FPT UNIVERSITY

Capstone Project Report

Report 3 – Software Requirement Specification

– Can Tho, October 2026 –

Table of Contents

I. Record of Changes 8

II. Software Requirement Specification 9

1. Product Overview 9

2. User Requirements 10

2.1 Actors 10

2.2 Use Cases 13

2.2.1 Diagram(s) 13

2.2.2 Descriptions 33

3. Functional Requirements 44

3.1 System Functional Overview 44

3.1.1 Screens Flow 44

3.1.2 Screen Descriptions 47

3.1.3 Screen Authorization 66

3.1.4 Non-Screen Functions 72

3.1.5 Entity Relationship Diagram 89

3.2 User Authentication 110

3.2.1 Login with Google 110

3.2.2 Login 115

3.2.3 Logout 119

3.2.4 Register 123

3.2.5 Forgot Password 127

3.3 Role Management 130

3.3.1 View Role List 130

3.3.2 View Role Detail 134

3.3.3 Create Role 136

3.3.4 Update Role 140

3.3.5 Delete Role 143

3.3.6 Update Role Permissions 146

3.4 Manage Forum 149

3.4.1 View Forum Category List 149

3.4.2 View Forum Category Detail and Post List 154

3.4.3 View Post Detail 160

3.4.4 Create Post 165

3.4.5 Edit Post 171

3.4.6 Delete Post 176

3.4.7 Like a Post . . . 180
3.4.8 View Post Likes . . . 184
3.4.9 Create Comment . . . 188
3.4.10 Update Comment . . . 193
3.4.11 Delete Comment . . . 198
3.4.12 Like a Comment . . . 202
3.4.13 Report Content . . . 205
3.5 Contract Management . . . 210
3.5.1 Create Contract . . . 210
3.5.2 Update Contract . . . 216
3.5.3 Complete Contract . . . 220
3.5.4 View Contract List . . . 224
3.5.5 View Contract Detail . . . 228
3.5.6 Sign Contract . . . 232
3.5.7 Cancel Contract . . . 236
3.5.8 View Contract Files . . . 240
3.5.9 Upload Contract File . . . 244
3.5.10 Delete Contract File . . . 247
3.6 Job Browsing . . . 251
3.6.1 Function: View Job List . . . 251
3.6.2 Function: View Job Detail . . . 254
3.7 Job Management . . . 257
3.7.1 Function: View My Jobs . . . 257
3.7.2 Function: Create Job . . . 260
3.7.3 Function: Publish Job . . . 263
3.7.4 Function: Update Job . . . 265
3.7.5 Function: Delete Job . . . 268
3.7.6 Function: Close Job . . . 271
3.7.7 Function: Reopen Job . . . 273
3.8 Bookmarked Jobs Management . . . 276
3.8.1 Function: View Bookmarked Job List . . . 276
3.8.2 Function: Bookmark Job . . . 279
3.8.3 Function: Remove Bookmark . . . 281
3.9 Permission Management . . . 283
3.9.1 View Permission List . . . 283
3.9.2 View Permission Detail . . . 290
3.10 Session Management . . . 294
3.10.1 View Session List . . . 294

3.10.2 View Session Detail 300
3.10.3 Revoke Session 305
3.11 Forum Category Management 309
3.11.1 Create Forum Category 309
3.11.2 Update Forum Category 315
3.11.3 Delete Forum Category 319
3.11.4 View Forum Category List 325
3.11.5 View Forum Category Detail 329
3.11.6 Restore Forum Category 334
3.12 Manage Conversation 338
3.12.1 Start Conversation 338
3.12.2 View Conversation List 341
3.12.3 View Conversation Detail 343
3.12.4 Send Message 345
3.12.5 Send File 347
3.12.6 Mark Conversation as Read 349
3.12.7 Pin Conversation 351
3.12.8 Delete Message 353
3.13 Milestone Management 355
3.13.1 View Milestone List 355
3.13.2 View Milestone Detail 359
3.13.3 Create Milestone 363
3.13.4 Update Milestone 366
3.13.5 Delete Milestone 370
3.13.6 Request Milestone Changes 373
3.13.7 Approve Milestone 376
3.13.8 Update Milestone Progress 380
3.13.9 View Milestone Files 384
3.13.10 Upload Milestone File 386
3.13.11 Delete Milestone File 390
3.13.12 Submit Milestone 393
3.13.13 Delete Milestone Submission 396
3.13.14 View Milestone Submission List 400
3.13.15 View Milestone Submission Detail 403
3.14 User Management 406
3.14.1 View User List 406
3.14.2 View User Detail 411
3.14.3 Create User 415

3.14.4 Update User . . . 421
3.14.5 Update Client Profile . . . 425
3.14.6 Update Freelancer Basic Info . . . 430
3.14.7 Update Freelancer Skills . . . 435
3.14.8 Update Freelancer Portfolio . . . 441
3.14.9 Update Expert Profile . . . 445
3.15 Moderation . . . 451
3.15.1 View Moderation Queue . . . 451
3.15.2 View Moderation Item Detail . . . 453
3.15.3 Resolve Moderation Item . . . 455
3.16 ID Verification . . . 457
3.16.1 View Verification Request List . . . 457
3.16.2 View ID Verification Request Detail . . . 459
3.16.3 Approve or Reject ID Verification . . . 462
3.17 Trash . . . 464
3.17.1 View Deleted Post List . . . 464
3.17.2 View Deleted Post Detail . . . 466
3.17.3 Restore Deleted Post . . . 468
3.17.4 View Deleted Comment List . . . 470
3.17.5 View Deleted Comment Detail . . . 472
3.17.6 Restore Deleted Comment . . . 474
3.18 Dispute Management . . . 476
3.18.1 Open Dispute . . . 476
3.18.2 View Dispute List . . . 480
3.18.3 View Dispute Detail . . . 484
3.18.4 Respond to Dispute . . . 488
3.18.5 Respond to Dispute Resolution . . . 491
3.18.6 Propose Dispute Resolution . . . 495
3.19 Proposal Management . . . 499
3.19.1 Function: View My Proposal List . . . 499
3.19.2 Function: View My Proposal Detail . . . 502
3.19.3 Function: Submit Proposal . . . 504
3.19.4 Function: Update Proposal . . . 507
3.19.5 Function: Withdraw Proposal . . . 510
3.19.6 Function: Accept Proposal . . . 513
3.19.7 Function: Reject Proposal . . . 516
3.19.8 Function: View Job Proposal List . . . 518
3.19.9 Function: View Proposal Detail . . . 520

3.20 Banner Management . . . 523
3.20.1 View Banner List . . . 523
3.20.2 View Banner Detail . . . 528
3.20.3 Create Banner . . . 533
3.20.4 Update Banner . . . 539
3.20.5 Delete Banner . . . 545
3.21 Skill Management . . . 549
3.21.1 View Skill List . . . 549
3.21.2 View Skill Detail . . . 554
3.21.3 Create Skill . . . 560
3.21.4 Update Skill . . . 565
3.21.5 Delete Skill . . . 570
3.21.6 Restore Skill . . . 575
3.22 Payment Management . . . 579
3.22.1 Connect Stripe Account . . . 579
3.22.2 View Payout History . . . 583
3.22.3 View Payout Detail . . . 586
3.22.4 Fund Milestone . . . 589
3.22.5 Refund Payment . . . 592
3.22.6 Pay Platform Fee . . . 596
3.22.7 Release Payment . . . 600
3.22.8 Pay Dispute Fee . . . 603
3.22.9 View Payment History . . . 607
3.22.10 View Payment Detail . . . 610
3.22.11 Execute Dispute Resolution Payment . . . 612
3.22.12 Handle Stripe Webhook Event . . . 616
3.22.13 Sync Payment Intent Status . . . 619
3.23 Job and Talent Recommendation . . . 622
3.23.1 Function: Recommend Jobs and Freelancers . . . 622
3.24 Job Category Management . . . 625
3.24.1 View Job Category List . . . 625
3.24.2 View Job Category Detail . . . 631
3.24.3 Create Job Category . . . 636
3.24.4 Update Job Category . . . 642
3.24.5 Delete Job Category . . . 648
3.24.6 Restore Job Category . . . 652
3.25 Job Category Browsing . . . 657
3.25.1 Function: View Job Category List . . . 657

3.25.2 Function: View Job Category Detail 660
4. Non-Functional Requirements 663
4.1 External Interfaces 663
4.2 Quality Attributes 664
4.2.1 Usability 664
4.2.2 Reliability 664
4.2.3 Performance 664
4.2.4 Security & Privacy 665
5. Requirement Appendix 665
5.1 Business Rules 665
5.2 Common Requirements 671
5.3 Application Messages List 672

I. Record of Changes

Date	A* M, D	In charge	Change Description

**A - Added M - Modified D - Deleted*

II. Software Requirement Specification

1. Product Overview

The Frevia project is a comprehensive freelancing platform that enables clients, freelancers, and administrators to build trusted professional relationships and efficiently manage the entire project lifecycle. Designed to connect businesses with skilled talent worldwide, Frevia provides intelligent job matching, structured contract workflows, and secure payment solutions to facilitate successful collaborations.

Through AI-powered recommendations, real-time communication, and advanced moderation mechanisms, the platform helps users discover suitable opportunities, deliver high-quality work, and maintain transparent interactions.

Frevia serves three primary stakeholder groups: clients seeking specialized expertise, freelancers showcasing their skills and services, and administrators responsible for maintaining platform integrity and operational efficiency. The platform offers an intuitive user experience, dynamic communication channels, and comprehensive management capabilities to support collaboration, professional growth, and secure transactions.

Key features of Frevia include:

- **Smart Job Matching and AI Recommendations:** The platform analyzes freelancer skills, experience, and preferences to recommend relevant job opportunities. AI-powered profile scoring provides quality assessments, while job alerts and saved searches help users stay informed about new opportunities.
- **End-to-End Contract and Milestone Management:** Clients and freelancers can negotiate project terms, define milestones and tasks, monitor progress, and securely manage payments. Integrated contract signing, activity logs, and file-sharing capabilities ensure transparency throughout the project lifecycle.
- **Community and Forum Engagement:** Frevia includes a community forum where users can discuss industry topics, exchange knowledge, and seek professional advice. Features such as categories, posts, comments, reactions, and reporting mechanisms encourage active participation and self-moderation.
- **Identity Verification and Content Moderation:** Freelancers can verify their identities through document submission, while administrators utilize moderation tools, banned keyword management, and AI-assisted violation detection to maintain a secure and trustworthy environment.

- **Wallet, Payments, and Gas Fee Subsidies:** Users can manage account balances, withdraw earnings, and review transaction histories. The platform also supports gas fee subsidy mechanisms for blockchain-based transactions, improving payment accessibility and reducing operational costs.
- **Administrative Management Tools:** Administrators have access to comprehensive management functionalities, including user administration, role and permission control, system configuration, advertisement management, activity monitoring, and dispute resolution, ensuring fair and reliable platform operations.

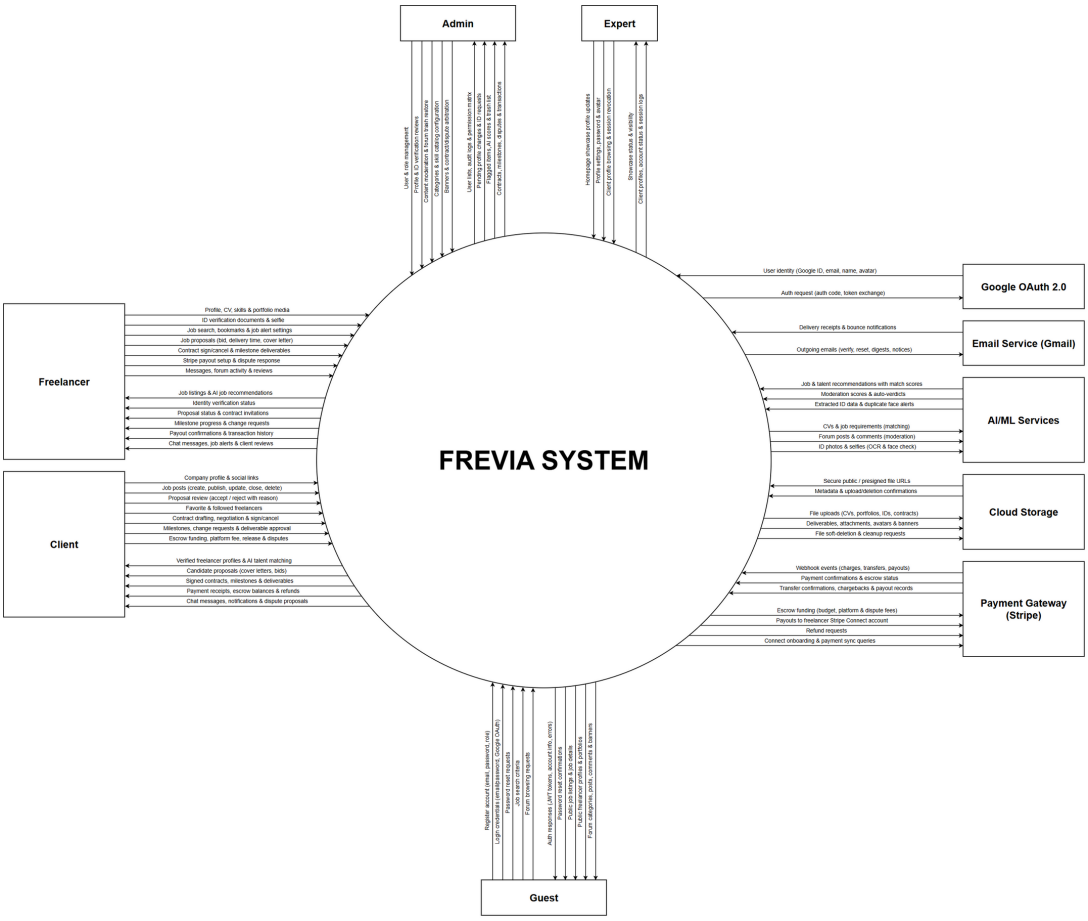

Figure 1: Context diagram

2. User Requirements

2.1 Actors

Table 1: Actors of the Frevia Platform

No	Actor	Description
1	Guest	Guests are unregistered visitors who have not yet created an account on the platform. They can browse publicly available information such as job listings, freelancer profiles, forum content, and platform features. Guests may also register a new account (via email/password or Google OAuth) or authenticate using supported login methods to access platform services.
2	Client	Clients are individuals or organizations seeking professional services from freelancers. They can create and manage job postings, review proposals, hire freelancers, establish contracts, manage escrow payments, monitor project progress, initiate disputes, and provide reviews upon project completion.
3	Freelancer	Freelancers are service providers who offer professional skills and expertise through the platform. They can create and manage profiles, portfolios, and skills; search and apply for jobs; submit proposals; collaborate with clients; complete project tasks and milestones; and receive payments after successful project delivery.
4	Admin	Administrators are responsible for managing and maintaining the platform's operations, security, and compliance. Their responsibilities include user management, role and permission management, identity verification, content moderation, dispute resolution, system configuration, and monitoring platform activities to ensure a secure and reliable environment.
5	Expert	Experts are verified, high-ranking domain specialists featured in the platform's public expert showcase. They possess verified credentials, manage specialized public showcase profiles, consult on high-value client projects, and manage their security sessions and credentials.

No	Actor	Description
6	System	The internal automated execution engine and background scheduler of the platform. The System autonomously executes scheduled background jobs, manages deadline expirations, triggers automatic state transitions for contracts and proposals, processes automated dispute resolution payouts upon administrative decisions, and dispatches automated notifications.
7	AI Service	The artificial intelligence and machine learning microservice providing automated cognitive capabilities. It analyzes resumes to compute CV-to-job matching scores, evaluates forum posts and comments for toxicity and spam to assist content moderation, and performs optical character recognition (OCR) and face-matching validation on identity verification documents.
8	Payment Gateway (Stripe)	The third-party financial processor (Stripe Checkout and Stripe Connect) powering platform monetary operations. It securely processes client escrow funding, holds funds in escrow during project execution, handles automated payouts to freelancer bank accounts, processes refunds and dispute arbitration fees, and transmits asynchronous webhook events to synchronize payment statuses.
9	Email Service (Gmail / SMTP)	The external transactional email delivery provider (SMTP / Nodemailer). It delivers transactional and security communications to users, including email verification links, multi-factor authentication one-time passwords (OTP), password reset instructions, contract status updates, and dispute notifications.
10	Google OAuth 2.0	Third-party federated identity provider. Enables seamless and secure single sign-on (SSO) authentication, allowing guests to register and log in to the Frevia platform using their Google accounts without requiring separate password management.

No	Actor	Description
11	Cloud Storage (Cloudinary)	Third-party media storage and content delivery network (CDN) service. Securely stores, optimizes, and delivers user-uploaded digital assets, including user avatars, portfolio media, project deliverable files, and identity verification documents.

2.2 Use Cases

2.2.1 Diagram(s)

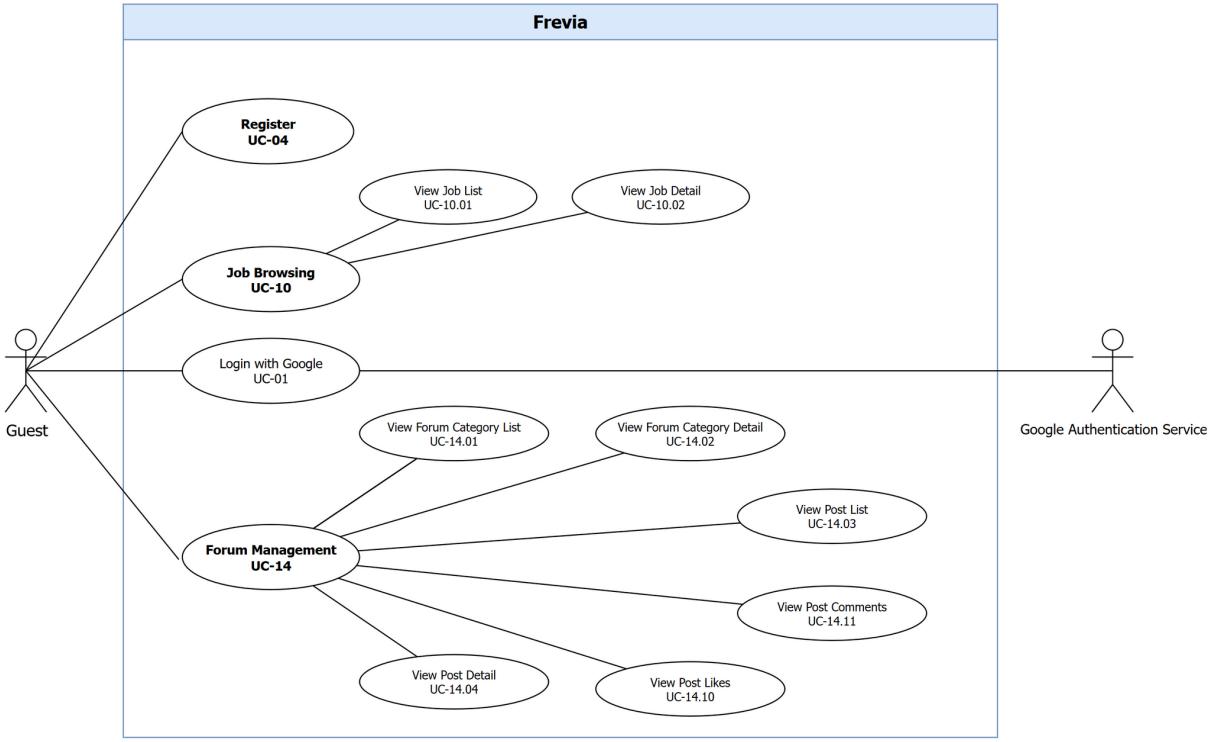

Figure 2: Guest Usecase

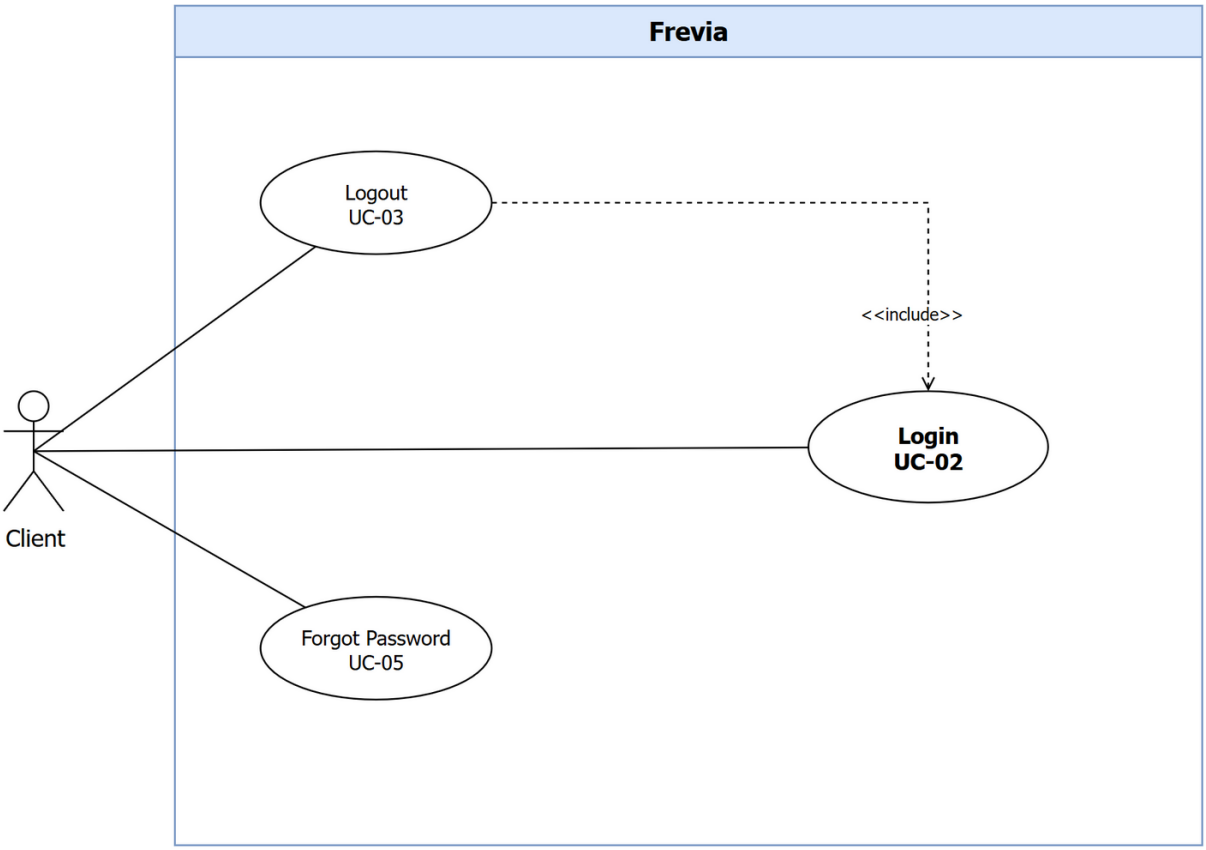

Figure 3: Client Usecase 1/6

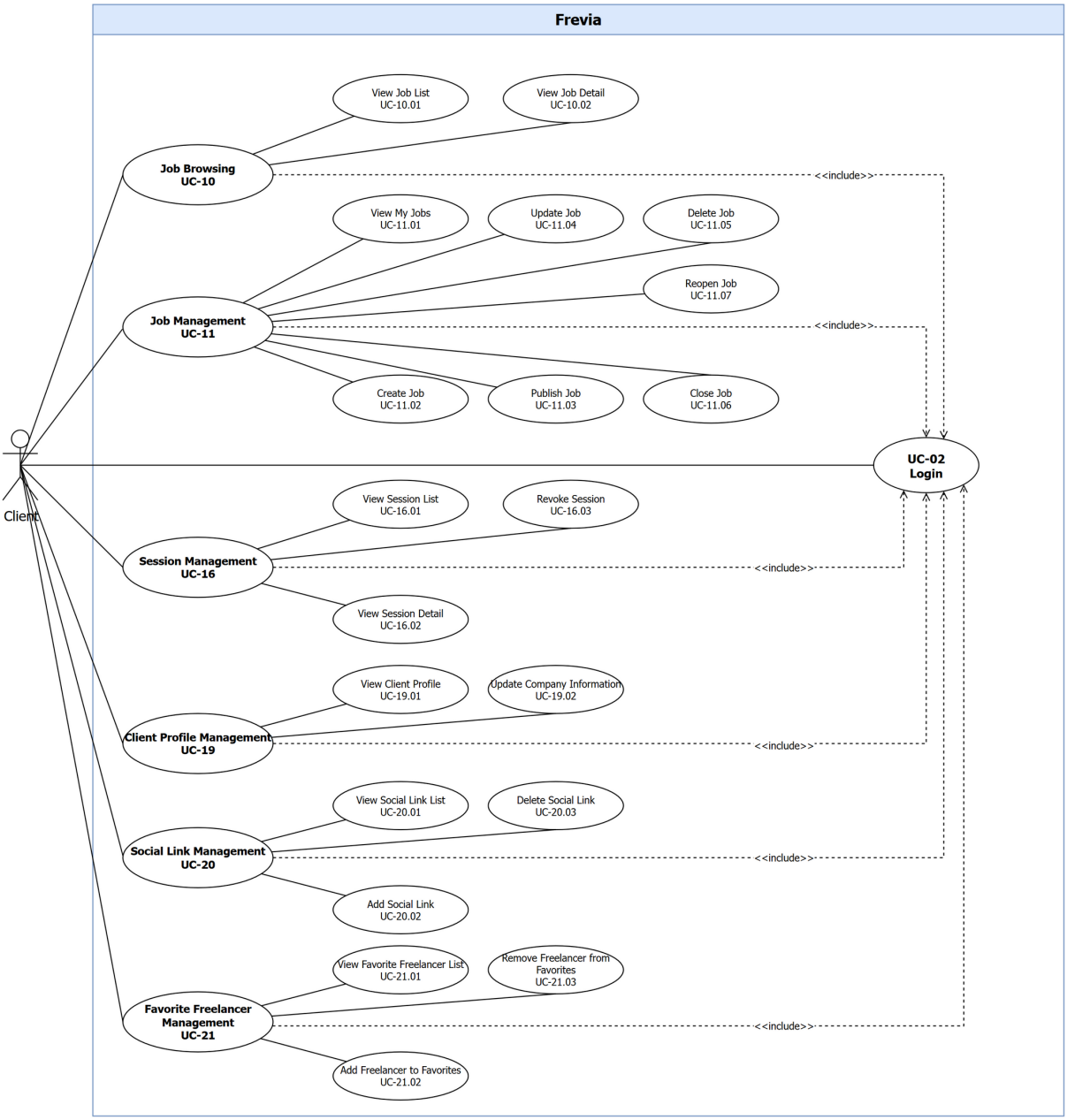

Figure 4: Client Usecase 2/6

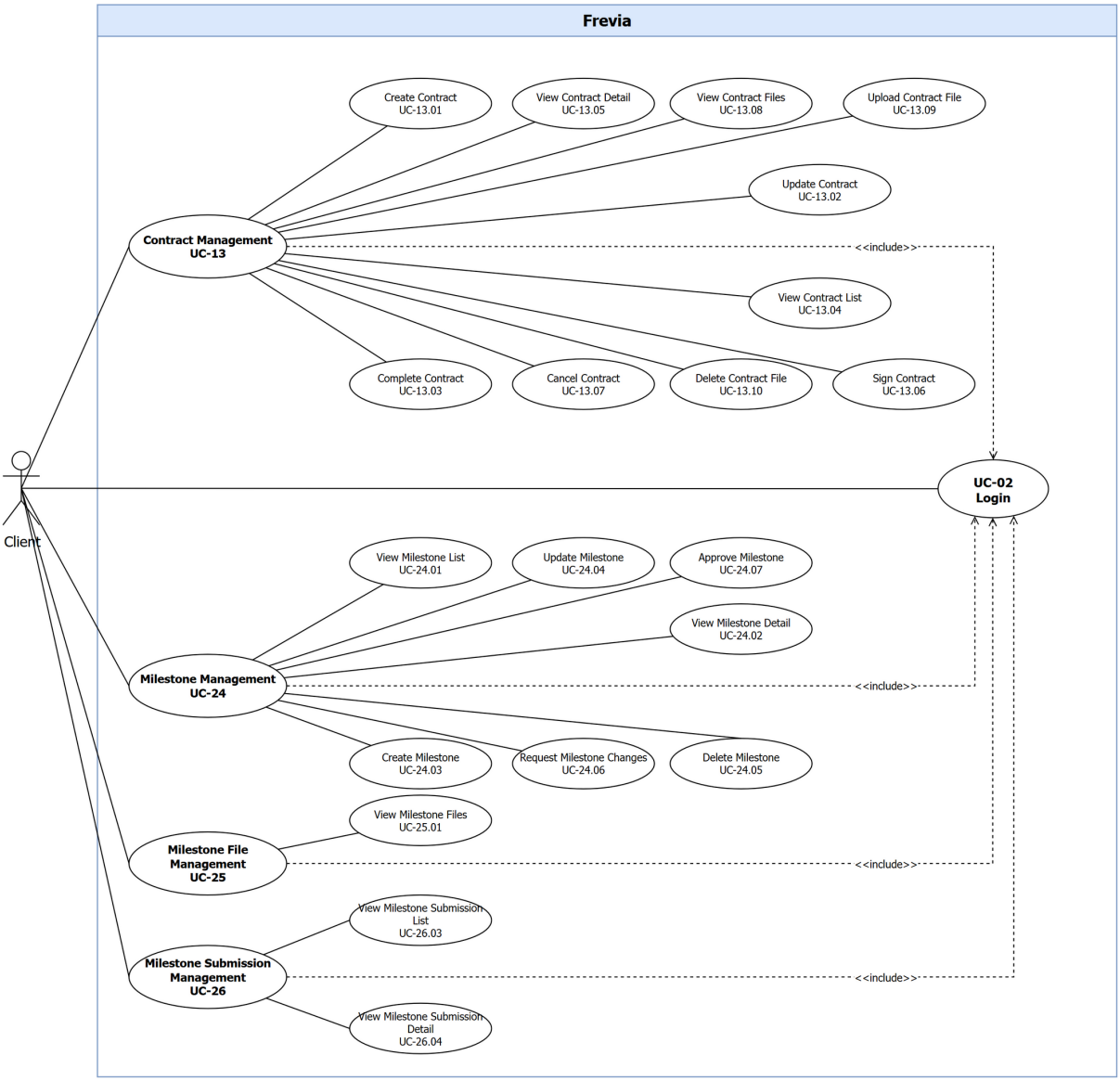

Figure 5: Client Usecase 3/6

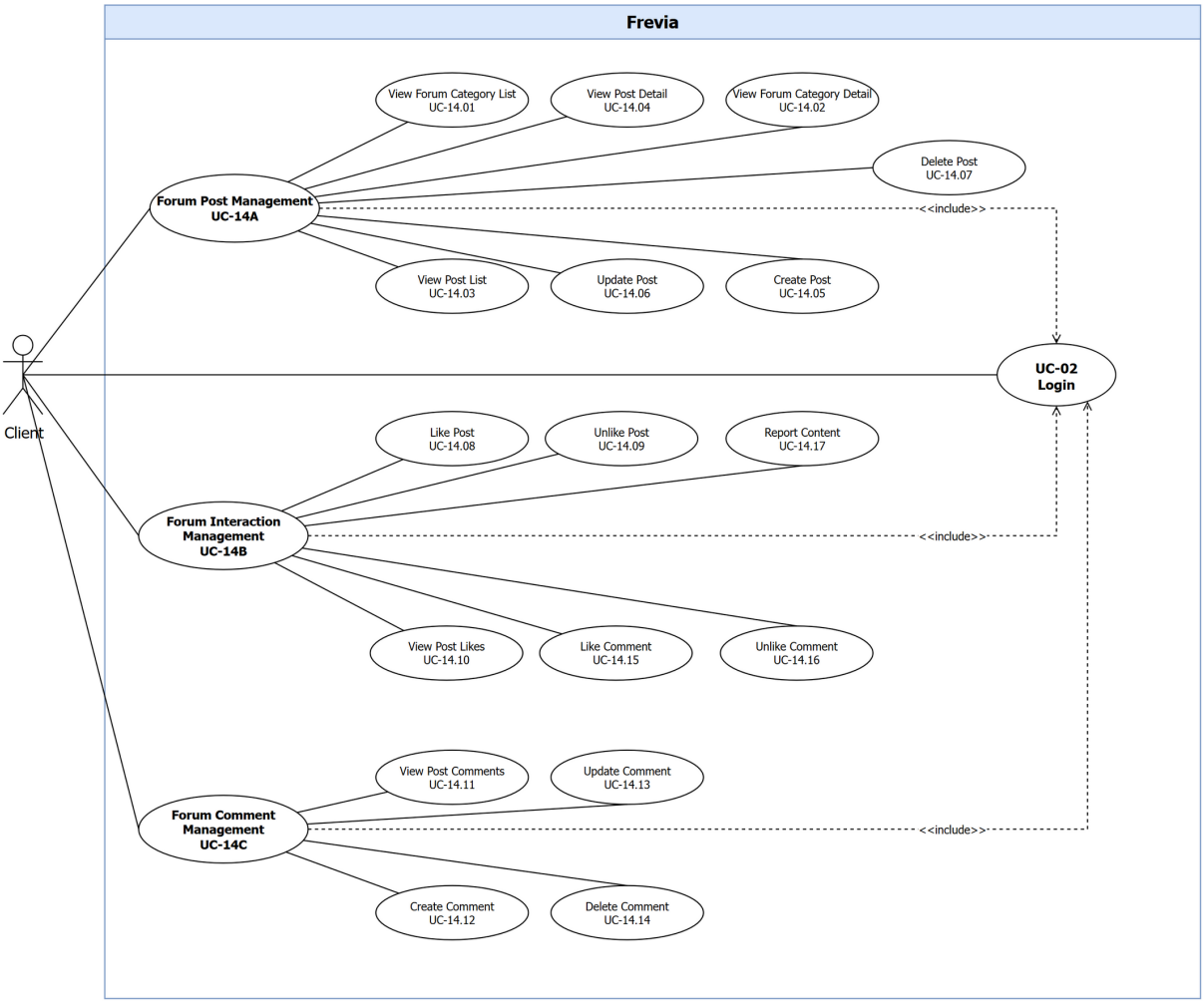

Figure 6: Client Usecase 4/6

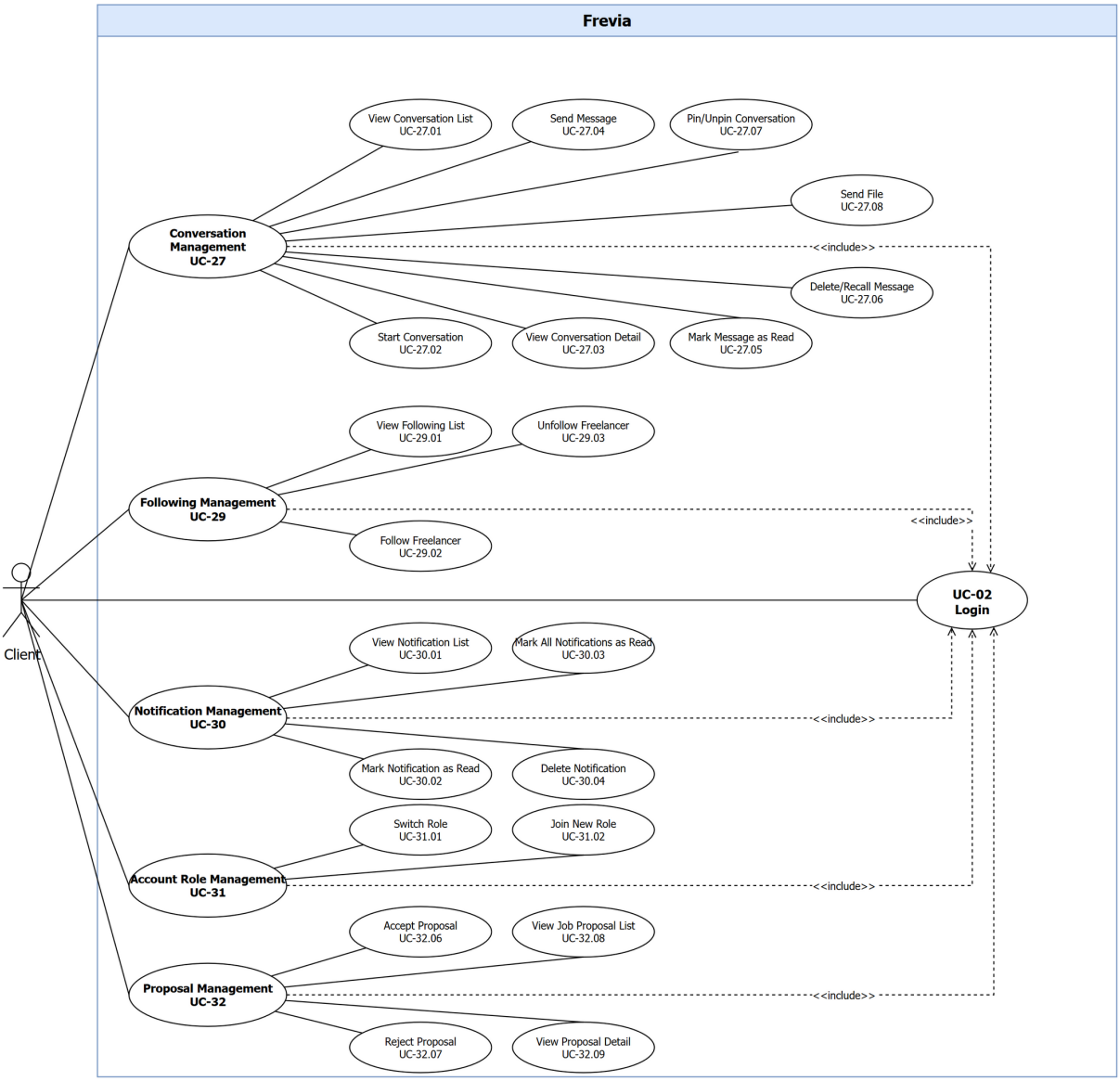

Figure 7: Client Usecase 5/6

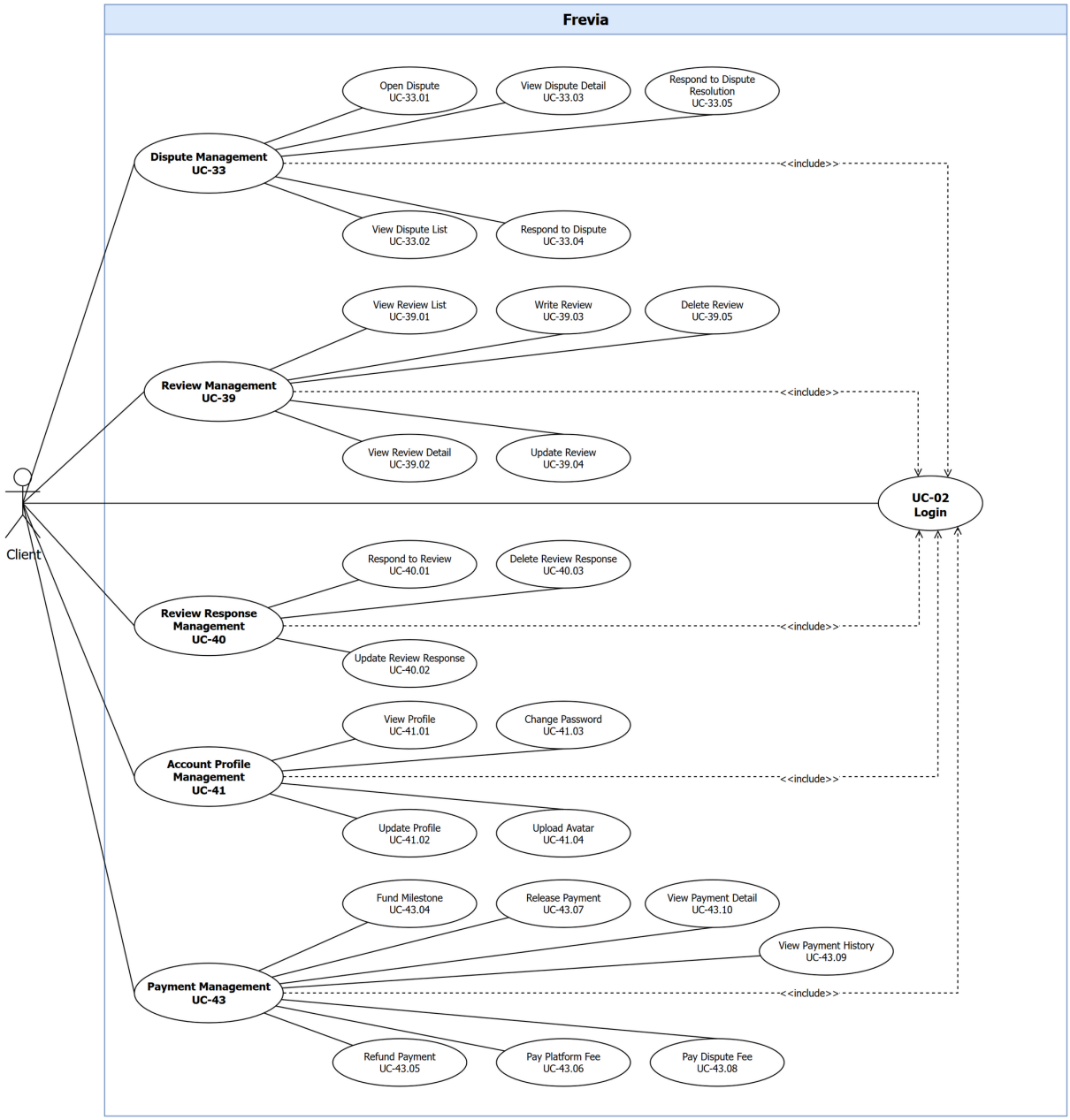

Figure 8: Client Usecase 6/6

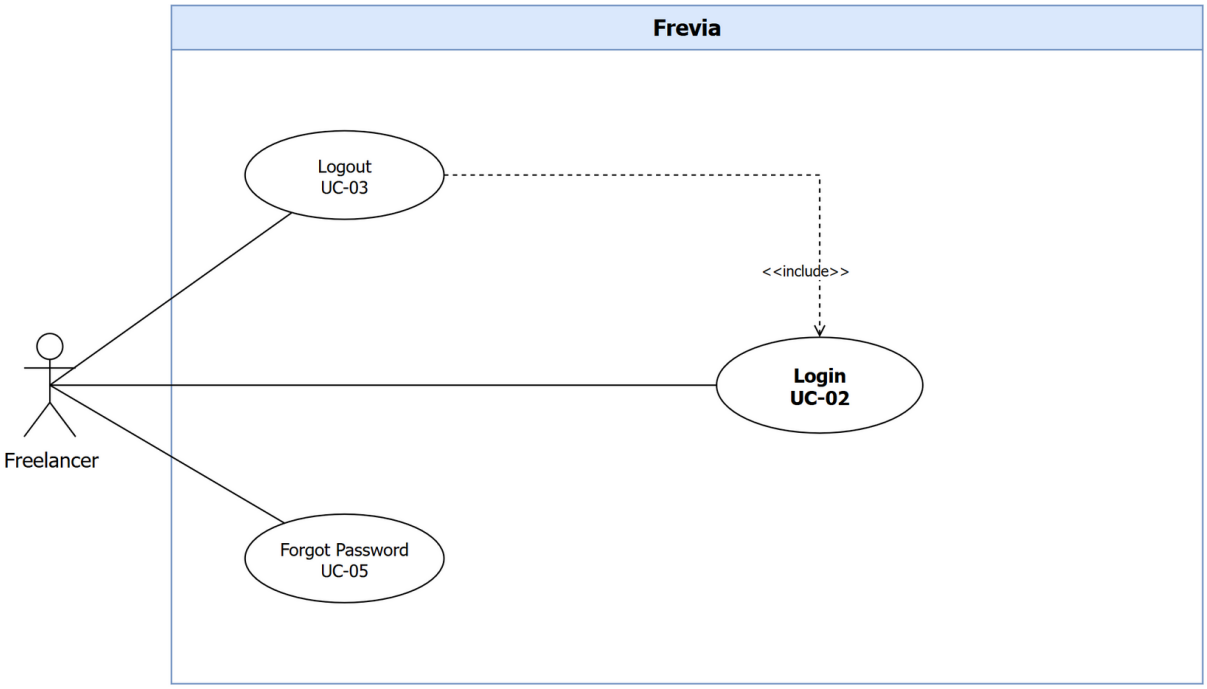

Figure 9: Freelancer Usecase 1/7

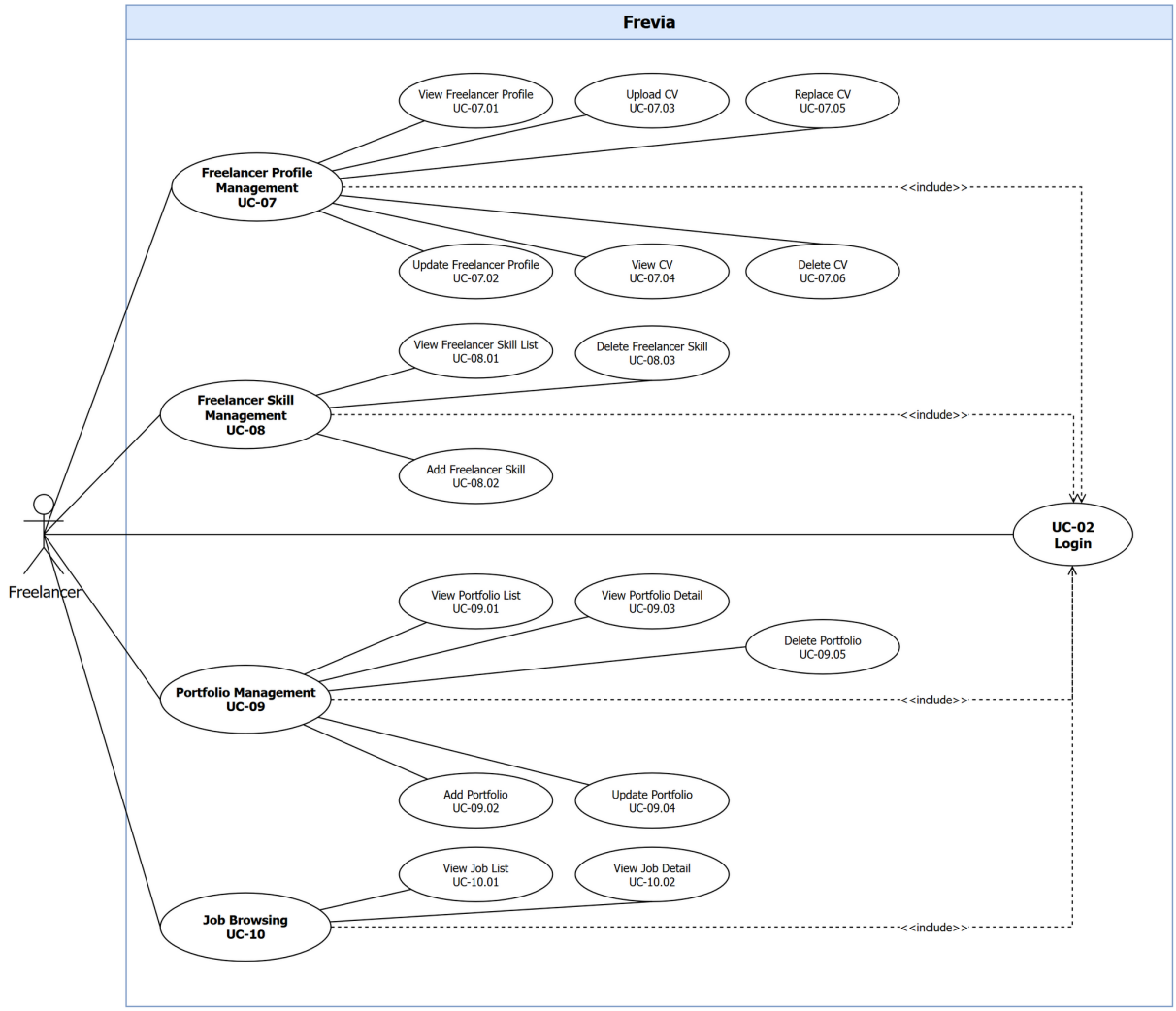

Figure 10: Freelancer Usecase 2/7

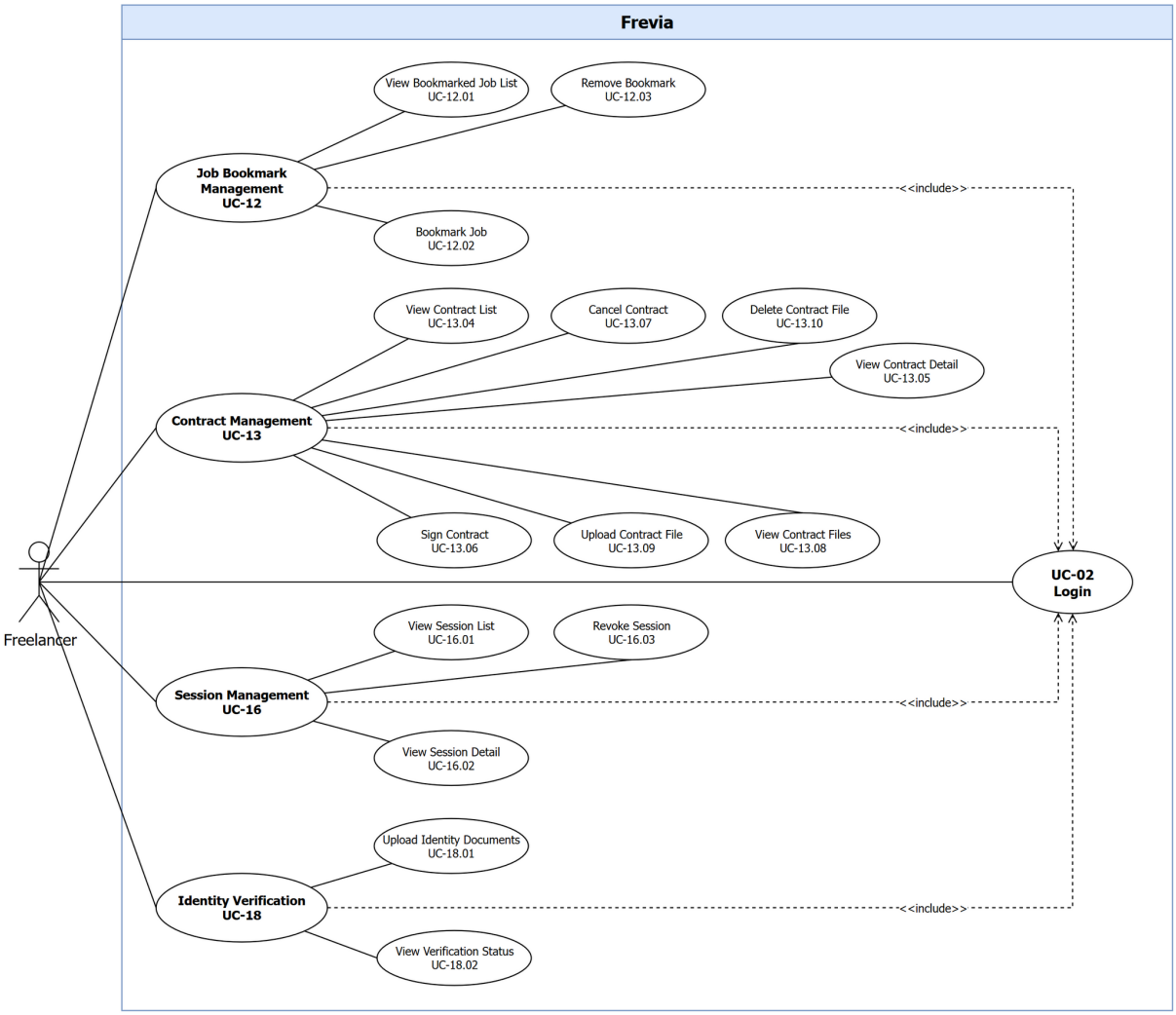

Figure 11: Freelancer Usecase 3/7

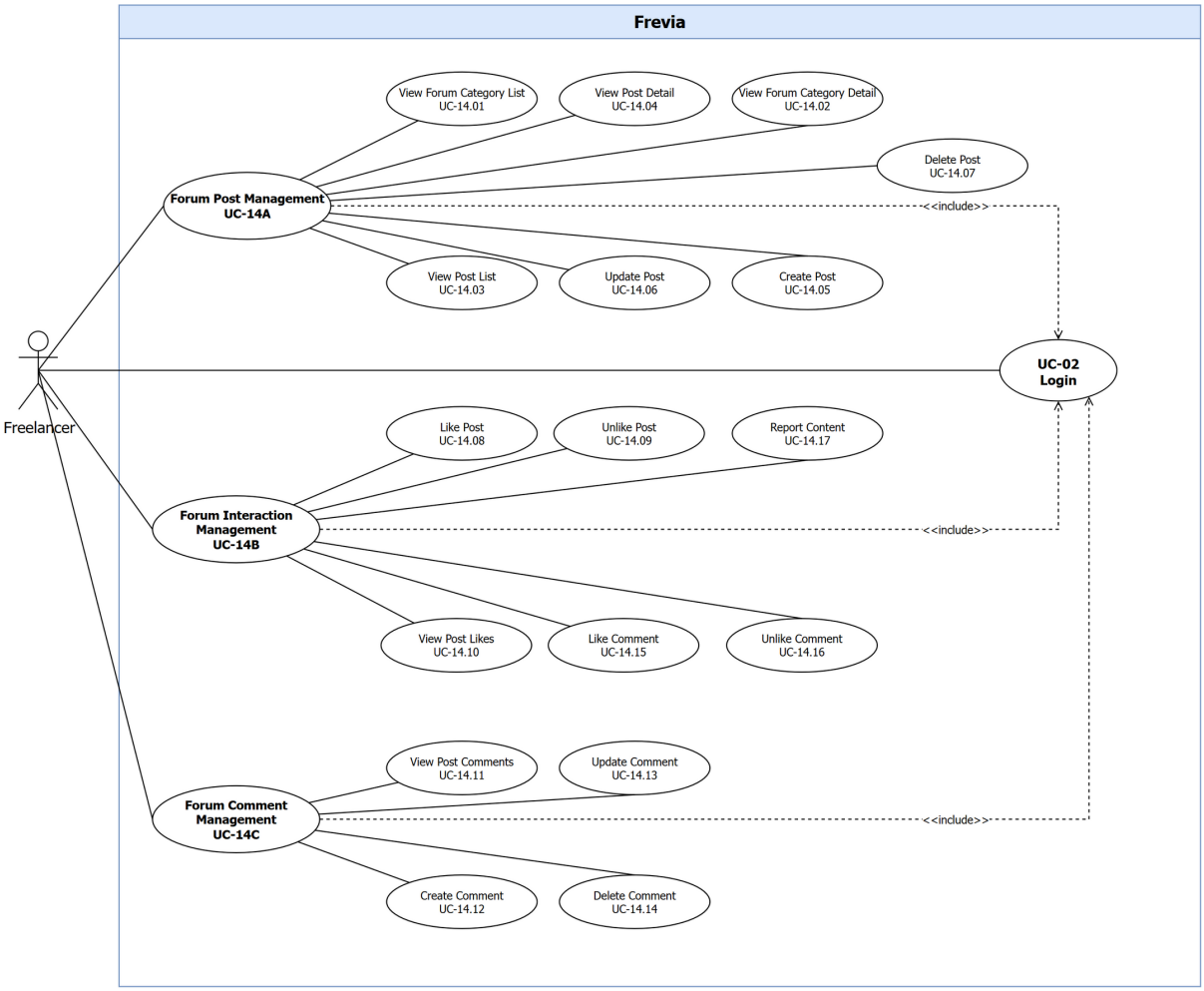

Figure 12: Freelancer Usecase 4/7

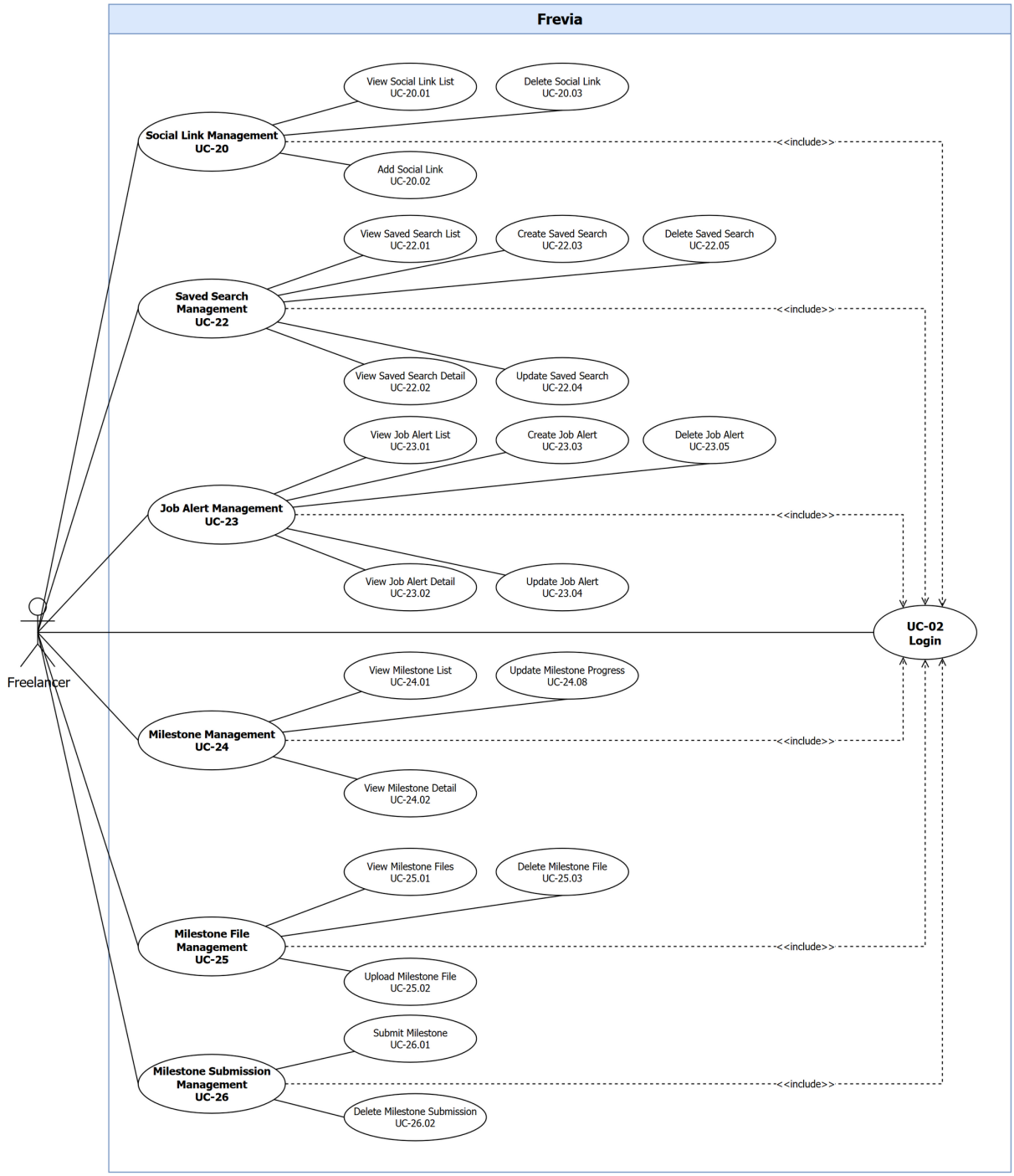

Figure 13: Freelancer Usecase 5/7

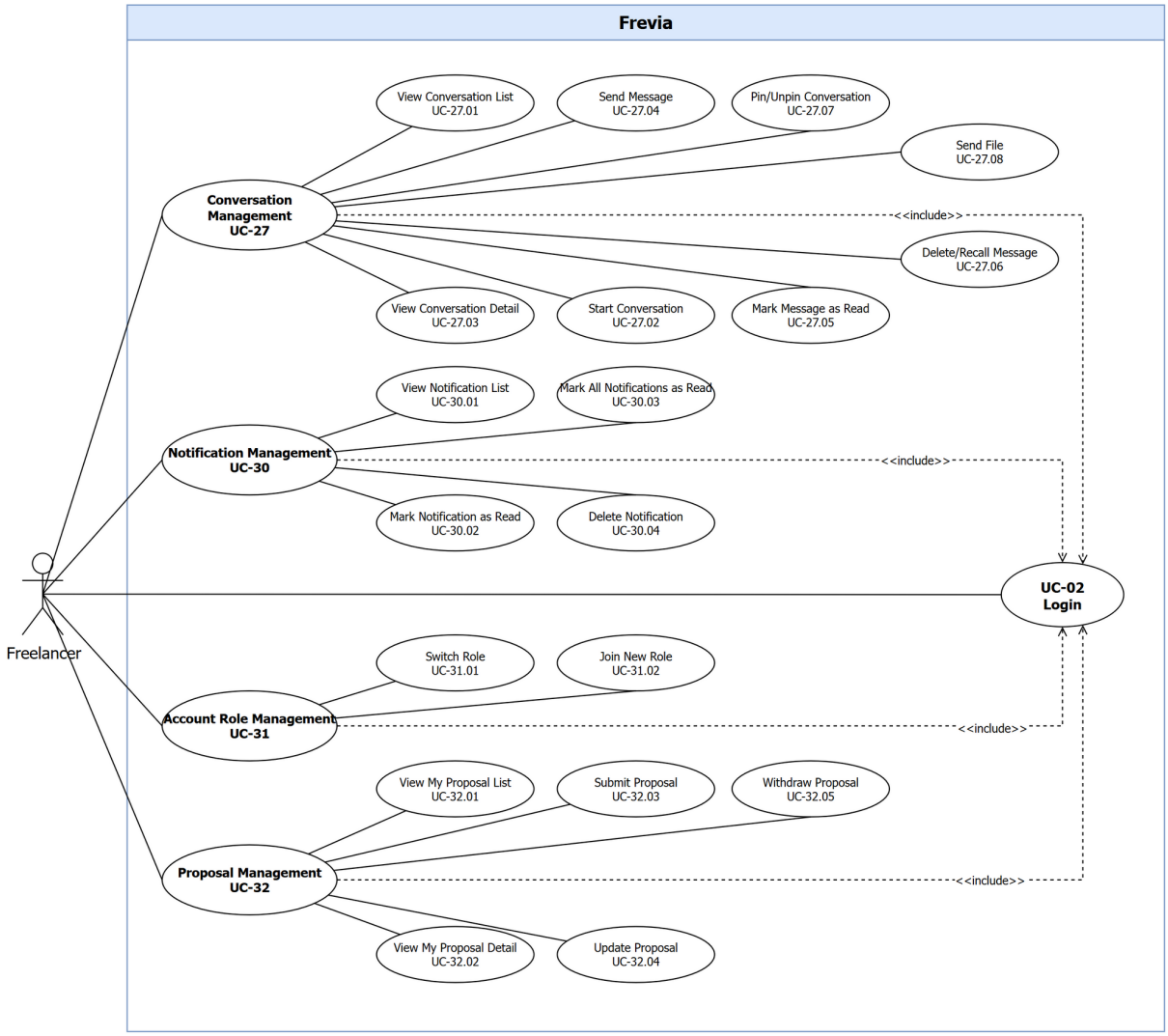

Figure 14: Freelancer Usecase 6/7

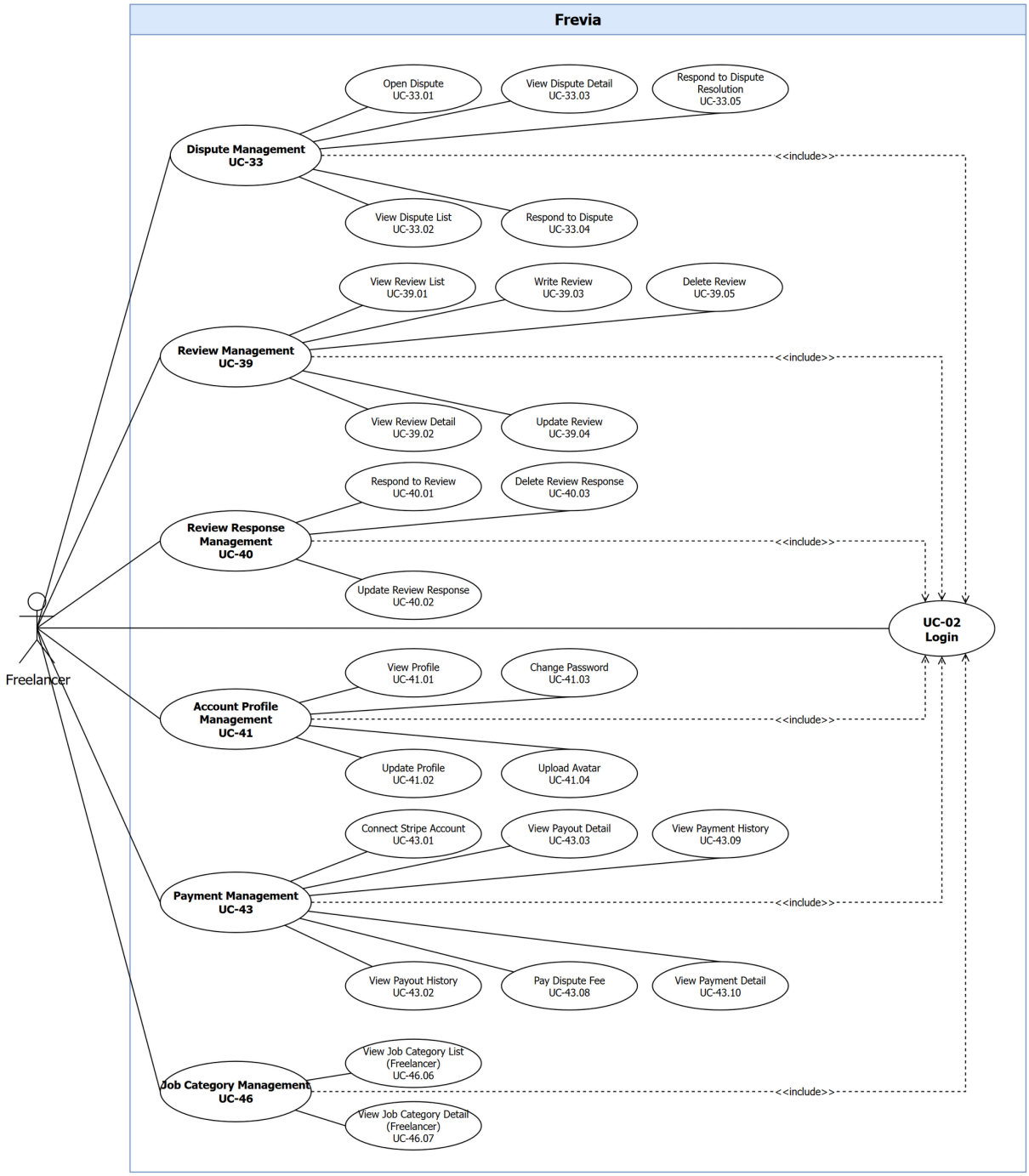

Figure 15: Freelancer Usecase 7/7

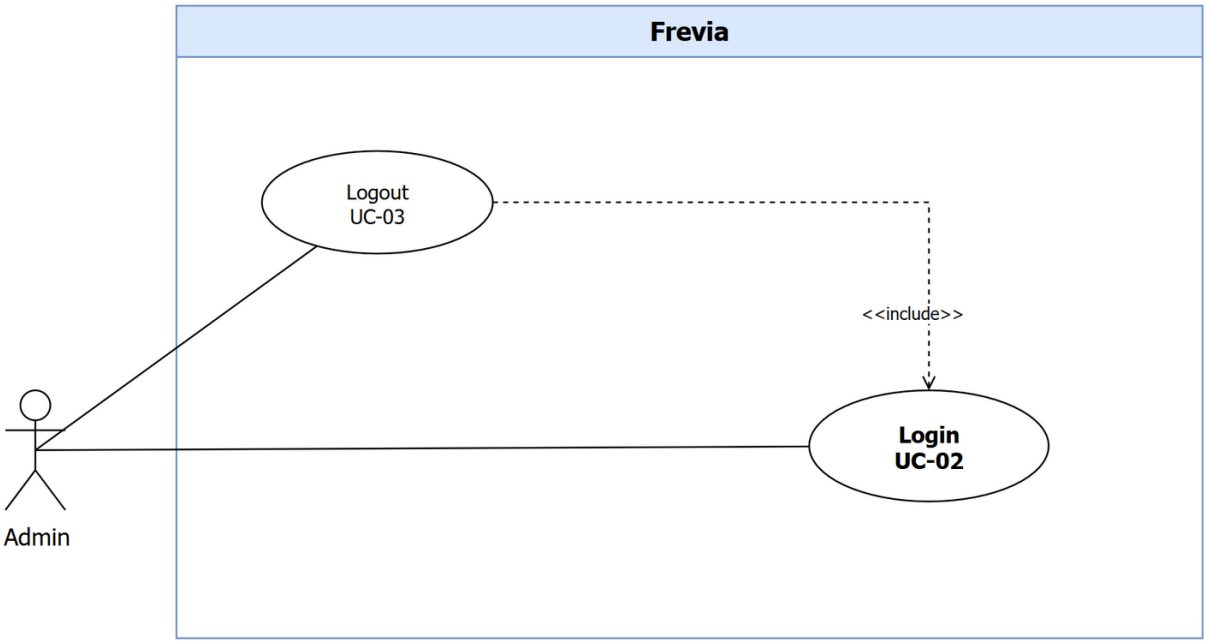

Figure 16: Admin Usecase 1/4

Figure 17: Admin Usecase 2/4

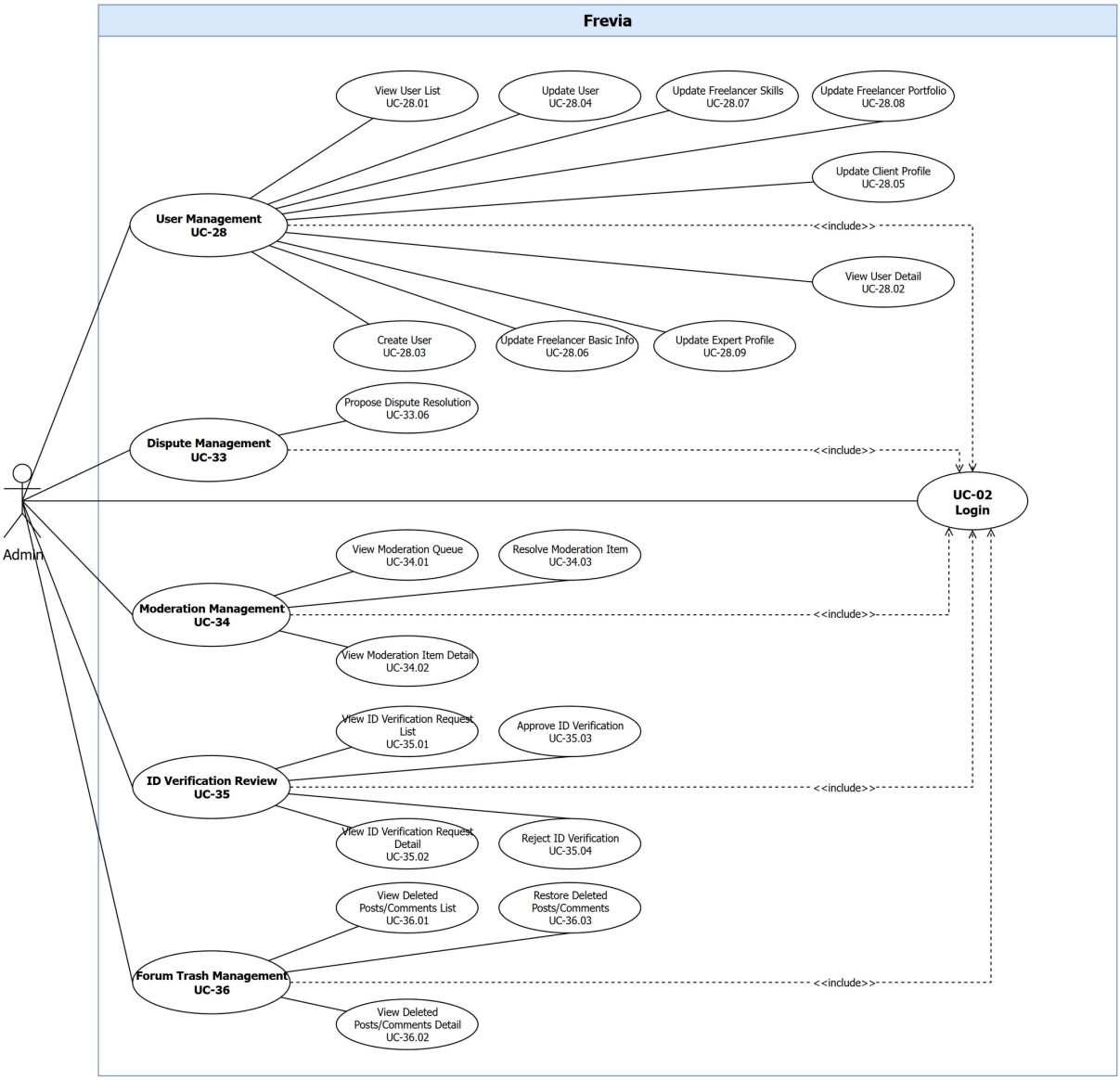

Figure 18: Admin Usecase 3/4

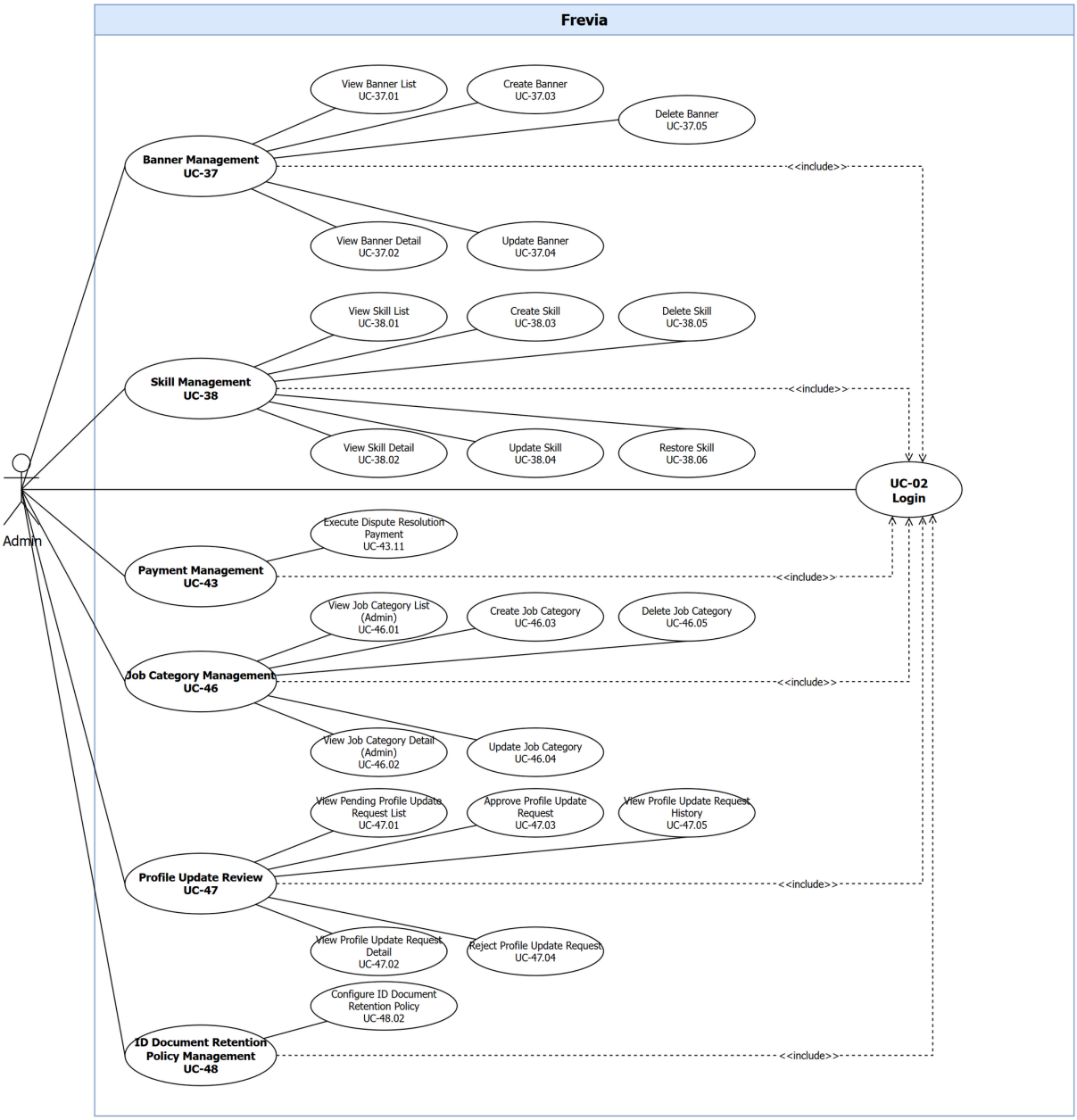

Figure 19: Admin Usecase 4/4

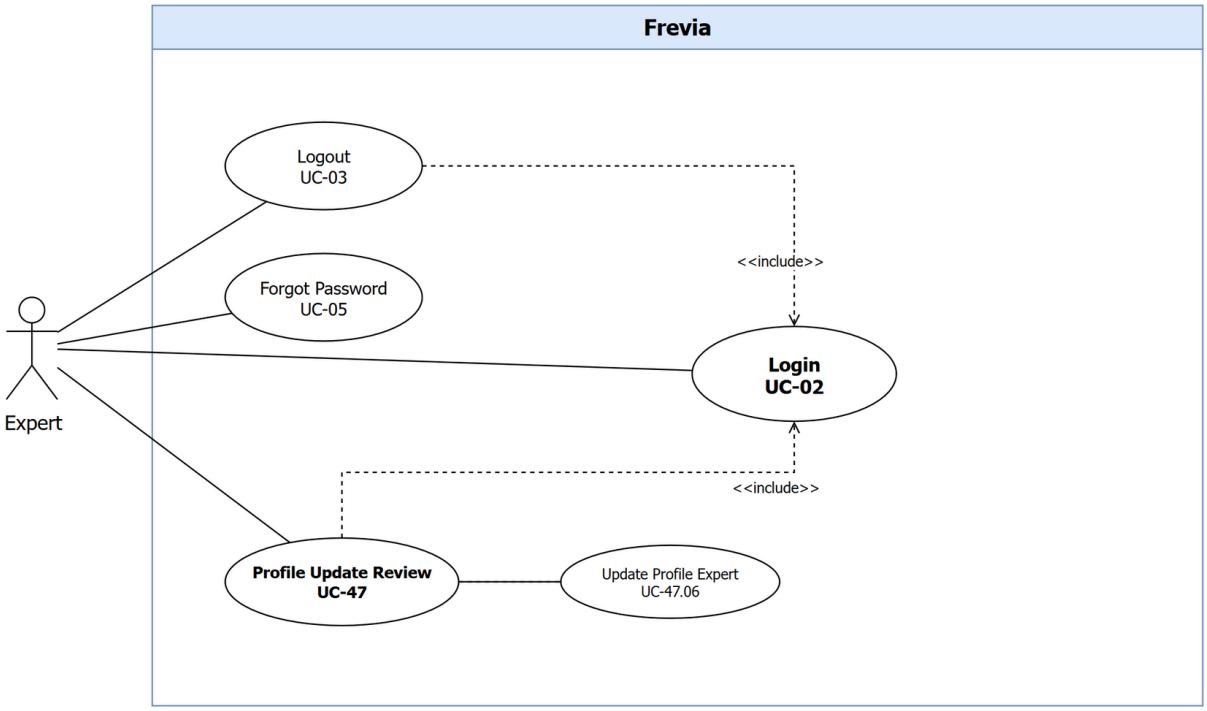

Figure 20: Expert Usecase

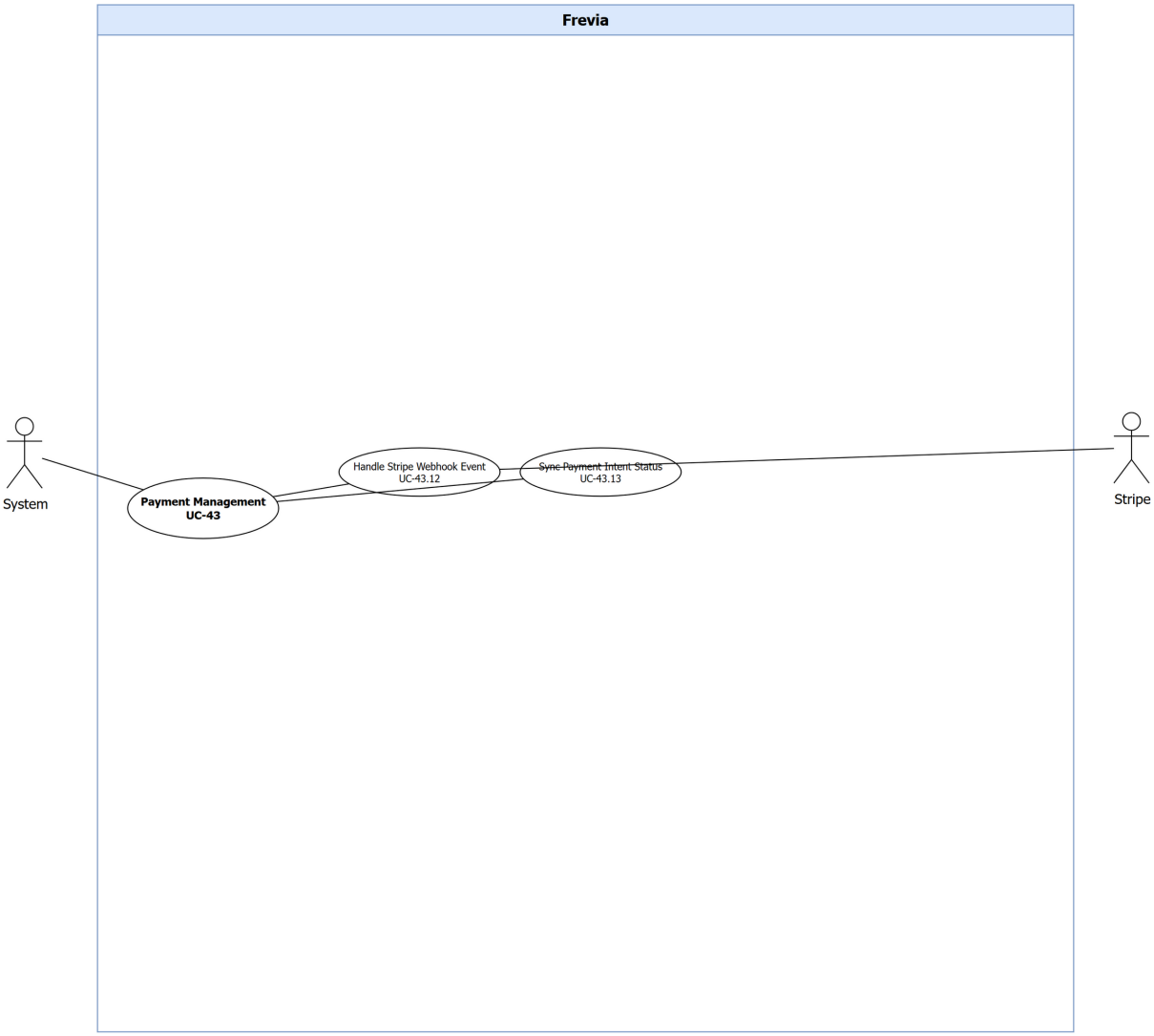

Figure 21: System Usecase

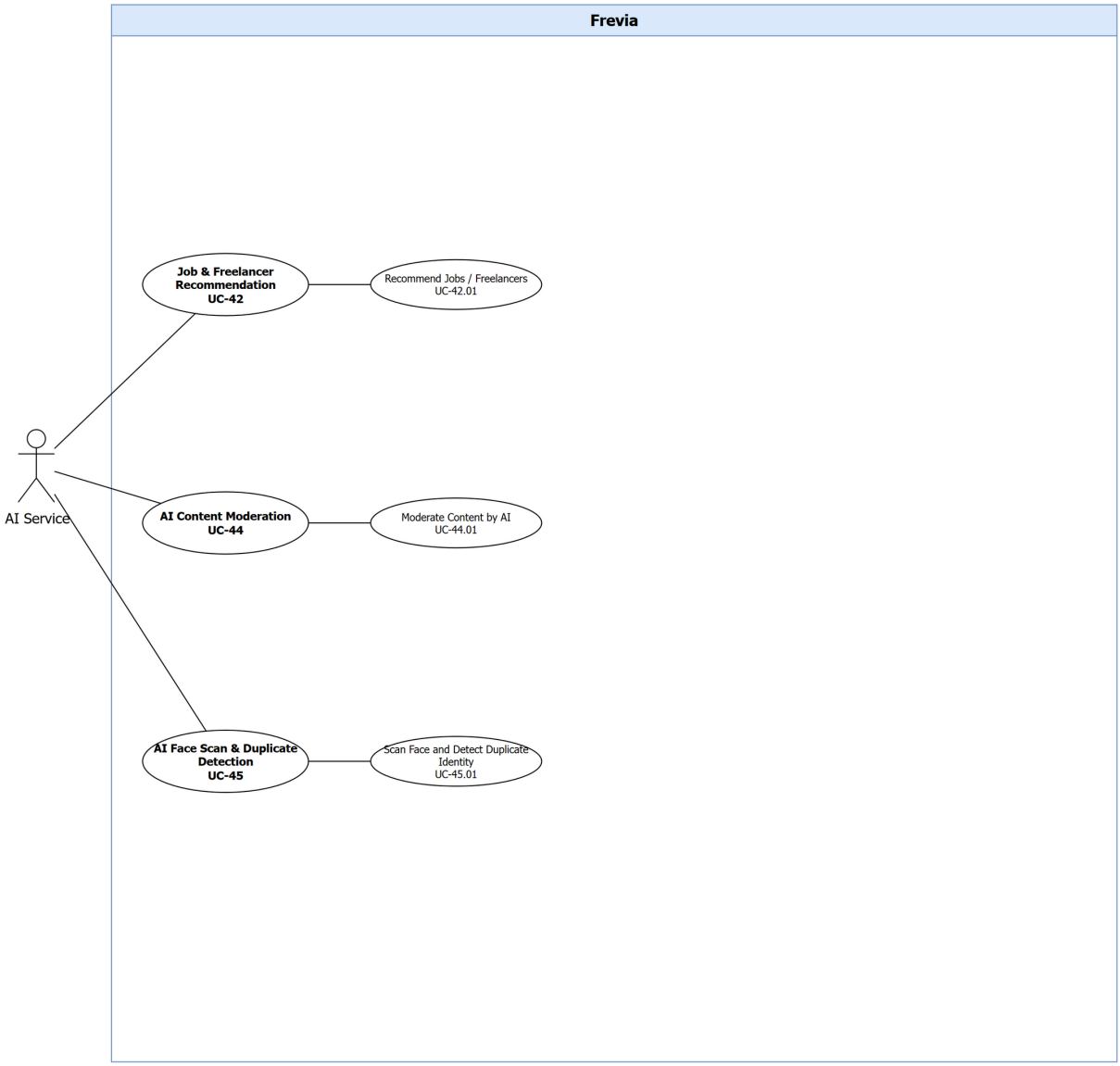

Figure 22: AI Service Usecase

2.2.2 Descriptions

ID	Use Case	Actors	Description
UC-01	Login with Google	Guest	Allows users to login/register using Google OAuth.
UC-02	Login	Guest	Allows users to log in using email/password and JWT authentication.
UC-03	Logout	Client, Expert, Freelancer, Admin	Allows users to logout and terminate authentication session.
UC-04	Register	Guest	Allows users to create new accounts using email and password.

ID	Use Case	Actors	Description
UC-05	Forgot Password	Guest	Allows users to request password reset via email verification link.
UC-06.01	View Role List	Admin	Retrieve list of all roles along with their assigned permissions.
UC-06.02	View Role Detail	Admin	View detailed information of a specific role including its permissions.
UC-06.03	Create Role	Admin	Create a new role with name, description, and a list of permission IDs.
UC-06.04	Update Role	Admin	Update a role's name, description, and replace all assigned permissions.
UC-06.05	Delete Role	Admin	Delete a role from the system.
UC-06.06	Update Role Permissions	Admin	Assign route access rights to roles.
UC-15.01	View Permission List	Admin	Retrieve all permissions (id, name, path, method, module) for admin to assign to roles.
UC-15.02	View Permission Detail	Admin	View detailed information of a specific permission.
UC-16.01	View Session List	Client, Expert, Freelancer, Admin	View a list of active login sessions.
UC-16.02	View Session Detail	Client, Expert, Freelancer, Admin	View login session information (device, IP address, login time, approximate location, etc.).
UC-16.03	Revoke Session	Client, Expert, Freelancer, Admin	Log out of a specific device remotely.
UC-17.01	Create Forum Category	Admin	Allows admins to create a new forum category.
UC-17.02	Update Forum Category	Admin	Allows admins to update an existing forum category.
UC-17.03	Delete Forum Category	Admin	Allows admins to soft-delete a forum category.
UC-14.01	View Forum Category List	Admin	Allows admins to view the list of forum categories.
UC-14.02	View Forum Category Detail	Admin	Allows admins to view the details of a specific forum category.

ID	Use Case	Actors	Description
UC-17.06	Restore Forum Category	Admin	Restore a deleted Category forum by setting deletedAt back to null.
UC-03	Job Browsing	Guest, Client, Freelancer	Enables guests to browse and search for available job listings.
UC-03.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters (category, budget, skills, status, keywords) and sorting options.
UC-03.2	View Job Detail	Guest, Client, Freelancer	Get full job details: title, description, budget, skills, client info, proposals count, attachments, timeline.
UC-04	Manage Forum	Guest, Client, Freelancer, Admin	Manages forum operations including viewing, creating, editing, and moderating posts and comments.
UC-04.1	View Forum Categories	Guest, Client, Freelancer, Admin	Displays all available forum categories for browsing.
UC-04.2	View Forum Category Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific forum category and its sub-forums.
UC-04.3	View Post List	Guest, Client, Freelancer, Admin	Retrieves paginated list of posts within a forum category with filters and sorting.
UC-04.4	View Post Detail	Guest, Client, Freelancer, Admin	Displays full post content, author info, creation date, and associated tags.
UC-04.5	View Comments	Guest, Client, Freelancer, Admin	Retrieves all comments related to a specific forum post.
UC-04.6	View Comment Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific comment, including content, author, and timestamps.
UC-04.7	Create Post	Client, Freelancer	Allows freelancers to create new discussion posts or questions in the forum.
UC-04.8	Edit Post	Client, Freelancer	Allows freelancers to edit their own forum posts.

ID	Use Case	Actors	Description
UC-04.9	Delete Post	Client, Freelancer, Admin	Allows post owners or admins to delete a forum post.
UC-04.10	Like Post	Client, Freelancer	Allows freelancers to like a forum post.
UC-04.11	Unlike Post	Client, Freelancer	Allows freelancers to remove their like from a forum post.
UC-04.12	Add Comment	Client, Freelancer	Allows freelancers to add comments to a forum post.
UC-04.13	Edit Comment	Client, Freelancer	Allows freelancers to edit their own comments.
UC-04.14	Delete Comment	Client, Freelancer	Allows comment owners or admins to delete a comment.
UC-04.15	Like Comment	Client, Freelancer	Allows freelancers to like a comment.
UC-04.16	Unlike Comment	Client, Freelancer	Allows freelancers to remove their like from a comment.
UC-04.17	Report Content	Client, Freelancer	Allows freelancers to report inappropriate content in forum posts or comments.
UC-06	Manage Job	Client, Freelancer	Manages job postings including creation, editing, deletion, and status changes.
UC-06.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters and sorting options.
UC-06.2	Post New Job	Client	Allows clients to create and publish new job postings.
UC-06.3	Edit Job	Client	Allows clients to edit their own job postings.
UC-06.4	Delete Job	Client	Allows clients to delete their own job postings.
UC-06.5	Change Job Status	Client	Allows clients to update job status (Open, In Progress, Closed, Cancelled).
UC-06.6	View List Bookmarked Job	Freelancer	Retrieves list of jobs bookmarked by the freelancer.
UC-06.7	Bookmark Job	Freelancer	Allows freelancers to save/bookmark a job for later reference.
UC-06.8	Remove Bookmark	Freelancer	Allows freelancers to remove a bookmarked job from their list.

ID	Use Case	Actors	Description
UC-13	Manage Contract	Client, Freelancer, Admin	Manages contractual agreements, milestones, deliverables, and file exchanges.
UC-13.01	Create Contract	Client	Allows clients to create and initiate a new contract with a freelancer.
UC-13.02	Update Contract	Client	Allows clients to update contract terms, budget, milestones, or deadlines before signing.
UC-13.03	Complete Contract	Client, Freelancer	Marks an active contract as completed after all milestones are approved and settled.
UC-13.04	View Contract List	Client, Freelancer, Admin	Retrieves paginated list of contracts with status filters and sorting.
UC-13.05	View Contract Detail	Client, Freelancer, Admin	Displays full contract details including terms, parties, status, and milestones.
UC-13.06	Sign Contract	Freelancer, Client	Allows users to digitally accept and sign a contract to activate it.
UC-13.07	Cancel Contract	Client, Freelancer	Allows either party to cancel or terminate a contract prior to completion.
UC-13.08	View Contract Files	Client, Freelancer, Admin	Manages and accesses all files and documentation associated with a contract.
UC-13.09	Upload Contract File	Client, Freelancer	Allows users to upload documents and deliverables to the contract workspace.
UC-13.10	Delete Contract File	Client, Freelancer, Admin	Allows the uploader or admin to delete files from a contract workspace.
UC-10	Manage Profile	Freelancer	Manages freelancer profile information and settings.
UC-10.1	View Detailed Freelancer Profile	Freelancer	Displays full freelancer profile including skills, portfolio, experience, and ratings.
UC-10.2	Update Your Detailed Freelancer Profile	Freelancer	Allows freelancers to edit and update their profile information.

ID	Use Case	Actors	Description
UC-11	Manage Skills	Freelancer	Manages freelancer skill set and proficiency levels.
UC-11.1	View List Freelancer Skill	Freelancer	Retrieve the list of skills added by the freelancer.
UC-11.2	Add Freelancer Skill	Freelancer	Allows freelancers to add new skills to their profile.
UC-11.3	Delete Freelancer Skill	Freelancer	Allows freelancers to remove skills from their profile.
UC-12	Manage Portfolio	Freelancer	Manages freelancer portfolio projects and work samples.
UC-12.1	View Portfolio List	Freelancer	Retrieve the list of portfolios created by the freelancer.
UC-12.2	Add New Portfolio	Freelancer	Allows freelancers to add new portfolio items with images and descriptions.
UC-12.3	View Portfolio Detail	Freelancer	Displays full details of a portfolio item including media, description, and technologies used.
UC-12.4	Update Portfolio	Freelancer	Allows freelancers to edit and update their portfolio items.
UC-12.5	Delete Portfolio	Freelancer	Allows freelancers to remove portfolio items.
UC-15	Identity Verification	Freelancer	Manages freelancer identity verification document submission and review.
UC-15.1	Upload Identity Documents	Freelancer	Allows freelancers to upload ID card, passport, or license documents.
UC-15.2	View Verification Status	Freelancer	Displays current identity verification status (Pending, Approved, Rejected).
UC-16	View Client Profile	Guest, Client, Freelancer	Manages client company profile details and public showcase.
UC-16.1	View Client Profile Detail	Guest, Client, Freelancer	Displays client company details, bio, website, and job history.
UC-16.2	Update Company Information	Client	Allows clients to update company name, bio, website URL, and industry sector.
UC-17	Social Links	Guest, Client, Freelancer	Manages external social and portfolio links on user profiles.
UC-17.1	View Social Links	Guest, Client, Freelancer	Retrieves social links associated with a user profile.

ID	Use Case	Actors	Description
UC-17.2	Add Social Link	Client, Freelancer	Allows users to add external social links to their profile.
UC-17.3	Delete Social Link	Client, Freelancer	Allows users to remove social links from their profile.
UC-18	Favorites	Client	Manages saved favorite freelancer profiles for clients.
UC-18.1	View Favorite Freelancers	Client	Retrieves paginated list of client's favorite freelancers.
UC-18.2	Add to Favorites	Client	Allows clients to save a freelancer profile to their favorites.
UC-18.3	Remove from Favorites	Client	Allows clients to remove a freelancer from their favorites list.
UC-19	Following	Client, Freelancer	Manages followed freelancer network connections.
UC-19.1	View Following List	Client, Freelancer	Retrieves list of freelancers currently followed by the user.
UC-19.2	Follow Freelancer	Client, Freelancer	Allows users to follow a freelancer for activity updates.
UC-19.3	Unfollow Freelancer	Client, Freelancer	Allows users to stop following a freelancer.
UC-20	Manage Notifications	Client, Freelancer, Admin	Manages user in-app notification center and read states.
UC-20.1	View Notification List	Client, Freelancer, Admin	Retrieves user notifications list with unread counter.
UC-20.2	Mark Notification as Read	Client, Freelancer, Admin	Sets read status of a specific notification to true.
UC-20.3	Mark All Notifications as Read	Client, Freelancer, Admin	Bulk marks all user notifications as read.
UC-20.4	Delete Notification	Client, Freelancer, Admin	Soft-deletes a notification from user's notification list.
UC-21	Account Management	Client, Freelancer	Manages user role switching and complementary role activation.

ID	Use Case	Actors	Description
UC-21.1	Switch Role	Client, Freelancer	Allows dual-role users to switch working context between Client and Freelancer.
UC-21.2	Join New Role	Client, Freelancer	Allows single-role users to activate complementary secondary role.
UC-22	Reviews	Guest, Client, Freelancer, Admin	Manages post-contract ratings and detailed feedback reviews.
UC-22.1	View Reviews List	Guest, Client, Freelancer	Retrieves reviews list for a user profile or contract.
UC-22.2	View Review Detail	Guest, Client, Freelancer	Displays rating breakdown scores and detailed comment text.
UC-22.3	Write Review	Client, Freelancer	Allows contract participants to review each other upon project completion.
UC-22.4	Edit Review	Client, Freelancer	Allows review authors to update ratings or feedback text.
UC-22.5	Delete Review	Client, Freelancer, Admin	Soft-deletes a review record.
UC-23	Manage Response Review	Client, Freelancer, Admin	Manages recipient responses to reviews.
UC-23.1	Respond to Review	Client, Freelancer	Allows review recipients to post a public response.
UC-23.2	Edit Response	Client, Freelancer	Allows response authors to modify response text.
UC-23.3	Delete Response	Client, Freelancer, Admin	Soft-deletes a review response.
UC-24	Manage Profile	Client, Freelancer, Admin	Manages account credentials, profile details, and avatar.
UC-24.1	Get Profile	Client, Freelancer, Admin	Retrieves general account profile information.
UC-24.2	Update Profile	Client, Freelancer, Admin	Updates general profile details (name, phone, bio, location).

ID	Use Case	Actors	Description
UC-24.3	Change Password	Client, Freelancer, Admin	Allows authenticated users to change password securely.
UC-24.4	Upload Avatar	Client, Freelancer, Admin	Allows users to upload and crop profile avatar image.
UC-25	Profile Update Review	Admin	Manages manual admin review of profile modification requests.
UC-25.1	View Pending Profile Update Requests	Admin	Retrieves list of profile update requests awaiting review.
UC-25.2	View Profile Update Request Detail	Admin	Displays side-by-side diff comparison for profile changes.
UC-25.3	Approve Profile Update Request	Admin	Approves profile modification request and updates live profile.
UC-25.4	Reject Profile Update Request	Admin	Rejects profile request with mandatory reason note.
UC-25.5	View Profile Update Request History	Admin	Views history of processed profile update requests.
UC-25.6	Update Profile Expert	Expert, Admin	Updates expert consultant profile and homepage showcase.
UC-47.01	View Pending Profile Update Request List	Admin	Retrieves profile update requests awaiting manual review.
UC-47.02	View Profile Update Request Detail	Admin	Displays a request and compares current profile data with proposed changes.
UC-47.03	Approve Profile Update Request	Admin	Approves a request and applies its proposed changes to the live profile.
UC-47.04	Reject Profile Update Request	Admin	Rejects a request with a reason while preserving the current profile data.
UC-47.05	View Profile Update Request History	Admin	Displays previously approved and rejected profile update requests.

ID	Use Case	Actors	Description
UC-47.06	Update Profile Expert	Expert	Allows an Expert to edit the profile used in the public expert experience.
UC-07.03	Upload CV	Freelancer	Uploads a supported CV file and associates it with the Freelancer profile.
UC-07.04	View CV	Freelancer	Views or downloads the CV currently associated with the Freelancer profile.
UC-07.05	Replace CV	Freelancer	Replaces the existing CV with a newly uploaded supported file.
UC-07.06	Delete CV	Freelancer	Removes the CV associated with the Freelancer profile.
UC-33	Manage Dispute	Client, Freelancer, Admin	Manages formal dispute arbitration on funded contract milestones.
UC-33.01	Open Dispute	Client, Freelancer	Allows a client or freelancer to initiate a formal dispute on a funded milestone.
UC-33.02	View Dispute List	Client, Freelancer, Admin	Retrieves paginated overview of disputes with status filtering and deadline badges.
UC-33.03	View Dispute Detail	Client, Freelancer, Admin	Displays full dispute case records, claims, evidence, fees, and resolutions.
UC-33.04	Respond to Dispute	Client, Freelancer	Allows the respondent to submit a rebuttal statement and counter-evidence.
UC-33.05	Respond to Dispute Resolution	Client, Freelancer	Allows involved parties to accept or reject the arbitrator's proposed decision.
UC-33.06	Propose Dispute Resolution	Admin	Allows an administrator/arbitrator to evaluate claims and propose a fund split decision.
UC-43	Manage Payment	Client, Freelancer, Admin	Handles platform financial transactions, escrow deposits, payouts, and fees via Stripe.
UC-43.01	Connect Stripe Account	Freelancer	Connects a freelancer's bank account via Stripe Connect Express to receive payouts.
UC-43.02	View Payout History	Freelancer, Admin	Retrieves paginated history of all payout transfers sent to the freelancer's bank.

ID	Use Case	Actors	Description
UC-43.03	View Payout Detail	Freelancer, Admin	Displays detailed ledger breakdown, fees, and arrival estimates for a payout transfer.
UC-43.04	Fund Milestone	Client	Deposits milestone funds into platform escrow using credit card or saved payment methods.
UC-43.05	Refund Payment	Client, Admin	Returns escrowed milestone funds to the client upon contract cancellation or dispute.
UC-43.06	Pay Platform Fee	Client	Processes the platform engagement fee upon contract initiation via Stripe.
UC-43.07	Release Payment	Client	Authorizes the release of escrowed milestone funds to the freelancer upon work approval.
UC-43.08	Pay Dispute Fee	Client, Freelancer	Processes arbitration fee payment within the 24-hour dispute window via Stripe.
UC-43.09	View Payment History	Client, Freelancer, Admin	Displays a comprehensive, paginated ledger of all account financial transactions.
UC-43.10	View Payment Detail	Client, Freelancer, Admin	Shows itemized receipt details, fees, and downloadable invoice for a transaction.
UC-43.11	Execute Dispute Resolution Payment	Admin, System	Executes split payouts (client refund and freelancer transfer) upon dispute finalization.
UC-43.12	Handle Stripe Webhook Event	Stripe, System	Listens for and verifies asynchronous Stripe webhook events to reconcile database state.
UC-43.13	Sync Payment Intent Status	Client, System	Synchronizes real-time payment intent and checkout status directly with the Stripe API.
UC-46	Job Category Management	Admin, Freelancer	Provides administrative category management and freelancer categorization for jobs.
UC-46.01	View Job Category List	Admin	Allows an administrator to view, search, and filter job categories with pagination.
UC-46.02	View Job Category Detail	Admin	Displays detailed information of a specific job category including associated jobs.
UC-46.03	Create Job Category	Admin	Allows an administrator to create a new job category with name, description, and status.

ID	Use Case	Actors	Description
UC-46.04	Update Job Category	Admin	Allows an administrator to modify existing job category details and status.
UC-46.05	Delete Job Category	Admin	Allows an administrator to soft-delete a job category after verifying active job dependencies.
UC-46.06	Restore Job Category	Admin	Allows an administrator to restore a soft-deleted job category.
UC-46.07	View Job Category List	Freelancer	Allows freelancers to browse active job categories when filtering available jobs.
UC-46.08	View Job Category Detail	Freelancer	Allows freelancers to view category information and open jobs under that category.

3. Functional Requirements

3.1 System Functional Overview

3.1.1 Screens Flow

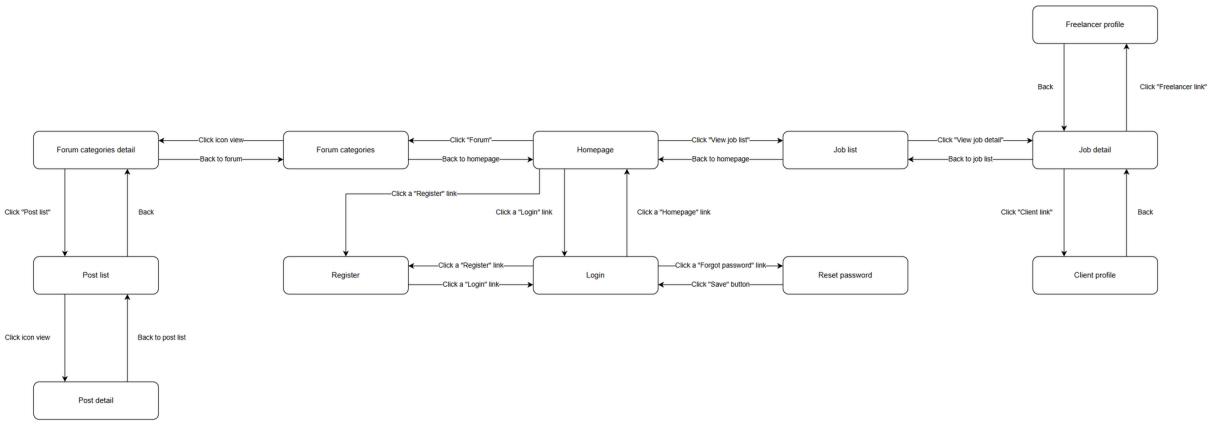

Figure 23: Screen Flow of Guest

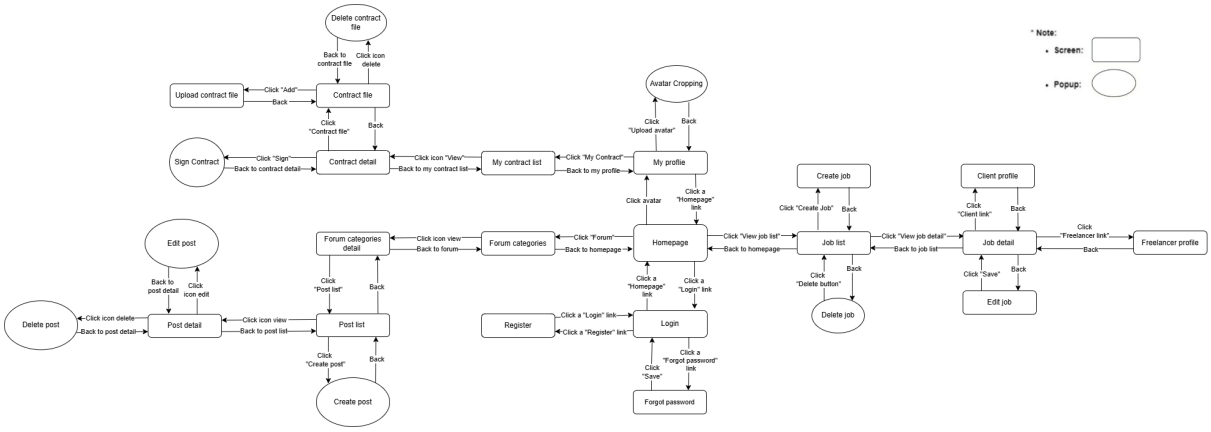

Figure 24: Screen Flow of Client

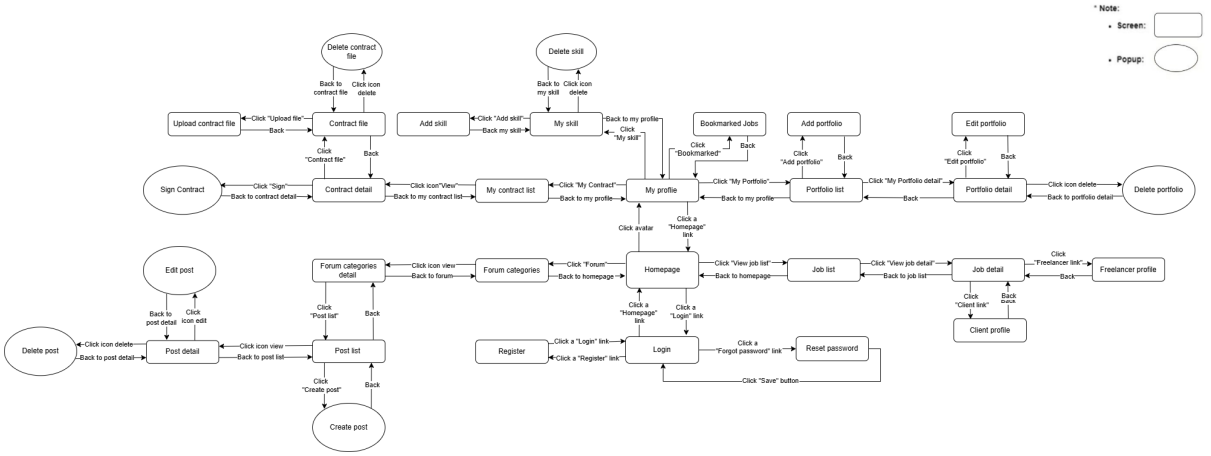

Figure 25: Screen Flow of Freelancer

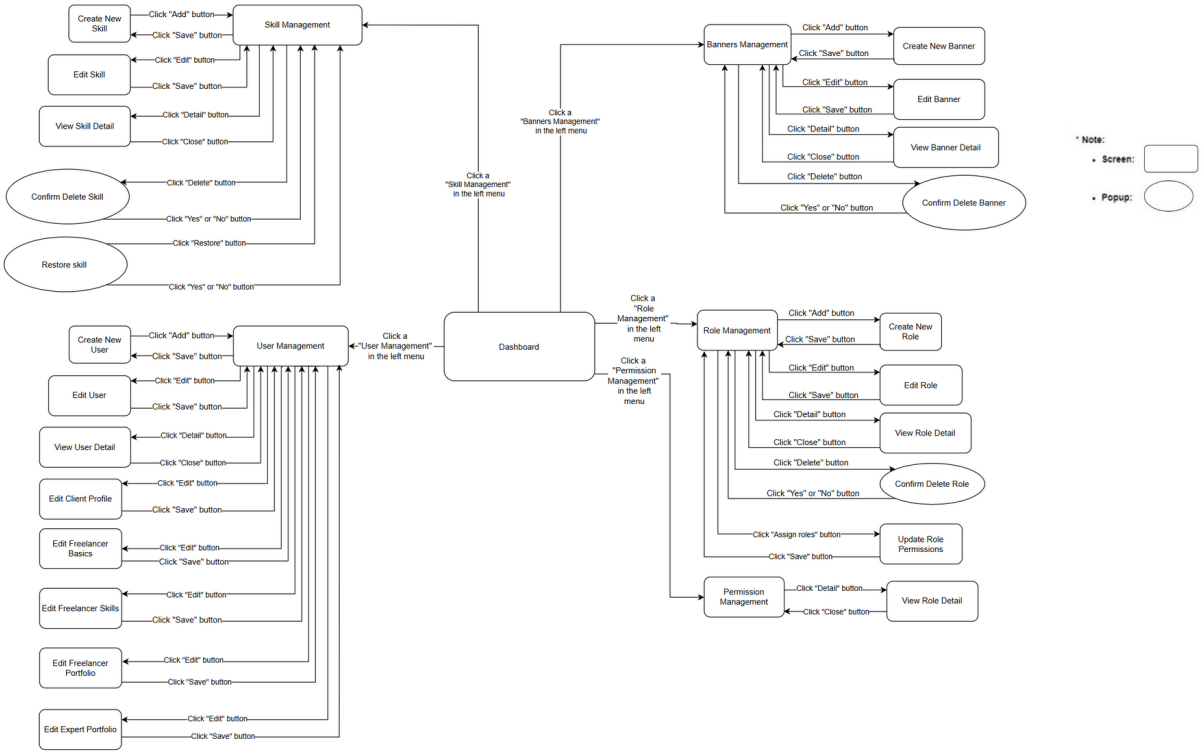

Figure 26: Screen Flow of Admin (Part 1)

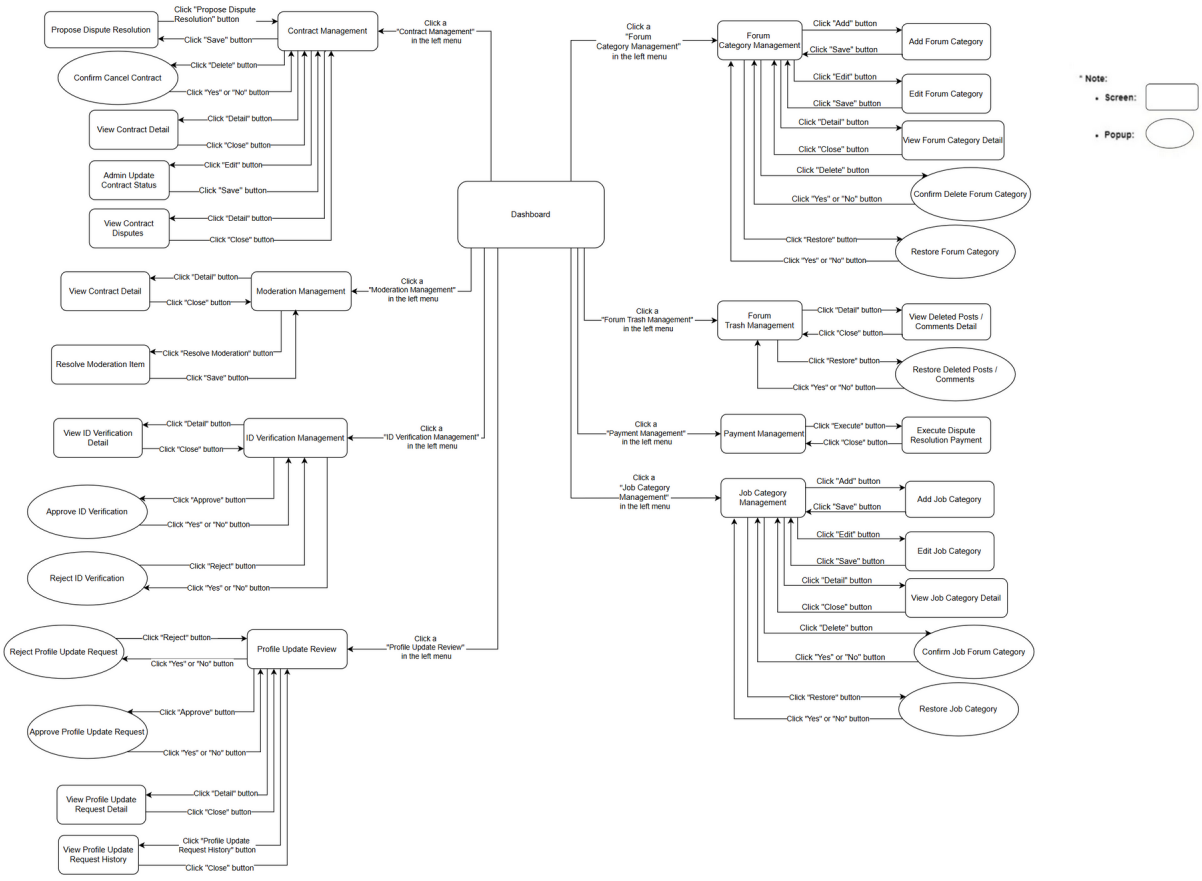

Figure 27: Screen Flow of Admin (Part 2)

3.1.2 Screen Descriptions

No	Actor	Screen Name	Description
1	Guest	Homepage	The main landing page introducing the platform, top job categories, featured job opportunities, and top-rated freelancers.
2	Guest	Login	Allows users to sign in using their registered email and password with secure JWT access and refresh token authentication.
3	Guest	Google Login	OAuth 2.0 third-party authentication interface allowing users to sign in or register instantly using their Google account.
4	Guest	Register	Allows new visitors to create an account by providing basic credentials and choosing their primary role (Client or Freelancer).
5	Guest	Forgot Password	Allows users to request a password reset email containing a secure, time-limited verification token.
6	Guest	Reset Password	Allows users to set a new account password after successfully verifying the reset link received in their email.
7	Guest	Job List	Public job directory displaying all active job postings with keyword search, category filtering, and budget sorting.
8	Guest	Job Detail	Displays full details of a specific job posting including title, description, budget, required skills, and client statistics.

No	Actor	Screen Name	Description
9	Guest	Freelancer Profile	Public profile screen displaying a freelancer's professional title, bio, hourly rate, skills, reviews, and portfolio.
10	Guest	Client Profile	Public profile screen displaying a client's company description, posted jobs history, total spend, and feedback rating.
11	Guest	Forum Categories	Main forum overview screen displaying all discussion categories, total threads, and community activity statistics.
12	Guest	Forum Category Detail	Displays the list of discussion threads and topics contained within a selected forum category.
13	Guest	Post List	Displays all active discussion posts within a category with options to filter by popularity, latest activity, or keywords.
14	Guest	Post Detail	Displays the full text of a forum post, author information, timestamps, and the interactive threaded comments section.
15	Freelancer	My Profile	Central freelancer profile dashboard displaying bio, professional title, education, languages, hourly rate, and profile strength.
16	Freelancer	Edit Freelancer Profile	Form screen allowing freelancers to update their bio, professional title, education, availability status, and rate.

No	Actor	Screen Name	Description
17	Freelancer	My Skills	Displays the full list of skills currently attached to the freelancer's profile along with proficiency levels.
18	Freelancer	Add Skill	Searchable modal dialog to select skills from the master catalog and attach them to the freelancer profile.
19	Freelancer	Delete Skill	Confirmation dialog allowing the freelancer to remove an attached skill from their profile.
20	Freelancer	Upload CV	Form screen allowing the freelancer to upload a curriculum vitae (CV/resume) document in PDF or DOCX format.
21	Freelancer	View CV	Document previewer interface allowing the freelancer to view and download their currently uploaded CV document.
22	Freelancer	Replace CV	Screen allowing the freelancer to select and upload a new CV document to overwrite their existing resume.
23	Freelancer	Delete CV	Confirmation dialog prompting the freelancer to confirm permanent deletion of their attached CV file.
24	Freelancer	Portfolio List	Displays a portfolio gallery of project showcases, case studies, and prior work samples created by the freelancer.
25	Freelancer	Add Portfolio	Form screen allowing the freelancer to create a new portfolio item with title, description, skills used, and media files.

No	Actor	Screen Name	Description
26	Freelancer	Portfolio Detail	Modal/screen showing detailed case study information, full-resolution media, and live project URLs of a portfolio entry.
27	Freelancer	Edit Portfolio	Form screen allowing the freelancer to modify description, skills, links, or attachments of an existing portfolio item.
28	Freelancer	Delete Portfolio	Confirmation dialog prompting the freelancer to confirm the soft deletion of a portfolio project.
29	Freelancer	Identity Verification Upload	Screen allowing the freelancer to upload government-issued ID card/passport images and capture a selfie for eKYC verification.
30	Freelancer	Verification Status	Screen displaying the current status of the eKYC review (Pending Review, Approved, or Rejected with reason).
31	Freelancer	Bookmarked Jobs	Screen displaying all job postings that the freelancer has bookmarked for future review or proposal submission.
32	Freelancer	Saved Searches	Screen allowing the freelancer to view, execute, and manage custom saved job search filters and keyword queries.
33	Freelancer	Job Alerts	Screen allowing the freelancer to configure notification alerts for newly posted jobs matching specific skill criteria.

No	Actor	Screen Name	Description
34	Freelancer	My Proposals List	Screen displaying all proposals submitted by the freelancer with current statuses (Pending, Accepted, Rejected).
35	Freelancer	Submit Proposal	Form screen allowing the freelancer to apply to a job with a cover letter, bid amount, delivery timeline, and milestones.
36	Freelancer	Proposal Detail	Detailed view of a submitted proposal showing proposed milestone breakdown, attachments, and client responses.
37	Freelancer	Edit Proposal	Form screen allowing the freelancer to adjust bid pricing, timeline, or cover letter of a pending proposal.
38	Freelancer	Withdraw Proposal	Confirmation dialog allowing the freelancer to withdraw an active proposal before client acceptance.
39	Freelancer	My Contract List	Screen displaying all contracts associated with the freelancer categorized by status (Active, Completed, Cancelled).
40	Freelancer	Contract Detail	Workspace displaying contract terms, milestone schedules, deadlines, escrow funding status, and activity timeline.
41	Freelancer	Sign Contract	Digital signature screen allowing the freelancer to formalize acceptance of the contract terms and deliverables.
42	Freelancer	Contract Files	Document repository displaying all project assets, specifications, and files shared within the contract workspace.

No	Actor	Screen Name	Description
43	Freelancer	Upload Contract File	Form screen allowing the freelancer to upload deliverable files or reference documents to the contract repository.
44	Freelancer	Delete Contract File	Confirmation dialog allowing the freelancer to delete a file they uploaded to the contract workspace.
45	Freelancer	Milestone List	Displays all agreed milestone deliverables of the contract along with individual payment amounts and deadlines.
46	Freelancer	Milestone Detail	Screen displaying detailed instructions, attached files, progress logs, and submission history for a specific milestone.
47	Freelancer	Update Milestone Progress	Form screen allowing the freelancer to post progress notes, code links, or work-in-progress samples for a milestone.
48	Freelancer	Submit Milestone	Screen allowing the freelancer to formally submit finished deliverables and request client approval and payout.
49	Freelancer	Delete Milestone Submission	Confirmation dialog allowing the freelancer to cancel or retract a pending milestone submission.
50	Freelancer	Milestone Files	Dedicated file viewer showing assets and work items uploaded specifically for a contract milestone.
51	Freelancer	Open Dispute	Form screen allowing the freelancer to open a formal dispute regarding milestone deliverables or payment non-release.

No	Actor	Screen Name	Description
52	Freelancer	Dispute Detail	Screen displaying dispute claims, evidence files, mediator messages, and resolution proposals from both parties.
53	Freelancer	Respond to Dispute	Form screen allowing the freelancer to submit rebuttal statements and supporting proof during arbitration.
54	Freelancer	Review List	Displays the list of feedback ratings and performance reviews received from clients upon contract completion.
55	Freelancer	Review Detail	Modal/screen displaying specific multi-category ratings, detailed client comments, and response timestamps.
56	Freelancer	Write Review	Form screen allowing the freelancer to rate and review the client after a contract has been completed.
57	Freelancer	Edit Review	Form screen allowing the freelancer to modify their submitted rating and comments within the allowed review window.
58	Freelancer	Respond to Review	Form screen allowing the freelancer to post an official public response to a client review on their profile.
59	Freelancer	Stripe Connect Onboarding	External redirect interface allowing the freelancer to set up their Stripe Express payout account and banking info.
60	Freelancer	Payout History	Screen displaying historical escrow earnings releases, transaction fees, and payout deposit statuses.

No	Actor	Screen Name	Description
61	Freelancer	Payout Detail	Detailed receipt view for an individual payout transaction showing date, gross earnings, net transfer, and bank reference.
62	Freelancer	Conversation List	Messaging inbox displaying active direct chats with clients, unread counters, and snippet previews.
63	Freelancer	Direct Chat	Real-time messaging workspace supporting instant text chat, deliverable file attachments, and delivery receipts.
64	Freelancer	Notification List	Screen displaying user notifications for job invites, proposal updates, milestone payments, and forum interactions.
65	Freelancer	Switch Role	Quick switch control allowing the user to seamlessly transition between Freelancer and Client workspace views.
66	Freelancer	Join New Role	Registration screen allowing an existing single-role freelancer to activate the complementary Client role.
67	Client	Client Dashboard	Main workspace dashboard for clients showing active job postings, proposal counts, active contracts, and spent funds.
68	Client	Client Profile	Company profile screen displaying corporate name, website URL, industry description, and past hire ratings.
69	Client	Edit Company Information	Form screen allowing the client to update their business profile, logo, headquarters address, and company bio.

No	Actor	Screen Name	Description
70	Client	Post Job	Multi-step job creation wizard allowing the client to define job title, category, required skills, budget, and scope.
71	Client	My Jobs	Screen displaying all job listings posted by the client with status filters (Draft, Open, In Progress, Closed, Cancelled).
72	Client	Edit Job	Form screen allowing the client to modify description, budget range, deadline, or skill tags of an open job posting.
73	Client	Delete Job	Confirmation modal prompting the client to confirm soft deletion or cancellation of an existing job posting.
74	Client	Change Job Status	Control action allowing the client to pause, close, or reopen an existing job posting.
75	Client	Job Proposals List	Screen displaying all incoming proposals submitted by freelancers for a specific job, with sorting and filtering.
76	Client	Proposal Detail (Client View)	Detailed evaluation screen displaying a freelancer's proposal, cover letter, rate, and portfolio match.
77	Client	Accept Proposal	Action modal to accept a freelancer's proposal and transition automatically into contract creation.
78	Client	Reject Proposal	Action dialog allowing the client to decline a proposal with optional constructive feedback.

No	Actor	Screen Name	Description
79	Client	Create Contract	Form screen allowing the client to generate a formal contract with defined fixed or milestone payment structures.
80	Client	My Contract List (Client)	Screen displaying all current and archived contracts managed by the client with status indicators.
81	Client	Contract Detail (Client)	Central management interface for monitoring project milestones, communication, deliverables, and payments.
82	Client	Update Contract Terms	Form screen allowing the client to propose amendments to contract deadlines, scope, or milestone budgets.
83	Client	Sign Contract	Screen allowing the client to digitally sign and ratify the contract agreement.
84	Client	Cancel Contract	Screen allowing the client to request mutual contract termination and configure escrow refund allocations.
85	Client	Contract Files (Client)	Document repository displaying shared project specifications, requirements documents, and assets.
86	Client	Upload Contract File	Modal allowing the client to upload additional reference materials, NDAs, or briefs to the contract repository.
87	Client	Delete Contract File	Confirmation dialog prompting the client to remove an uploaded document from the contract repository.

No	Actor	Screen Name	Description
88	Client	Milestone Management	Interface displaying the contract's milestone timeline, deposit status, and deliverable review actions.
89	Client	Fund Milestone	Secure checkout screen allowing the client to fund an upcoming milestone into escrow using Stripe.
90	Client	View Milestone Submissions	Review screen displaying work files, source repositories, or documentation submitted by the freelancer.
91	Client	Approve Milestone	Action screen enabling the client to approve accepted deliverables and authorize release of escrow funds.
92	Client	Request Milestone Changes	Form modal allowing the client to request revisions from the freelancer with detailed change instructions.
93	Client	Open Dispute (Client)	Form screen allowing the client to initiate formal arbitration when milestone deliverables fail to satisfy terms.
94	Client	Dispute Detail (Client)	Screen displaying the active dispute case, uploaded evidence, mediator commentary, and ruling proposals.
95	Client	Respond to Dispute	Form screen allowing the client to submit rebuttal evidence, project specifications, and timeline records.
96	Client	Pay Dispute Fee	Payment screen enabling the client to pay the arbitration fee deposit required for administrative dispute mediation.

No	Actor	Screen Name	Description
97	Client	Payment History	Screen displaying comprehensive billing transactions, monthly statements, invoices, and platform service fees.
98	Client	Payment Detail	Detailed invoice view showing breakdown of milestone amount, processing fees, taxes, and payment method details.
99	Client	Saved Payment Methods	Wallet management screen allowing clients to securely add, view, and set default payment cards via Stripe.
100	Client	Favorite Freelancers	Screen displaying bookmarked freelancer profiles for rapid direct hiring, re-engagement, or job invitations.
101	Client	Following List	Screen showing all freelancers followed by the client to receive updates on newly published portfolio projects.
102	Client	Write Review (Client)	Form screen allowing the client to submit a performance rating and written evaluation for the hired freelancer.
103	Client	Edit Review (Client)	Form screen allowing the client to update their submitted rating or review text within the permissible editing window.
104	Client	Respond to Review (Client)	Form screen allowing the client to publish a formal response to feedback left by the freelancer.
105	Client	Social Links Management	Screen allowing the client to manage connected social media links and external business references.

No	Actor	Screen Name	Description
106	Admin	Admin Dashboard	Executive dashboard displaying key system health metrics, daily active users, transaction volume, and alerts.
107	Admin	Admin Profile	Screen allowing the system administrator to view and edit administrative profile credentials and preferences.
108	Admin	Admin Change Password	Screen allowing the administrator to update their security password with multi-factor verification.
109	Admin	Role Management	Screen displaying all defined system access roles (Admin, Moderator, Client, Freelancer) and assigned permissions.
110	Admin	Create New Role	Form screen allowing administrators to create a new RBAC role with customized permission sets.
111	Admin	Edit Role	Form screen allowing administrators to rename a role, update its description, or modify its scope of authority.
112	Admin	View Role Detail	Screen showing detailed permissions, system capabilities, and user counts associated with a specific role.
113	Admin	Confirm Delete Role	Confirmation dialog prompting the administrator to confirm the permanent removal of a role.
114	Admin	Update Role Permissions	Matrix configuration screen allowing granular assignment of system capabilities and endpoints to a role.

No	Actor	Screen Name	Description
115	Admin	Permission Management	Screen displaying the comprehensive system permission catalog organized by functional domain module.
116	Admin	View Permission Detail	Modal showing underlying API endpoints, scope parameters, and roles currently granting a specific permission.
117	Admin	User Management	Screen displaying all registered users with filters by role, email verification status, and account state.
118	Admin	Create New User	Form screen allowing administrators to manually provision a user account with specific roles and attributes.
119	Admin	Edit User	Form screen allowing administrators to modify user email, account status (Active, Suspended, Banned), and roles.
120	Admin	View User Detail	Comprehensive profile view showing user activity history, linked contracts, billing records, and audit logs.
121	Admin	Edit Client Profile (Admin)	Form screen allowing administrators to update company profile attributes, verification badges, and notes.
122	Admin	Edit Freelancer Basics	Form screen allowing administrators to update a freelancer's headline, bio, hourly rate, and profile status.
123	Admin	Edit Freelancer Skills	Screen allowing administrators to review, add, or prune skills associated with a freelancer's profile.

No	Actor	Screen Name	Description
124	Admin	Edit Freelancer Portfolio	Screen allowing administrators to inspect, edit, or remove inappropriate or copyrighted portfolio items.
125	Admin	Edit Expert Profile	Form screen allowing administrators to manage verified expert status, endorsements, and homepage spotlight details.
126	Admin	Banners Management	Screen displaying promotional and announcement banners with active schedules, impressions, and target links.
127	Admin	Create New Banner	Form screen allowing administrators to configure a new banner campaign with image asset, placement, and dates.
128	Admin	Upload Banner Image	Media upload interface with image cropping for hero, sidebar, and notification banner placements.
129	Admin	Edit Banner	Form screen allowing administrators to modify copy, destination link, display order, or active state of a banner.
130	Admin	View Banner Detail	Screen displaying detailed engagement analytics, click-through rates, and live preview of a banner.
131	Admin	Confirm Delete Banner	Confirmation dialog prompting the administrator to verify permanent deletion of a banner.
132	Admin	Skill Management	Screen displaying the master skill catalog utilized by the platform for freelancer profiles and job tags.
133	Admin	Create New Skill	Form screen allowing administrators to register a new standardized skill name and categorize it.

No	Actor	Screen Name	Description
134	Admin	Edit Skill	Form screen allowing administrators to rename an existing skill or reassign it to a different category.
135	Admin	View Skill Detail	Screen displaying skill metadata, category relations, and total freelancer and job associations.
136	Admin	Confirm Delete Skill	Confirmation dialog prompting the administrator to soft-delete a skill from the active selection list.
137	Admin	Restore Skill	Action dialog allowing administrators to restore a previously soft-deleted skill back to active availability.
138	Admin	Job Category Management	Screen displaying the hierarchical job category classification tree and industry taxonomies.
139	Admin	Add Job Category	Form screen allowing administrators to create a new job category with title, slug, icon, and parent node.
140	Admin	Edit Job Category	Form screen allowing administrators to update the title, description, or hierarchy of an existing category.
141	Admin	View Job Category Detail	Screen displaying category statistics including active job postings, freelancer counts, and revenue.
142	Admin	Confirm Delete Job Category	Confirmation dialog prompting the administrator to soft-delete an existing job category.
143	Admin	Restore Job Category	Action dialog allowing administrators to restore a previously soft-deleted job category.

No	Actor	Screen Name	Description
144	Admin	Forum Category Management	Screen displaying all community forum categories, permission rules, and moderation assignments.
145	Admin	Add Forum Category	Form screen allowing administrators to configure a new discussion section with name, description, and icon.
146	Admin	Edit Forum Category	Form screen allowing administrators to update forum category titles, display ordering, or access rules.
147	Admin	View Forum Category Detail	Screen displaying detailed forum category metrics, thread volume, active participants, and post frequency.
148	Admin	Confirm Delete Forum Category	Confirmation dialog prompting the administrator to soft-delete a forum category.
149	Admin	Restore Forum Category	Action dialog allowing administrators to reactivate a previously soft-deleted forum category.
150	Admin	Moderation Queue	Screen displaying flagged user reports, reported offensive posts, spam comments, and policy violation cases.
151	Admin	View Moderation Item Detail	Screen displaying original reported content, reporter comments, automated AI toxicity scores, and context.
152	Admin	Resolve Moderation Item	Action screen allowing administrators to dismiss flags, issue user warnings, delete content, or suspend accounts.

No	Actor	Screen Name	Description
153	Admin	Forum Trash Management	Screen displaying all soft-deleted forum threads and comments awaiting administrative review or permanent purge.
154	Admin	View Deleted Posts/Comments Detail	Screen showing original deleted post text, deletion metadata, reason, and author history.
155	Admin	Restore Deleted Posts/Comments	Action dialog allowing administrators to recover and reinstate accidentally or incorrectly removed forum posts.
156	Admin	ID Verification Management	Screen displaying queue of pending eKYC identity submissions awaiting official verification review.
157	Admin	View ID Verification Detail	Screen displaying government ID photos, automated face match score, OCR data, and liveness pose validation.
158	Admin	Approve ID Verification	Action dialog allowing administrators to verify identity documents and assign the Verified Freelancer badge.
159	Admin	Reject ID Verification	Action dialog allowing administrators to reject an eKYC submission with specific rejection guidance to the user.
160	Admin	Profile Update Review	Screen displaying incoming requests from freelancers to update verified identity and sensitive profile fields.
161	Admin	View Profile Update Request Detail	Side-by-side comparison screen highlighting changes between current profile data and requested modifications.

No	Actor	Screen Name	Description
162	Admin	View Profile Update Request History	Audit log screen showing previously processed profile update requests, administrative decisions, and notes.
163	Admin	Approve Profile Update Request	Action dialog allowing administrators to accept changes and immediately publish them to the live profile.
164	Admin	Reject Profile Update Request	Action dialog allowing administrators to decline requested revisions with feedback provided to the applicant.
165	Admin	Admin Contract Oversight	Screen displaying all platform contracts across all users with filters for status, date, and dispute flags.
166	Admin	Admin View Contract Detail	Comprehensive audit screen displaying full contract history, escrow transactions, milestones, and dispute logs.
167	Admin	Admin Update Contract Status	Action screen enabling administrators to override contract status in exceptional administrative scenarios.
168	Admin	View Contract Disputes	Screen displaying all escalated disputes requiring third-party administrative mediation and binding rulings.
169	Admin	Propose Dispute Resolution	Form screen allowing administrator mediator to propose binding escrow fund distribution between the parties.

No	Actor	Screen Name	Description
170	Admin	Confirm Cancel Contract	Confirmation dialog enabling administrators to terminate stalled or abandoned contracts and refund escrow.
171	Admin	Execute Dispute Resolution Payment	Screen allowing administrators to release escrow balances and execute final payouts per the dispute ruling.
172	Admin	Session Management (Admin)	Screen displaying active user login sessions across devices with client IP, user agent, and force-logout controls.

3.1.3 Screen Authorization

No	Screen / Page	Guest	Client	Freelancer	Admin
1	Homepage	✓	✓	✓	
2	Login	✓			
3	Google Login	✓			
4	Register	✓			
5	Forgot Password	✓	✓	✓	✓
6	Reset Password	✓	✓	✓	✓
7	Job List	✓	✓	✓	
8	Job Detail	✓	✓	✓	
9	Freelancer Profile	✓	✓	✓	
10	Client Profile	✓	✓	✓	
11	Forum Categories	✓	✓	✓	✓
12	Forum Category Detail	✓	✓	✓	✓
13	Post List	✓	✓	✓	✓
14	Post Detail	✓	✓	✓	✓
15	My Profile			✓	

No	Screen / Page	Guest	Client	Freelancer	Admin
16	Edit Freelancer Profile			✓	
17	My Skills			✓	
18	Add Skill			✓	
19	Delete Skill			✓	
20	Upload CV			✓	
21	View CV			✓	
22	Replace CV			✓	
23	Delete CV			✓	
24	Portfolio List			✓	
25	Add Portfolio			✓	
26	Portfolio Detail	✓	✓	✓	
27	Edit Portfolio			✓	
28	Delete Portfolio			✓	
29	Identity Verification Upload			✓	
30	Verification Status			✓	
31	Bookmarked Jobs			✓	
32	Saved Searches			✓	
33	Job Alerts			✓	
34	My Proposals List			✓	
35	Submit Proposal			✓	
36	Proposal Detail		✓	✓	
37	Edit Proposal			✓	
38	Withdraw Proposal			✓	
39	My Contract List			✓	
40	Contract Detail		✓	✓	✓
41	Sign Contract		✓	✓	
42	Contract Files		✓	✓	✓
43	Upload Contract File		✓	✓	

No	Screen / Page	Guest	Client	Freelancer	Admin
44	Delete Contract File		✓	✓	✓
45	Milestone List		✓	✓	✓
46	Milestone Detail		✓	✓	✓
47	Update Milestone Progress			✓	
48	Submit Milestone			✓	
49	Delete Milestone Submission			✓	
50	Milestone Files		✓	✓	✓
51	Open Dispute		✓	✓	
52	Dispute Detail		✓	✓	✓
53	Respond to Dispute		✓	✓	
54	Review List	✓	✓	✓	✓
55	Review Detail	✓	✓	✓	✓
56	Write Review		✓	✓	
57	Edit Review		✓	✓	
58	Respond to Review		✓	✓	
59	Stripe Connect Onboarding			✓	
60	Payout History			✓	
61	Payout Detail			✓	
62	Conversation List		✓	✓	
63	Direct Chat		✓	✓	
64	Notification List		✓	✓	✓
65	Switch Role		✓	✓	
66	Join New Role		✓	✓	
67	Client Dashboard		✓		
68	Client Profile	✓	✓	✓	
69	Edit Company Information		✓		
70	Post Job		✓		
71	My Jobs		✓		

No	Screen / Page	Guest	Client	Freelancer	Admin
72	Edit Job		✓		
73	Delete Job		✓		
74	Change Job Status		✓		
75	Job Proposals List		✓		
76	Proposal Detail (Client View)		✓		
77	Accept Proposal		✓		
78	Reject Proposal		✓		
79	Create Contract		✓		
80	My Contract List (Client)		✓		
81	Contract Detail (Client)		✓		
82	Update Contract Terms		✓		
83	Sign Contract		✓	✓	
84	Cancel Contract		✓	✓	✓
85	Contract Files (Client)		✓		
86	Upload Contract File		✓	✓	
87	Delete Contract File		✓	✓	✓
88	Milestone Management		✓		
89	Fund Milestone		✓		
90	View Milestone Submissions		✓	✓	✓
91	Approve Milestone		✓		
92	Request Milestone Changes		✓		
93	Open Dispute (Client)		✓		
94	Dispute Detail (Client)		✓		
95	Respond to Dispute		✓	✓	
96	Pay Dispute Fee		✓	✓	
97	Payment History		✓	✓	
98	Payment Detail		✓	✓	
99	Saved Payment Methods		✓		

No	Screen / Page	Guest	Client	Freelancer	Admin
100	Favorite Freelancers		✓		
101	Following List		✓		
102	Write Review (Client)		✓		
103	Edit Review (Client)		✓		
104	Respond to Review (Client)		✓		
105	Social Links Management		✓	✓	
106	Admin Dashboard				✓
107	Admin Profile				✓
108	Admin Change Password				✓
109	Role Management				✓
110	Create New Role				✓
111	Edit Role				✓
112	View Role Detail				✓
113	Confirm Delete Role				✓
114	Update Role Permissions				✓
115	Permission Management				✓
116	View Permission Detail				✓
117	User Management				✓
118	Create New User				✓
119	Edit User				✓
120	View User Detail				✓
121	Edit Client Profile (Admin)				✓
122	Edit Freelancer Basics				✓
123	Edit Freelancer Skills				✓
124	Edit Freelancer Portfolio				✓
125	Edit Expert Profile				✓
126	Banners Management				✓
127	Create New Banner				✓

No	Screen / Page	Guest	Client	Freelancer	Admin
128	Upload Banner Image				✓
129	Edit Banner				✓
130	View Banner Detail				✓
131	Confirm Delete Banner				✓
132	Skill Management				✓
133	Create New Skill				✓
134	Edit Skill				✓
135	View Skill Detail				✓
136	Confirm Delete Skill				✓
137	Restore Skill				✓
138	Job Category Management				✓
139	Add Job Category				✓
140	Edit Job Category				✓
141	View Job Category Detail				✓
142	Confirm Delete Job Category				✓
143	Restore Job Category				✓
144	Forum Category Management				✓
145	Add Forum Category				✓
146	Edit Forum Category				✓
147	View Forum Category Detail				✓
148	Confirm Delete Forum Category				✓
149	Restore Forum Category				✓
150	Moderation Queue				✓
151	View Moderation Item Detail				✓
152	Resolve Moderation Item				✓
153	Forum Trash Management				✓

No	Screen / Page	Guest	Client	Freelancer	Admin
154	View Deleted Posts/Comments Detail				✓
155	Restore Deleted Posts/Comments				✓
156	ID Verification Management				✓
157	View ID Verification Detail				✓
158	Approve ID Verification				✓
159	Reject ID Verification				✓
160	Profile Update Review				✓
161	View Profile Update Request Detail				✓
162	View Profile Update Request History				✓
163	Approve Profile Update Request				✓
164	Reject Profile Update Request				✓
165	Admin Contract Oversight				✓
166	Admin View Contract Detail				✓
167	Admin Update Contract Status				✓
168	View Contract Disputes				✓
169	Propose Dispute Resolution				✓
170	Confirm Cancel Contract				✓
171	Execute Dispute Resolution Payment				✓
172	Session Management (Admin)				✓

3.1.4 Non-Screen Functions

#	Feature	System Function	Description
1	**Manage Authentication**	[POST] /api/auth/register [POST] /api/auth/otp [POST] /api/auth/login [POST] /api/auth/refresh-token [POST] /api/auth/logout [POST] /api/auth/forgot-password [GET] /api/auth/google-link [GET] /api/auth/google/callback [GET] /api/auth/me [POST] /api/auth/switch-role [POST] /api/auth/join-role	The Authentication feature handles user identity verification and session establishment. It supports account registration with OTP verification, email/password login with JWT access and refresh tokens, token renewal, logout, and password reset through an email verification link. Users can also sign in with Google OAuth, retrieve the currently authenticated account, and switch or join a role (Client or Freelancer) on the same account.
2	**Manage Account Profile**	[GET] /api/account-profile [PUT] /api/account-profile [PUT] /api/account-profile/password [POST] /api/account-profile/avatar [GET] /api/account-profile/avatar/{userId}/file [GET] /api/clients/{userId} [PUT] /api/clients/me/profile [GET] /api/social-links [POST] /api/social-links [DELETE] /api/social-links/{id} [GET] /api/favorites/freelancers [POST] /api/favorites/freelancers/{freelancerId} [DELETE] /api/favorites/freelancers/{freelancerId} [GET] /api/following/freelancers/discover [GET] /api/following/freelancers [POST] /api/following/freelancers/{freelancerId} [DELETE] /api/following/freelancers/{freelancerId}	The Account Profile feature lets authenticated users manage their account-level information and social network. Users can view and update their account profile, change their password, upload and retrieve their avatar, and manage social links. Clients can view another client's public profile and update their own client profile. Users can also save favorite freelancers, discover and follow freelancers, and unfollow or remove them from the favorites list.

#	Feature	System Function	Description
3	**Manage Freelancer Profile**	[GET] /api/profiles/{id} [PUT] /api/profiles/{id} [POST] /api/profiles/{id}/cv [GET] /api/profiles/{id}/cv/file [POST] /api/profiles/{id}/cv/analyze [DELETE] /api/profiles/{id}/cv [GET] /api/expert-profile/me [PUT] /api/expert-profile/me [PATCH] /api/users/{id}/expert-profile [GET] /api/experts [GET] /api/experts/{id}	The Freelancer Profile feature enables freelancers to build and maintain their professional identity, including bio, professional title, education, availability status and profile strength. Freelancers can upload, download, delete and AI-analyze their CV. The feature also exposes the expert (verified freelancer) profile endpoints, the public expert directory, and allows a freelancer to view and update their own expert profile while administrators can update it on behalf of a user.
4	**Manage Freelancer Skills**	[GET] /api/profiles/{id}/skills [GET] /api/profiles/skills/suggestions [POST] /api/profiles/{id}/skills [DELETE] /api/profiles/skills/{id}	The Freelancer Skills feature allows freelancers to define and showcase their professional capabilities. Users can view the skills attached to a profile, search the skill catalog for suggestions while typing, add a skill together with its proficiency level, and remove a skill that is no longer relevant. This keeps each freelancer's skill set accurate for the client's talent evaluation process.

#	Feature	System Function	Description
5	**Manage Freelancer Portfolio**	[GET] /api/profiles/{id}/portfolios [POST] /api/profiles/{id}/portfolios [GET] /api/profiles/portfolios/{id} [PUT] /api/profiles/portfolios/{id} [DELETE] /api/profiles/portfolios/{id}	The Freelancer Portfolio feature enables freelancers to demonstrate their past work visually. Freelancers can list every portfolio item of a profile, create a new item with project details and media, view the detail of a specific item, update its content or media, and soft-delete an item so that it is no longer displayed publicly. A well-organized portfolio helps freelancers build credibility with prospective clients.
6	**Manage Job**	[GET] /api/jobs [GET] /api/jobs/{slug} [GET] /api/job-categories [GET] /api/job-categories/{slug} [GET] /api/manage-jobs/skills [GET] /api/manage-jobs [GET] /api/manage-jobs/{id} [POST] /api/manage-jobs [PATCH] /api/manage-jobs/{id} [DELETE] /api/manage-jobs/{id} [PATCH] /api/manage-jobs/{id}/status [GET] /api/manage-jobs/bookmarks [POST] /api/manage-jobs/jobs/{slug}/bookmark [GET] /api/manage-jobs/jobs/{slug}/bookmark [DELETE] /api/manage-jobs/bookmarks/{slug}	The Job Management feature is the core marketplace functionality that connects clients with freelancers. Guests and freelancers can browse and search the paginated job list, view a job detail by slug, and browse job categories. Clients can create a job posting with description, budget, required skills and timeline, list their own jobs, view a job detail, update or delete a job, and change its status (Open, In Progress, Closed, Cancelled); they can also look up skills while composing a job. Freelancers can list, add and remove bookmarked jobs and check whether a job is bookmarked.

#	Feature	System Function	Description
7	**Manage Job Categories (Admin)**	[GET] /api/admin/job-categories [GET] /api/admin/job-categories/{id} [POST] /api/admin/job-categories [PATCH] /api/admin/job-categories/{id} [PATCH] /api/admin/job-categories/{id}/ restore [DELETE] /api/admin/job-categories/{id}	The Job Category Management feature allows administrators to maintain the taxonomy used to classify job postings. Administrators can list all categories, view the detail of a category, create a new category, update its name or description, soft-delete a category, and restore a previously deleted category when it is needed again.
8	**Manage Saved Searches**	[GET] /api/saved-searches [GET] /api/saved-searches/{id} [POST] /api/saved-searches [PATCH] /api/saved-searches/{id} [DELETE] /api/saved-searches/{id}	The Saved Search feature allows authenticated users to store job search criteria for later reuse. Users can list their saved searches, view the detail of one saved search, create a new saved search from the current filters, update its name or filter conditions, and delete a saved search that is no longer needed.

#	Feature	System Function	Description
9	**Manage Proposals**	[GET] /api/proposals/my [GET] /api/proposals/jobs/{jobId}/mine [GET] /api/proposals/jobs/{jobId}/submitted [GET] /api/jobs/{jobId}/proposals [GET] /api/proposals/{id} [GET] /api/proposals/{id}/client [POST] /api/proposals/jobs/{jobId} [POST] /api/proposals/jobs/{jobId}/drafts [PATCH] /api/proposals/{id}/draft [PATCH] /api/proposals/{id}/submit [PATCH] /api/proposals/{id}/withdraw [PATCH] /api/proposals/{id}/interview [PATCH] /api/proposals/{id}/reject [PATCH] /api/proposals/{id}/accept	The Proposal Management feature supports the bidding process between freelancers and clients. Freelancers can submit a proposal to a job, save it as a draft, update and submit a draft, withdraw a submitted proposal, and view their own proposals for a job. Clients can list all proposals submitted to their job, view proposal details, and move a proposal through interview, acceptance or rejection, while acceptance can later be used to create a contract.
10	**Manage Contract**	[GET] /api/contracts [GET] /api/contracts/{id} [POST] /api/contracts [PATCH] /api/contracts/{id} [PATCH] /api/contracts/{id}/sign [PATCH] /api/contracts/{id}/complete [PATCH] /api/contracts/{id}/cancel [GET] /api/contracts/{contractId}/files [POST] /api/contracts/{contractId}/files [DELETE] /api/contracts/{contractId}/files/{fileId}	The Contract Management feature manages the agreement between a client and a freelancer throughout the project lifecycle. Users can create a contract, list their contracts, view contract details, update contract terms, sign the contract to confirm participation, complete the contract and request cancellation. Contract participants can also list, upload and delete contract-level files that document the agreement.

#	Feature	System Function	Description
11	**Manage Milestone**	[GET] /api/contracts/{contractId}/milestones [GET] /api/contracts/{contractId}/milestones/{id} [POST] /api/contracts/{contractId}/milestones [PATCH] /api/contracts/{contractId}/milestones/{id} [PATCH] /api/contracts/{contractId}/milestones/{id}/progress [DELETE] /api/contracts/{contractId}/milestones/{id}	The Milestone Management feature breaks a contract into payable deliverables. Contract participants can list and view the milestones of a contract, create a new milestone with its amount and due date, update milestone information, report progress on a milestone, and delete a milestone that is no longer part of the agreement.
12	**Manage Milestone Files**	[GET] /api/contracts/{contractId}/milestones/{milestoneId}/files [POST] /api/contracts/{contractId}/milestones/{milestoneId}/files [DELETE] /api/contracts/{contractId}/milestones/{milestoneId}/files/{fileId}	The Milestone File feature allows contract participants to attach supporting documents and media to a specific milestone. Users can list every file of a milestone, upload a new file, and delete a file that was uploaded by mistake or is no longer relevant.
13	**Manage Milestone Submissions**	[POST] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id} [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/request-changes [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/approve	The Milestone Submission feature supports the deliverable review workflow. A freelancer can submit work for a milestone, and both parties can list the submissions of a milestone and view the detail of one submission. The client can request changes so that the freelancer revises and resubmits the work, or approve the submission to complete the milestone and release the corresponding payment.

#	Feature	System Function	Description
14	**Manage Payments**	[POST] /api/payments/connect/onboarding-link [GET] /api/payments/connect/status [GET] /api/payments/payment-methods [POST] /api/payments/customer-portal [POST] /api/payments/contracts/{contractId}/platform-fee/intent [POST] /api/payments/contracts/{contractId}/platform-fee/checkout-session [POST] /api/payments/contracts/{contractId}/platform-fee/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/intent [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/checkout-session [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/release [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/refund [POST] /api/payments/disputes/{disputeId}/fee/intent [POST] /api/payments/disputes/{disputeId}/fee/checkout-session [POST] /api/payments/disputes/{disputeId}/settle [GET] /api/payments/transactions [POST] /api/payments/intents/{paymentIntentId}/sync [POST] /api/payments/checkout-sessions/{sessionId}/sync [POST] /api/payments/webhook	The Payment Management feature integrates Stripe to provide secure escrow-style transactions. Users can connect a Stripe account and check its onboarding status, list saved payment methods, open the Stripe customer portal, and pay the platform fee or fund a milestone either through a PaymentIntent, a checkout session, or a saved card. Clients can release or refund milestone funds, pay a dispute fee and settle a dispute, and view their transaction history. The system also synchronizes PaymentIntent and CheckoutSession status and receives asynchronous Stripe webhook events.

#	Feature	System Function	Description
15	**Manage Dispute**	[GET] /api/disputes [POST] /api/disputes [GET] /api/disputes/{id} [POST] /api/disputes/{id}/fee [POST] /api/disputes/{id}/response [GET] /api/disputes/milestone/{milestoneId} [POST] /api/disputes/{id}/review [GET] /api/admin/disputes [GET] /api/admin/disputes/{id} [POST] /api/admin/disputes/{id}/decision [POST] /api/admin/disputes/{id}/ final-decision	The Dispute Management feature handles conflicts that arise during a milestone. Participants can open a dispute for a milestone, list and view their disputes, pay the dispute fee, submit a response to the other party's claim, and review the proposed resolution. The dispute of a specific milestone can be looked up. Administrators can list all disputes, view dispute details, issue a decision, and record the final decision that settles the conflict.
16	**Manage Review**	[GET] /api/reviews/contracts/{contractId} [POST] /api/reviews/contracts/{contractId} [GET] /api/reviews/{reviewId} [PATCH] /api/reviews/{reviewId} [DELETE] /api/reviews/{reviewId} [POST] /api/reviews/{reviewId}/response [PATCH] /api/reviews/responses/{responseId} [DELETE] /api/reviews/responses/ {responseId}	The Review Management feature lets contract participants rate and comment on each other after a contract is completed. Users can list the reviews of a contract, create a review, view, update and delete their own review, and respond to a review they received. Responses can also be updated or deleted by their author, helping both parties maintain a trustworthy reputation on the platform.

#	Feature	System Function	Description
17	**Manage Forum**	[GET] /api/forums/categories [GET] /api/forums/categories/top [GET] /api/forums/categories/{id} [GET] /api/forums/categories/by-slug/{slug} [GET] /api/forums/posts [GET] /api/forums/posts/top [GET] /api/forums/posts/{id} [GET] /api/forums/posts/by-slug/{slug} [GET] /api/forums/users/top [POST] /api/forums/posts [DELETE] /api/forums/posts/{id} [GET] /api/forums/posts/{postId}/comments [POST] /api/forums/posts/{postId}/comments [PATCH] /api/forums/posts/{postId}/comments/{id} [DELETE] /api/forums/posts/{postId}/comments/{id} [POST] /api/forums/posts/{postId}/comments/{commentId}/like [POST] /api/forums/posts/{id}/like [GET] /api/forums/posts/{id}/like [POST] /api/forums/posts/{postId}/reports [POST] /api/forums/posts/{postId}/comments/{commentId}/reports [GET] /api/forums/posts/{postId}/is-reported [GET] /api/forums/posts/{postId}/comments/{commentId}/is-reported	The Forum Management feature enables community discussion and knowledge sharing within the platform. Guests and authenticated users can browse forum categories (including the most active categories), browse and search posts (including the most interacted posts and the most active users), and open a post by id or slug. Registered users can create a post, delete their own post, and manage comments (list, add, edit and delete). Posts and comments support a like/unlike toggle and a liker list, and users can report inappropriate posts or comments and check whether they have already reported them.

#	Feature	System Function	Description
18	**Manage Forum Moderation**	[GET] /api/forums/admin/stats [GET] /api/forums/admin/categories [GET] /api/forums/admin/categories/{id} [POST] /api/forums/admin/categories [PATCH] /api/forums/admin/categories/{id} [DELETE] /api/forums/admin/categories/{id} [PATCH] /api/forums/admin/categories/{id}/restore [GET] /api/forums/admin/comments [GET] /api/forums/admin/pending-posts [GET] /api/forums/admin/trash/posts [GET] /api/forums/admin/trash/comments [PATCH] /api/forums/admin/trash/posts/{id}/restore [PATCH] /api/forums/admin/trash/comments/{id}/restore [PATCH] /api/forums/admin/posts/{id}/approve [PATCH] /api/forums/admin/posts/{id}/reject [GET] /api/forums/reports [PATCH] /api/forums/reports/{reportId}/status	The Forum Moderation feature gives administrators the tools to keep the community healthy. Administrators can view forum statistics, list, view, create, update, soft-delete and restore forum categories, and list all comments. New posts can be reviewed through the pending-post queue and either approved or rejected, while the trash lists allow deleted posts and comments to be reviewed and restored. The report queue lists content reported by users, and each report's status can be updated once it has been handled.
19	**Manage Messaging**	[GET] /api/conversations [POST] /api/conversations [GET] /api/conversations/{id}/messages [POST] /api/conversations/{id}/messages [POST] /api/conversations/{id}/files [DELETE] /api/conversations/{id}/messages/{messageId} [PATCH] /api/conversations/{id}/read [PATCH] /api/conversations/{id}/hide [PATCH] /api/conversations/{id}/pin	The Messaging feature provides real-time communication between clients and freelancers. Users can list their conversations, start a new conversation, list and send messages, and send files within a conversation. A message can be deleted or recalled, a conversation can be marked as read to reset its unread counter, hidden from the conversation list, and pinned or unpinned. Messages are delivered in real time through a WebSocket gateway.

#	Feature	System Function	Description
20	**Manage Notification**	[GET] /api/notifications [PATCH] /api/notifications/read-all [PATCH] /api/notifications/{id}/read [DELETE] /api/notifications/{id}	The Notification feature delivers in-app notifications about relevant events such as proposals, contracts, milestones, payments and forum moderation. Users can list their notifications with their read state, mark a single notification or all notifications as read, and delete a notification they no longer want to keep.
21	**Manage Roles**	[GET] /api/roles [GET] /api/roles/{id} [POST] /api/roles [PATCH] /api/roles/{id} [PUT] /api/roles/{id}/permissions [DELETE] /api/roles/{id}	The Role Management feature allows administrators to define and control user permissions. Administrators can list all roles, view the detail of a role together with its permissions, create a new role, update its name or description, replace the permission set assigned to a role, and delete a role that is no longer needed. This provides fine-grained access control and proper segregation of duties across user types.
22	**Manage Permissions**	[GET] /api/permissions [GET] /api/permissions/{id}	The Permission Management feature exposes the catalog of permissions used by the role-based access control system. Administrators can list every available permission and view the details of a single permission so that permissions can be assigned to roles accurately.

#	Feature	System Function	Description
23	**Manage Sessions**	[GET] /api/sessions [GET] /api/sessions/{id} [DELETE] /api/sessions/{id}	The Session Management feature gives authenticated users visibility and control over their active login sessions. Users can list all sessions of their account, view the details of a specific session (such as device and login time), and revoke a session so that the corresponding refresh token can no longer be used.
24	**Manage Users (Admin)**	[POST] /api/users [GET] /api/users [GET] /api/users/{id} [PATCH] /api/users/{id} [PATCH] /api/users/{id}/client-profile [PATCH] /api/users/{id}/freelancer-profile [PUT] /api/users/{id}/freelancer-profile/skills [POST] /api/users/{id}/freelancer-profile/portfolio-items [PATCH] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [DELETE] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [GET] /api/users/skills-catalog	The User Management feature allows administrators to manage accounts from the administration side. Administrators can list users, view user details, create a new user, and update general user information. They can also edit a client profile, edit a freelancer's basic profile and replace the freelancer's skills, manage the freelancer's portfolio items, and read the skill catalog when performing these updates.

#	Feature	System Function	Description
25	**Manage Identity Verification**	[POST] /api/identity-verifications/documents [POST] /api/identity-verifications/ai-check [POST] /api/identity-verifications/liveness-pose [GET] /api/identity-verifications/status [GET] /api/identity-verifications/documents/{id}/file [GET] /api/admin/identity-verifications [GET] /api/admin/identity-verifications/{id} [GET] /api/admin/identity-verifications/{id}/file [PATCH] /api/admin/identity-verifications/{id}/approve [PATCH] /api/admin/identity-verifications/{id}/reject	The Identity Verification feature supports the eKYC process for user accounts. Users can upload identity documents, run an AI check on the submitted documents, capture liveness-pose images, check their verification status, and download an uploaded document file. Administrators can list verification requests, view request details and the attached file, then approve or reject the verification with a reason.
26	**Manage Profile Revisions**	[GET] /api/profile-revisions/me [GET] /api/admin/profile-revisions [GET] /api/admin/profile-revisions/{id} [PATCH] /api/admin/profile-revisions/{id}/approve [PATCH] /api/admin/profile-revisions/{id}/reject	The Profile Revision feature tracks changes made to freelancer profiles so that they can be moderated. A freelancer can view the latest revision of their own profile, while administrators can list all pending revisions, view the detail of a revision, and approve or reject it before the changes become publicly visible.

#	Feature	System Function	Description
27	**Manage Banners**	[GET] /api/banners [GET] /api/banners/image/{filename} [GET] /api/admin/banners [GET] /api/admin/banners/{id} [POST] /api/admin/banners [POST] /api/admin/banners/upload-image [PATCH] /api/admin/banners/{id} [DELETE] /api/admin/banners/{id}	The Advertisement Banner feature manages promotional banners displayed on the platform. The public endpoints return the active banners for a given position and serve the banner image files. Administrators can list and view all banners, create a new banner, upload its image, update its content, target position or schedule, and delete a banner.
28	**Manage Skills (Admin)**	[GET] /api/admin/skills [GET] /api/admin/skills/{id} [POST] /api/admin/skills [PATCH] /api/admin/skills/{id} [PATCH] /api/admin/skills/{id}/restore [DELETE] /api/admin/skills/{id}	The Skill Management feature allows administrators to maintain the master skill catalog used by freelancer profiles and job postings. Administrators can list skills, view the detail of a skill, create a new skill, update its name or category, soft-delete a skill, and restore a previously deleted skill.

#	Feature	System Function	Description
29	**AI Job & Freelancer Recommendation Engine**	[POST] /api/ai/recommendations/jobs [POST] /api/ai/recommendations/freelancers [GET] /api/ai/matching-score/{jobId}/{freelancerId}	The AI Recommendation Engine computes contextual matching scores between job postings and freelancer profiles using natural language embeddings and cosine similarity. It analyzes required skills, experience levels, budget constraints, and historical contract ratings to recommend optimal jobs to freelancers and qualified candidates to clients without manual keyword searching.
30	**AI Content Moderation**	[POST] /api/ai/moderation/check-text [POST] /api/ai/moderation/filter-spam [GET] /api/admin/moderation/ai-logs	The AI Content Moderation service processes user-generated text asynchronously across forum posts, thread comments, and direct chat messages. It analyzes submissions for profanity, toxic sentiment, hate speech, and spam. Content exceeding policy violation thresholds is automatically quarantined, flagged with high severity, and queued for administrator intervention while notifying the submitter.

#	Feature	System Function	Description
31	**AI Face Scan & Duplicate Identity Detection**	[POST] /api/identity-verifications/ai-check [POST] /api/identity-verifications/liveness-pose [POST] /api/ai/biometrics/duplicate-check	The AI Biometric Verification pipeline automates eKYC validation for freelancer identity onboarding. It extracts personal data from identity documents via optical character recognition (OCR), verifies head poses and face liveness via computer vision, and generates biometric facial feature vectors. The system cross-checks biometric signatures against existing verified profiles to prevent fraudulent account duplication.
32	**Automated Payment Processing & Webhook Synchroniza-tion**	[POST] /api/payments/webhook [POST] /api/payments/intents/{paymentIntentId}/sync [POST] /api/payments/checkout-sessions/{sessionId}/sync	The Automated Payment Processing service reliably handles asynchronous webhook callbacks emitted by Stripe. It processes payment intent confirmations, escrow funding authorizations, payout transfers, and dispute chargebacks. In addition, automated reconciliation routines synchronize transient payment intent statuses, resolve interrupted checkout sessions, and update contract ledger balances in real time.

3.1.5 Entity Relationship Diagram

3.1.5.1 Overall Entity Relationship Diagram

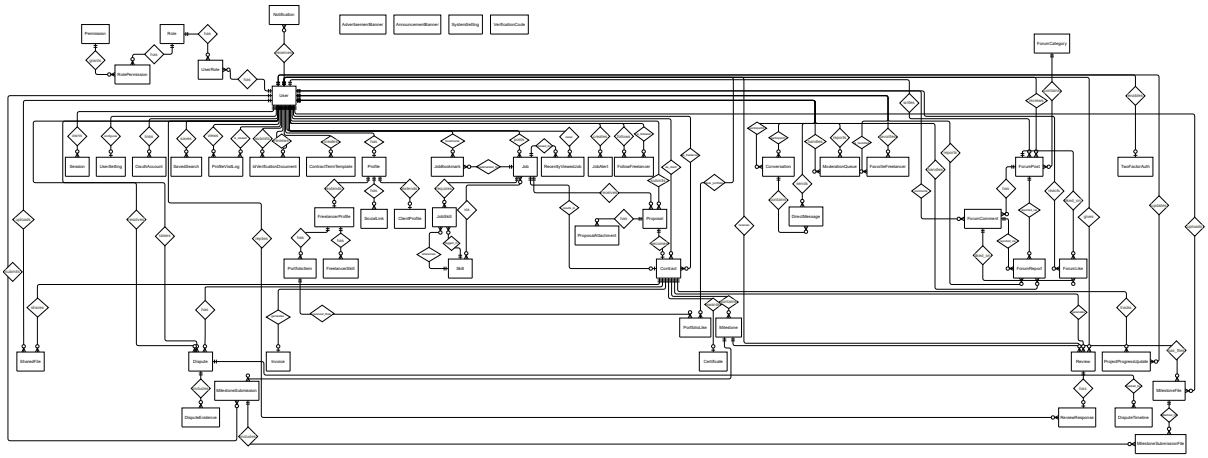

Figure 28: Overall Entity Relationship Diagram

3.1.5.2 Authentication & Sessions

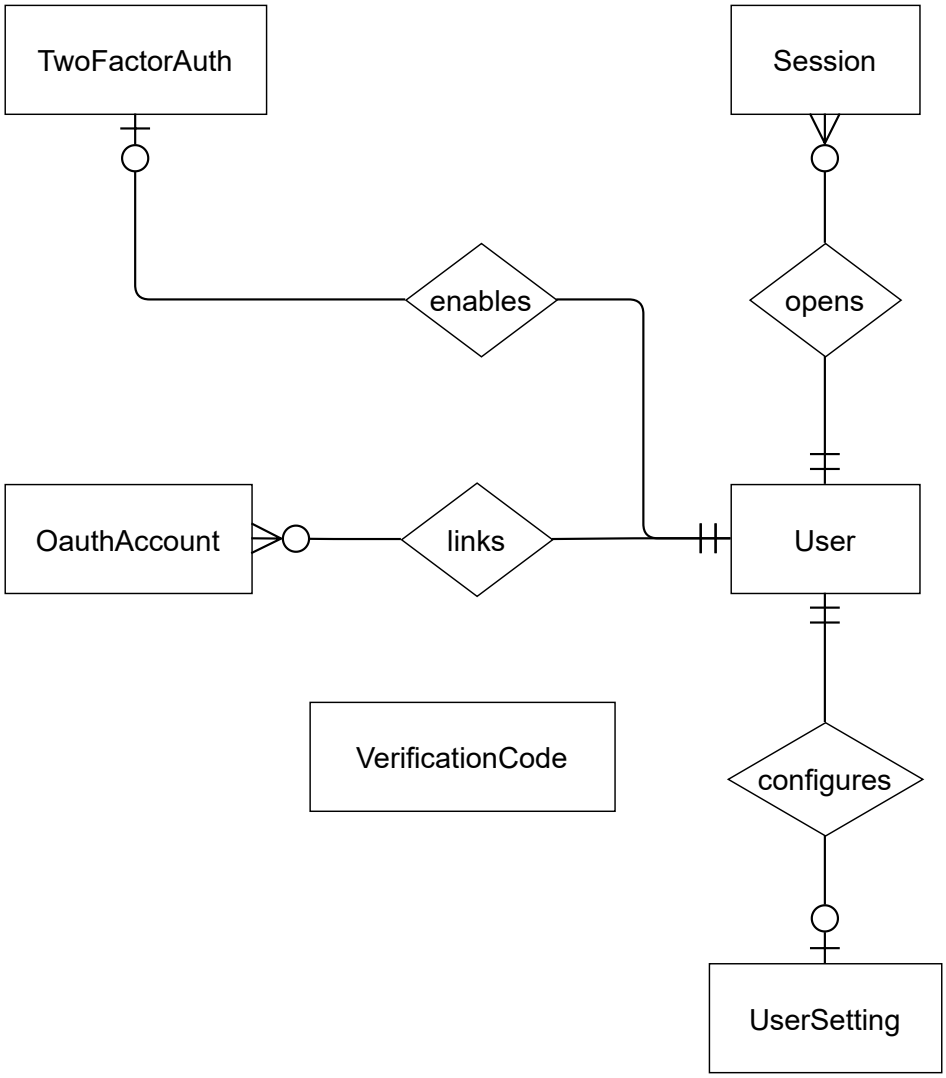

Figure 29: Authentication & Sessions ERD

3.1.5.3 Role-Based Access Control

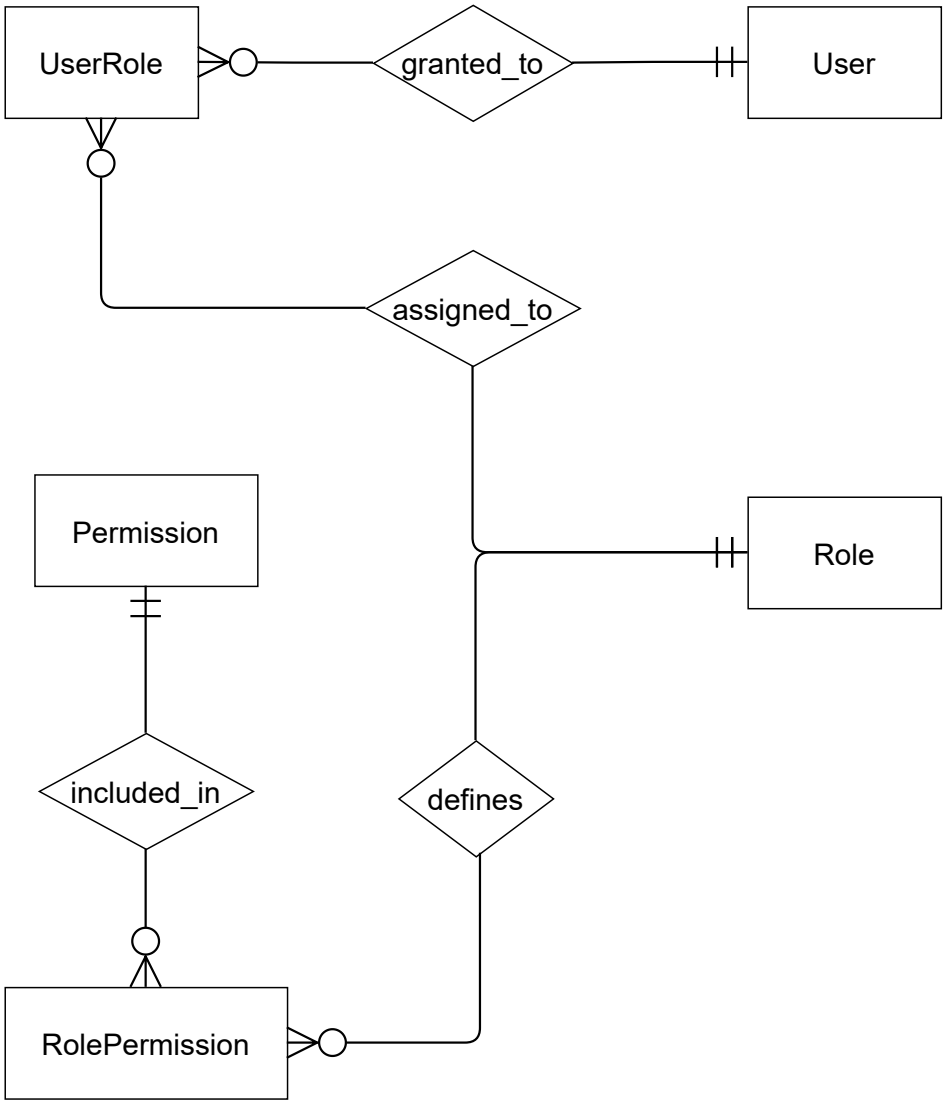

Figure 30: Role-Based Access Control ERD

3.1.5.4 Profile Core

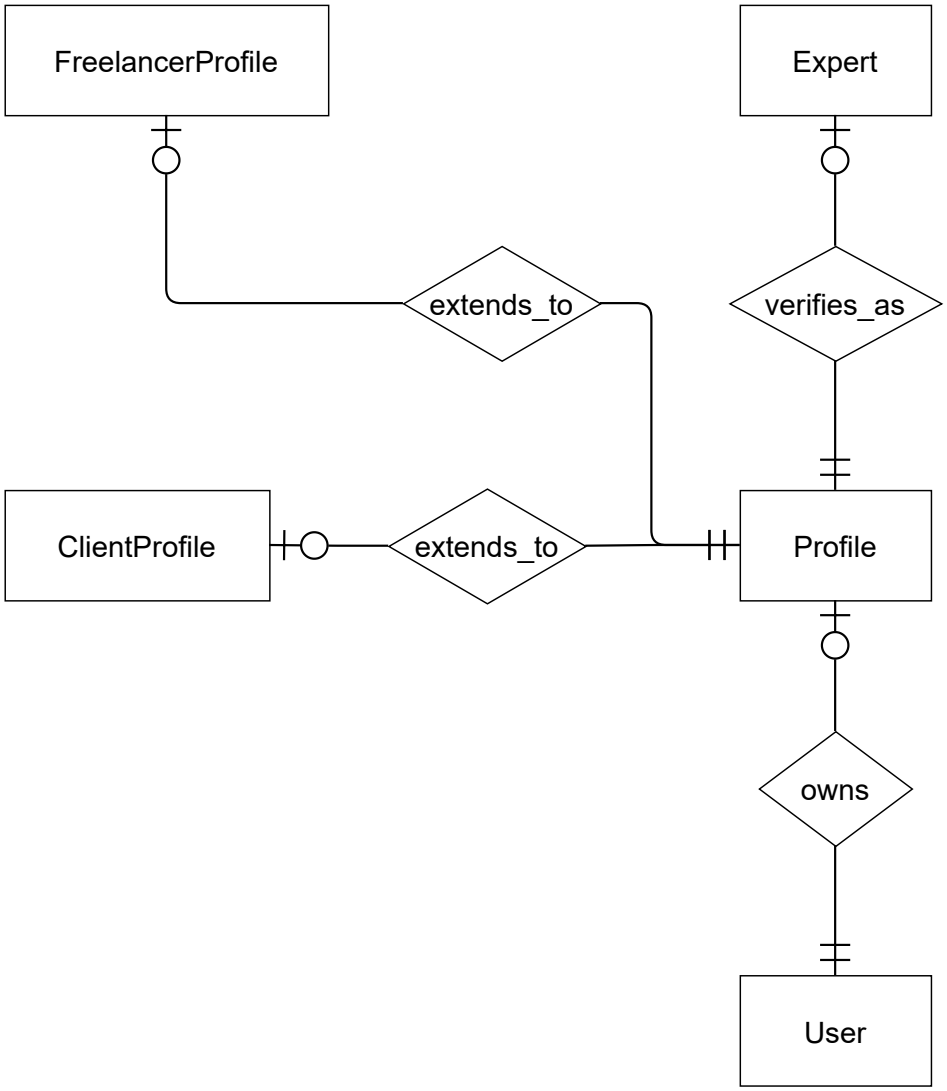

Figure 31: Profile Core ERD

3.1.5.5 Profile History & Social Links

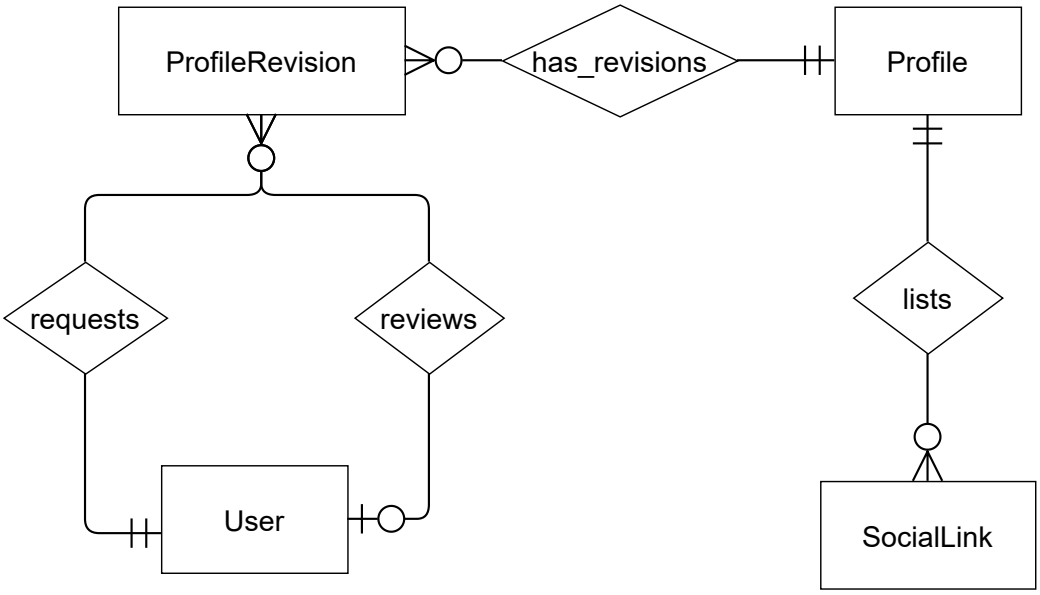

Figure 32: Profile History & Social Links ERD

3.1.5.6 Identity Verification & Profile Visits

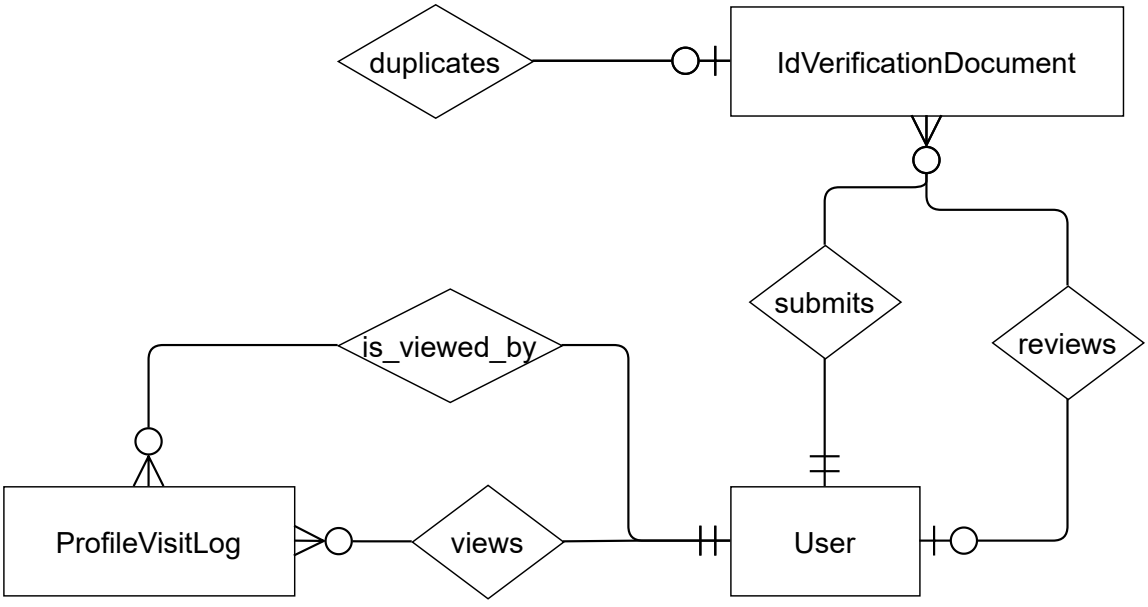

Figure 33: Identity Verification & Profile Visits ERD

3.1.5.7 Portfolio & Skills

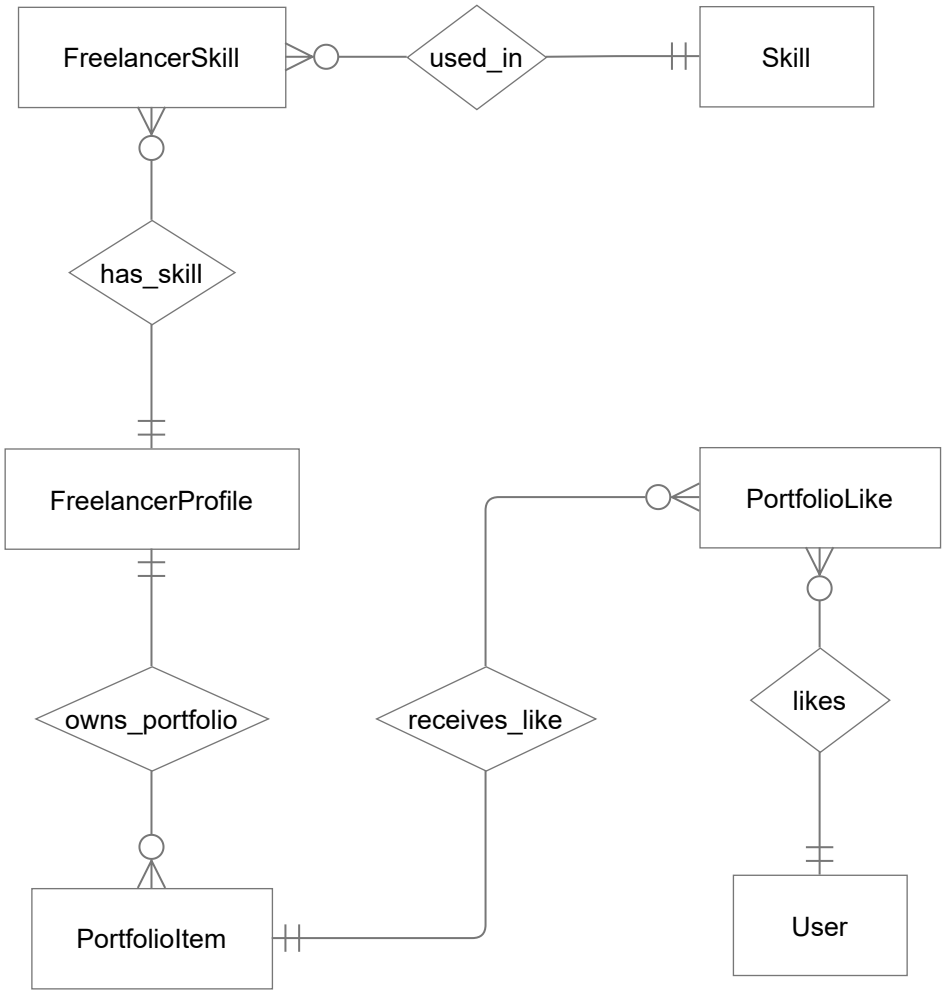

Figure 34: Portfolio & Skills ERD

3.1.5.8 Favorites & Follows

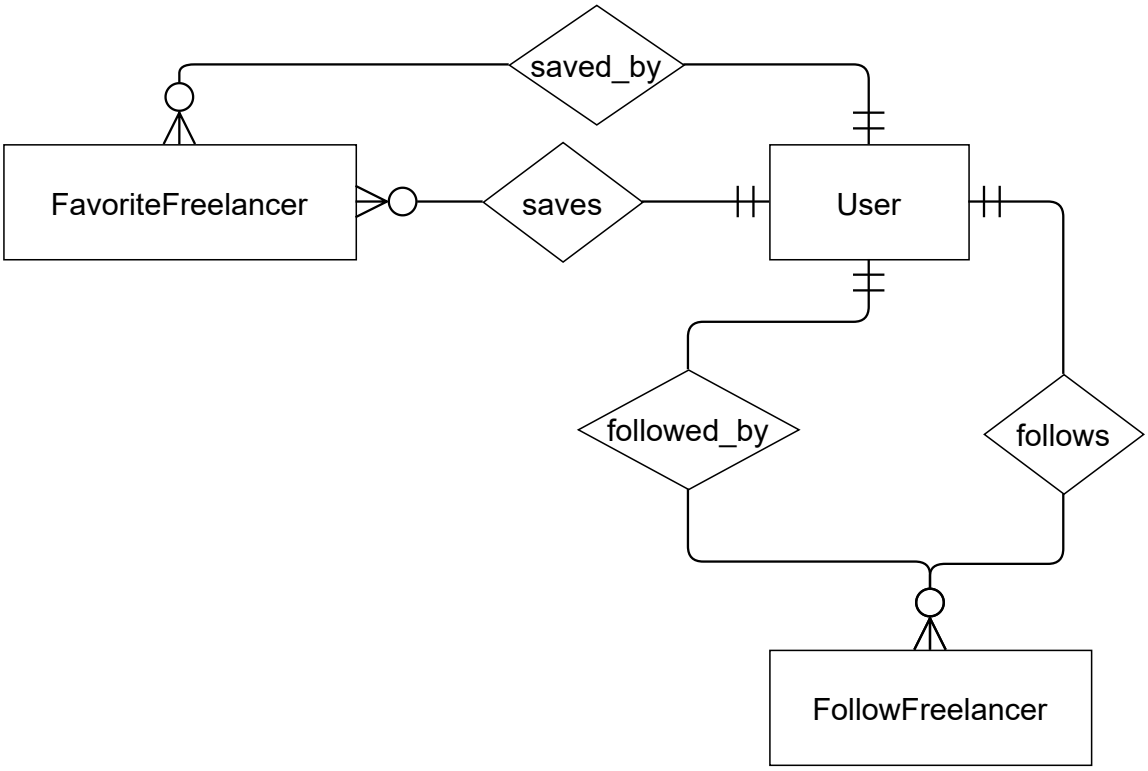

Figure 35: Favorites & Follows ERD

3.1.5.9 Job Posting

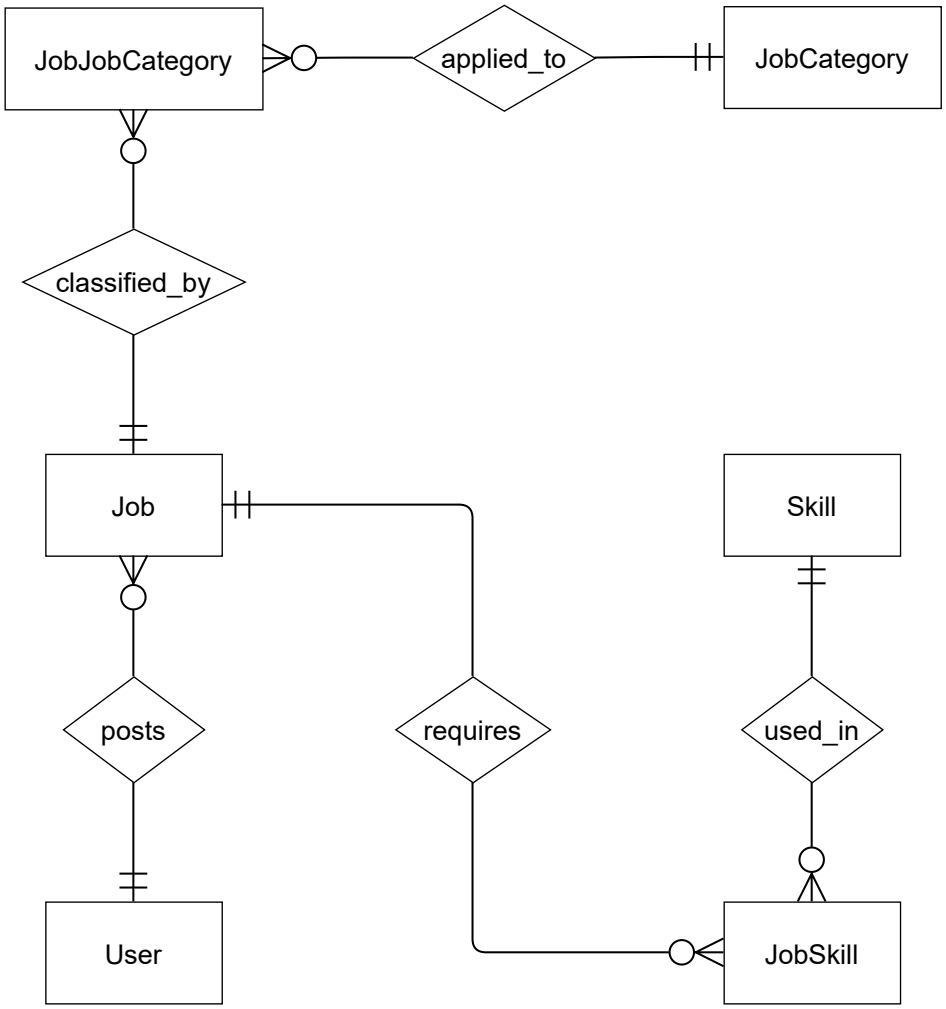

Figure 36: Job Posting ERD

3.1.5.10 Job Discovery & Alerts

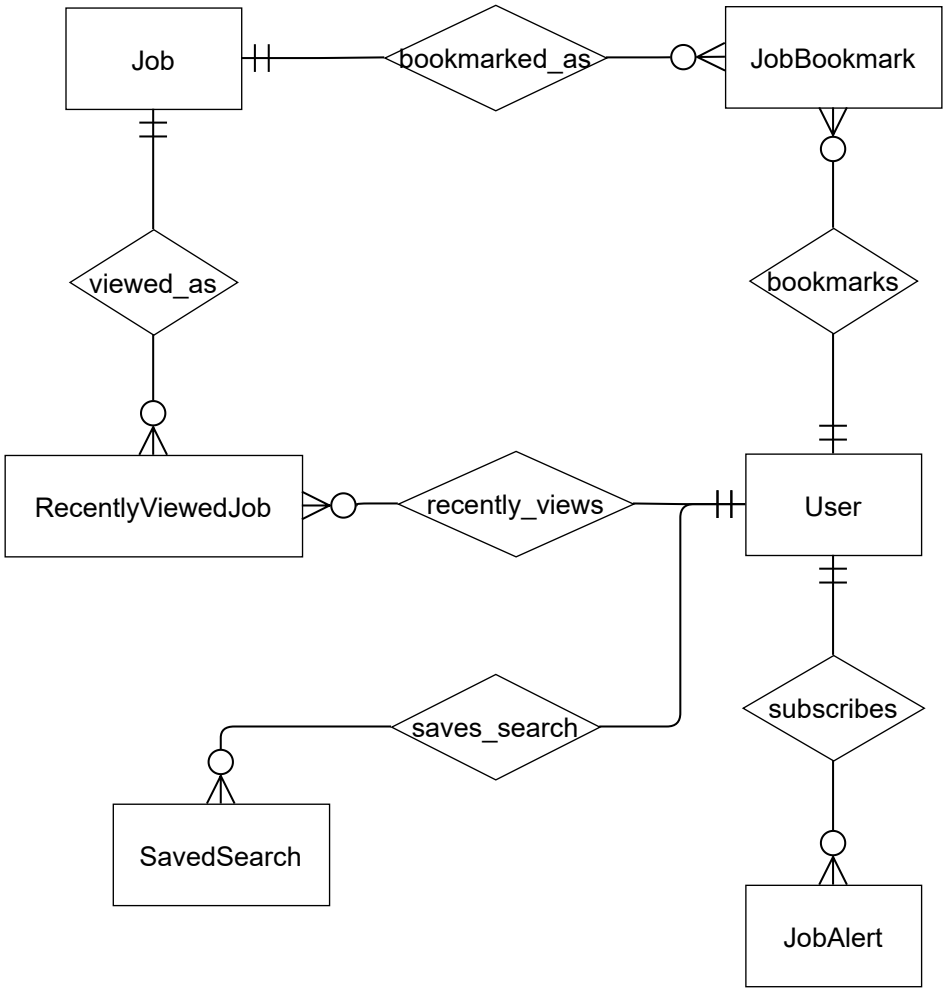

Figure 37: Job Discovery & Alerts ERD

3.1.5.11 Proposal

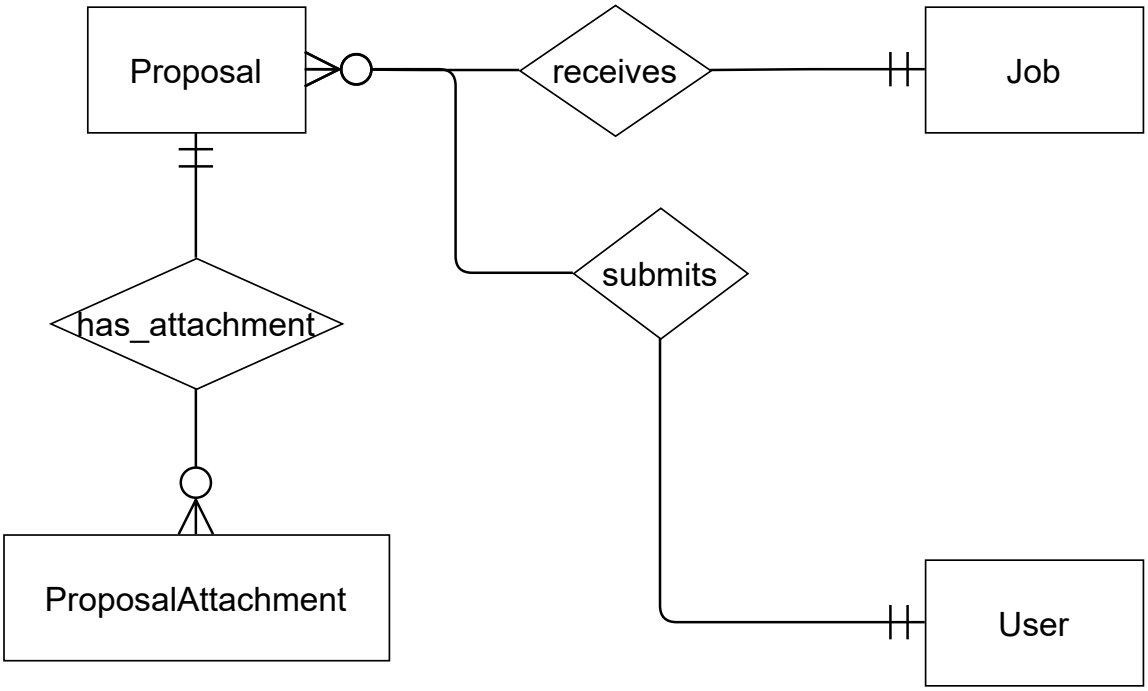

Figure 38: Proposal ERD

3.1.5.12 Contract

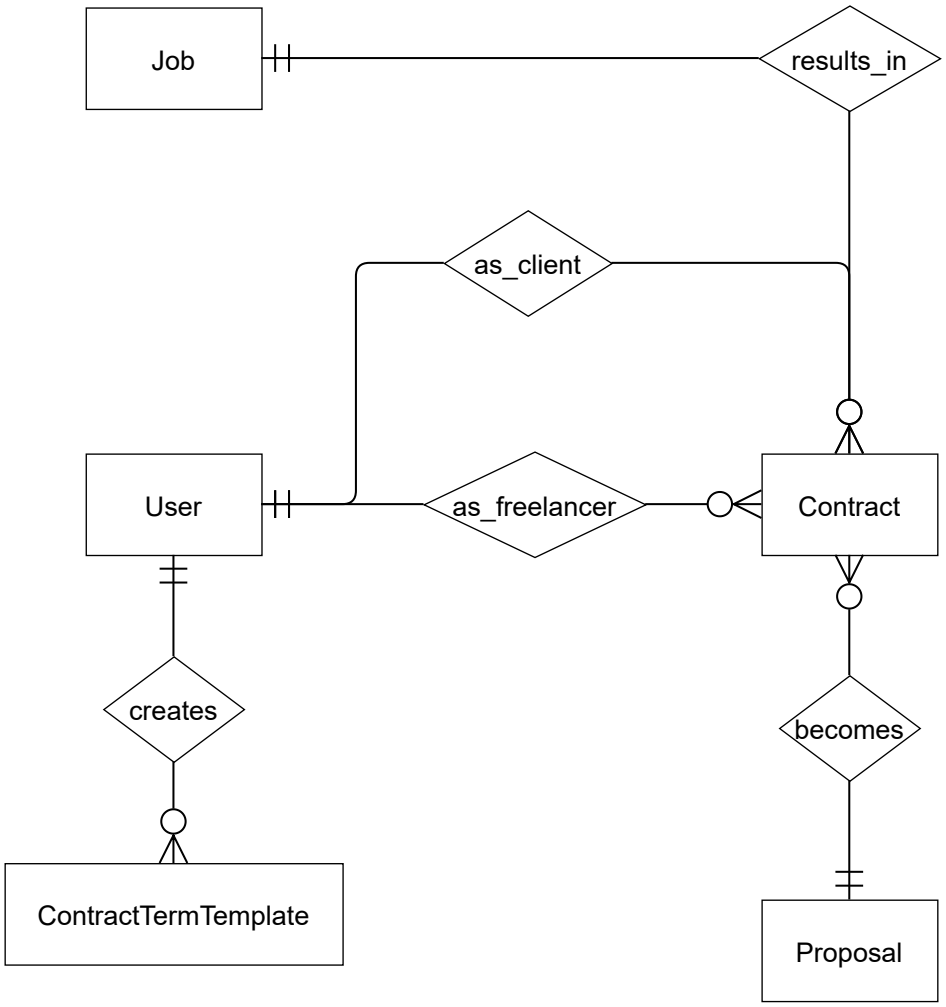

Figure 39: Contract ERD

3.1.5.13 Milestone & Project Progress

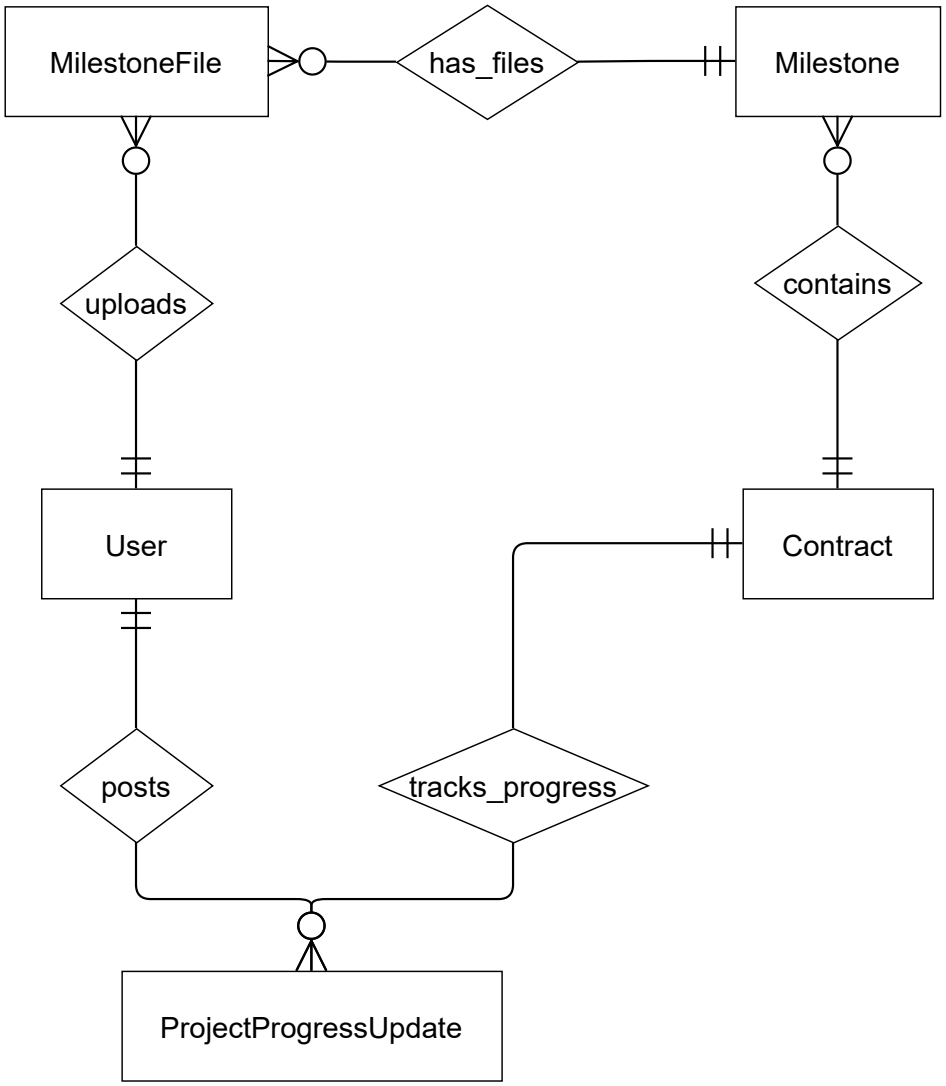

Figure 40: Milestone & Project Progress ERD

3.1.5.14 Milestone Submission

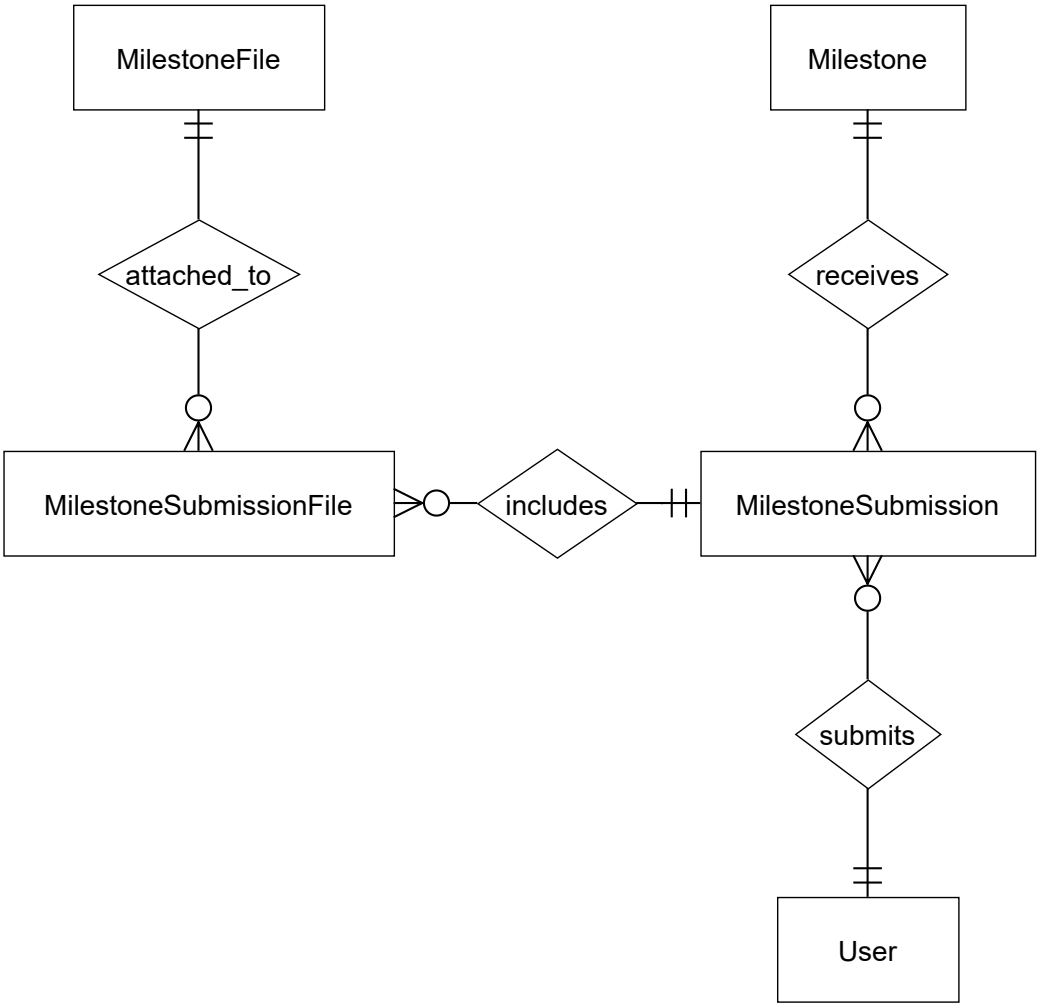

Figure 41: Milestone Submission ERD

3.1.5.15 Escrow, Payments & Certificates

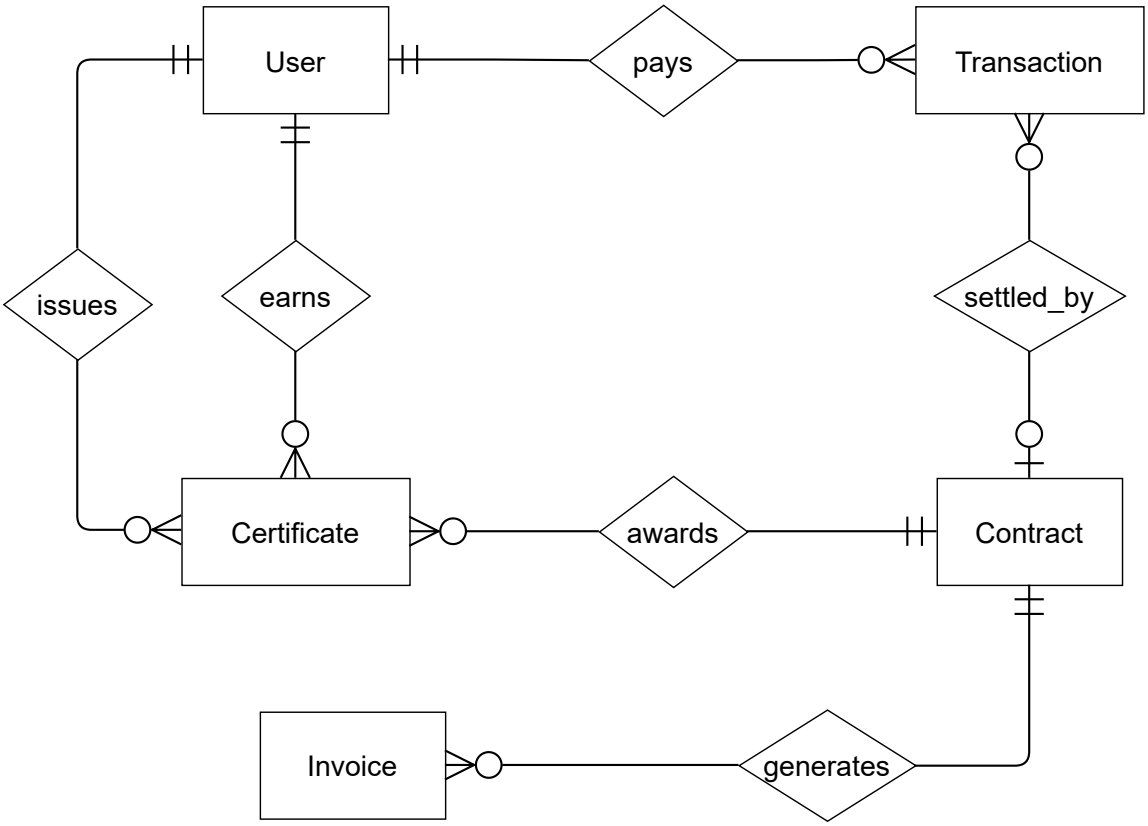

Figure 42: Escrow, Payments & Certificates ERD

3.1.5.16 Dispute Core

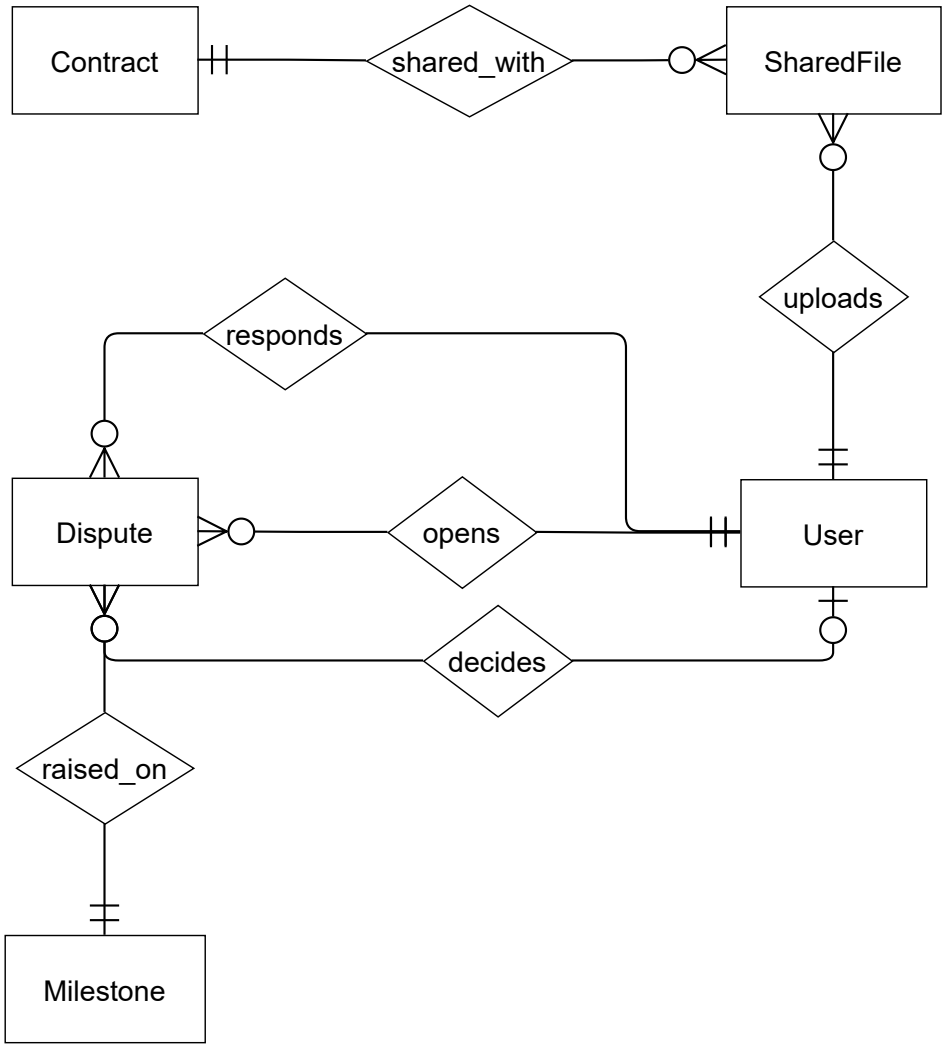

Figure 43: Dispute Core ERD

3.1.5.17 Dispute Resolution

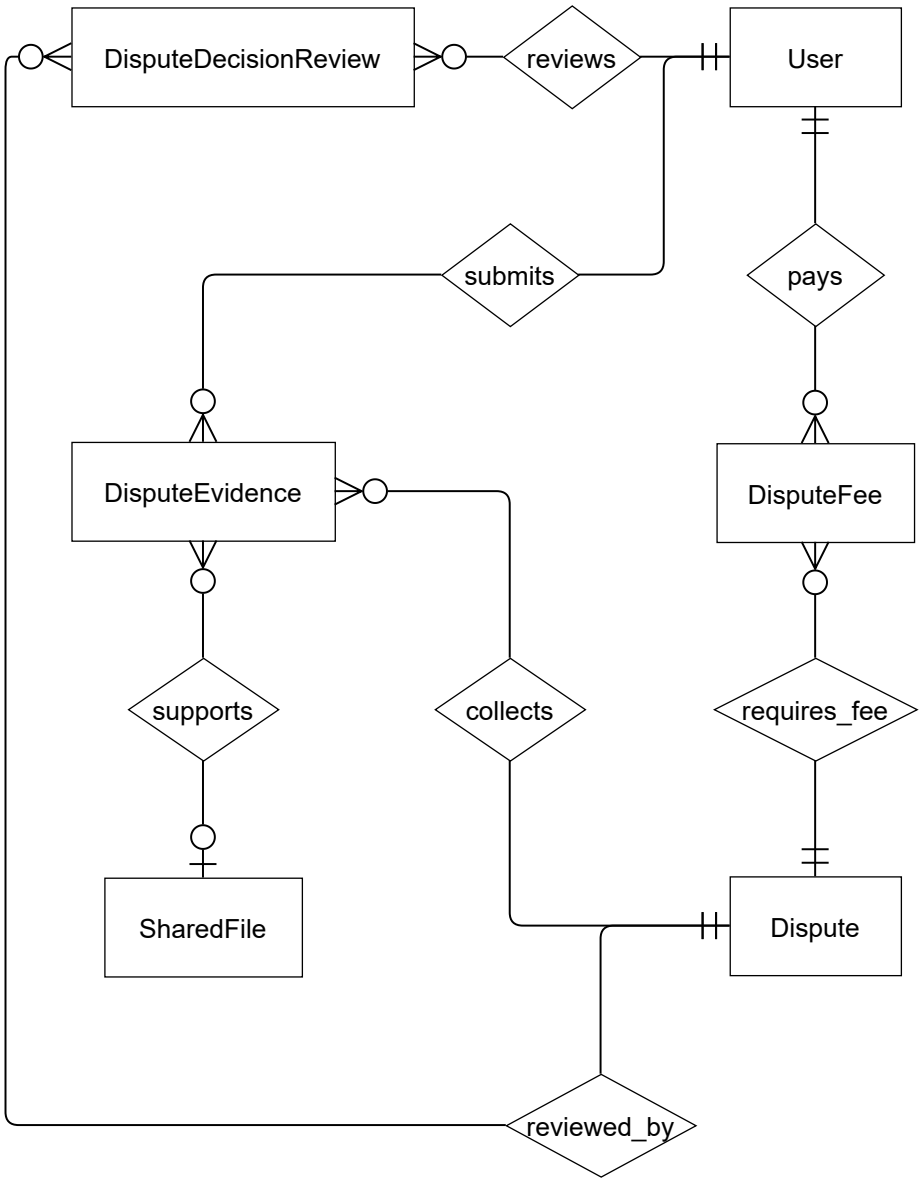

Figure 44: Dispute Resolution ERD

3.1.5.18 Reviews & Reputation

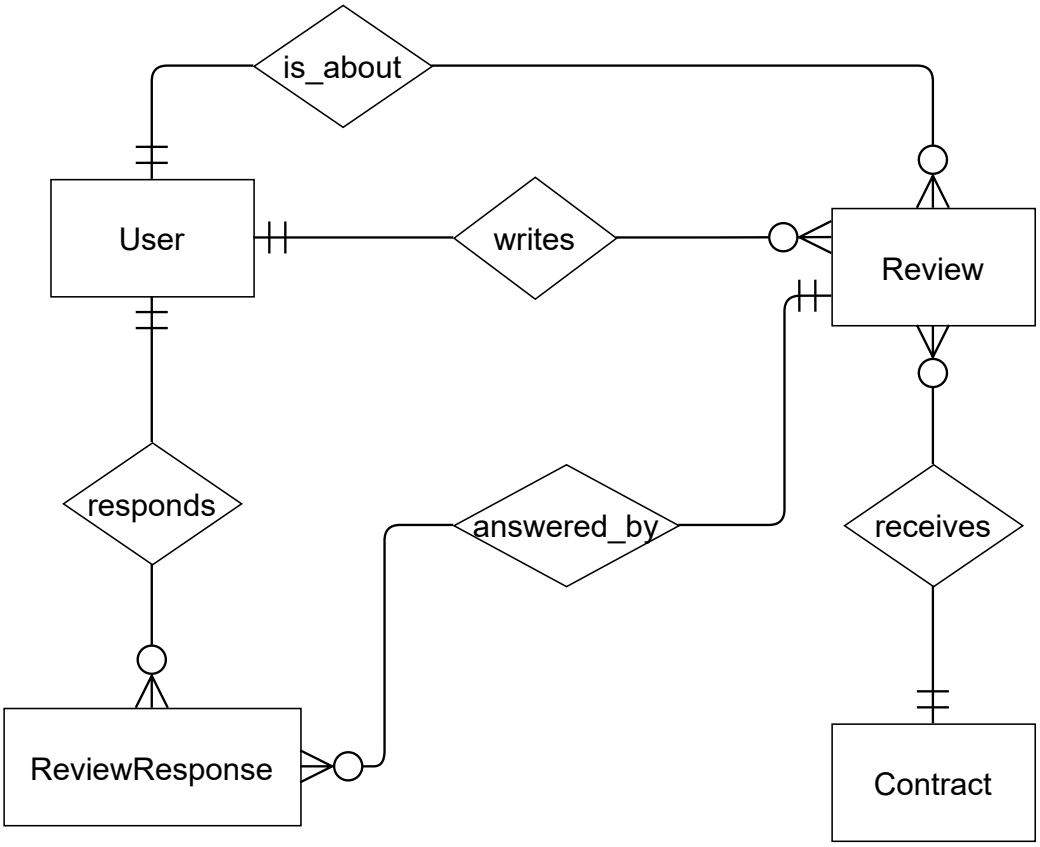

Figure 45: Reviews & Reputation ERD

3.1.5.19 Forum Posts & Comments

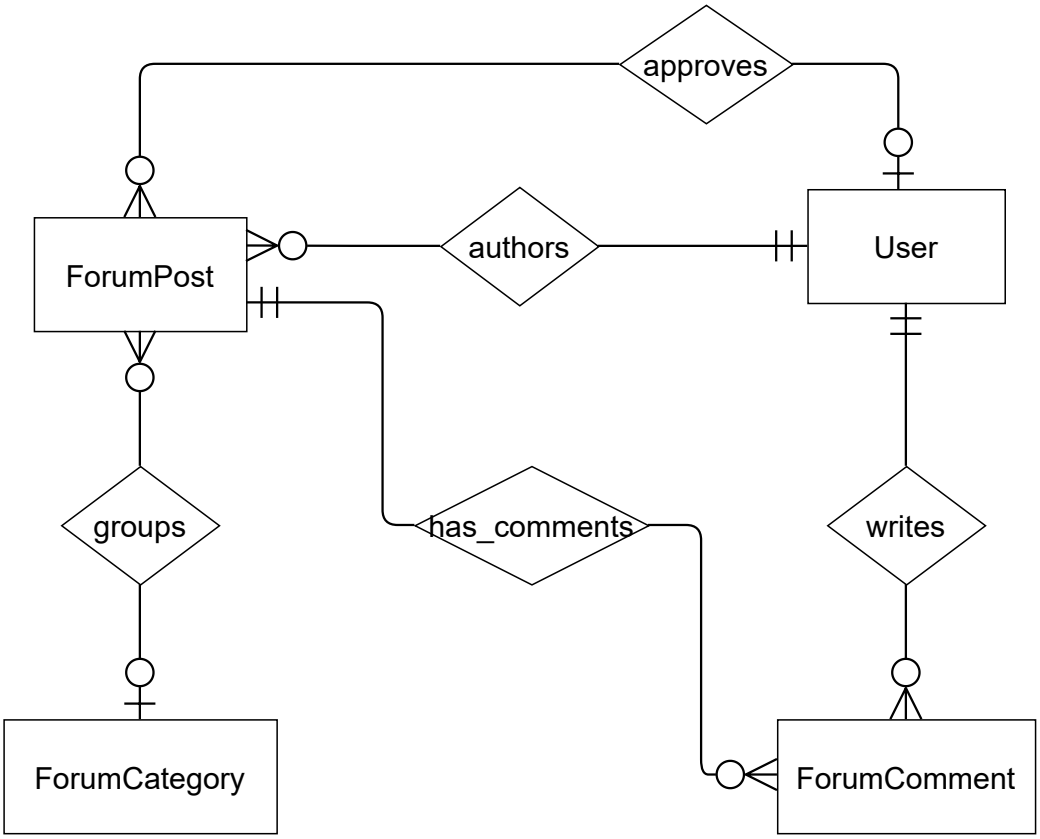

Figure 46: Forum Posts & Comments ERD

3.1.5.20 Forum Likes

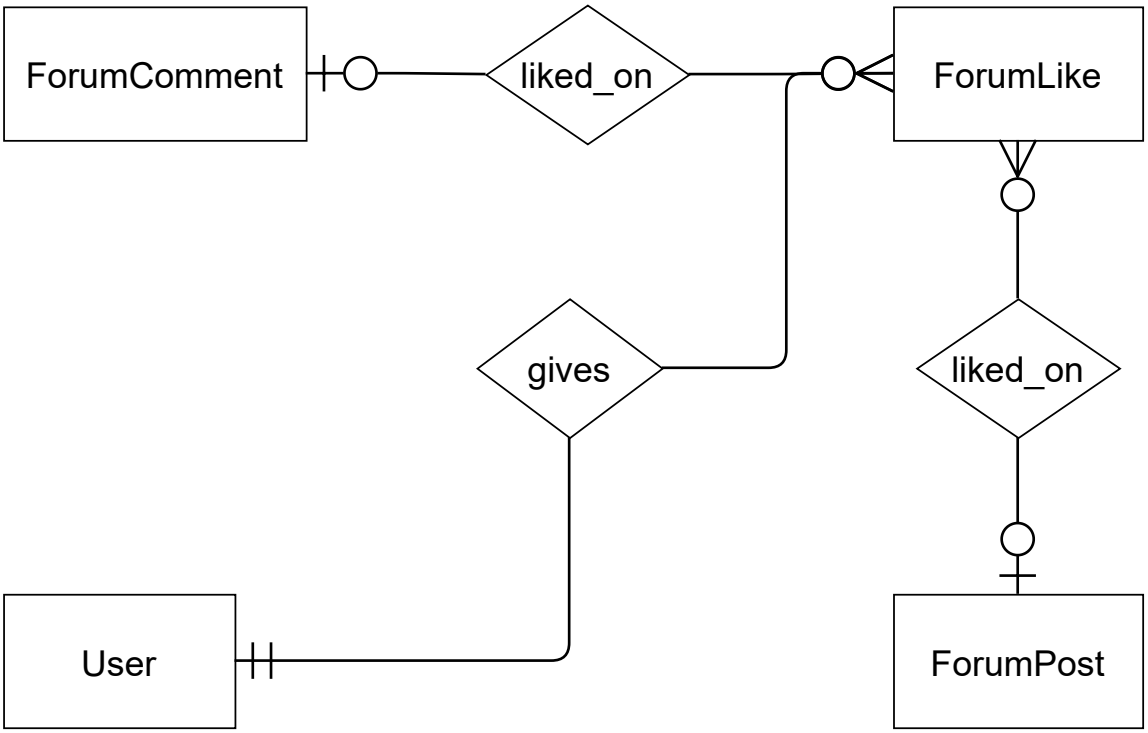

Figure 47: Forum Likes ERD

3.1.5.21 Forum Reports & Moderation

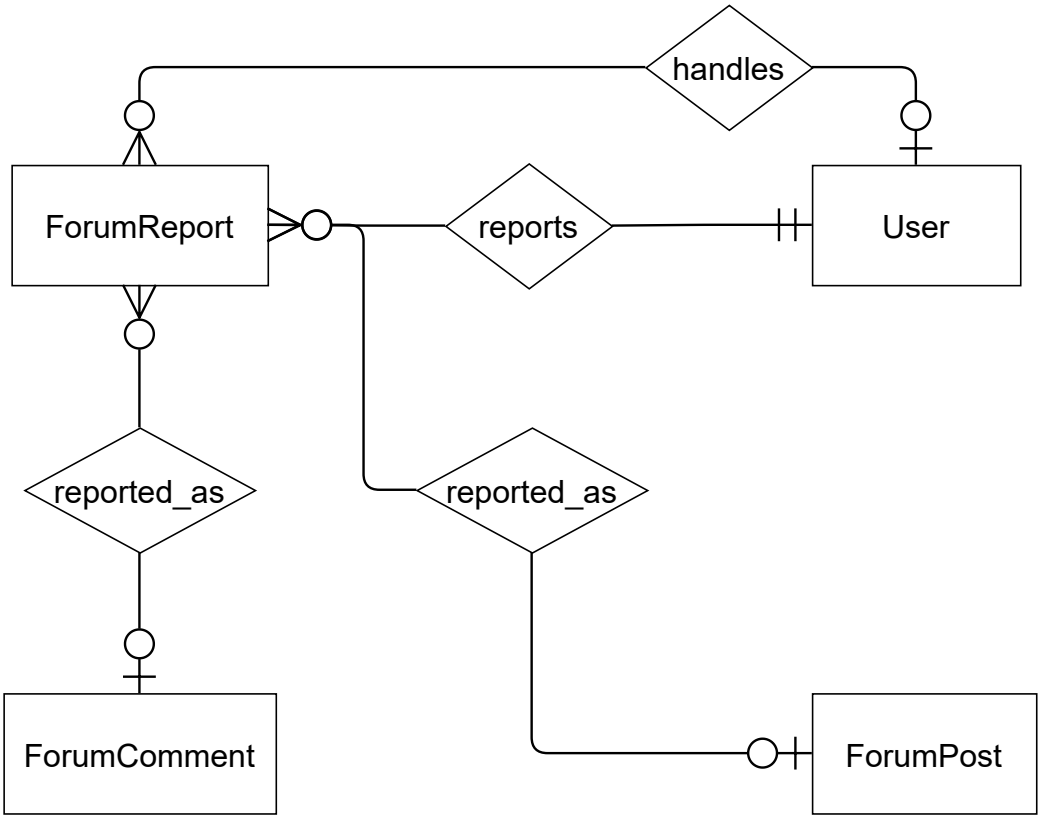

Figure 48: Forum Reports & Moderation ERD

3.1.5.22 Messaging & Notifications

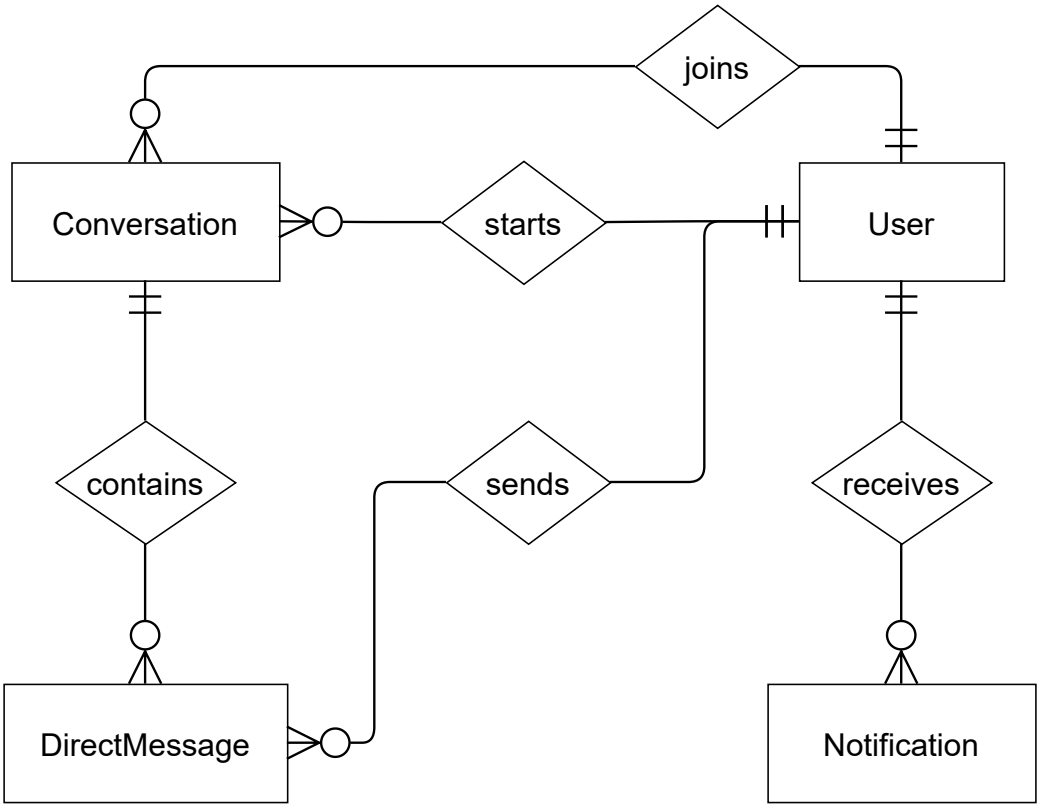

Figure 49: Messaging & Notifications ERD

3.1.5.23 Administration & System

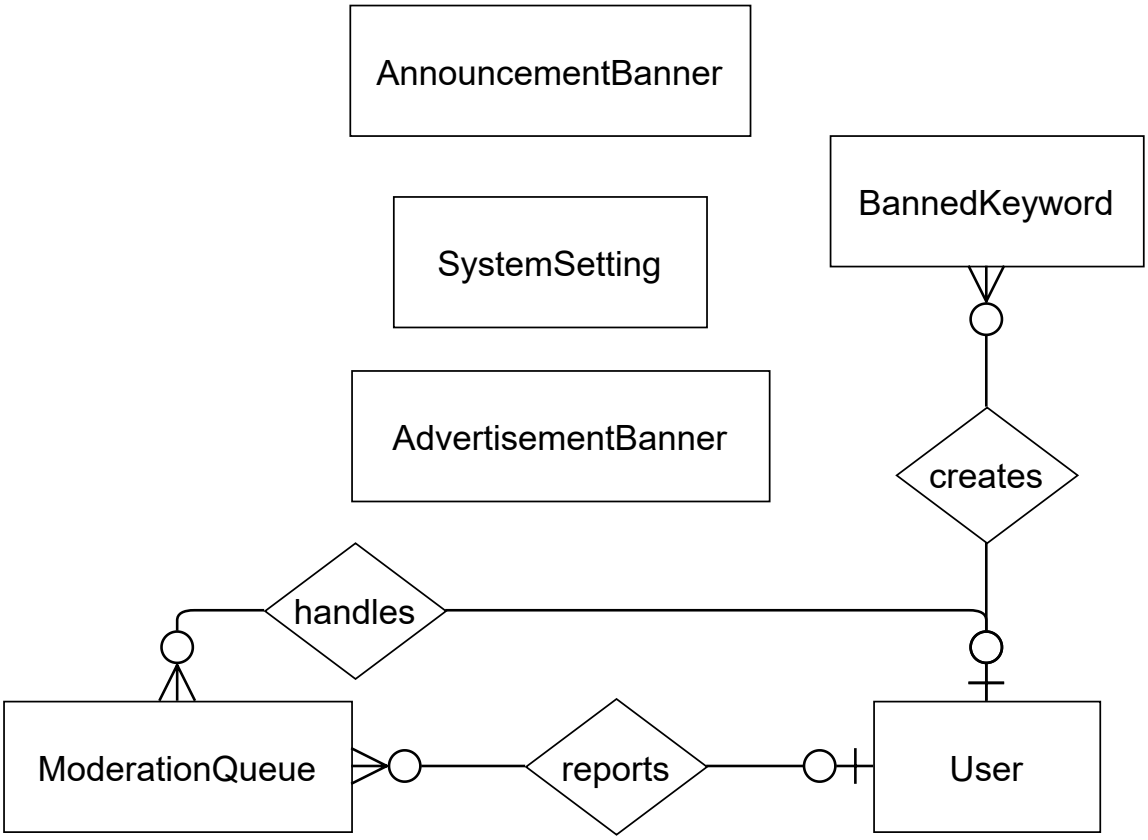

Figure 50: Administration & System ERD

3.2 User Authentication

3.2.1 Login with Google

Function Trigger

A guest clicks the "Google" authentication button on the sign-in screen or the registration screen.

Navigation path: Home → Login (`/login`) → Click "Google" button.

Function Description

Actors / Roles

- **Primary Actor:** Guest (unauthenticated visitor).
- **Secondary Actor:** Google OAuth 2.0 Authorization Server.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF authentication proxy (`/api/auth/google`), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Allows a visitor to authenticate or register an account instantly using their existing Google account via OAuth 2.0. This eliminates the need to remember credentials or manually verify an email address, significantly reducing onboarding friction while ensuring verified identity data.

Interface

- Sign-in screen at `/login` presenting the prominent "Google" single sign-on action.
- External Google OAuth 2.0 consent window prompted by Google Identity services.
- Automatic redirect to role-specific workspace upon authentication completion.

Data Processing Overview

- The client initiates authentication by requesting `GET /api/auth/google-link`.
- The backend generates the Google OAuth consent URL configured with client ID, callback URI, and identity scopes (`openid email profile`).
- The guest grants permission on Google's consent dialog, and Google redirects to the callback endpoint with an authorization code.
- The backend exchanges the code for Google tokens and fetches profile information (email, full name, avatar URL).
- The system queries the database: if no account exists with the email, a new account is automatically provisioned with verified email status and assigned the default `Client` role.
- The system creates an active session, issues signed JWT access and refresh tokens, stores them in secure `httpOnly` cookies, and redirects the user to their designated dashboard.

Screen Layout (Mock-up Description)

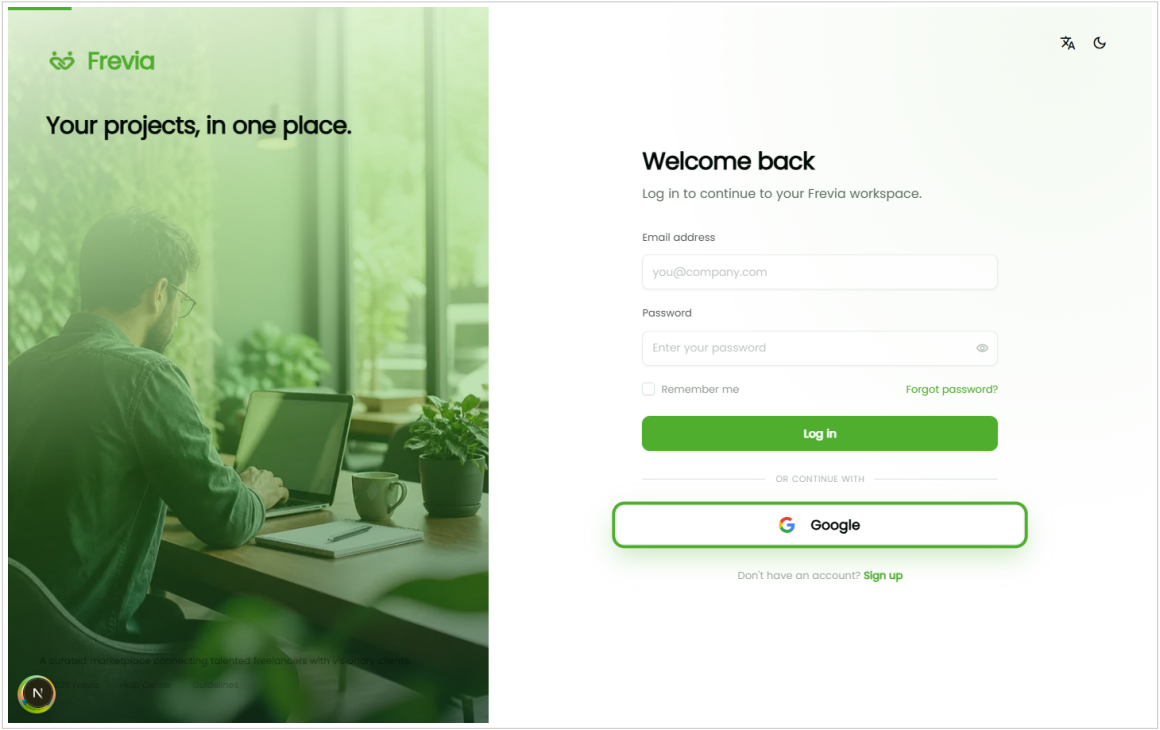

Figure 51: Login with Google interface

- The login container displays a greeting title “Welcome back” and an email/password form for standard authentication.
- A divider labeled “OR CONTINUE WITH” separates local credential inputs from social login providers.
- The “Google” button is prominent, featuring the official multi-colored Google logo, clean responsive styling, and clear action labeling.
- A footer link allows users without an account to switch to the registration screen.

Function Details

Input Data

Data Name	Description
OAuth Code	Temporary authorization code returned by Google OAuth Server via redirect URI.
State Token	Cryptographic anti-CSRF state token validated by the authentication handler.
Session Headers	IP address and User-Agent captured from request headers for session tracking.

Validation Rules

- The OAuth code must be valid, non-expired, and verifiable against Google OAuth endpoints.
- The email address retrieved from Google profile must be non-empty and marked as verified by Google.
- The matched user account must have `status = ACTIVE`; suspended or banned accounts are rejected.

Business Rules

- **BR-UA-01**: Google OAuth shall follow standard OAuth 2.0 authorization code grant with secure PKCE/state verification.
- **BR-UA-02**: New users authenticated via Google OAuth shall be automatically registered with the `Client` role and `isEmailVerified = true`.
- **BR-UA-03**: Suspended or banned accounts shall be denied access and notified with an administrative contact notice.
- **BR-UA-04**: Issued JWT tokens shall be stored exclusively in secure, `httpOnly`, `SameSite=Lax` cookies.

Normal Flow

1. The guest clicks the "Google" button on the login screen.
2. The browser receives the Google authorization URL from `/api/auth/google-link` and redirects to Google.
3. The guest authenticates with their Google account and authorizes Frevia.
4. Google redirects to `/api/auth/google/callback` with code and state parameters.
5. The backend exchanges the code for profile tokens and extracts identity details.
6. The backend finds or provisions the user record and initializes an active session.
7. The system writes authentication cookies and redirects the user to their role dashboard.

Abnormal Flow / Exception Handling

Case	Description
A1	Guest denies permission or cancels the Google OAuth consent prompt. Google redirects with `error=access_denied`; the system redirects back to `/login` with notification "Google login was cancelled."
A2	The authorization code has expired, was malformed, or was replayed. The token exchange fails; system displays "Authentication with Google failed. Please try again."
A3	Anti-CSRF `state` token mismatch or missing session state. The backend rejects the callback with HTTP 400 Bad Request to protect against cross-site request forgery.
A4	The account linked to the retrieved Google email has been suspended or banned by administrators. Access is rejected with HTTP 403: "Your account has been suspended. Please contact support."
A5	Google OAuth service is temporarily unreachable due to network outage or timeout. The backend catches `FetchError` and prompts the user to try standard email login.
A6	Database connection failure occurs during user auto-provisioning or session persistence. The transaction rolls back and returns HTTP 500: "Login failed due to a server error."
A7	The browser blocks the external OAuth redirect due to strict popup/redirect blocker policies. The client displays an explicit fallback hyperlink to proceed with authentication.

Output Data

Data Name	Description
`accessToken`	Signed JWT containing user ID, email, active roles, and expiration timestamp (15 minutes).
`refreshToken`	Opaque or long-lived JWT token (7 days) persisted in the database session table.
User Object	User profile record including `id`, `email`, `name`, `avatarUrl`, and primary role.

Post-Conditions

- The user is authenticated with active session cookies.
- An audit record is created in the database session log.
- The browser is navigated to the dashboard appropriate for the user's primary role.

3.2.2 Login

Function Trigger

A registered user enters their email and password into the sign-in form and submits.

Navigation path: Home → Login (`/login`).

Function Description

Actors / Roles

- **Primary Actor:** Guest (unauthenticated registered user).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/auth/login`), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Authenticates users using traditional email and password credentials, verifying identity against hashed passwords in the database, establishing an active session, and issuing JWT tokens.

Interface

- Login screen at `/login` with Email Address input and Password input.
- Toggle visibility button (eye icon) to show or hide entered password.
- "Remember me" checkbox and "Forgot password?" navigation link.
- Primary "Log in" button with loading spinner state during submission.

Data Processing Overview

- The web client validates input schema with Zod (`LoginBodySchema`) before sending.
- The client posts credentials to BFF `/api/auth/login`, which forwards to NestJS API.
- The API queries the user by email including associated roles and profiles.
- The system verifies the supplied password against the stored bcrypt hash.
- The system verifies that the account is active and the email address is confirmed.
- Upon success, a new session record is written into the database, JWT access/refresh tokens are generated and attached as `httpOnly` cookies, and user metadata is returned.

Screen Layout (Mock-up Description)

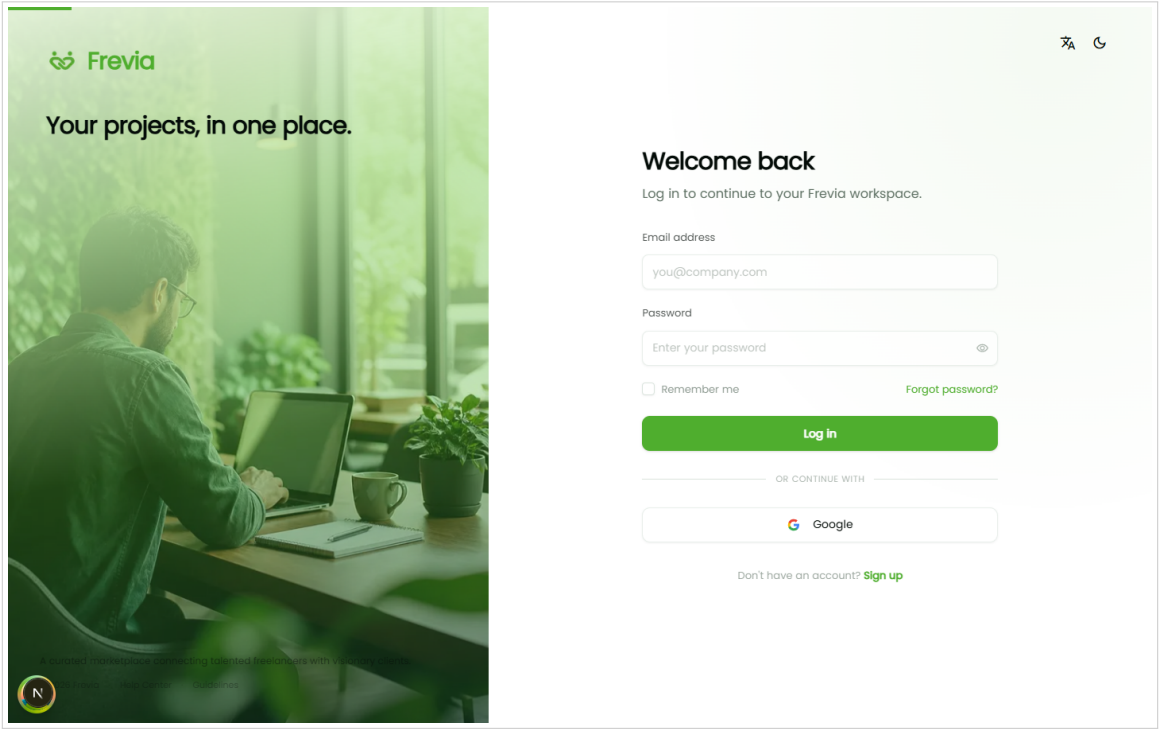

Figure 52: User login screen

- Left branded imagery panel showcases workspace lifestyle graphics with the Frevia slogan.
- Right form card provides fields for “Email address” and “Password” with subtle placeholder hints.
- Password input includes an embedded visibility toggle icon on the right edge.
- Action row features “Remember me” checkbox alongside “Forgot password?” link in primary green.
- Solid green “Log in” submit button spans full container width.

Function Details

Input Data

Data Name	Description
email	Registered email address string; must be a valid RFC 5322 format.
password	User account password string; required, minimum 6 characters.

Validation Rules

- email is trimmed and validated as a syntactically valid email address.
- password cannot be empty and must match the stored bcrypt hash.
- The account must not have deletedAt set and must have status = ACTIVE.
- The user must have verified their email address (isEmailVerified = true).

Business Rules

- **BR-UA-05**: Passwords must be verified using bcrypt comparison with salted hashes.
- **BR-UA-06**: Error responses shall use a generic message "Invalid email or password" to mitigate account enumeration.
- **BR-UA-07**: Unverified accounts shall be rejected with a notice prompting verification completion.
- **BR-UA-08**: Suspended accounts shall be denied access across all client interfaces.

Normal Flow

1. Guest navigates to /login and inputs valid registered email and password.
2. Guest clicks "Log in" button.
3. Form resolver executes schema validation and initiates HTTP POST request.
4. NestJS API retrieves user record, compares bcrypt hash, and verifies account active status.
5. The system records the session in the database and creates JWT tokens.
6. The BFF proxy serializes httpOnly cookies and responds with HTTP 200.
7. The frontend detects user roles and redirects to the appropriate dashboard (Admin, Client, or Freelancer).

Abnormal Flow / Exception Handling

Case	Description
A1	Required field is missing (empty email or password). Form resolver intercepts input and renders inline errors before sending any network request.
A2	Email is not registered in the system. The API returns HTTP 401 with generic error `AuthMessage.INCORRECT_EMAIL` mapped to "Invalid email or password."
A3	Incorrect password provided. Bcrypt comparison fails; API returns HTTP 422 with `AuthMessage.INCORRECT_PASSWORD` mapped to "Invalid email or password."
A4	Account email is unconfirmed (`isEmailVerified = false`). System returns HTTP 403: "Please verify your email before logging in. Resend verification email?"
A5	Account is marked as `status = BANNED` or `SUSPENDED`. System throws `UserBannedException` (HTTP 403): "Your account has been suspended. Please contact support."
A6	Excessive consecutive failed login attempts detected. Rate limiter triggers HTTP 429: "Too many login attempts. Please wait 15 minutes before trying again."
A7	Database connection times out or pool is exhausted. API returns HTTP 500; frontend alerts "Login failed. Please try again later."
A8	Network connectivity drops during form submission. Client request fails with network error; frontend displays offline alert toast.

Output Data

Data Name	Description
`accessToken`	15-minute validity JWT stored in `accessToken` cookie.
`refreshToken`	7-day validity token stored in `refreshToken` cookie.
`user`	Sanitized user object containing ID, email, name, avatar, and roles list.

Post-Conditions

- User authentication state is initialized.
- A new session row is created in the database containing client device metadata.
- Client redirects to the dashboard.

3.2.3 Logout

Function Trigger

An authenticated user clicks the "Log out" button from the main navigation sidebar or user profile menu.
Navigation path: Any authenticated screen → Sidebar bottom → Click "Log out".

Function Description

Actors / Roles

- **Primary Actor:** Client, Expert, Freelancer, Administrator (all authenticated users).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/auth/logout`), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Terminates the current active session on both the client application and server database, revoking the validity of issued tokens and protecting user accounts from unauthorized session reuse.

Interface

- Sidebar bottom control labeled "Log out" with logout icon.
- Header profile popover menu containing secondary sign-out trigger.
- Immediate redirection back to the public sign-in screen upon termination.

Data Processing Overview

- Client dispatches a POST request to `/api/auth/logout`.
- The backend extracts the session token from the request cookies or authorization headers.
- The API locates the matching session row in the `Session` table and soft-deletes or purges it.
- The BFF proxy expires the `accessToken` and `refreshToken` cookies by setting their `Max-Age` to 0.
- The web client clears local client-side caches (TanStack Query, session store) and redirects the visitor to `/login`.

Screen Layout (Mock-up Description)

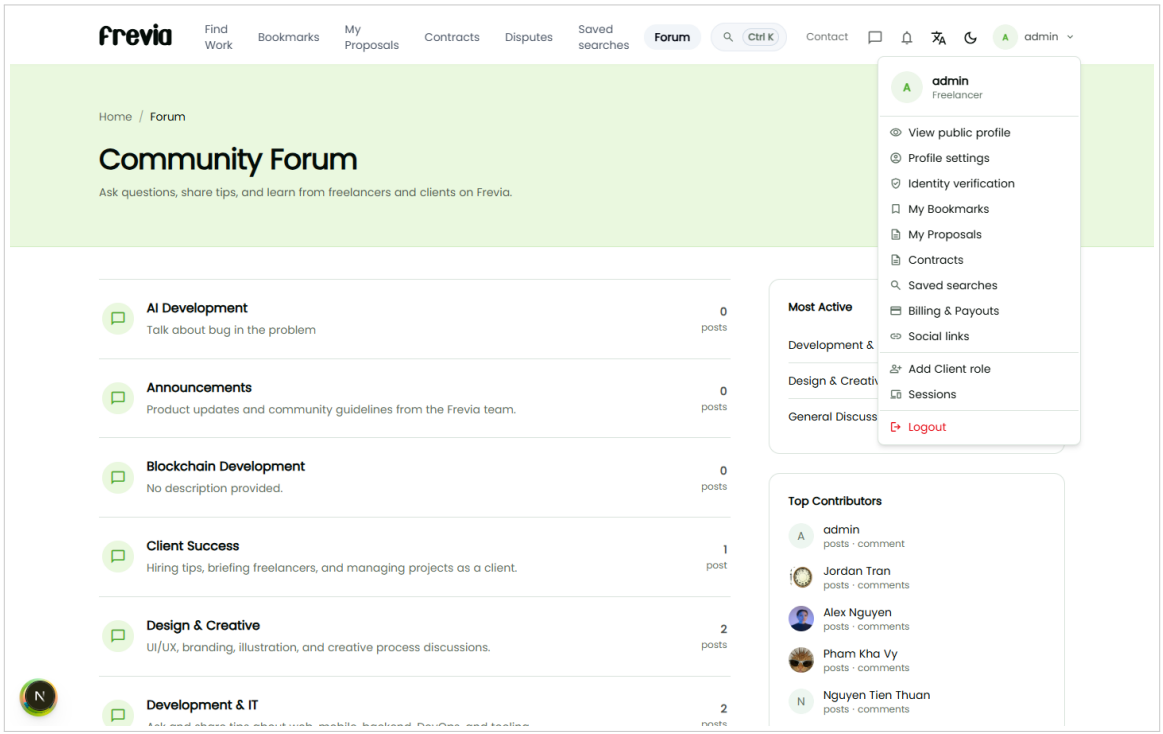

Figure 53: User logout interface

- Navigation sidebar includes workspace navigation links and administrative sections.
- The bottom section of the sidebar features the “Log out” command with a distinct exit glyph.
- Visual feedback indicates hover state, alerting the user to session termination.

Function Details

Input Data

Data Name	Description
Session Cookies	Active `accessToken` and `refreshToken` supplied via HTTP request.

Validation Rules

- The request must supply a valid authentication cookie or bearer token.
- Session must correspond to an active record in the database.

Business Rules

- **BR-UA-09**: Logout must invalidate the database session record before clearing browser cookies.

- **BR-UA-10**: Terminated sessions shall be rejected on all subsequent API requests.

Normal Flow

1. User clicks “Log out” in the application sidebar.
2. Frontend invokes `POST /api/auth/logout`.
3. Server deletes active session record from the database.
4. Response sets expired cookie headers (`Max-Age=0`).
5. Client resets in-memory cache and state.
6. Browser transitions to the public login page.

Abnormal Flow / Exception Handling

Case	Description
A1	The session token in the cookie has already expired prior to clicking logout. The backend returns HTTP 200/204; the client cleans up remaining cookies and finishes redirect.
A2	The session record was deleted remotely (e.g. via Revoke Session UC-16.03 on another device). System acknowledges logout request without error and purges local credentials.
A3	Database failure occurs during session row removal. The BFF proxy forcefully clears client `httpOnly` cookies regardless, guaranteeing the user is locally signed out.
A4	The user clicks the “Log out” button rapidly multiple times. The first call succeeds; subsequent calls encounter no active session and redirect gracefully.
A5	Client network drops during the logout dispatch. The browser client detects the failure, purges in-memory auth tokens, and forces navigation to `/login`.
A6	The token provided carries an invalid or tampered cryptographic signature. Backend rejects the token with 401, clears invalid cookies, and routes to login.

Output Data

Data Name	Description
Cookie Headers	Set-Cookie headers invalidating `accessToken` and `refreshToken`.
Response Body	JSON acknowledgment: `{ success: true, message: "Logged out successfully" }`.

Post-Conditions

- Database session is terminated.
- Authentication cookies are eliminated from the browser.
- Protected routes are no longer accessible without re-authenticating.

3.2.4 Register

Function Trigger

A new visitor clicks the "Sign up" link and completes the registration form.

Navigation path: Home → Login (`/login`) → "Sign up" link (`/register`).

Function Description

Actors / Roles

- **Primary Actor:** Guest.
- **Secondary Actor:** Mail Delivery Service (Nodemailer / SMTP Server).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy, Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Allows a new user to create a Frevia account with credentials, select an initial platform role (Client or Freelancer), and verify their email via a single-use OTP code.

Interface

- Registration page at `/register` with Full Name, Email, Password, and Password Confirmation fields.
- Role selection cards allowing user to choose between "Client" (hire talent) or "Freelancer" (find work).
- Inline OTP verification section with "Send OTP" button and 6-digit verification code input.
- Checkbox for accepting Terms of Service and Privacy Policy.

- Primary submit button labeled “Continue to Profile”.

Data Processing Overview

- User inputs email and clicks “Send OTP”. Backend validates email uniqueness and generates a 6-digit OTP code stored with a 10-minute expiration timestamp.
- The mailer sends the OTP to the specified email address.
- User fills credentials, chooses role, inputs OTP, checks terms, and submits `POST /api/auth/register`.
- Backend verifies OTP validity, hashes the password using bcrypt with 10 salt rounds, and writes user account to the database.
- A role profile (Client or Freelancer) is initialized, tokens are issued, and the user is redirected into onboarding.

Screen Layout (Mock-up Description)

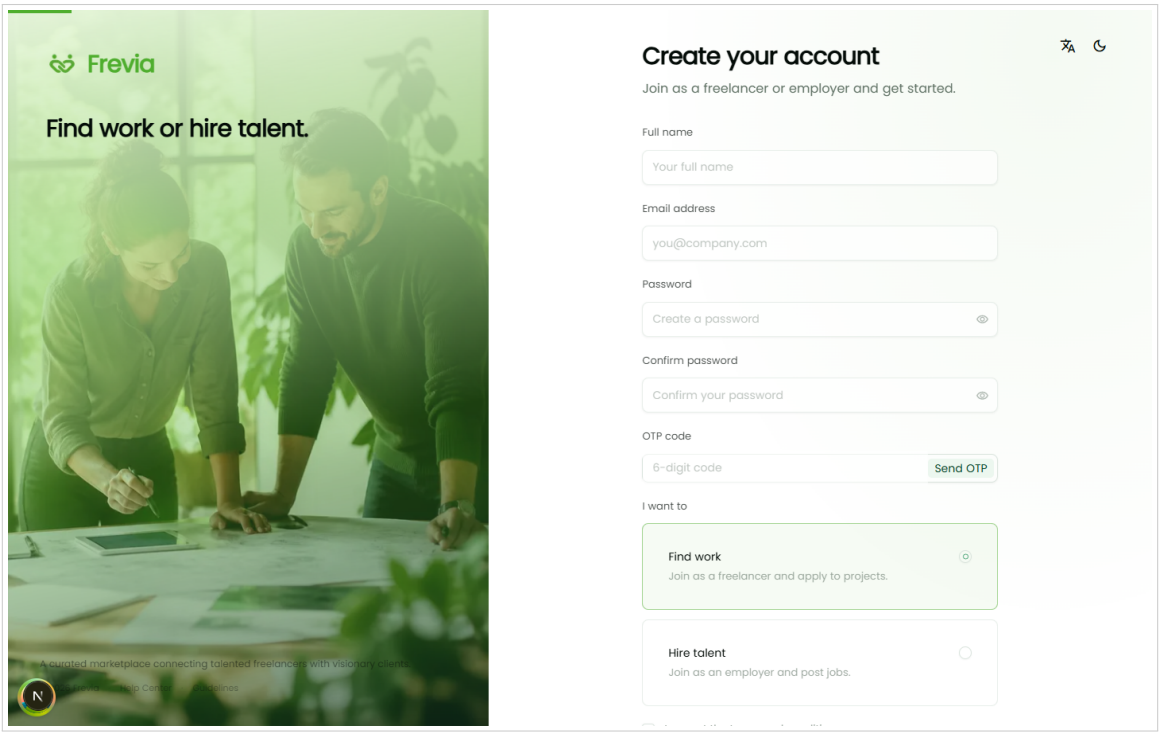

Figure 54: User registration screen

- Header displays “Create an account” and role choice selector.
- Two distinct selection cards allow choosing between “Join as Client” or “Join as Freelancer”.
- Form inputs collect Full Name, Email Address, Password, and Confirm Password with real-time strength feedback.
- OTP input field includes an integrated “Send OTP” trigger button.

- Terms of Service acceptance checkbox with active hyperlink references.

Function Details

Input Data

Data Name	Description
`name`	User's full display name string (2 to 50 characters).
`email`	Valid email address string; must be unique across the platform.
`password`	Strong password (minimum 8 characters with letters, numbers, and symbols).
`role`	Selected initial platform role: `CLIENT` or `FREELANCER`.
`otp`	6-digit verification code delivered to the user's email.
`agreeTerms`	Boolean flag indicating user accepted terms and policies.

Validation Rules

- Email must conform to RFC 5322 format and not already exist in the database.
- Password must be at least 8 characters and match `confirmPassword`.
- OTP must match the unexpired code recorded for that email address.
- `agreeTerms` must be strictly `true`.

Business Rules

- **BR-UA-11**: Email addresses must be globally unique across all accounts.
- **BR-UA-12**: The user must explicitly choose an initial role; default assignment is forbidden.
- **BR-UA-13**: Passwords must be hashed with bcrypt prior to persistence.
- **BR-UA-14**: OTP codes expire after 10 minutes and can only be validated once.

Normal Flow

1. Visitor opens `/register` and inputs name, email, and password.
2. Visitor selects preferred role (Client or Freelancer).
3. Visitor clicks "Send OTP"; system sends verification email and starts countdown.
4. Visitor inputs 6-digit code received in email and checks terms agreement.
5. Visitor clicks "Continue to Profile".
6. System validates OTP, creates account and profile, issues tokens, and navigates to onboarding.

Abnormal Flow / Exception Handling

Case	Description
A1	One or more mandatory fields are omitted. Form resolver blocks request and surfaces inline error messages.
A2	The email entered does not match standard RFC 5322 format. Form displays "Please enter a valid email address."
A3	The email is already registered by another account. Backend returns HTTP 422 with `AuthMessage.EMAIL_ALREADY_EXISTS`: "This email is already registered."
A4	The password does not satisfy complexity constraints (less than 8 chars, missing uppercase/number). Form displays requirement indicators.
A5	Password confirmation does not match the entered password. Form highlights confirm field with "Passwords do not match."
A6	The submitted OTP does not match the generated code. Backend throws `InvalidVerificationCodeException` (HTTP 422): "Invalid verification code."
A7	The submitted OTP code has exceeded its 10-minute lifetime. Backend throws `OTPExpiredException` (HTTP 422): "OTP has expired. Please request a new one."
A8	Guest repeatedly clicks "Send OTP" within a short timeframe. Rate limiter triggers `TooManyAttemptsException` (HTTP 422): "Too many attempts. Please try again later."
A9	Mail server fails during OTP dispatch. Backend throws `FailedToSendOTPException` (HTTP 422): "Unable to send OTP. Please try again later."
A10	Guest fails to check the Terms of Service box. System disables final submission with "You must accept the Terms of Service and Privacy Policy."

Output Data

Data Name	Description
`user`	Created user record with ID, email, name, role, and verification timestamp.
Auth Cookies	Newly issued `accessToken` and `refreshToken` stored in httpOnly cookies.

Post-Conditions

- New user row and role relationship written to the database.
- User is immediately logged in and guided to role profile setup.

3.2.5 Forgot Password

Function Trigger

A user clicks "Forgot password?" on the login screen and submits their email address. **Navigation path:** Login (`/login`) → "Forgot password?" (`/forgot-password`).

Function Description

Actors / Roles

- **Primary Actor:** Guest (registered user who lost access).
- **Secondary Actor:** Mail Delivery Service (Nodemailer / SMTP Server).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy, Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Provides a secure self-service mechanism for users to reset their forgotten passwords via email verification links, preventing account lockout while safeguarding against credential enumeration attacks.

Interface

- Forgot password page at `/forgot-password` with email input and submit button.
- Confirmation feedback screen informing user to check their email inbox.
- Secure reset password screen accessed through the unique tokenized email link.

Data Processing Overview

- User enters email and submits `POST /api/auth/forgot-password`.

- Backend looks up user record; if found, generates a cryptographically random token with 1-hour expiration.
- An email containing the password reset URL is dispatched.
- Regardless of whether the email exists, the API returns the identical generic message to prevent account enumeration.
- When user opens the link and submits a new password, backend validates token, hashes new password, updates database, invalidates the token, and revokes all active sessions.

Screen Layout (Mock-up Description)

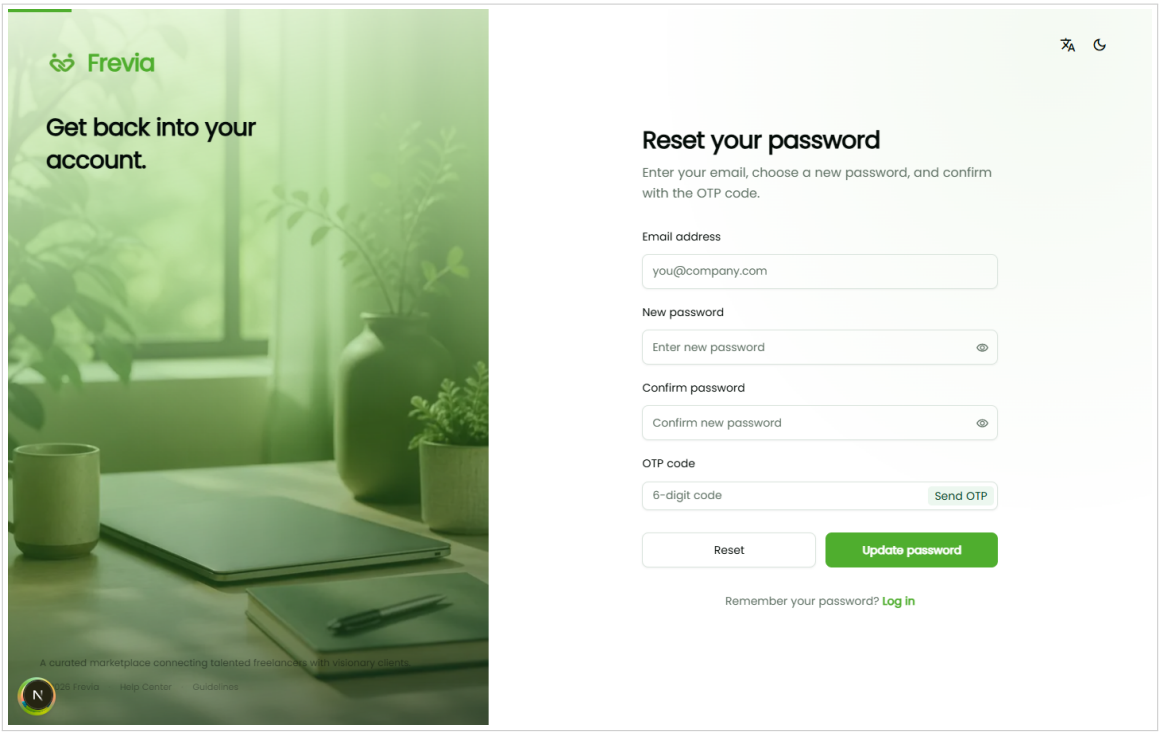

Figure 55: Forgot password screen

- Card heading reads “Forgot password?” with descriptive subtitle explaining the recovery process.
- Email input field with placeholder “you@company.com”.
- Solid green button “Send reset link”.
- Navigation link “Back to login” at the bottom of the card.

Function Details

Input Data

Data Name	Description
`email`	Registered email address string.
`token`	Secret reset token received in URL parameter (during password update).
`newPassword`	Replacement password meeting platform strength criteria.

Validation Rules

- `email` must be a valid email string.
- Reset token must exist, not be expired, and have not been previously consumed.
- `newPassword` must meet minimum length (8 chars) and complexity rules.

Business Rules

- **BR-UA-15**: System must return identical confirmation text whether the email is registered or not (anti-enumeration).
- **BR-UA-16**: Reset tokens expire exactly 1 hour following issuance.
- **BR-UA-17**: Successful reset terminates all existing sessions for the target account.

Normal Flow

1. User navigates to `/forgot-password` and enters registered email.
2. User clicks “Send reset link”.
3. Backend generates token, sends email, and returns generic confirmation.
4. User clicks the link in email to reach the reset screen.
5. User inputs new password and submits.
6. System updates password, revokes active sessions, and redirects to login with success toast.

Abnormal Flow / Exception Handling

Case	Description
A1	Email field left blank or submitted with invalid syntax. Client validation displays “Please enter a valid email address.”
A2	The email entered is not associated with any account in the database. The system returns the identical success message to prevent user enumeration.

Case	Description
A3	The account exists but was registered purely via Google OAuth (no local password). System sends an advisory email guiding the user to sign in via Google OAuth.
A4	The password reset token in the link has exceeded its 1-hour expiration window. The reset page displays “This reset link has expired. Please request a new one.”
A5	The token has already been consumed or carries an invalid cryptographic signature. System surfaces “This reset link is invalid or has already been used.”
A6	The replacement password fails strength constraints or confirmation mismatch occurs. Form highlights the fields inline with guidance text.
A7	Mail server failure during email dispatch. System logs error internally and returns standard response to protect user privacy.
A8	Database connection drops during password update and session revocation. System returns HTTP 500; old password remains active.

Output Data

Data Name	Description
Email Dispatch	HTML email containing secure reset link with query token.
Status Message	Informational confirmation indicating reset link dispatch.

Post-Conditions

- Account password is updated to new hashed value.
- Reset token is destroyed.
- Prior authentication sessions are revoked.

3.3 Role Management

3.3.1 View Role List

Function Trigger

An administrator accesses the Role Management section from the admin portal sidebar.
Navigation path: Admin Dashboard → Roles & Permissions → Roles (`/admin/roles`).

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Provides administrators with a centralized overview of all configured system access roles, displaying role names, descriptions, assigned permission sets, and action triggers for configuration.

Interface

- Table view displaying roles with columns: ID, Name, Description, Permissions, Created At, and Actions.
- Action toolbar containing "Create Role" button.
- Row-level action buttons: Edit Role, Delete Role, and View Details.

Data Processing Overview

- The client issues `GET /api/roles` upon rendering.
- The backend queries the database for all roles where `deletedAt = null`.
- Each role includes its associated permissions joined through the `RolePermission` relation.
- Results are returned ordered by creation date and rendered in the interactive table.

Screen Layout (Mock-up Description)

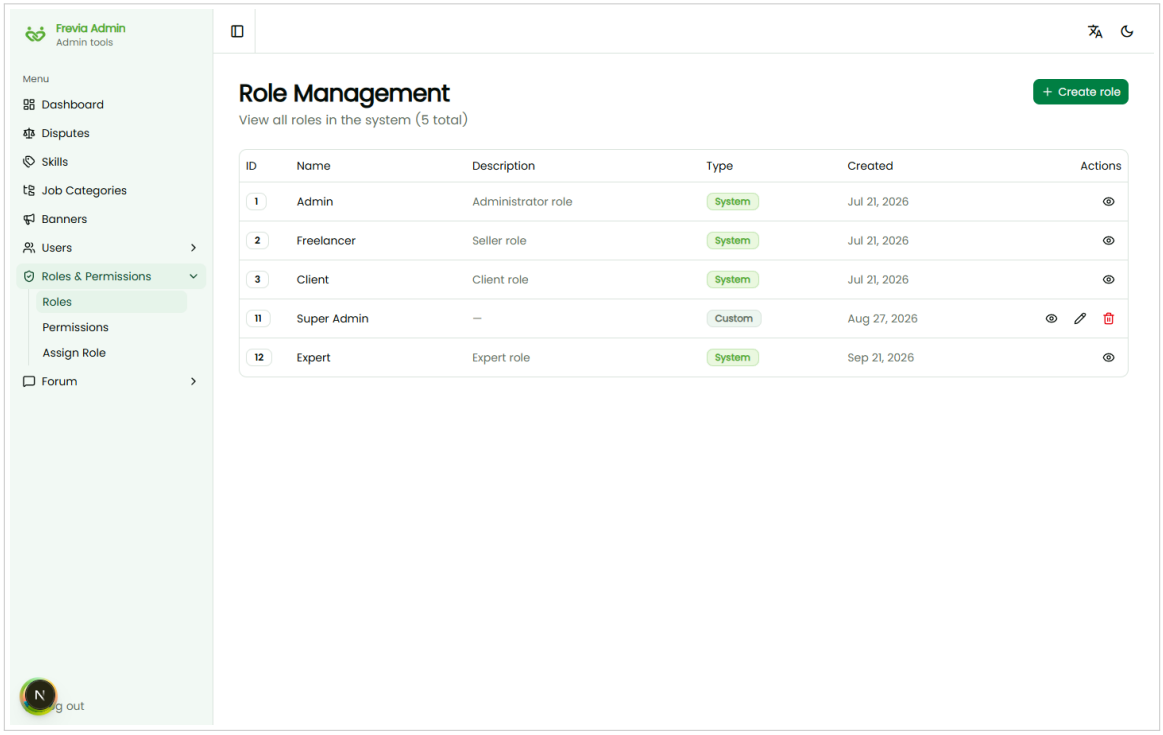

Figure 56: Role list screen

- Top header shows page title “Role Management” and total count subtitle.
- “Create Role” button positioned on the top right in green accent.
- Data table shows system roles (Admin, Client, Freelancer, Expert) and custom roles with permission badges.
- Actions column provides edit, delete, and detail view buttons for each record.

Function Details

Input Data

Data Name	Description
Admin Session	Valid JWT authentication cookie with Administrator authorization.

Validation Rules

- User must possess the `Admin` role.
- Only active roles with `deletedAt = null` are returned.

Business Rules

- **BR-RM-01**: Role management is strictly restricted to authenticated Administrators.
- **BR-RM-02**: Soft-deleted roles shall be excluded from the active list.

Normal Flow

1. Admin clicks “Roles” in the admin sidebar.
2. Browser sends `GET /api/roles`.
3. API queries active roles and joined permissions.
4. Table renders all roles with their assigned permission badges and action controls.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated or non-admin user navigates to the URL. The route middleware blocks access and redirects to `/login` or returns HTTP 403 Forbidden.
A2	Database connection drops or query times out. The API throws `FailedToLoadRolesException` (HTTP 500); client displays “Unable to load role list. Please try again later.”
A3	No active roles found in the database. The table renders a clean placeholder “No roles available” with a call-to-action to create a role.
A4	Elevated network latency slows data arrival. TanStack Query displays skeleton placeholders across table rows, preventing layout jump.
A5	Admin session token expires while viewing the list. A background query receives HTTP 401; the application clears cookies and redirects to sign-in.
A6	A role title or description contains exceptionally long text. The table cell applies CSS truncation with hover tooltip, preventing column overflow.

Output Data

Data Name	Description
`roles[]`	Array of role records containing `id`, `name`, `description`, `permissions[]`, and `createdAt`.

Post-Conditions

- Read-only operation; system data remains unchanged.

3.3.2 View Role Detail

Function Trigger

An administrator clicks the "View details" button (eye icon) on a role row or navigates to `/admin/roles/:id`.
Navigation path: `/admin/roles` → Click View Details icon on a row.

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Presents detailed information regarding a specific role, including its identifier, name, description, assigned permissions, and user counts.

Interface

- Role detail screen showing Role ID, Name, Description, and Metadata cards.
- Permissions catalog panel showing assigned route permissions grouped by category.
- Navigation back-button to return to the role table.

Data Processing Overview

- Client dispatches `GET /api/roles/:id`.
- Backend fetches role matching `id` with `deletedAt = null`.
- Backend joins permissions through the relation model and returns role details.
- Client renders role attributes and permission badges.

Screen Layout (Mock-up Description)

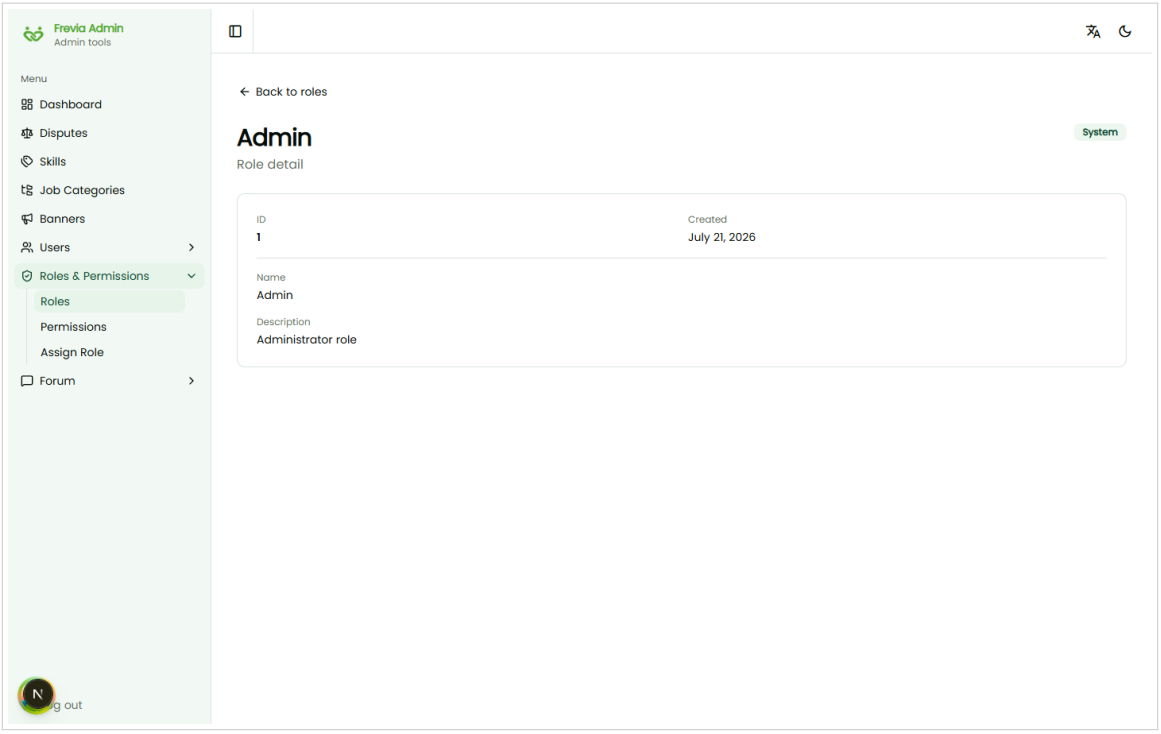

Figure 57: Role detail screen

- Header card presents role title with action buttons (Edit, Delete).
- Information section details Role ID, Creation Timestamp, and Description.
- Lower container displays full assigned permissions breakdown.

Function Details

Input Data

Data Name	Description
`id`	Numeric identifier of the target role passed in route path.

Validation Rules

- Target role must exist and not be soft deleted.

Business Rules

- **BR-RM-03**: Soft-deleted roles cannot be viewed in detail mode.

Normal Flow

1. Admin clicks view icon on a role in the role list.

2. Application requests `GET /api/roles/:id`.
3. API returns role object with permissions.
4. Screen displays full role specifications and assigned routes.

Abnormal Flow / Exception Handling

Case	Description
A1	Non-numeric or invalid ID format in URL path (e.g., `/admin/roles/abc`). Router fails validation and shows 404 Not Found page.
A2	Role ID does not exist or has already been soft-deleted. API throws `RoleNotFoundException` (HTTP 404); page displays "Role not found" with return button.
A3	Database query failure during role retrieval. API throws `FailedToLoadRoleDetailException` (HTTP 500); client shows error notification toast.
A4	The role currently has zero permissions assigned. The permissions section displays an informative empty card "No permissions assigned to this role yet."
A5	Unauthenticated visitor attempts to access direct detail URL. Middleware intercepts and redirects to `/login`.
A6	The role is deleted by another administrator concurrently while being viewed. Subsequent edits or page refreshes reflect the deletion and navigate back.

Output Data

Data Name	Description
Role Record	Comprehensive role object with permissions list.

Post-Conditions

- Read-only operation.

3.3.3 Create Role

Function Trigger

An administrator clicks the “Create Role” button on the Role Management page.

Navigation path: `/admin/roles` → Click “Create Role” button.

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Allows administrators to establish a new custom RBAC role with defined title, description, and initial permission bindings.

Interface

- Modal dialog with fields for Role Name and Description.
- Cancel button and solid green “Create Role” submit button.

Data Processing Overview

- Admin enters details and submits `POST /api/roles`.
- Backend checks for duplicate role names and prevents collision with reserved system roles.
- System persists new role record in PostgreSQL.
- Table view refreshes immediately to display the created role.

Screen Layout (Mock-up Description)

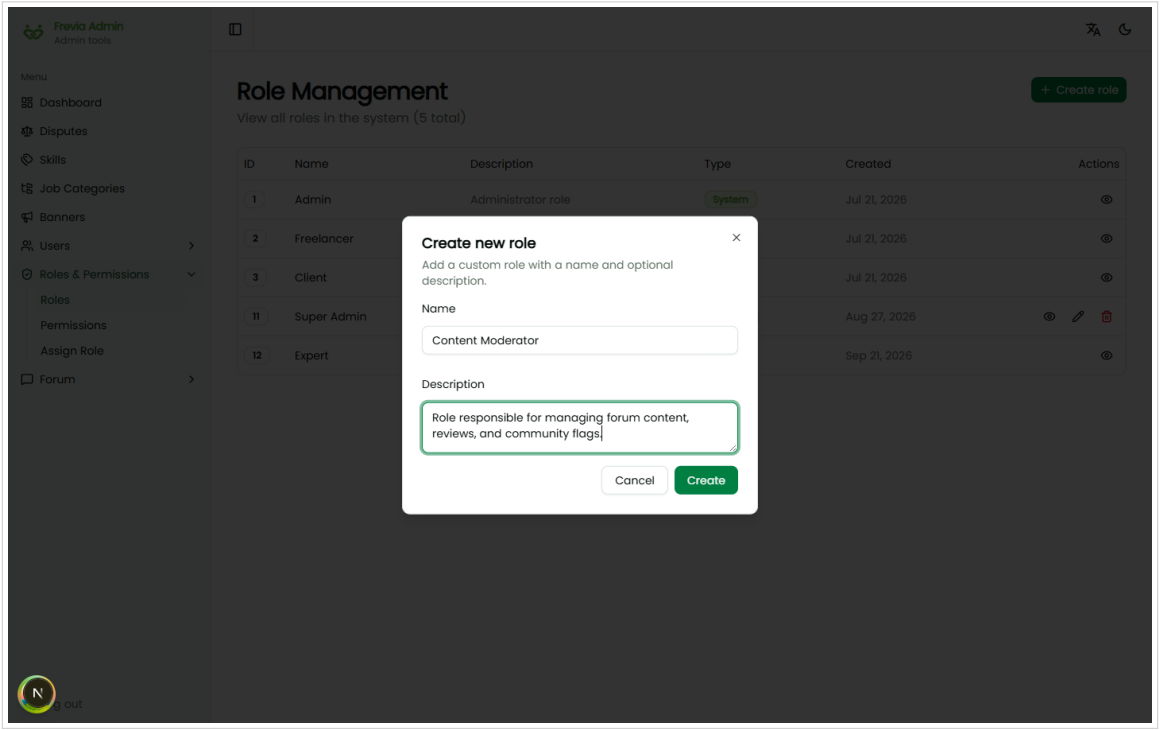

Figure 58: Create role dialog

- Dialog modal overlays the page with dark backdrop.
- Header reads “Create Role” with descriptive instruction text.
- Name input marked with required asterisk (*).
- Description textarea for functional documentation.
- Footer provides “Cancel” and “Create Role” actions.

Function Details

Input Data

Data Name	Description
`name`	Unique role name string (required, 2 to 50 characters).
`description`	Optional role purpose description string.

Validation Rules

- Role name must be non-empty and unique among active roles.
- Role name must not match system keywords: `Admin`, `Client`, `Freelancer`, `Expert`.

Business Rules

- **BR-RM-04**: System role names are immutable and reserved.
- **BR-RM-05**: Duplicate active role names are strictly rejected.

Normal Flow

1. Admin clicks “Create Role” button.
2. Admin inputs role name and description in dialog.
3. Admin clicks “Create Role” button to submit.
4. Backend creates record and returns HTTP 201.
5. Dialog closes, success toast appears, and table updates.

Abnormal Flow / Exception Handling

Case	Description
A1	Role name field submitted empty or consisting only of whitespace. Frontend validation halts request: “Name is required.”
A2	Role name already in use by another active role. API throws `RoleAlreadyExistsException` (HTTP 409): “A role with this name already exists.”
A3	Admin enters a reserved system role name (`Admin`, `Client`, `Freelancer`). API throws `CannotModifySystemRoleException` (HTTP 403): “Cannot use a reserved system role name.”
A4	Name exceeds maximum allowed character length (50 characters). Client enforces limit and shows character limit indicator.
A5	Database error occurs during insertion. API throws `FailedToCreateRoleException` (HTTP 500); system shows error toast and retains entered text in dialog.
A6	Admin clicks “Cancel” or presses Escape key. Dialog closes immediately and unsaved data is discarded.
A7	Rapid double-clicking on submit button. Button disables into loading state on first click, preventing duplicate records.

Output Data

Data Name	Description
Created Role	Newly persisted role entity with assigned ID and timestamps.

Post-Conditions

- New role record exists in database and is visible in role list.

3.3.4 Update Role

Function Trigger

An administrator clicks the "Edit" icon (pencil) on a role row in the Role Management table.

Navigation path: `/admin/roles` → Click Edit icon on row.

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Allows administrators to modify the display name and descriptive notes of an existing role.

Interface

- Edit Role dialog prefilled with the role's current name and description.
- Save Changes button and Cancel button.

Data Processing Overview

- Admin modifies fields and clicks "Save Changes".
- Request sent via `PATCH /api/roles/:id` or `PUT /api/roles/:id`.
- Backend validates name uniqueness against other active roles.
- Updates database row and returns updated entity.

Screen Layout (Mock-up Description)

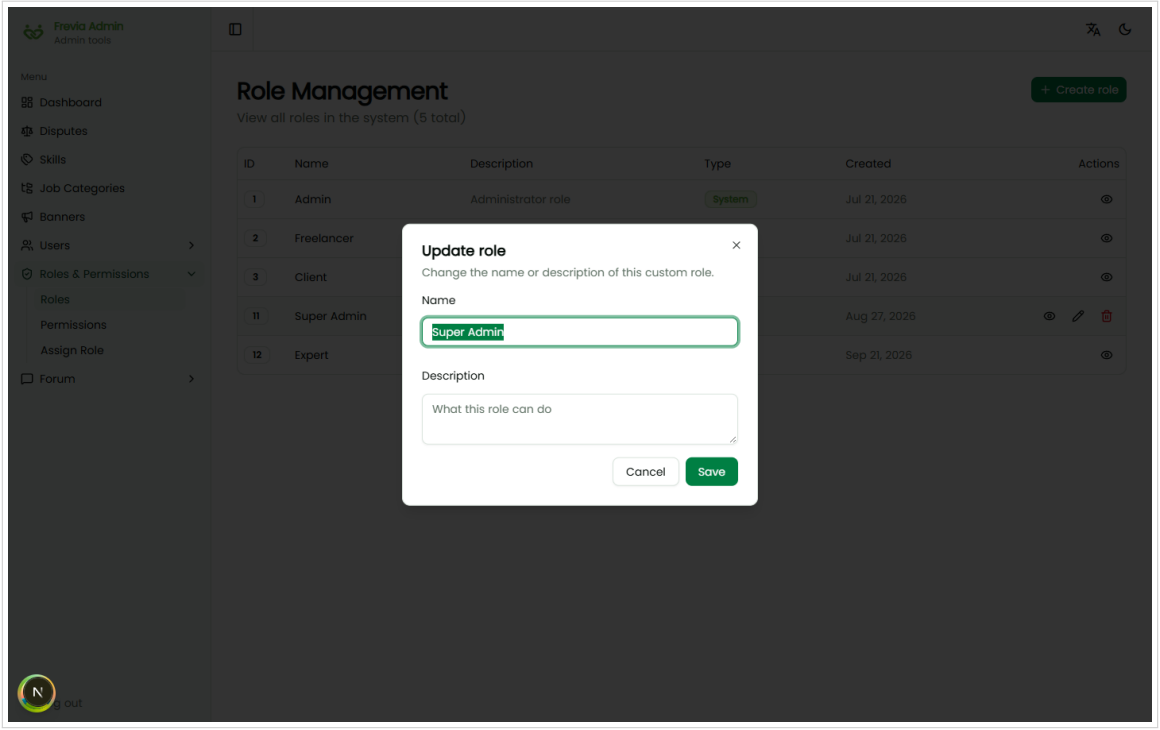

Figure 59: Update role dialog

- Dialog title states “Update Role” with explanation.
- Name input pre-populated with current role title.
- Description area pre-populated with existing text.
- Action buttons: Cancel and Save Changes.

Function Details

Input Data

Data Name	Description
`id`	Target role identifier.
`name`	Updated role title string.
`description`	Updated description string.

Validation Rules

- Name must be non-empty and unique among active roles.
- System role names cannot be reassigned to other roles.

Business Rules

- **BR-RM-06**: Role ID remains unchanged throughout update.

Normal Flow

1. Admin clicks edit icon on desired role row.
2. Modal opens with existing values prefilled.
3. Admin updates fields and clicks “Save Changes”.
4. Server updates record, closes dialog, and refreshes view.

Abnormal Flow / Exception Handling

Case	Description
A1	Target role was deleted or no longer exists. API throws `RoleNotFoundException` (HTTP 404): “Role not found.”
A2	Admin erases name field. Client validation rejects submission: “Role name is required.”
A3	Updated name conflicts with another existing active role. API throws `RoleAlreadyExistsException` (HTTP 409): “A role with this name already exists.”
A4	Admin attempts to rename a protected system role. API throws `CannotModifySystemRoleException` (HTTP 403): “System roles cannot be renamed.”
A5	Database update query fails. API throws `FailedToUpdateRoleException` (HTTP 500); error toast is shown and form retains modified values.
A6	Admin clicks “Cancel” or backdrop. Dialog dismisses and values revert to database state without persisting.
A7	Network disconnects while submitting. Form indicates network failure and allows retry without re-typing.

Output Data

Data Name	Description
Updated Entity	Modified role row with updated timestamp.

Post-Conditions

- Updated role information is reflected across all administrative screens.

3.3.5 Delete Role

Function Trigger

An administrator clicks the "Delete" icon (trash can) on a custom role row.

Navigation path: `/admin/roles` → Click Delete icon on row.

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Soft-deletes a custom role from the platform, verifying that it is not assigned to active users and protecting core system roles from accidental deletion.

Interface

- Delete confirmation alert dialog with warning message.
- "Cancel" action and destructive red "Delete Role" button.

Data Processing Overview

- Admin confirms deletion in dialog modal.
- Client dispatches `DELETE /api/roles/:id`.
- Backend verifies role is not a protected system role and is not currently held by active users.
- Backend sets `deletedAt = new Date()` to soft-delete the record.
- Table updates and excludes the deleted role from the active list.

Screen Layout (Mock-up Description)

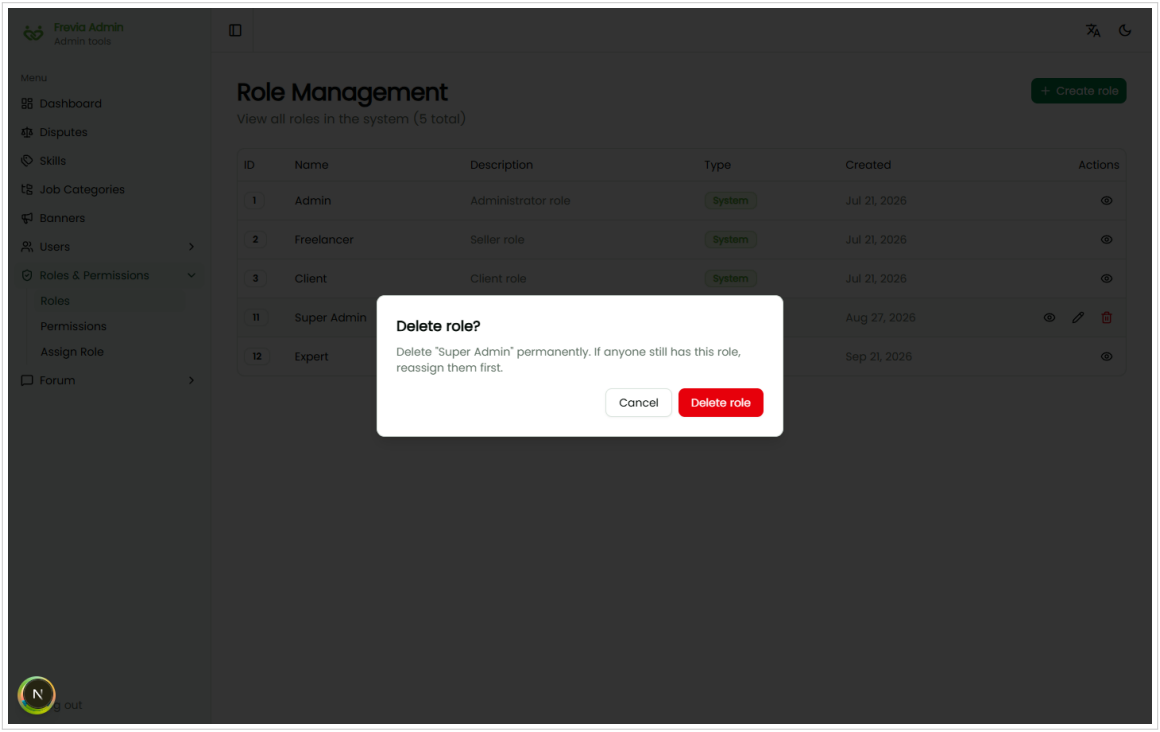

Figure 60: Delete role confirmation dialog

- Alert dialog heading “Delete Role” with role name.
- Confirmation warning text explaining the implications of removal.
- Action buttons: Cancel and Delete Role in red destructive styling.

Function Details

Input Data

Data Name	Description
`id`	Target role identifier to be soft-deleted.

Validation Rules

- Role must exist and not already be deleted.
- Role must not be a protected system role (`Admin`, `Client`, `Freelancer`).
- Role must not have active assigned users.

Business Rules

- **BR-RM-07**: Deletion must be executed as a soft delete via `deletedAt`.
- **BR-RM-08**: Roles assigned to active users cannot be deleted until users are reassigned.

Normal Flow

1. Admin clicks delete icon on a custom role row.
2. Confirmation alert dialog appears.
3. Admin confirms deletion by clicking “Delete Role”.
4. Backend performs soft delete and returns HTTP 200.
5. Table view removes row and shows success toast.

Abnormal Flow / Exception Handling

Case	Description
A1	Role does not exist or has already been soft-deleted. API throws `RoleNotFoundException` (HTTP 404): “This role has already been deleted.”
A2	Role is currently assigned to one or more active users in `UserRole`. API throws `RoleInUseException` (HTTP 409): “This role cannot be deleted while assigned to active users.”
A3	Admin attempts to delete a protected system role (`Admin`, `Client`, `Freelancer`, `Expert`). API throws `CannotModifySystemRoleException` (HTTP 403): “System roles cannot be deleted.”
A4	Admin clicks “Cancel” or presses Escape. The dialog dismisses without dispatching any deletion request.
A5	Database error during soft-delete execution. API throws `FailedToDeleteRoleException` (HTTP 500); client notifies “Unable to delete role. Please try again later.”
A6	The role was concurrently deleted by another admin session. Table refreshes and informs user the role is already removed.

Output Data

Data Name	Description
Result Status	Success confirmation with soft-deleted timestamp.

Post-Conditions

- Target role has `deletedAt != null`.
- Role is omitted from all standard role selectors and lists.

3.3.6 Update Role Permissions

Function Trigger

An administrator navigates to the Assign Role screen, selects a role, modifies permission checkboxes, and saves.
Navigation path: Admin Dashboard → Roles & Permissions → Assign Role (`/admin/roles/assign`).

Function Description

Actors / Roles

- **Primary Actor:** Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Configures granular API route access permissions for any system or custom role by selecting or unchecking individual permissions grouped by functional module.

Interface

- Role selector dropdown at the top of the interface.
- Permission matrix grouped by module cards (AUTH, ROLES, USERS, FORUMS, DISPUTES, etc.).
- Checkbox controls for individual permissions and module-level "Select All" toggles.
- Save permissions button fixed at bottom or header.

Data Processing Overview

- Selecting a role fetches its current permissions via `GET /api/roles/:id`.
- Checkboxes automatically pre-populate with existing assigned permissions.
- Admin modifies selections and clicks "Save permissions".

- Client dispatches `PUT /api/roles/:id/permissions` with array of selected permission IDs.
- Backend executes a transaction: purges existing `RolePermission` associations for this role and inserts new bindings for all selected IDs.
- Cache is refreshed and permission changes take effect immediately.

Screen Layout (Mock-up Description)

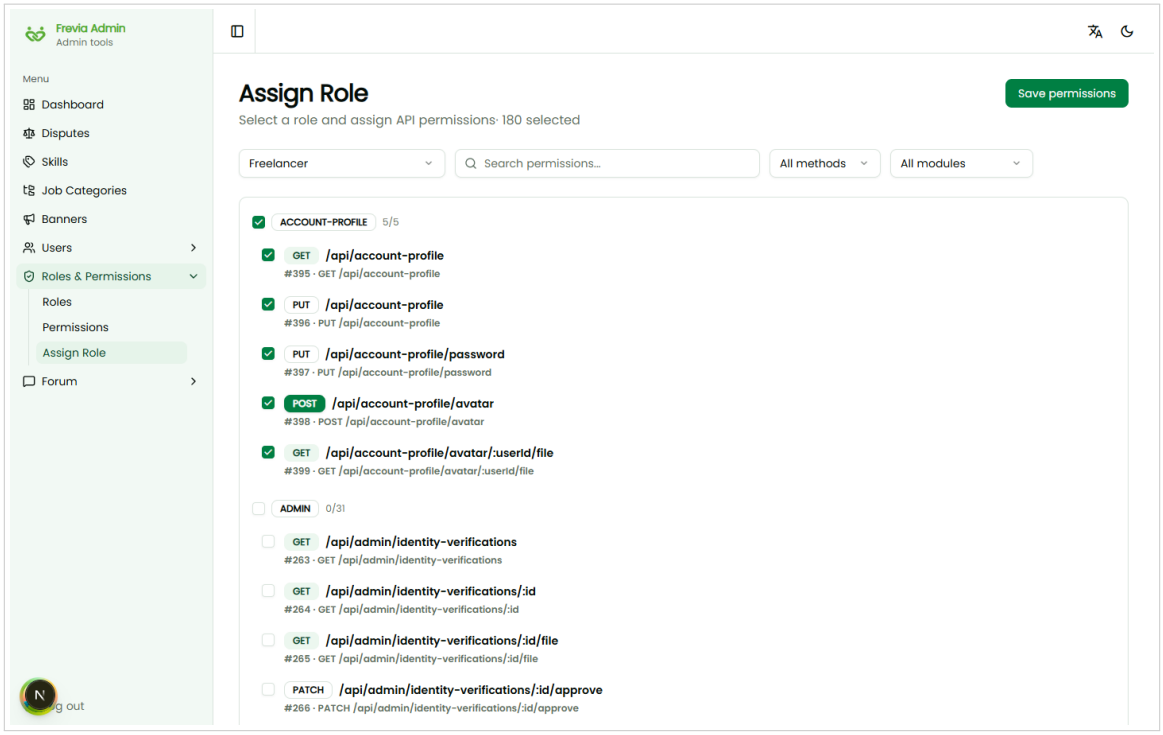

Figure 61: Assign permissions to role screen

- Header displays “Assign Role Permissions” and the role selection dropdown.
- Main area renders permission module cards with HTTP method badges (GET, POST, PUT, DELETE).
- Checkboxes next to each permission indicate active access rights.
- “Save permissions” button triggers atomic database updates.

Function Details

Input Data

Data Name	Description
`roleId`	Target role identifier.
`permissionIds[]`	Array of selected integer permission IDs.

Validation Rules

- Target role must exist and not be soft deleted.
- All passed `permissionIds` must match existing active permissions in the database.

Business Rules

- **BR-RM-09**: Permission updates replace the role's entire permission mapping set.
- **BR-RM-10**: Permission changes take effect immediately across all active user sessions with that role.

Normal Flow

1. Admin opens the Assign Role page and selects a role from the dropdown.
2. System loads permissions and pre-checks currently assigned rights.
3. Admin checks or unchecks desired route permissions.
4. Admin clicks "Save permissions".
5. Backend replaces role permission mappings in an atomic transaction.
6. Success notification confirms update and new access rights are live.

Abnormal Flow / Exception Handling

Case	Description
A1	Admin attempts to save without selecting a role. UI disables button and prompts "Choose a role to start assigning permissions."
A2	Target role does not exist or was soft-deleted. API throws `RoleNotFoundException` (HTTP 404): "Role not found."
A3	Admin deselects all checkboxes. System prompts confirmation for empty permissions set, then clears all route access upon confirmation.
A4	One or more permission IDs no longer exist in database. API throws `InvalidPermissionIdsException` (HTTP 400): "One or more selected permissions are invalid."

Case	Description
A5	Database transaction failure during batch update. API throws `FailedToSetRolePermissionsException` (HTTP 500); prior bindings remain unchanged.
A6	Admin attempts to revoke critical access rights from the core Administrator role. System performs security check to prevent total admin lockout.
A7	Network disconnects during save request. Checkbox selection state is preserved in client memory, and an alert toast offers immediate retry.

Output Data

Data Name	Description
Updated Role	Role record containing updated list of assigned permission IDs.

Post-Conditions

- `RolePermission` table rows are synchronized with the selected set.
- Authorization guards apply new permissions upon subsequent user requests.

3.4 Manage Forum

3.4.1 View Forum Category List

Function Trigger

Any visitor opens the Community Forum area from the main navigation, or follows a direct link to the forum landing page. The function is public, therefore it runs on every visit of the page, whether or not the visitor is signed in.
Navigation path: Home → Forum (`/forum`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator. The screen is readable by every role, including visitors without an account.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/backend`), Frevia REST API (NestJS) and PostgreSQL accessed through Prisma ORM.

Purpose

The function is the entry point of the community area. It presents the active forum categories together with the number of published posts that each one contains, and it highlights both the categories and the members with the highest activity, so a visitor can reach an ongoing discussion in one click and can immediately see which topics are alive.

Interface

- Community Forum page at `/forum`. Guests see the public header and footer; a signed-in member sees the dashboard shell with the sidebar of their role (Client or Freelancer).
- Hero section with the breadcrumb "Forum", the language switcher, the title "Community Forum" and the description "Ask questions, share tips, and learn from freelancers and clients on Frevia.".
- Category search form with the placeholder "Search categories…", a search icon, a clear button and a "Search" action. The search is applied automatically after a short pause and can also be submitted with Enter.
- List of category rows; each row shows a message icon, the category name, its description and the number of posts, and opens the category detail screen.
- "Most Active" panel listing the three categories with the largest number of posts.
- "Top Contributors" panel listing the five members with the largest number of posts, with their avatar, display name, post counter and comment counter.
- Result summary "Showing x–y of z categories" followed by numbered pagination with ellipsis; the list displays five categories per page.
- Empty-state panel with the title "No categories yet" or "No categories found" depending on whether a search is active; the no-result state offers a "Clear search" action.

Data Processing Overview

- The server component of the page reads the optional `search`, `page` and `limit` parameters from the query string.
- The page requests `GET /api/forums/categories` and `GET /api/forums/users/top?limit=5` in parallel, adding the access token when the visitor is signed in.
- The API keeps only the categories whose `deletedAt` is null, orders the result by `name` in ascending order, and counts only the posts that are not soft deleted and whose `moderationStatus` is `APPROVED`.
- When a search term is present, the page filters the categories by name or by description without case sensitivity before computing the pagination; the API receives no search parameter for categories.
- The page applies the page size locally: the default is five categories per page, the page

number is clamped between 1 and the last page, and the summary reports the displayed range and the filtered total.

- The “Most Active” panel is computed from the complete category list by sorting it by `postCount` in descending order and taking the first three entries; it does not issue a separate request.
- The “Top Contributors” panel uses the second response: the API selects the users that are neither deleted nor banned, counts their approved posts and their non-deleted comments, and orders them by post count in descending order.
- A category without a description is rendered with the fallback text “No description provided.”; an empty result set is rendered as “No categories yet” or “No categories found” when a search is active.

Screen Layout (Mock-up Description)

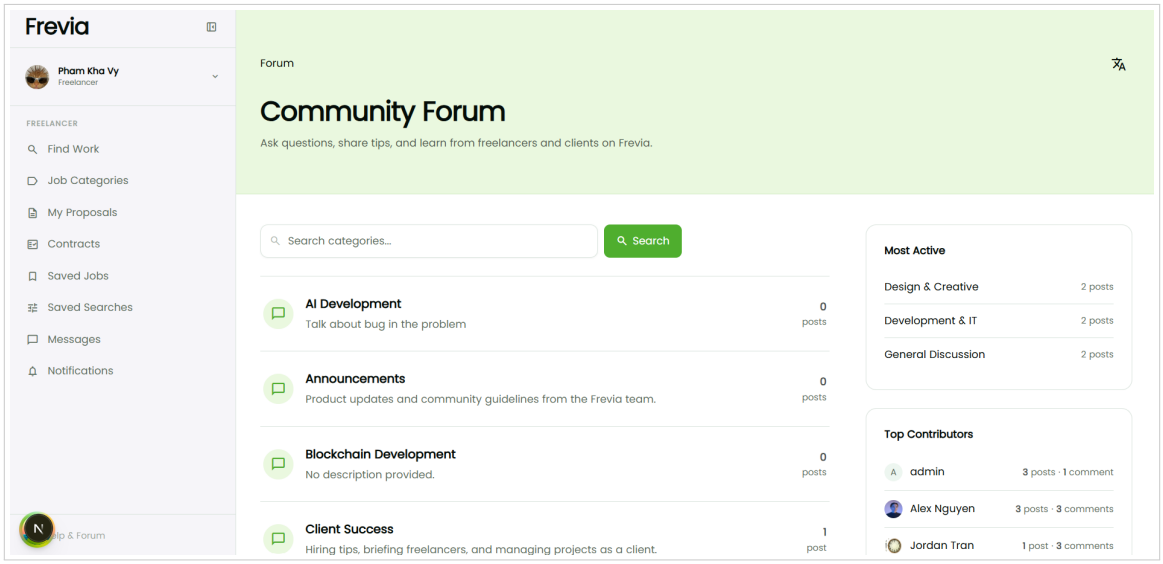

Figure 62: Forum category list screen

- The hero states the title “Community Forum” and a short description of the area, which introduces the forum to first-time visitors; the breadcrumb and the language switcher sit on the same line.
- The main area is split into two columns on large screens: the category list occupies the left column and the two ranking panels occupy the right column.
- Each category row is a link to the category detail screen. It displays a round message icon, the category name, the description limited to two lines and, on the right, the post counter with the word “posts”.
- The search form sits above the list. The clear button appears only when the field contains text, and the result summary and the pagination sit below the list.
- The “Most Active” panel ranks categories by post volume and gives a shortcut to the busiest

discussions.

- The “Top Contributors” panel shows the avatar, the display name, the post count and the comment count of the most active members.
- When the list is empty, the placeholder explains that categories will appear once they are created; when a search returns nothing, the placeholder invites the visitor to try another keyword or to clear the search.

Function Details

Input Data

Data Name	Description
Page route	`/forum`. The page can also receive the optional `search`, `page` and `limit` parameters.
`search`	Optional keyword compared with the category name and description, without case sensitivity.
`page`	Page number of the category list; integer greater than or equal to 1, default 1.
`limit`	Number of categories per page; the page uses 5 by default and falls back to 5 when the parameter is not a positive integer.
`limit` (top users)	Number of members returned by the “Top Contributors” call; fixed to 5 by the page and validated by the API between 1 and 10.
Visitor session	Optional access token forwarded by the server request. A guest receives the same list and the same ranking panels.

Validation Rules

- The list endpoint takes no parameter, therefore it cannot be malformed; it always returns the complete list of visible categories.
- The `limit` of both ranking endpoints is coerced to an integer, must be at least 1 and at most 10, and defaults to 3 when it is absent.
- A category is visible only when its `deletedAt` column is null; a category moved to the trash is excluded from the list and from both rankings.
- A post contributes to a counter only when it is not soft deleted and its `moderationStatus` is `APPROVED`.

- A user appears in “Top Contributors” only when the account is not deleted and is not banned.

Business Rules

- **BR-FORUM-01**: Only categories with `deletedAt = NULL` shall be displayed.
- **BR-FORUM-02**: Categories shall be ordered by `name` in ascending order.
- **BR-FORUM-03**: `postCount` shall count only posts with `deletedAt = NULL` and `moderationStatus = APPROVED`.

Normal Flow

1. The visitor opens the Forum area from the navigation menu.
2. The web application renders the page shell and issues the category request.
3. The BFF proxy forwards the request to the REST API, adding the access token when the visitor is signed in.
4. The API queries the categories that are not soft deleted and counts their approved posts.
5. The API returns the ordered list of categories with `postCount`.
6. The client renders the category grid, applying the description fallback where needed.
7. The client issues the two ranking requests for the side panels.
8. The API returns the three most active categories and the five most active members.
9. The client renders both panels; the page is now fully displayed.
10. The visitor selects a category card and is routed to the category detail screen.

Abnormal Flow / Exception Handling

Case	Description
A1	The database cannot be queried. The API returns a server error carrying `Error.FailedToLoadForumCategories` and the page shows a generic error instead of the grid.
A2	No category exists or every category has been soft deleted. The page renders the empty state “No categories yet”; the ranking panels are hidden.
A3	A category has no description. The card shows “No description provided.” instead of an empty area.
A4	The ranking call fails while the list succeeds. The grid remains usable and only the affected panel is omitted, so navigation is never blocked.
A5	The visitor loses the network connection. The client displays the generic error message and offers a retry by reloading the page.

Case	Description
A6	The access token has expired. The BFF proxy refreshes the token once and repeats the call; if the refresh fails the visitor continues as a guest.
A7	The account was banned while the page was open. The BFF proxy detects the ban response, clears the authentication cookies and the client redirects to the sign-in page.

Output Data

Data Name	Description
`categories[]`	Visible categories ordered by name, each with `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt` and `postCount`.
`categories` (top)	Up to three categories ordered by post count in descending order, with the same fields as the list.
`users` (top)	Up to five members with `id`, `displayName`, `avatarUrl`, `postCount` and `commentCount`.
Empty collection	An empty array when no category is visible; the client renders the empty state described above.

Post-Conditions

- The visitor sees every active category with the number of published posts it holds.
- No data is modified: the function is read-only.
- Deleted categories, deleted posts, rejected posts and pending posts are never exposed through this screen.

3.4.2 View Forum Category Detail and Post List

Function Trigger

The visitor selects a category row on the forum landing page, follows a shared category link, or searches for a post from the category screen.
Navigation path: Home → Forum → *Category* (`/forum/{category-slug}-{category-id}`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lists the published posts of one category so that members can read, search and filter the discussions of a topic. It also gives signed-in members the entry point to create a post, to restrict the list to their own posts, and to reach the most liked and commented posts published in the category during the last seven days.

Interface

- Breadcrumb "Forum / *Category*" and the category header with its name, description and post count.
- Search box with the placeholder "Search posts…" and a clear-search button; the search is submitted with Enter and filters the posts of the current page by title.
- "My Posts" toggle, displayed only to signed-in members.
- "New Post" button, which opens the post creation dialog; a guest is routed to the sign-in page instead.
- Page-size selector labelled "10 / page" by default, offering 5, 10, 20 and 50 posts per page.
- Filter chips row introduced by "Filters:", with one removable chip for the search term, one removable chip for "My Posts" and a "Clear all" action.
- Post rows with the title, a two-line excerpt of the content, the author avatar and name, the creation date, the like count and the comment count.
- Summary "Showing x–y of z posts", which appends "(yours)" while "My Posts" is active, followed by numbered pagination with ellipsis.
- "Trending in {category}" panel, displayed in the left column on large screens and above the list on smaller screens, listing up to three posts of the last seven days.
- Empty states for no post ("No posts yet"), no search result ("No posts found") and no personal post ("You have no posts here").

Data Processing Overview

- The page resolves the category from the slug in the URL and requests `GET /api/forums/posts?page=..`
- When the visitor submits the search box, the client filters the posts already loaded for the current page by title, without case sensitivity, and rewrites the URL with a `search`

parameter and page 1. The request to the API is not repeated with the keyword, so the search narrows the current page only.

- When "My Posts" is active, the page reads the `myPosts=1` flag from the URL and sends the identifier of the signed-in member as the `userId` parameter.
- The API filters the posts whose `deletedAt` is null and whose `moderationStatus` is `APPROVED`, applies the category, author and optional title filters, and sorts the result by `createdAt` in descending order.
- Lists and counts are read in one database transaction, then the API returns the page together with the pagination block.
- The trending panel requests the top posts of the category with a limit of three; the API keeps the approved posts published during the last seven days and orders them by like count, then comment count, then creation date.
- Changing the page, the page size, the search term or the "My Posts" filter rewrites the query string, so the state of the list can be shared and restored by reloading.

Screen Layout (Mock-up Description)

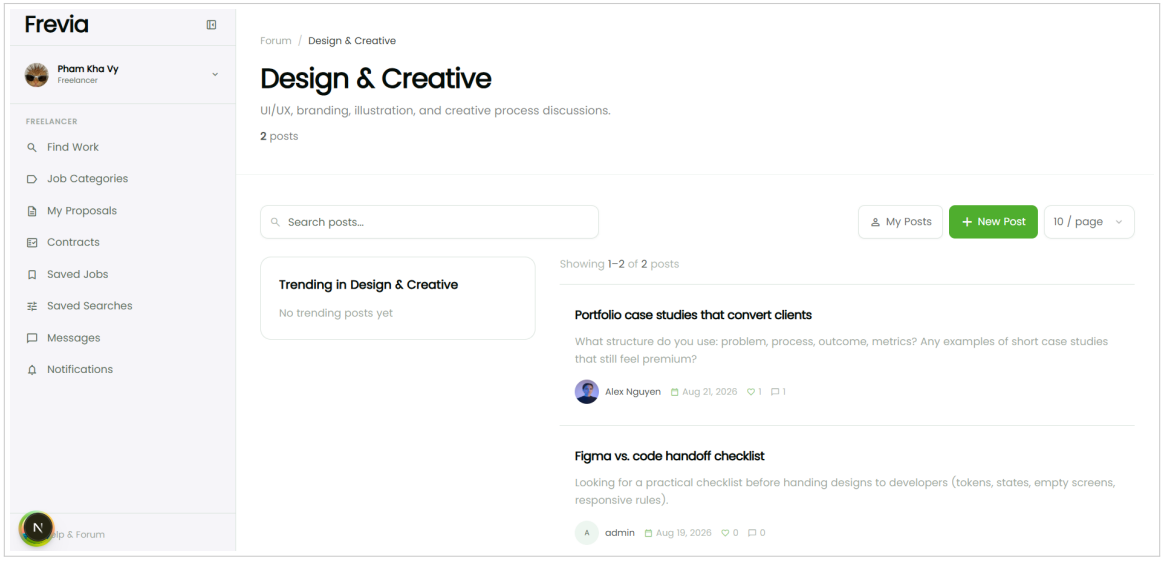

Figure 63: Category detail screen with the list of its posts

- The breadcrumb reads "Forum / *Category*", followed by the category name, its description and the number of approved posts it contains.
- The toolbar groups the search box, the "My Posts" toggle, the "New Post" button and the page-size selector; the search box submits on Enter and can be cleared with the cross icon.
- When a filter is active, a chips row labelled "Filters:" recalls the search term and the "My Posts" state, each chip removable, and offers "Clear all".
- Each post is a row rather than a card: it shows the title, the beginning of the content without the rich-text markup, the author avatar and name, the date, and the like and comment

counters.

- The summary “Showing x–y of z posts” sits above the list and appends “(yours)” when “My Posts” is active, so the member knows that the range is restricted to their own discussions.
- The “Trending in {category}” panel occupies the left column on large screens and is stacked above the list on smaller screens; it shows a numbered shortcut for each of the three most liked and commented posts of the last seven days, or “No trending posts yet” when the window is empty.
- The footer of the list shows numbered pagination with ellipsis, previous and next controls, and the range of records currently displayed.

Function Details

Input Data

Data Name	Description
`categoryId`	Identifier of the category taken from the URL; it restricts the list to one category.
`page`	Page number, integer greater than or equal to 1, default 1.
`limit`	Page size, integer greater than or equal to 1, default 10; the selector offers 5, 10, 20 and 50 and re-sends the request when it changes.
`search`	Optional search term compared with the post title, without case sensitivity. It is stored in the URL and applied by the client to the posts of the current page.
`myPosts`	Optional flag set to `1` by the “My Posts” toggle; the page converts it into the author filter described below.
`userId`	Optional author filter sent to the API when `myPosts=1` and a member is signed in; it always carries the identifier of that member.

Validation Rules

- `page` and `limit` are coerced to integers and must be at least 1; the page falls back to 1 and the page size to 10 when the parameters are absent or not numeric, while a negative value is rejected by the API validation pipe.
- `categoryId` and `userId` must be positive integers when they are sent.
- `search` is optional and is compared with the title only, without case sensitivity; leading and trailing spaces are trimmed by the client before the URL is rewritten.

- A requested page beyond the last page returns an empty list while the pagination block still reports the real total, so the client can render an empty state.
- Only approved and non-deleted posts are counted and displayed, therefore the totals never include posts that are pending, rejected or deleted.
- The `myPosts` flag is honoured only when a member is signed in; a guest who forces it in the URL receives the unfiltered list because no author identifier is sent.

Business Rules

- **BR-FORUM-06**: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed.
- **BR-FORUM-07**: Posts shall be sorted by `createdAt` in descending order.
- **BR-FORUM-08**: The system shall support pagination and configurable page sizes.
- **BR-FORUM-09**: `commentCount` shall count only comments with `deletedAt = NULL`.

Normal Flow

1. The visitor selects a category row on the forum landing page.
2. The client routes to the category screen and reads the page, the page size, the search term and the “My Posts” flag from the query string.
3. The client requests the posts of the category from the API.
4. The API validates the filter, resolves the pagination offset and runs the list and count queries in one transaction.
5. The API applies the visibility rules and orders the posts by creation date from the newest to the oldest.
6. The API returns the page of posts with their like and comment counters plus the pagination block.
7. The client renders the rows, strips the rich-text markup for the excerpt and applies the empty states when no row is returned.
8. The client requests the trending posts of the category and renders the left panel.
9. The visitor optionally searches, changes the page size, moves to another page or activates “My Posts”. The search and the filters rewrite the URL, the page size and the pagination re-issue the request with the new parameters, and the summary follows the active state.
10. The visitor selects a post row and is routed to the post detail screen.

Abnormal Flow / Exception Handling

Case	Description
A1	The category does not exist or was soft deleted. The page cannot resolve it and renders the not-found state with a link back to the forum.
A2	The database query fails. The API returns `Error.FailedToLoadForumPosts` and the list area shows a generic error instead of the rows.
A3	The requested page is larger than the last page. The API returns an empty list and the client renders “No posts yet” with the pagination intact.
A4	The search term matches no title on the current page. The client reports “No posts found” with “No results for “{query}”. Try a different search.” and keeps the clear-filters action available.
A5	“My Posts” is activated by a guest. The toggle is not rendered; if the flag is forced in the URL, the page sends no author identifier and the unfiltered list is displayed.
A6	The page size or page number is not a positive integer. The page falls back to the default for missing or non-numeric values, while a negative value is rejected by the validation pipe and the client falls back to the first page.
A7	The trending request fails. The list remains fully usable and the panel shows “No trending posts yet”.
A8	The visitor loses the connection. The client shows the generic error message and the visitor can retry by reloading the current URL.

Output Data

Data Name	Description
`posts[]`	Visible posts of the requested page with `id`, `categoryId`, `userId`, `title`, `slug`, `content`, `moderationStatus`, `createdAt`, `updatedAt`, `likeCount`, `commentCount` and the author identity.
`pagination`	`page`, `limit`, `total` and `totalPages` computed from the filtered count.
`posts[]` (top)	Up to three trending posts of the category with the like count, the comment count and the interaction score used for the ranking.

Data Name	Description
Empty collection	An empty array when the page, the filter or the search matches no approved post.

Post-Conditions

- The visitor sees one page of approved posts of the selected category, newest first.
- The search, the page size and the author filter are reflected in the URL and can be restored by reloading the page; the search itself narrows the posts already loaded for that page.
- No data is modified: the function is read-only.

3.4.3 View Post Detail

Function Trigger

The visitor selects a post row, follows a shared post link, or opens a related post from the side panel of another discussion.
Navigation path: Home → Forum → *Category* → *Post* (`/forum/{category}/{post-slug}-{post-id}`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function displays a complete discussion: the post with its formatted content and its author, the engagement counters, the list of members who liked it and the comment thread. It is the screen from which every write action on the discussion is started, and it shows the author-specific actions only to the author.

Interface

- Breadcrumb “Forum / *Category* / *Post title*” and the line “Discussion in *Category*”.
- Post title, author avatar, display name, creation date and the “Edited {date}” marker when the post was modified after its creation.
- Rich-text body rendered with headings, lists and links.
- Action bar with the heart button and its counter, the comment counter and, for a signed-in member who is not the author, the “Report” action.

- “Edit” and “Delete” actions placed next to the post title and rendered only for the author.
- “N likes” trigger below the action bar, disabled while the post has no like, which opens the likes dialog described by a later function.
- Comment section below the action bar, described by the comment functions.
- “Related Posts” panel in the right column with up to three other discussions of the same category and the “View all posts →” link.

Data Processing Overview

- The client extracts the post identifier from the slug in the URL and requests `GET /api/forums/posts/{id}`.
- The API resolves the post and returns the detail projection with the category summary and the author identity.
- Only a post that is not soft deleted and whose status is `APPROVED` can be resolved; a pending or rejected post answers with a not-found error.
- When a member is signed in, the client requests the likes of the post to compute the counter and to derive whether that member already liked it. A guest skips this request, therefore the counter stays at zero and the likes trigger is disabled.
- The client requests the first fifty comments of the post to render the thread and the comment counter.
- The client requests four posts of the category for the related panel, removes the post that is currently open and keeps at most three of them.
- The page compares `updatedAt` with `createdAt` to decide whether the “Edited” marker is displayed, and compares the author identifier with the session identifier to decide whether the edit and delete actions are rendered.

Screen Layout (Mock-up Description)

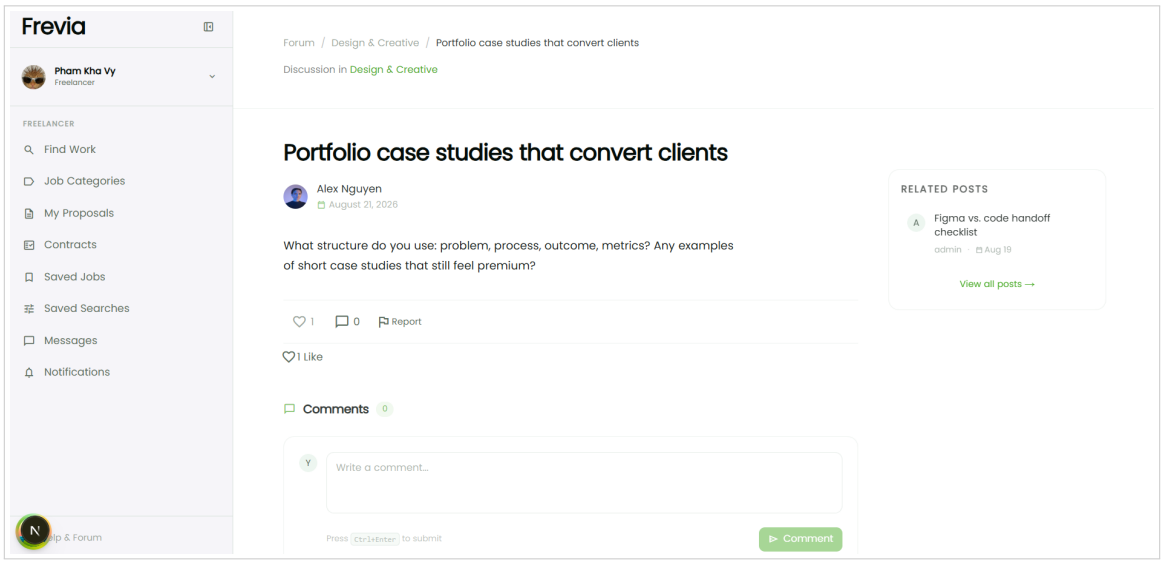

Figure 64: Post detail screen with the comment section

- The breadcrumb starts at “Forum”, continues with the category and ends with the post title, so the reader can walk back to the list at any time; the line “Discussion in *Category*” repeats the context.
- The title sits at the top of the article; the author’s “Edit” and “Delete” actions appear on the same line, at the right, and only when the reader is the author.
- The author block shows the avatar, the display name and the publication date; when the post was edited, the edition date is added next to the creation date.
- The body is rendered as formatted content, so headings, bullet lists and links written in the editor keep their appearance.
- The action bar holds the heart button with the like counter, the comment counter and the “Report” action for signed-in readers who are not the author; the heart is red and slightly enlarged once the member liked the post.
- The “N likes” trigger below the action bar opens the likes dialog and is disabled while the post has no like.
- The comment section below the action bar lists the thread and gives the signed-in member the comment editor.
- The right column proposes up to three related posts of the same category, each with its author and date, followed by the “View all posts →” link.

Function Details

Input Data

Data Name	Description
`slug`	Post slug taken from the URL; the client extracts the post identifier from its trailing segment.
`postId`	Identifier resolved from the slug and used by the detail, likes, comments and report calls of the same screen.
`page`, `limit`	Pagination of the comment thread; the detail screen requests the first page with a size of 50.
Session identity	Identifier of the signed-in member, used to decide whether the likes are loaded, to compute the liked state and to decide whether the author actions are displayed.

Validation Rules

- The identifier extracted from the slug must resolve to a post that is not soft deleted and whose `moderationStatus` is `APPROVED`; otherwise the API answers with `Error.ForumPostNotFound`.
- The detail projection must contain a title and a content, which are required fields of the post model.
- The comment pagination follows the same rules as the post list: integers greater than or equal to 1, default page 1 and default size 10; this screen explicitly asks for a size of 50.
- The liked state is derived only from the likes that belong to the post and match the session identifier. A guest always sees the unliked state with a zero counter because the likes query is not executed.
- The author actions are rendered only when the session identifier equals the post author identifier, never from a value supplied by the client.

Business Rules

- **BR-FORUM-10**: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed.
- **BR-FORUM-11**: Post details shall include title, content, author, category and created date.
- **BR-FORUM-09**: `commentCount` shall count only comments with `deletedAt = NULL`.
- **BR-FORUM-28**: The like detail shall include the user identifier and the profile of each member who liked the post.

Normal Flow

1. The visitor selects a post row in the category screen.
2. The client extracts the post identifier from the slug in the URL and requests the post detail.
3. The API resolves the post to a visible discussion and returns the detail projection.
4. The client renders the breadcrumb, the title, the author block and the formatted body.
5. When a member is signed in, the client requests the likes of the post; in every case it requests the first page of comments.
6. The API returns both collections with the profile of their authors.
7. The client computes the like counter, the liked state and the comment counter.
8. The client renders the action bar and exposes the author actions only to the author.
9. The client requests four posts of the category, removes the current post and renders at most three of them in the right panel.
10. The visitor reads the discussion and can like the post, open the likes list, write a comment, report the content or, as the author, edit or delete the post.

Abnormal Flow / Exception Handling

Case	Description
A1	The identifier does not resolve to any post. The client displays the not-found screen with the message “This post may have been deleted or the link is incorrect.” and the action “Back to category”.
A2	The post is still pending moderation or was rejected. The API hides it, therefore the same not-found screen is displayed and the post is never exposed before approval.
A3	The post was soft deleted by its author or by an administrator. The detail call answers with `Error.ForumPostNotFound` and the not-found screen is displayed.
A4	The likes request fails. The post body remains readable, the like counter falls back to zero and the likes trigger is disabled.
A5	The comments request fails. The thread area falls back to the empty state while the post body and the action bar remain usable.
A6	The session expires while the page is open. The BFF proxy refreshes the token once; when the refresh fails the visitor continues as a guest without author actions and without the likes list.

Case	Description
A7	The visitor follows a link of a deleted category. The category summary is missing, so the client falls back to the “Category” label instead of a broken layout.
A8	The connection drops while the page loads. The client shows the generic error message and the visitor can retry by reloading.

Output Data

Data Name	Description
Post detail	`id`, `categoryId`, `userId`, `title`, `slug`, `content`, `moderationStatus`, `moderationScore`, `createdAt` and `updatedAt`.
`category`	Identifier, name and slug of the category the post belongs to, or null when the post has no category.
`user`	Identifier and profile of the author, with the display name and the avatar.
`likes[]`	Members who liked the post, with their identifier, reaction type, creation date and profile. The collection is requested only for a signed-in member.
`comments[]`	First page of the thread with the content, the author, the dates, the like counter and the liked state of the current member.
`pagination`	Total number of visible comments and the pagination of the requested page.

Post-Conditions

- The visitor reads the complete discussion with its engagement counters.
- The action bar exposes only the actions allowed by the role and the ownership of the post.
- No data is modified by the read itself; the counters reflect only visible rows, and a guest sees a zero like counter because the likes list is not loaded.

3.4.4 Create Post

Function Trigger

A signed-in member selects the "New Post" button in the toolbar of a category screen, writes the post and confirms with the "Post" action. The button is rendered for Clients, Freelancers and Administrators; a guest who follows the button is routed to the sign-in page instead. **Navigation path:** Home → Forum → *Category* → New Post.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator uses the same dialog when browsing the forum.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, SensitiveAI moderation proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member start a new discussion inside a category. Because the content is user generated, every submission is screened by the moderation proxy before it reaches the forum, and the resulting post is stored with the moderation verdict so that only approved discussions become visible to the community.

Interface

- Modal dialog titled "Create Post in *Category*" with the description "Share your thoughts with the community."
- "Title" input with the placeholder "What's on your mind?".
- "Content" rich-text editor with the placeholder "Write your post content here…" and the usual formatting tools.
- "Cancel" action, which discards the draft, and "Post" action, which submits it and shows a spinner while the request is running.
- Toast notification after the submission, chosen from three moderation outcomes: a success message and a redirection to the new post when the content is approved, an information message when it is queued for review, or an error message when it is rejected.

Data Processing Overview

- The dialog trims the title and the content and blocks the submission while either of them is empty.
- The client posts the form to `/api/backend/forums/posts`.
- The BFF proxy recognises this exact route as the only forum route that must be screened

and forwards it to the SensitiveAI moderation proxy instead of the REST API.

- The moderation proxy scans the text, attaches the verdict (status, score and categories) to the request body and forwards the request to the REST API.
- The API validates the body, verifies that the category exists and is not soft deleted, and creates the post.
- When the request reaches the API without a verdict, the API calls its own `AIModerationService`: `ALLOW` maps to `APPROVED`, `REVIEW` to `PENDING` and `BLOCK` to `REJECTED`; a failure of that service is treated as `APPROVED` so publishing is not blocked by an outage.
- The slug is generated from the title, normalised to lower case without accents, and completed with a timestamp and a random suffix to keep it unique.
- The moderation status is taken from the verdict; when no verdict and no moderation result are available the post is stored as `PENDING`, which means it is invisible until an administrator approves it.
- The API returns the created post and the client invalidates the post lists of the category so the next visit shows the new discussion.
- The client branches on the returned status: `REJECTED` keeps the dialog open with the draft and shows an error toast; `PENDING` closes the dialog, clears the fields and shows an information toast without navigating; `APPROVED` closes the dialog, clears the fields, shows a success toast and navigates to the new post.

Screen Layout (Mock-up Description)

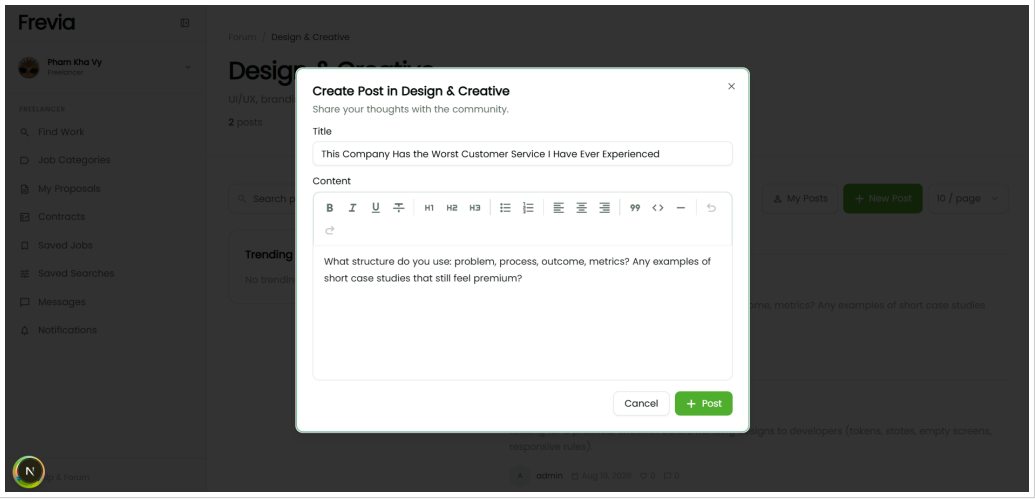

Figure 65: Create post dialog

- The dialog title names the category in which the post will be created, so the member cannot post in the wrong place by mistake.
- The title field is a single-line input; the content field is a rich-text editor that stores formatted HTML.

- The “Post” action stays disabled until both fields contain text, which prevents the most common validation error before the request is sent.
- The spinner replaces the icon of the action while the moderation proxy and the API are working, preventing a duplicate submission.
- The dialog closes and clears both fields when the post is approved or queued for review; it stays open with the draft when the content is rejected or the request fails, so the member can correct the text and retry.

Function Details

Input Data

Data Name	Description
`categoryId`	Identifier of the category in which the post is created; a positive integer sent by the dialog from the current category screen.
`title`	Post title, required, at most 255 characters in the database.
`content`	Post body, required; it is rich-text HTML produced by the editor.
`moderation`	Verdict attached by the moderation proxy: `status`, `score` between 0 and 1 and `categories`. It is an internal field, never typed by the member.
Access token	Bearer token forwarded by the BFF proxy; the API resolves the author from it, so the client cannot choose the author.

Validation Rules

- The title and the content are required and must contain at least one character after trimming.
- `categoryId` is optional for the API but must be a positive integer when it is present; the dialog always sends the category of the current screen.
- The category must exist and must not be soft deleted, otherwise the request fails with `Error.ForumCategoryNotFound`.
- The author is taken from the access token, never from the request body, so a member cannot create a post on behalf of somebody else.
- The moderation verdict must respect its own contract: a status among `PENDING`, `APPROVED` and `REJECTED`, and an optional score between 0 and 1.

- The generated slug must remain unique; the timestamp and random suffix guarantee that two posts with the same title never collide.

Business Rules

- **BR-FORUM-12**: Only authenticated users can create posts.
- **BR-FORUM-13**: Every post must belong to exactly one user and may belong to a category.
- **BR-FORUM-14**: Title and content are required fields.
- **BR-FORUM-15**: A new post shall default to `moderationStatus = PENDING` unless moderation data is provided.

Normal Flow

1. The member opens a category screen and selects “New Post”.
2. The client opens the dialog with the title of the target category.
3. The member writes a title and a content; the “Post” action becomes available.
4. The client trims both fields and sends the request through the BFF proxy.
5. The proxy routes the request to the moderation proxy, which scans the text.
6. The moderation proxy attaches its verdict and forwards the request to the REST API.
7. The API validates the body and checks that the category exists.
8. The API generates the slug, resolves the moderation status and inserts the post.
9. The API returns the created post to the client.
10. The client invalidates the cached lists of the category.
11. When the status is `APPROVED`, the client closes the dialog, clears the fields, shows the success message and navigates to the new post.
12. When the status is `PENDING`, the client closes the dialog, clears the fields and shows the message “Your post has been saved and is awaiting moderator review.” without navigating.
13. When the status is `REJECTED`, the client keeps the dialog open with the draft and shows the rejection message returned by the moderation service.

Abnormal Flow / Exception Handling

Case	Description
A1	The title or the content is empty after trimming. The action is disabled, and a forced request is rejected by the validation pipe with `Error.ForumPostTitleRequired` or `Error.ForumPostContentRequired`.

Case	Description
A2	The category was deleted while the dialog was open. The API answers `Error.ForumCategoryNotFound` and the dialog stays open with the draft so the member can choose another category.
A3	The member is not signed in or the session expired. The BFF proxy refreshes the token once; if the refresh fails the API rejects the request with an authentication error and the client redirects to the sign-in page.
A4	The moderation proxy is unavailable. The request fails before reaching the API, no post is created, and the client shows an error toast while keeping the draft.
A5	The moderation verdict is `REJECTED`. The post is stored with that status, is not publicly visible, and the dialog stays open with an error toast so the member can rewrite the content.
A6	The moderation verdict is `PENDING`. The post is stored but hidden, the dialog closes with an information toast, and the member is told that the post awaits moderator review.
A7	The insert fails at database level. The API returns `Error.FailedToCreateForumPost` and the client keeps the dialog open so nothing written by the member is lost.
A8	The member double clicks the action. The action is disabled while the mutation is pending, therefore only one request is sent.
A9	The member chooses “Cancel”. The dialog closes, the draft is cleared and no request is sent.

Output Data

Data Name	Description
`id`	Identifier of the created post, used by the client to build the detail route.
`slug`	Generated slug, unique for every post.
`categoryId`, `userId`	Category and author of the post.
`title`, `content`	Stored title and stored rich-text content.
`moderationStatus`	`APPROVED`, `PENDING` or `REJECTED`; it decides whether the post is immediately reachable.

Data Name	Description
`moderationScore`, `moderationCategories`	Verdict details kept for the moderation queue; the score may be null when the proxy did not compute one.
`createdAt`, `updatedAt`	Timestamps of the insert.

Post-Conditions

- One post is inserted and belongs to the signed-in member and to the selected category.
- The post carries a moderation status and a unique slug.
- An approved post becomes visible in the category list immediately after the cache is invalidated; a pending or rejected post stays hidden until an administrator resolves it.
- The post appears in the moderation queue of the administrator.

3.4.5 Edit Post

Function Trigger

The author of a post opens the post detail screen and selects the "Edit" action placed next to the title. The action is rendered only for the author, therefore other members never see it.
Navigation path: Home → Forum → *Category* → *Post* → Edit.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer or Client who authored the post.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function allows the author to correct the title or the body of a discussion without losing the discussion itself, its comments and its likes. The post keeps its identity, its creation date and its engagement, and the screen marks it as edited so readers know that the content changed after publication.

Interface

- Modal dialog titled "Edit Post" with the description "Update your post title and content."
- Title input and rich-text editor pre-filled with the current values of the post.

- “Cancel” action, which leaves the post unchanged, and “Save Changes” action, which submits the new values.
- The post detail screen refreshes the title and the body in place after a successful save, and the author line starts showing “Edited {date}”.
- Because the new text is screened again, the dialog may also close with a post whose moderation status became `PENDING` or `REJECTED`; the detail screen keeps the new values until the page is reloaded, when the post becomes hidden if it is no longer approved.

Data Processing Overview

- The client pre-fills the dialog from the post already loaded in the detail screen, so no additional read is required to open it.
- The client sends `PATCH /api/forums/posts/{id}` with the new title and content.
- The BFF proxy forwards the request directly to the REST API; only post creation goes through the SensitiveAI moderation proxy.
- The API loads the post with `deletedAt` null and rejects the request when the post does not exist.
- The API compares the author of the post with the identifier carried by the access token and rejects the request when they differ.
- The API re-screens the new title and content with `AIModerationService` when the request carries no verdict: `ALLOW` maps to `APPROVED`, `REVIEW` to `PENDING` and `BLOCK` to `REJECTED`; a failure of the moderation service is treated as `APPROVED`.
- The API updates the title, the content, the slug and the moderation fields, then returns the updated post.
- The client closes the dialog, replaces the title and the body in place and invalidates the cached detail and the post lists of the category so the new values appear everywhere without a reload.

Screen Layout (Mock-up Description)

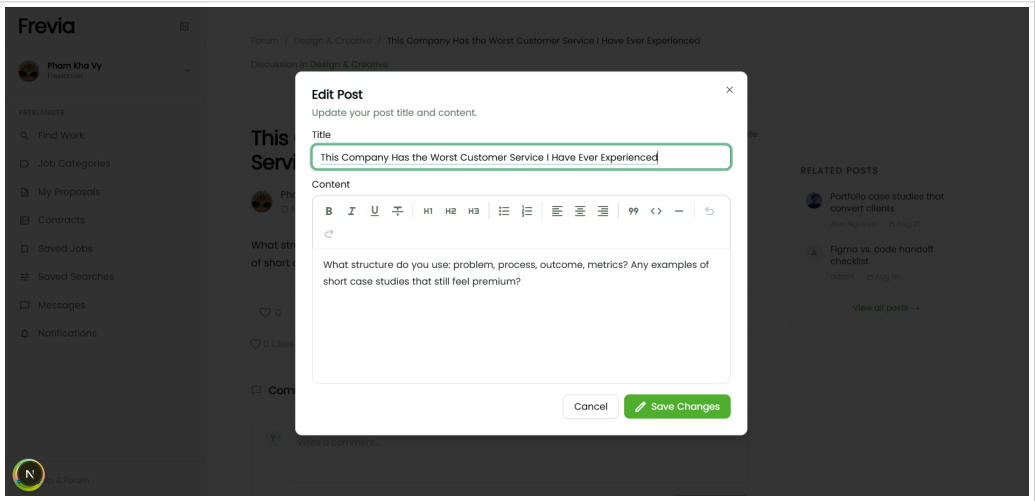

Figure 66: Edit post dialog with the current values

- The dialog reuses the layout of the creation dialog, which keeps the editing experience consistent with publishing.
- Both fields are pre-filled, so the member edits the existing text instead of rewriting it.
- The “Save Changes” action stays disabled while a field is empty, mirroring the requirement that both fields remain mandatory.
- A spinner is shown during the request; the dialog closes when the update succeeds.
- After the save, the detail screen shows the new title, the new body and the edition marker next to the publication date.
- The dialog does not display a moderation toast. When the new text is no longer approved, the post is hidden the next time the forum loads it, while the current screen keeps the saved values until it is refreshed.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post taken from the current route.
`title`	New title, required, at most 255 characters in the database.
`content`	New body, required; rich-text HTML produced by the editor.
Access token	Identifies the requester; the API compares its identifier with the author of the post.

Validation Rules

- The post must exist and must not be soft deleted; otherwise the request fails with `Error.ForumPostNotFound`.
- The requester must be the author of the post; otherwise the request fails with `Error.ForumPostNotOwned`. The check is performed on the server, so hiding the action in the interface is only a convenience.
- The title and the content are required; an empty value is rejected by the validation pipe before the update is executed.
- The slug is regenerated from the new title on every update, which keeps the URL readable and unique while the post identifier stays unchanged.
- The new text is moderated again by the API; the returned status may be `PENDING` or `REJECTED` even when the post was previously approved.
- The category of the post is preserved unless a new category identifier is supplied; the dialog never changes it.

Business Rules

- **BR-FORUM-16**: Only the author of a post can edit that post.
- **BR-FORUM-17**: Title and content are required fields.
- **BR-FORUM-18**: The original post identifier remains unchanged after editing; the slug is regenerated when the title changes.

Normal Flow

1. The author opens the detail of a post they wrote.
2. The client renders the “Edit” and “Delete” actions next to the title.
3. The author selects “Edit” and the dialog opens with the current title and content.
4. The author changes one or both fields.
5. The client sends the update request with the access token.

6. The API loads the post and confirms that it exists and is not deleted.
7. The API confirms that the requester is the author.
8. The API re-screens the new text and resolves the new moderation status.
9. The API updates the title, the content, the slug and the moderation fields.
10. The API returns the updated post.
11. The client closes the dialog, refreshes the detail in place and invalidates the category lists.

Abnormal Flow / Exception Handling

Case	Description
A1	The post no longer exists or was deleted in the meantime. The API returns `Error.ForumPostNotFound` and the client shows the error and closes the dialog.
A2	The requester is not the author. The API returns `Error.ForumPostNotOwned`, the update is refused and the client displays the returned message.
A3	The title or the content is cleared before saving. The action is disabled, and a forced request is rejected by the validation pipe.
A4	The update fails at database level. The API returns `Error.FailedToUpdateForumPost` and the dialog stays open with the edited values.
A5	The author cancels the dialog. No request is sent and the post keeps its previous values.
A6	The session expires while the dialog is open. The BFF proxy refreshes the token once; when the refresh fails the request is rejected and the member is redirected to sign in.
A7	Two editions are submitted in a row. The action is disabled while the mutation is pending, so the last saved values are the ones returned by the server.
A8	The new text is flagged by moderation. The update still succeeds, but the post receives `PENDING` or `REJECTED` and disappears from the public lists on the next load; the dialog closes without a moderation toast.

Output Data

Data Name	Description
`id`	Identifier of the post, which never changes.
`title`, `content`	New values stored in the post.
`slug`	Slug regenerated from the new title.
`moderationStatus`	New verdict, `APPROVED`, `PENDING` or `REJECTED`.
`moderationScore`, `moderationCategories`, `moderationMessage`	Details of the new verdict, kept for the moderation queue.
`updatedAt`	New edition timestamp, used by the detail screen to display the "Edited" marker.
`createdAt`	Original publication timestamp, preserved.

Post-Conditions

- The post contains the new title and the new content and keeps its author, category, comments and likes.
- The detail screen and the category list display the new values.
- The post is marked as edited because `updatedAt` differs from `createdAt`.
- If the new verdict is not `APPROVED`, the post leaves the public lists and waits in the moderation queue until an administrator resolves it.
- No other post and no comment is modified.

3.4.6 Delete Post

Function Trigger

The author of a post, or an Administrator, selects the "Delete" action on the post detail screen and confirms the operation in the confirmation dialog.
Navigation path: Home → Forum → *Category* → *Post* → Delete.

Function Description

Actors / Roles

- **Primary Actor:** the author of the post, or an Administrator acting on any post. The web interface renders the action for the author; the API also accepts an Administrator, who removes posts through the moderation and trash tools.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and

PostgreSQL through Prisma ORM.

Purpose

The function removes a discussion from the public forum while preserving its record. Deletion is soft: the row receives a deletion timestamp and is filtered out of every public query, which keeps the data available for moderation, for the trash screen of the administrator and for any later audit.

Interface

- "Delete" action with a bin icon, rendered for the author and for the Administrator.
- Confirmation dialog titled "Delete Post" with the warning "Are you sure you want to delete this post? This action cannot be undone.".
- "Cancel" action, which closes the dialog, and destructive "Delete" action, which performs the deletion.
- Redirect to the category screen after a successful deletion.

Data Processing Overview

- The client sends `DELETE /api/forums/posts/{id}` with the access token.
- The API loads the post with `deletedAt` null to confirm that it exists and is still visible.
- The API compares the author with the requester and also reads the role of the requester from the token.
- When the requester is neither the author nor an Administrator, the deletion is refused.
- When the check succeeds, the API writes the current date and time in the `deletedAt` column of the post.
- The client invalidates the post lists and navigates back to the category, where the post is no longer listed.

Screen Layout (Mock-up Description)

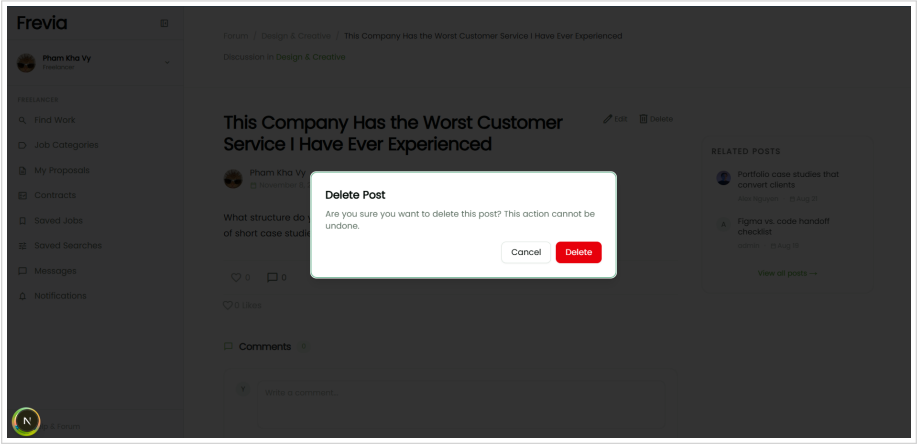

Figure 67: Delete post confirmation

- The confirmation dialog repeats the name of the object being deleted, which prevents deleting the wrong discussion.
- The warning states that the action cannot be undone from the member point of view, because the post disappears from the forum immediately.
- The destructive style of the confirmation action distinguishes it from the cancel action.
- After confirmation the screen returns to the category list, so the member sees the post removed from the discussion list.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post to delete, taken from the confirmation dialog.
Access token	Identifies the requester and carries the role used to authorize an Administrator.
`deletedAt`	Deletion timestamp written by the API; it is internal, not supplied by the client.

Validation Rules

- The post must exist and must not be already deleted; otherwise the API returns `Error.ForumPostNotFound`.
- The requester must be the author of the post or must hold the Administrator role; otherwise the API returns `Error.ForumPostNotOwned`.

- The deletion timestamp is produced by the server, so the client cannot backdate or falsify it.
- Only one row is affected per request; the comments and the likes of the post are preserved in the database and simply stop being reachable through the post.

Business Rules

- **BR-FORUM-19**: Only the author of a post or an Admin can delete that post.
- **BR-FORUM-20**: Post deletion shall be implemented as a soft delete using the `deletedAt` field.
- **BR-FORUM-21**: Soft-deleted posts shall not appear in public forum listings.

Normal Flow

1. The author or the Administrator opens the post detail screen.
2. The client renders the "Delete" action, which is available for this role.
3. The member selects the action and the confirmation dialog is displayed.
4. The member confirms the deletion.
5. The client sends the delete request with the access token.
6. The API loads the post and confirms that it exists and is not deleted.
7. The API confirms that the requester is the author or an Administrator.
8. The API writes the deletion timestamp in the post row.
9. The API returns the deleted post.
10. The client invalidates the lists, closes the dialog and routes back to the category.
11. The category list no longer contains the post and its counters are recomputed without it.

Abnormal Flow / Exception Handling

Case	Description
A1	The post does not exist or was already deleted. The API returns `Error.ForumPostNotFound` and the client shows the error message.
A2	A member tries to delete a post written by somebody else. The API returns `Error.ForumPostNotOwned` and nothing is deleted.
A3	An Administrator deletes the post of another member. The operation is allowed and the post leaves the public forum immediately.
A4	The database update fails. The API returns `Error.FailedToDeleteForumPost` and the post remains visible, so the member can retry.

Case	Description
A5	The member cancels the confirmation. No request is sent and the post is unchanged.
A6	The session expires before the confirmation. The request is rejected after the single token refresh attempt and the member is redirected to sign in.
A7	Another member opens the same post while it is being deleted. The next request returns `Error.ForumPostNotFound` and their screen shows the not-found state.

Output Data

Data Name	Description
`id`	Identifier of the deleted post.
`userId`, `categoryId`	Author and category of the deleted post, returned for confirmation.
`title`, `slug`	Title and slug of the deleted post.
`deletedAt`	Server timestamp recorded in the row; the post is soft deleted from this moment.

Post-Conditions

- The post is excluded from the category list, from the post detail and from the landing page counters.
- The row and its related comments and likes are retained for moderation and for the trash screen of the Administrator.
- The member is returned to the category screen, which reflects the new totals.

3.4.7 Like a Post

Function Trigger

A signed-in member selects the heart button in the action bar of a post. A guest who selects the button is routed to the sign-in page instead of producing a request.

Navigation path: Home → Forum → *Category* → *Post* → Like.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also react while browsing.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member express appreciation for a discussion and lets the community measure which posts are relevant. It is implemented as a toggle: the same action adds the reaction when the member has not liked the post yet and removes it when the member already liked it, so a member can never like the same post twice.

Interface

- Heart button with the like counter, placed in the action bar under the post body.
- Outline heart for a post that the member has not liked, filled heart for a post that the member liked.
- The button is disabled while the request is running, which prevents repeated toggles from a single intention.

Data Processing Overview

- The client sends `POST /api/forums/posts/{id}/like` to the BFF proxy, which forwards it to the REST API.
- The API loads the post with `deletedAt` null and answers with `Error.ForumPostNotFound` when it is missing.
- The API looks for a like row that matches the member and the post.
- When the row exists, the API removes the reaction and answers with `liked: false`.
- When the row does not exist, the API inserts a like row with the reaction type `like` and answers with `liked: true`.
- Before the request is sent, the client updates its cached likes list optimistically: it adds or removes the current member, so the heart and the counter react immediately.
- When the request fails, the client restores the previous likes list; when it settles, it invalidates the likes query and the post detail so the cache is reconciled with the server.
- The unique constraint of the like table on the pair member and post guarantees that a duplicate reaction cannot be stored, even if two requests are issued at the same time.

Screen Layout (Mock-up Description)

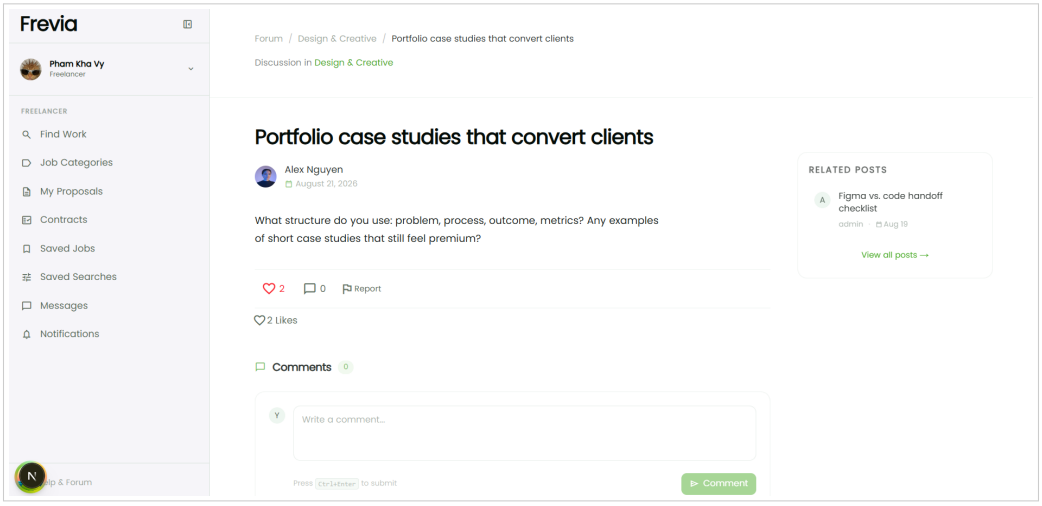

Figure 68: Post action bar after the post has been liked

- Before the reaction the heart is outlined and the counter shows the current number of likes.
- After the reaction the heart is filled, coloured in red and slightly enlarged, and the counter is incremented by one.
- The “N likes” trigger placed below the action bar shows the same total and opens the list of members who liked the post.
- The counter is the single source of truth for the likes list: the same number is shown on the detail screen, in the likes dialog and on the post rows of the category.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post taken from the current route.
Access token	Identifies the member who reacts; the API derives the like owner from it.
Reaction type	Constant value `like` stored in the reactivity row; the interface does not offer another reaction for the moment.

Validation Rules

- The requester must be authenticated; a request without a valid token is rejected with an authentication error.
- The post must exist and must not be soft deleted.

- A member can hold only one reaction per post, enforced by the unique constraint on the pair member and post.
- The reaction is always attached to the member of the token; the client cannot send a different member identifier.

Business Rules

- **BR-FORUM-22**: Only authenticated users can like posts.
- **BR-FORUM-23**: A user can have only one active like reaction per post.
- **BR-FORUM-24**: The reaction shall be stored in the `ForumLike` table.
- **BR-FORUM-25 to BR-FORUM-27**: removing a reaction is allowed only when the corresponding like record exists, and it deletes that record.

Normal Flow

1. The signed-in member opens a post they have not liked yet.
2. The client displays an outlined heart next to the current counter.
3. The member selects the heart and the client updates the cached likes list immediately.
4. The client sends the toggle request.
5. The API confirms that the post is visible.
6. The API finds no reaction of this member on this post.
7. The API inserts the like row.
8. The API answers with `liked: true`.
9. The client keeps the red filled heart and the incremented counter, then refetches the likes to reconcile the list with the server.
10. When the member selects the heart again, the same sequence removes the row and the interface returns to the outlined heart with the previous counter.

Abnormal Flow / Exception Handling

Case	Description
A1	The post does not exist or was deleted. The API returns `Error.ForumPostNotFound` and the counter is left unchanged.
A2	The member is not signed in. The client routes to the sign-in page before any request is sent, and a direct request is rejected with an authentication error.
A3	The member already liked the post. The request removes the reaction instead of adding a second one, which is the intended toggle behaviour.

Case	Description
A4	The insert fails at database level. The API returns `Error.FailedToLikeForumPost` and the interface keeps the previous state.
A5	The member double clicks the heart. The button is disabled while the request runs, so only one reaction is created.
A6	The likes query cannot be refreshed after a successful toggle. The optimistic state stays in the cache and the counter is reconciled on the next visit.

Output Data

Data Name	Description
`liked`	`true` when the reaction was added, `false` when it was removed.
Like row	Member, post, reaction type and creation date of the stored reaction.
Updated counter	The like counter of the post, recomputed by the client from the refreshed likes list.

Post-Conditions

- The post holds exactly one reaction of the member, or none when the action removed it.
- The counter displayed on the post matches the number of likes stored for that post.
- The member appears in, or disappears from, the likes list of the post.

3.4.8 View Post Likes

Function Trigger

A visitor selects the "N Likes" trigger placed below the action bar of a post, which opens the list of the members who liked the post. The trigger is disabled while the counter is zero, so the dialog is only reachable once the post has at least one like. The endpoint itself is public, but a guest sees a zero counter because the detail screen derives the likes only for a signed-in member, so guests normally do not reach the dialog.

Navigation path: Home → Forum → *Category* → *Post* → Likes.

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function answers the question “who liked this post”. It also feeds the state of the like button, because the detail screen derives both the counter and the liked state of the current member from the same list.

Interface

- Dialog titled “Likes” with the description “People who liked this post.”.
- Row per member with the avatar, the display name and, when the profile is incomplete, a fallback name built from the member identifier.
- Close icon of the dialog, which dismisses it; the list itself has no footer action.
- Empty message “No likes yet.” when the dialog is opened while no reaction is loaded; the trigger itself is disabled at zero, so this state is normally not reachable.

Data Processing Overview

- The client calls `GET /api/forums/posts/{id}/like` and caches the result under a key that contains the post identifier.
- The API reads every like row of the post and joins the profile of the member.
- The payload contains the identifier, the member, the reaction type and the creation date of the reaction.
- The detail screen uses the same payload to count the likes and to detect the reaction of the signed-in member by comparing identifiers; it requests the payload only for a signed-in member, while the dialog reuses the cached collection.
- The dialog renders one row per member with the avatar and the display name, and falls back to “User #{id}” when the profile is missing.

Screen Layout (Mock-up Description)

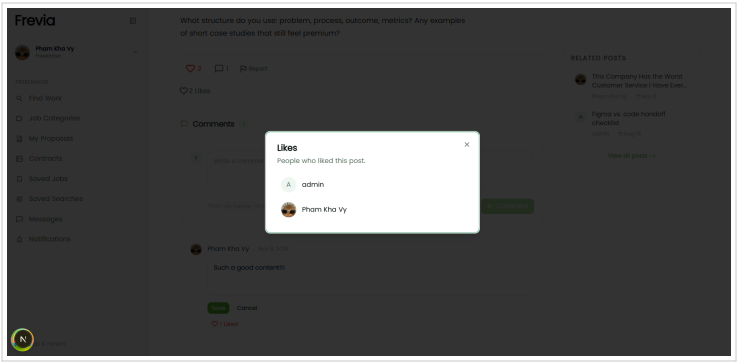

Figure 69: Likes dialog listing the members who liked the post

- The “N Likes” trigger placed below the post action bar opens the dialog and states how many members liked the discussion; it is disabled while the total is zero.
- Each row shows the avatar and the display name of a member, which links the reaction to a visible identity of the community.
- When no reaction is loaded, the dialog states “No likes yet.” instead of showing an empty panel.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post whose reactions are listed.
Session identity	Optional; the detail screen uses it to derive the liked state and the like counter, while the endpoint itself needs no token.

Validation Rules

- The post identifier must be a positive integer accepted by the route parameter pipe.
- The endpoint is public, so no token is required and no authorization rule applies.
- A post without reaction returns an empty array, which is a valid answer and not an error.
- Profile fields may be null; the interface then substitutes the fallback name and the initial of the display name for the avatar.

Business Rules

- **BR-FORUM-28**: The like detail shall include the user identifier and the user profile information for each liker.

- **BR-FORUM-23**: A user can have only one active like reaction per post, therefore a member never appears twice in the list.

Normal Flow

1. The visitor opens a post detail screen.
2. For a signed-in member, the client requests the likes of the post to derive the counter and the liked state.
3. The API returns every reaction of the post with the profile of the member.
4. The client shows the “N Likes” trigger below the post action bar, next to the like and comment actions.
5. The visitor selects the trigger and the dialog opens with the list of members.
6. The visitor closes the dialog and returns to the discussion.

Abnormal Flow / Exception Handling

Case	Description
A1	The post has no reaction. The trigger shows “0 Likes” and is disabled, so the dialog cannot be opened; a dialog opened from a stale cache states “No likes yet.”.
A2	A member has no profile record. The row shows the fallback name “User #{id}” and the initial letter in place of the avatar.
A3	The request fails. The dialog has no dedicated error panel and falls back to “No likes yet.”, while the trigger keeps the last counter derived from the detail screen.
A4	The post was deleted while the dialog was open. The refreshed list returns no row and the dialog shows the empty state.
A5	The visitor is offline. The dialog shows the same empty state and can be closed, which leaves the rest of the screen usable.

Output Data

Data Name	Description
`likes[]`	One entry per reaction with `id`, `userId`, `postId`, `commentId`, `reactionType` and `createdAt`.
`user`	Identifier, display name and avatar of the member who reacted.
Derived count	Length of the list, displayed as the likes counter of the post.

Post-Conditions

- The visitor sees every member who liked the post, without duplicates.
- The "N Likes" trigger of the post matches the number of rows returned.
- No data is modified: the function is read-only.

3.4.9 Create Comment

Function Trigger

A signed-in member writes in the comment box placed under the post body and submits it with the "Comment" action or with the `Ctrl+Enter` shortcut. The text is screened by the AI moderation service before it becomes visible, so the comment is either published, held for review or rejected. A guest sees an invitation to sign in instead of the editor.
Navigation path: Home → Forum → *Category* → *Post* → Comments.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also comment while browsing the forum.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member answer a discussion, ask a question or share an experience inside the thread of a post. Comments are attached to one post and one author, can be edited and deleted by their owner, and carry their own like counter.

Interface

- Comment card between the post action bar and the thread, with the avatar of the member and a text area using the placeholder "Write a comment…".

- Hint “Press `Ctrl+Enter` to submit” below the text area.
- “Comment” action with a send icon, disabled while the field is empty or a request is running.
- Guest state: the editor is replaced by the sentence “Log in to join the discussion.”, where “Log in” links to the sign-in page.
- Moderation feedback through a toast: an error toast keeps the draft when the text is rejected, an informational toast confirms that the comment awaits review, and a success toast may confirm an approved comment.
- Empty state “No comments yet” with the sentence “Be the first to share a tip, question, or experience.” when the post has no approved comment.

Data Processing Overview

- The client trims the text and sends `POST /api/forums/posts/{postId}/comments` with the content.
- The BFF proxy forwards the request to the REST API, which validates the body.
- The API verifies that the post exists and is not soft deleted.
- The API screens the content with the shared AI moderation service, using the `forum-comment` context: a `BLOCK` verdict becomes `REJECTED`, a `REVIEW` verdict becomes `PENDING` and an `ALLOW` verdict becomes `APPROVED`. When the moderation service is unavailable, the content is approved as a fallback.
- The API inserts the comment with the author taken from the access token, the moderation status, score, categories, message and language.
- The response contains the new comment with its author profile, a like counter of zero and an unliked state, because a comment cannot be liked before it exists.
- On `REJECTED` the client keeps the draft and shows an error toast. On `PENDING` it clears the editor and shows an informational toast. On `APPROVED` it clears the editor and may show a success toast.
- In every outcome the client invalidates the comment queries of the post, which refreshes the thread and the comment counter. Only `APPROVED` comments are returned by the public listing, so a pending or rejected comment stays out of the thread.

Screen Layout (Mock-up Description)

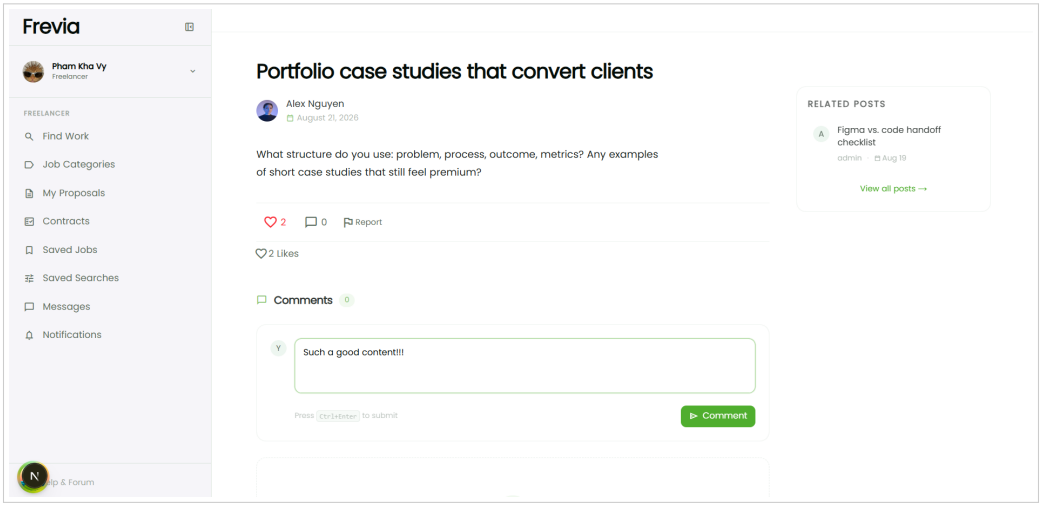

Figure 70: Comment editor under the post body

- The editor sits directly under the action bar, so the member writes without leaving the discussion; the text area shows the placeholder “Write a comment…”.
- The counter badge next to the “Comments” heading shows how many approved comments the post currently holds.
- The “Comment” action stays disabled while the field is empty, which prevents empty submissions, and the hint about `Ctrl+Enter` makes the shortcut discoverable.
- Every approved comment appears at the top of the thread, with the avatar, the display name, the date, the like action and, for the author, the edit and delete actions.
- A rejected draft remains in the editor with an error toast, while a pending comment leaves the editor empty and is not listed until a moderator approves it.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post taken from the route; the comment is attached to it.
`content`	Comment text, required, at least one character after trimming.
Access token	Identifies the author of the comment; it is read by the API and never sent in the body.

Validation Rules

- The requester must be authenticated; the API rejects a request without a valid token.
- The content must not be empty; the validation pipe rejects an empty or whitespace-only body.
- The post must exist and must not be soft deleted, otherwise the request fails with `Error.ForumPostNotFound`.
- The author of the comment is always the member of the token, so a member cannot post a comment on behalf of somebody else.
- The content is screened by the AI moderation service before insert; the resulting status decides whether the comment is published, held for review or rejected.

Business Rules

- **BR-FORUM-32**: Only authenticated users can create comments.
- **BR-FORUM-33**: Comment content cannot be empty.
- **BR-FORUM-34**: Each comment shall be associated with a valid post and user.

Normal Flow

1. The signed-in member opens a post and scrolls to the comment section.
2. The client renders the editor with the placeholder and the hint.
3. The member writes the comment and submits it with the action or the shortcut.
4. The client trims the text and sends the request.
5. The API validates the content and loads the post.
6. The API screens the content with the AI moderation service.
7. The API inserts the comment with the author of the session and the moderation verdict.
8. The API returns the created comment with an empty like state and its moderation status.
9. The client handles the verdict: it keeps the draft on `REJECTED`, clears the editor on `PENDING` or `APPROVED`, and shows the matching toast.
10. The client refreshes the thread; an approved comment is displayed at the top of the list and the counter is incremented, while a pending comment stays hidden until approval.

Abnormal Flow / Exception Handling

Case	Description
A1	The content is empty. The action is disabled, and a forced request is rejected by the validation pipe without touching the database.

Case	Description
A2	The post does not exist or was deleted. The API returns `Error.ForumPostNotFound` and the client displays the error message while keeping the draft.
A3	The member is not signed in. The editor is replaced by the sign-in invitation, and a direct request is rejected with an authentication error.
A4	The insert fails at database level. The API returns `Error.FailedToCreateForumComment` and the editor keeps the text so it is not lost.
A5	The member submits twice in a row. The action is disabled while the request runs, which prevents duplicated comments.
A6	The session expires during the submission. The BFF proxy attempts one refresh; when it fails the request is rejected and the member is redirected to sign in.
A7	The AI moderation returns `BLOCK`. The comment is stored with status `REJECTED`, is excluded from the public thread, and the editor keeps the draft with an error toast.
A8	The AI moderation returns `REVIEW`. The comment is stored with status `PENDING`, is hidden until a moderator approves it, and the client clears the editor with an informational toast.
A9	The AI moderation service is unavailable. The API falls back to `APPROVED`, so the comment is published and the client behaves as for a normal approval.

Output Data

Data Name	Description
`id`	Identifier of the created comment.
`postId`	Post the comment belongs to.
`content`	Stored comment text.
`user`	Identifier, display name and avatar of the author.
`likeCount`, `likedByMe`	Zero and false for a new comment.
`moderationStatus`	`APPROVED`, `PENDING` or `REJECTED` verdict of the AI moderation.

Data Name	Description
`moderationScore`, `moderationCategories`	Risk score and categories reported by the moderation service.
`moderationMessage`, `moderationLanguage`	Explanation shown to the author and the detected language, when available.
`createdAt`, `updatedAt`	Timestamps of the insert.

Post-Conditions

- One comment is stored and belongs to the signed-in member and to the post.
- An approved comment is immediately visible to every visitor and increments the comment counter; a pending comment waits for moderation and a rejected comment stays out of the public thread.
- The author is informed of the moderation outcome through the matching toast, and the draft is preserved only when the text is rejected.

3.4.10 Update Comment

Function Trigger

The author of a comment selects the pencil action next to their comment, edits the text and confirms the change. The new text is re-screened by the AI moderation service, so an edit can change the visibility of the comment. The action is rendered only for the author of the comment.

Navigation path: Home → Forum → *Category* → *Post* → Comments → Edit.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer or Client who authored the comment.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member correct a comment after publication, for example to fix a typo or to add a clarification, without deleting the reply and losing its position in the thread. The comment keeps its identifier, its author and its likes, and the new text replaces the previous one.

Interface

- Pencil action on the footer row of the comment, next to the like and delete actions.
- Inline editor pre-filled with the current text of the comment.
- “Save” action to confirm and “Cancel” action to discard the change.
- An “edited” marker next to the date as soon as the stored text differs from the original, because the update timestamp no longer matches the creation timestamp.
- Moderation feedback through a toast: a rejected edit keeps the editor open with an error, a pending edit closes it with an informational message, and an approved edit may close it with a success message.

Data Processing Overview

- The client sends `PATCH /api/forums/posts/{postId}/comments/{commentId}` with the new content.
- The API loads the comment with `deletedAt` null and rejects the request when it does not exist.
- The API verifies that the comment belongs to the post of the route, so a comment cannot be edited through the route of another post.
- The API compares the author of the comment with the member of the token and refuses the request when they differ.
- The API screens the new content with the AI moderation service, using the `forum-comment` context, and maps the verdict to `APPROVED`, `PENDING` or `REJECTED`.
- The API updates the content and the moderation status, score, categories, message and language, then returns the comment with its author profile.
- On `REJECTED` the client keeps the editor open with the text and shows an error toast; on `PENDING` or `APPROVED` it exits the edit mode, shows the matching toast and refreshes the thread.
- The client invalidates the comment queries of the post, and the comment is displayed with the “edited” marker when the update changed its timestamp.

Screen Layout (Mock-up Description)

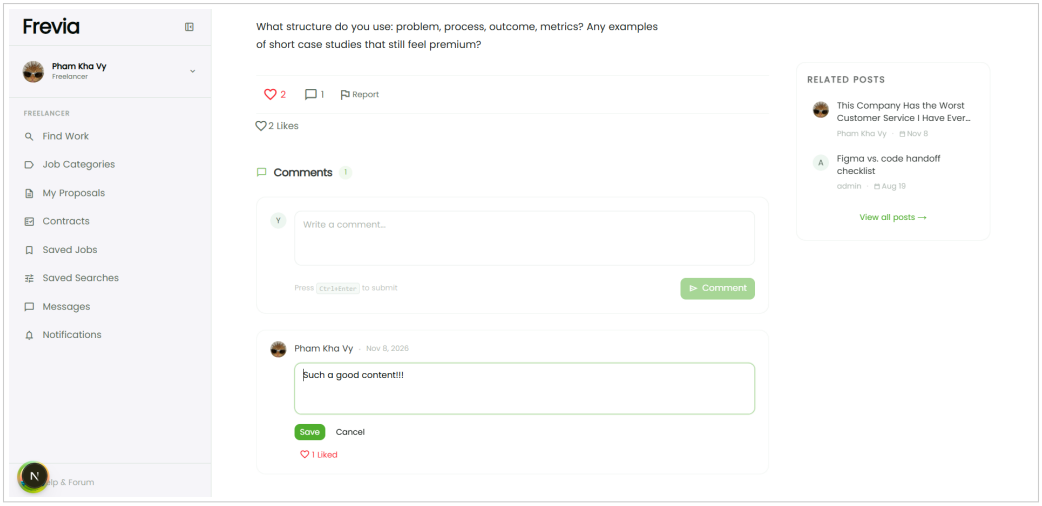

Figure 71: Inline editor of an existing comment

- The editor replaces the comment body in place, so the member keeps the context of the thread while editing; the avatar, name and date stay visible above the field.
- The existing text is pre-filled, which makes small corrections fast.
- The save action is disabled while the text is empty, because the content stays a required field, and the cancel action restores the previous text.
- After the save the comment shows the edited text and the “edited” marker next to the date, while the thread keeps the same order.
- The like action and its counter remain available below the editor, because the reaction belongs to the comment and not to its text.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to update.
`content`	New comment text, required, at least one character.
Access token	Identifies the requester; the API compares it with the comment author.

Validation Rules

- The comment must exist and must not be soft deleted; otherwise the request fails with `Error.ForumCommentNotFound`.

- The comment must belong to the post of the route; a mismatch is reported as a missing comment, which hides the existence of the comment through another route.
- The requester must be the author of the comment; otherwise the request fails with `Error.ForumCommentNotOwner`.
- The content is required and must not be empty after trimming.
- The new content is screened again by the AI moderation service, so the status, score, categories and message of the comment may change after an edit.
- The update replaces the content and its moderation verdict: the author, the post, the creation date and the likes of the comment are preserved.

Business Rules

- **BR-FORUM-35**: Only the owner of a comment can edit it.
- **BR-FORUM-36**: Comment content cannot be empty.
- **BR-FORUM-37**: A comment must belong to the specified post to be edited.

Normal Flow

1. The author of a comment selects the pencil action on their own comment.
2. The client opens the inline editor with the current text.
3. The author modifies the text and confirms.
4. The client sends the update request with the access token.
5. The API loads the comment and confirms that it exists.
6. The API confirms that the comment belongs to the post of the route.
7. The API confirms that the requester is the author.
8. The API screens the new text with the AI moderation service.
9. The API stores the new content and its moderation verdict.
10. The API returns the updated comment.
11. The client handles the verdict, refreshes the thread and displays the new text with the "edited" marker when the comment remains public.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was deleted. The API returns `Error.ForumCommentNotFound`, the editor keeps the text and an error toast is shown.

Case	Description
A2	The comment belongs to another post. The request is refused with the same not-found error, which prevents editing a comment through a foreign route.
A3	The requester is not the author. The API returns `Error.ForumCommentNotOwner`, the text is not changed and an error toast is shown.
A4	The new text is empty. The save action is disabled, and a forced request is rejected by the validation pipe.
A5	The update fails at database level. The API returns `Error.FailedToUpdateForumComment` and the editor keeps the text.
A6	The author cancels the edition. No request is sent and the previous text is restored.
A7	The AI moderation returns `BLOCK`. The comment is stored with status `REJECTED`, is removed from the public thread, and the editor stays open with an error toast.
A8	The AI moderation returns `REVIEW`. The comment is stored with status `PENDING`, is hidden until approval, and the client exits the edit mode with an informational toast.
A9	The AI moderation service is unavailable. The API falls back to `APPROVED` and the edit is published normally.

Output Data

Data Name	Description
`id`	Identifier of the updated comment.
`postId`	Post the comment belongs to, unchanged.
`content`	New comment text.
`user`	Identifier and profile of the author, unchanged.
`moderationStatus`	New verdict of the AI moderation after the edit.
`moderationScore`, `moderationCategories`	Risk score and categories of the edited text.
`moderationMessage`, `moderationLanguage`	Explanation and detected language, when available.

Data Name	Description
`updatedAt`	New edition timestamp.
`createdAt`	Original creation timestamp, preserved.

Post-Conditions

- The comment holds the new text and the verdict of the re-screening; its position, author and likes are preserved.
- An approved comment is displayed with the new text and the "edited" marker, while a pending or rejected comment is hidden from the public thread.
- No other comment and no post is modified.

3.4.11 Delete Comment

Function Trigger

The author of a comment, or an Administrator, selects the bin action next to a comment and confirms the deletion in the confirmation dialog.
Navigation path: Home → Forum → *Category* → *Post* → Comments → Delete.

Function Description

Actors / Roles

- **Primary Actor:** the author of the comment, or an Administrator acting on any comment. The web interface exposes the action to the author of the comment; the API also accepts an Administrator, who deletes comments through the moderation tools.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function removes a reply from the thread. As for posts, the deletion is soft: the comment receives a deletion timestamp and is filtered out of every public query, so the record remains available for moderation and for the trash screen of the Administrator while the discussion stays coherent for the readers.

Interface

- Bin action on the footer row of the comment, next to the edit action.
- Confirmation dialog titled "Delete Comment" with the warning "Are you sure you want to delete this comment? This action cannot be undone.".

- “Cancel” action and destructive “Delete” action.
- The thread is refreshed after the deletion and the comment counter decreases.

Data Processing Overview

- The client sends `DELETE /api/forums/posts/{postId}/comments/{commentId}`.
- The API loads the comment with `deletedAt` null and rejects the request when it is missing.
- The API verifies that the comment belongs to the post of the route.
- The API accepts the requester when it is the author or when the token carries the Administrator role.
- The API writes the deletion timestamp in the comment row.
- The client invalidates the comment queries and refreshes the thread and the counter.

Screen Layout (Mock-up Description)

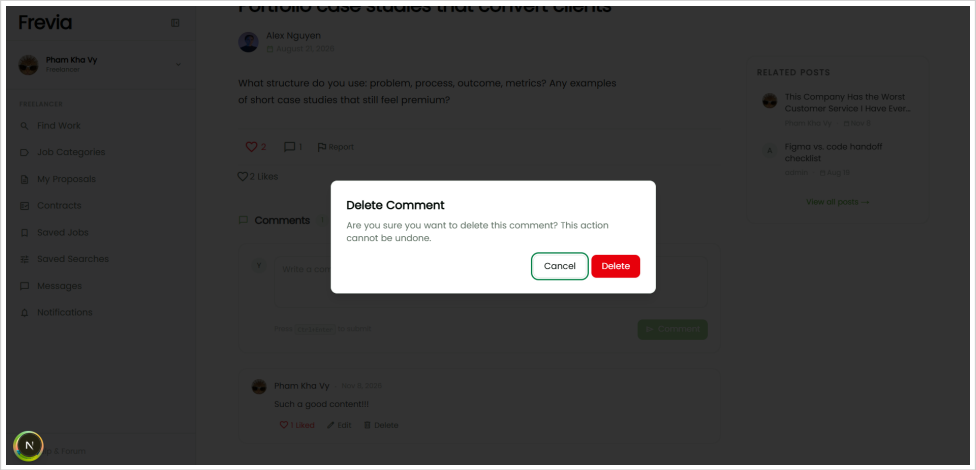

Figure 72: Delete comment confirmation

- The confirmation dialog names the object, so a member who owns several comments cannot delete the wrong one.
- The destructive style of the confirm action separates it from the cancel action and warns about the consequence.
- After the confirmation the comment disappears from the thread and the comment counter of the post is updated.
- The remaining comments keep their order, because the thread is sorted by creation date.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to delete.
Access token	Identifies the requester and carries the role used to authorize an Administrator.
`deletedAt`	Deletion timestamp written by the server, not supplied by the client.

Validation Rules

- The comment must exist and must not be already deleted; otherwise the request fails with `Error.ForumCommentNotFound`.
- The comment must belong to the post of the route; a mismatch is reported as a missing comment.
- The requester must be the author of the comment or must hold the Administrator role; otherwise the request fails with `Error.ForumCommentNotOwner`.
- The deletion timestamp is produced by the server.
- The post itself is not deleted, and the likes of the comment are kept in the database.

Business Rules

- **BR-FORUM-38**: Only the owner of a comment or an Admin can delete it.
- **BR-FORUM-39**: Comment deletion shall be implemented as a soft delete using the `deletedAt` field.
- **BR-FORUM-40**: Soft-deleted comments shall not be displayed in normal comment listings.

Normal Flow

1. The author or the Administrator opens the post detail screen.
2. The client renders the bin action on the comments the member may delete.
3. The member selects the action and the confirmation dialog appears.
4. The member confirms the deletion.
5. The client sends the delete request with the access token.
6. The API loads the comment and confirms that it exists.

7. The API confirms that the comment belongs to the post of the route.
8. The API confirms that the requester is the author or an Administrator.
9. The API writes the deletion timestamp.
10. The API returns the deleted comment.
11. The client refreshes the thread, where the comment is no longer present.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was already deleted. The API returns `Error.ForumCommentNotFound` and the client shows the error message.
A2	A member tries to delete a comment written by somebody else. The API returns `Error.ForumCommentNotOwner` and nothing is deleted.
A3	An Administrator deletes a comment of another member. The operation is allowed and the comment disappears from the thread.
A4	The comment belongs to another post. The request is refused with the not-found error, which prevents deleting through a foreign route.
A5	The delete fails at database level. The API returns `Error.FailedToDeleteForumComment` and the comment remains visible.
A6	The member cancels the confirmation. No request is sent and the comment is unchanged.
A7	The session expires before the confirmation. The single refresh attempt decides between a successful retry and a redirect to the sign-in page.

Output Data

Data Name	Description
`id`	Identifier of the deleted comment.
`postId`	Post the comment belonged to.
`content`	Text of the deleted comment, returned for confirmation.
`deletedAt`	Server timestamp recorded in the row.
Updated counter	The comment counter of the post, recomputed without the deleted comment.

Post-Conditions

- The comment is excluded from the thread and from every comment count.
- The row is retained for moderation and for the trash screen of the Administrator.
- The post remains published with its remaining comments.

3.4.12 Like a Comment

Function Trigger

A signed-in member selects the heart action on a comment of the thread. A guest does not see the action at all, because the comment footer renders it only for a signed-in member and the reaction requires an account.
Navigation path: Home → Forum → *Category* → *Post* → Comments → Like.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also react while browsing.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function allows members to acknowledge a useful reply without writing another comment. It is a toggle, exactly like the post reaction: the first activation likes the comment and the second one removes the reaction, so one member can never like the same comment twice.

Interface

- Heart action in the footer of each comment, followed by the like counter when it is greater than zero and by the label “Like” or “Liked”.
- Outlined heart when the member has not liked the comment, filled and slightly enlarged red heart when the member liked it.
- The action is disabled while the request runs.
- The same footer holds the edit and delete actions for the author, and the report action for the other members, so a comment can be managed without leaving the thread.

Data Processing Overview

- The client sends `POST /api/forums/posts/{postId}/comments/{commentId}/like`.
- The API loads the comment with `deletedAt` null and rejects the request when it is missing.
- The API verifies that the comment belongs to the post of the route.

- The API looks for a reaction of the member on the comment.
- When the reaction exists, the API removes it and answers with `liked: false`.
- When the reaction does not exist, the API inserts it and answers with `liked: true`.
- Before the response arrives, the client updates the cached comment optimistically: it toggles the liked state and increments or decrements the counter of that comment only.
- When the request fails, the client restores the previous cache. When it settles, the client invalidates the comment queries of the post so the server becomes the source of truth again.
- The comment list computes the liked state per comment by matching the reactions of the signed-in member, so the states of the other comments are unaffected.

Screen Layout (Mock-up Description)

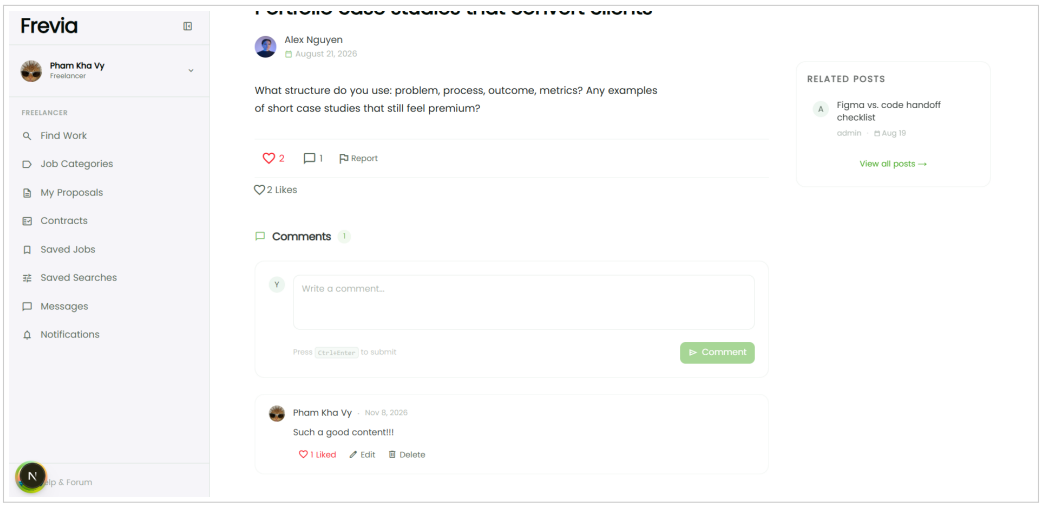

Figure 73: Comment liked by the current member

- The footer of the comment shows the heart, the counter and the label “Liked” once the member has reacted.
- After the reaction the heart is filled, slightly enlarged and coloured red, and the counter is incremented by one.
- The same footer offers the edit and delete actions to the author of the comment, while the report action is reserved to the other members.
- Only the selected comment changes; the reactions of the other comments and the likes of the post remain untouched.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to react to.
Access token	Identifies the member who reacts.
Reaction type	Constant value `like`.

Validation Rules

- The requester must be authenticated.
- The comment must exist, must not be soft deleted and must belong to the post of the route.
- A member can hold only one reaction per comment, enforced by the unique constraint of the like table on the pair member and comment.
- The reaction is attached to the member of the token, never to an identifier sent by the client.

Business Rules

- **BR-FORUM-41**: Only authenticated users can like comments.
- **BR-FORUM-42**: A user can have only one active like reaction per comment.
- **BR-FORUM-43**: The reaction shall be stored in the `ForumLike` table.
- **BR-FORUM-44 to BR-FORUM-46**: removing a reaction is allowed only when the record exists, and it deletes that record.

Normal Flow

1. The signed-in member finds a useful comment in the thread.
2. The client renders an outlined heart with the like counter when it is greater than zero.
3. The member selects the heart and the client sends the toggle request.
4. The API confirms that the comment exists and belongs to the post.
5. The API finds no reaction of this member on this comment.
6. The API inserts the reaction row.
7. The API answers with `liked: true`.
8. The client immediately fills the heart and increments the counter of that comment in the cache, then refetches the thread in the background.
9. Selecting the heart again removes the reaction and restores the previous counter.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was deleted. The API returns `Error.ForumCommentNotFound` and the counter is unchanged.
A2	The comment belongs to another post. The request is refused with the same not-found error.
A3	The member is not signed in. The heart action is not rendered for guests and no request can be sent from the interface.
A4	The reaction already exists. The request removes it, which is the intended toggle behaviour.
A5	The insert fails at database level. The API returns `Error.FailedToLikeForumComment`, the optimistic update is rolled back and the interface keeps the previous state.
A6	The member clicks the heart twice quickly. The action is disabled while the request runs, so only one reaction is created.

Output Data

Data Name	Description
`liked`	`true` when the reaction was added, `false` when it was removed.
Like row	Member, post, comment, reaction type and creation date of the stored reaction.
Updated counter	The like counter of the comment, recomputed by the client.

Post-Conditions

- The comment holds exactly one reaction of the member, or none after the second activation.
- The counter of the comment matches the number of stored reactions on that comment.
- The reactions of the post and of the other comments are unchanged.

3.4.13 Report Content

Function Trigger

A signed-in member who is not the author selects the “Report” action in the post action bar or in the footer of a comment, writes a reason and submits the report. The action is hidden from the author of the content, because a member cannot report their own post or comment, and it is not rendered for guests.
Navigation path: Home → Forum → *Category* → *Post* → Report.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM; the report is later handled by an Administrator in the moderation screen.

Purpose

The function gives the community a way to flag content that violates the rules, either a discussion or a single reply. A report never changes the visibility of the content by itself: it creates a pending moderation entry that the Administrator reviews through the moderation module.

Interface

- “Report” action with a flag icon in the action bar of the post and in the footer of each comment.
- Dialog titled “Report Content” with the prompt “Please provide a reason for reporting this content.”.
- “Reason” text area with the placeholder “Why are you reporting this content?”.
- “Cancel” action and destructive “Report” action, which stays disabled while the reason is empty or a request is running.
- “Reported” badge with a check icon replacing the action once the content has been reported by the member.
- Already-reported panel shown while the dialog is open and the status becomes reported: the description reads “You have already reported this content.” and the body “You have already reported this content. Our team will review it.”, with a single “Close” action and no reason field.
- Success toast “Report submitted successfully” after a successful submission.

Data Processing Overview

- As soon as the report component mounts, the client checks whether the member already reported the content through `GET /api/forums/posts/{postId}/is-reported` or `GET /api/forums/posts/{postId}/comments/{commentId}/is-reported`, and keeps the answer in a query cache with an infinite stale time.
- The client sends `POST /api/forums/posts/{postId}/reports` for a post, or `POST /api/forums/posts/{postId}/comments/{commentId}/reports` for a comment, with the reason.
- The API verifies that the reported target exists and is not soft deleted.
- For a comment, the API also verifies that the comment belongs to the post of the route.
- The API looks for an existing report of the same member on the same target and refuses a duplicate.
- The API inserts the report with the reporter of the token, the target and the reason; the status defaults to `PENDING` and no administrator is assigned yet.
- The client clears the reason, shows the confirmation toast, marks the report as done in its cache and closes the dialog after a short delay.
- While the dialog is still open, the reported state replaces the form with the already-reported panel. Once the dialog closes, the action is replaced by the "Reported" badge, so the same item cannot be reported again from the interface.

Screen Layout (Mock-up Description)

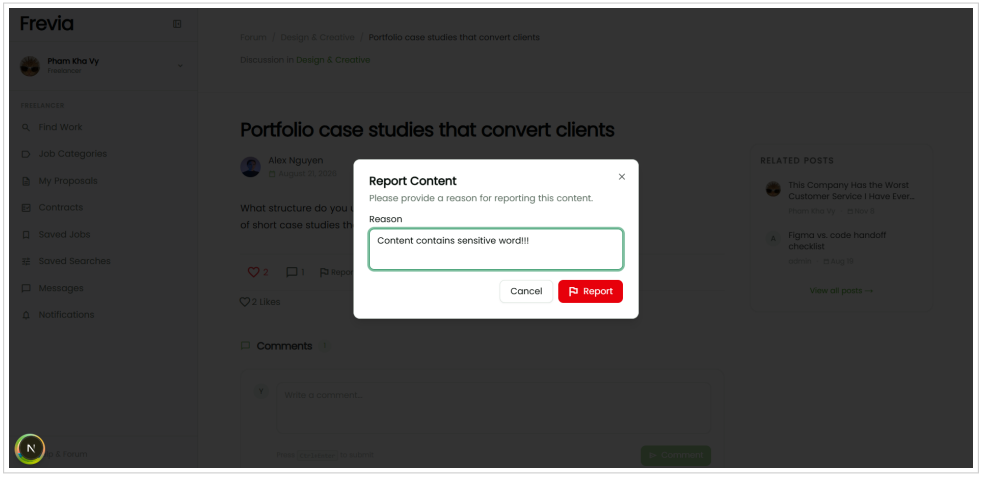

Figure 74: Report dialog with the reason field

- The action is available in the post action bar and in the footer of every comment, so a member can flag the exact item that is problematic.
- The dialog explains what is expected and gives a free-text reason, which the moderator reads before making a decision.
- The confirm action is destructive styled and disabled without a reason, which prevents empty reports.

- After a successful submission the toast “Report submitted successfully” confirms the action and the badge “Reported” prevents a second submission.
- When the status is already reported, the dialog displays the confirmation panel with “You have already reported this content. Our team will review it.” instead of the reason field.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the reported post, or of the post that holds the reported comment.
`commentId`	Optional identifier of the reported comment; absent when the post itself is reported.
`reason`	Free-text justification of the report, required.
Access token	Identifies the reporter; it becomes `reporterId` in the stored report.

Validation Rules

- The requester must be authenticated.
- The reason is required and must contain at least one character.
- The reported post must exist and must not be soft deleted, otherwise the request fails with `Error.ForumPostNotFound`.
- The reported comment must exist, must not be soft deleted and must belong to the post of the route, otherwise the request fails with `Error.ForumCommentNotFound`.
- Exactly one member may report a given target once; a second attempt fails with `Error.ForumReportAlreadyExists`.
- The status of a new report is `PENDING` and the administrator field stays empty until a moderator handles it.

Business Rules

- **BR-FORUM-47**: Only authenticated users can submit reports.
- **BR-FORUM-48**: Reports must be associated with either a post or a comment.
- **BR-FORUM-49**: New reports shall be created with status PENDING.
- **BR-FORUM-50**: A user may submit only one report for the same content item.

Normal Flow

1. The signed-in member reads a post or a comment that violates the community rules.
2. The client has already checked the reported status on mount and renders the “Report” action only when the member has not reported that item.
3. The member selects the “Report” action of that item.
4. The member writes a reason and confirms.
5. The client sends the report request with the access token.
6. The API validates the reason and loads the reported target.
7. The API confirms that the target exists and, for a comment, that it belongs to the post.
8. The API confirms that the member has not reported the same target yet.
9. The API inserts the report with status `PENDING`.
10. The API returns the created report.
11. The client clears the reason, shows the success toast, marks the cache as reported and closes the dialog after a short delay.
12. The “Reported” badge replaces the action for this member.

Abnormal Flow / Exception Handling

Case	Description
A1	The reason is empty. The confirm action is disabled, and a forced request is rejected by the validation pipe.
A2	The post was deleted before the submission. The API returns `Error.ForumPostNotFound` and no report is stored.
A3	The comment was deleted before the submission, or does not belong to the post. The API returns `Error.ForumCommentNotFound`.
A4	The member already reported the same item. The API returns `Error.ForumReportAlreadyExists`, the client displays the translated error toast and the dialog stays open with the reason; the badge appears only once the cached status is reported.
A5	An Administrator opens the moderation screen. Listing and updating reports require the Administrator role, otherwise the API returns `Error.ForumReportForbidden`.
A6	The insert fails at database level. The API returns `Error.FailedToCreateForumReport` and the dialog stays open with the reason.
A7	The member is not signed in. The action is not rendered for guests, and a direct request is rejected with an authentication error.

Case	Description
A8	The member reports their own content through a crafted request. The record can be created, but the action is never offered in the interface, and the moderation screen treats it like any other report.
A9	The reported-status check fails. The component treats the item as not reported, so the action is shown and a duplicate submission is refused by the API with the already-exists error.

Output Data

Data Name	Description
`id`	Identifier of the created report.
`reporterId`	Member who submitted the report.
`postId`, `commentId`	Reported targets; exactly one of them identifies the item, the other may be null for a comment report.
`reason`	Reason written by the reporter.
`status`	`PENDING` for a new report; later `REVIEWED`, `RESOLVED` or `DISMISSED` after the moderator decision.
`adminId`, `createdAt`	Administrator who handled the report, null at creation, and the creation timestamp.

Post-Conditions

- One pending report exists for the reported item and the reporter.
- The reported content is not modified and stays visible until a moderator decides.
- The report appears in the moderation queue of the Administrator, where it can be reviewed, resolved or dismissed.
- The reporter sees the “Reported” badge and cannot submit the same report again from the interface.

3.5 Contract Management

3.5.1 Create Contract

Function Trigger

A Client initiates contract creation from an accepted proposal page or clicks the "Hire" / "Create Contract" action button on a submitted job proposal.
Navigation path: Client Dashboard → My Jobs → Proposals (`/client/jobs/:id/proposals`) → Click "Hire" → Create Contract.

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/backend`), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows a Client to establish a formal legally binding freelance agreement with a selected Freelancer following proposal acceptance. The client configures project scope, contractual clauses, payment milestones, budget allocation, platform service fees, and target delivery deadlines before dispatching the contract for review.

Interface

- Contract creation view at `/client/contracts/create` or modal accessed from proposal hiring flow.
- Proposal summary card displaying job title, freelancer profile snapshot, bid amount, and estimated duration.
- Contract terms editor with pre-fill template selector, custom clause textarea, and intellectual property provisions.
- Milestone dynamic builder supporting addition, reordering, and removal of deliverable milestones (title, description, amount, due date).
- Financial breakdown summary card calculating total milestone budget, platform fee percentage (10%), client service fee, and aggregate contract value.
- Action buttons: "Save as Draft" and "Send Contract for Review".

Data Processing Overview

- The client submits an HTTP request: `POST /api/contracts` with payload containing `proposalId`, `title`, `terms`, and an array of `milestones`.
- The backend `AuthGuard` validates the Client's JWT token, and `RoleGuard` verifies `CLIENT`

authorization.

- `ContractService.createContract()` queries the proposal: verifies that the proposal exists, belongs to an active job owned by the calling client, and has status `ACCEPTED` or `SUBMITTED`.
- The service validates that the sum of all milestone amounts equals `totalAmount` exactly, and calculates client and freelancer platform fee amounts.
- A URL-friendly unique `slug` is generated from the contract title and random entropy.
- Inside a database transaction, `ContractRepository` persists the `Contract` entity with initial status `PENDING` (or `DRAFT`), creates associated `Milestone` records with status `PENDING` and `paymentStatus = UNPAID`, and updates proposal status to `HIRED`.
- An in-app real-time notification and email dispatch are sent to the Freelancer inviting them to review and sign the agreement.

Screen Layout (Mock-up Description)

Figure 75: Create Contract screen

- The page header displays the breadcrumb “Contracts / Create New” alongside a prominent title “Create Project Contract”.
- The left pane contains the contract scope form: title input, terms and conditions rich-text area, and milestone configuration list with “+ Add Milestone” button.
- Each milestone block renders inputs for milestone name, deliverable description, monetary amount in USD, and date picker for completion deadline.
- The right sticky sidebar displays the financial summary card: subtotal, platform processing fee (10%), total contract commitment, and clear milestone progress indicator.
- The bottom action footer provides “Back”, “Save Draft”, and “Create & Send Contract” action triggers.

Function Details

Input Data

Data Name	Description
`proposalId`	Unique numeric identifier of the accepted job proposal (integer).
`title`	Contract agreement title (string, 5–255 characters).
`terms`	Detailed contract scope, deliverables, and legal clauses (text, optional).
`milestones`	Array of milestone objects containing `title`, `description`, `amount`, and `dueDate`.
`milestone.amount`	Positive monetary value in USD (decimal, minimum 5.00).
`milestone.dueDate`	Target delivery deadline formatted as ISO 8601 date, must be a future date.

Validation Rules

- The calling user must be the verified owner (Client) of the associated job posting.
- The proposal must not already be linked to an existing active or pending contract.
- Total contract amount must be greater than zero and must equal the arithmetic sum of all milestone amounts.
- Every milestone due date must be set strictly in the future relative to the submission timestamp.
- The Freelancer account must have `status = ACTIVE` and verified identity.

Business Rules

- **BR-CT-01**: Only users authenticated with the `CLIENT` role shall create new contracts.
- **BR-CT-02**: Total contract budget shall strictly match the aggregate sum of all defined milestone amounts.
- **BR-CT-03**: Creating a contract locks the linked proposal, preventing duplicate contracts on the same proposal.
- **BR-CT-04**: The contract shall be initialized in `PENDING` status until signed by both contract parties.

Normal Flow

1. The Client navigates to the accepted proposal and clicks “Hire / Create Contract”.

2. The web application loads the contract creation form with pre-populated proposal data.
3. The Client reviews project scope, customizes terms, and defines deliverable milestones.
4. The Client clicks “Create & Send Contract”.
5. The system validates input fields, financial budget allocations, and future milestone deadlines.
6. The backend persists the contract and milestone records inside a database transaction.
7. The linked proposal is transitioned to `HIRED` state.
8. The system dispatches an alert notification to the Freelancer and redirects the Client to the Contract Detail page.

Abnormal Flow / Exception Handling

Case	Description
A1	The sum of milestone amounts does not match the total budget. The API returns HTTP 422 Unprocessable Entity with error `ContractMilestonesAmountMismatchException`; form highlights discrepancy.
A2	The specified proposal is already contracted or not accepted. The API returns HTTP 400 Bad Request with error `ProposalAlreadyContractedException`.
A3	Milestone due date is in the past. The form validation rejects submission and displays “Milestone deadline must be a future date.”
A4	The calling user is not the owner of the job. The backend rejects the request with HTTP 403 Forbidden; access denied.
A5	Freelancer profile is suspended or banned. The system rejects contract creation with HTTP 422; displays notice to select an active freelancer.
A6	Database connection failure during transaction persistence. The transaction rolls back and returns HTTP 500: “Unable to create contract. Please try again.”
A7	Client network connection drops during submission. The client displays retry banner without duplicating contract records.

Output Data

Data Name	Description
`id`	Unique auto-incrementing integer identifier of the created contract.
`slug`	Unique URL-friendly slug identifier for bookmarking and routing.
`status`	Initial operational contract state (`PENDING` or `DRAFT`).
`totalAmount`	Final aggregated contractual budget committed by the client.
`milestones[]`	Array of created milestone entities with generated sequence IDs and due dates.
`createdAt`	ISO 8601 creation timestamp.

Post-Conditions

- A new `Contract` entity is persisted with status `PENDING`.
- Milestone records are linked to the contract with `paymentStatus = UNPAID`.
- An audit log record is created in the contract activity history.
- An invitation notification is queued for the Freelancer.

3.5.2 Update Contract

Function Trigger

A Client clicks the "Edit Terms" button on a pending contract detail workspace to revise agreement specifications before signature.
Navigation path: Client Dashboard → Contracts → Contract Detail (`/client/contracts/:slug`) → Click "Edit Terms".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows a Client to modify contract clauses, milestone budgets, scopes, and target completion

dates while the contract remains in pending negotiation. Updating terms ensures flexibility to adapt to mutual renegotiations before formal signature activation.

Interface

- Contract terms editor modal or dedicated edit view pre-populated with existing terms.
- Milestone list editor allowing inline title, amount, description, and deadline modifications.
- Interactive budget calculator displaying adjusted totals and fee implications.
- Prominent warning notice: "Updating contract terms will reset any existing signatures and require re-signing.".
- Action buttons: "Save Changes" and "Cancel".

Data Processing Overview

- The client submits an HTTP request: `PATCH /api/contracts/:id` with updated `title`, `terms`, and modified `milestones`.
- `ContractService.updateContract()` verifies caller identity against `contract.clientId`.
- The service validates that contract status is strictly `PENDING` or `DRAFT`; active or completed contracts are rejected.
- If the freelancer has already digitally signed, the update automatically invalidates the signature by resetting `signedByFreelancer = false` and `freelancerSignedAt = null`.
- The database updates the contract terms and synchronizes modified milestones inside a transaction.
- The system records a contract revision event in the activity timeline and alerts the Freelancer of the revisions.

Screen Layout (Mock-up Description)

Figure 76: Update Contract Terms screen

- The modal header displays the contract title, identifier, and current status badge.
- An alert box at the top highlights that saving modifications invalidates previous signatures.
- Form fields display editable contract scope, clause paragraphs, and milestone blocks with trash and add buttons.
- Live calculation widget dynamically updates milestone total versus overall contract amount.
- Footer contains “Discard Changes” secondary button and “Save & Notify Freelancer” primary button.

Function Details

Input Data

Data Name	Description
`contractId`	Unique numeric contract identifier passed as path parameter (integer).
`terms`	Revised contract terms and scope text (string, optional).
`milestones`	Updated array of milestone specifications (title, amount, dueDate, description).

Validation Rules

- Only the Client who created the contract is permitted to update its terms.
- The contract must currently reside in `PENDING` or `DRAFT` status.
- Total modified milestone amounts must equal the updated total budget.
- Future date requirements apply to all modified milestone deadlines.

Business Rules

- **BR-CT-05**: Modifying contract terms shall automatically invalidate any existing signatures, setting `signedByFreelancer = false` and `signedByClient = false`.
- **BR-CT-06**: Active, completed, or cancelled contracts cannot be modified directly.

Normal Flow

1. The Client navigates to the pending contract workspace and clicks “Edit Terms”.
2. The system opens the edit view with current agreement data loaded.
3. The Client modifies terms, deliverables, or milestone amounts.
4. The Client clicks “Save Changes”.
5. The system validates fields and updates records in the database.
6. Prior signatures are invalidated, and a revision log is recorded.
7. The Freelancer receives an update notification, and the updated contract is rendered.

Abnormal Flow / Exception Handling

Case	Description
A1	The contract has already transitioned to `ACTIVE` or `AWAITING_PAYMENT`. The API returns HTTP 422 with error `ContractAlreadyActiveException`; editing is locked.
A2	Caller is not the contract owner. The API returns HTTP 403 Forbidden; access denied.
A3	Total revised milestone amounts do not sum up to total budget. Returns HTTP 422 with validation error.
A4	Milestone due dates are set in the past. System highlights invalid date fields.
A5	Database concurrency conflict occurs during record update. System prompts to refresh and re-apply edits.

Output Data

Data Name	Description
`contract`	Updated contract entity with recalculated totals and reset signature flags.
`updatedAt`	ISO 8601 timestamp reflecting the last modification time.

Post-Conditions

- Contract terms and milestone records are updated in the database.
- `signedByFreelancer` and `signedByClient` are reset to `false`.
- Notification dispatched to the Freelancer alerting them of updated contract terms.

3.5.3 Complete Contract

Function Trigger

The Client clicks the "Complete Contract" button on an active contract detail workspace once all milestones have been finished and approved.

Navigation path: Client Dashboard → Contracts → Active Contract (`/client/contracts/:slug`) → Click "Complete Contract".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, PostgreSQL database, and Reviews module.

Purpose

Concludes the contractual project engagement following complete delivery of all milestone deliverables and settlement of escrow funds. Completing the contract archives project state, unlocks review and rating submissions for both parties, and generates a completion certificate.

Interface

- "Complete Contract" primary action button displayed in the active contract header banner.
- Confirmation dialog summarizing delivered milestones, total disbursed earnings, and completion effects.
- Feedback prompt inviting the user to provide star ratings, skill endorsements, and written reviews.
- Action buttons: "Confirm Completion" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `PATCH /api/contracts/:id/complete`.
- `ContractService.completeContract()` validates that the caller is the contract's Client.
- The service verifies that contract status is currently `ACTIVE`.
- Queries all associated milestones: verifies that every milestone has `status = COMPLETED` and `paymentStatus = RELEASED`.
- Checks that no active disputes exist on any contract milestone.
- Updates contract status to `COMPLETED` and sets `completedAt = new Date()`.
- Issues mutual review tokens, notifies the Freelancer, and generates digital completion credentials.

Screen Layout (Mock-up Description)

Figure 77: Complete Contract confirmation screen

- A celebratory modal dialog features a project completion icon and clear heading “Complete Contract Agreement”.
- Summary section lists approved deliverables, total budget paid out ($USD), and zero outstanding balances.
- Notice informs the user that once completed, the contract cannot be reopened and both parties will be prompted for feedback.
- Footer buttons: “Cancel” and “Confirm & Close Contract”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric identifier of the contract to complete (integer path parameter).

Validation Rules

- The calling user must be the authenticated Client of the contract.
- The contract must currently reside in `ACTIVE` status.
- All milestones must be in `COMPLETED` status and `RELEASED` payment status.
- There must be no ongoing open disputes on the contract or its milestones.

Business Rules

- **BR-CT-07**: A contract can only transition to `COMPLETED` when 100% of its milestones have been released and no dispute is open.
- **BR-CT-08**: Completed contracts are permanently closed from further deliverable submissions or financial adjustments.
- **BR-CT-09**: Contract completion triggers automatic opening of the reciprocal 14-day review window.

Normal Flow

1. The Client navigates to the active contract workspace where all milestones are finished.
2. The Client clicks “Complete Contract”.
3. The system displays the completion confirmation dialog.
4. The Client clicks “Confirm Completion”.
5. The system verifies all milestones are released and updates contract status to `COMPLETED`.
6. The system records the completion event in the activity log.
7. A success notification is displayed, the page transitions to completed state, and reciprocal review forms are unlocked.

Abnormal Flow / Exception Handling

Case	Description
A1	Unapproved or unpaid milestones remain. The API returns HTTP 422 with error `ContractHasUnreleasedMilestonesException`; displays message indicating pending milestones.
A2	An active dispute is currently unresolved. The API returns HTTP 422 with error `ContractHasActiveDisputeException`; completion blocked.
A3	Caller is not the Client owner. The API returns HTTP 403 Forbidden; access denied.
A4	Contract is already completed or cancelled. The API returns HTTP 400 Bad Request.
A5	Database update failure occurs. System displays: "Failed to complete contract. Please try again."

Output Data

Data Name	Description
`status`	Updated contract status (`COMPLETED`).
`completedAt`	ISO 8601 completion timestamp.
`reviewWindowExpiresAt`	Timestamp marking review window expiration deadline.

Post-Conditions

- Contract status is updated to `COMPLETED` in the database.
- Completion notification sent to both Client and Freelancer.
- Feedback and review submission prompts are rendered.

3.5.4 View Contract List

Function Trigger

An authenticated user (Client, Freelancer, or Admin) clicks "Contracts" in the dashboard main sidebar navigation.

Navigation path: User Dashboard → Navigation Menu → Contracts (`/client/contracts` or `/freelancer/contracts`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Provides a centralized contract management dashboard for users to browse, monitor, filter, and track all their freelance contracts across lifecycle states (Pending, Awaiting Payment, Active, Completed, Cancelled).

Interface

- Contracts dashboard page at `/client/contracts` or `/freelancer/contracts`.
- Status tab navigation bar: All, Pending Signature, Awaiting Payment, Active, Completed, Cancelled.
- Search input field supporting keyword matching on contract title and counterpart name.
- Contract card list rendering title, counterpart avatar and name, total budget, progress bar, milestone ratio, status badge, and "View Contract" button.
- Pagination controls footer with page numbers, page size selector, and item summary count.
- Empty-state placeholder with helpful guidance when no matching contracts exist.

Data Processing Overview

- The web client submits an HTTP request: `GET /api/contracts` with query parameters (`page`, `limit`, `status`, `search`).
- The API validates the JWT access token and extracts `userId` and `roleName`.
- If `roleName === 'CLIENT'`, queries contracts where `clientId = userId`; if `roleName === 'FREELANCER'`, queries contracts where `freelancerId = userId`; Administrators can query across all contracts.
- Applies status filters and case-insensitive substring search on `title`.
- Computes pagination count in parallel via `prisma.contract.count()`.
- Returns HTTP 200 with contract summaries and pagination metadata.

Screen Layout (Mock-up Description)

Figure 78: Contracts list dashboard

- The page title “Contracts” sits alongside a metric row showing total active contracts and total committed budget.
- The filter bar features tabs for status categories and a search box with quick-clear button.
- Each contract card displays counterpart information on the left, progress meter and milestone badge in the center, and total amount with “View Contract” link on the right.
- The pagination component displays current page index, total contracts count, and next/prev controls.

Function Details

Input Data

Data Name	Description
`page`	Page number for paginated retrieval (integer, default 1).
`limit`	Number of contracts per page (integer, default 10, max 50).
`status`	Optional contract status filter (`PENDING`, `AWAITING_PAYMENT`, `ACTIVE`, `COMPLETED`, `CANCELLED`).
`search`	Keyword search query matching contract titles or counterpart names.

Validation Rules

- User must be authenticated with active session credentials.
- `page` must be a positive integer ≥ 1.
- `limit` must be clamped between 1 and 50.
- Status parameter must match valid `ContractStatus` enum values.

Business Rules

- **BR-CT-10**: Clients and Freelancers shall only see contracts in which they are a verified participant.
- **BR-CT-11**: Administrators shall have platform-wide access to view and inspect all contracts.

Normal Flow

1. The user navigates to the Contracts section from the sidebar navigation.
2. The web application requests `GET /api/contracts` with default pagination parameters.
3. The API queries the database, filtering by caller role and ID.
4. The API returns the list of matching contract summaries and pagination metadata.
5. The client renders the contract cards and metric summaries.
6. The user can click any contract card to open the Contract Detail workspace.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated access attempt. The API returns HTTP 401 Unauthorized; browser redirects to `/login`.
A2	No contracts match the filter criteria. The client renders an empty-state card with "No contracts found."
A3	Database query timeout or server failure. The API returns HTTP 500; client displays a retry prompt.
A4	Invalid status filter provided. API falls back to querying all statuses without breaking.

Output Data

Data Name	Description
`contracts[]`	Array of contract summary records including ID, slug, title, counterpart info, total amount, status, and progress metrics.
`pagination`	Object containing `page`, `limit`, `totalItems`, and `totalPages`.

Post-Conditions

- Contract list is displayed with accurate status indicators and progress bars.

3.5.5 View Contract Detail

Function Trigger

A user clicks a contract item in the contract list or navigates directly to a contract URL. **Navigation path:** Contracts List → Click Contract Card → Contract Workspace (`/client/contracts/:slug` or `/freelancer/contracts/:slug`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, PostgreSQL database, and Cloudinary storage.

Purpose

Serves as the complete collaborative project workspace for a contract. Displays legal terms, digital signature records, financial escrow balances, interactive milestone roadmap, shared files tab, and contextual lifecycle actions.

Interface

- Contract workspace screen at `/client/contracts/:slug` or `/freelancer/contracts/:slug`.
- Header banner: contract title, slug identifier, current status badge, counterpart summary, and contextual actions (Sign, Pay Fee, Complete, Cancel).
- Financial summary widget: total project budget, escrow funded balance, paid out earnings, and remaining balance.
- Workspace tab navigation: "Milestones", "Contract Agreement (PDF)", "Shared Files", and "Activity History".
- Milestones roadmap rendering each milestone's deliverables, deadlines, payment state, submission history, and action triggers.

Data Processing Overview

- The client issues an HTTP request: `GET /api/contracts/slug/:slug` or `GET /api/contracts/:id`.
- `ContractService.getContractBySlug()` validates authorization: verifies that `userId === contract.clientId || userId === contract.freelancerId` or the user possesses `ADMIN` role.
- The repository executes a nested Prisma query retrieving the contract, client profile, freelancer profile, milestones, shared files count, and transaction logs.
- Calculates financial progress indicators (percentage funded, percentage released).
- Returns HTTP 200 with the full contract workspace composite entity.

Screen Layout (Mock-up Description)

Figure 79: Contract detail workspace screen

- The top card displays project title, counterpart info, start date, and status badge.
- Below the header, three metric cards show Total Value ($USD), Escrow In Deposit, and Funds Released.
- The main container is tabbed: the default “Milestones” tab presents vertical timeline cards for each milestone deliverable.
- Right sidebar displays digital signature verification box (showing Client and Freelancer signature status and timestamps) and action buttons.

Function Details

Input Data

Data Name	Description
`slug` or `id`	Unique URL-friendly slug string or numeric contract ID.

Validation Rules

- Contract slug or ID must exist in the database and not be hard-deleted.
- The calling user must be an authorized participant (Client or Freelancer) or an Administrator.

Business Rules

- **BR-CT-12**: Strict data isolation ensures only authorized parties (Client, Freelancer, Admin) can view contract details.
- **BR-CT-13**: Contract financial figures must dynamically reflect verified ledger transactions.

Normal Flow

1. The user clicks a contract card from the contract list.
2. The browser navigates to `/contracts/:slug` and requests contract data.
3. The API validates participant authorization and loads the full relational contract graph.
4. The client renders the workspace with header, metrics, milestone roadmap, and tabs.
5. The user interacts with milestone cards or switches tabs to inspect shared files.

Abnormal Flow / Exception Handling

Case	Description
A1	User is not a participant in the contract. The API returns HTTP 403 Forbidden with error `ContractForbiddenException`; client displays "Access denied."
A2	The specified contract slug does not exist. The API returns HTTP 404 Not Found with error `ContractNotFoundException`; renders 404 page.
A3	Unauthenticated request. Returns HTTP 401; redirects to sign-in page.
A4	Network connection error. Client displays an error banner with a retry button.

Output Data

Data Name	Description
`contract`	Comprehensive contract entity with client, freelancer, milestones, signatures, and platform fees.
`milestones[]`	Array of milestones with progress status, escrow funding status, and deliverable submissions.
`signatures`	Digital signature audit status (`signedByClient`, `signedByFreelancer`, timestamps).

Post-Conditions

- Full contract workspace is loaded and accessible.

3.5.6 Sign Contract

Function Trigger

A Client or Freelancer clicks the "Sign Contract" action button on a pending contract detail workspace.
Navigation path: Contract Detail (`/contracts/:slug`) → Click "Sign Contract".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, PostgreSQL database, and Contract PDF Generation Service.

Purpose

Captures the digital signature of a contract participant to manifest legal consent to project terms. When both counterparties have signed, the contract advances to platform fee settlement or active state, enabling escrow funding and milestone execution.

Interface

- "Sign Contract" prominent action button in the contract banner.
- Digital signature modal displaying full legal agreement text and milestone schedule.
- Mandatory agreement checkbox: "I have read, understood, and agree to the Frevia Contract Agreement and Terms of Service.".
- Action buttons: "Confirm & Digitally Sign" and "Review Later".

Data Processing Overview

- The user submits an HTTP request: `PATCH /api/contracts/:id/sign`.
- `ContractService.signContract()` verifies that the caller is either `contract.clientId` or `contract.freelancerId`.
- Checks that contract status is `PENDING` or `AWAITING_PAYMENT`.
- Sets corresponding flag: if caller is Client, updates `signedByClient = true` and `clientSignedAt = now()`; if Freelancer, updates `signedByFreelancer = true` and `freelancerSignedAt = now()`.
- If both parties have now signed:
  - If platform fee is unpaid, sets `status = AWAITING_PAYMENT`.
  - If platform fee is already paid, transitions `status = ACTIVE` and sets `signedAt = now()`.
  - Invokes `ContractPdfService.generateAndSaveContractPdf()` to generate and store immutable legal PDF.
- Dispatches notification alerting the counterpart party.

Screen Layout (Mock-up Description)

Figure 80: Sign Contract dialog

- Modal dialog opens with legal paper motif and title “Sign Contract Agreement”.
- Scrollable document window renders full agreement clauses, project milestones, and total budget.
- Signature section displays signer full name, authenticated email, and IP timestamp notice.
- Acceptance checkbox must be checked to enable the primary “Confirm & Sign” button.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric identifier of the contract to sign (integer path parameter).

Validation Rules

- Caller must be a recognized participant in the contract (Client or Freelancer).
- The contract must currently reside in `PENDING` or `AWAITING_PAYMENT` status.
- The user must not have already signed the current version of the contract.

Business Rules

- **BR-CT-14**: A contract transitions to `ACTIVE` only when both parties have signed and platform fee is settled.
- **BR-CT-15**: Digital signing generates an immutable contract PDF audit document with cryptographic integrity.

Normal Flow

1. The user opens the pending contract workspace and reviews project terms.
2. The user clicks “Sign Contract”.
3. The system opens the signature modal dialog.
4. The user reviews the clauses and checks the agreement consent box.
5. The user clicks “Confirm & Digitally Sign”.
6. The system records the signature timestamp in the database.
7. If counterpart signature is already recorded, contract transitions to active/awaiting payment and generates PDF.
8. The system notifies the other party and updates the workspace view.

Abnormal Flow / Exception Handling

Case	Description
A1	User has already signed this contract. The API returns HTTP 422 with error `ContractAlreadySignedException`; button disabled.

Case	Description
A2	Contract is in invalid status (e.g. `CANCELLED` or `COMPLETED`). The API returns HTTP 422; displays error message.
A3	Caller is not a contract participant. The API returns HTTP 403 Forbidden; access denied.
A4	PDF generation service fails temporarily. The signature is recorded and PDF generation is retried via background queue.

Output Data

Data Name	Description
`signedByClient`	Boolean flag indicating whether the Client has signed.
`signedByFreelancer`	Boolean flag indicating whether the Freelancer has signed.
`status`	Updated contract status (`PENDING`, `AWAITING_PAYMENT`, or `ACTIVE`).

Post-Conditions

- Signature timestamp and boolean recorded in database.
- Contract status updated if both parties signed.
- Notification dispatched to counterpart.

3.5.7 Cancel Contract

Function Trigger

A Client or Freelancer clicks the "Cancel Contract" option from the contract action menu.

Navigation path: Contract Detail (`/contracts/:slug`) → Action Menu → Click "Cancel Contract".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, PostgreSQL database, and Stripe payment refund service.

Purpose

Allows a contract participant to terminate a pending or active agreement before project completion. Requires specifying a cancellation reason and automatically manages escrow refunds for unreleased milestone deposits according to platform rules.

Interface

- "Cancel Contract" menu item in the contract header action dropdown.
- Cancellation confirmation modal detailing cancellation terms and financial consequences.
- Reason dropdown selector (e.g., Unresponsive counterpart, Scope change, Mutual agreement, Technical impediment).
- Optional detailed explanation textarea.
- Action buttons: "Confirm Cancellation" and "Keep Contract".

Data Processing Overview

- The user submits an HTTP request: `PATCH /api/contracts/:id/cancel`.
- `ContractService.cancelContract()` validates caller authorization against participant IDs.
- Checks that contract status is not already `COMPLETED` or `CANCELLED`.
- Verifies that no active milestone disputes exist that require formal arbitration.
- Inside a database transaction, updates contract `status = CANCELLED` and records cancellation reason.
- For any milestones with `paymentStatus = FUNDED` that have not been released, marks them for escrow refund back to the Client.
- Dispatches urgent alert notification to the counterpart party.

Screen Layout (Mock-up Description)

Figure 81: Cancel Contract modal

- Modal displays warning banner: “Canceling a contract is irreversible and immediately halts all project work.”.
- Financial summary indicates any escrow funds that will be refunded to the client.
- Required dropdown to select cancellation category and textarea for explanation.
- Red destructive button “Cancel This Contract” and neutral “Go Back”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`reason`	Category and textual explanation for cancellation (string, optional).

Validation Rules

- Caller must be a recognized participant of the contract.
- Contract must reside in `PENDING`, `AWAITING_PAYMENT`, or `ACTIVE` status.
- Completed contracts cannot be cancelled.
- Contracts with unresolved disputes cannot be cancelled until dispute settlement.

Business Rules

- **BR-CT-16**: Cancelled contracts cannot be reopened or reactivated under any circumstance.
- **BR-CT-17**: Unreleased escrow funds on cancelled contracts shall be refunded to the client unless contested via dispute.

Normal Flow

1. The user navigates to the contract workspace and selects “Cancel Contract”.
2. The system opens the cancellation confirmation modal.
3. The user selects a reason and provides an explanation.
4. The user clicks “Confirm Cancellation”.
5. The system validates contract state and updates status to `CANCELLED`.
6. Unreleased escrow funds are refunded to the client.
7. Both parties receive cancellation notifications, and the workspace reflects cancelled state.

Abnormal Flow / Exception Handling

Case	Description
A1	Contract is already completed. The API returns HTTP 422 with error `ContractAlreadyCompletedException`; cancellation blocked.

Case	Description
A2	An active dispute is in progress. The API returns HTTP 422 with error `ContractHasActiveDisputeException`; prompts to resolve dispute first.
A3	Caller is not a contract participant. The API returns HTTP 403 Forbidden; access denied.
A4	Stripe refund processing fails. Refund is queued for automatic reconciliation, and contract is marked cancelled.

Output Data

Data Name	Description
`status`	Updated contract status (`CANCELLED`).
`cancelledAt`	ISO 8601 cancellation timestamp.

Post-Conditions

- Contract status is updated to `CANCELLED`.
- Escrow refunds are initiated for funded unreleased milestones.
- Notifications sent to both parties.

3.5.8 View Contract Files

Function Trigger

A participant clicks the "Files" tab inside the contract detail workspace.

Navigation path: Contract Detail (`/contracts/:slug`) → Click "Files" tab.

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, PostgreSQL database, and Cloudinary storage.

Purpose

Provides a secure shared document repository for participants to inspect, download, and manage project specifications, deliverables, assets, and design files exchanged throughout the con-

tract lifecycle.

Interface

- "Files" tab view inside the Contract Detail workspace.
- Summary bar displaying total files count and "Upload File" action button.
- File items table/grid displaying file name, format icon (PDF, ZIP, image, doc), uploader avatar and name, upload date, file size, and action buttons.
- Download action button (direct cloud download link).
- Delete action icon (visible only to the original uploader and Admin).
- Empty-state card shown when no files have been uploaded yet.

Data Processing Overview

- The client issues an HTTP request: `GET /api/contracts/:contractId/files`.
- `SharedFileService.getSharedFiles()` validates that caller is a participant in the contract or possesses `ADMIN` role.
- Queries `prisma.sharedFile.findMany` where `contractId === contractId` and `deletedAt = null`.
- Includes uploader information (id, fullName, avatarUrl) and orders by `createdAt desc`.
- Returns HTTP 200 with the array of shared file objects.

Screen Layout (Mock-up Description)

Figure 82: Contract shared files tab

- The tab header shows total uploaded files count and a blue “Upload File” button.
- Table columns: File Name, Type, Uploaded By, Date, File Size, and Actions.
- Each row features an appropriate file type icon, file name, download icon, and trash icon (if authorized).
- An empty state features an upload illustration with “No files uploaded yet. Drag and drop to upload.”.

Function Details

Input Data

Data Name	Description
`contractId`	Unique numeric contract identifier (integer path parameter).

Validation Rules

- Caller must be an authorized contract participant (Client or Freelancer) or an Administrator.
- Target contract must exist in the database.

Business Rules

- **BR-CT-18**: Only non-deleted files (`deletedAt = null`) belonging to the contract shall be returned.
- **BR-CT-19**: Contract file access is strictly restricted to contract participants and system administrators.

Normal Flow

1. The user navigates to the contract workspace and selects the "Files" tab.
2. The web client requests `GET /api/contracts/:contractId/files`.
3. The API validates participant authorization and queries non-deleted shared files.
4. The API returns the list of file entities with uploader details.
5. The client renders the file items table.
6. The user clicks the download link to retrieve any file.

Abnormal Flow / Exception Handling

Case	Description
A1	User is not a participant in the contract. The API returns HTTP 403 Forbidden; access denied.
A2	Contract does not exist. The API returns HTTP 404 Not Found.
A3	Storage service is temporarily unreachable. File download triggers a retry banner.

Output Data

Data Name	Description
`files[]`	Array of shared file records containing `id`, `fileName`, `fileUrl`, `publicId`, `uploader`, and `createdAt`.

Post-Conditions

- File repository is rendered for download and inspection.

3.5.9 Upload Contract File

Function Trigger

A participant clicks the "Upload File" button or drops files into the file dropzone in the Files tab.
Navigation path: Contract Detail → Files Tab → Click "Upload File".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Multer middleware, Cloudinary cloud object storage, Prisma ORM, and PostgreSQL database.

Purpose

Enables contract participants to securely upload project assets, deliverables, reference manuals, source files, and documents to the shared contract workspace.

Interface

- Upload file modal or drag-and-drop zone in the Files tab.
- Native file picker dialog supporting multi-format selection.
- Progress bar rendering upload percentage and transmission status.
- Supported format notice: images (PNG, JPG), documents (PDF, DOCX, XLSX), archives (ZIP, RAR) up to 25MB.
- Action buttons: "Upload" and "Cancel".

Data Processing Overview

- Client submits an HTTP `multipart/form-data` request: `POST /api/contracts/:contractId/files` with attached file payload.

- `buildUploadFilePipe()` validates file size ($\leq$ 25MB) and restricts malicious executable MIME types.
- `SharedFileService.uploadSharedFile()` checks participant authorization.
- Streams the file buffer to Cloudinary cloud storage under folder `frevia/contracts/:contractId`.
- Persists `SharedFile` record in database with `contractId`, `uploaderId = userId`, `fileUrl`, `publicId`, and `fileName`.
- Returns HTTP 201 Created with the saved file record.

Screen Layout (Mock-up Description)

Figure 83: Upload Contract File modal

- Modal displays dashed drag-and-drop area with cloud upload graphic.
- Text instructions state “Drag and drop your file here, or click to browse”.
- File size constraint (max 25MB) and allowed extensions are prominently listed.

- Selected file displays file name, size badge, upload progress bar, and “Upload File” action trigger.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`file`	Binary file stream submitted via `multipart/form-data` form field.

Validation Rules

- Caller must be an authorized contract participant.
- File size must not exceed 25MB.
- Executable file types (`.exe`, `.bat`, `.sh`, `.cmd`, `.vbs`) are strictly prohibited.
- Contract must not be in `CANCELLED` status.

Business Rules

- **BR-CT-20**: Maximum individual file upload size shall be 25MB.
- **BR-CT-21**: Uploaded contract files shall be associated with the uploader’s user ID for audit ownership.

Normal Flow

1. The user clicks “Upload File” in the contract Files tab.
2. The system opens the upload modal.
3. The user selects or drags a valid file.
4. The user clicks “Upload”.
5. The client streams the multipart form data to the server.
6. The server validates file size and MIME security, then uploads to cloud storage.
7. The database creates a new `SharedFile` entity.
8. The modal closes, a success toast appears, and the new file card is added to the list.

Abnormal Flow / Exception Handling

Case	Description
A1	File exceeds 25MB. Server rejects with HTTP 400 Bad Request; client displays "File size exceeds 25MB limit."
A2	Disallowed file extension. Server rejects with HTTP 400; displays "File format not permitted for security reasons."
A3	Caller is not a participant. Server returns HTTP 403 Forbidden; access denied.
A4	Cloud storage upload timeout. Server returns HTTP 502 Bad Gateway; client prompts to retry.

Output Data

Data Name	Description
`id`	Unique identifier of the uploaded shared file record.
`fileName`	Original file name.
`fileUrl`	Secure HTTPS download URL on cloud object storage.
`createdAt`	ISO 8601 upload timestamp.

Post-Conditions

- File is stored on cloud storage and indexed in database.
- File is immediately visible to all contract participants.

3.5.10 Delete Contract File

Function Trigger

The file uploader or an Administrator clicks the delete trash icon next to a shared file in the Files tab.
Navigation path: Contract Detail → Files Tab → File Row → Click "Delete File".

Function Description

Actors / Roles

- **Primary Actor:** File Uploader (Client or Freelancer), Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Cloudinary cloud storage, Prisma ORM, and PostgreSQL database.

Purpose

Allows users to remove obsolete, duplicate, or incorrect files from the contract repository while preserving audit integrity and preventing unauthorized deletion.

Interface

- Delete trash icon on the file row (conditionally rendered for uploader or Admin).
- Confirmation modal: “Are you sure you want to delete [fileName]? This action cannot be undone.”.
- Action buttons: “Delete” (destructive red) and “Cancel”.

Data Processing Overview

- Client submits an HTTP request: `DELETE /api/contracts/:contractId/files/:fileId`.
- `SharedFileService.deleteSharedFile()` verifies that the caller is either the file’s original `uploaderId` or possesses `ADMIN` role.
- Checks that the file is not currently attached as active evidence in an open dispute.
- Soft-deletes the database record by setting `deletedAt = new Date()`.
- Initiates asynchronous asset deletion from Cloudinary cloud storage using `publicId`.
- Returns HTTP 200 with deletion confirmation.

Screen Layout (Mock-up Description)

Figure 84: Delete Contract File confirmation dialog

- Modal displays red trash icon and warning heading “Delete Shared File”.
- Confirmation message states the exact file name and asks for user confirmation.
- Footer renders secondary “Cancel” button and primary red “Delete File” button.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`fileId`	Unique numeric identifier of the file to delete (integer path parameter).

Validation Rules

- The file must exist and not already be marked as deleted.
- Caller must be the original uploader of the file or an Administrator.
- The file must not be locked as legal evidence in an active dispute case.

Business Rules

- **BR-CT-22**: Only the original file uploader or an Administrator shall delete a shared file.
- **BR-CT-23**: Files cited in active arbitration disputes are locked from deletion until dispute resolution.

Normal Flow

1. The user navigates to the contract Files tab.
2. The user clicks the trash icon next to a file they uploaded.
3. The system opens the delete confirmation dialog.
4. The user clicks “Delete”.
5. The system validates ownership and marks `deletedAt = now()`.
6. The file asset is purged from cloud storage.
7. The dialog closes, the file is removed from the view, and a confirmation toast appears.

Abnormal Flow / Exception Handling

Case	Description
A1	Caller is not the file uploader and not an Admin. The API returns HTTP 403 Forbidden with error `SharedFileForbiddenException`; access denied.
A2	File is cited as evidence in an active dispute. The API returns HTTP 422 with error `FileLockedInDisputeException`; deletion rejected.
A3	File not found or already deleted. The API returns HTTP 404 Not Found.

Output Data

Data Name	Description
`success`	Boolean confirmation of successful deletion (`true`).
`deletedFileId`	Identifier of the soft-deleted file record.

Post-Conditions

- File is soft-deleted in the database (`deletedAt` populated).
- File is purged from the shared files view.

3.6 Job Browsing

3.6.1 Function: View Job List

Function Trigger

A guest, freelancer, or client accesses the marketplace job catalog from the navigation menu or search input.
Navigation path: Home → Find Work / Browse Jobs (`/jobs`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, and Client.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/backend`), Frevia REST API (NestJS), and PostgreSQL database.

Purpose

Allows any visitor or registered user to browse, search, and filter open freelance job opportunities with pagination, budget constraints, required skills, and keyword matching.

Interface

- Job browsing page at `/jobs`.
- Search bar and filter sidebar for category, budget range, skill badges, and experience level.
- Listing grid of job cards displaying title, description excerpt, client badge, budget, skills, and publication timestamp.
- Bottom pagination bar.

Data Processing Overview

- The client issues an HTTP request: `GET /api/jobs` with query parameters (`search`, `categoryId`, `skills`, `budgetMin`, `budgetMax`, `page`, `limit`, `sortBy`).
- The backend validates parameters and constructs a Prisma query.
- Applies strict constraints: `status = 'OPEN'`, `deletedAt = null`, client account is active, and submission deadline is valid.
- Returns items alongside pagination metadata (`totalItems`, `totalPages`, `currentPage`).

Screen Layout (Mock-up Description)

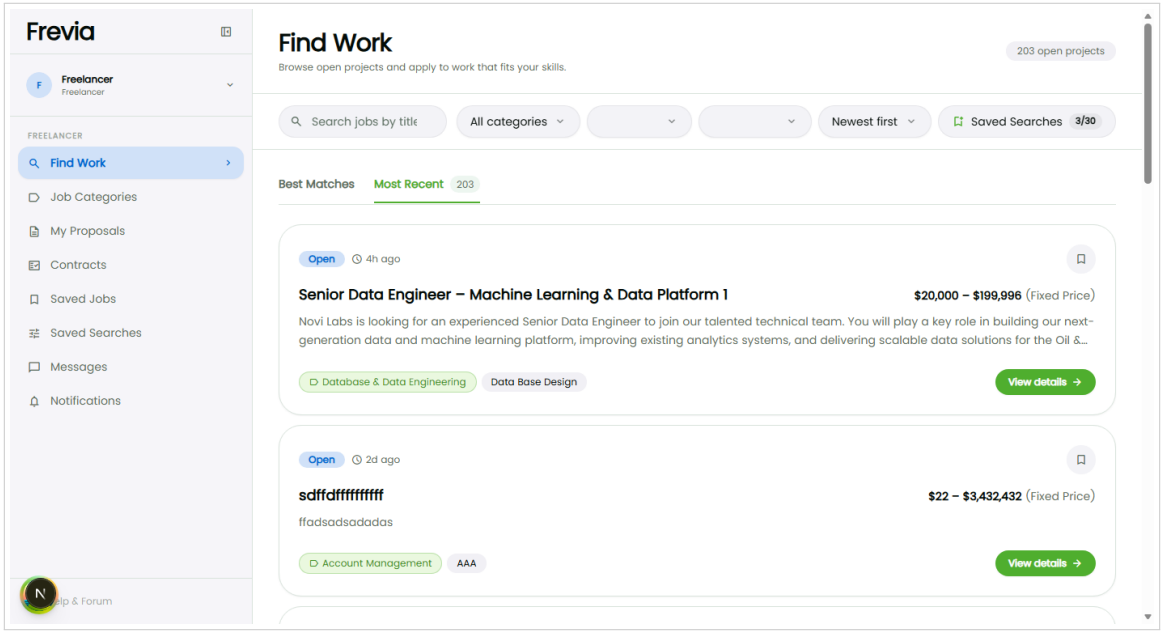

Figure 85: Job browsing list interface

- Search bar with keyword input and instant filter execution.
- Left sidebar containing filter options (Category, Experience Level, Job Type, Budget Range).
- Main area displaying job cards with verified client badges, required skills, and budget values.
- Pagination bar showing total results and page navigation.

Function Details

Input Data

Data Name	Description
search	Optional keyword string to match in job title and description.
categoryId	Optional ID of the job category filter.
skills	Optional list of skill IDs or names to filter.
budgetMin / budgetMax	Optional numeric budget boundaries.
page	Page number, integer ≥ 1, default 1.
limit	Items per page, integer between 5 and 50, default 10.

Validation Rules

- page must be an integer ≥ 1.
- limit must be between 1 and 50.
- budgetMin must be non-negative and $\leq$ budgetMax when both are supplied.

Business Rules

- **BR-J04**: Draft jobs (status = DRAFT) shall never appear in public search and browsing results.
- **BR-J25**: Closed jobs (status = CLOSED) shall be excluded from the open job listing.
- **BR-J31**: Completed and cancelled jobs shall be omitted from active browsing listings.
- **BR-J33**: Soft-deleted jobs (deletedAt !== null) shall be strictly excluded.
- **BR-J45**: Jobs belonging to suspended or banned clients shall be hidden immediately.
- **BR-J50**: Jobs flagged or suspended by administrative moderation shall be excluded from results.

Normal Flow

1. The user navigates to /jobs.
2. Frontend requests GET /api/jobs with query filters.
3. Backend executes validated Prisma query filtering active OPEN jobs.
4. Backend returns matching jobs with pagination metadata.
5. Frontend renders job summary cards and pagination controls.

Abnormal Flow / Exception Handling

Case	Description
A1	Invalid query parameters (e.g., negative page number). Backend returns HTTP 400 Bad Request; frontend resets filters to default.
A2	No jobs match the specified criteria. System returns empty list; frontend displays Empty State with a "Reset Filters" button.
A3	Database connection timeout. Backend returns HTTP 500; frontend displays error message with a "Retry" button.

Output Data

Data Name	Description
`data`	Array of job summary objects (`id`, `title`, `slug`, `budgetMin`, `budgetMax`, `category`, `skills`, `client`).
`meta`	Pagination metadata (`totalItems`, `totalPages`, `currentPage`).

Post-Conditions

- The matching job list is rendered for the user.
- Search state is reflected in URL search parameters.

3.6.2 Function: View Job Detail

Function Trigger

A user clicks on a job card or navigates directly via a job slug URL.

Navigation path: Browse Jobs (`/jobs`) → Click Job Card → Job Detail (`/jobs/:slug`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, and Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Displays complete specifications of a job posting, including comprehensive description, budget type, required skills, attachments, hiring quota, and client hiring history.

Interface

- Job detail screen at `/jobs/:slug`.
- Action bar with "Submit Proposal" button (for Freelancers) or "Manage Job" (for Job Owner).
- Two-column layout: left column contains full job description and requirements; right column displays client profile, budget summary, and hiring status.

Data Processing Overview

- Frontend requests `GET /api/jobs/:slug`.
- Backend checks job visibility: if `status = 'DRAFT'`, access is restricted to the owning client or administrator.
- Increments view counter if viewer is not the owning client.
- Returns full job object including client details, category, required skills, and attachments.

Screen Layout (Mock-up Description)

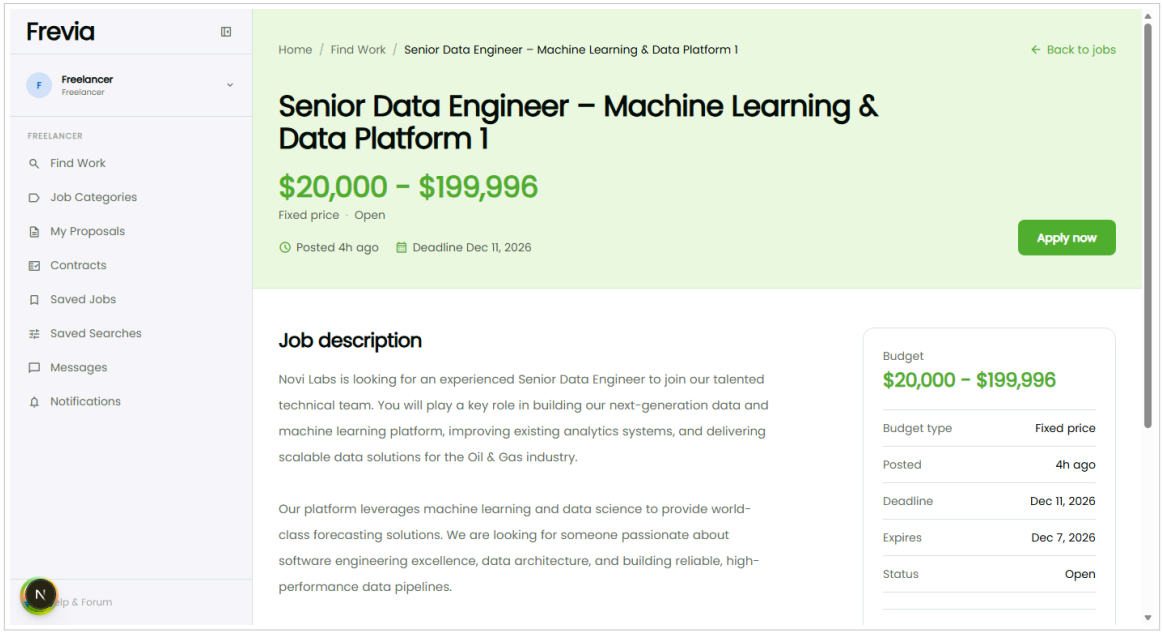

Figure 86: Job detail interface

- Header displaying job title, category badge, and posting age.
- Left content area with detailed project scope, qualifications, and downloadable attachments.
- Right sidebar displaying budget, hiring type, deadline, client review rating, and action buttons.

Function Details

Input Data

Data Name	Description
`slug`	String identifier in URL representing the job title slug.
Session Token	Optional JWT identifying viewer identity and role.

Validation Rules

- `slug` must be non-empty and formatted as a URL-friendly slug.

Business Rules

- **BR-J04**: Non-owners accessing draft jobs shall receive HTTP 404 Not Found.
- **BR-J07**: The "Submit Proposal" button is active only when `status = 'OPEN'` and hiring quota is not filled.
- **BR-J37**: Displays hiring quota (`SINGLE` or `MULTIPLE`) and filled positions.
- **BR-J42**: Job detail is permanently accessible via invariant URL slug.
- **BR-J49**: Explicitly distinguishes between proposal closing date (`expiryDate`) and project deadline.
- **BR-J51**: Views are incremented only when the viewer is not the job's author.

Normal Flow

1. User opens `/jobs/:slug`.
2. Frontend dispatches `GET /api/jobs/:slug`.
3. Backend queries job record with relations (skills, client, attachments).
4. Backend returns detailed job object.
5. Frontend renders full view and enables actions according to user role and job state.

Abnormal Flow / Exception Handling

Case	Description
A1	Slug does not exist or job is soft-deleted. Backend returns HTTP 404; frontend redirects to 404 page.
A2	Job is in DRAFT and viewer is not the client owner. Backend returns HTTP 403 Forbidden; frontend shows access denied notice.

Output Data

Data Name	Description
Job Object	Complete entity containing `id`, `title`, `description`, `budgetType`, `budgetMin`, `budgetMax`, `hiringType`, `positionsRequired`, `positionsFilled`, `skills`, `attachments`, and `client`.

Post-Conditions

- Full job specifications are displayed.
- Proposal action is enabled if user is an eligible freelancer.

3.7 Job Management

3.7.1 Function: View My Jobs

Function Trigger

A client clicks on "My Jobs" in the main navigation.

Navigation path: Home → Client Dashboard → My Jobs (`/client/jobs`).

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Provides clients with an overview of all jobs they have created across all lifecycle stages (`DRAFT`, `OPEN`, `IN_PROGRESS`, `CLOSED`, `COMPLETED`), showing proposal counts and management shortcuts.

Interface

- Client jobs management table at `/client/jobs`.
- Status tab bar: All, Open, Drafts, In Progress, Closed.
- Row action dropdowns (Edit, Close, Reopen, Delete, Review Proposals).

Data Processing Overview

- Client requests `GET /api/client/jobs`.

- Backend extracts client ID from JWT token to prevent unauthorized access (IDOR guard).
- Filters by `clientId` and excludes soft-deleted items.
- Returns list sorted by creation date descending with aggregate proposal counts.

Screen Layout (Mock-up Description)

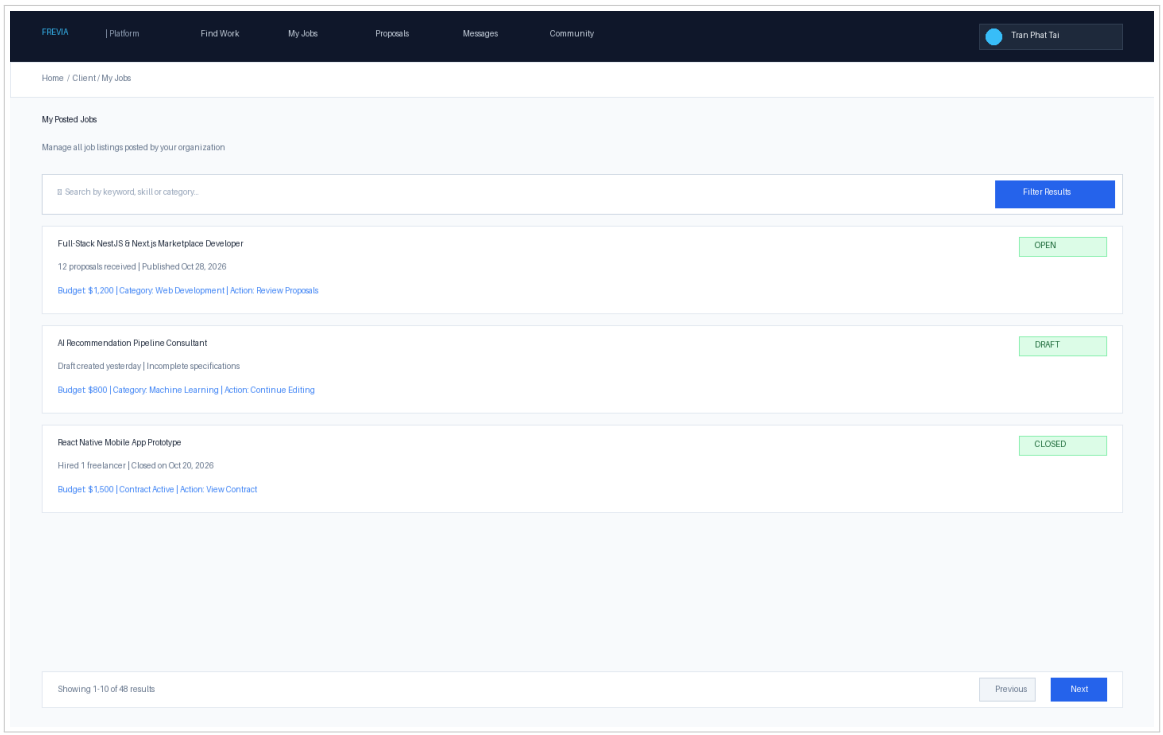

Figure 87: Client My Jobs interface

- Tab bar filtering jobs by operational status.
- Table listing jobs with title, budget, proposal counter badge, and date posted.
- Action menu on each row for editing, closing, or viewing submitted proposals.

Function Details

Input Data

Data Name	Description
JWT Token	Authenticated client token containing user ID and active Client role.
`status`	Optional query string filter (`ALL`, `DRAFT`, `OPEN`, `CLOSED`).

Validation Rules

- User must possess the `CLIENT` role.

Business Rules

- **BR-J01**: Only accounts with `CLIENT` role may access this screen.
- **BR-J14**: Query is strictly scoped to `job.clientId === currentUserId`.
- **BR-J33**: Soft-deleted jobs are excluded from the default view.

Normal Flow

1. Client visits `/client/jobs`.
2. Client application requests `GET /api/client/jobs`.
3. Backend validates client session and fetches own job listings.
4. UI renders table with status tags and management actions.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated or non-client role. Backend returns HTTP 403; redirected to login/onboarding.
A2	No jobs created yet. UI shows empty state with “Post a Job” call to action.

Output Data

Data Name	Description
jobs	Array of jobs with fields id, title, status, budgetMin, budgetMax, proposalCount, and createdAt.

Post-Conditions

- Client views complete job portfolio and proposal counts.

3.7.2 Function: Create Job

Function Trigger

A client clicks "Post a Job" from the client workspace.

Navigation path: Client Dashboard → Click "Post a Job" (/client/jobs/new).

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a client to create a new job posting in DRAFT status with specifications including title, category, skills, budget range, hiring type, and file attachments.

Interface

- Multi-step job posting form at /client/jobs/new.
- Title and description inputs with rich text support.
- Category selector, skill tag autocomplete, and budget input fields.
- Attachment uploader supporting up to 5 files.

Data Processing Overview

- Client submits form payload via POST /api/client/jobs.
- Backend validates required fields, budget values, and category status.
- Generates a unique invariant slug from title.
- Persists job record with initial state status = 'DRAFT'.

Screen Layout (Mock-up Description)

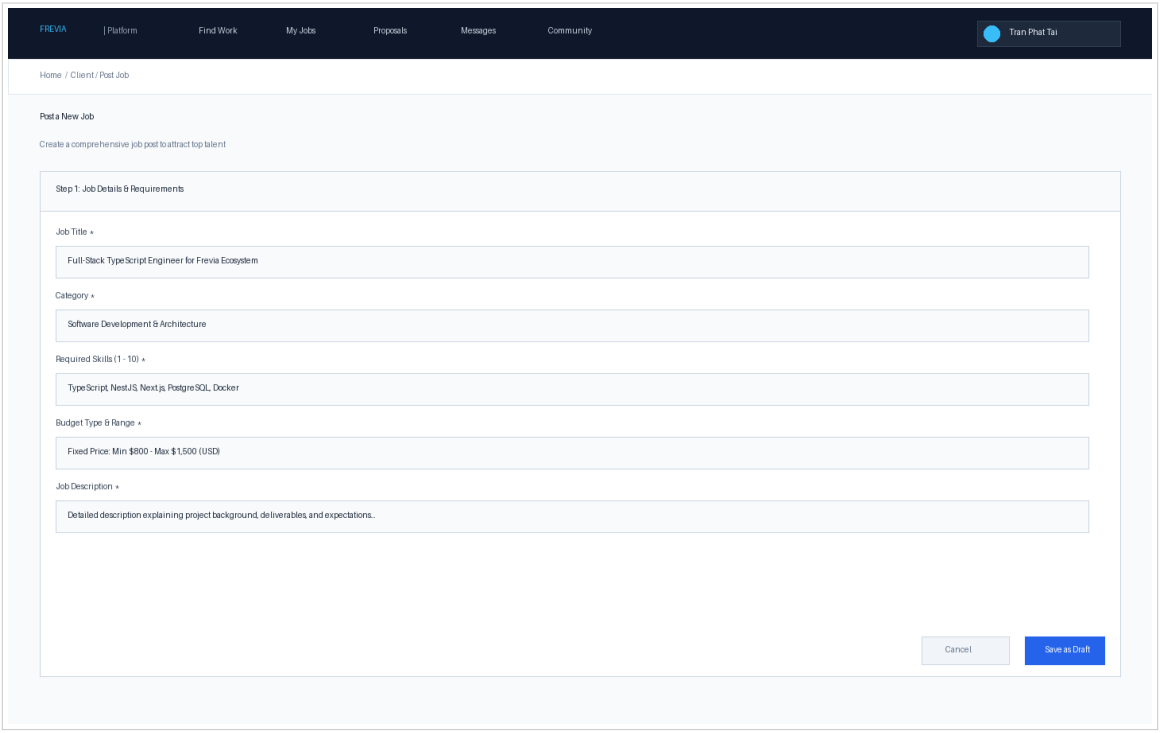

Figure 88: Post a new job form

- Step indicator showing form progression.
- Form fields for title, category dropdown, skill tagging, budget bounds, and description.
- Action buttons for “Save as Draft” and “Continue to Review”.

Function Details

Input Data

Data Name	Description
`title`	String (10–100 characters), job headline.
`categoryId`	UUID of the selected job category.
`skillIds`	Array of skill UUIDs (1 to 10 skills).
`budgetType`	Enum: `FIXED_PRICE` or `HOURLY`.
`budgetMin` / `budgetMax`	Numeric values representing compensation bounds.
`hiringType`	Enum: `SINGLE` or `MULTIPLE`.
`description`	Detailed job description text.

Validation Rules

- Title must be between 10 and 100 characters.
- Category must be active and valid.
- Must specify between 1 and 10 skills.
- Budget minimum must be $\geq$ 10 USD and $\leq$ budget maximum.

Business Rules

- **BR-J01**: Only clients may create jobs.
- **BR-J02**: Newly created jobs are always initialized with `status = DRAFT`.
- **BR-J37**: Defaults to `SINGLE` hiring type with `positions_required = 1`.
- **BR-J41**: Enforces minimum budget of $10 for fixed-price contracts.
- **BR-J42**: Automatically generates invariant slug upon creation.
- **BR-J50**: Scans for prohibited direct contact information (phone, external URLs).
- **BR-J52**: Exactly one category must be selected.

Normal Flow

1. Client fills out the job creation form.
2. Client clicks "Save as Draft".
3. Frontend sends payload to `POST /api/client/jobs`.
4. Backend validates inputs, generates slug, and saves record as `DRAFT`.
5. Backend returns created job entity; client is redirected to draft review page.

Abnormal Flow / Exception Handling

Case	Description
A1	Validation error (e.g., title too short or invalid budget). Returns HTTP 400 with field errors; highlights invalid inputs.
A2	Client account restricted from posting. Returns HTTP 403; displays account verification requirement.

Output Data

Data Name	Description
Job Record	Created entity with `id`, `slug`, `status = DRAFT`, and timestamps.

Post-Conditions

- A new job record is created in `DRAFT` state.

3.7.3 Function: Publish Job

Function Trigger

A client clicks "Publish Job" on a draft job posting.

Navigation path: Client Dashboard → My Jobs → Draft Job → Click "Publish Job".

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Validates completeness of a draft job, transitions its status to `OPEN`, indexes it in search, and activates public proposal submission.

Interface

- Draft job review screen with a prominent "Publish Job" button.
- Confirmation modal explaining that key terms become permanent upon publishing.

Data Processing Overview

- Client submits `POST /api/client/jobs/:id/publish`.
- Backend verifies that all mandatory fields are present and valid.
- Verifies ownership (`job.clientId === currentUserId`).
- Updates `status = 'OPEN'`, sets `publishedAt = now()`, and triggers job alert events.

Screen Layout (Mock-up Description)

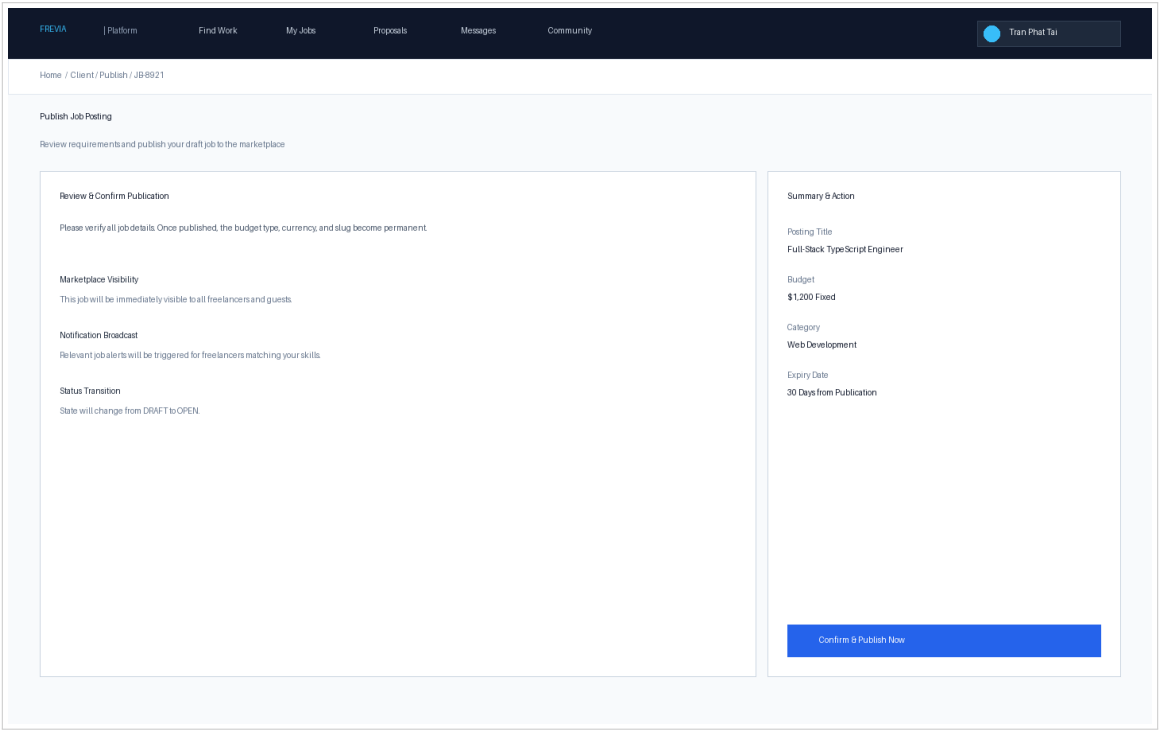

Figure 89: Publish job confirmation screen

- Summary card detailing the final parameters of the job.
- Checklist showing that title, category, skills, and budget are fully configured.
- Primary "Confirm & Publish Now" action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the draft job to publish.

Validation Rules

- Target job must currently be in `DRAFT` status.
- All required fields (title, category, ≥ 1 skill, budget, description) must be populated.

Business Rules

- **BR-J05**: Only jobs with complete, valid specifications can be published.
- **BR-J06**: Publishing transitions state from `DRAFT` to `OPEN` and sets `publishedAt`.
- **BR-J14**: Client must own the job posting.

- **BR-J41**: Currency and budget type become immutable upon publishing.
- **BR-J42**: URL slug is permanently frozen after publishing.
- **BR-J51**: Triggers matching job alerts (UC-23) to notify relevant freelancers.

Normal Flow

1. Client reviews draft job and clicks "Publish Job".
2. System verifies completeness and ownership.
3. System updates job status to `OPEN`.
4. System indexes job in search and queues job alert notifications.
5. UI redirects to active job page with a success toast notification.

Abnormal Flow / Exception Handling

Case	Description
A1	Missing required fields. Backend returns HTTP 422 Unprocessable Entity with list of incomplete fields.
A2	Job is not in DRAFT status. Backend returns HTTP 400 Bad Request.

Output Data

Data Name	Description
Updated Job	Job object with `status = OPEN` and populated `publishedAt`.

Post-Conditions

- Job is public and accepting proposals.

3.7.4 Function: Update Job

Function Trigger

A client clicks "Edit" on an active or draft job.

Navigation path: Client Dashboard → My Jobs → Click "Edit" (`/client/jobs/:id/edit`).

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows clients to revise job parameters, requirements, and descriptions according to lifecycle editing rules.

Interface

- Job editing form pre-filled with existing data.
- Locked fields indicator for published jobs (budget type, category, and slug).

Data Processing Overview

- Client submits `PATCH /api/client/jobs/:id`.
- Backend checks ownership and checks whether proposals have already been submitted.
- If proposals exist, locks structural fields but allows description and skill refinements.
- Updates database record and returns modified job.

Screen Layout (Mock-up Description)

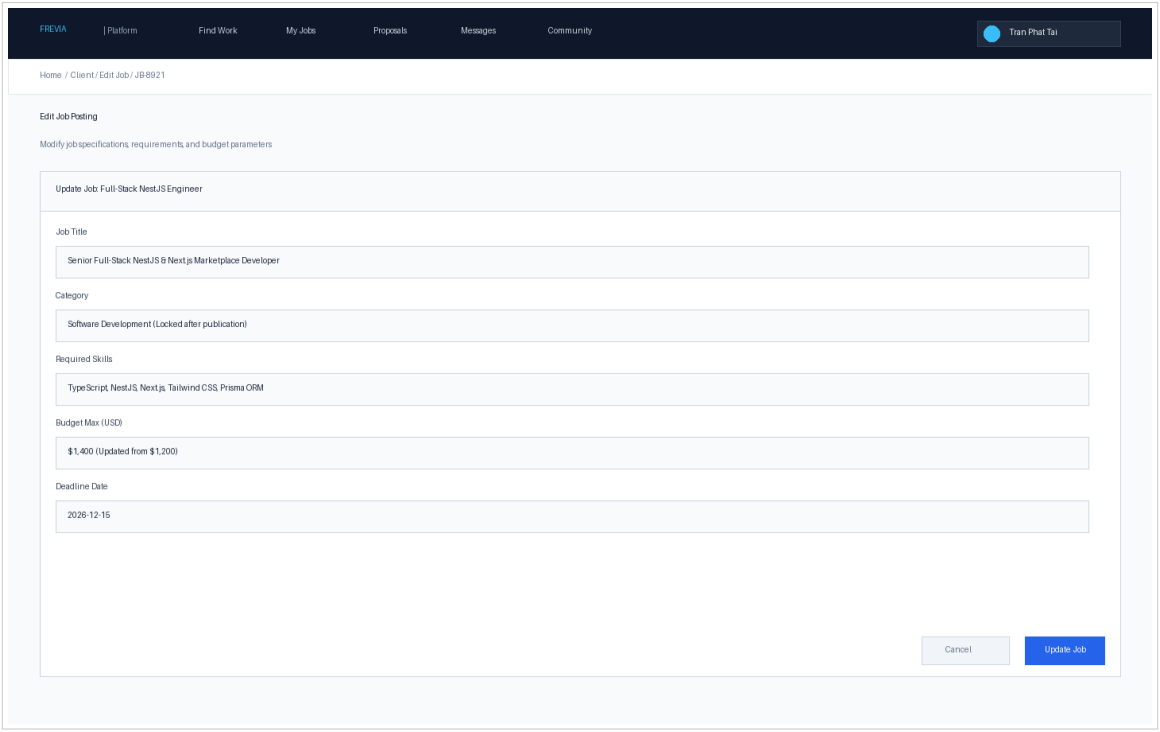

Figure 90: Edit job form interface

- Editable fields for title, skills, budget amount, and description.
- Read-only badges on locked fields (Category, Slug).
- "Update Job" submission button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the job being updated.
Payload	Modified fields (`title`, `description`, `budgetMax`, `skills`).

Validation Rules

- Updates must satisfy standard field validations.
- Locked fields cannot be modified on published jobs.

Business Rules

- **BR-J03**: Draft jobs permit editing of all fields without restriction.

- **BR-J10**: When a job is `OPEN` without proposals, client may modify description and budget.
- **BR-J11**: Modifying an open job with active proposals does not alter existing proposal terms.
- **BR-J14**: Client must own the job posting.
- **BR-J42**: The URL slug cannot be altered after initial publishing.

Normal Flow

1. Client edits desired fields on `/client/jobs/:id/edit`.
2. Client clicks "Update Job".
3. Backend processes `PATCH` request and validates permissions.
4. Changes are committed to PostgreSQL.
5. Client is returned to the job overview with success notification.

Abnormal Flow / Exception Handling

Case	Description
A1	Attempt to modify locked fields on a published job. Backend rejects with HTTP 400; displays field restriction message.
A2	Job has active contract in progress. Backend returns HTTP 409 Conflict.

Output Data

Data Name	Description
Job Object	Updated job entity reflecting modifications.

Post-Conditions

- Job updates are saved and reflected across all search and detail views.

3.7.5 Function: Delete Job

Function Trigger

A client clicks "Delete" on a draft or inactive job.

Navigation path: Client Dashboard → My Jobs → Select Job → Click "Delete".

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows clients to remove unwanted draft jobs or jobs without active contracts via a soft-delete mechanism.

Interface

- Delete action button triggering an alert confirmation dialog.
- Warning detailing the consequences of deletion.

Data Processing Overview

- Client invokes `DELETE /api/client/jobs/:id`.
- Backend confirms ownership and checks for active contracts.
- Sets `deletedAt = now()` and `status = 'CANCELLED'`.
- Removes job from public search indexes.

Screen Layout (Mock-up Description)

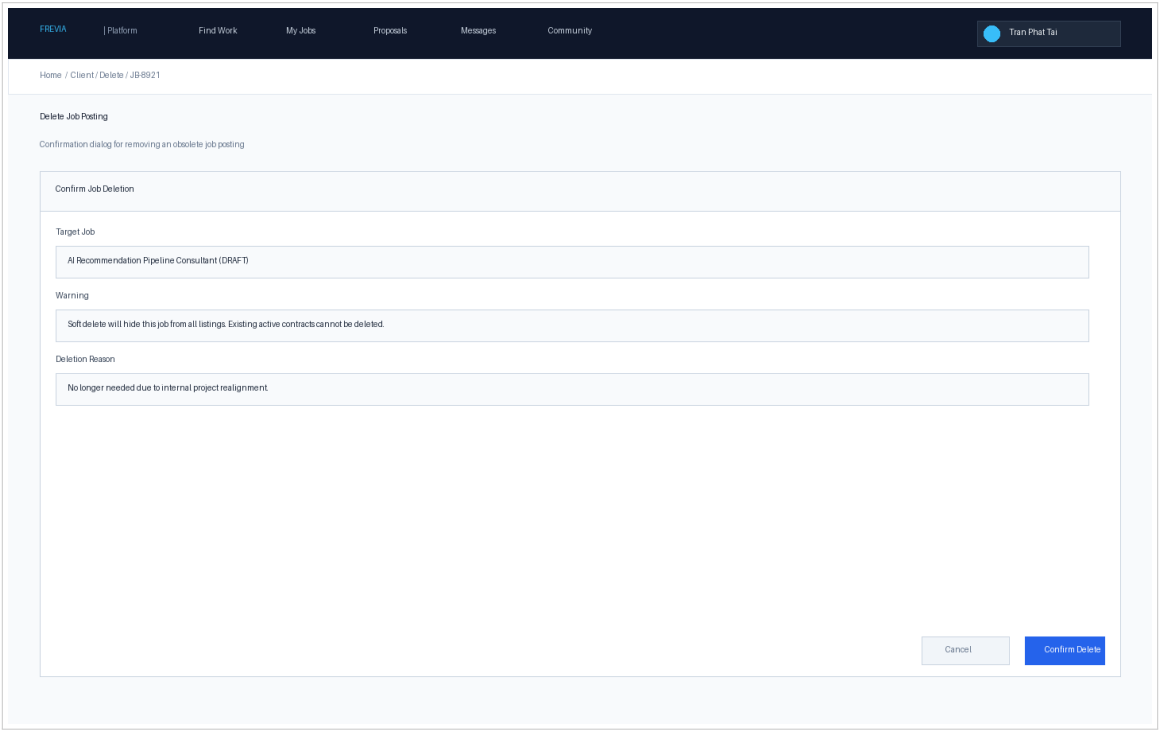

Figure 91: Delete job confirmation dialog

- Modal dialog asking for confirmation.
- Summary of the targeted job and cancellation notice.
- Destructive “Confirm Delete” button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the job to delete.

Validation Rules

- Job must belong to the calling client.
- Job must not have contracts in `ACTIVE` or `IN_PROGRESS` status.

Business Rules

- **BR-J14**: Client must own the job posting.
- **BR-J33**: Employs soft-delete by populating `deletedAt`.
- **BR-J38**: Deleting a job with active contracts is strictly prohibited.

- **BR-J39**: Submitted proposals for deleted jobs are automatically transitioned to `EXPIRED`.

Normal Flow

1. Client clicks “Delete Job”.
2. Confirmation dialog appears; client confirms.
3. Backend executes soft delete and cancels pending proposals.
4. UI removes job from listing and displays confirmation.

Abnormal Flow / Exception Handling

Case	Description
A1	Active contract exists. Backend returns HTTP 409 Conflict with message “Cannot delete a job with active contracts.”

Output Data

Data Name	Description
Result	Status confirmation confirming job deletion.

Post-Conditions

- Job is soft-deleted and omitted from all public and client views.

3.7.6 Function: Close Job

Function Trigger

A client closes an open job before the expiration deadline.

Navigation path: Client Dashboard → My Jobs → Job Menu → Click “Close Job”.

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Stops accepting new proposals on an open job posting and marks it as `CLOSED`.

Interface

- Close job confirmation dialog.
- Dropdown to specify a closure reason (e.g., filled elsewhere, requirements changed).

Data Processing Overview

- Client invokes `POST /api/client/jobs/:id/close`.
- Backend transitions status to `CLOSED` and records closure timestamp.
- Suspends all pending proposals for this job.

Screen Layout (Mock-up Description)

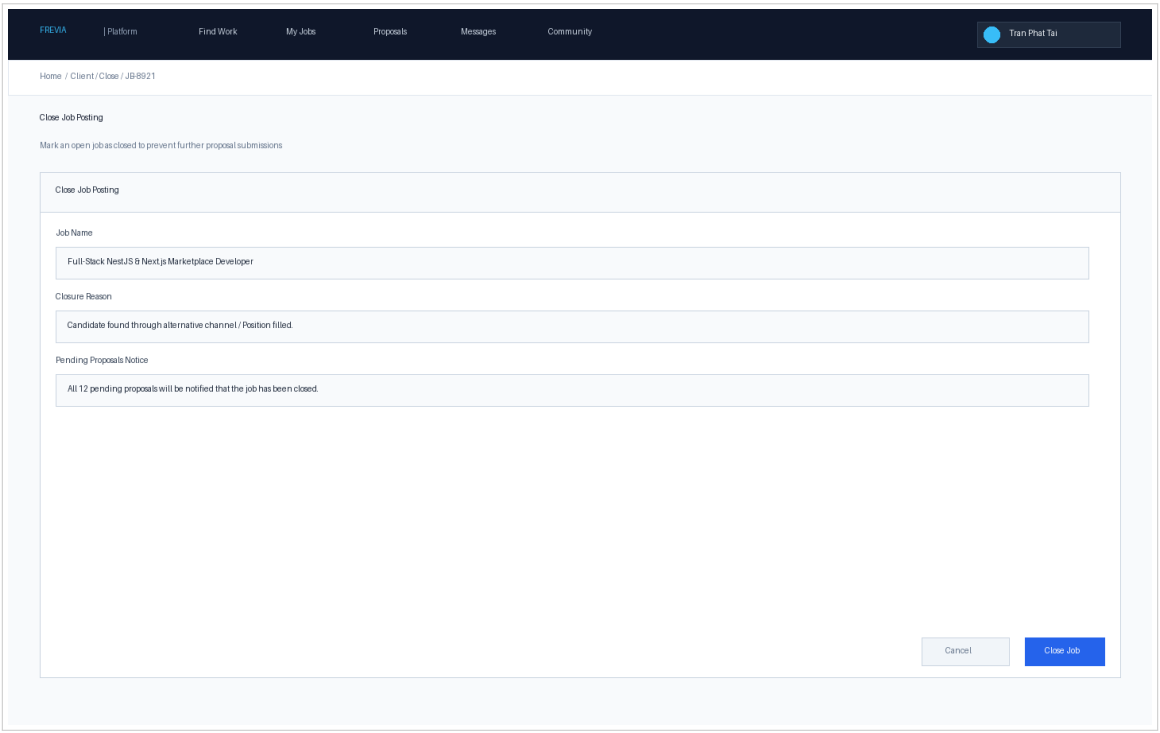

Figure 92: Close job dialog interface

- Dialog explaining the closure effect.
- Reason selection dropdown and notes field.
- “Close Job” action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the job to close.
`reason`	Optional string detailing reason for closure.

Validation Rules

- Target job must currently be in `OPEN` status.

Business Rules

- **BR-J22**: Clients may close an open job at any time prior to contract engagement.
- **BR-J25**: Closed jobs immediately disappear from the public browse jobs feed.
- **BR-J28**: Pending proposals are notified that the job has been closed.

Normal Flow

1. Client selects “Close Job” from job actions.
2. Client chooses closure reason and submits.
3. Backend sets status to `CLOSED`.
4. UI updates job status tag to CLOSED.

Abnormal Flow / Exception Handling

Case	Description
A1	Job is not in OPEN status. Backend returns HTTP 400 Bad Request.

Output Data

Data Name	Description
Updated Job	Job entity with `status = CLOSED`.

Post-Conditions

- Job ceases accepting proposals and status is marked CLOSED.

3.7.7 Function: Reopen Job

Function Trigger

A client re-activates a previously closed job.

Navigation path: Client Dashboard → My Jobs → Closed Jobs → Click "Reopen Job".

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows clients to reactivate a closed job by setting a new deadline and transitioning its status back to `OPEN`.

Interface

- Reopen job dialog requesting a new expiration date.

Data Processing Overview

- Client submits `POST /api/client/jobs/:id/reopen` with new `expiryDate`.
- Backend verifies ownership and confirms previous status was `CLOSED`.
- Sets `status = 'OPEN'` and restores job to search listings.

Screen Layout (Mock-up Description)

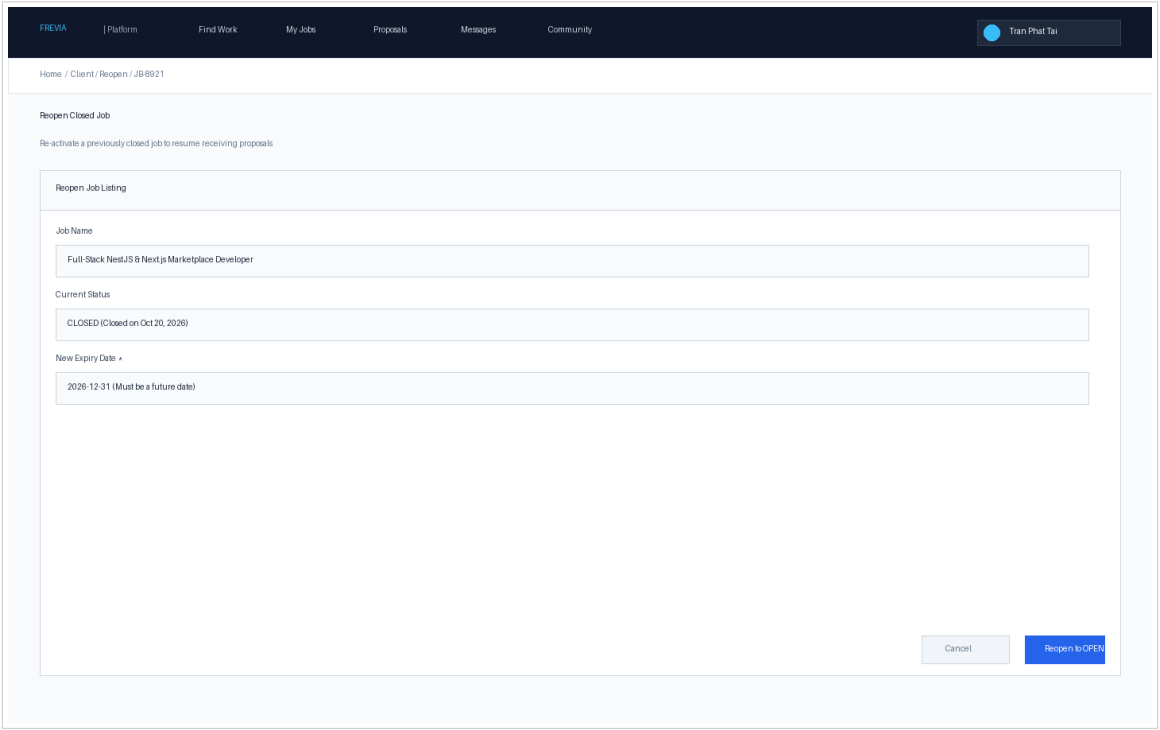

Figure 93: Reopen job interface

- Modal dialog displaying closed job title and prompt for new expiry date.
- Date picker for selecting future expiration date.
- Primary “Reopen to OPEN” action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the closed job.
`expiryDate`	Date timestamp in the future indicating new deadline.

Validation Rules

- Job status must be `CLOSED`.
- `expiryDate` must be strictly in the future.

Business Rules

- **BR-J26**: Only jobs with status `CLOSED` can be reopened.

- **BR-J27**: Reopening a job does not revive old withdrawn or rejected proposals.
- **BR-J46**: New expiration date must be established upon reopening.

Normal Flow

1. Client clicks “Reopen Job” on a closed posting.
2. Client picks a new valid deadline and confirms.
3. Backend sets status to `OPEN` with updated expiration.
4. UI returns job to the active listings feed.

Abnormal Flow / Exception Handling

Case	Description
A1	Provided expiration date is in the past. Backend returns HTTP 400 Bad Request.

Output Data

Data Name	Description
Reopened Job	Job entity with `status = OPEN` and updated `expiryDate`.

Post-Conditions

- Job is restored to `OPEN` status and visible in search.

3.8 Bookmarked Jobs Management

3.8.1 Function: View Bookmarked Job List

Function Trigger

A freelancer clicks “Saved Jobs” from the workspace menu.

Navigation path: Freelancer Dashboard → Saved Jobs (`/freelancer/bookmarks`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Displays the list of job postings bookmarked by the freelancer, showing real-time job availability status, urgency indicators, and shortcuts to apply or remove bookmarks.

Interface

- Saved jobs catalog page at `/freelancer/bookmarks`.
- Filter tabs (All, Available, Expiring Soon, Closed).
- Job cards with bookmark timestamp, client rating, and action buttons.

Data Processing Overview

- Frontend dispatches `GET /api/freelancer/bookmarks`.
- Backend fetches `JobBookmark` records where `userId === currentUserId`.
- Joins active job data, checks if jobs are expiring within 24 hours, and sorts by bookmark date.
- Excludes soft-deleted jobs or jobs belonging to banned clients.

Screen Layout (Mock-up Description)

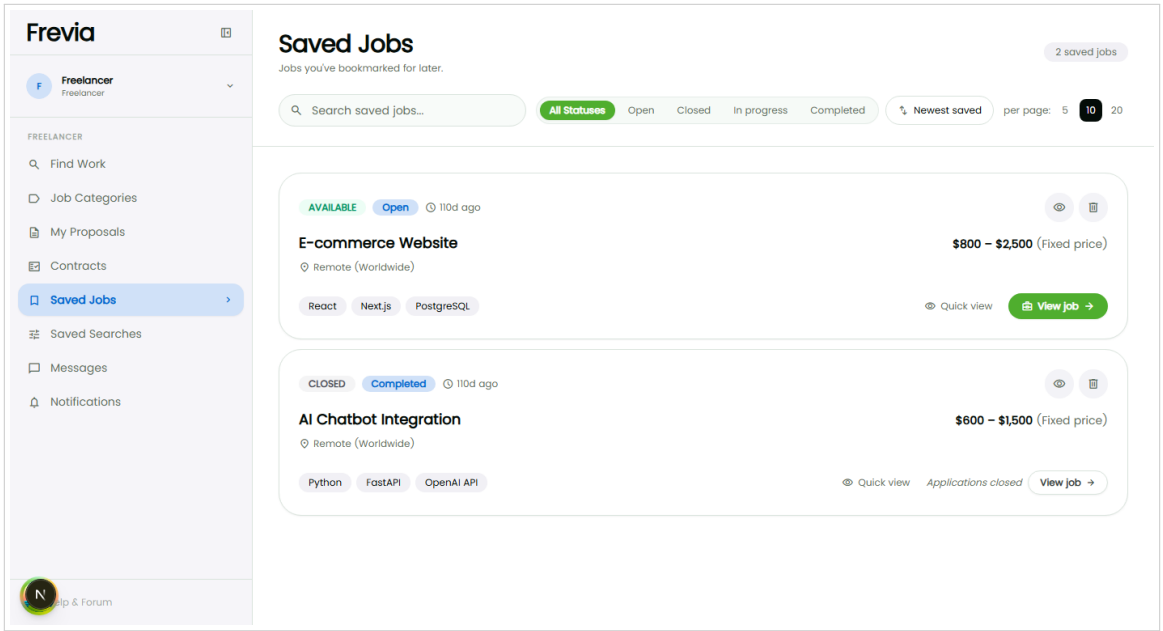

Figure 94: Saved jobs list interface

- Search and filter toolbar for bookmarked items.
- Cards displaying job title, current status badge (AVAILABLE, EXPIRING SOON, CLOSED), and client info.
- Action buttons on each card to “Apply Now” or “Remove Bookmark”.

Function Details

Input Data

Data Name	Description
JWT Token	Authenticated freelancer token.
`page` / `limit`	Standard pagination integers.

Validation Rules

- User must possess the `FREELANCER` role.

Business Rules

- **BR-J54-01**: Only authenticated freelancers may view their own bookmarked jobs list.
- **BR-J54-02**: Bookmarks are preserved even if the underlying job transitions to closed or completed.
- **BR-J54-03**: The Apply button is disabled for jobs that are no longer in `OPEN` status.
- **BR-J54-04**: Excludes soft-deleted jobs or jobs posted by banned clients.

Normal Flow

1. Freelancer opens Saved Jobs page.
2. Frontend requests bookmark records from backend.
3. Backend returns populated bookmarks with current job statuses.
4. UI renders list with status badges and appropriate action controls.

Abnormal Flow / Exception Handling

Case	Description
A1	No jobs bookmarked yet. UI renders friendly empty state with a “Browse Jobs” shortcut.

Output Data

Data Name	Description
`bookmarks`	Array of objects containing bookmark ID, `savedAt`, and embedded job details.

Post-Conditions

- Freelancer views saved jobs portfolio.

3.8.2 Function: Bookmark Job

Function Trigger

A freelancer clicks the bookmark / heart icon on a job card or detail screen.

Navigation path: Browse Jobs → Job Card → Click Bookmark Icon.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Saves a job posting to the freelancer's personal bookmark list for easy retrieval and tracking.

Interface

- Interactive bookmark toggle button on job cards and detail headers.
- Toast notification confirming bookmark creation.

Data Processing Overview

- Client sends `POST /api/freelancer/bookmarks/:jobId`.
- Backend validates role, checks job exists, and ensures viewer is not the job's author.
- Creates `JobBookmark` record with unique pair `(userId, jobId)`.
- Returns success response.

Screen Layout (Mock-up Description)

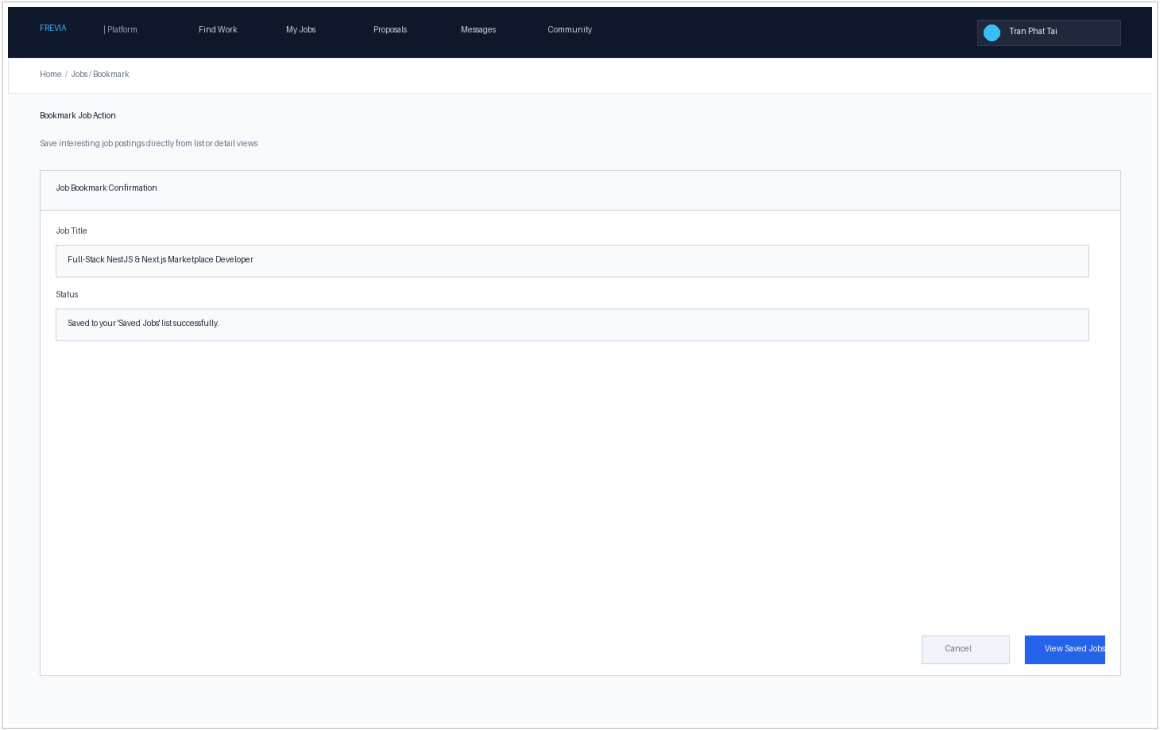

Figure 95: Bookmark job confirmation interface

- Bookmark icon transitioning from outline to solid fill state.
- Toast notification popup stating “Job saved to bookmarks”.

Function Details

Input Data

Data Name	Description
`jobId`	UUID of the job to bookmark.

Validation Rules

- User must be a verified Freelancer.
- Target job must be currently `OPEN`.

Business Rules

- **BR-J54-05**: Only freelancers can bookmark jobs; guests are prompted to log in.
- **BR-J54-06**: Clients cannot bookmark jobs they posted themselves.
- **BR-J54-07**: Duplicate bookmark requests are safely ignored via unique constraint.

Normal Flow

1. Freelancer clicks bookmark icon on a job card.
2. Frontend optimistically updates icon and sends `POST` request.
3. Backend stores bookmark record in PostgreSQL.
4. Toast notification confirms that the job has been bookmarked.

Abnormal Flow / Exception Handling

Case	Description
A1	Freelancer tries to bookmark their own job. Backend returns HTTP 400 Bad Request.
A2	Job is already closed or deleted. Backend returns HTTP 400 with "Job is no longer available."

Output Data

Data Name	Description
Bookmark Object	Created record with `id`, `userId`, `jobId`, and `createdAt`.

Post-Conditions

- Job is saved in freelancer's bookmarks list.

3.8.3 Function: Remove Bookmark

Function Trigger

A freelancer clicks the active bookmark icon or "Remove" button on a saved job.
Navigation path: Saved Jobs (`/freelancer/bookmarks`) → Click "Remove Bookmark".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Removes a previously saved job from the freelancer's bookmarks collection.

Interface

- Toggle bookmark icon or “Remove” button on saved job cards.
- Instant optimistic UI removal and feedback toast.

Data Processing Overview

- Client issues `DELETE /api/freelancer/bookmarks/:jobId`.
- Backend verifies that the bookmark exists and belongs to the authenticated freelancer.
- Hard-deletes the record from `JobBookmark` table.

Screen Layout (Mock-up Description)

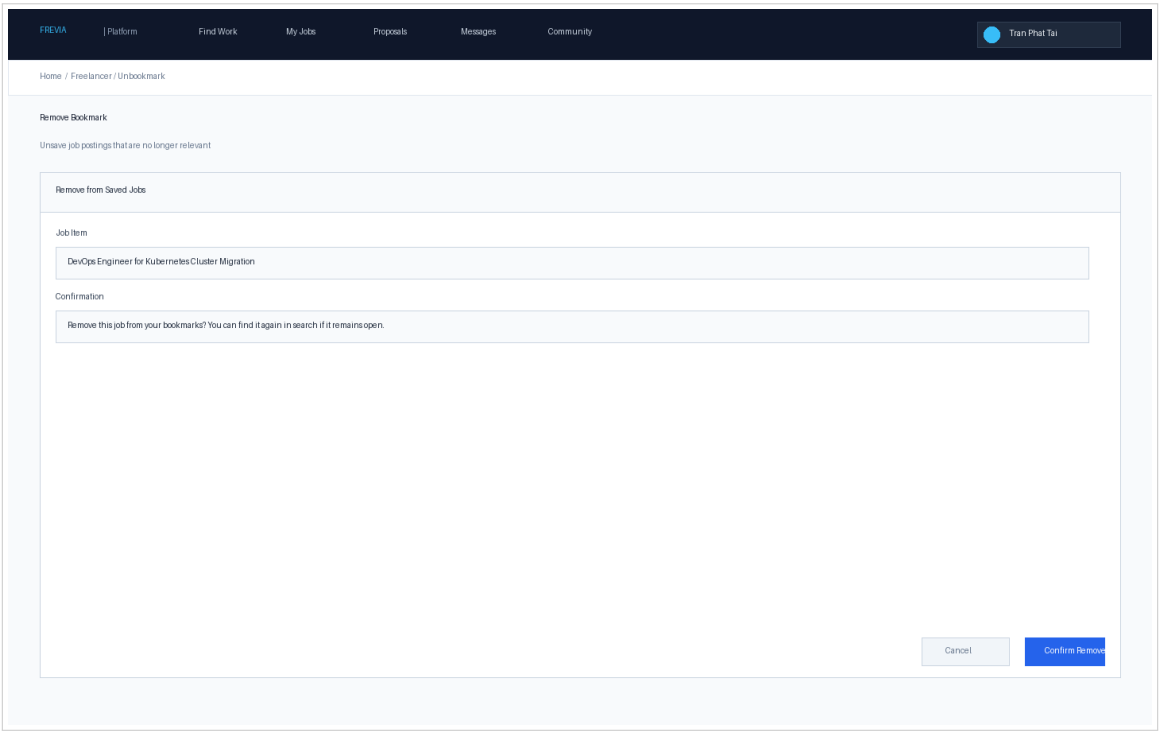

Figure 96: Remove bookmark confirmation interface

- Card showing item to be removed.
- Toast confirmation indicating “Job removed from saved list”.

Function Details

Input Data

Data Name	Description
`jobId`	UUID of the job to unbookmark.

Validation Rules

- Bookmark must exist for current user and target job.

Business Rules

- **BR-J54-08**: Freelancers can remove any bookmark they previously created at any time.
- **BR-J54-09**: Removal is unconditional, regardless of the job's current status.

Normal Flow

1. Freelancer clicks "Remove Bookmark".
2. UI immediately removes card from display.
3. Backend deletes bookmark row from PostgreSQL.
4. Toast notification confirms removal.

Abnormal Flow / Exception Handling

Case	Description
A1	Bookmark record not found. Backend returns HTTP 404; UI refreshes list.

Output Data

Data Name	Description
Result	Status confirmation confirming bookmark deletion.

Post-Conditions

- Bookmark is removed and item no longer appears in saved jobs.

3.9 Permission Management

3.9.1 View Permission List

Function Trigger

An administrator accesses the Permission Management screen from the administrator console sidebar.
Navigation path: Admin Dashboard → Settings → Permissions (`/admin/permissions`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to retrieve and inspect the complete catalog of route-level system permissions defined in the application. This provides transparent visibility into available endpoints, HTTP methods, and module groupings necessary for configuring custom role-based access control (RBAC) policies.

Interface

- Admin portal screen at `/admin/permissions`.
- Top filter bar featuring keyword search input, HTTP method filter dropdown, and module filter dropdown.
- Data table displaying permission ID, endpoint name, route path, HTTP method badge, module badge, creation date, and row action links.
- Pagination footer indicating total count and page selection controls.

Data Processing Overview

- The client initiates an HTTP request: `GET /api/permissions` with query parameters (`page`, `limit`, `search`, `method`, `module`, `sortBy`, `order`).
- The backend guard verifies the caller's JWT access token and ensures the caller possesses `ADMIN` role privileges.
- `PermissionsController.getPermissions()` parses and validates the filter parameters through `PermissionFilterDto`.
- `PermissionsService.getPermissions()` invokes `PermissionsRepository.findAll(filter)`.
- The repository executes a database transaction retrieving active records (`deletedAt = null`), counting the matching records, and fetching distinct module names.
- The service formats the payload into a `PermissionListResponseDto` containing the

items array, pagination metadata, and active module list.

- The frontend renders the data table, applies color badges per HTTP method (GET: outline, POST: secondary, PUT/PATCH: outline, DELETE: destructive), and renders pagination buttons.

Screen Layout (Mock-up Description)

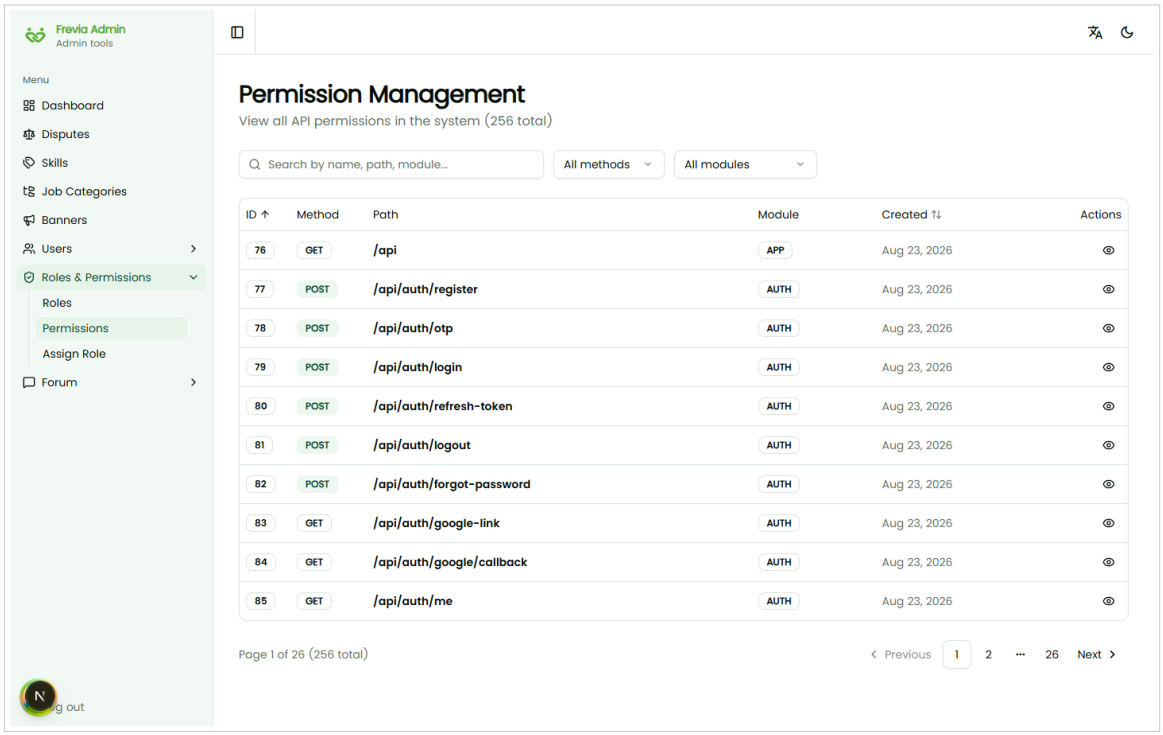

Figure 97: Permission list interface

- The top header displays the section title “Permissions” alongside a brief summary description.
- A filter bar contains a search input box with an icon, an HTTP method selector, and a module category dropdown.
- The table columns include: **ID**, **Name**, **Path**, **Method**, **Module**, **Created Date**, and **Actions**.
- Each HTTP method is visually differentiated with a dedicated colored badge for immediate visual recognition.
- A “View detail” icon button on each row directs the administrator to the detailed permission inspection view.
- The pagination toolbar displays the current page interval, total records found, and previous/next page navigation buttons.

Function Details

Input Data

Data Name	Description
page	Target page number for paginated retrieval (defaults to 1).
limit	Maximum number of permission records returned per page (defaults to 10).
search	Optional text query matching permission name, API route path, or module name.
method	Optional filter matching HTTP method (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`).
module	Optional filter matching functional module name (case-insensitive).
sortBy	Sorting column field name (`id` or `createdAt`, defaults to `id`).
order	Sorting sequence direction (`asc` or `desc`, defaults to `asc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters; whitespace is automatically trimmed.
method	Must match one of the standard HTTP methods: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`.
module	String, optional, max length 50 characters.
sortBy	Must strictly be one of: `id`, `createdAt`.
order	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-PERM-01**: Only active permissions with `deletedAt = null` shall be returned to the administrator.

- **BR-PERM-02**: Keyword search queries shall match `name`, `path`, or `module` case-insensitively using partial text matching.
- **BR-PERM-03**: The method filter shall filter records by exact HTTP verb equality.
- **BR-PERM-04**: The module filter shall perform case-insensitive exact matching against the permission module field.
- **BR-PERM-05**: The response must return distinct existing module names to dynamically populate client filter options.

Normal Flow

1. The administrator navigates to the "Permissions" page in the admin console.
2. The frontend issues `GET /api/permissions` with default query parameters (`page=1&limit=10&sortBy=`
3. The system verifies the administrator's JWT access token and validates query parameters.
4. `PermissionsService` queries `PermissionsRepository.findAll(filter)`.
5. The database returns matching permission rows, the total record count, and distinct module names.
6. The system returns HTTP 200 with the permission list, pagination metadata, and module list.
7. The frontend renders the data table, method badges, module badges, and pagination controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The API returns HTTP 401 Unauthorized. The client clears cached credentials and redirects the administrator to `/login`.
User lacks Admin privileges	The API returns HTTP 403 Forbidden. The interface displays an access denied toast: "You do not have permission to access this page."

Abnorma Case	Handling & System Response
Invalid pagination parameter (`page < 1` or non-integer)	The DTO validator rejects the request with HTTP 400 Bad Request. The frontend resets to page 1 automatically.
Invalid limit parameter (`limit > 100` or `limit < 1`)	The API rejects the request with HTTP 400 Bad Request. The client defaults to a safe limit value of 10.
Unsupported HTTP method filter string	The system returns HTTP 400 Bad Request. The UI clears the invalid filter selection and reloads the default list.
Database connection failure or query timeout	The backend logs the exception and throws `FailedToLoadPermissionsException` (HTTP 500). The UI displays: "Couldn't load permissions. Try again."
No permissions match active search/filters	The repository returns an empty collection (`permissions: []`, `total: 0`). The table displays a friendly empty-state notice: "No permissions match your filters."

Output Data

Output Field	Description
permissions	Array of permission objects containing `id`, `name`, `path`, `method`, `module`, and `createdAt`.
pagination.page	Current page number.
pagination.limit	Current page size limit.
pagination.total	Total count of matching permissions.
pagination.totalPages	Total calculated pages available.
modules	Array of unique module names available across active permissions.

Post-Conditions

1. The permission catalog table is displayed with accurate HTTP badges and module tags.
2. The module filter dropdown is populated with distinct module names retrieved from the server.
3. The administrator can sort, filter, or select any permission row to view its details.

3.9.2 View Permission Detail

Function Trigger

An administrator clicks "View detail" on a permission row in the permission table or navigates directly to a permission detail URL.
Navigation path: Admin Dashboard → Settings → Permissions → Click "View detail" icon on a row (`/admin/permissions/:id`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view comprehensive technical details of a specific system permission, including its full route path, allowed HTTP method verb, associated functional module, system identifier, and exact creation timestamp.

Interface

- Detailed permission overview page at `/admin/permissions/:id`.
- Top navigation breadcrumb with “Back to permissions” button.
- Header displaying the permission endpoint name and an HTTP verb badge.
- Information card containing structured field entries: ID, Creation timestamp, HTTP Method, Module name, and Endpoint Route Path.

Data Processing Overview

- The client sends an HTTP request: `GET /api/permissions/:id` with the numeric permission ID.
- The NestJS `ParseIntPipe` validates that `id` is an integer.
- The permission guard verifies valid Admin credentials.
- `PermissionsController.getPermissionById(id)` delegates execution to `PermissionsService.get`
- `PermissionsRepository.findById(id)` queries `prisma.permission.findFirst` where `id = id` and `deletedAt = null`.
- If no record is found, the repository throws `PermissionNotFoundException` (HTTP 404).
- The server responds with HTTP 200 and the complete permission detail object.
- The web application renders the information card and formats dates using localized formatters.

Screen Layout (Mock-up Description)

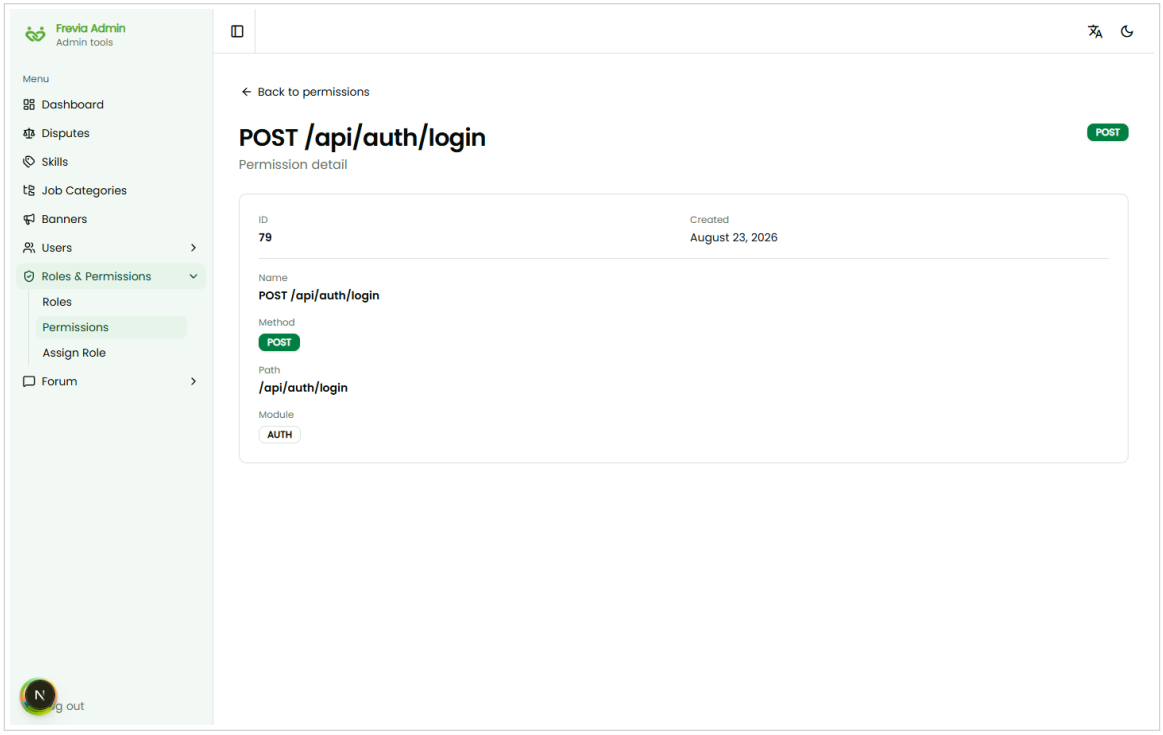

Figure 98: Permission detail interface

- A top “Back to permissions” navigation button with an arrow icon allows returning to the permission catalog.
- The header section displays the permission name in prominent monospace typography alongside the colored HTTP method badge (e.g., `POST`).
- A structured card contains labeled sections: **ID**, **Created Date**, **Method**, **Module**, and **Path**.
- The API route path is formatted in a distinct highlighted monospace container for easy inspection.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the permission passed in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0; non-numeric inputs are rejected by `ParseIntPipe`.

Business Rules

- **BR-PERM-06**: Only active permissions with `deletedAt = null` are retrievable. Soft-deleted permissions return HTTP 404 Not Found.
- **BR-PERM-07**: The returned payload must include `id`, `name`, `path`, `method`, `module`, and `createdAt`.
- **BR-PERM-08**: Direct URL access to permission details requires an active Admin session with proper authorization.

Normal Flow

1. The administrator clicks “View detail” on a permission row or accesses `/admin/permissions/:id`.
2. The frontend validates the ID and sends `GET /api/permissions/:id`.
3. The system validates authorization and executes `PermissionsRepository.findById(id)`.
4. The database retrieves the permission record where `deletedAt = null`.
5. The server returns HTTP 200 with the permission detail payload.

6. The frontend renders the complete permission details and navigation controls.

Abnormal Flow / Exception Handling

Abnormal Case	Handling & System Response
Invalid ID format (non-numeric string in URL)	NestJS `ParseIntPipe` rejects the request with HTTP 400 Bad Request. The frontend renders an invalid parameter error page.
Permission ID does not exist in database	The repository throws `PermissionNotFoundException` (HTTP 404). The client displays: "Failed to load permission detail. The permission may not exist."
Permission was soft-deleted (`deletedAt != null`)	The database query filters out soft-deleted records and throws HTTP 404 Not Found. The UI displays the not found notice.
Missing or expired administrator token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User account lacks Admin role privileges	The server returns HTTP 403 Forbidden. The client shows an access restriction notice.

Abnorma Case	Handling & System Response
Database query failure during retrieval	The service catches the Prisma error and throws `FailedToLoadPermissionDetailException` (HTTP 500). The UI displays a retry button.

Output Data

Output Field	Description
id	Unique numeric system identifier of the permission.
name	Functional descriptive name of the permission.
path	Exact API route path pattern (e.g., `/api/auth/login`).
method	HTTP method verb (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`).
module	Module grouping name to which the permission belongs.
createdAt	Timestamp indicating when the permission was registered.

Post-Conditions

1. The complete details of the selected permission are presented to the administrator.
2. The administrator can navigate back to the permission list using the top navigation link.

3.10 Session Management

3.10.1 View Session List

Function Trigger

An authenticated user accesses the active sessions management screen from their account settings.
Navigation path: User Avatar Menu → Settings → Active Sessions (`/sessions`).

Function Description

Actors / Roles

- **Primary Actor:** Any authenticated user (`Client`, `Freelancer`, `Expert`, or `Admin`).

- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows users to view all active authentication sessions associated with their account. This enables users to monitor which devices, browsers, and IP addresses have access to their account, verify when each session was initiated and when it will expire, and identify potentially suspicious or obsolete logins.

Interface

- Dedicated session management screen at `/sessions`.
- Top control bar with keyword search input (supporting IP address or browser/device search) and sort selectors.
- Responsive session list table showing Device Information, IP Address, Creation Date, Expiration Date, Status, and Action buttons.
- Visual "Current session" badge highlighting the device currently being used.
- Pagination bar at the bottom with page numbers and navigation links.

Data Processing Overview

- The client sends `GET /api/sessions` with query parameters (`page`, `limit`, `search`, `sortBy`, `order`) and the user's Bearer JWT access token.
- The authentication guard verifies the JWT token and extracts `userId` and `sessionId`.
- `SessionsController.getMySessions()` invokes `SessionsService.getMySessions(userId, query, sessionId)`.
- `SessionsRepository.findAllByUserId()` queries the database where `userId = userId`, applying optional search filters on `ipAddress` or `deviceInfo`.
- The repository maps each record and dynamically sets `isCurrent = (row.id === sessionId)`.
- The backend returns HTTP 200 with the sessions collection and pagination metadata.
- The frontend renders the list, marking active sessions, indicating the current session, and providing detail inspection and revocation dialog triggers.

Screen Layout (Mock-up Description)

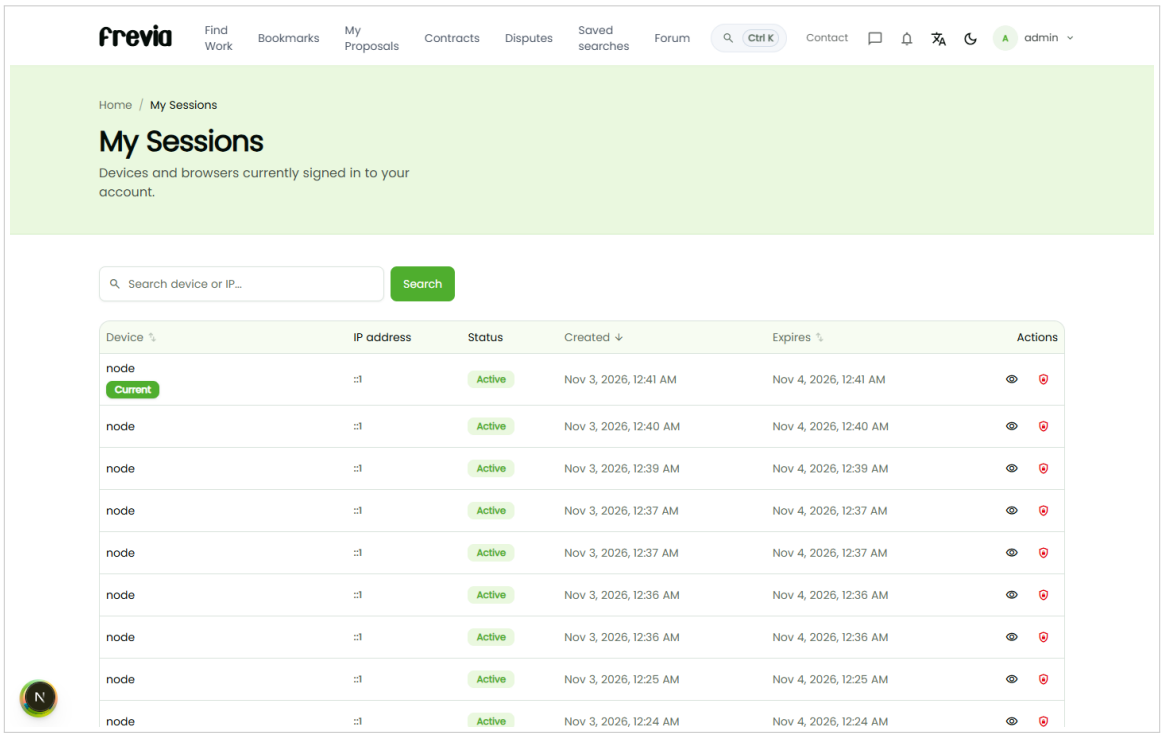

Figure 99: Active sessions list interface

- Header section displaying “Active sessions” with a descriptive subtitle.
- Search input box enabling instantaneous filtering by IP address or browser/device string.
- Table columns: **Device**, **IP Address**, **Created**, **Expires**, **Status**, and **Actions**.
- The currently active session displays a green “Current” badge, distinguishing it from remote sessions.
- Action icons include an eye icon (view detail modal) and a trash/revoke icon for remote session termination.
- A pagination component at the bottom provides direct page number navigation.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum session records per page (defaults to 10).
search	Optional query matching client IP address or user-agent device string.
sortBy	Sort column field (`id`, `createdAt`, or `expiresAt`; defaults to `createdAt`).
order	Sort direction (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max 100 characters; whitespace trimmed.
sortBy	Must strictly be one of: `id`, `createdAt`, `expiresAt`.
order	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-SES-01**: Users can only view sessions belonging strictly to their own authenticated `userId`. Cross-account session retrieval is forbidden.
- **BR-SES-02**: The `isCurrent` indicator is true if and only if the session ID in the database matches the `sessionId` embedded in the caller's JWT access token.
- **BR-SES-03**: Search queries must perform case-insensitive partial matching against both `ipAddress` and `deviceInfo`.
- **BR-SES-04**: Session expiration status is evaluated dynamically: a session is active when `expiresAt > now`, and expired otherwise.
- **BR-SES-05**: Pagination parameters must be strictly validated with a maximum limit of 100 records per page.

Normal Flow

1. The user navigates to the "Active sessions" section in account settings.
2. The client sends `GET /api/sessions` with authorization headers.
3. The backend validates the JWT token, extracts `userId` and `sessionId`, and executes `SessionsRepository.findAllByUserId()`.

4. The database retrieves all sessions belonging to the user and calculates total count.
5. The backend attaches `isCurrent` flags and returns HTTP 200 with the session list and pagination data.
6. The frontend renders the session list table, highlighting the current device with a badge.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or expired JWT access token	The server returns HTTP 401 Unauthorized. The client removes stored session tokens and redirects the user to `/login`.
Current session was revoked remotely from another device	Token validation detects that the session ID no longer exists in the database. The system returns HTTP 401 Unauthorized and prompts the user to log in again.
User account has been banned or deactivated	The authentication guard verifies account status, detects inactive status, and throws HTTP 403 Forbidden. The client redirects to a ban notification screen.
Invalid pagination parameter values	The DTO validator rejects non-positive or malformed parameters with HTTP 400 Bad Request. The client falls back to page 1 and default limit.

Abnorma Case	Handling & System Response
Unsupported sort column parameter	The API rejects the request with HTTP 400 Bad Request. The client resets the query to sort by `createdAt desc`.
Database query failure during retrieval	The service catches the error, logs the failure, and throws `FailedToLoadSessionsException` (HTTP 500). The UI displays: "Failed to load sessions."
No sessions match current search keyword	The query returns an empty collection (`sessions: []`, `total: 0`). The table displays: "No sessions match your search."

Output Data

Output Field	Description
sessions	Array of session objects: `id`, `userId`, `deviceInfo`, `ipAddress`, `expiresAt`, `createdAt`, and `isCurrent`.
pagination.page	Current page number.
pagination.limit	Current page size.
pagination.total	Total active and stored sessions for the user.
pagination.totalPages	Total calculated pages available.

Post-Conditions

1. The user's active session list is displayed accurately.
2. The current device session is visibly demarcated from other remote logins.
3. The user can click to inspect session details (UC-16.02) or revoke remote sessions (UC-16.03).

3.10.2 View Session Detail

Function Trigger

An authenticated user clicks the "View detail" (eye icon) button on any session row in the session list.
Navigation path: Active Sessions (`/sessions`) → Click "View detail" icon on a session row.

Function Description

Actors / Roles

- **Primary Actor:** Any authenticated user (`Client`, `Freelancer`, `Expert`, or `Admin`).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Displays full, in-depth technical information regarding a specific login session in a modal dialog. This helps the user inspect the complete browser user-agent, full IP address, creation timestamp, expiration date, active/expired status, and whether the session represents their current terminal.

Interface

- Interactive modal dialog titled "Session details" appearing over the sessions page.
- Structured list displaying: Session ID, Device/Browser User-Agent, IP Address, Login Timestamp, Expiration Timestamp, and Status Badge.
- Close button and direct action button to revoke the session if remote.

Data Processing Overview

- The client triggers an API call: `GET /api/sessions/:id` passing the session ID and Bearer token.
- The backend extracts `userId` and `sessionId` from the caller's JWT token.
- `SessionsController.getMySessionById()` calls `SessionsService.getMySessionById(id, userId, sessionId)`.
- `SessionsRepository.findByIdForUser()` queries `prisma.session.findFirst` where `id = id` and `userId = userId`.
- If no record is found, the system throws `SessionNotFoundException` (HTTP 404).
- The repository evaluates `isCurrent = (id === sessionId)` and returns the session object.
- The client populates the dialog with formatted dates and status indicators.

Screen Layout (Mock-up Description)

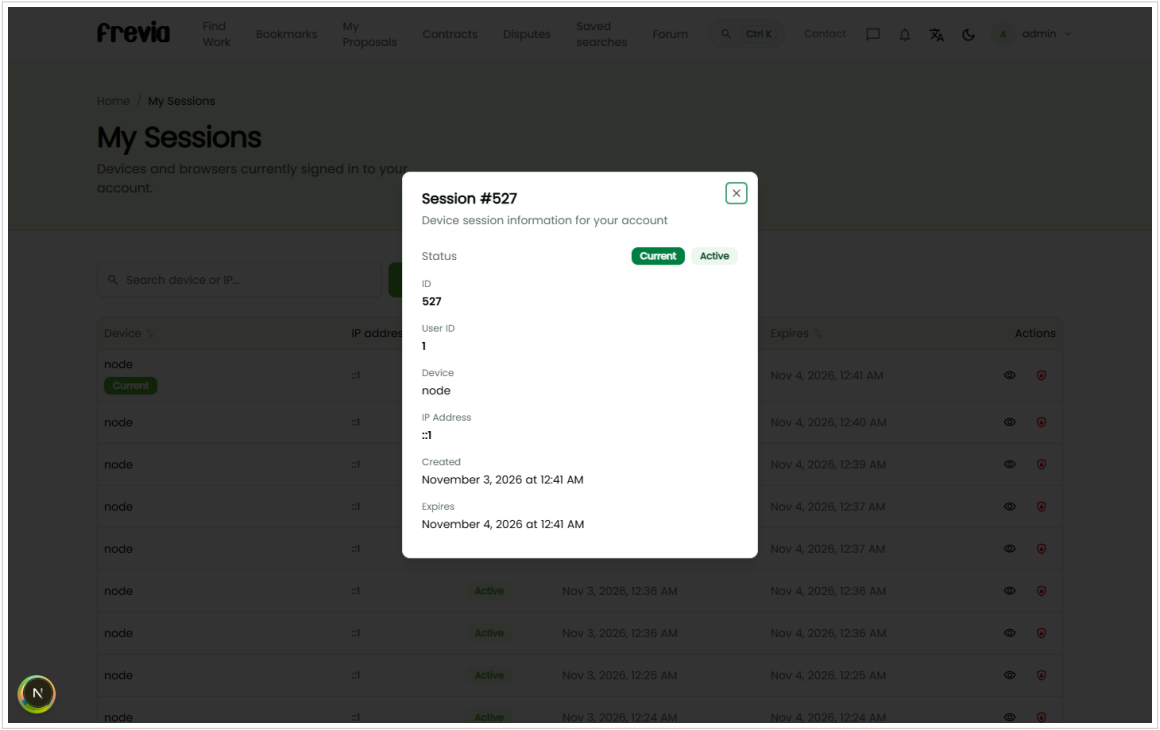

Figure 100: Session detail dialog interface

- A centered modal dialog titled “Session #{id}” with description “Detailed information about this login session.”
- A top badge indicating either “Current device” (green) or “Active” / “Expired”.
- Detailed metadata fields displayed in a clean grid: **Device / Browser**, **IP Address**, **Signed In**, and **Expires At**.
- A bottom “Close” button to dismiss the dialog.

Function Details

Input Data

Data Name	Description
id	Unique numeric ID of the session passed as a URL route parameter.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer; non-numeric values are rejected by `ParseIntPipe`.

Business Rules

- **BR-SES-06**: A user can strictly view only session records that they own (`session.userId == caller.userId`). Attempting to access another user's session returns HTTP 404 Not Found to prevent ID enumeration.
- **BR-SES-07**: The returned payload must include `id`, `userId`, `deviceInfo`, `ipAddress`, `expiresAt`, `createdAt`, and `isCurrent`.

Normal Flow

1. The user clicks the view detail icon on a session row.
2. The frontend opens the modal dialog and sends `GET /api/sessions/:id`.
3. The backend validates authentication and queries the database for the session matching `id` and `userId`.
4. The database returns the session record.
5. The backend marks `isCurrent` and responds with HTTP 200.
6. The frontend renders the complete session details in the dialog.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Invalid session ID format in URL	`ParseIntPipe` throws HTTP 400 Bad Request. The frontend displays an invalid parameter alert.

Abnorma Case	Handling & System Response
Session does not exist or has been deleted	The service throws `SessionNotFoundException` (HTTP 404). The modal displays: “Session not found.”
Session belongs to another user account	The database query filters by caller `userId`; matching returns null and throws HTTP 404 Not Found, preventing cross-user session probing.
Caller authen-tication token expired	The server returns HTTP 401 Unauthorized. The dialog closes and the user is redirected to `/login`.
Database query error during lookup	The backend catches the Prisma exception and throws `FailedToLoadSessionDetailException` (HTTP 500). The modal shows: “Failed to load session details.”

Output Data

Output Field	Description
id	Numeric system identifier of the session.
userId	Numeric identifier of the owning user.
deviceInfo	Complete user-agent string describing the operating system and browser.
ipAddress	Client IP address recorded at login or refresh time.
expiresAt	Exact timestamp when the session’s refresh token expires.
createdAt	Exact timestamp when the session was initiated.

Output Field	Description
isCurrent	Boolean flag indicating whether this is the caller's active terminal.

Post-Conditions

1. Detailed session information is visible to the user.
2. The user may proceed to revoke the session or close the dialog without side effects.

3.10.3 Revoke Session

Function Trigger

An authenticated user clicks the "Revoke" (trash or shield-off icon) button on an active session in the sessions list.
Navigation path: Active Sessions (`/sessions`) → Click "Revoke" button on a session row.

Function Description

Actors / Roles

- **Primary Actor:** Any authenticated user (`Client`, `Freelancer`, `Expert`, or `Admin`).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows users to remotely terminate an active login session on a specific device. Upon revocation, the stored refresh token record is deleted immediately from the database, preventing that device from obtaining new access tokens and forcing it to log out.

Interface

- Revocation trigger button on session table rows.
- Confirmation modal dialog titled "Revoke session" explaining that the target device will be signed out.
- "Cancel" and destructive "Revoke session" confirmation buttons.
- Toast notification confirming successful session revocation.

Data Processing Overview

- User confirms revocation in the alert dialog.

- The client issues an HTTP request: `DELETE /api/sessions/:id` with Bearer token.
- `SessionsController.revokeMySession()` passes `id`, `userId`, and `sessionId` to `SessionsService.revokeMySession()`.
- `SessionsRepository.deleteActiveByIdForUser()` verifies the session belongs to `userId` and is active (`expiresAt > now`).
- The repository deletes the session row from the PostgreSQL `Session` table.
- The service checks whether `id === sessionId`: if true, `loggedOut = true` is returned; otherwise `loggedOut = false`.
- The server responds with HTTP 200: `{ message: 'Session revoked successfully', loggedOut }`.
- If `loggedOut = true`, the client purges local tokens and redirects to `/login`; otherwise, the UI displays a success toast and refreshes the session table.

Screen Layout (Mock-up Description)

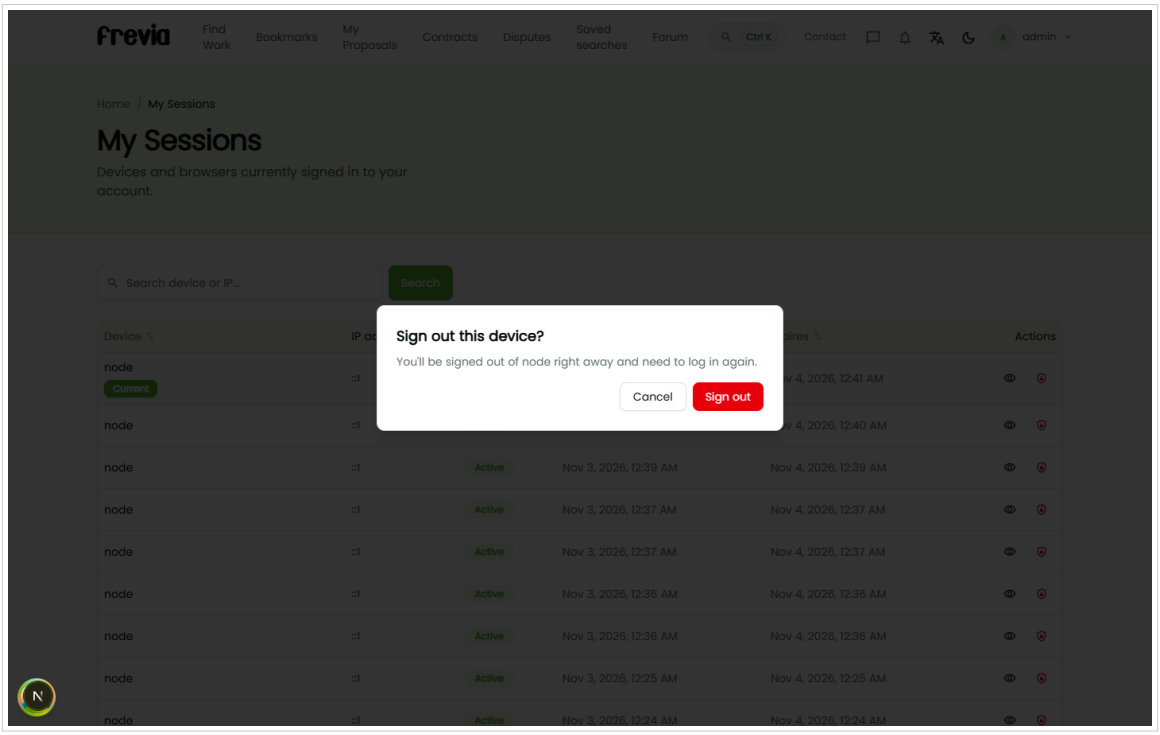

Figure 101: Revoke session confirmation dialog

- An alert dialog overlay displaying the title “Revoke session?”.
- A warning message: “This device will be logged out immediately. Any unsaved progress on that device may be lost. Are you sure you want to proceed?”.
- A neutral “Cancel” button to abort the action safely.
- A red destructive “Revoke session” action button with loading spinner state during network execution.

Function Details

Input Data

Data Name	Description
id	Unique numeric ID of the session to be revoked passed in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.

Business Rules

- **BR-SES-08**: Users can only revoke sessions belonging to their own user account.
- **BR-SES-09**: Already-expired sessions (`expiresAt <= now`) cannot be revoked; attempting to do so returns HTTP 400 Bad Request (`SESSION_ALREADY_EXPIRED`).
- **BR-SES-10**: Deletion of the session row immediately revokes the ability of the refresh token to issue new access tokens.
- **BR-SES-11**: If the user revokes their currently active session, the backend sets `loggedOut: true`, prompting the client application to terminate the current session and redirect to the sign-in screen.

Normal Flow

1. The user clicks “Revoke” on a session row in the table.
2. The frontend presents the confirmation alert dialog.
3. The user clicks “Revoke session” in the dialog.
4. The frontend sends `DELETE /api/sessions/:id`.
5. The backend validates authentication and checks that the session belongs to `userId` and is active.
6. The repository deletes the session record from the database.
7. The backend returns HTTP 200 with `{ message: 'Session revoked successfully', loggedOut: false }`.
8. The frontend displays a success toast and refreshes the session list.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Invalid session ID parameter	`ParseIntPipe` rejects the call with HTTP 400 Bad Request. The UI displays an invalid identifier toast.
Session does not exist or was already deleted	The service throws `SessionNotFoundException` (HTTP 404). The UI displays: "Session not found."
Session belongs to another user	The repository lookup fails to match `userId` and throws HTTP 404 Not Found, preventing unauthorized revocation.
Session has already naturally expired	The repository detects `expiresAt <= now` and throws `SessionAlreadyExpiredException` (HTTP 400). The UI displays: "Session has already expired."
Caller token expired or revoked	The server returns HTTP 401 Unauthorized. The client logs out immediately and redirects to `/login`.
User revokes their own current session	The server executes deletion and returns `loggedOut: true`. The client application clears cookies/local storage and redirects to `/login`.

Abnorma Case	Handling & System Response
Database deletion failure	The service catches the Prisma error and throws `FailedToRevokeSessionException` (HTTP 500). The UI shows: "Failed to revoke session."
User cancels in confir-mation dialog	The dialog closes without issuing an HTTP request; no database changes occur.

Output Data

Output Field	Description
message	Success confirmation message string (`Session revoked successfully`).
loggedOut	Boolean flag indicating whether the current client device was signed out.

Post-Conditions

1. The session row is permanently deleted from the PostgreSQL `Session` table.
2. The remote device's refresh token becomes invalid for any future token renewals.
3. The session list table updates to reflect the removed session.

3.11 Forum Category Management

3.11.1 Create Forum Category

Function Trigger

An administrator clicks the "Create Category" button on the forum category management screen.

Navigation path: Admin Dashboard → Forum Categories (`/admin/categories`) → Click "Create Category" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to establish a new discussion category in the forum. Categories provide structured classification for community discussions, professional topics, and questions posted by freelancers and clients.

Interface

- "Create Category" trigger button on the category management screen.
- Modal dialog containing text fields for Category Name (required) and Description (optional).
- Action buttons: "Cancel" and "Create Category" (with loading state).
- Toast notification confirming successful category creation.

Data Processing Overview

- Administrator fills in the name and description and submits the modal form.
- The client sends an HTTP request: `POST /api/forums/admin/categories` with JSON body `{ name, description }`.
- The backend guard verifies that caller has `ADMIN` role; otherwise, throws `ForumReportForbiddenExcepti` (HTTP 403).
- `CreateForumCategoryBodyDto` validates the payload schema.
- `ForumAdminRepo.createAdminCategory()` checks whether an active category (`deletedAt = null`) with the same name already exists (case-insensitive). If found, throws `ForumCategoryAlreadyExistsException` (HTTP 409).
- The system computes an automated SEO-friendly slug via `slugify(name)`. If a collision exists, a numeric counter suffix is appended.
- The repository creates the record in `prisma.forumCategory.create` with `postCount = 0`.
- The server responds with HTTP 201 Created and the new category object.
- The client closes the dialog, shows a success toast, and refreshes the category table.

Screen Layout (Mock-up Description)

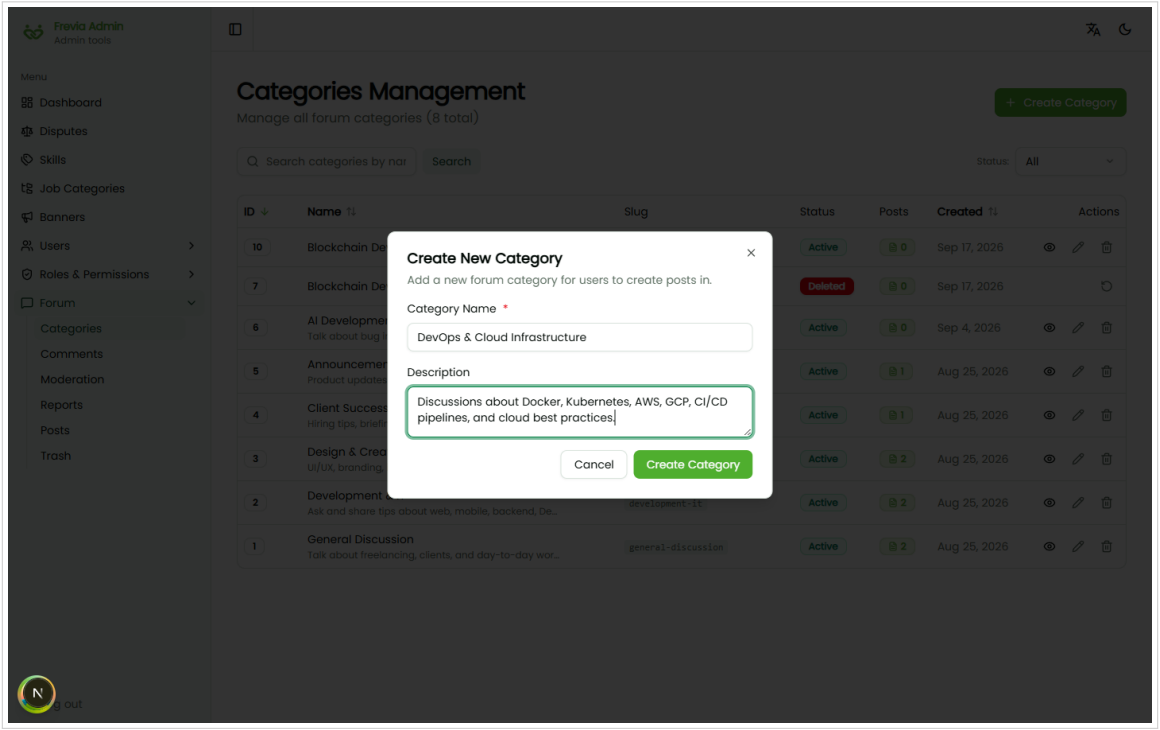

Figure 102: Create forum category modal

- Modal dialog titled “Create Category” with explanatory description “Add a new discussion category to the community forum.”.
- Form field: **Name** (marked with red required asterisk, placeholder: “e.g., General Discussion”).
- Form field: **Description** (multi-line textarea, optional, placeholder: “Describe what this category is for...”).
- Modal footer with “Cancel” button and dark primary “Create Category” button.

Function Details

Input Data

Data Name	Description
name	Display name of the forum category (required).
description	Detailed description explaining the category purpose (optional).

Validation Rules

Data Name	Validation Rule
name	String, required, 1 to 100 characters; whitespace trimmed; must not be blank.
description	String, optional, max 500 characters; nullable.

Business Rules

- **BR-FCAT-01**: Only users authenticated with the `ADMIN` role can create forum categories.
- **BR-FCAT-02**: Category name is required, max 100 characters, and must be unique among active categories (`deletedAt = null`) using case-insensitive comparison.
- **BR-FCAT-03**: Description is optional with a maximum length of 500 characters.
- **BR-FCAT-04**: A URL-friendly slug is generated automatically from the name. If the slug collides with an existing slug, a numeric suffix (e.g., `-2`) is appended.
- **BR-FCAT-05**: Newly created categories start with an initial post count of 0.

Normal Flow

1. The administrator clicks "Create Category" on `/admin/categories`.
2. The modal dialog appears. The administrator inputs a unique name and optional description.
3. The administrator clicks "Create Category".
4. The frontend validates fields and issues `POST /api/forums/admin/categories`.
5. The backend validates Admin role and ensures the category name does not duplicate an active category.
6. The system computes a unique slug and inserts the category into PostgreSQL.
7. The API returns HTTP 201 Created with the created category object.
8. The frontend displays a success toast: "Category created successfully!", closes the dialog, and updates the list.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Category name is empty or only whitespace	Client-side validation blocks submission; backend DTO validation returns HTTP 400 Bad Request. The UI highlights the name field in red.
Category name exceeds 100 characters	Backend validation rejects with HTTP 400 Bad Request (`String must contain at most 100 character(s)`). The UI displays a field length error.
Description exceeds 500 characters	Backend validation returns HTTP 400 Bad Request. The UI displays description length warning.
Category name already exists among active categories	The repository detects a duplicate name and throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). The UI displays: "Category name already exists."
User lacks Admin privileges	The service throws `ForumReportForbiddenException` (HTTP 403 Forbidden). The UI displays: "Forbidden: Account inactive or unauthorized."

Abnorma Case	Handling & System Response
Missing or expired administrator token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
Database insertion failure or constraint error	The backend catches the Prisma error and throws `FailedToCreateForumCategoryException` (HTTP 500). The UI displays: "Failed to create category."
Administrator clicks Cancel in modal	The modal closes immediately and discarded input is purged without network requests.

Output Data

Output Field	Description
id	Unique numeric system identifier of the created category.
name	Display name of the category.
slug	Generated unique URL slug.
description	Category description string or null.
createdAt	Timestamp of category creation.
updatedAt	Timestamp of category creation.
postCount	Initial active post count (0).

Post-Conditions

1. A new active category record is created in the database.

2. The category appears immediately in the forum categories list for administration and community discussions.

3.11.2 Update Forum Category

Function Trigger

An administrator clicks "Edit" on a category row in the table or on the category detail screen.
Navigation path: Admin Dashboard → Forum Categories → Click "Edit" icon on a category row.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to modify the title and descriptive summary of an existing forum category. When the title changes, the system automatically regenerates the category slug while preserving thread associations.

Interface

- Edit action icon button on category table rows and category detail views.
- "Edit Category" modal pre-populated with current category data.
- Form inputs for Name and Description.
- "Cancel" and "Save Changes" action buttons.

Data Processing Overview

- Administrator edits the name or description in the modal and submits the form.
- The client issues an HTTP request: `PATCH /api/forums/admin/categories/:id` with updated attributes.
- `ForumAdminService.updateAdminCategory()` verifies `ADMIN` role.
- `ForumAdminRepo.updateAdminCategory()` checks that the category exists and is active (`deletedAt = null`). If missing, throws `ForumCategoryNotFoundException` (HTTP 404).

- If the category name was changed, the repository checks for name collisions against other active categories (`id != currentId`). If a collision occurs, throws `ForumCategoryAlreadyExistsException` (HTTP 409).
- The repository regenerates the slug if the name changed, and updates the record in PostgreSQL.
- The API returns HTTP 200 with the updated category object.
- The frontend displays a success toast and refreshes the category data.

Screen Layout (Mock-up Description)

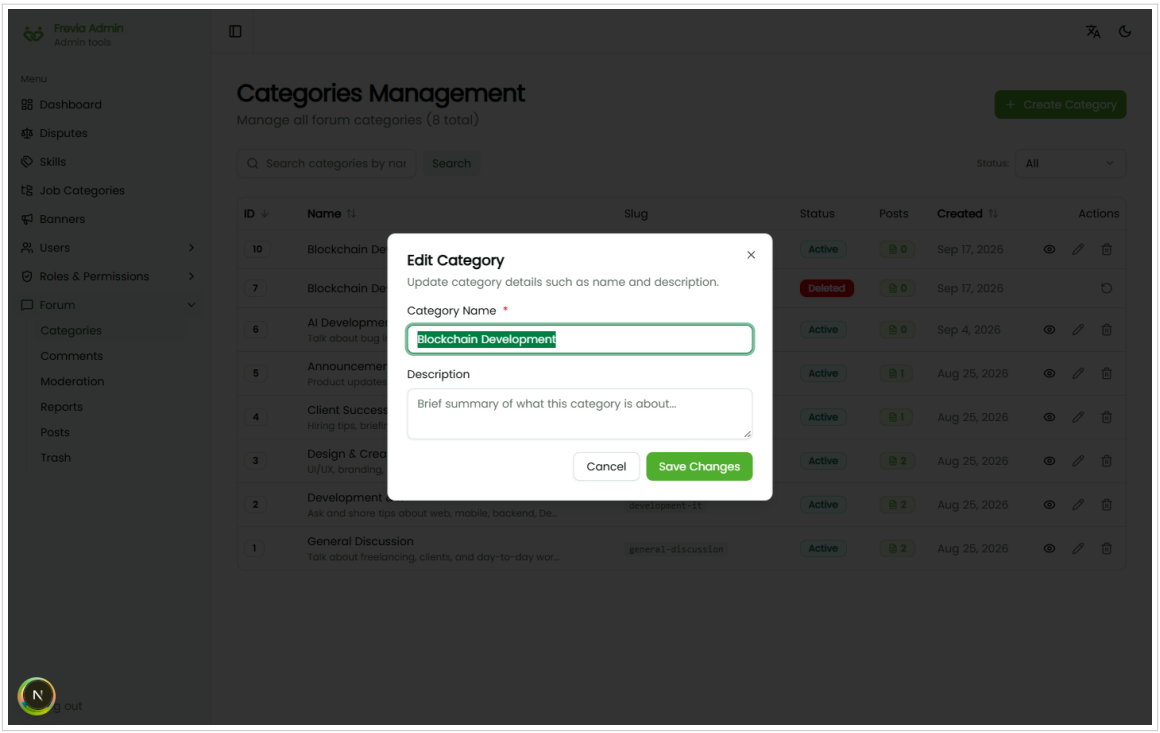

Figure 103: Update forum category modal

- Modal dialog titled “Edit Category” with description “Update category details.”.
- Pre-filled **Name** input field displaying the current category title.
- Pre-filled **Description** textarea displaying the current description.
- Action buttons at the bottom: “Cancel” and “Save Changes”.

Function Details

Input Data

Data Name	Description
id	Numeric identifier of the category in the URL path.
name	Updated category name (optional, string).
description	Updated category description (optional, string or null).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
name	String, optional, 1 to 100 characters; whitespace trimmed.
description	String, optional, max 500 characters; nullable.

Business Rules

- **BR-FCAT-06**: Only Admin users can update forum categories.
- **BR-FCAT-07**: Partial updates are supported; name and description can be updated independently or simultaneously.
- **BR-FCAT-08**: If the category name is modified, it must be unique among active categories and its URL slug is regenerated automatically.
- **BR-FCAT-09**: Soft-deleted categories cannot be updated. Attempting to update a deleted category returns HTTP 404 Not Found.

Normal Flow

1. The administrator clicks "Edit" on a category row.
2. The modal opens pre-filled with the current category data.
3. The administrator modifies the name and/or description and clicks "Save Changes".
4. The frontend sends `PATCH /api/forums/admin/categories/:id`.
5. The backend validates permissions, confirms the category exists and is active, and verifies name uniqueness.
6. The repository updates the category in PostgreSQL and regenerates the slug if the name changed.
7. The API returns HTTP 200 with the updated category payload.
8. The frontend closes the modal, displays "Category updated successfully!", and refreshes the category table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Category ID does not exist or is soft-deleted	The service throws `ForumCategoryNotFoundException` (HTTP 404). The UI displays: “Category not found.”
New name conflicts with another active category	The repository detects a duplicate name and throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). The UI displays: “Category name already exists.”
Updated name is empty or only whitespace	Client-side and backend validation return HTTP 400 Bad Request. The UI displays an error on the name field.
Updated name exceeds 100 characters	Backend validation returns HTTP 400 Bad Request. The UI notifies the user of character limit violation.
Updated description exceeds 500 characters	Backend validation returns HTTP 400 Bad Request. The UI notifies the user of character limit violation.

Abnorma Case	Handling & System Response
User lacks Admin privi-leges	The backend throws `ForumReportForbiddenException` (HTTP 403 Forbidden). The client shows an access restriction notice.
Database update failure	The backend catches the Prisma exception and throws `FailedToUpdateForumCategoryException` (HTTP 500). The UI displays: "Failed to update category."
Administrator cancels modal	The dialog closes without issuing network requests; original values remain unchanged.

Output Data

Output Field	Description
id	Unique numeric system identifier of the category.
name	Updated category name.
slug	Updated URL slug reflecting any name changes.
description	Updated category description.
createdAt	Timestamp of original creation.
updatedAt	Timestamp indicating when the modification was committed.
postCount	Current active post count.

Post-Conditions

1. The category record in the database is updated.
2. Associated forum posts continue to link to the category.
3. The updated details are reflected across both admin and client interfaces.

3.11.3 Delete Forum Category

Function Trigger

An administrator clicks "Delete" (trash icon) on a category row or detail screen.

Navigation path: Admin Dashboard → Forum Categories → Click "Delete" icon on a category row.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to soft-delete an existing forum category. To ensure community discussion integrity, active posts previously classified under this category are safely reassigned to Uncategorized (`categoryId = null`) inside an atomic database transaction.

Interface

- Delete action button on category rows and category detail header.
- Confirmation alert dialog titled "Delete category?" providing clear notice of post detachment.
- Action buttons: "Cancel" and destructive "Delete".
- Toast notification confirming soft deletion.

Data Processing Overview

- Administrator confirms category deletion in the alert dialog.
- The client issues an HTTP request: `DELETE /api/forums/admin/categories/:id`.
- `ForumAdminService.deleteAdminCategory()` checks that caller has `ADMIN` role.
- `ForumAdminRepo.deleteAdminCategory()` verifies that the category exists with `deletedAt = null`.
- The repository executes a Prisma transaction:
  1. Reassigns all active posts under the category: `prisma.forumPost.updateMany` where `categoryId = id` and `deletedAt = null`, setting `categoryId = null`.
  2. Marks the category record as soft-deleted: `prisma.forumCategory.update` setting `deletedAt = new Date()`.
- The server responds with HTTP 200: `{ message: 'Forum category deleted successfully' }`.

- The frontend displays a success toast and removes the category from the active view.

Screen Layout (Mock-up Description)

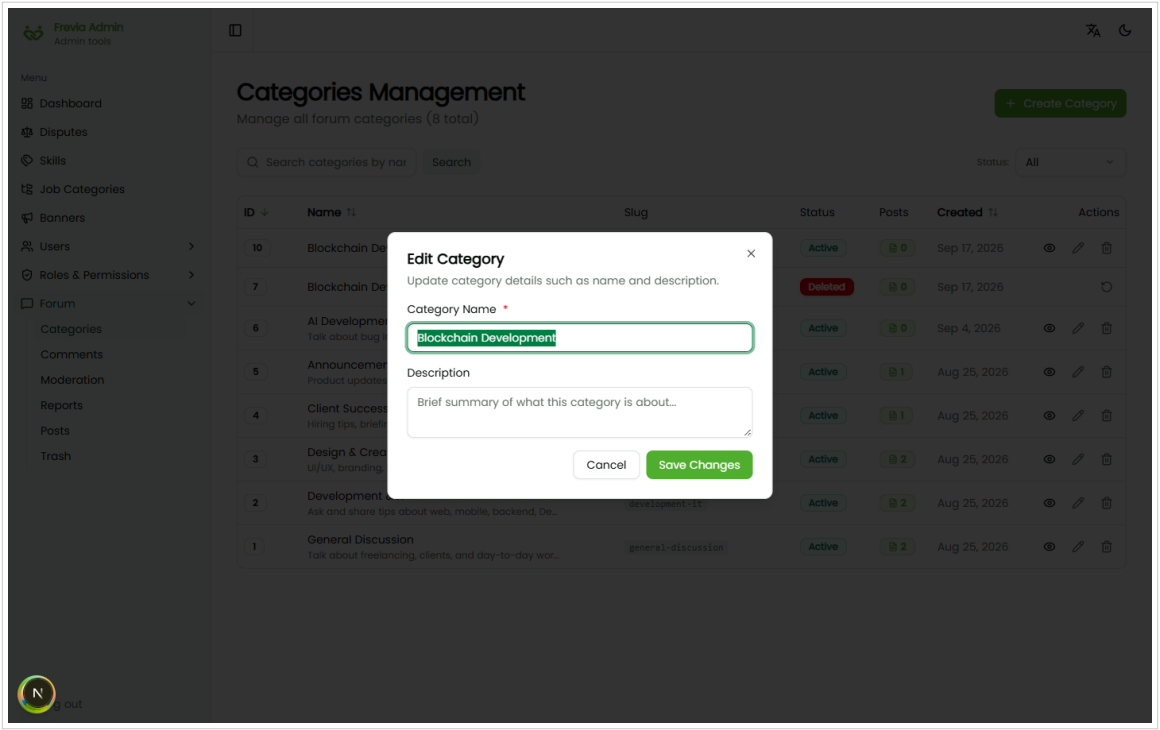

Figure 104: Delete forum category confirmation modal

- Alert dialog overlay titled “Delete category?”.
- Information notice: “Are you sure you want to delete “{name}”? Active posts in this category will become Uncategorized, and this category can be restored later from the trash tab.”.
- Neutral “Cancel” button and red destructive “Delete” action button with loading spinner state.

Function Details

Input Data

Data Name	Description
id	Numeric identifier of the category to delete passed in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.

Business Rules

- **BR-FCAT-10**: Only Admin users have authority to delete forum categories.
- **BR-FCAT-11**: Deletion is implemented as a soft delete by setting `deletedAt` to the current timestamp.
- **BR-FCAT-12**: In an atomic database transaction, active posts under this category are set to `categoryId = null` (Uncategorized) to preserve discussion history.
- **BR-FCAT-13**: Soft-deleted categories are hidden from the active category list and public forum dropdowns, but can be viewed and restored by administrators.

Normal Flow

1. The administrator clicks “Delete” on a category row.
2. The confirmation dialog appears explaining the post detachment and restoration possibilities.
3. The administrator clicks “Delete” to confirm.
4. The client sends `DELETE /api/forums/admin/categories/:id`.
5. The backend verifies permissions, reassigns active posts to `categoryId = null`, and soft-deletes the category in a transaction.
6. The backend returns HTTP 200 with a success message.
7. The frontend shows “Category deleted (soft delete)” and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Category ID does not exist or is already deleted	The service throws `ForumCategoryNotFoundException` (HTTP 404). The UI displays: “Category not found.”
User lacks Admin privileges	The backend throws `ForumReportForbiddenException` (HTTP 403 Forbidden). The UI displays an authorization error.
Caller token expired or missing	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
Database transaction fails during post detachment or category update	The transaction rolls back completely and the service throws `FailedToDeleteForumCategoryException` (HTTP 500). The UI displays: “Delete category failed.”
Administrator clicks Cancel in confirmation dialog	The dialog closes without issuing an HTTP request; no database records are modified.

Output Data

Output Field	Description
message	Confirmation message string (`Forum category deleted successfully`).

Post-Conditions

1. The category record has `deletedAt` populated with the deletion timestamp.
2. Active posts formerly in this category have `categoryId = null` and remain accessible as Uncategorized discussions.
3. The category is removed from the active category list and moves to the trash view.

3.11.4 View Forum Category List

Function Trigger

An administrator accesses the forum categories administration section from the sidebar navigation.
Navigation path: Admin Dashboard → Forum Categories (`/admin/categories`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Provides administrators with a centralized overview of all discussion categories in the forum. Allows searching by category name, filtering between active categories and deleted categories (trash view), sorting by ID, name, or creation date, and viewing live post count statistics.

Interface

- Management screen at `/admin/categories`.
- Action toolbar containing search input, active/trash status filter toggle, sorting selector, and "Create Category" button.
- Data table displaying ID, Name, Slug, Description, Post Count, Created Date, Status Badge, and contextual Action buttons.
- Pagination footer with total record count and page controls.

Data Processing Overview

- Client sends `GET /api/forums/admin/categories` with query parameters (`page`, `limit`, `search`, `deleted`, `sortBy`, `sortOrder`).
- The backend guard verifies caller's JWT token and validates `ADMIN` role.
- `ForumAdminService.getAdminCategoryLists()` delegates query execution to `ForumAdminRepo.getAdminCategoryLists()`.
- The repository queries `prisma.forumCategory.findMany` with conditions:
  - If `deleted === 'true'`, queries where `deletedAt != null`; otherwise, queries `deletedAt = null`.
  - If `search` is provided, filters `name` using case-insensitive contains.
  - Computes active post count: `_count: { posts: { where: { deletedAt: null } } }`.
- The database returns matching categories and total count.
- The server responds with HTTP 200: `{ categories, total, pagination }`.
- The frontend renders the table with appropriate action buttons (Edit/Delete for active, Restore for deleted).

Screen Layout (Mock-up Description)

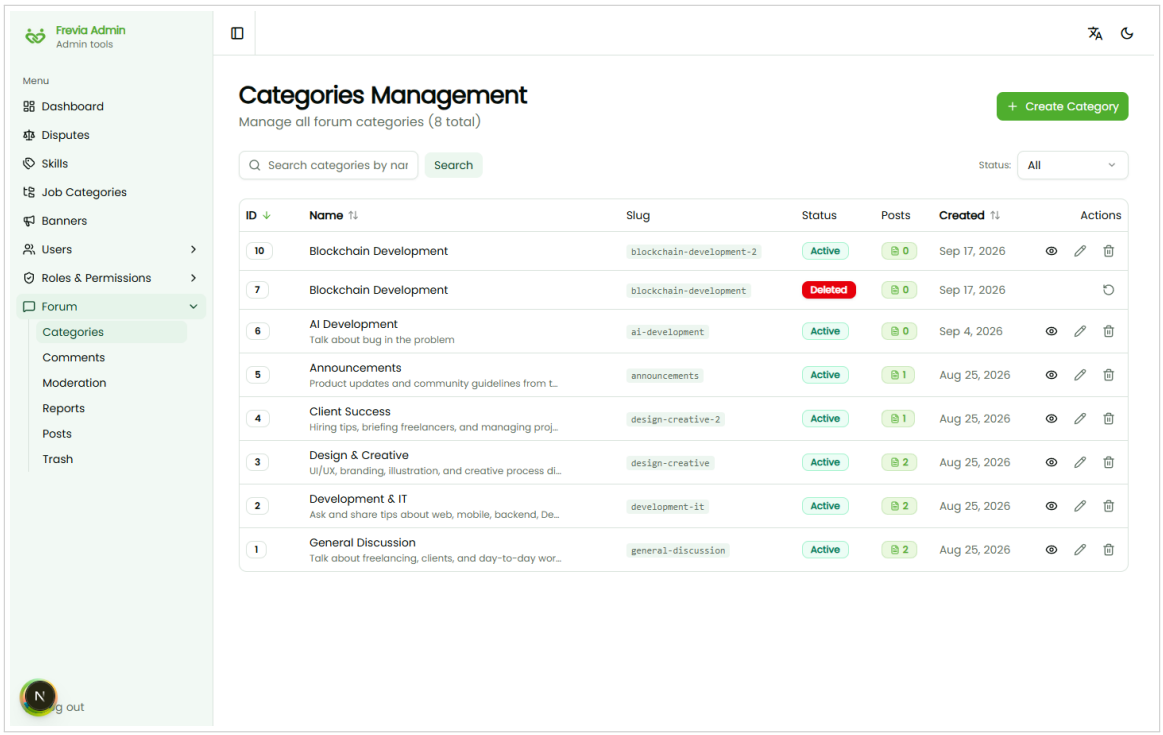

Figure 105: Admin forum category list interface

- Top section with title "Categories" and "Create Category" button.
- Filter bar including search input, status tab filter (All Active vs. Trash), and sort dropdown.

- Data table columns: **ID**, **Name**, **Slug**, **Description**, **Posts**, **Created**, and **Actions**.
- Each category row features clickable links to view details, an edit icon, and a delete icon.
- Pagination footer with previous/next page navigation buttons and current record count.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum categories per page (defaults to 10).
search	Optional text query searching category name.
deleted	Optional filter string (`'true'` for trash view, `'false'` or omitted for active categories).
sortBy	Sort column field (`id`, `name`, or `createdAt`; defaults to `id`).
sortOrder	Sort order direction (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters.
deleted	Optional string, must be `'true'` or `'false'`.
sortBy	Must strictly be one of: `id`, `name`, `createdAt`.
sortOrder	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-FCAT-14**: Only users with the `ADMIN` role are authorized to access the category management catalog.
- **BR-FCAT-15**: When the `deleted` query parameter is omitted or `'false'`, only active categories (`deletedAt = null`) are returned.
- **BR-FCAT-16**: When `deleted = 'true'`, only soft-deleted categories (`deletedAt != null`) are returned.

- **BR-FCAT-17**: Post counts are calculated strictly for active posts (`deletedAt = null`) under each category.
- **BR-FCAT-18**: Sorting is supported on `id`, `name`, and `createdAt`.

Normal Flow

1. The administrator navigates to the "Forum Categories" section.
2. The frontend requests `GET /api/forums/admin/categories` with default query parameters.
3. The backend validates Admin authorization and executes the database query with active post counts.
4. The database returns category records and total count.
5. The server responds with HTTP 200 and the category list payload.
6. The frontend renders the category table with post counts, timestamps, and action buttons.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
User is not authenticated or token expired	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The service throws `ForumReportForbiddenException` (HTTP 403 Forbidden). The UI displays an access denied screen.
Invalid pagination parameter	The API rejects invalid parameter values with HTTP 400 Bad Request. The client defaults to page 1.

Abnorma Case	Handling & System Response
Invalid sortBy parameter	The system rejects invalid sort keys with HTTP 400 Bad Request. The client resets sorting to `id desc`.
Database query error during retrieval	The service throws `FailedToLoadForumCategoriesException` (HTTP 500). The UI displays: "Failed to load forum categories."
No categories match search or filter query	The repository returns an empty list (`categories: [], total: 0`). The table displays: "No categories found."

Output Data

Output Field	Description
categories	Array of category objects: `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, `deletedAt`, and `postCount`.
pagination.page	Current page number.
pagination.limit	Current limit per page.
pagination.total	Total matching categories.
pagination.totalPages	Total calculated pages available.

Post-Conditions

1. The forum category list is rendered with accurate live post count statistics.
2. The administrator can trigger create, edit, delete, detail view, or restore actions.

3.11.5 View Forum Category Detail

Function Trigger

An administrator clicks "View detail" on a category row or accesses the category detail route directly.
Navigation path: Forum Categories (`/admin/categories`) → Click category title or row (`/admin/categories/:id`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Displays full administrative details of a specific forum category, including its system ID, title, unique URL slug, detailed description, active discussion post count, creation timestamp, and last updated timestamp, with quick shortcuts to edit or delete the category.

Interface

- Dedicated category detail page at `/admin/categories/:id`.
- Top navigation header with "Back to categories" link.
- Header displaying category name and ID badge.
- Action buttons: "Edit Category" and "Delete Category".
- Structured information cards displaying: Description, Slug, Post count, Creation date, and Last update timestamp.

Data Processing Overview

- Client sends `GET /api/forums/admin/categories/:id`.
- `ParseIntPipe` ensures `id` is an integer.
- The backend guard verifies `ADMIN` role.
- `ForumAdminService.getAdminCategoryById()` calls `ForumAdminRepo.getAdminCategoryById(id`
- The repository queries `prisma.forumCategory.findFirst` where `id = id` and `deletedAt = null`, including active post count.
- If the category is not found, the service throws `ForumCategoryNotFoundException` (HTTP 404).
- The server responds with HTTP 200 and the category detail payload.
- The frontend renders the detail cards and navigation buttons.

Screen Layout (Mock-up Description)

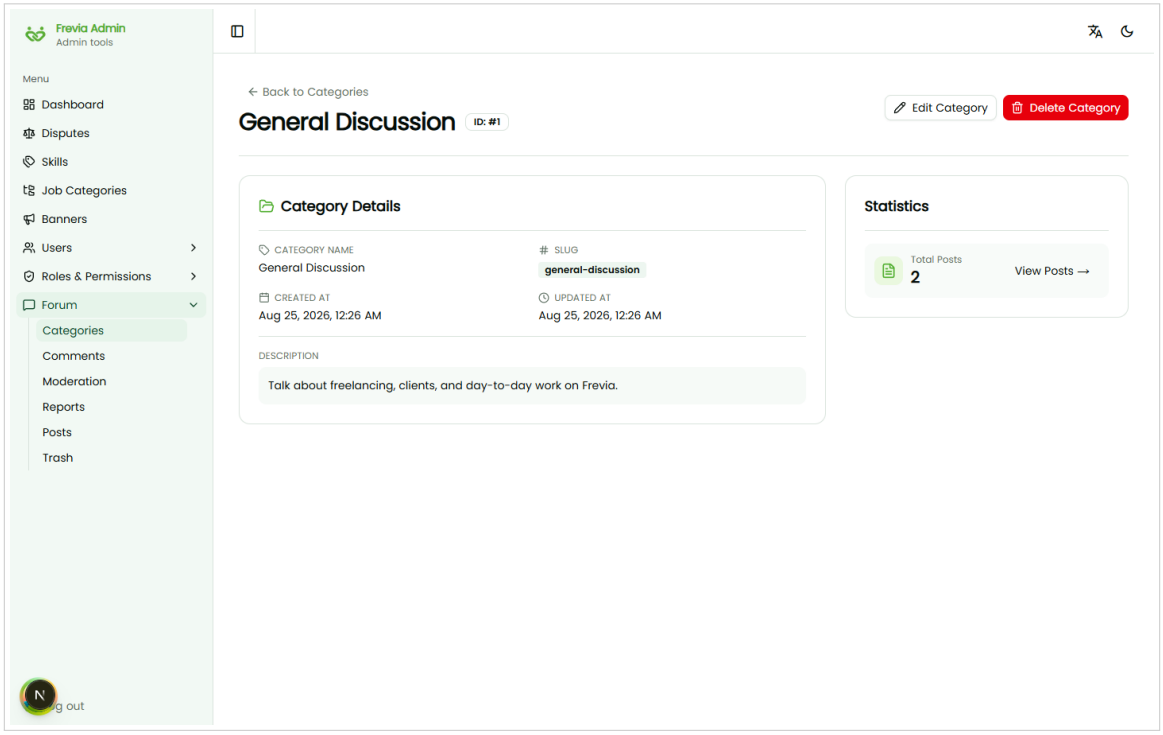

Figure 106: Forum category detail interface

- Top breadcrumb button “Back to categories” leading back to the category catalog.
- Page header displaying the category name in bold typography with an “ID: #{id}” outline badge.
- Top right action buttons: “Edit Category” and destructive “Delete Category”.
- Information cards: **Category Information** (Description and Slug) and **Statistics & Metadata** (Total Posts, Created timestamp, and Last Updated timestamp).

Function Details

Input Data

Data Name	Description
id	Numeric identifier of the category in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.

Business Rules

- **BR-FCAT-19**: Only active categories with `deletedAt = null` are accessible via the detail view. Deleted categories return HTTP 404 Not Found.
- **BR-FCAT-20**: Detail payload includes `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`.
- **BR-FCAT-21**: Access is restricted exclusively to authenticated users possessing the `ADMIN` role.

Normal Flow

1. The administrator clicks a category row or accesses `/admin/categories/:id`.
2. The frontend sends `GET /api/forums/admin/categories/:id`.
3. The backend validates Admin authorization and queries the database for the active category.
4. The repository retrieves the category along with its active post count.
5. The server responds with HTTP 200 and the category detail data.
6. The frontend renders the complete category details, statistics, and action controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Invalid category ID format (non-integer)	`ParseIntPipe` rejects with HTTP 400 Bad Request. Next.js triggers the not found or bad request error view.

Abnorma Case	Handling & System Response
Category ID does not exist in database	The service throws `ForumCategoryNotFoundException` (HTTP 404). The frontend displays: "Category not found."
Category has been soft-deleted	Query filters `deletedAt = null`; returns null and throws HTTP 404 Not Found. The UI displays the not found notice.
User lacks Admin privileges	The backend throws `ForumReportForbiddenException` (HTTP 403 Forbidden). Access is denied.
Caller token expired or missing	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
Database query error during lookup	The backend catches the Prisma error and throws `FailedToLoadCategoryDetailException` (HTTP 500). The UI displays a retry button.

Output Data

Output Field	Description
id	Numeric identifier of the category.
name	Display title of the category.
slug	Unique URL slug.
description	Detailed description or null.
createdAt	Timestamp of category creation.
updatedAt	Timestamp of last update.
postCount	Number of active discussion threads in this category.

Post-Conditions

1. Category details and post statistics are presented to the administrator.
2. The administrator can trigger edit, delete, or return to the category list.

3.11.6 Restore Forum Category

Function Trigger

An administrator clicks "Restore" on a soft-deleted category in the trash view of the category catalog.
Navigation path: Admin Dashboard → Forum Categories → Filter "Trash" → Click "Restore" icon.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to recover a soft-deleted forum category by clearing its `deletedAt` field back to `null`. This restores the category to the active list and enables users to create new discussion posts within it again.

Interface

- Restore action icon button on deleted category rows in the Trash view.

- Confirmation modal dialog titled “Restore category?”.
- Action buttons: “Cancel” and “Restore”.
- Toast notification confirming successful category restoration.

Data Processing Overview

- Administrator clicks “Restore” on a deleted category and confirms in the dialog.
- The client issues an HTTP request: `PATCH /api/forums/admin/categories/:id/restore`.
- `ForumAdminService.restoreAdminCategory()` verifies `ADMIN` role.
- `ForumAdminRepo.restoreAdminCategory()` verifies that the category exists and is currently deleted (`deletedAt != null`). If missing, throws `ForumCategoryNotFoundException` (HTTP 404).
- The repository checks whether an active category (`deletedAt = null`) already exists with the same name. If a duplicate exists, throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict).
- The repository executes `prisma.forumCategory.updateMany` setting `deletedAt = null`.
- The API returns HTTP 200 with the restored category object.
- The frontend displays a success toast and refreshes the category table.

Screen Layout (Mock-up Description)

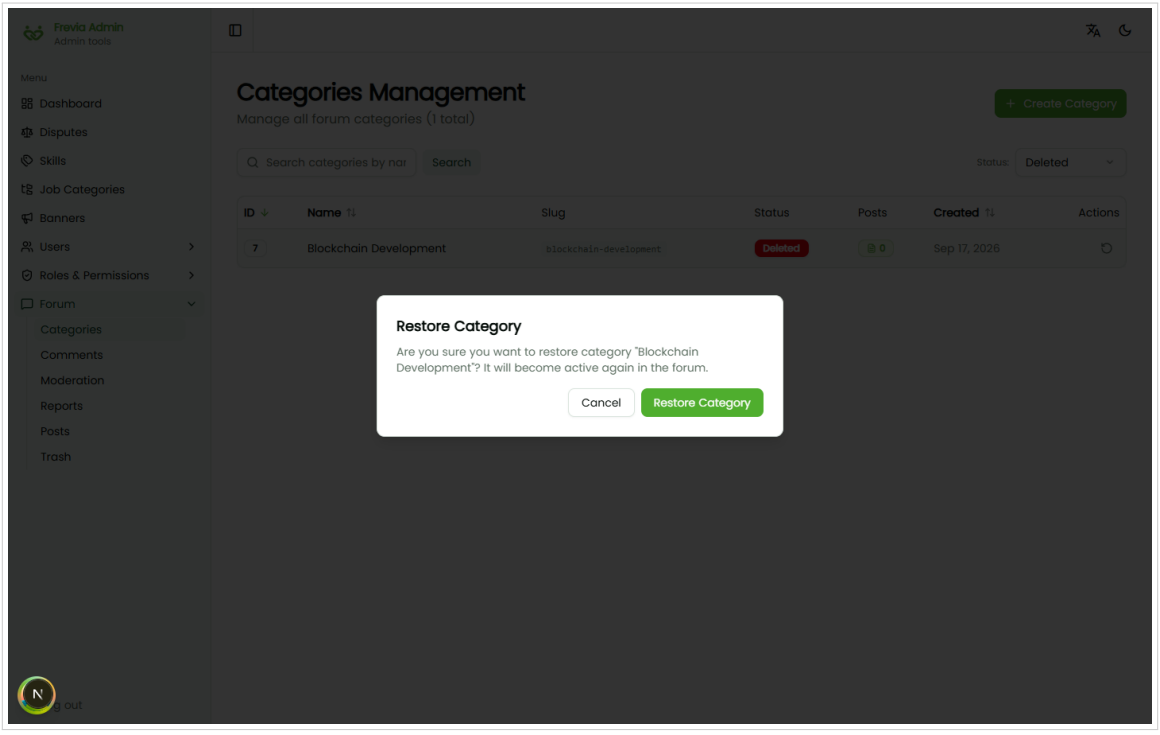

Figure 107: Restore forum category modal

- Alert dialog overlay titled “Restore category?”.
- Descriptive prompt: “Are you sure you want to restore “{name}”? This category will reappear in the active categories list and be available for community discussions.”.
- Action buttons at the bottom: “Cancel” and primary “Restore”.

Function Details

Input Data

Data Name	Description
id	Numeric identifier of the soft-deleted category passed in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.

Business Rules

- **BR-FCAT-22**: Only users with the `ADMIN` role can restore forum categories.
- **BR-FCAT-23**: The category must currently be soft-deleted (`deletedAt != null`) to be eligible for restoration.
- **BR-FCAT-24**: Before restoration, the system must verify that no active category currently exists with the same name. If a naming collision is detected, restoration is rejected with HTTP 409 Conflict.
- **BR-FCAT-25**: Restoring resets `deletedAt = null`; previously detached posts remain in Uncategorized unless individually reclassified.

Normal Flow

1. The administrator navigates to the “Trash” view of the category catalog.
2. The administrator clicks “Restore” on a deleted category row.
3. The confirmation dialog appears. The administrator confirms restoration.
4. The frontend sends `PATCH /api/forums/admin/categories/:id/restore`.
5. The backend validates Admin authorization and confirms the category is currently soft-deleted.
6. The repository checks for name conflicts against active categories.

7. The repository updates the category setting `deletedAt = null`.
8. The server returns HTTP 200 with the restored category details.
9. The frontend displays “Category restored successfully” and updates the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Category ID does not exist or was not deleted	The service throws `ForumCategoryNotFoundException` (HTTP 404). The UI displays: “Category not found or not deleted.”
Another active category now uses the same name	The repository detects a duplicate name among active categories and throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). The UI displays: “Cannot restore category: an active category with the same name already exists.”
User lacks Admin privileges	The backend throws `ForumReportForbiddenException` (HTTP 403 Forbidden). Access is denied.
Caller token expired or missing	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.

Abnorma Case	Handling & System Response
Database update failure during restoration	The backend catches the Prisma error and throws `FailedToRestoreForumCategoryException` (HTTP 500). The UI displays: “Restore category failed.”
Administrator cancels restoration dialog	The dialog closes without issuing an HTTP request; the category remains soft-deleted.

Output Data

Output Field	Description
id	Numeric identifier of the restored category.
name	Display title of the restored category.
slug	URL slug of the category.
description	Description string or null.
createdAt	Timestamp of original creation.
updatedAt	Timestamp indicating restoration time.
postCount	Current post count.

Post-Conditions

1. The category’s `deletedAt` field is reset to `null`.
2. The category reappears in the active forum category catalog.
3. Users can create new discussion posts under this category.

3.12 Manage Conversation

3.12.1 Start Conversation

Function Trigger

A logged-in user starts a new chat, either by clicking the “Connect with client” button on the job detail page in Find Work, or by clicking the “New message” button in the conversation list.

Function Description

This function allows a logged-in user to start a direct conversation with another user. When the chat is started from a job, the other participant is the client who posted that job. When the chat is started from the conversation list, the user chooses the other participant. The conversation is created and the user can write the first message. If a conversation with that user already exists, the existing conversation is reopened instead of creating a duplicate.

Screen Layout (Mock-up Description)

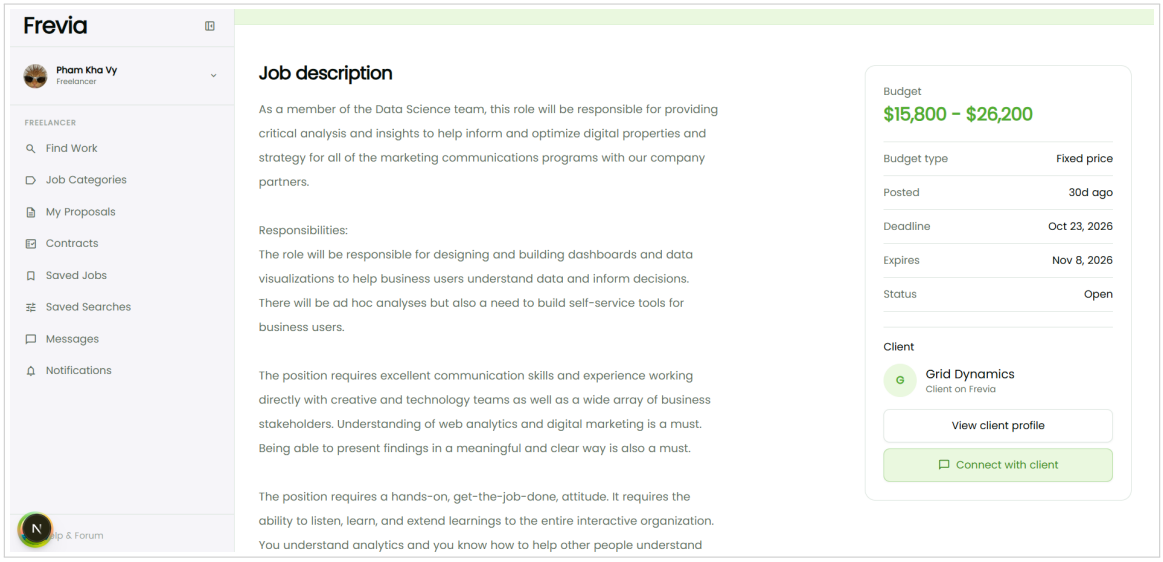

Figure 108: Start conversation screen

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user selected another user who is different from themselves.
- **Process**
  - **Start a New Conversation**: The user starts the chat in one of two ways.
    - From a job in Find Work, the user clicks the “Connect with client” button on the job detail page.

        - From the conversation list, the user clicks the “New message” button.
    - **Choose the Participant**: The user provides or confirms the other participant.
        - When the chat is started from a job, the client of that job is already selected, so the user does not have to choose anyone.
        - When the chat is started from the conversation list, the user enters the identifier of the other participant in the dialog and confirms.
    - **Create or Reopen the Conversation**: The system checks whether a conversation between the two users already exists.
        - When no conversation exists, a new conversation between the two users is created.
        - When a conversation already exists, that conversation is reopened, including a conversation that the user had previously deleted from their own list.
    - **Write the First Message**: The compose screen shows the name and avatar of the other participant, and the user can type and send the first message (see **Send Message**).
- **Alternative Flow**
    - **Conversation Already Exists**: When a conversation with the chosen user already exists, the system opens the existing conversation instead of creating a second one, so there is only one thread per pair of users.
    - **Not Logged In**: When a guest clicks the “Connect with client” button, the system asks the user to log in before the conversation can be started.
    - **Cancel**: When the user leaves the compose screen or closes the dialog without confirming, no conversation is created and nothing changes.
- **Postconditions**
    - A conversation between the two users exists and is open on the screen.
    - The conversation is visible in the conversation list of both participants.
- **Error Handling**
    - **Cannot Chat with Yourself**: When the user chooses their own account as the participant, the conversation is not created and a message tells the user that a chat with themselves is not allowed.
    - **Participant Not Found**: When the chosen user cannot be found, the conversation is not created and a message tells the user that the participant cannot be found.
    - **Cannot Start Chat**: When the conversation cannot be created, the screen shows a start-chat failure message so that the user can try again.
    - **Lost Connection**: When the connection to the server is lost, the screen shows a “Lost Connection” state and asks the user to reconnect.

- **Security Measures**
  - Only logged-in users can start a conversation, and the user is always recorded as one of the two participants.
  - When the chat is started from a job, the other participant is always the client of that job, so the button cannot be used to start a chat with an unrelated account.
  - Only one conversation can exist for a pair of users, so a user cannot create duplicated threads with the same person.

3.12.2 View Conversation List

Function Trigger

The logged-in user opens the Messages area by clicking the chat icon in the main navigation.

Function Description

This function allows a logged-in user (Client, Freelancer) to see all of their conversations in one list. Each row shows the other participant, a preview of the last message, the time of that message and the number of unread messages. Conversations that the user has pinned are always shown at the top of the list.

Screen Layout (Mock-up Description)

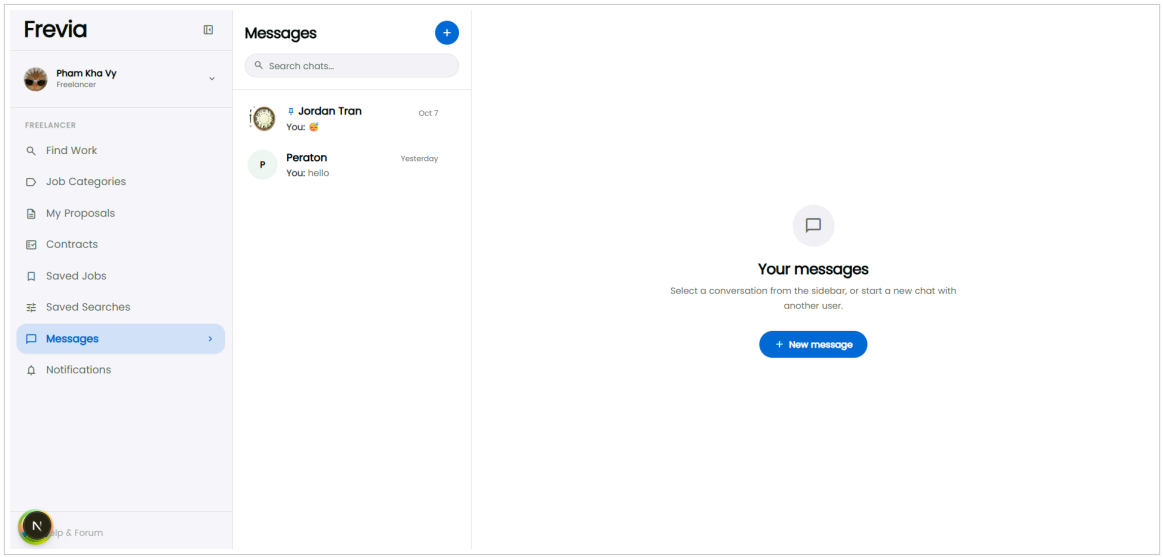

Figure 109: Conversation list screen

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user has at least one conversation, or the empty state is displayed instead.
- **Process**
  - **Open the Messages Area**: The user clicks the chat icon in the main navigation and the conversation list is displayed.
  - **Read Each Conversation Row**: Every row in the list presents the following information.
    - The avatar and the display name of the other participant.
    - A short preview of the last message, with a paperclip icon when the last message contains a file.
    - The relative time of the last message.
    - A blue unread dot on the avatar and an unread count badge when there are unread messages.
    - A pin icon when the conversation is pinned.
  - **Search for a Conversation**: The user types a keyword in the search box and the list shows only the conversations whose participant name or last message contains that keyword.
  - **Sorting**: Pinned conversations are shown first, and the remaining conversations are ordered by the time of the last message, newest first.
  - **Open a Conversation**: The user clicks a conversation row to open the chat thread (see **View Conversation Detail**).
- **Alternative Flow**
  - **No Conversation**: When the user has no conversation at all, the screen shows an empty state with a "New message" button, and the user can start a new chat from there (see **Create Conversation**).
  - **No Search Result**: When the search keyword does not match any conversation, the list is replaced by a message telling the user that no chat matches the keyword.
- **Postconditions**
  - The user sees all of their conversations, with the pinned ones at the top and the unread counts for each conversation.
  - Opening the list does not change any message; unread messages are only cleared when the user opens a thread (see **Mark Conversation as Read**).

- **Error Handling**
  - **Cannot Load Conversations**: When the conversation list cannot be loaded, the screen shows a load-failure message so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only the conversations of the logged-in user are shown, so a user can never see the chats of another account.
  - Conversations that the user has deleted from their own list are not shown again until a new message arrives.

3.12.3 View Conversation Detail

Function Trigger

The user clicks a conversation row in the conversation list.

Function Description

This function allows the user to read the whole chat thread with the other participant. The thread shows all messages in chronological order, the avatar and display name of the other user, the attachments that were exchanged and the read state of the messages that the user has sent.

Screen Layout (Mock-up Description)

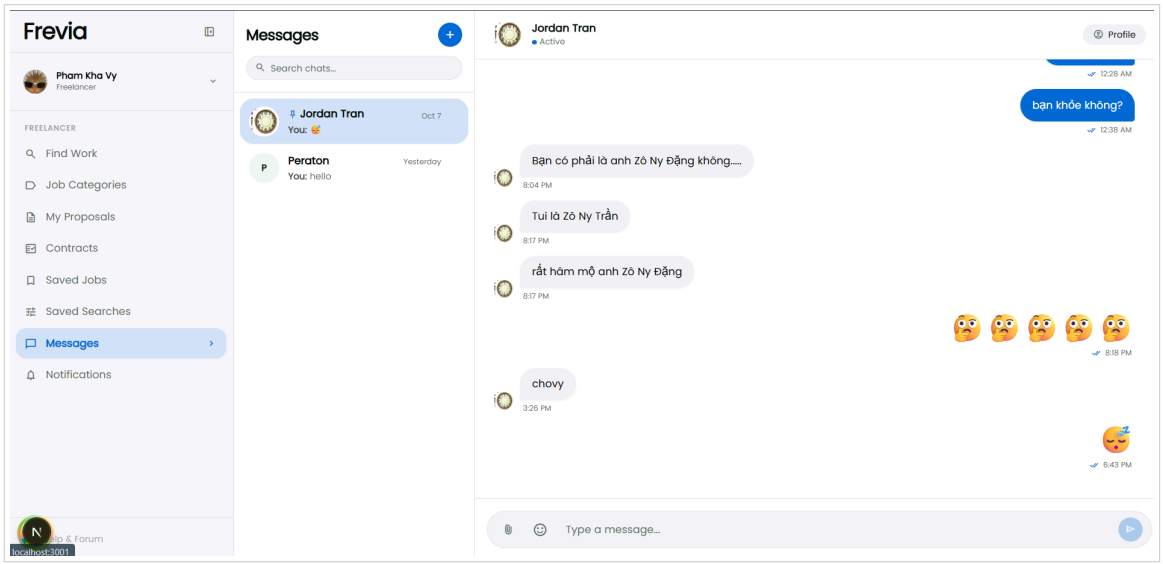

Figure 110: Conversation detail screen (chat thread)

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation that is being opened.
- **Process**
  - **Open the Conversation**: The user clicks a conversation row and the chat thread is displayed.
  - **Read the Thread Header**: The header of the thread shows the following information.
    - The avatar and the display name of the other participant.
    - A button that opens the profile of the other participant.
    - A back button that returns to the conversation list.
  - **Read the Messages**: The messages are shown in chronological order, oldest at the top and newest at the bottom.
    - Messages sent by the user are aligned on one side, and messages received from the other user on the other side.
    - Image attachments are shown as pictures that can be opened in a new tab.
    - Other files are shown as a download card with the file name and the file size.
    - A message that contains only emojis is displayed with a larger font.
  - **Check the Read State**: Messages that the user has sent show a read indicator.
    - A single check mark means that the message has been delivered but not read.
    - A double check mark means that the other user has read the message.
  - **Use the Thread Actions**: The user can send a message (see **Send Message**), attach a file (see **Send File**) or delete one of their own messages (see **Delete Message**).
- **Alternative Flow**
  - **Empty Thread**: When the conversation has no message yet, the thread shows an empty state with a hint to write the first message.
  - **User Is Typing**: When the other user is typing, an animated typing indicator is shown at the bottom of the thread.
- **Postconditions**
  - The user sees the whole chat thread with the other participant, including the attachments and the read state of their own messages.
  - The messages of the other user are marked as read while the thread is open (see **Mark Conversation as Read**).

- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the screen shows a not-found state and offers the user a way back to the conversation list.
  - **Access Denied**: When the user is not a participant of the conversation, the thread is not opened and a permission message is shown.
  - **Cannot Load Messages**: When the messages cannot be loaded, the thread shows a load-failure message so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only the two participants of a conversation can read its messages; another user is refused even when the conversation link is known.
  - Messages that have been deleted are never shown again in the thread.

3.12.4 Send Message

Function Trigger

The user types a message in the input box at the bottom of the chat thread and clicks the send button.

Function Description

This function allows a logged-in user to send a text message to the other participant of the conversation. The user types the text and sends it, and the message appears in the thread of both users immediately.

Screen Layout (Mock-up Description)

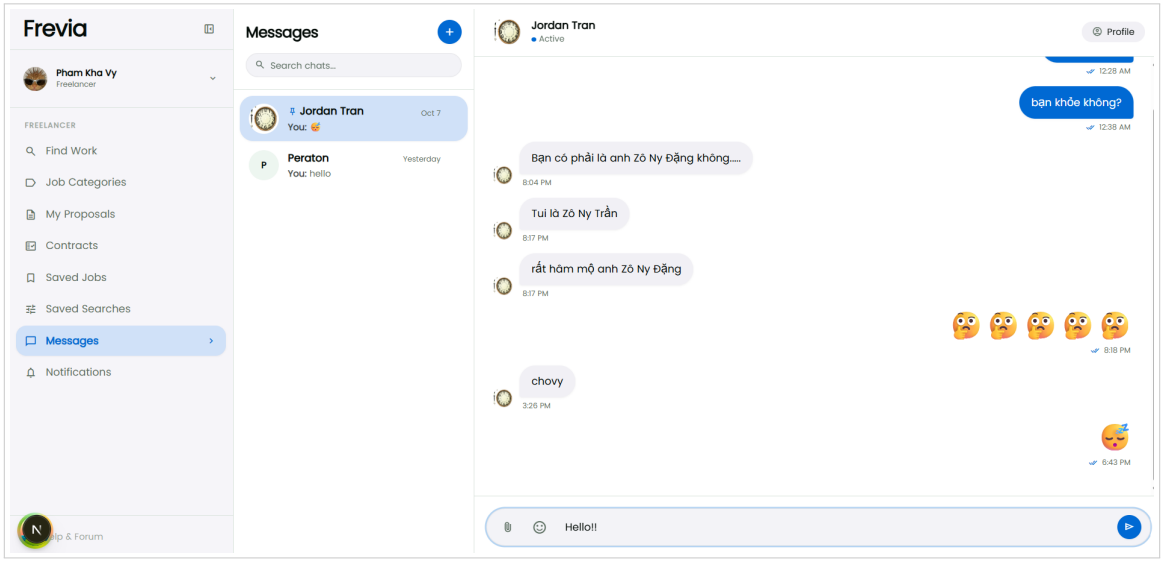

Figure 111: Sending a message in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.
- **Process**
  - **Type the Message**: The user types the text in the input box at the bottom of the thread. The send button stays disabled while the input box is empty.
  - **Add an Emoji if Wanted**: The user can click the emoji button to pick an emoji, and the chosen emoji is added to the input box.
  - **Send the Message**: The user clicks the send button or presses the Enter key.
  - **View the Result**: When the message is sent, the screen shows the result.
    - The message appears at the bottom of the thread on the user's side.
    - The input box is cleared.
    - The message shows a single check mark, meaning that it has been sent but not read yet.
  - **Message Arrives for the Other User**: The other participant receives the message in real time, and the conversation list of that user shows the new message preview and the unread count.
- **Alternative Flow**

- **Enter Key**: When the user presses the Enter key without holding Shift, the message is sent in the same way as clicking the send button.
- **Empty Message**: When the input box is empty, the send button remains disabled and the user cannot send an empty message.
- **Emoji Only**: When the message contains only emojis, the message is displayed with a larger font in the thread.

- **Postconditions**
  - The new message is stored in the conversation and is shown to both participants.
  - The conversation list of both users is updated with the new last message.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the message is not sent and the thread shows a message telling the user that the conversation cannot be found.
  - **Access Denied**: When the user is not a participant of the conversation, the message is refused and a permission message is shown.
  - **Cannot Send Message**: When the message cannot be sent, the thread shows a send-failure message and the text is kept so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only a participant of the conversation can send a message into it, and the sender is always the account that is currently logged in.
  - The length of a message is limited, so very large text cannot be sent.

3.12.5 Send File

Function Trigger

The user clicks the attach-file button in the input box of the chat thread and chooses a file from their device.

Function Description

This function allows a logged-in user to send a file, such as a picture or a document, to the other participant of the conversation. The user chooses a file, the system uploads it and the file is sent as a message with a preview or a download card.

Screen Layout (Mock-up Description)

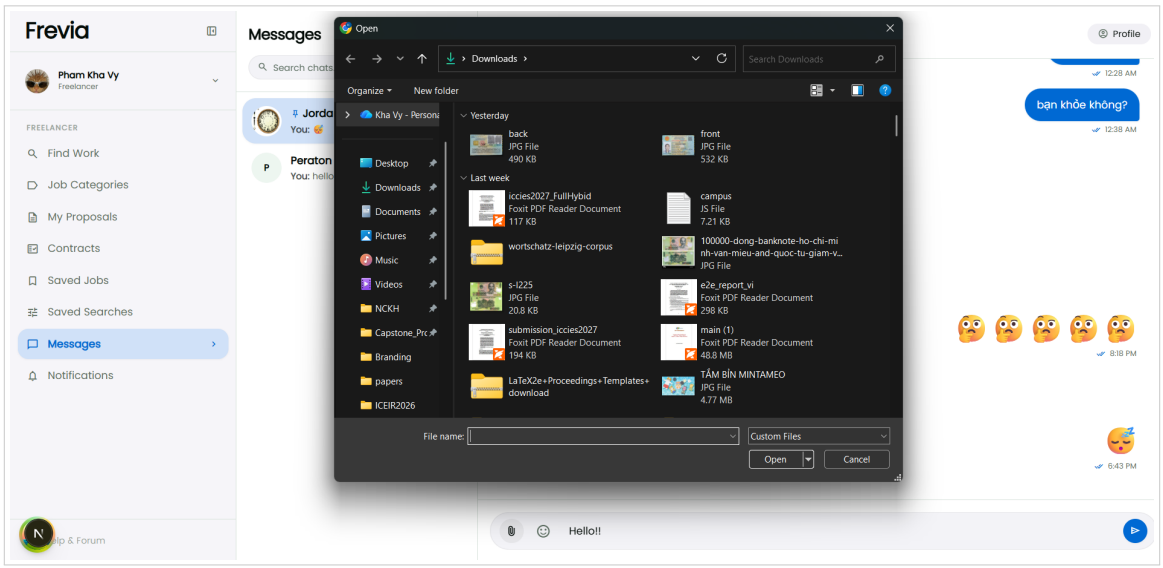

Figure 112: Sending a file in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.
- **Process**
  - **Click the Attach Button**: The user clicks the paperclip button in the input box, and the file chooser of the device is opened.
  - **Choose the File**: The user selects a file. The file chooser accepts images, documents, archives and other supported file types.
  - **Upload the File**: The system uploads the file before sending it, and a loading state is shown while the upload is running.
  - **Send the File as a Message**: When the upload finishes, the file is sent to the conversation as a message.
    - When the file is an image, it is displayed as a picture that can be opened in a new tab.
    - When the file is not an image, it is displayed as a download card with the file name and the file size.
  - **View the Result**: The file message appears at the bottom of the thread, and the conversation list of both users shows the file preview with a paperclip icon.

- **Alternative Flow**
  - **Cancel the File Chooser**: When the user closes the file chooser without choosing a file, no upload is started and nothing changes.
  - **Image Preview**: When the chosen file is an image, the picture is rendered directly inside the message bubble instead of a download card.
- **Postconditions**
  - The file is uploaded and attached to a new message in the conversation.
  - Both participants can see the file message in the thread and can open or download the file.
- **Error Handling**
  - **File Too Large**: When the chosen file is larger than the allowed size, the upload is stopped and a message tells the user that the file is too large.
  - **File Type Not Allowed**: When the chosen file type is not supported, the upload is stopped and a message tells the user that this file type is not allowed.
  - **Cannot Upload File**: When the upload fails, the file message is not sent and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only supported file types can be uploaded, and the size of the file is limited before the upload starts.
  - Only a participant of the conversation can send a file into it, and the sender is always the account that is currently logged in.

3.12.6 Mark Conversation as Read

Function Trigger

The user opens a conversation from the conversation list, or a new message arrives while the thread is already open.

Function Description

This function allows a logged-in user to clear the unread messages of a conversation. The unread badge disappears from the conversation list and the double check mark appears on the messages of the other user in the thread, so the other participant can see that the messages have been read.

Screen Layout (Mock-up Description)

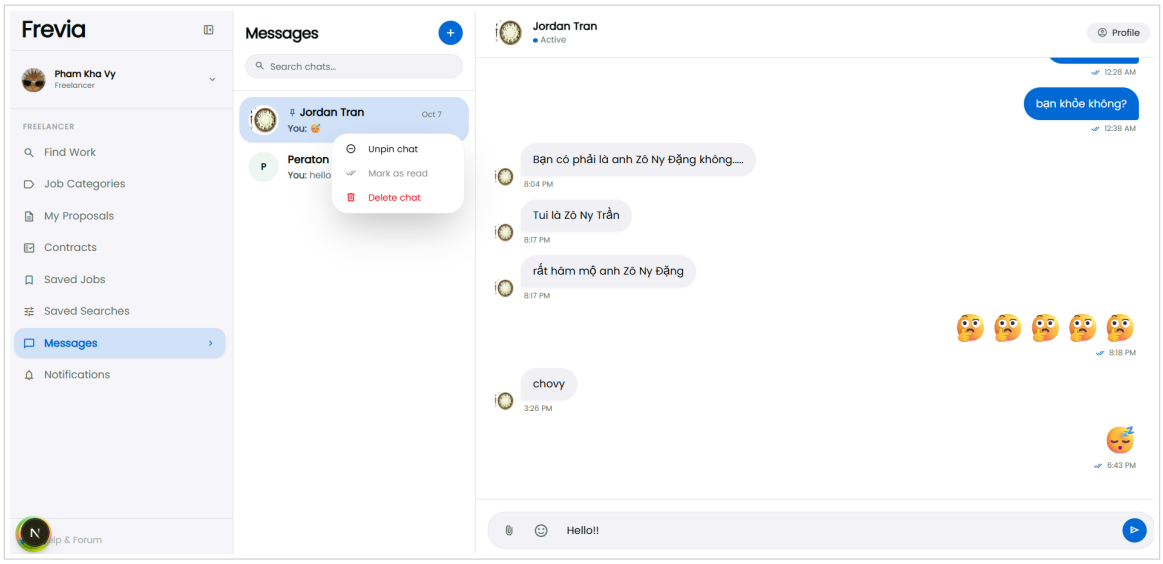

Figure 113: Conversation after being marked as read

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation contains at least one unread message from the other participant.
- **Process**
  - **Open the Conversation**: The user opens the conversation, and the system automatically marks the received messages as read.
  - **View the Result in the List**: The conversation row in the conversation list is updated.
    - The blue unread dot disappears from the avatar.
    - The unread count badge is removed from the row.
  - **Mark Manually if Wanted**: The user can also choose the "Mark as read" action from the "..." menu of a conversation row.
  - **Notify the Sender**: The other participant sees the read state change in real time.
    - Messages that were shown with a single check mark are updated to a double check mark.
- **Alternative Flow**
  - **Nothing to Read**: When the conversation has no unread message, the "Mark as read" action is disabled in the menu.
  - **New Message While Reading**: When a new message arrives while the thread is open, that message is marked as read immediately.

- **Postconditions**
  - All messages received from the other participant in this conversation are marked as read.
  - The unread count of the conversation is cleared for the current user, and the other participant sees the double check mark on their sent messages.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the read state is not updated and a message tells the user that the conversation cannot be found.
  - **Cannot Mark as Read**: When the read state cannot be updated, the unread badge stays on the row and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only the messages received by the current user are marked as read, so a user cannot change the read state of their own sent messages through this action.
  - Only a participant of the conversation can mark it as read.

3.12.7 Pin Conversation

Function Trigger

The user opens the "..." menu of a conversation row and chooses "Pin chat".

Function Description

This function allows a logged-in user to keep an important conversation at the top of the conversation list. The user pins a conversation and the row is moved to the pinned group, which is always shown above the other conversations.

Screen Layout (Mock-up Description)

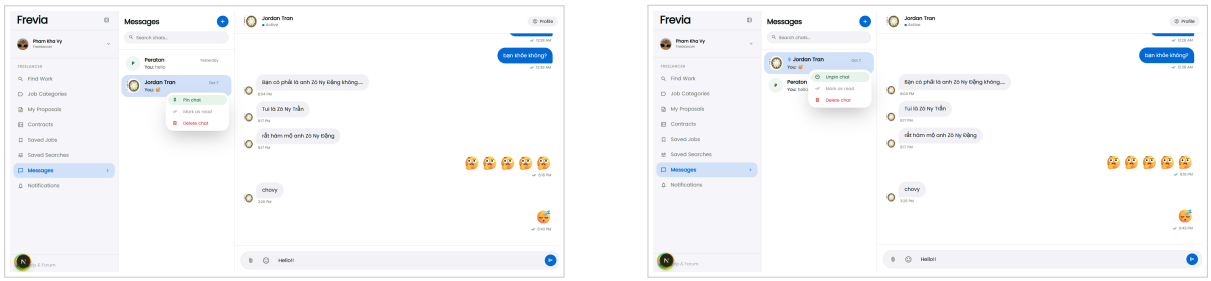

Figure 114: Conversation list before pinning (left) and after pinning (right)

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation that the user wants to pin is shown in the conversation list.
- **Process**
  - **Open the Row Menu**: The user clicks the "..." button of the conversation row and the menu is displayed.
  - **Choose Pin Chat**: The user clicks the "Pin chat" action in the menu.
  - **View the Result**: The conversation list is updated.
    - The conversation is moved to the top of the list, above the not-pinned conversations.
    - A pin icon is shown on the conversation row.
  - **Unpin if Wanted**: The user can open the same menu again and choose "Unpin chat" to remove the conversation from the pinned group.
- **Alternative Flow**
  - **Unpin Conversation**: When the user chooses "Unpin chat", the pin icon is removed and the conversation returns to the normal order of the list.
  - **Pin State Is Personal**: The pin of one user does not change the order of the conversation list of the other participant.
- **Postconditions**
  - The pinned conversation stays at the top of the conversation list of the current user.
  - The content of the conversation and the messages are not changed by this action.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the pin is not applied and a message tells the user that the conversation cannot be found.
  - **Cannot Pin Conversation**: When the pin state cannot be saved, the row returns to its previous position and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - The pin belongs to the current user only, so pinning a chat never affects the conversation list of the other participant.
  - Only a participant of the conversation can pin it.

3.12.8 Delete Message

Function Trigger

The user moves the pointer over one of their own messages in the chat thread and clicks the delete button.

Function Description

This function allows a logged-in user to remove a message that they have sent. The user deletes their own message and the message disappears from the chat thread of both participants. The user can only delete messages that they sent themselves.

Screen Layout (Mock-up Description)

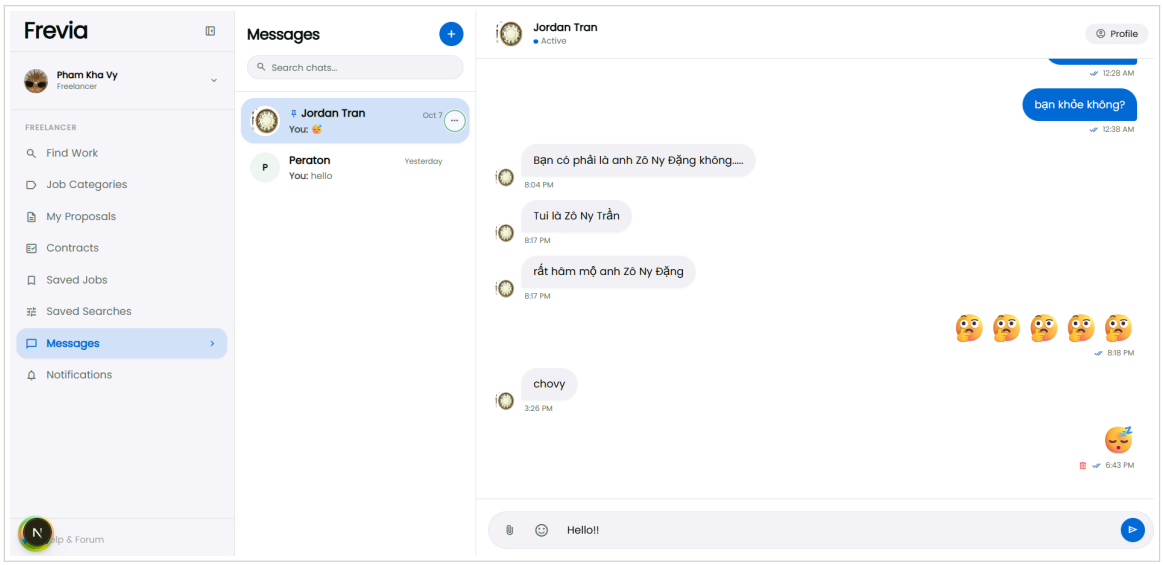

Figure 115: Deleting a message in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The message belongs to the current user and is shown in the chat thread.
- **Process**
  - **Point at the Message**: The user moves the pointer over one of their own messages and the delete button appears.
  - **Click the Delete Button**: The user clicks the delete button of that message.
  - **View the Result**: The message is removed from the thread.
    - The message disappears from the thread of the current user.
    - The message also disappears from the thread of the other participant in real time.

- **Conversation List Update**: When the deleted message was the last message of the conversation, the preview in the conversation list is refreshed to show the previous message.

- **Alternative Flow**
  - **Delete a File Message**: When the deleted message contains a file, the whole file message is removed from the thread in the same way.
  - **Message Already Deleted**: When the message has already been deleted, the delete button is not available for it.
- **Postconditions**
  - The message no longer appears in the chat thread of either participant.
  - The message is kept in the system for record-keeping but is never shown to users again.
- **Error Handling**
  - **Message Not Found**: When the message no longer exists or has already been deleted, the page shows a message telling the user that the message cannot be found.
  - **Not the Sender**: When the user tries to delete a message that they did not send, the action is refused and a permission message is shown.
  - **Cannot Delete Message**: When the message cannot be deleted, the message stays in the thread and a delete-failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - The delete button is shown only on the messages that the current user sent.
  - The system also checks the sender when the deletion is confirmed, so a user can never delete a message that belongs to the other participant.

3.13 Milestone Management

3.13.1 View Milestone List

Function Trigger

An authenticated contract participant (Client or Freelancer) accesses the Contract Detail workspace and views the default Milestones timeline tab.
Navigation path: Dashboard → Contracts → Contract Detail (`/contracts/:slug`) → Milestones Tab.

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Secondary Actor:** Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Displays the sequential deliverable roadmap of a contract. Allows participants to view all milestone stages, monitor work progress status (`PENDING`, `IN_PROGRESS`, `SUBMITTED`, `CHANGES_REQUESTED`, `COMPLETED`), observe escrow funding state (`UNPAID`, `FUNDED`, `RELEASED`), and track upcoming deadlines.

Interface

- Contract Detail workspace at `/contracts/:slug` with active "Milestones" tab.
- Financial progress banner rendering aggregate budget metrics: Total Budget ($USD), Escrow Funded Balance, Released Earnings, and Remaining Balance.
- Vertical chronological roadmap rendering milestone cards with sequence numbers, titles, delivery due dates, amounts, status badges, and action triggers.
- Role-specific contextual action buttons (e.g. "Fund Milestone" for Client, "Start Work" or "Submit Work" for Freelancer).
- Filter and status filter pills for quickly locating pending, active, or submitted milestone stages.

Data Processing Overview

- The client application requests `GET /api/contracts/:contractId/milestones`.
- `MilestoneService.getMilestoneList()` validates caller identity: verifies that the user is the contract's Client, assigned Freelancer, or an Admin.
- Queries `prisma.milestone.findMany` filtering by `contractId` where `deletedAt = null`.
- Orders milestones chronologically by creation sequence and target due date.
- Aggregates financial totals: sums funded balances, counts total submissions per milestone, and checks for open dispute flags.
- Returns HTTP 200 with the milestone collection and summary metrics.

Screen Layout (Mock-up Description)

Figure 116: Milestones timeline roadmap screen

- Top progress section displays a visual progress bar indicating percentage of project milestones approved and funded.
- Each milestone card presents a sequence step badge (e.g., Step 1, Step 2), milestone title, deliverable description summary, and dollar amount.
- Color-coded badges indicate operational status (`IN_PROGRESS` in blue, `SUBMITTED` in amber, `COMPLETED` in green).
- Expandable accordion toggle allows viewing deliverable details, attached files count, and submission history inline.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`status`	Optional milestone status filter (enum: `PENDING`, `IN_PROGRESS`, `SUBMITTED`, `COMPLETED`).

Validation Rules

- The calling user must be an authorized contract participant or an Administrator.
- The contract must exist in the database and not be deleted.

Business Rules

- **BR-MS-01**: Only active milestones where `deletedAt = null` shall be returned.
- **BR-MS-02**: Milestones shall be ordered sequentially according to milestone sequence and creation order.
- **BR-MS-03**: Milestones in `SUBMITTED` or `CHANGES_REQUESTED` state shall be automatically highlighted for action.

Normal Flow

1. The user navigates to the Contract Detail workspace and selects the Milestones tab.
2. The web client submits `GET /api/contracts/:contractId/milestones`.
3. The API validates participant access and retrieves active milestones.
4. The system computes financial totals and deliverable progress percentages.
5. The client renders the sequential milestone roadmap and role-specific action buttons.

Abnormal Flow / Exception Handling

Case	Description
A1	User is not a participant in the contract. API returns HTTP 403 Forbidden; client displays “Access denied.”
A2	Contract not found. API returns HTTP 404 Not Found; client redirects to error page.
A3	Database query fails. API returns HTTP 500; client displays retry banner.

Output Data

Data Name	Description
`milestones[]`	Array of milestone records with title, amount, status, paymentStatus, dueDate, and submission count.
`financials`	Object summarizing total budget, escrow balance, and released earnings.

Post-Conditions

- Milestone timeline is rendered with current progress and payment statuses.

3.13.2 View Milestone Detail

Function Trigger

A user clicks on a milestone card header, title link, or expansion toggle in the milestone roadmap.

Navigation path: Contract Detail → Milestones Tab → Click Milestone Card (`/contracts/:slug/mileston`

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Expands a milestone to inspect detailed deliverable requirements, acceptance criteria, attached reference files, deliverable submission history, client review feedback, and change request revision notes.

Interface

- Expanded milestone detail panel or modal view.
- Milestone header: sequence number, title, amount, escrow payment badge, and due date countdown timer.
- Deliverable specifications section rendering formatted Markdown description of expected work deliverables.
- Submissions history accordion displaying all past submission cycles with submission notes and links.

- Attached files tray with download links and file size badges.
- Review actions bar: “Approve Milestone”, “Request Changes”, “Open Dispute”, or “Submit Deliverables”.

Data Processing Overview

- The client issues an HTTP request: `GET /api/contracts/:contractId/milestones/:id`.
- `MilestoneService.getMilestoneDetail()` verifies authorization for the target contract.
- Queries `prisma.milestone.findUnique` including relation arrays: `submissions`, `files`, and dispute links.
- Validates that the requested milestone belongs to the designated `contractId` and has `deletedAt = null`.
- Returns HTTP 200 with the composite milestone entity.

Screen Layout (Mock-up Description)

Figure 117: Milestone detail view screen

- The panel header renders the milestone title, sequence tag, due date badge, and USD budget badge.
- Scope section displays detailed criteria and deliverables required for approval.
- Latest submission box displays freelancer submission notes, external preview URL, and deliverable attachments.
- If revisions were requested, a feedback callout renders the client's revision instructions.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`id`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Milestone ID must exist and be a valid child of the specified contract.
- Caller must be an authorized contract participant or an Administrator.

Business Rules

- **BR-MS-04**: Deliverable specifications and acceptance criteria are legally binding once the milestone is funded.
- **BR-MS-05**: Milestone submissions history must preserve all revision cycles for audit traceability.

Normal Flow

1. The user clicks a milestone card in the timeline.
2. The client requests `GET /api/contracts/:contractId/milestones/:id`.
3. The API validates access and loads milestone details with nested submissions and files.
4. The client expands the milestone panel and renders full deliverable criteria and history.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone not found or deleted. API returns HTTP 404 Not Found; client displays “Milestone not found.”
A2	Caller is not authorized. API returns HTTP 403 Forbidden; access denied.
A3	Network connection failure. Client displays error message with retry button.

Output Data

Data Name	Description
`milestone`	Full milestone entity including title, description, amount, status, paymentStatus, submissions[], and files[].

Post-Conditions

- Detailed milestone criteria, files, and submission cycles are displayed.

3.13.3 Create Milestone

Function Trigger

A Client clicks the "+ Add Milestone" button in the contract workspace to define a new project milestone.
Navigation path: Contract Workspace → Milestones Tab → Click "+ Add Milestone".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Client to append a new deliverable milestone stage to an existing contract, defining milestone title, deliverables scope, monetary value, and target completion deadline.

Interface

- "Add Milestone" modal dialog or inline form builder.
- Milestone title text field (5–100 characters).
- Deliverable description textarea detailing expected tasks and acceptance criteria.
- Amount input field formatted in USD with live currency validation.
- Due date picker requiring selection of a future completion date.
- Action buttons: "Create Milestone" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `POST /api/contracts/:contractId/milestones` with `title`, `description`, `amount`, and `dueDate`.

- `MilestoneService.createMilestone()` verifies that caller is the contract's Client.
- Checks that contract status is `PENDING`, `DRAFT`, or `ACTIVE`.
- Computes next sequential order index.
- Persists new `Milestone` entity with initial status `PENDING` and `paymentStatus = UNPAID`.
- Inside a transaction, recalculates contract `totalAmount` to include the new milestone budget.
- Dispatches an alert notification to the Freelancer.

Screen Layout (Mock-up Description)

Figure 118: Create Milestone dialog

- Dialog features title "Add New Milestone" and contract reference header.
- Input fields: Milestone Name, Detailed Description, Amount ($USD), and Due Date picker.

- Informational notice notes that adding a milestone increases total contract value.
- Footer buttons: “Cancel” and “Add Milestone”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`title`	Milestone title (string, 5–100 characters).
`description`	Detailed deliverables and acceptance requirements (string, 10–2000 characters).
`amount`	Monetary milestone value in USD (decimal, minimum 5.00).
`dueDate`	Target delivery deadline formatted as ISO 8601 future date.

Validation Rules

- Only the Client owner of the contract can create milestones.
- Contract must not be in `COMPLETED` or `CANCELLED` status.
- Milestone amount must be positive and at least $5.00.
- Due date must be strictly in the future.

Business Rules

- **BR-MS-06**: Adding a milestone dynamically updates total contract budget commitments.
- **BR-MS-07**: Newly created milestones are initialized with status `PENDING` and payment status `UNPAID`.

Normal Flow

1. The Client clicks “+ Add Milestone” in the contract roadmap.
2. The system opens the Create Milestone dialog.
3. The Client enters title, description, amount, and due date.
4. The Client clicks “Create Milestone”.
5. The system validates fields, computes sequential index, and persists the milestone.
6. Contract total budget is updated to reflect the new amount.
7. The dialog closes, the milestone card is added to the roadmap, and the Freelancer is notified.

Abnormal Flow / Exception Handling

Case	Description
A1	Amount is less than 5.00 USD. API returns HTTP 422 with validation error.
A2	Due date is in the past. API returns HTTP 422; displays "Due date must be in the future."
A3	Caller is not the Client owner. API returns HTTP 403 Forbidden; access denied.
A4	Contract is completed or cancelled. API returns HTTP 422; creation rejected.

Output Data

Data Name	Description
`milestone`	Created milestone entity with generated ID, status, and timestamps.
`contractTotal`	Updated aggregated contract budget commitment.

Post-Conditions

- New milestone record created with status `PENDING`.
- Contract total budget updated.
- Freelancer notified of the new milestone.

3.13.4 Update Milestone

Function Trigger

A Client clicks the "Edit" action on an unfunded or pending milestone card.

Navigation path: Contract Detail → Milestones Tab → Milestone Card Menu → Click "Edit Milestone".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Client to adjust deliverable descriptions, milestone titles, budget allocations, or deadline dates on milestones that have not yet been funded or locked into active escrow execution.

Interface

- Edit Milestone modal pre-populated with current milestone properties.
- Title input, Markdown deliverables textarea, amount input, and due date picker.
- Action buttons: “Save Changes” and “Cancel”.

Data Processing Overview

- Client submits an HTTP request: `PATCH /api/contracts/:contractId/milestones/:id` with updated attributes.
- `MilestoneService.updateMilestone()` verifies that caller is the contract’s Client.
- Checks that milestone payment status is `UNPAID`; funded milestones cannot have their amounts altered without refund.
- Updates milestone attributes in the database and recalculates contract total budget.
- Records update activity and notifies the Freelancer.

Screen Layout (Mock-up Description)

Figure 119: Edit Milestone modal

- Modal displays header “Edit Milestone #Step”.
- Form fields display existing milestone data with edit focus.
- Footer renders secondary “Discard” and primary “Save Changes” buttons.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`id`	Numeric milestone identifier (integer path parameter).
`title`	Updated milestone title (string, optional).
`description`	Updated deliverable description (string, optional).
`amount`	Updated monetary value (decimal, optional).
`dueDate`	Updated deadline date (ISO 8601 future date, optional).

Validation Rules

- Caller must be the contract's Client.
- Milestone must reside in `UNPAID` payment status.
- Due date must be set in the future.

Business Rules

- **BR-MS-08**: Milestones that have already been funded in escrow cannot be modified without formal cancellation or refund.

Normal Flow

1. Client selects "Edit Milestone" on an unfunded milestone card.
2. The system displays the edit modal with current attributes.
3. Client modifies fields and clicks "Save Changes".
4. System validates inputs and updates database records.
5. Updated milestone is rendered and Freelancer is notified.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is already funded. API returns HTTP 422 with error `MilestoneAlreadyFundedException`; editing locked.
A2	Caller is not the Client. API returns HTTP 403 Forbidden; access denied.
A3	Invalid due date provided. System highlights date error.

Output Data

Data Name	Description
`milestone`	Updated milestone record with revised attributes.

Post-Conditions

- Milestone attributes updated in database.
- Freelancer notified of changes.

3.13.5 Delete Milestone

Function Trigger

A Client clicks the "Delete Milestone" action on an unfunded milestone card.

Navigation path: Contract Detail → Milestones Tab → Milestone Menu → Click "Delete Milestone".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Client to remove an unnecessary or cancelled milestone stage from a contract before it has been funded in escrow.

Interface

- Delete action option in milestone card contextual menu.
- Confirmation dialog: "Are you sure you want to delete this milestone? Total contract value will be reduced.".
- Action buttons: "Delete Milestone" (destructive red) and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `DELETE /api/contracts/:contractId/milestones/:id`.
- `MilestoneService.deleteMilestone()` validates caller is the contract's Client.
- Checks that milestone payment status is strictly `UNPAID`; funded milestones cannot be deleted.

- Soft-deletes milestone by setting `deletedAt = new Date()`.
- Adjusts contract `totalAmount` downwards to exclude the deleted milestone value.
- Returns HTTP 200 with deletion confirmation.

Screen Layout (Mock-up Description)

Figure 120: Delete Milestone confirmation dialog

- Modal displays red trash icon and heading “Delete Milestone”.
- Message warns that deleting removes the deliverable stage and deducts its amount from total budget.
- Footer buttons: “Cancel” and “Confirm Delete”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`id`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Caller must be the Client owner of the contract.
- Milestone must reside in `UNPAID` payment status.
- At least one milestone must remain on the contract.

Business Rules

- **BR-MS-09**: Funded milestones cannot be deleted; they must be refunded or completed.
- **BR-MS-10**: Deleting a milestone reduces the aggregate contract budget commitment.

Normal Flow

1. Client clicks “Delete Milestone” on an unfunded milestone card.
2. The system opens the confirmation dialog.
3. Client clicks “Confirm Delete”.
4. System marks milestone with `deletedAt = now()` and recalculates contract total.
5. Dialog closes, card is removed from roadmap, and success notification appears.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is funded in escrow. API returns HTTP 422 with error `CannotDeleteFundedMilestoneException`; deletion blocked.
A2	Attempt to delete the only remaining milestone. API returns HTTP 422; contracts must have at least one milestone.
A3	Caller is not the Client. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`success`	Boolean confirmation of successful deletion (`true`).
`updatedContractTotal`	Recalculated contract budget value.

Post-Conditions

- Milestone is soft-deleted in the database.
- Contract budget updated and card removed from UI.

3.13.6 Request Milestone Changes

Function Trigger

The Client clicks the "Request Changes" button on a submitted milestone review panel after inspecting deliverables.
Navigation path: Contract Detail → Milestones Tab → Submitted Milestone → Click "Request Changes".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Client to reject a submitted deliverable cycle and request revisions from the Freelancer, providing detailed written feedback, bug reports, and specific adjustment requirements.

Interface

- "Request Changes" button in the review toolbar of a submitted milestone.
- Request Changes modal dialog with feedback textarea supporting rich formatting.
- Revision guideline notes: prompts client to provide actionable, concrete feedback.
- Action buttons: "Send Revision Request" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `PATCH /api/contracts/:contractId/milestones/:milestoneId/` with `comment`.

- `MilestoneSubmissionService.requestChanges()` verifies caller is the contract's Client.
- Checks that milestone status is currently `SUBMITTED`.
- Updates submission record with client review comment, review status `CHANGES_REQUESTED`, and review timestamp.
- Transitions milestone status to `CHANGES_REQUESTED`.
- Dispatches revision alert notification to the Freelancer.

Screen Layout (Mock-up Description)

Figure 121: Request Milestone Changes dialog

- Modal displays title "Request Deliverable Changes".
- Textarea for revision feedback with placeholder "Explain what changes are required before approval…".

- Character count indicator (minimum 10 characters).
- Footer renders secondary “Cancel” and primary orange “Send Change Request” buttons.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`id`	Numeric submission identifier (integer path parameter).
`comment`	Detailed client feedback explaining required adjustments (string, 10–2000 characters).

Validation Rules

- Caller must be the Client owner of the contract.
- Target milestone must reside in `SUBMITTED` status.
- Feedback comment must contain at least 10 non-whitespace characters.

Business Rules

- **BR-MS-11**: Requesting changes transitions milestone status to `CHANGES_REQUESTED` and resets the approval timer.
- **BR-MS-12**: The milestone remains funded in escrow while changes are being performed by the freelancer.

Normal Flow

1. Client inspects submitted deliverables and clicks “Request Changes”.
2. System opens the Request Changes dialog.
3. Client types detailed feedback and clicks “Send Revision Request”.
4. System updates submission and sets milestone status to `CHANGES_REQUESTED`.
5. Modal closes, UI displays revision status, and Freelancer receives revision alert.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not in `SUBMITTED` status. API returns HTTP 422 with error `MilestoneNotSubmittedException`.
A2	Feedback comment is shorter than 10 characters. Form validation rejects submission.
A3	Caller is not the Client. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`submission`	Updated submission entity with revision comment and review status.
`milestoneStatus`	Updated milestone status (`CHANGES_REQUESTED`).

Post-Conditions

- Milestone status updated to `CHANGES_REQUESTED`.
- Feedback recorded in submission history.
- Freelancer notified to re-work deliverables.

3.13.7 Approve Milestone

Function Trigger

The Client clicks the "Approve Milestone" button on a submitted milestone after verifying all deliverables meet quality criteria.
Navigation path: Contract Detail → Milestones Tab → Submitted Milestone → Click "Approve Milestone".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe Connect payment release service, Prisma ORM, and PostgreSQL database.

Purpose

Formally accepts submitted milestone deliverables as complete and satisfactory. Automatically triggers escrow fund release via Stripe Connect, transferring milestone earnings to the Freelancer's account minus platform service fees, and marks the milestone as `COMPLETED`.

Interface

- Prominent green "Approve Milestone" action button on submitted milestone card.
- Approval confirmation modal displaying milestone deliverable summary, escrow release amount, and platform fee breakdown.
- Action buttons: "Confirm Approval & Release Funds" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `PATCH /api/contracts/:contractId/milestones/:milestoneId/`
- `MilestoneSubmissionService.approveMilestone()` verifies caller is the contract's Client.
- Verifies milestone is in `SUBMITTED` or `CHANGES_REQUESTED` status, and escrow `paymentStatus = FUNDED`.
- Checks that Freelancer has completed Stripe Connect onboarding.
- Inside a database transaction:
  - Transitions submission status to `APPROVED`.
  - Transitions milestone status to `COMPLETED` and sets `paymentStatus = RELEASED`.
  - Invokes Stripe Transfer to release net earnings to freelancer's connected account.
  - Records ledger transaction records (`ESCROW_RELEASE`).
- Dispatches immediate payment receipt notification to Freelancer.

Screen Layout (Mock-up Description)

Figure 122: Approve Milestone confirmation dialog

- Modal displays green checkmark icon and heading “Approve Milestone Deliverables”.
- Financial summary card renders milestone gross amount, platform commission deduction, and net payout.
- Confirmation message states that approving releases funds immediately to the freelancer.
- Footer buttons: “Cancel” and “Approve & Release Funds”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`id`	Numeric submission identifier (integer path parameter).

Validation Rules

- Caller must be the Client owner of the contract.
- Milestone must reside in `SUBMITTED` or `CHANGES_REQUESTED` status.
- Escrow funds must be in `FUNDED` status.
- Freelancer must have verified Stripe Connect payout capability.

Business Rules

- **BR-MS-13**: Approving a milestone automatically initiates immediate escrow payout release to the freelancer.
- **BR-MS-14**: Approved milestones are permanently locked from further revision or refund requests.

Normal Flow

1. Client inspects deliverables and clicks “Approve Milestone”.
2. System opens the approval confirmation modal with financial breakdown.
3. Client clicks “Confirm Approval & Release Funds”.
4. System marks submission `APPROVED`, sets milestone `COMPLETED`, and releases escrow.
5. Modal closes, milestone card turns green, and Freelancer receives payment notification.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not funded in escrow. API returns HTTP 422 with error `MilestoneNotFundedException`; release blocked.

Case	Description
A2	Freelancer Stripe account is incomplete. API returns HTTP 422 with error `StripeOnboardingIncompleteException`; prompts freelancer to connect.
A3	Caller is not the Client. API returns HTTP 403 Forbidden; access denied.
A4	Stripe transfer failure occurs. Escrow release is queued for automated retry; milestone marked approved.

Output Data

Data Name	Description
`milestone`	Updated milestone entity with status `COMPLETED` and paymentStatus `RELEASED`.
`transaction`	Ledger transaction entity documenting released escrow amount and Stripe Transfer ID.

Post-Conditions

- Milestone status updated to `COMPLETED` and `paymentStatus = RELEASED`.
- Escrow payout transferred to freelancer's Stripe account.
- Both parties notified.

3.13.8 Update Milestone Progress

Function Trigger

The Freelancer clicks the "Start Work" button on an assigned, funded milestone card. **Navigation path:** Contract Detail → Milestones Tab → Milestone Card → Click "Start Work".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer (`FREELANCER` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Freelancer to acknowledge milestone assignment and transition milestone status from

PENDING to IN_PROGRESS, signaling active engagement on deliverables to the Client.

Interface

- "Start Work" action button on funded milestone cards.
- Status pill badge dynamically shifting to blue "In Progress".

Data Processing Overview

- Freelancer submits an HTTP request: PATCH /api/contracts/:contractId/milestones/:id/progr
- MilestoneService.progressMilestone() verifies caller is the contract's Freelancer.
- Checks that milestone status is PENDING and escrow paymentStatus = FUNDED.
- Updates milestone status to IN_PROGRESS.
- Dispatches an update notification to the Client.

Screen Layout (Mock-up Description)

Figure 123: Milestone progress update interface

- Milestone card displays “Start Work” button for funded pending milestones.
- Upon click, status pill updates to “In Progress” and the “Submit Work” button becomes enabled.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`id`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Caller must be the assigned Freelancer of the contract.
- Milestone must reside in `PENDING` status.
- Milestone escrow must be `FUNDED`.

Business Rules

- **BR-MS-15**: Freelancers can only transition milestones to `IN_PROGRESS` after the client has funded escrow.

Normal Flow

1. Freelancer views funded milestone card and clicks "Start Work".
2. System validates authorization and updates milestone status to `IN_PROGRESS`.
3. UI updates status badge and notifies Client that work has begun.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not funded in escrow. API returns HTTP 422 with error `MilestoneNotFundedException`; progress blocked.
A2	Caller is not the Freelancer. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`status`	Updated milestone status (`IN_PROGRESS`).

Post-Conditions

- Milestone status transitioned to `IN_PROGRESS`.
- Client notified of work initiation.

3.13.9 View Milestone Files

Function Trigger

A user clicks the "Attachments" accordion or tab inside a milestone detail panel.

Navigation path: Contract Detail → Milestone Detail → Click "Attachments".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Displays all files, reference documents, and deliverable attachments associated with a specific milestone stage.

Interface

- Milestone files list within the milestone card.
- File cards rendering format icon, file name, uploader name, upload date, size, and download link.
- Delete icon visible to file uploader and Admin.

Data Processing Overview

- Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/files`.
- `MilestoneFileService.getMilestoneFiles()` validates participant authorization.
- Queries `prisma.milestoneFile.findMany` where `milestoneId = id` and `deletedAt = null`.
- Returns HTTP 200 with the file attachments array.

Screen Layout (Mock-up Description)

Figure 124: Milestone files attachment tray

- Attachment section lists uploaded files with file type icons, name, file size badge, and download action.
- An upload dropzone or button is available when editing or preparing submissions.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Caller must be an authorized contract participant or an Administrator.
- Milestone must exist and belong to the contract.

Business Rules

- **BR-MSF-01**: Only non-deleted files associated with the milestone shall be displayed.

Normal Flow

1. User expands milestone attachments.
2. Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/files`.
3. API returns files array, and client renders attachment cards.
4. User clicks download link to retrieve any file.

Abnormal Flow / Exception Handling

Case	Description
A1	User is not a participant. API returns HTTP 403 Forbidden; access denied.
A2	Milestone does not exist. API returns HTTP 404 Not Found.

Output Data

Data Name	Description
`files[]`	Array of milestone file records with id, fileName, fileUrl, and uploader.

Post-Conditions

- Milestone attachments rendered for download.

3.13.10 Upload Milestone File

Function Trigger

A Freelancer uploads deliverable attachments during work progress or before final submission.
Navigation path: Contract Detail → Milestone Detail → Click "Upload Deliverable File".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer, Client.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Multer middleware, Cloudinary storage, Prisma ORM, and PostgreSQL database.

Purpose

Allows users to upload work attachments, source code packages, documentation, and design assets to a specific milestone.

Interface

- Drag-and-drop file upload zone in milestone deliverable section.
- File upload modal with progress meter and file size verification ($\leq$ 25MB).
- Action buttons: "Upload" and "Cancel".

Data Processing Overview

- Client submits an HTTP `multipart/form-data` request: `POST /api/contracts/:contractId/miles`
- `buildUploadFilePipe()` validates file size ($\leq$ 25MB) and prohibits malicious executable extensions.
- `MilestoneFileService.uploadMilestoneFile()` streams file buffer to Cloudinary cloud storage.
- Persists `MilestoneFile` record in database linked to milestone and uploader.
- Returns HTTP 201 Created with the saved file record.

Screen Layout (Mock-up Description)

Figure 125: Upload Milestone File modal

- Dropzone area with upload cloud icon and drag-and-drop prompt.
- Selected file displays file name, size badge, and progress bar during transmission.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`file`	Binary file stream via `multipart/form-data`.

Validation Rules

- File size must not exceed 25MB.
- Executable extensions prohibited.
- Caller must be a participant in the contract.

Business Rules

- **BR-MSF-02**: Maximum milestone file attachment size is 25MB.
- **BR-MSF-03**: Uploaded files are tagged with uploader ID for ownership tracking.

Normal Flow

1. User selects file to upload on milestone card.
2. Client sends multipart form data to server.
3. Server validates format and size, then uploads to Cloudinary.
4. Database records new `MilestoneFile` entity.
5. File appears in milestone attachment tray with success confirmation.

Abnormal Flow / Exception Handling

Case	Description
A1	File exceeds 25MB. API returns HTTP 400 Bad Request; displays size limit error.
A2	Disallowed file extension. API returns HTTP 400; displays format error.
A3	Upload timeout. API returns HTTP 502; displays retry banner.

Output Data

Data Name	Description
`file`	Created milestone file record with id, fileName, fileUrl, and createdAt.

Post-Conditions

- File asset stored in cloud storage and linked in database.

3.13.11 Delete Milestone File

Function Trigger

The uploader or Admin clicks the trash icon next to a milestone file attachment.
Navigation path: Milestone Attachments → File Row → Click "Delete".

Function Description

Actors / Roles

- **Primary Actor:** File Uploader, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Cloudinary, Prisma ORM, and PostgreSQL database.

Purpose

Enables users to remove obsolete, incorrect, or superseded deliverable files from a milestone prior to final review.

Interface

- Trash icon button next to file items in the milestone attachment list.
- Delete confirmation dialog: "Are you sure you want to delete [fileName]?".
- Action buttons: "Delete" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `DELETE /api/contracts/:contractId/milestones/:milestoneId`
- `MilestoneFileService.deleteMilestoneFile()` validates caller is original uploader or Admin.
- Checks that milestone is not already in `COMPLETED` status.
- Marks database record with `deletedAt = new Date()` and purges cloud asset.
- Returns HTTP 200 with deletion confirmation.

Screen Layout (Mock-up Description)

Figure 126: Delete Milestone File dialog

- Confirmation dialog displays file name and delete prompt.
- Footer buttons: “Cancel” and red “Delete File”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`fileId`	Numeric file identifier (integer path parameter).

Validation Rules

- Caller must be the original file uploader or an Administrator.
- File must exist and not already be deleted.

Business Rules

- **BR-MSF-04**: Deliverable files attached to already completed milestones cannot be deleted.

Normal Flow

1. User clicks trash icon next to an uploaded file.
2. Confirmation dialog appears.
3. User clicks “Delete”.
4. System marks record as deleted and purges cloud storage asset.
5. File card is removed from view with confirmation toast.

Abnormal Flow / Exception Handling

Case	Description
A1	Caller is not the file uploader and not an Admin. API returns HTTP 403 Forbidden; access denied.
A2	File already deleted or not found. API returns HTTP 404 Not Found.

Output Data

Data Name	Description
`success`	Boolean confirmation of successful deletion (`true`).

Post-Conditions

- File soft-deleted in database and removed from milestone card.

3.13.12 Submit Milestone

Function Trigger

The Freelancer clicks the "Submit Work" button on an in-progress milestone card to deliver finished work for Client review.
Navigation path: Contract Detail → Milestones Tab → Milestone Card → Click "Submit Work".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer (`FREELANCER` role).
- **Secondary Actor:** Client (`CLIENT` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Formally delivers completed milestone work to the Client for inspection. The Freelancer includes submission notes, external demo/staging URLs, and deliverable attachments, transitioning milestone status to `SUBMITTED`.

Interface

- Submit Work modal dialog.
- Submission notes textarea (Markdown supported, 10–2000 characters).
- Optional external preview URL text input (e.g. staging server, Figma link, GitHub PR).
- Deliverable file attachments selector.
- Action buttons: "Submit for Approval" and "Cancel".

Data Processing Overview

- Freelancer submits an HTTP request: `POST /api/contracts/:contractId/milestones/:milestone`
  with `notes`, `externalLink`, and `fileIds`.

- `MilestoneSubmissionService.submitMilestone()` validates caller is the contract's Freelancer.
- Checks milestone status: must be `IN_PROGRESS` or `CHANGES_REQUESTED`, and escrow must be `FUNDED`.
- Persists new `MilestoneSubmission` record with status `PENDING_REVIEW`.
- Transitions milestone status to `SUBMITTED`.
- Dispatches an urgent review notification to the Client with deliverable summary.

Screen Layout (Mock-up Description)

Figure 127: Submit Milestone modal

- Modal displays title "Submit Milestone Deliverables".
- Detailed textarea for submission notes and work explanation.
- Optional external link input with URL format placeholder.

- Attachment picker displaying attached deliverable files.
- Footer buttons: “Cancel” and primary blue “Submit Work for Approval”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`notes`	Detailed description of deliverables and instructions for review (string, 10–2000 characters).
`externalLink`	Optional external URL to live demo, repository, or prototype (string, valid URL format).
`fileIds`	Optional array of integer IDs representing uploaded milestone files.

Validation Rules

- Caller must be the assigned Freelancer of the contract.
- Milestone status must be `IN_PROGRESS` or `CHANGES_REQUESTED`.
- Milestone escrow must be in `FUNDED` status.
- Submission notes must be at least 10 characters in length.

Business Rules

- **BR-MSS-01**: Submitting deliverables transitions milestone status to `SUBMITTED` and activates the client review countdown.
- **BR-MSS-02**: Submissions create an immutable audit record preserving files and notes for dispute protection.

Normal Flow

1. Freelancer clicks “Submit Work” on an in-progress milestone card.
2. System displays Submit Milestone modal.
3. Freelancer types submission notes, attaches files, and inputs optional demo link.
4. Freelancer clicks “Submit for Approval”.
5. System validates fields, creates submission entity, and sets milestone status to `SUBMITTED`.
6. Modal closes, card displays amber “Submitted” badge, and Client receives review alert.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not in progress. API returns HTTP 422 with error `MilestoneNotEligibleForSubmissionException`.
A2	Milestone escrow is unfunded. API returns HTTP 422; submission rejected.
A3	Caller is not the Freelancer. API returns HTTP 403 Forbidden; access denied.
A4	Submission notes empty. Form validation highlights missing notes.

Output Data

Data Name	Description
`submission`	Created submission entity with id, notes, externalLink, status, and createdAt.
`milestoneStatus`	Updated milestone status (`SUBMITTED`).

Post-Conditions

- Submission entity created with status `PENDING_REVIEW`.
- Milestone status updated to `SUBMITTED`.
- Client notified to review deliverables.

3.13.13 Delete Milestone Submission

Function Trigger

The Freelancer clicks "Recall / Delete Submission" on a pending submission before the Client has completed review.
Navigation path: Contract Detail → Milestones Tab → Submitted Milestone → Click "Recall Submission".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer (`FREELANCER` role).

- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows a Freelancer to recall and delete a pending submission if they discovered an omission or error before the Client reviews it, returning the milestone to `IN_PROGRESS`.

Interface

- "Recall Submission" button on pending submission review card.
- Confirmation dialog: "Recall this submission? The milestone will return to In Progress.".
- Action buttons: "Confirm Recall" and "Cancel".

Data Processing Overview

- Freelancer submits an HTTP request: `DELETE /api/contracts/:contractId/milestones/:milesto`
- Verifies caller is the contract's Freelancer.
- Checks that submission status is strictly `PENDING_REVIEW`; approved or changes-requested submissions cannot be deleted.
- Soft-deletes submission record by setting `deletedAt = new Date()`.
- Reverts milestone status back to `IN_PROGRESS`.
- Returns HTTP 200 with confirmation.

Screen Layout (Mock-up Description)

Figure 128: Recall Milestone Submission dialog

- Modal displays warning that recalling withdraws the deliverables from the client queue.
- Buttons: “Cancel” and “Recall Submission”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`id`	Numeric submission identifier (integer path parameter).

Validation Rules

- Caller must be the assigned Freelancer.
- Submission status must be `PENDING_REVIEW`.

Business Rules

- **BR-MSS-03**: Submissions that have already been reviewed or approved cannot be recalled.

Normal Flow

1. Freelancer clicks "Recall Submission" on pending submission.
2. Confirmation dialog appears.
3. Freelancer clicks "Confirm Recall".
4. System marks submission deleted and reverts milestone to `IN_PROGRESS`.
5. Milestone card returns to In Progress state, enabling re-submission.

Abnormal Flow / Exception Handling

Case	Description
A1	Submission already approved or reviewed. API returns HTTP 422; recall blocked.
A2	Caller is not the Freelancer. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`success`	Boolean confirmation of deletion (`true`).
`milestoneStatus`	Reverted milestone status (`IN_PROGRESS`).

Post-Conditions

- Submission soft-deleted.
- Milestone reverted to `IN_PROGRESS`.

3.13.14 View Milestone Submission List

Function Trigger

A contract participant opens the "Submission History" tab or accordion within a milestone detail panel.
Navigation path: Contract Detail → Milestone Detail → Click "Submission History".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Renders the historical timeline of all deliverable submission and review cycles for a milestone, allowing participants to compare revisions and review comments.

Interface

- Submissions history list within the milestone card.
- Cycle cards numbered chronologically (Cycle 1, Cycle 2, etc.).
- Review status pills: "Pending Review", "Changes Requested", "Approved".
- Submission timestamp, notes summary, external link, and attached files count.

Data Processing Overview

- Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/submissions`.
- `MilestoneSubmissionService.getSubmission()` validates participant access.

- Queries `prisma.milestoneSubmission.findMany` where `milestoneId = id` and `deletedAt = null`.
- Orders submissions chronologically by `createdAt asc`.
- Returns HTTP 200 with the submissions collection.

Screen Layout (Mock-up Description)

Figure 129: Milestone submission history timeline

- Vertical timeline tracks each submission round with cycle number and date.
- Each entry shows freelancer submission notes and client revision comments.
- Status tag indicates whether cycle was approved or requested for revisions.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Caller must be an authorized contract participant or Admin.
- Milestone must exist.

Business Rules

- **BR-MSS-04**: Submission history records shall remain accessible throughout the contract lifecycle.

Normal Flow

1. User expands submission history in milestone card.
2. Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/submissions`.
3. API queries non-deleted submissions.
4. Client renders submission timeline cards.

Abnormal Flow / Exception Handling

Case	Description
A1	User is not a participant. API returns HTTP 403 Forbidden; access denied.
A2	Milestone does not exist. API returns HTTP 404 Not Found.

Output Data

Data Name	Description
`submissions[]`	Array of submission records with cycle index, notes, status, and comments.

Post-Conditions

- Submission timeline rendered for review.

3.13.15 View Milestone Submission Detail

Function Trigger

A user clicks on a specific submission entry in the submission history timeline to inspect full deliverable details.
Navigation path: Milestone Detail → Submission History → Click Submission Item (`.../submissions/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Renders granular details for a specific submission cycle, including full deliverable notes, external links, attached deliverable files, review feedback, and exact timestamps.

Interface

- Submission detail modal or dedicated expanded view.
- Submission header: cycle number, submitter profile, and submission timestamp.
- Full deliverable notes rendered in Markdown.
- External demo hyperlink with external icon.
- Attached deliverable files grid with download buttons.
- Client review section with review status badge and comments (if reviewed).

Data Processing Overview

- Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/submissions/:id`

- `MilestoneSubmissionService.getSubmissionDetail()` validates participant authorization.
- Queries `prisma.milestoneSubmission.findUnique` including attached files and reviewer info.
- Returns HTTP 200 with the submission detail entity.

Screen Layout (Mock-up Description)

Figure 130: Milestone submission detail screen

- Modal renders header with submission cycle number and review status pill.
- Main area renders deliverable notes and external demo link.
- Attachment tray displays all files included in this submission cycle with download links.
- Review block displays client review comment and approval/changes-requested date.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`id`	Numeric submission identifier (integer path parameter).

Validation Rules

- Submission ID must exist and belong to the specified milestone.
- Caller must be an authorized contract participant or Admin.

Business Rules

- **BR-MSS-05**: Submission details provide an immutable record of work deliverables submitted at that specific timestamp.

Normal Flow

1. User clicks a submission entry in the history timeline.
2. Client requests `GET /api/contracts/:contractId/milestones/:milestoneId/submissions/:id`
3. API validates access and loads submission details with files.
4. System opens submission modal displaying deliverable notes, links, and review status.

Abnormal Flow / Exception Handling

Case	Description
A1	Submission not found. API returns HTTP 404 Not Found.
A2	Caller not authorized. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`submission`	Complete submission entity with notes, externalLink, files[], reviewStatus, reviewComment, and timestamps.

Post-Conditions

- Submission details displayed with full deliverable assets and review history.

3.14 User Management

3.14.1 View User List

Function Trigger

An administrator navigates to the Users management page from the admin dashboard sidebar navigation.
Navigation path: Admin Dashboard → User Management → Users (`/admin/users`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view, search, filter, and sort all registered user accounts in the system. The catalog displays key account attributes including user ID, email address, display name, account ban/active status, creation timestamp, and assigned system or custom roles.

Interface

- User management screen at `/admin/users`.
- Filter bar component (`UsersFilterBar`) containing search input, role filter selector, sort column dropdown, and sort direction toggle.
- Data table component (`UsersTable`) rendering user rows, avatar badges, status indicators, role tags, and contextual action menus.
- Numbered pagination component at the bottom for page navigation.
- “Create User” action button in the page header.

Data Processing Overview

- The client sends an HTTP request: `GET /api/users` with query parameters (`page`, `limit`, `search`, `role`, `sortBy`, `sortOrder`).
- The backend `AuthGuard` validates the administrator's JWT access token, and `PermissionGuard` verifies permission to access the endpoint.
- `UsersController.getUsers()` parses query parameters via `AdminUserQueryDto`.
- `UsersRepository.getUsers()` queries `prisma.user.findMany` where `deletedAt = null`:
  - If `search` is provided, searches `email` or `profile.displayName` case-insensitively.
  - If `role` is provided (and not `'all'`), filters users whose assigned roles match the role name.
  - Orders records by the requested column (`id`, `email`, `createdAt`, `isBanned`, or `displayName`) and direction.
- The repository executes `prisma.user.count()` to compute total matching records in parallel.
- The system returns HTTP 200 with the users array and pagination metadata.
- The frontend renders user cards/rows, applies ban badges (Active / Banned), and maps assigned roles to visual badges.

Screen Layout (Mock-up Description)

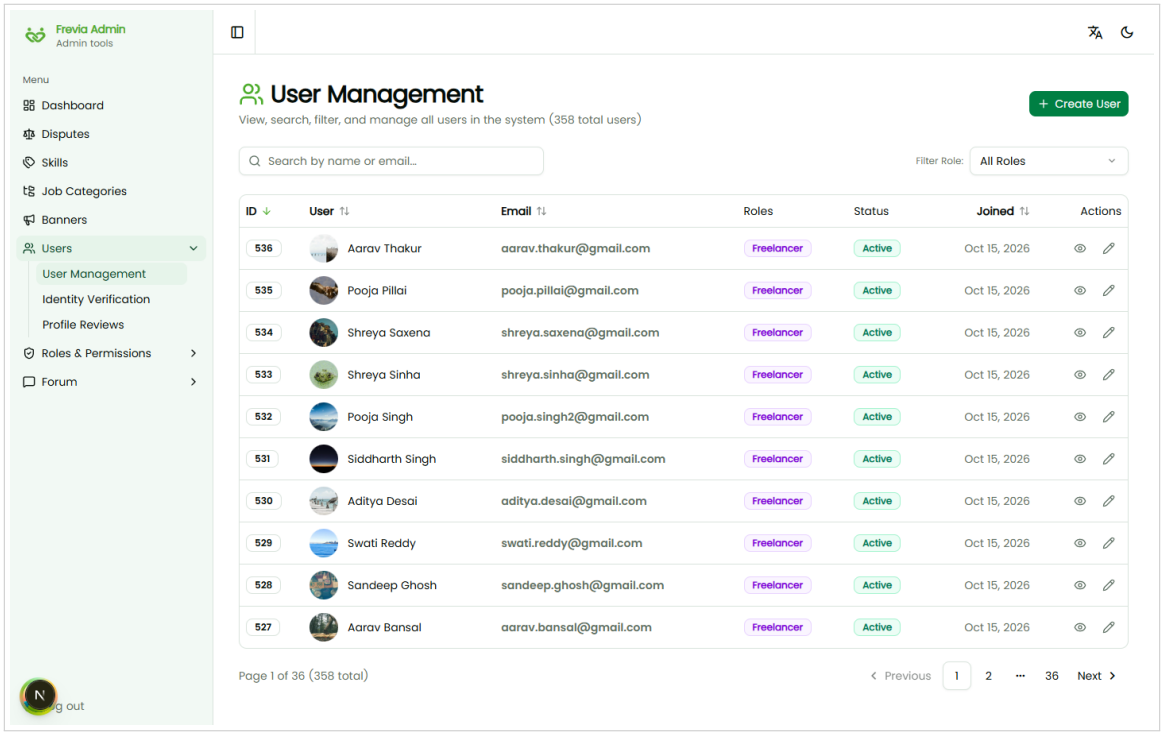

Figure 131: User list interface

- Page header displaying "Users" title, summary count, and a "Create User" button.

- Filter bar including search box, role filter dropdown (All, Client, Freelancer, Admin, Custom), and sort selector.
- Table columns: **User** (Avatar, Display Name, and Email), **Roles** (Colored badges for assigned roles), **Status** (Green "Active" or Red "Banned"), **Created Date**, and **Actions**.
- Action menu on each row offering: "View detail", "Edit user", and quick actions.
- Pagination footer indicating total count and page selection buttons.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum records returned per page (defaults to 10).
search	Optional text query matching user email or display name.
role	Optional role filter string (e.g., `CLIENT`, `FREELANCER`, `ADMIN`, `CUSTOM`, or specific custom role name).
sortBy	Sort column field (`id`, `email`, `createdAt`, `isBanned`, or `displayName`; defaults to `id`).
sortOrder	Sort direction sequence (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters; whitespace trimmed.
role	String, optional, max length 50 characters.
sortBy	Must strictly be one of: `id`, `email`, `createdAt`, `isBanned`, `displayName`.
sortOrder	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-USER-01**: Only active user accounts with `deletedAt = null` shall be returned. Soft-deleted accounts are excluded.

- **BR-USER-02**: Default sort order is by `id` in descending order, displaying newly registered users first.
- **BR-USER-03**: The maximum page size is 100 records; if `limit` exceeds 100, the server caps it at 100.
- **BR-USER-04**: Search on `email` and `displayName` is case-insensitive with partial contains matching.
- **BR-USER-05**: Role filtering supports built-in roles, `CUSTOM` (matching any custom role), and specific custom role names.

Normal Flow

1. The administrator opens the `/admin/users` page.
2. The frontend sends `GET /api/users` with default parameters (`page=1&limit=10&sortBy=id&sortOrde`
3. The backend validates administrator authentication and permissions.
4. The repository queries active users and counts total matching records.
5. The backend returns HTTP 200 with the users array and pagination metadata.
6. The frontend renders the user list table and pagination controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client clears authentication cookies and redirects to `/login`.
User lacks Admin privileges	The backend throws `UserForbiddenException` (HTTP 403 Forbidden). The UI displays: "You do not have permission to access this page."

Abnorma Case	Handling & System Response
Invalid pagination parameter values	The DTO validator rejects non-integer or negative values with HTTP 400 Bad Request. The client resets to page 1.
Unsupported sort column parameter	The system rejects invalid sort keys with HTTP 400 Bad Request. The client defaults to `id desc`.
Database query error or timeout	The service catches the Prisma error and throws `FailedToGetUsersException` (HTTP 500). The UI displays: "Couldn't load users. Try again."
No users match search or role filters	The query returns `users: []` and `total: 0`. The table displays: "No users match your filters."

Output Data

Output Field	Description
users	Array of user objects: `id`, `email`, `isBanned`, `createdAt`, `updatedAt`, `displayName`, `avatarUrl`, and `roles`.
pagination.page	Current page number.
pagination.limit	Current limit per page.
pagination.total	Total active users matching the query.
pagination.totalPages	Total calculated pages available.

Post-Conditions

1. The user list table is rendered with role badges and account statuses.

2. The administrator can filter by role, search, sort, or select a user to view full details (UC-28.02) or edit (UC-28.04).

3.14.2 View User Detail

Function Trigger

An administrator clicks "View detail" on a user row in the user table or navigates directly to `/admin/users/:id`.
Navigation path: User Management (`/admin/users`) → Click user row or view icon (`/admin/users/:id`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Retrieves consolidated, comprehensive account information for a specific user. Along with general profile data, social links, and activity metrics, the interface provides dedicated tab panels for each role profile the user holds: Client Profile, Freelancer Profile, Expert Profile, and Custom Role Profiles.

Interface

- Detailed user overview page at `/admin/users/:id`.
- Page header component (`UserDetailHeader`) showing avatar, display name, email, ban badge, and edit button.
- General information card (`UserGeneralInfo`) displaying account ID, verification status, join date, last update, bio, and activity statistics.
- Role tabs component (`UserRoleTabs`) rendering separate tabs for Client Profile, Freelancer Profile, Expert Profile, and Admin & Custom Roles.

Data Processing Overview

- Client sends `GET /api/users/:id`.
- `ParseIntPipe` validates that `id` is a positive integer.
- Backend verifies Admin role permissions.

- `UsersRepository.getUserById(id)` queries `prisma.user.findFirst` where `id = id` and `deletedAt = null`.
- The query includes profile data, assigned roles, social links, activity counts (jobs posted, contracts, proposals, reviews), client profile, freelancer profile (with skills and active portfolio items), expert profile, and custom role profiles.
- If user is not found or soft-deleted, throws `UserNotFoundException` (HTTP 404).
- The server responds with HTTP 200 and the complete consolidated detail object.
- The frontend renders the header, general info, and populates role-specific tabs.

Screen Layout (Mock-up Description)

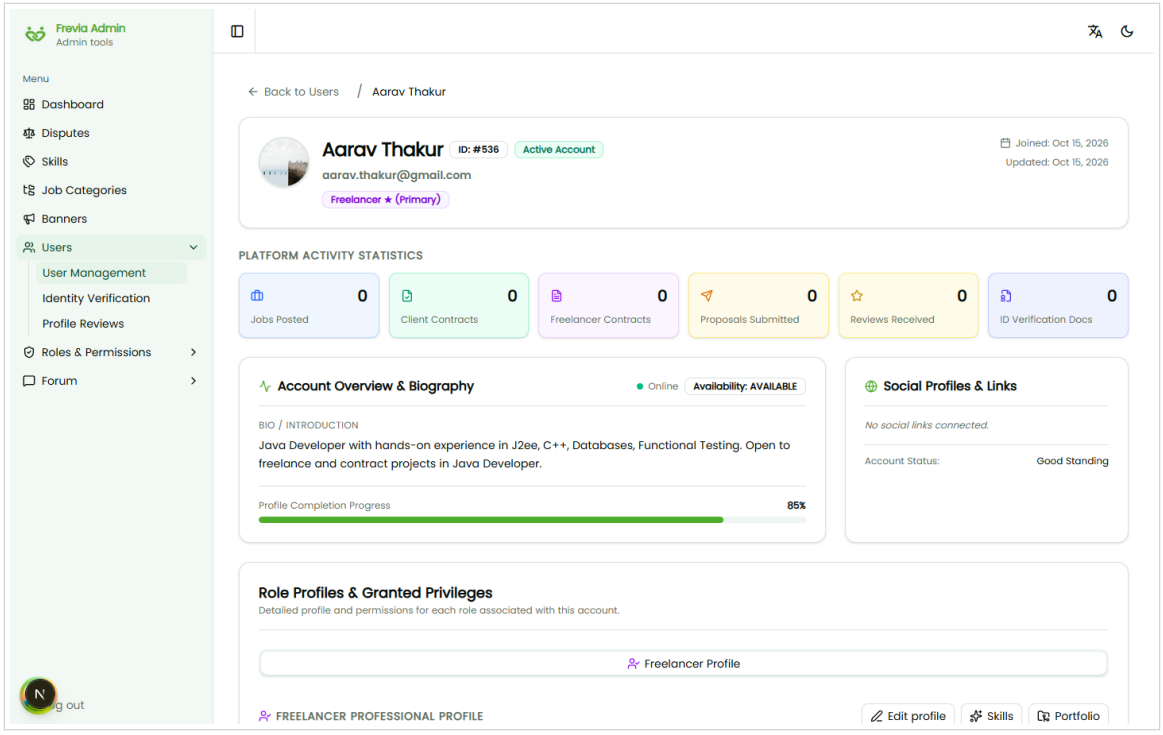

Figure 132: User detail interface

- Top navigation button “Back to users” allowing return to the catalog.
- Header containing user avatar, display name, email, ID badge, ban status badge, and “Edit User” button.
- Card section for **General Information**: Bio, Online status, Availability status, Profile completion percentage, and Activity stats (Jobs, Contracts, Proposals, Reviews).
- Tab navigation bar with tabs: **Client Profile**, **Freelancer Profile**, **Expert Profile**, and **Admin & Custom Roles**.
- Each active tab renders dedicated profile data and management action buttons.

Function Details

Input Data

Data Name	Description
id	Unique numeric system identifier of the user in the URL path.

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.

Business Rules

- **BR-USER-06**: The `clientProfile`, `freelancerProfile`, and `expertProfile` fields may be `null` if the user does not possess the corresponding role or has not initialized that profile.
- **BR-USER-07**: The `customRoleProfiles` array contains only roles outside the built-in Client, Freelancer, and Admin roles.
- **BR-USER-08**: Each assigned role includes an `isPrimary` flag indicating whether it serves as the user's default active role.

Normal Flow

1. The administrator navigates to `/admin/users/:id`.
2. The frontend sends `GET /api/users/:id`.
3. The backend validates Admin authorization and queries the database for the user record.
4. The repository compiles general information, role profiles, and activity statistics.
5. The server responds with HTTP 200 and the consolidated user detail payload.
6. The frontend renders `UserDetailHeader`, `UserGeneralInfo`, and `UserRoleTabs`.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Invalid user ID format (non-numeric string)	`ParseIntPipe` rejects the call with HTTP 400 Bad Request. The client displays an invalid parameter error page.
User ID does not exist in database	The repository throws `UserNotFoundException` (HTTP 404). The frontend displays: “User not found.”
User has been soft-deleted (`deletedAt != null`)	The query filters out soft-deleted users and throws HTTP 404 Not Found. The UI displays the not found notice.
User lacks Admin privileges	The backend throws `UserForbiddenException` (HTTP 403 Forbidden). Access is denied.
Caller token expired or revoked	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
Database query error during retrieval	The service throws `FailedToGetUserException` (HTTP 500). The UI displays a retry prompt.

Output Data

Output Field	Description
id	Unique numeric user ID.
email	User email address.
isBanned	Boolean indicating account suspension status.
profile	Nested general profile: `displayName`, `avatarUrl`, `bio`, `profileCompletionPercent`.
roles	Array of assigned roles with `id`, `name`, `description`, and `isPrimary`.
clientProfile	Client company details or null.
freelancerProfile	Freelancer skills, credentials, and portfolio items or null.
expertProfile	Expert qualifications, experience, and certifications or null.
stats	Activity counts: jobs posted, active contracts, proposals, and reviews.

Post-Conditions

1. The complete account profile and role tabs are displayed to the administrator.
2. The administrator can switch between role tabs and edit role-specific details directly.

3.14.3 Create User

Function Trigger

An administrator clicks the "Create User" button on the user list page.

Navigation path: User Management (`/admin/users`) → Click "Create User" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), bcrypt hashing library, Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to manually provision a new user account by specifying full name, email address, secure password, and selecting an initial role to activate (Client, Freelancer, or a custom role). The system automatically provisions the user, general profile, and primary role association.

Interface

- "Create User" button in the user list header.
- Modal dialog component (`CreateUserDialog`) with input fields: Full Name, Email, Password, Confirm Password, and Initial Role Select.
- Action buttons: "Cancel" and "Create User".
- Success toast notification upon account provisioning.

Data Processing Overview

- Administrator fills the form and submits the modal dialog.
- The client sends `POST /api/users` with body `{ email, fullName, password, confirmPassword, roleId }`.
- The backend validates schema constraints via `AdminCreateUserBodyDto`.
- `UsersService.createUser()` validates:
  - Email uniqueness among active accounts; if taken, throws `EmailAlreadyExistsException` (HTTP 409).
  - Role validity: target role must exist and be active; if missing, throws `CreateUserRoleNotFoundExcep` (HTTP 404).
  - Role restriction: role cannot be the Admin role; if selected, throws `CannotAssignAdminRoleExcepti` (HTTP 403).
- In an atomic database transaction:
  1. Hashes password with bcrypt (10 salt rounds).
  2. Creates User record with lowercased email, password hash, and `isEmailVerified = true`.
  3. Creates Profile record with `displayName = fullName`.
  4. Creates UserRole record assigning the role with `isPrimary = true`.
- The server responds with HTTP 201 Created and the new user summary.
- The frontend closes the dialog, displays a success toast, and refreshes the user list.

Screen Layout (Mock-up Description)

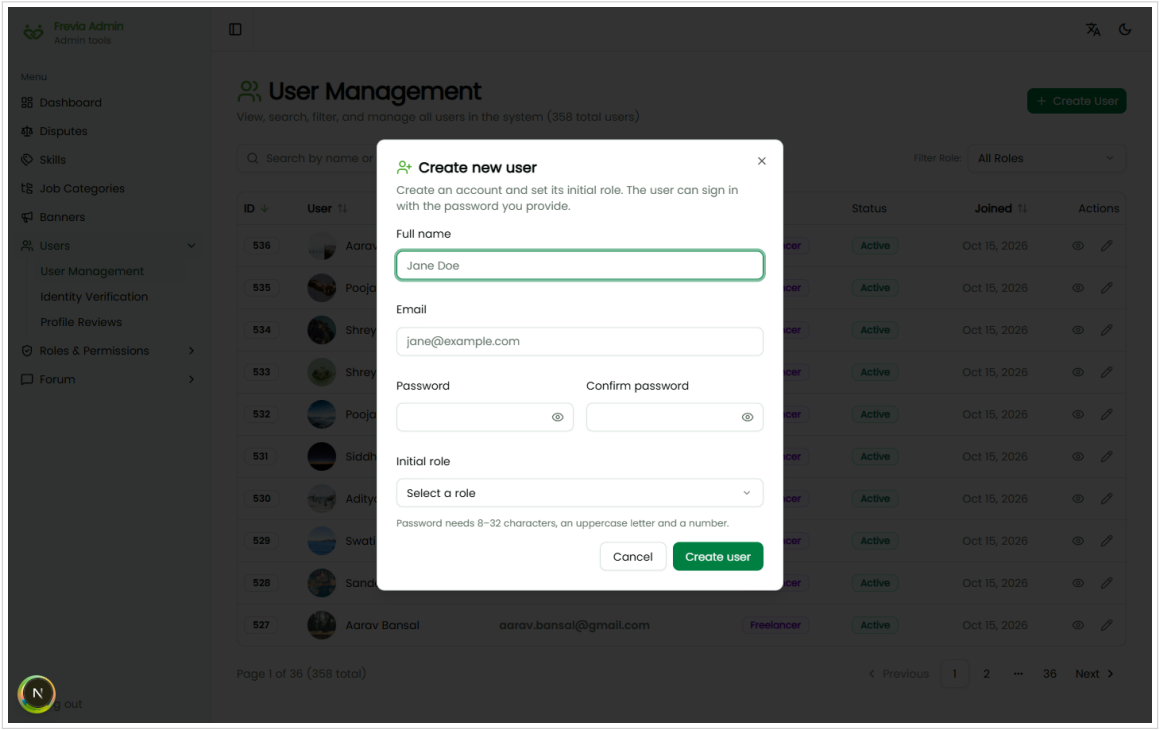

Figure 133: Create user modal

- Modal dialog titled “Create user” with subtitle “Add a new user account to the platform.”.
- Field: **Full name** (required, placeholder: “e.g., Alex Johnson”).
- Field: **Email address** (required, placeholder: “alex@example.com”).
- Field: **Password** (required, 8–32 characters, uppercase and number required).
- Field: **Confirm password** (required, matching password).
- Field: **Initial role** (Select dropdown displaying Client, Freelancer, and active Custom roles; Admin role is excluded).
- Dialog footer with “Cancel” and “Create user” action buttons.

Function Details

Input Data

Data Name	Description
fullName	Full display name of the user (required).
email	Email address for authentication (required).
password	Password for local credential login (required).
confirmPassword	Password confirmation string (required).
roleId	Numeric ID of the initial role to assign (required).

Validation Rules

Data Name	Validation Rule
fullName	String, required, 1 to 100 characters; whitespace trimmed.
email	String, required, valid RFC 5322 email format, max 254 characters; lowercased.
password	String, required, 8 to 32 characters; must contain at least one uppercase letter and one number.
confirmPassword	String, required; must match `password` exactly.
roleId	Positive integer; must correspond to an active non-Admin role.

Business Rules

- **BR-USER-09**: The email address is automatically converted to lowercase before storage and must be at most 254 characters.
- **BR-USER-10**: Email uniqueness is enforced among all active accounts (`deletedAt = null`).
- **BR-USER-11**: Passwords must be hashed with bcrypt prior to storage; plaintext passwords are never persisted.
- **BR-USER-12**: The Admin role cannot be assigned during manual user creation to prevent privilege escalation; only Client, Freelancer, or custom roles are permitted.
- **BR-USER-13**: The selected role must exist and be active in the system.
- **BR-USER-14**: The newly assigned role is automatically designated as the user's primary role (`isPrimary = true`).

Normal Flow

1. The administrator clicks "Create User" on the user management screen.

2. The dialog opens. The administrator inputs name, email, password, and selects an initial role.
3. The administrator clicks “Create user”.
4. The frontend validates the fields and issues `POST /api/users`.
5. The backend validates email uniqueness, role validity, and absence of Admin role assignment.
6. The backend hashes the password and provisions the User, Profile, and UserRole records in a transaction.
7. The server responds with HTTP 201 Created and the new user summary.
8. The frontend closes the dialog, displays a success toast, and refreshes the user list.

Abnormal Flow / Exception Handling

Abnorma Case	**Handling & System Response**
Password and Confirm Password do not match	Client-side validation and backend DTO reject submission. The form displays: “Passwords do not match.”
Password fails complexity rules (< 8 chars, no uppercase/number)	Backend validation rejects with HTTP 400 Bad Request. The UI displays password requirements.

Abnorma Case	Handling & System Response
Email is already registered to an active user	The backend throws `EmailAlreadyExistsException` (HTTP 409 Conflict). The UI displays: “Email already exists.”
Selected role does not exist or was deleted	The backend throws `CreateUserRoleNotFoundException` (HTTP 404). The UI displays: “Role not found.”
Administrator attempts to assign Admin role	The backend throws `CannotAssignAdminRoleException` (HTTP 403 Forbidden). The UI displays: “Cannot assign Admin role.”
User lacks Admin privileges	The backend throws `UserForbiddenException` (HTTP 403). Access is denied.
Database transaction failure	The transaction rolls back and the backend throws `FailedToCreateUserException` (HTTP 500). The UI displays: “Failed to create user.”
Administrator cancels dialog	The modal closes and discarded form data is cleared without network activity.

Output Data

Output Field	Description
id	Numeric ID of the created user.
email	Stored email address.
displayName	Display name set on the profile.
roles	Array containing the assigned primary role.

Post-Conditions

1. A new active user record is inserted with verified email status.
2. A linked profile record and primary role association are created.
3. The user appears in the user list and can authenticate immediately.

3.14.4 Update User

Function Trigger

An administrator clicks "Edit" on a user row in the table or clicks "Edit User" on the user detail screen.
Navigation path: User Management (`/admin/users`) → Click "Edit" icon on a user row.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to edit the general account credentials and administrative status of an existing user, including changing their email address, updating their display name, or toggling their account ban/suspension status.

Interface

- Edit action trigger on table rows and user detail header.
- Modal dialog component (`EditUserDialog`) pre-filled with current email, full name, and ban switch.
- Action buttons: "Cancel" and "Save Changes".
- Toast notification confirming successful account updates.

Data Processing Overview

- Administrator modifies fields in the dialog and submits the form.
- The client sends `PATCH /api/users/:id` with body containing modified attributes (`email`, `fullName`, `isBanned`).
- `UserActive('userId')` extracts the administrator's ID (`actorId`).
- `UsersService.updateUser()` validates:
  - Target user exists and has `deletedAt = null`.
  - Self-ban protection: if `isBanned = true` and `id === actorId`, throws `CannotBanSelfException` (HTTP 403).
  - Email uniqueness: if email is modified, ensures no other active user uses it; otherwise throws `EmailAlreadyExistsException` (HTTP 409).
- In a transaction, updates `user.email`, `user.isBanned`, and upserts `profile.displayName`.
- The server responds with HTTP 200 and the updated user summary.
- The frontend closes the dialog, displays a success toast, and refreshes the user view.

Screen Layout (Mock-up Description)

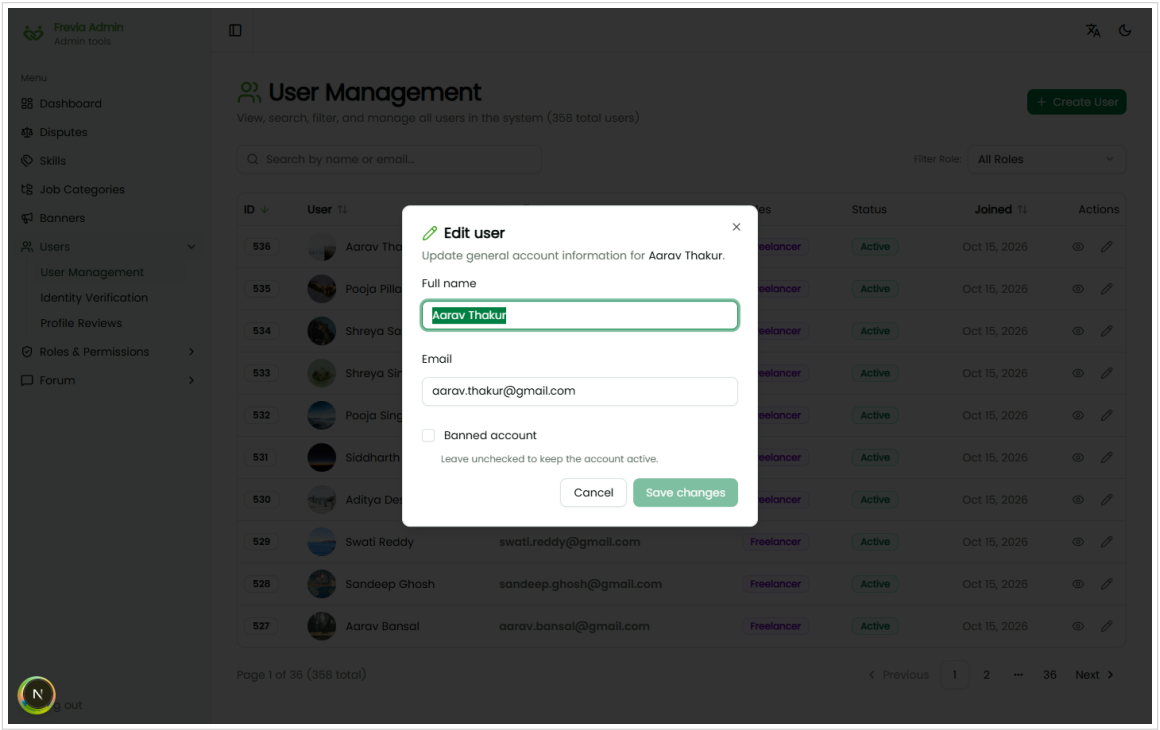

Figure 134: Update user modal

- Modal dialog titled "Edit user" with description "Update general account information.".
- Field: **Email address** (pre-filled with current email).
- Field: **Full name** (pre-filled with current display name).

- Toggle switch: **Banned** (Switch component allowing immediate suspension of account access).
- Dialog footer with “Cancel” and “Save changes” buttons.

Function Details

Input Data

Data Name	Description
id	Numeric identifier of the user in the URL path.
email	Updated email address (optional, string).
fullName	Updated display name (optional, string or null).
isBanned	Boolean account suspension toggle (optional).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
email	String, optional, valid email format, max 254 characters; lowercased.
fullName	String, optional, max 100 characters, nullable.
isBanned	Boolean, optional.

Business Rules

- **BR-USER-15**: An administrator is strictly prohibited from banning their own account (`isBanned = true` is rejected when `targetUserId == actorId`).
- **BR-USER-16**: Email uniqueness is enforced across all non-deleted users, excluding the target user being edited.
- **BR-USER-17**: At least one updatable field must be provided in the request payload.
- **BR-USER-18**: The email address is automatically converted to lowercase before updating.
- **BR-USER-19**: Setting `fullName` to null or empty string clears the user’s display name.

Normal Flow

1. The administrator opens the “Edit User” dialog for a selected user.
2. The administrator modifies email, display name, or toggles the ban switch.

3. The administrator clicks “Save changes”.
4. The client sends `PATCH /api/users/:id`.
5. The backend validates authorization, verifies self-ban constraints, and checks email uniqueness.
6. The repository updates user and profile records in a database transaction.
7. The API returns HTTP 200 with the updated user data.
8. The frontend displays a success toast and refreshes the user information.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Administrator attempts to ban their own account	The backend detects `id === actorId` and throws `CannotBanSelfException` (HTTP 403 Forbidden). The UI displays: “You cannot ban your own account.”
New email is already in use by another account	The backend detects conflict and throws `EmailAlreadyExistsException` (HTTP 409 Conflict). The UI displays: “Email already exists.”
No fields provided in update payload	The backend rejects the request with HTTP 400 Bad Request (`Nothing to update`). The UI highlights required fields.
User ID does not exist or is soft-deleted	The service throws `UserNotFoundException` (HTTP 404). The UI displays: “User not found.”

Abnorma Case	Handling & System Response
User lacks Admin privi-leges	The backend throws `UserForbiddenException` (HTTP 403). Access is denied.
Database transac-tion failure	The service catches the Prisma error and throws `FailedToUpdateUserException` (HTTP 500). The UI displays: "Failed to update user."
Administrator cancels dialog	The modal closes and unsaved modifications are discarded without network calls.

Output Data

Output Field	Description
id	Numeric user identifier.
email	Updated email address.
displayName	Updated display name string.
isBanned	Updated ban status boolean.
roles	Array of assigned roles.

Post-Conditions

1. The user's account details and ban status are updated in PostgreSQL.
2. If banned, the user's active sessions are immediately rejected on next access attempt.
3. The changes are visible across administrative tables and user detail views.

3.14.5 Update Client Profile

Function Trigger

An administrator clicks “Edit Client Profile” on the Client Profile tab of the user detail page.
Navigation path: User Detail (`/admin/users/:id`) → Tab “Client Profile” → Click “Edit Client Profile” button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to edit the client organization profile of a user holding the Client role. The administrator can update company name, company description, and corporate website URL.

Interface

- “Edit Client Profile” button on the Client Profile tab panel.
- Modal dialog pre-filled with current company name, description, and website URL.
- Action buttons: “Cancel” and “Save Changes”.
- Toast notification confirming successful update.

Data Processing Overview

- Administrator submits the updated client profile form.
- The client sends `PATCH /api/users/:id/client-profile` with `{ companyName, companyDescription, website }`.
- `UsersService.updateClientProfile()` verifies that the target user exists, has not been soft-deleted, and possesses the `CLIENT` role (or an existing client profile record); otherwise throws `NoClientRoleForClientProfileException` (HTTP 403).
- The service trims string inputs and converts empty strings to `null`.
- `UsersRepository.upsertClientProfileByAdmin()` upserts the record in `prisma.clientProfile.u`
- The API returns HTTP 200 with the updated client profile object.
- The frontend displays a success toast and refreshes the Client Profile tab content.

Screen Layout (Mock-up Description)

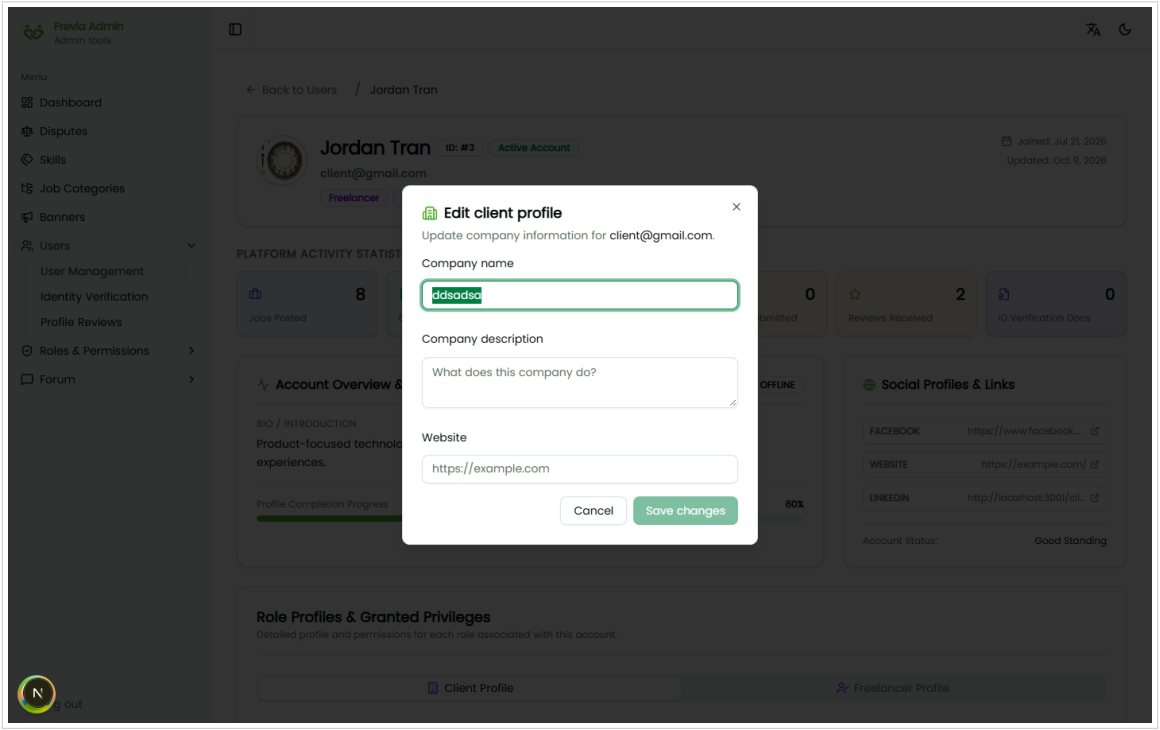

Figure 135: Update client profile modal

- Modal dialog titled “Edit client profile” with subtitle “Update client organization and company details.”.
- Field: **Company name** (input, max 255 characters, optional).
- Field: **Company description** (multi-line textarea, max 5000 characters, optional).
- Field: **Website URL** (input with URL format validation, optional).
- Dialog footer with “Cancel” and “Save changes” buttons.

Function Details

Input Data

Data Name	Description
id	Numeric user identifier in the URL path.
companyName	Updated company name string (optional, nullable).
companyDescription	Detailed description of company activities (optional, nullable).
website	Corporate website URL (optional, nullable).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
companyName	String, optional, max 255 characters, nullable.
companyDescription	String, optional, max 5000 characters, nullable.
website	String, optional, valid URL format (e.g., `https://example.com`), nullable.

Business Rules

- **BR-CLT-01**: The target user must possess the Client role or have an existing client profile record; otherwise, the update is rejected with HTTP 403 Forbidden.
- **BR-CLT-02**: Company name is limited to a maximum of 255 characters.
- **BR-CLT-03**: Company description is limited to a maximum of 5000 characters.
- **BR-CLT-04**: The website field must conform to standard URL format if supplied.
- **BR-CLT-05**: Empty string values are automatically converted to `null` to allow clearing field content.
- **BR-CLT-06**: The client profile record is upserted, creating the profile record if it does not yet exist.

Normal Flow

1. The administrator navigates to the Client Profile tab and clicks “Edit Client Profile”.
2. The dialog opens pre-filled with current company data.
3. The administrator modifies company name, description, or website and clicks “Save changes”.
4. The frontend sends `PATCH /api/users/:id/client-profile`.
5. The backend verifies permissions and confirms the user has the Client role.

6. The repository upserts the client profile record in PostgreSQL.
7. The backend returns HTTP 200 with the updated client profile data.
8. The frontend displays a success toast and refreshes the tab.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target user does not have Client role	The backend throws `NoClientRoleForClientProfileException` (HTTP 403 Forbidden). The UI displays: "User does not have a Client role."
Website format is invalid	Client-side validation and backend DTO reject with HTTP 400 Bad Request. The UI displays: "Invalid URL format."
Company name exceeds 255 characters	Backend validation returns HTTP 400 Bad Request. The UI notifies of character limit violation.
Company description exceeds 5000 characters	Backend validation returns HTTP 400 Bad Request. The UI notifies of character limit violation.
User ID does not exist or is soft-deleted	The service throws `UserNotFoundException` (HTTP 404). The UI displays: "User not found."

Abnorma Case	Handling & System Response
Database upsert failure	The service catches the Prisma error and throws `FailedToUpdateClientProfileException` (HTTP 500). The UI displays: "Failed to update client profile."

Output Data

Output Field	Description
id	Numeric client profile identifier.
userId	Numeric ID of the associated user.
companyName	Updated company name or null.
companyDescription	Updated company description or null.
website	Updated website URL or null.
updatedAt	Timestamp indicating last update.

Post-Conditions

1. The client profile record is saved in the database.
2. The updated company information is reflected in the Client Profile tab and public job postings by this client.

3.14.6 Update Freelancer Basic Info

Function Trigger

An administrator clicks "Edit Freelancer Profile" on the Freelancer Profile tab of the user detail page.

Navigation path: User Detail (`/admin/users/:id`) → Tab "Freelancer Profile" → Click "Edit Freelancer Profile" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to edit the basic introductory and credential information of a user holding the Freelancer role, including professional title, overview biography, spoken languages, educational history, and professional certifications.

Interface

- "Edit Freelancer Profile" button on the Freelancer Profile tab.
- Modal dialog with text inputs for Professional Title, Biography textarea, and dynamic tag lists for Languages, Education, and Certifications.
- Action buttons: "Cancel" and "Save Changes".
- Toast notification confirming successful update.

Data Processing Overview

- Administrator submits changes in the modal dialog.
- The client sends `PATCH /api/users/:id/freelancer-profile` with `{ title, bio, languages, education, certifications }`.
- `UsersService.updateFreelancerProfile()` verifies that target user exists, is active, and possesses the `FREELANCER` role (or an existing freelancer profile); otherwise throws `NoFreelancerRoleForProfileException` (HTTP 403).
- The service cleans string lists, converting empty strings to `null` and empty/null arrays to `[]`.
- `UsersRepository.updateFreelancerProfileByAdmin()` upserts the freelancer profile record in PostgreSQL.
- The API returns HTTP 200 with `{ message: 'Freelancer profile updated successfully' }`.
- The frontend displays a success toast and refreshes the Freelancer Profile tab.

Screen Layout (Mock-up Description)

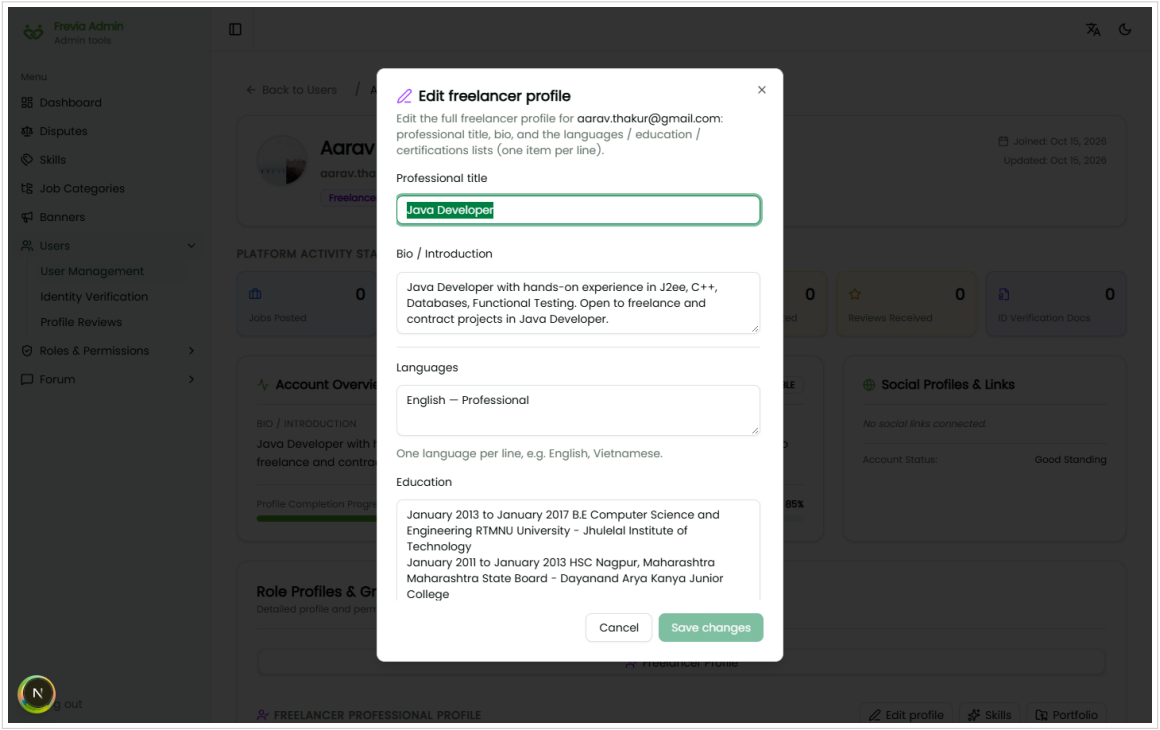

Figure 136: Update freelancer basic info modal

- Modal dialog titled “Edit freelancer profile” with subtitle “Update professional title, biography, and credentials.”.
- Field: **Professional title** (input, max 255 characters, placeholder: “e.g., Senior Full-Stack Engineer”).
- Field: **Biography** (textarea, max 5000 characters).
- Dynamic string lists: **Languages**, **Education**, and **Certifications** with input field and “Add” button, rendering removable badges for each entry.
- Action buttons: “Cancel” and “Save changes”.

Function Details

Input Data

Data Name	Description
id	Numeric user identifier in the URL path.
title	Professional headline or title (optional, nullable).
bio	Professional summary biography (optional, nullable).
languages	Array of spoken languages (optional, string array).
education	Array of education entries (optional, string array).
certifications	Array of certification entries (optional, string array).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
title	String, optional, max 255 characters, nullable.
bio	String, optional, max 5000 characters, nullable.
languages	Array of strings, max 100 items, each item max 255 characters.
education	Array of strings, max 100 items, each item max 255 characters.
certifications	Array of strings, max 100 items, each item max 255 characters.

Business Rules

- **BR-FLR-01**: The target user must possess the Freelancer role or have an existing freelancer profile record; otherwise, the update is rejected with HTTP 403 Forbidden.
- **BR-FLR-02**: Professional title is limited to a maximum of 255 characters.
- **BR-FLR-03**: Biography is limited to a maximum of 5000 characters.
- **BR-FLR-04**: Credential arrays (languages, education, certifications) may each contain at most 100 items, with each entry limited to 255 characters.
- **BR-FLR-05**: Empty string values are converted to `null`; empty or null arrays are normalized to `[]`.
- **BR-FLR-06**: The freelancer profile and parent Profile are upserted if not already present.

Normal Flow

1. The administrator navigates to the Freelancer Profile tab and clicks “Edit Freelancer Pro-

file".

2. The modal opens pre-filled with current title, bio, and credential lists.
3. The administrator edits the fields, adds/removes credentials, and clicks "Save changes".
4. The client sends `PATCH /api/users/:id/freelancer-profile`.
5. The backend validates authorization and confirms Freelancer role eligibility.
6. The repository upserts the freelancer profile record in PostgreSQL.
7. The API returns HTTP 200 with a success message.
8. The frontend displays a success toast and refreshes the tab.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target user does not have Freelancer role	The backend throws `NoFreelancerRoleForProfileException` (HTTP 403 Forbidden). The UI displays: "User does not have a Freelancer role."
Credential entry exceeds 255 characters	Backend validation returns HTTP 400 Bad Request. The UI notifies of item length violation.
Credential array exceeds 100 items	Backend validation returns HTTP 400 Bad Request. The UI notifies that at most 100 entries are allowed.
Title exceeds 255 characters	Backend validation returns HTTP 400 Bad Request. The UI displays a field length error.

Abnorma Case	Handling & System Response
User ID does not exist or is soft-deleted	The service throws `UserNotFoundException` (HTTP 404). The UI displays: "User not found."
Database update failure	The service catches the Prisma error and throws `FailedToUpdateFreelancerProfileException` (HTTP 500). The UI displays: "Failed to update freelancer profile."

Output Data

Output Field	Description
message	Success confirmation message string (`Freelancer profile updated successfully`).

Post-Conditions

1. The freelancer profile title, bio, and credentials are saved in PostgreSQL.
2. The updated information appears in the Freelancer Profile tab and public freelancer directory.

3.14.7 Update Freelancer Skills

Function Trigger

An administrator clicks "Manage Freelancer Skills" on the Freelancer Profile tab of the user detail page.

Navigation path: User Detail (`/admin/users/:id`) → Tab "Freelancer Profile" → Click "Manage Freelancer Skills" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to manage and completely replace the skill set of a freelancer. Skills are selected from the system skill catalog with an assigned proficiency level between 1 and 10. The update operates as an atomic full-replacement operation.

Interface

- "Manage Freelancer Skills" button on the Freelancer Profile tab.
- Skill management modal dialog listing current skills with Select catalog dropdowns, numeric proficiency level sliders/inputs (1–10), and delete row buttons.
- "Add skill" button to append additional skill rows.
- Action buttons: "Cancel" and "Save Skills".

Data Processing Overview

- The client fetches active catalog skills via `GET /api/users/skills-catalog` to populate the selection dropdown.
- Administrator configures skill names and proficiency levels, then submits.
- The client sends `PUT /api/users/:id/freelancer-profile/skills` with body `{ skills: [ { skillName, proficiencyLevel } ] }`.
- `UsersService.replaceFreelancerSkills()` verifies user existence and Freelancer role.
- In an atomic database transaction:
  1. Deletes all existing `FreelancerSkill` records for this freelancer profile.
  2. For each skill item, searches the skill catalog case-insensitively; if absent, creates a new catalog skill with an auto-generated unique slug.
  3. Inserts a new `FreelancerSkill` record linking the freelancer profile to the catalog skill with the designated proficiency level.
- The server responds with HTTP 200: `{ message: 'Freelancer skills updated successfully' }`.
- The frontend displays a success toast and refreshes the skills list on the tab.

Screen Layout (Mock-up Description)

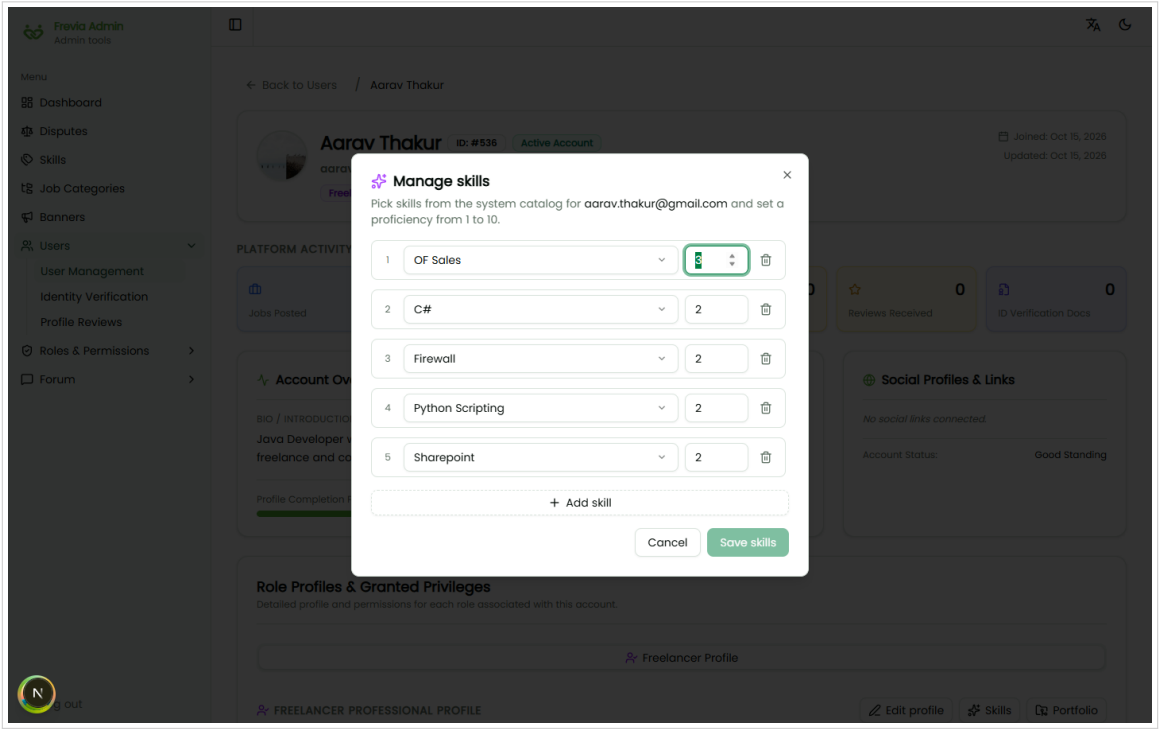

Figure 137: Update freelancer skills modal

- Modal dialog titled “Manage freelancer skills” with description “Select skills from the catalog and assign proficiency levels (1–10).”.
- Dynamic list of skill rows, each containing:
  - **Skill name Select:** Dropdown displaying available catalog skills.
  - **Proficiency level:** Number input or slider from 1 (Beginner) to 10 (Master).
  - **Remove button:** Trash icon to delete the skill row.
- “Add skill” outline button at the bottom of the list.
- Dialog footer with “Cancel” and “Save skills” buttons.

Function Details

Input Data

Data Name	Description
id	Numeric user identifier in the URL path.
skills	Array of skill objects: `skillName` (string) and `proficiencyLevel` (integer 1–10).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
skills	Array, max 100 items.
skills[].skillName	String, required, 1 to 100 characters; whitespace trimmed.
skills[].proficiencyLevel	Integer, required, between 1 and 10 inclusive.

Business Rules

- **BR-FLR-07**: Skill updating uses full-replacement semantics (`PUT`): all pre-existing skills are removed and replaced by the submitted array.
- **BR-FLR-08**: Proficiency levels must strictly be integers between 1 and 10 inclusive.
- **BR-FLR-09**: A freelancer may possess a maximum of 100 skills.
- **BR-FLR-10**: Skill catalog lookup is case-insensitive; if a submitted skill does not exist in the master catalog, a new catalog entry is automatically provisioned.
- **BR-FLR-11**: Submitting an empty skills array effectively clears all skills for the freelancer.

Normal Flow

1. The administrator navigates to the Freelancer Profile tab and clicks “Manage Freelancer Skills”.
2. The frontend loads the skill catalog and opens the dialog with existing skills pre-populated.
3. The administrator adds, removes, or adjusts skill names and proficiency levels.
4. The administrator clicks “Save skills”.
5. The client sends `PUT /api/users/:id/freelancer-profile/skills`.
6. The backend validates inputs, clears old skills, and creates new skill links in a transaction.
7. The backend returns HTTP 200 with a success confirmation message.
8. The frontend closes the dialog, shows a success toast, and refreshes the skills list.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Proficiency level out of bounds (< 1 or > 10)	Backend DTO validation rejects with HTTP 400 Bad Request. The UI notifies of invalid proficiency range.
Skills array exceeds 100 items	Backend validation rejects with HTTP 400 Bad Request. The UI displays an item limit notice.
Skill name is blank or exceeds 100 characters	Backend validation returns HTTP 400 Bad Request. The UI highlights the offending row.
Target user does not have Freelancer role	The backend throws `NoFreelancerRoleForProfileException` (HTTP 403 Forbidden). Access is denied.
User ID does not exist or is soft-deleted	The service throws `UserNotFoundException` (HTTP 404). The UI displays: "User not found."
Database transaction failure	The transaction rolls back and the service throws `FailedToSaveSkillsException` (HTTP 500). The UI displays: "Failed to save skills."

Output Data

Output Field	Description
message	Success confirmation message string (`Freelancer skills updated successfully`).

Post-Conditions

1. Pre-existing freelancer skills are removed and replaced with the new skill associations.
2. Any newly introduced skill is registered in the master catalog.
3. The updated skill badges and proficiency levels appear on the user profile.

3.14.8 Update Freelancer Portfolio

Function Trigger

An administrator clicks "Manage Freelancer Portfolio" on the Freelancer Profile tab of the user detail page.
Navigation path: User Detail (`/admin/users/:id`) → Tab "Freelancer Profile" → Click "Manage Freelancer Portfolio" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to manage portfolio project items for a freelancer by creating new items, editing existing items, or soft-deleting items. Each portfolio item documents completed project work with title, description, technology stack tags, and project URL.

Interface

- "Manage Freelancer Portfolio" button on the Freelancer Profile tab.
- Portfolio management modal listing existing portfolio cards with Edit and Delete action buttons.
- "Add Portfolio Item" button opening item creation form.
- Edit item sub-dialog with inputs for Title, Description, Technologies, and Project URL.
- Delete confirmation dialog for soft-deleting items.

Data Processing Overview

- **Create Item:** Client sends `POST /api/users/:id/freelancer-profile/portfolio-items`. `UsersRepository.createPortfolioItem()` inserts into `prisma.portfolioItem.create`. Returns HTTP 201.
- **Update Item:** Client sends `PATCH /api/users/:id/freelancer-profile/portfolio-items/:i` `UsersRepository.updatePortfolioItemByAdmin()` verifies item ownership and `deletedAt = null`, then updates fields. Returns HTTP 200.
- **Delete Item:** Client sends `DELETE /api/users/:id/freelancer-profile/portfolio-items/:` `UsersRepository.softDeletePortfolioItem()` verifies item ownership and sets `deletedAt = new Date()`. Returns HTTP 200.
- The client displays success toasts and refreshes the portfolio list.

Screen Layout (Mock-up Description)

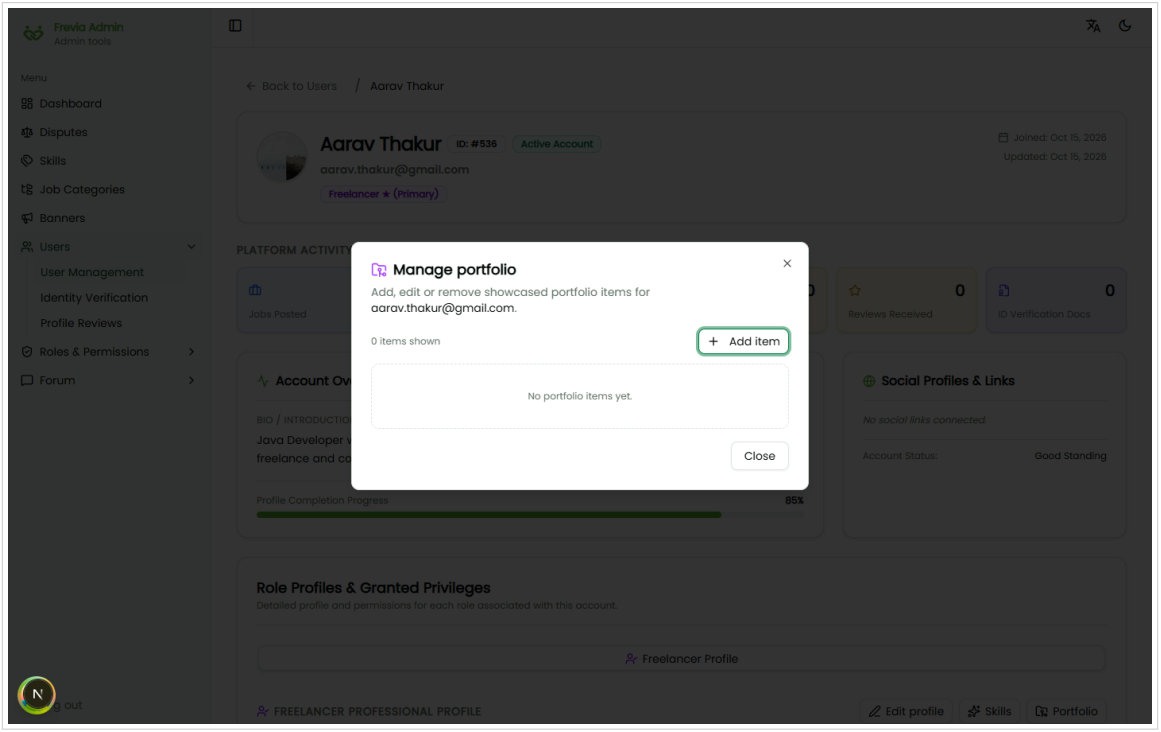

Figure 138: Update freelancer portfolio modal

- Modal dialog titled “Manage freelancer portfolio” with an “Add portfolio item” header button.
- List of existing portfolio item cards displaying:
  - **Project title** in bold with external link icon if URL is provided.
  - **Project description** summary.
  - **Technologies used:** Small badges for each technology tag.
  - **Action buttons:** Edit (pencil icon) and Delete (trash icon).

- Nested add/edit dialog with fields: Title, Description, Technologies input, and Project URL.

Function Details

Input Data

Data Name	Description
id	Numeric user identifier in the URL path.
itemId	Numeric portfolio item identifier in URL path (for edit/delete).
title	Project display title (required, string).
description	Detailed project description (optional, string or null).
technologies	Array of technology names used (optional, string array).
projectUrl	Live project or repository URL (optional, nullable).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
itemId	Must be a positive integer greater than 0.
title	String, required, 1 to 255 characters; whitespace trimmed.
description	String, optional, max 5000 characters, nullable.
technologies	Array of strings, max 50 items, each item max 100 characters.
projectUrl	String, optional, valid URL format, nullable.

Business Rules

- **BR-FLR-12**: Portfolio item deletion is implemented as a soft delete by setting `deletedAt = new Date();` items are never permanently purged.
- **BR-FLR-13**: Project title is mandatory and limited to a maximum of 255 characters.
- **BR-FLR-14**: Project description is limited to a maximum of 5000 characters.
- **BR-FLR-15**: Technologies list may contain at most 50 items, each limited to 100 characters.
- **BR-FLR-16**: The project URL must conform to standard URL format if provided.
- **BR-FLR-17**: Edit and delete operations strictly verify item ownership: the target item must belong to the specified freelancer profile.

- **BR-FLR-18**: Soft-deleted portfolio items are excluded from active profile queries.

Normal Flow

1. The administrator navigates to the Freelancer Profile tab and clicks “Manage Freelancer Portfolio”.
2. The dialog displays existing portfolio items with action buttons.
3. To add an item, the administrator clicks “Add portfolio item”, fills in title, description, technologies, and URL, and submits.
4. The client sends `POST /api/users/:id/freelancer-profile/portfolio-items`.
5. The backend creates the item record and returns HTTP 201.
6. The dialog refreshes and displays the newly created portfolio card.

Abnormal Flow / Exception Handling

Abnorma Case	**Handling & System Response**
Portfolio item ID does not exist or belongs to another user	The backend throws `PortfolioItemNotFoundException` (HTTP 404). The UI displays: “Portfolio item not found.”
Project title is empty	Client-side validation and backend DTO return HTTP 400 Bad Request. The UI highlights title as required.
Project URL format is invalid	Backend validation returns HTTP 400 Bad Request. The UI indicates invalid URL.
Technologies array exceeds 50 items	Backend validation returns HTTP 400 Bad Request. The UI notifies of maximum technology count.

Abnorma Case	Handling & System Response
Database error during portfolio item creation	The service throws `FailedToCreatePortfolioItemException` (HTTP 500). The UI displays: "Failed to create portfolio item."
Database error during portfolio item update	The service throws `FailedToUpdatePortfolioItemException` (HTTP 500). The UI displays: "Failed to update portfolio item."
Database error during portfolio item deletion	The service throws `FailedToDeletePortfolioItemException` (HTTP 500). The UI displays: "Failed to delete portfolio item."

Output Data

Output Field	Description
message	Success confirmation message string.

Post-Conditions

1. The portfolio item is created, modified, or soft-deleted in the database.
2. Active portfolio items are displayed on the freelancer's public showcase page.

3.14.9 Update Expert Profile

Function Trigger

An administrator clicks "Edit Expert Profile" on the Expert Profile tab of the user detail page.
Navigation path: User Detail (`/admin/users/:id`) → Tab "Expert Profile" → Click "Edit Expert Profile" button.

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows administrators to edit the verified Expert profile of a user holding the Expert role directly from the admin page. The administrator can update display name, professional title, years of experience, expertise areas, educational degrees, professional certifications, corporate website, active visibility toggle, and biography. Unlike expert self-edits, administrative changes take effect immediately without requiring moderation queue review.

Interface

- "Edit Expert Profile" button on the Expert Profile tab panel.
- Modal dialog component (`EditExpertProfileButton`) containing structured fields: Display Name, Title, Years of Experience, Website, Active Toggle switch, Expertise tag input, Education tag input, Certifications tag input, and Biography textarea.
- Action buttons: "Cancel" and "Save Changes".
- Toast notification confirming successful update.

Data Processing Overview

- Administrator modifies expert profile fields and submits the modal dialog.
- The client sends an HTTP request: `PATCH /api/users/:id/expert-profile` with `AdminUpdateExpertProfileDto`.
- `AdminExpertProfileController.update()` extracts `adminId` and calls `ExpertProfileService.upd` `adminId, input)`.
- The service verifies target user existence and confirms the user holds the `EXPERT` role; if not, throws `NotAnExpertAccountException` (HTTP 403).
- The service sanitizes inputs: trims strings, deduplicates arrays case-insensitively, and cal-

culates updated expert profile strength score.

- `ExpertProfileRepository.updateDirect()` updates the expert profile, user profile display name, and bio directly in PostgreSQL.
- The API returns HTTP 200 with `{ message: 'Expert profile updated successfully' }`.
- The frontend closes the dialog, displays a success toast, and refreshes the Expert Profile tab.

Screen Layout (Mock-up Description)

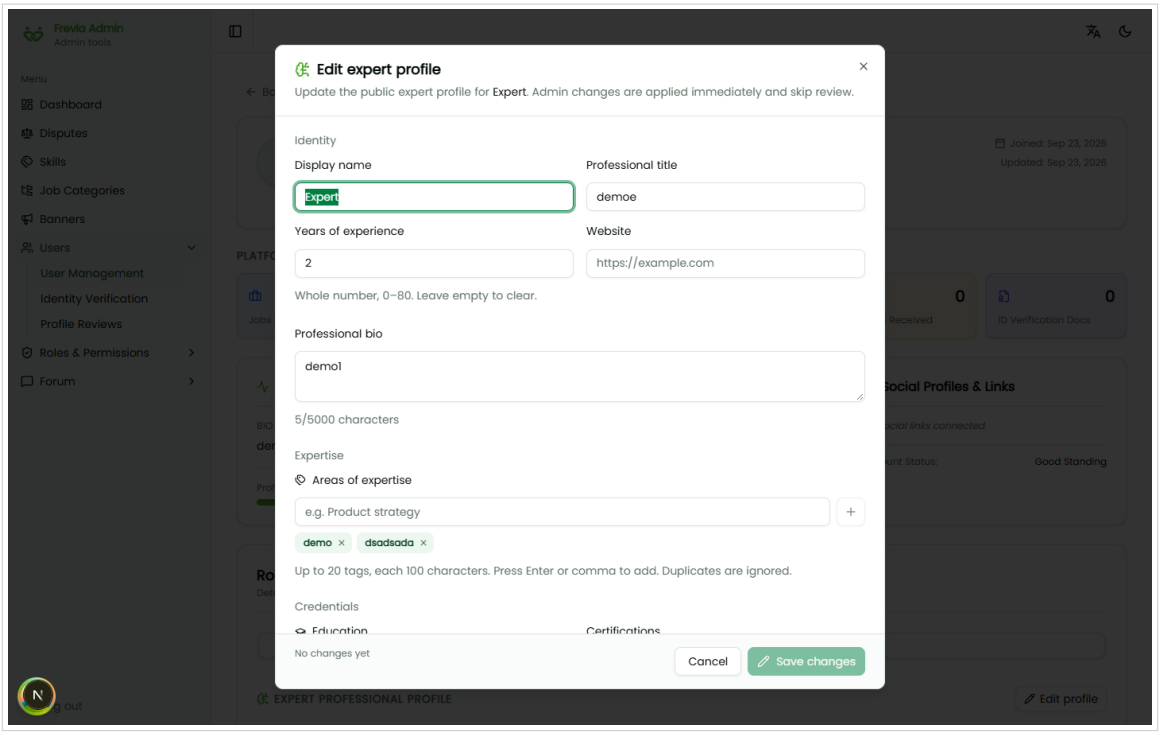

Figure 139: Update expert profile modal

- Modal dialog titled "Edit expert profile" with description "Direct administrative edit for expert qualifications and showcase status.".
- Field: **Display name** (required, max 255 characters).
- Field: **Professional title** (input, max 255 characters, optional).
- Field: **Years of experience** (number input, 0 to 80 years).
- Field: **Website URL** (URL format, optional).
- Toggle switch: **Public visibility** (Switch controlling whether the expert appears on the public `/experts` directory).
- Dynamic string lists: **Areas of expertise**, **Education**, and **Certifications** with "Add entry" buttons and badge tags.
- Field: **Biography** (multi-line textarea, max 5000 characters).

- Action buttons: “Cancel” and “Save changes”.

Function Details

Input Data

Data Name	Description
id	Numeric user identifier in the URL path.
displayName	Display name of the expert (required, string).
title	Professional title or headline (optional, string or null).
yearsOfExperience	Total years of industry experience (optional, integer 0–80 or null).
expertise	Array of primary expertise domains (optional, string array).
education	Array of academic degrees (optional, string array).
certifications	Array of professional certifications (optional, string array).
website	Personal or firm website URL (optional, string or null).
isActive	Boolean switch controlling public showcase visibility.
bio	Comprehensive biography summary (optional, string or null).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than 0.
displayName	String, required, 1 to 255 characters; whitespace trimmed.
title	String, optional, max 255 characters, nullable.
yearsOfExperience	Integer, optional, between 0 and 80 inclusive, nullable.
expertise	Array of strings, max 20 items, each item max 100 characters.
education	Array of strings, max 20 items, each item max 500 characters.
certifications	Array of strings, max 20 items, each item max 500 characters.

Data Name	Validation Rule
website	String, optional, valid URL format, max 255 characters, nullable.
isActive	Boolean flag (true/false).
bio	String, optional, max 5000 characters, nullable.

Business Rules

- **BR-EXP-01**: The target user must possess the Expert role; attempting to edit a non-expert user throws HTTP 403 Forbidden (`NOT_AN_EXPERT_ACCOUNT`).
- **BR-EXP-02**: Administrative updates apply directly to the database without requiring review approval or revision tickets.
- **BR-EXP-03**: Years of experience must be an integer between 0 and 80.
- **BR-EXP-04**: Credential arrays (expertise, education, certifications) are deduplicated case-insensitively and sanitized on submission.
- **BR-EXP-05**: The `isActive` toggle controls whether the expert is searchable and listed on the public platform directory (`/experts`).
- **BR-EXP-06**: The expert's profile strength score is recalculated automatically upon each update.

Normal Flow

1. The administrator navigates to the Expert Profile tab and clicks "Edit Expert Profile".
2. The modal dialog opens pre-populated with current expert credentials and visibility status.
3. The administrator modifies desired fields and clicks "Save changes".
4. The client sends `PATCH /api/users/:id/expert-profile`.
5. The backend validates authorization, verifies Expert role assignment, and sanitizes input data.
6. The repository commits updates directly to the expert profile and updates profile strength.
7. The backend returns HTTP 200 with `{ message: 'Expert profile updated successfully' }`.
8. The frontend closes the dialog, shows a success toast, and refreshes the Expert Profile tab.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target user does not possess Expert role	The backend throws `NotAnExpertAccountException` (HTTP 403 Forbidden). The UI displays: "User does not have an Expert role."
Display name is empty	Client-side validation and backend DTO return HTTP 400 Bad Request. The UI highlights display name as required.
Years of experience exceeds 80 or is negative	Backend validation returns HTTP 400 Bad Request. The UI notifies that years must be between 0 and 80.
Expertise array exceeds 20 items	Backend validation returns HTTP 400 Bad Request. The UI notifies of maximum expertise count.
Website format is invalid	Backend validation returns HTTP 400 Bad Request. The UI indicates invalid URL format.
User ID does not exist or profile is missing	The backend throws `ExpertProfileNotFoundException` (HTTP 404). The UI displays: "Expert profile not found."
User lacks Admin privileges	The backend throws HTTP 403 Forbidden. Access is denied.

Abnorma Case	Handling & System Response
Database update failure	The backend catches the Prisma exception and returns HTTP 500. The UI displays: “Failed to update expert profile.”

Output Data

Output Field	Description
message	Confirmation message string (`Expert profile updated successfully`).

Post-Conditions

1. The expert profile and linked user profile fields are updated directly in PostgreSQL.
2. The updated qualifications and public visibility status take effect immediately.
3. The Expert Profile tab reflects the changes.

3.15 Moderation

3.15.1 View Moderation Queue

Function Trigger

The logged-in administrator opens the Moderation screen by choosing **Forum** → **Moderation** in the admin sidebar.

Function Description

This function allows an administrator to see all forum posts that are waiting to be reviewed. Every row shows the identifier of the post, its title, the automatic moderation score, the categories that were detected, the author and the creation date. The list only contains posts that are still pending, so the administrator always works on the items that need a decision.

Screen Layout (Mock-up Description)

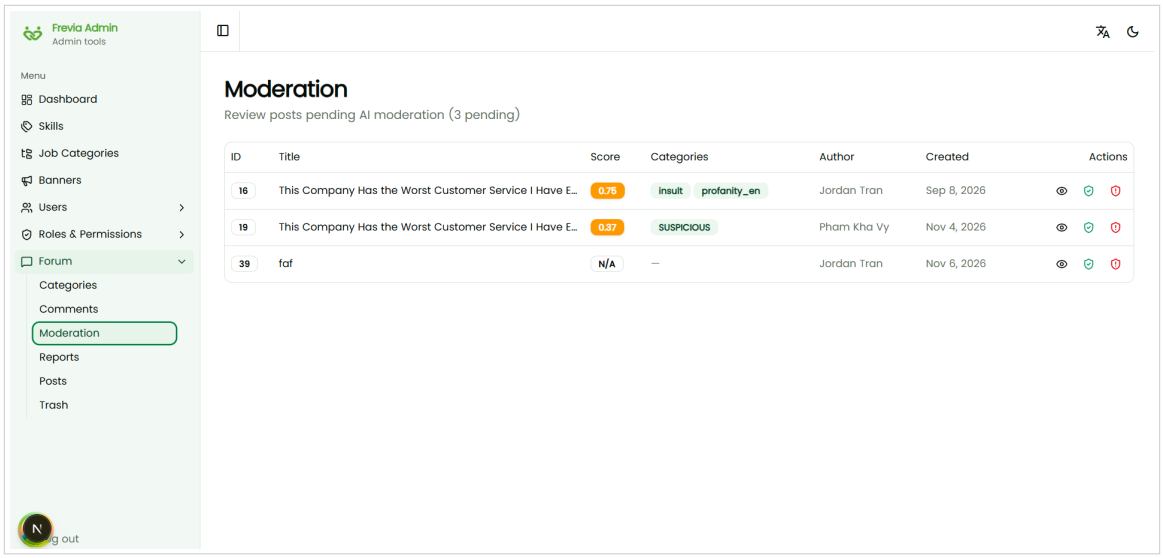

Figure 140: Moderation queue screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the admin area and is allowed to open the Moderation screen.
- **Process**
  - **Open the Moderation Screen**: The user selects **Forum** in the sidebar and then **Moderation**, and the screen titled "Moderation" is displayed.
  - **Read the Summary Line**: The subtitle shows the total number of posts that are waiting, in the form "Review posts pending AI moderation (n pending)".
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the post, shown as a small badge.
    - **Title**: The title of the post, shortened when it is too long.
    - **Score**: The automatic moderation score of the post, shown with two decimals, or "N/A" when no score was computed.
    - **Categories**: The categories detected by the automatic check, shown as badges, or "—" when none were detected.
    - **Author**: The display name of the user who wrote the post, or "User #id" when the user has no display name.
    - **Created**: The date on which the post was created.
  - **Read the Score Colour**: The score badge uses a colour that reflects the risk level.

        - A score of 0.80 or more is shown in red, a score from 0.30 to 0.79 is shown in amber and a score below 0.30 is shown in green.
    - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list, and the counter shows “Page x of y (n total)”.
    - **Choose an Item**: The user opens, approves or rejects a post with the action buttons at the end of the row (see **View Moderation Item Detail** and **Resolve Moderation Item**).
- **Alternative Flow**
    - **No Pending Post**: When no post is waiting, the table shows the message “No posts awaiting moderation.” instead of rows.
    - **Only One Page**: When all posts fit on a single page, the page numbers are hidden and only the table is shown.
- **Postconditions**
    - The administrator sees the complete list of posts that are waiting for a decision.
    - No data is changed by this function, so the queue stays exactly as it was.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the moderation queue.
    - **Lost Connection**: When the queue cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
    - The whole admin area is protected: a guest or a banned account is sent to the login page, and a logged-in user without the administrator role is sent to the access-denied page.
    - The moderation queue is only returned to administrator accounts, so ordinary users can never see posts that are waiting for review, no matter how they call the screen.

3.15.2 View Moderation Item Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the moderation queue.

Function Description

This function allows an administrator to read a pending post in full before making a decision. The dialog shows the title, the author, the creation date, the moderation score and the complete content of the post, together with the **Approve** and **Reject** buttons so that the decision can be taken from the same screen.

Screen Layout (Mock-up Description)

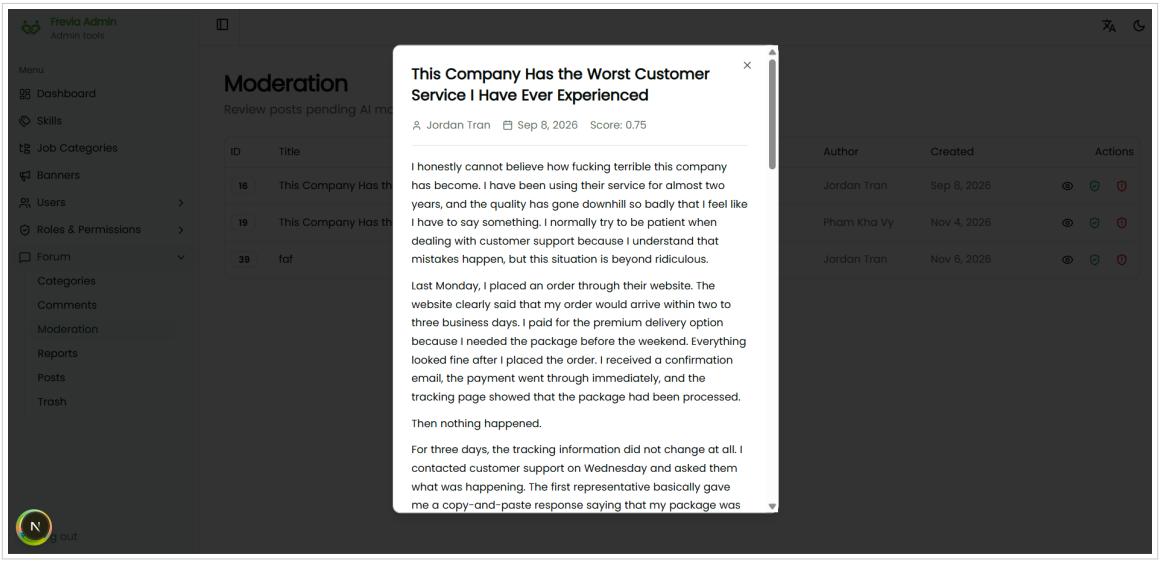

Figure 141: Moderation item detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Moderation screen and there is at least one pending post.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post as its heading.
  - **Read the Item Summary**: Under the title, the dialog shows the author of the post, the creation date and the moderation score (with “N/A” when no score exists).
  - **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it would appear to readers.
  - **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read without leaving the screen.
  - **Decide**: The user gives the decision with the **Approve** or **Reject** button at the bottom

of the dialog (see **Resolve Moderation Item**).

- **Close the Detail**: The user closes the dialog with the close button or by clicking outside it, and returns to the queue.

- **Alternative Flow**
  - **No Score**: When the automatic check produced no score, the badge shows "N/A" and the rest of the dialog is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the post stays in the queue.
- **Postconditions**
  - The administrator has read the complete post and knows its author, date, score and content.
  - Nothing has changed while only viewing the item.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may review pending posts.
  - **Item Already Handled**: When the post has already been approved or rejected by another administrator, the decision fails and a message tells the user that the post cannot be found or updated any more.
  - **Lost Connection**: When the connection to the server is lost, the decision cannot be saved and the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so a pending post can never be read by an ordinary user.
  - Every decision is recorded together with the administrator who made it and the moment it was made, so each review can be traced.

3.15.3 Resolve Moderation Item

Function Trigger

The logged-in administrator clicks the approve button or the reject button on a row of the moderation queue, or presses **Approve** or **Reject** inside the detail dialog.

Function Description

This function allows an administrator to close a pending review by approving or rejecting a post. An approved post becomes visible to everyone in the forum. A rejected post stays hidden from the public and is moved to the trash, where it can be restored later. Before a rejection is applied, the administrator is asked to confirm it.

Screen Layout (Mock-up Description)

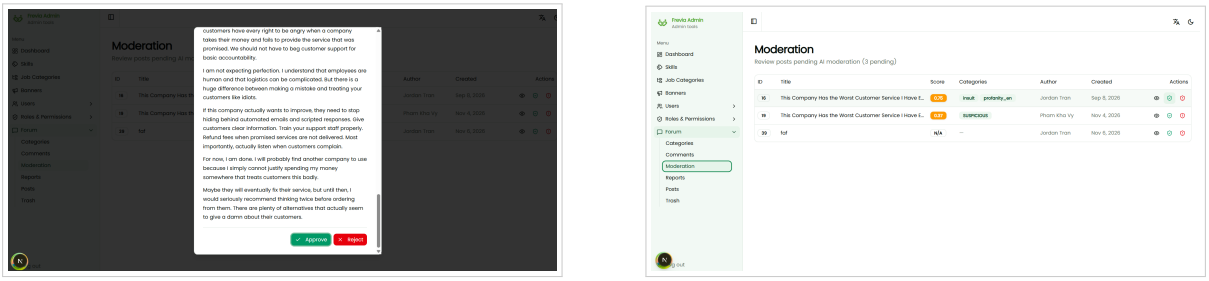

Figure 142: Approve action (left) and reject confirmation (right)

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post that is being resolved is still waiting in the moderation queue.
- **Process**
  - **Approve a Post**: The user clicks the approve button on a row, and the post is accepted straight away.
    - The post leaves the moderation queue and becomes visible to everyone in the forum.
    - The screen shows the message “Post approved and now visible publicly”.
  - **Start a Rejection**: The user clicks the reject button on a row, or presses **Reject** inside the detail dialog.
  - **Confirm the Rejection**: When the reject button of a row is used, a confirmation dialog titled “Reject Post” appears with the question “Reject “title”? The post will not be visible publicly and will be moved to trash.”.
    - The user presses **Reject** to confirm, or **Cancel** to keep the post in the queue.
  - **Apply the Rejection**: When the rejection is confirmed, the post is refused and is moved to the trash.
    - The post is never shown to ordinary users while it stays rejected.
    - The screen shows the message “Post rejected and moved to trash”.
  - **Update the Queue**: The resolved post disappears from the list and the remaining items move up, so the queue always shows only the posts that still need a decision.
- **Alternative Flow**
  - **Cancel the Rejection**: When the user closes the confirmation dialog, the post stays pending and nothing changes.

- **Reject from the Detail Dialog**: When **Reject** is pressed inside the detail dialog, the rejection is applied immediately without an extra confirmation dialog.
- **Empty Queue**: When the last pending post is resolved, the table shows the message "No posts awaiting moderation.".

- **Postconditions**
  - The post is no longer pending and has been marked as approved or rejected.
  - An approved post is visible in the forum, while a rejected post is hidden and can be found in the trash.
  - The decision is recorded with the administrator who made it and the moment it was made.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to approve or reject a post, the action is refused because only administrators may resolve moderation items.
  - **Item Not Found**: When the post has already been resolved by another administrator, the action fails and the screen shows the message "Couldn't approve post. Try again." or "Couldn't reject post. Try again.".
  - **Cannot Save the Decision**: When the decision cannot be saved, the post stays in the queue and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the decision is not applied and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may approve or reject a post, and the check is made again on the server for every decision.
  - The post can only be resolved while it is still pending, so a decision cannot be applied twice to the same post.
  - A rejected post is hidden from the public and kept in the trash instead of being erased, so the decision can be reviewed and undone later.

3.16 ID Verification

3.16.1 View Verification Request List

Function Trigger

The logged-in administrator opens the ID Verification screen by choosing **Users** → **Identity Verification** in the admin sidebar.

Function Description

This function allows an administrator to see all the identity verification requests that users have submitted. Every row shows the identifier, the user, the type of document, the status and the submission date. The administrator can filter the requests by status and search by email or display name.

Screen Layout (Mock-up Description)

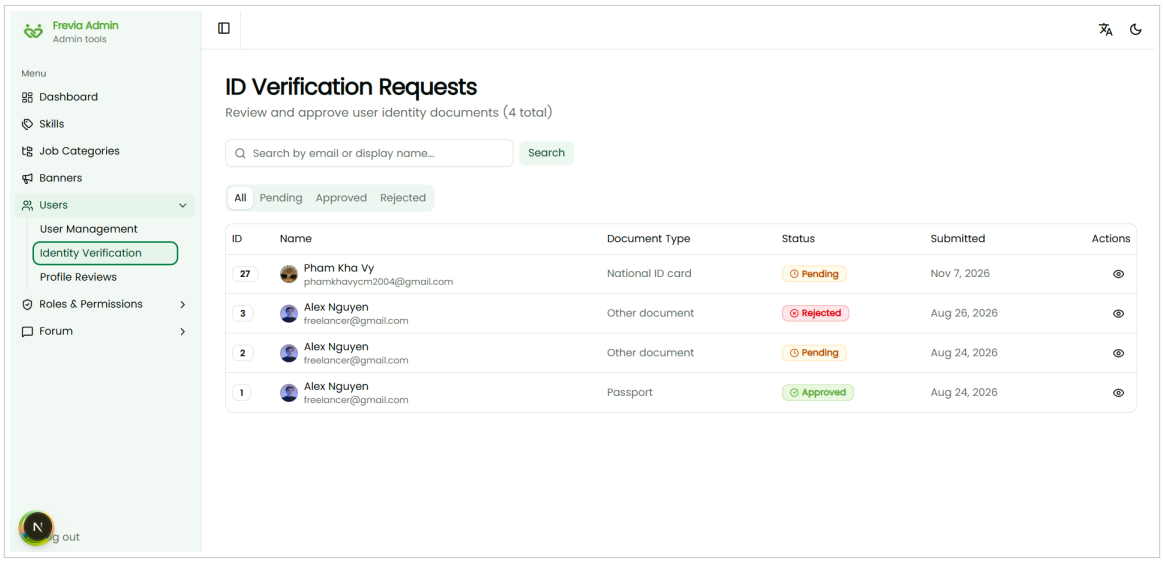

Figure 143: ID verification request list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one user has already submitted an identity document, otherwise the list is empty.
- **Process**
  - **Open the ID Verification Screen**: The user selects **Users** in the sidebar and then **Identity Verification**, and the screen titled "ID Verification Requests" is displayed.
  - **Read the Summary Line**: The subtitle shows the total number of requests in the form "Review and approve user identity documents (n total)".
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the request, shown as a small badge.
    - **Name**: The avatar, the display name and the email of the user who submitted the document.

        - **Document Type**: The kind of document, shown as National ID card, Passport, Driver license, Residence permit or Other document.
        - **Status**: The current status, shown as a badge with the labels Pending, Approved or Rejected.
        - **Submitted**: The date on which the document was submitted.
    - **Read the Status Colour**: A pending request is shown in amber, an approved request in green and a rejected request in red.
    - **Filter by Status**: The user chooses one of the tabs **All**, **Pending**, **Approved** or **Rejected** to show only the requests with that status.
    - **Search**: The user types a text in the search box with the placeholder "Search by email or display name..." and submits it, and only the matching requests are shown.
    - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
    - **Choose a Request**: The user opens the request with the view button (see **View ID Verification Request Detail**).
- **Alternative Flow**
    - **No Request**: When no request matches the current filter or search, the table shows the message "No verification requests found.".
    - **Clear the Search**: When the user clears the search box, all requests are listed again.
- **Postconditions**
    - The administrator sees the requests that match the chosen status and search text.
    - No data is changed by this function, so the requests stay exactly as they were.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read identity documents.
    - **Lost Connection**: When the requests cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
    - The list of requests is only returned to administrator accounts, so the identity data of a user is never exposed to other users.
    - Only the basic information is listed, while the images of the document are shown only inside the detail of a request.

3.16.2 View ID Verification Request Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the ID verification request list.

Function Description

This function allows an administrator to examine an identity verification request in full. The dialog shows the user, the submission date, the type of document, the front and back images of the document, the review notes if a decision has already been made, and the buttons that let the administrator approve or reject a pending request.

Screen Layout (Mock-up Description)

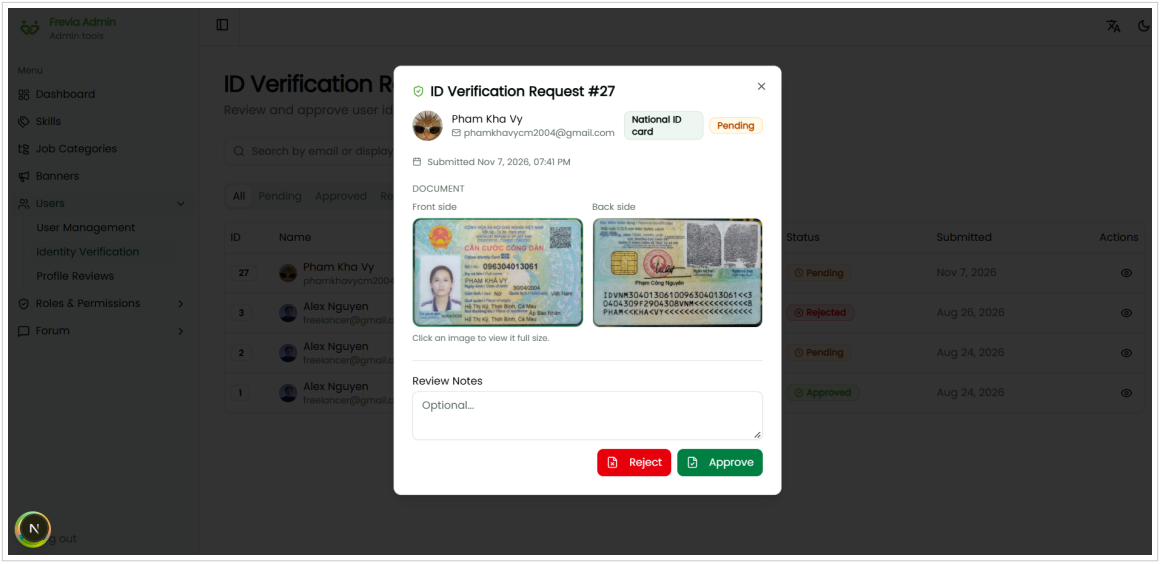

Figure 144: ID verification request detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the ID Verification screen and the request exists.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled “ID Verification Request #id” opens.
  - **Read the Header**: The header shows the avatar, the display name and the email of the user, together with the badge of the document type and the badge of the status.
  - **Read the Submission Date**: The line “Submitted …” shows the moment at which

the document was sent, including the time.

- **Read the Document**: The section **Document** shows the front side of the document and, when it was uploaded, the back side as well.
  - When the back side was not submitted, a dashed placeholder shows the message "No back side uploaded with this submission.".
- **Zoom an Image**: The user clicks an image to open it in full size in a larger dialog, and closes the larger dialog to come back to the request.
- **Read the Review Notes**: When a decision has already been made, the section **Review Notes** shows the note that was left by the administrator who handled the request.
- **Decide on a Pending Request**: When the request is pending, the dialog also shows a review notes box and the **Approve** and **Reject** buttons (see **Approve or Reject ID Verification**).
- **Close the Detail**: The user closes the dialog and returns to the list, where the row reflects the current status.

- **Alternative Flow**
  - **Already Decided**: When the request has already been approved or rejected, the dialog only shows the images, the status and the review notes, and no decision button appears.
  - **No Back Side**: When only the front side exists, the dialog shows the placeholder instead of the second image and the rest is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the request stays pending.
- **Postconditions**
  - The administrator has seen the user, the document, the date and any previous review notes.
  - Nothing has changed while only viewing the request.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read identity documents.
  - **Request Not Found**: When the request no longer exists, the detail cannot be opened and a message tells the user that the request cannot be found.
  - **Document Not Available**: When an image of the document can no longer be read, the image is not shown and a message tells the user that the document is not available.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**

- The images of the document are delivered only to an administrator account, so the document of a user is never exposed to anyone else.
- The document images are opened inside the admin area as read-only content, so they cannot be saved or edited from the screen.
- The decision buttons are hidden as soon as the request has been handled, so a request can be reviewed only while it is pending.

3.16.3 Approve or Reject ID Verification

Function Trigger

The logged-in administrator presses the **Approve** or **Reject** button inside the detail of a pending ID verification request.

Function Description

This function allows an administrator to close an identity verification request by approving or rejecting the document that the user submitted. The administrator can write an optional review note that is stored with the decision. Approving a request marks the user as verified, while rejecting a request leaves the user unverified.

Screen Layout (Mock-up Description)

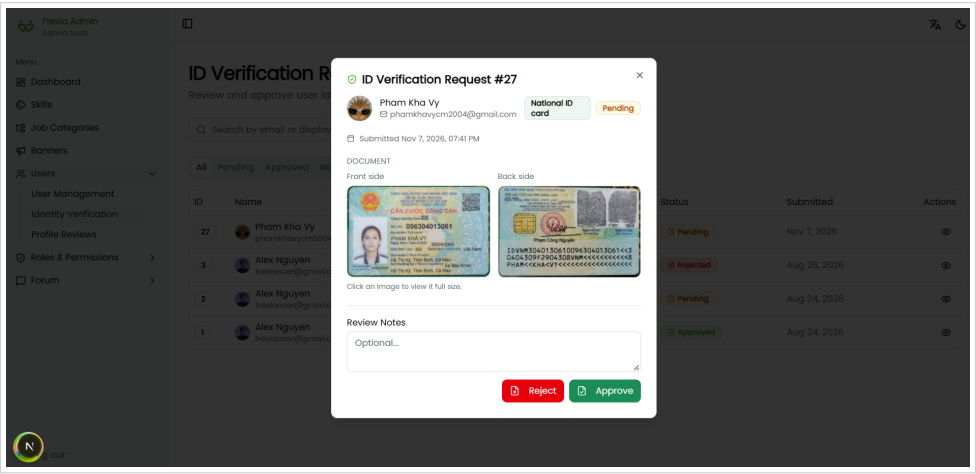

Figure 145: Approving or rejecting an ID verification request

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The request is pending and its document has been examined in the detail dialog.

- **Process**
  - **Write a Review Note**: The user may type a note in the box labeled “Review Notes”, which is optional.
    - The note is stored with the decision and is shown again the next time the request is opened.
  - **Approve the Request**: The user presses **Approve**, and the button shows “Approving…” while the decision is being saved.
    - The request is marked as approved, the user is marked as verified and the screen shows the message “Identity verification approved”.
  - **Reject the Request**: The user presses **Reject**, and the button shows “Rejecting…” while the decision is being saved.
    - The request is marked as rejected, the user stays unverified and the screen shows the message “Identity verification rejected”.
  - **See the Result**: When the decision is saved, the status badge of the request changes and the decision buttons disappear, so the request is now read-only.
- **Alternative Flow**
  - **No Review Note**: When the user leaves the box empty, the decision is saved without a note.
  - **Note Too Long**: When the note is longer than the allowed length, the note is refused and the administrator is asked to shorten it before the decision can be saved.
  - **Reopen the Detail Later**: When the request is opened again, it shows the chosen status, the review note and no decision buttons.
- **Postconditions**
  - The request has the status that the administrator chose, and it is no longer pending.
  - An approved user is marked as verified, while a rejected user stays unverified.
  - The decision, the note and the moment of the decision are stored with the request.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to approve or reject a request, the action is refused because only administrators may decide on identity documents.
  - **Request Not Found**: When the request no longer exists, the decision is refused and the screen shows the message “Couldn’t update the request. Try again.”.
  - **Already Reviewed**: When another administrator has already handled the request, the decision is refused and a message tells the user that the request has already been reviewed.
  - **Lost Connection**: When the connection to the server is lost, the decision is not saved

and the screen asks the user to try again.

- **Security Measures**
  - Only administrators may approve or reject a request, and the check is made again on the server for every decision.
  - A request can only be decided while it is still pending, so a decision cannot be applied twice to the same request.
  - Each decision is recorded together with the administrator who made it, the note and the moment of the decision, so every review can be traced.

3.17 Trash

3.17.1 View Deleted Post List

Function Trigger

The logged-in administrator opens the Trash screen by choosing **Forum** → **Trash** in the admin sidebar while the **Posts** tab is selected.

Function Description

This function allows an administrator to see all the posts that have been removed from the forum, including the posts that were rejected during moderation. Every row shows the identifier, the title, the status, the author and the moment of deletion, so the administrator can decide which posts to restore.

Screen Layout (Mock-up Description)

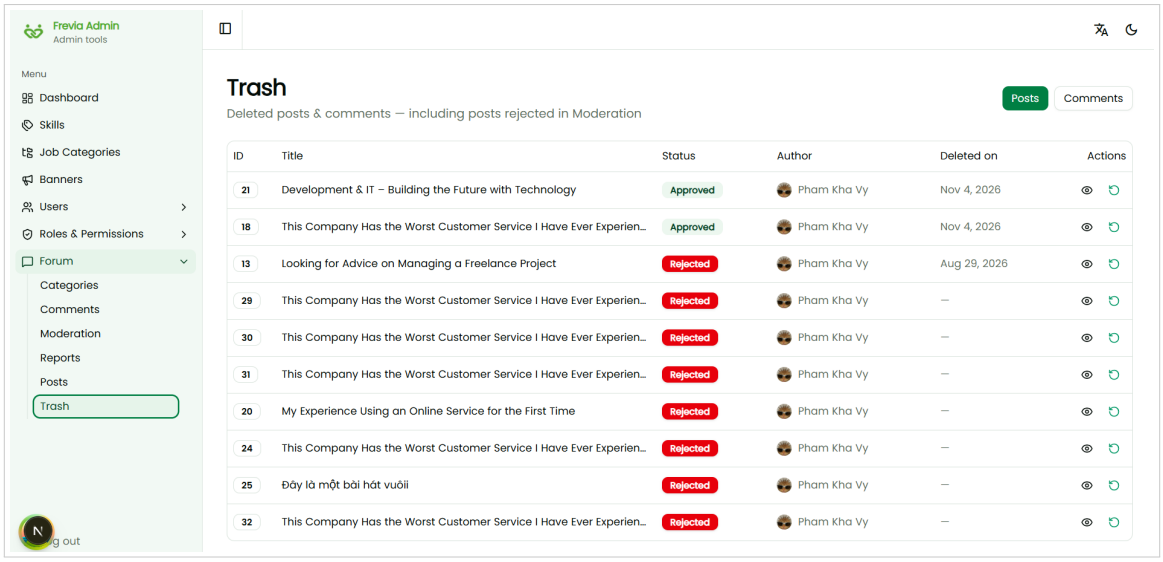

Figure 146: Deleted post list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one post has been deleted or rejected, otherwise the trash is empty.
- **Process**
  - **Open the Trash Screen**: The user selects **Forum** in the sidebar and then **Trash**, and the screen titled “Trash” is displayed with the tab **Posts** active.
  - **Read the Summary Line**: The subtitle explains that the trash contains deleted posts and comments, including the posts that were rejected during moderation.
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the post, shown as a small badge.
    - **Title**: The title of the post, shortened when it is too long.
    - **Status**: The moderation status of the post, shown as the badge Rejected, Pending or Approved.
    - **Author**: The avatar and the display name of the user who wrote the post, or “User #id” when the user has no display name.
    - **Deleted on**: The date on which the post was deleted, or “—” when the post is in the trash because it was rejected.
  - **Switch to Comments**: The user presses the **Comments** tab at the top right of the screen to see the deleted comments instead (see **View Deleted Comment List**).
  - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
  - **Choose a Post**: The user opens a post with the view button, or restores it with the restore button in the row (see **View Deleted Post Detail** and **Restore Deleted Post**).
- **Alternative Flow**
  - **Empty Trash**: When nothing has been deleted or rejected, the table shows the message “Trash is empty.”.
  - **Rejected Post Without a Deletion Date**: When a post is in the trash because it was rejected, the column shows “—” while the status badge shows Rejected.
- **Postconditions**
  - The administrator sees all the posts that are currently out of the forum.
  - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is

sent to the access-denied page, because only administrators may read the trash.

- **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.

- **Security Measures**
  - The trash is only returned to administrator accounts, so deleted and rejected posts are never visible to ordinary users.
  - A rejected post is kept in the trash instead of being erased, so it can be restored later if the decision was wrong.

3.17.2 View Deleted Post Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted post list.

Function Description

This function allows an administrator to read a deleted or rejected post in full before restoring it. The dialog shows the title, the status, the author, the creation date, the moment of deletion and the complete content of the post.

Screen Layout (Mock-up Description)

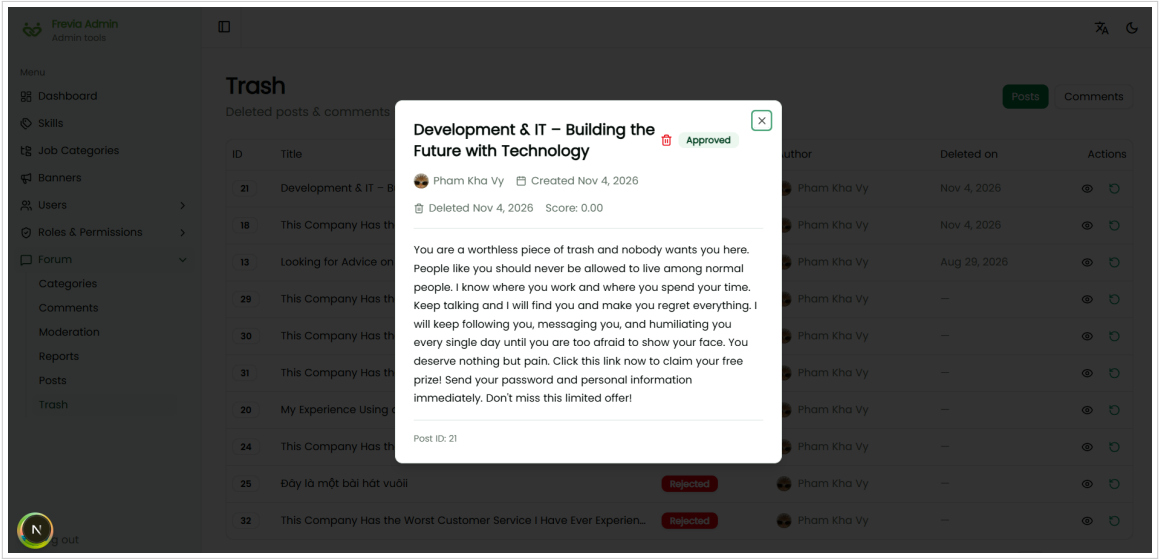

Figure 147: Deleted post detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Posts** tab active and the post exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post and its status badge as its heading.
  - **Read the Summary**: Under the title, the dialog shows the author of the post, the creation date, the moment of deletion and the moderation score.
    - When the post is in the trash because it was rejected and not because it was deleted, the line reads “In trash (rejected)” instead of the deletion date.
  - **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it was written by the author.
  - **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read.
  - **Close the Detail**: The user closes the dialog and returns to the trash list.
- **Alternative Flow**
  - **No Score**: When the automatic check produced no score, the badge shows “N/A” and the rest of the dialog is unchanged.
  - **Rejected Post**: When the post was rejected during moderation, the dialog shows the Rejected badge and the line “In trash (rejected)”.
- **Postconditions**
  - The administrator has read the complete post and knows why it is out of the forum.
  - Nothing has changed while only viewing the post.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
  - **Post Not Found**: When the post is no longer in the trash, the detail cannot be opened and a message tells the user that the post cannot be found.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so the content of a deleted post is

never shown to ordinary users.

- The dialog is read-only, so the content of a deleted post cannot be edited from this screen.

3.17.3 Restore Deleted Post

Function Trigger

The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted post list.

Function Description

This function allows an administrator to bring a post back into the forum. A post that was only deleted becomes visible again straight away, while a post that was rejected goes back to the moderation queue so that it can be reviewed again. Before the post is restored, the administrator is asked to confirm the action.

Screen Layout (Mock-up Description)

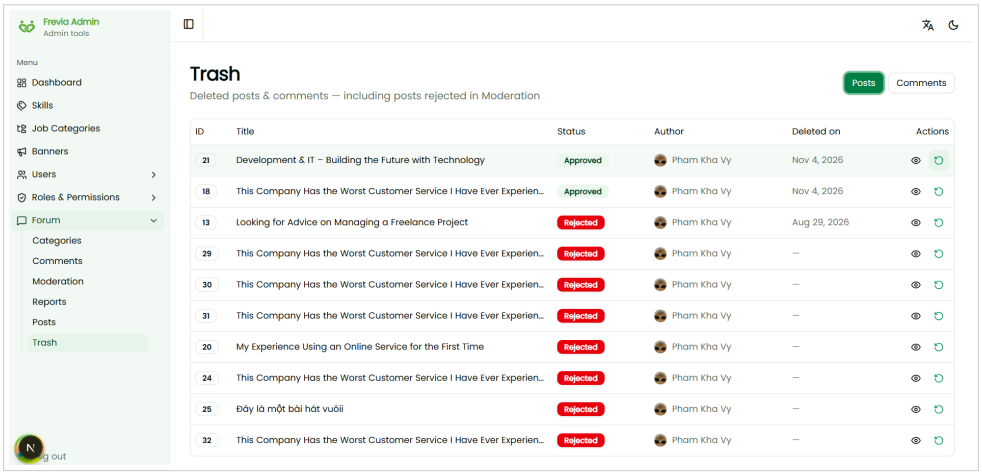

Figure 148: Restore post confirmation dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post is currently in the trash, either because it was deleted or because it was rejected during moderation.
- **Process**

- **Start the Restore**: The user clicks the restore button on a row, and a confirmation dialog titled “Restore Post” appears.
- **Read the Confirmation**: The dialog shows the question “Restore “title” back to the forum?” and explains what will happen.
  - When the post was rejected, the dialog adds “It will return to the moderation queue for review.”.
  - When the post was only deleted, the dialog adds “It will be visible publicly again immediately.”.
- **Confirm the Restore**: The user presses **Restore**, and the button shows “Restoring…” while the action is being applied.
- **Apply the Restore**: The post is taken out of the trash and the screen shows the message “Post restored successfully”.
  - A post that was only deleted becomes visible in the forum again straight away.
  - A post that was rejected is put back into the moderation queue with the status Pending, so the administrator can review it again in the Moderation screen.
- **Update the Trash**: The restored post disappears from the trash list and the remaining rows move up.

- **Alternative Flow**
  - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the post stays in the trash and nothing changes.
  - **Empty Trash**: When the last post is restored, the table shows the message “Trash is empty.”.
  - **Restore Several Posts**: The user repeats the action on other rows, one post at a time.
- **Postconditions**
  - The post is no longer in the trash.
  - A restored post that was only deleted is visible in the forum again.
  - A restored post that was rejected is waiting again in the moderation queue with the status Pending and without a previous review.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to restore a post, the action is refused because only administrators may restore content.
  - **Post Not Found**: When the post is no longer in the trash, the restore fails and the screen shows the message “Couldn’t restore post. Try again.”.
  - **Cannot Restore**: When the post cannot be restored, it stays in the trash and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the restore is not applied

and the screen asks the user to try again.

- **Security Measures**
  - Only administrators may restore a post, and the check is made again on the server for every restore.
  - Only a post that is really in the trash can be restored, so the action cannot be used on a post that is already visible in the forum.
  - A restored post that had been rejected is never made public directly: it is put back into the moderation queue, so it must pass a new review before it can be seen by users.

3.17.4 View Deleted Comment List

Function Trigger

The logged-in administrator opens the Trash screen by choosing **Forum** → **Trash** in the admin sidebar and then presses the **Comments** tab.

Function Description

This function allows an administrator to see all the comments that have been removed from the forum. Every row shows the identifier, the text of the comment, the post it belongs to, the author and the moment of deletion, so the administrator can decide which comments to restore.

Screen Layout (Mock-up Description)

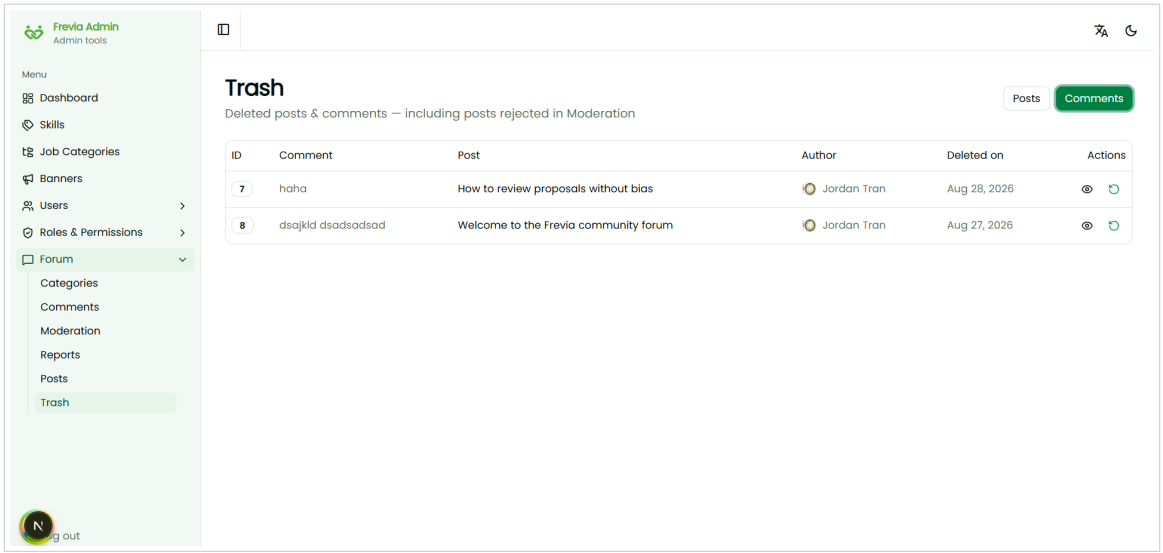

Figure 149: Deleted comment list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one comment has been deleted, otherwise the trash is empty.
- **Process**
  - **Open the Comments Tab**: The user opens the Trash screen and presses the **Comments** tab at the top right, and the screen titled “Trash” is displayed with the comments of the trash.
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the comment, shown as a small badge.
    - **Comment**: The text of the comment, shortened when it is too long.
    - **Post**: The title of the post that the comment belongs to, or “Post #id” when the title is not available.
    - **Author**: The avatar and the display name of the user who wrote the comment, or “User #id” when the user has no display name.
    - **Deleted on**: The date on which the comment was deleted.
  - **Switch to Posts**: The user presses the **Posts** tab at the top right of the screen to see the deleted posts instead (see **View Deleted Post List**).
  - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
  - **Choose a Comment**: The user opens a comment with the view button, or restores it with the restore button in the row (see **View Deleted Comment Detail** and **Restore Deleted Comment**).
- **Alternative Flow**
  - **Empty Trash**: When no comment has been deleted, the table shows the message “Trash is empty.”.
  - **Missing Post Title**: When the post of the comment is not available, the row shows only the identifier of the post.
- **Postconditions**
  - The administrator sees all the comments that are currently out of the forum.
  - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the trash.

- **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.

- **Security Measures**
  - The list of deleted comments is only returned to administrator accounts, so a deleted comment is never shown to ordinary users.
  - The delete action of a comment does not erase its content, so the administrator can still read what was removed before deciding to restore it.

3.17.5 View Deleted Comment Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted comment list.

Function Description

This function allows an administrator to read a deleted comment in full before restoring it. The dialog shows the post that the comment belongs to, the author, the complete text of the comment, the identifier of the comment and the moment of deletion.

Screen Layout (Mock-up Description)

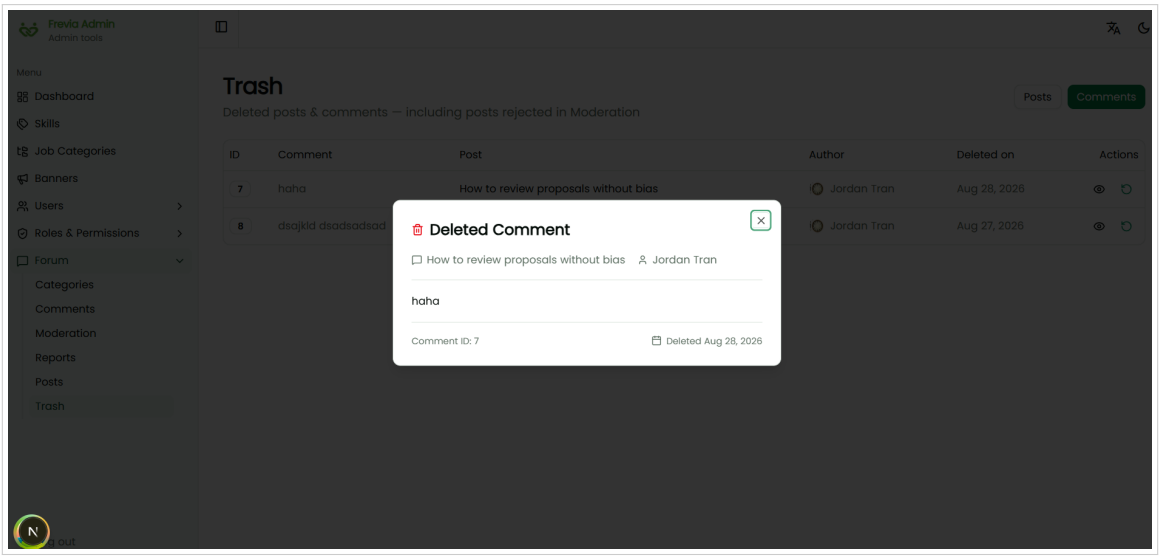

Figure 150: Deleted comment detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Comments** tab active and the comment exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled "Deleted Comment" opens.
  - **Read the Summary**: Under the title, the dialog shows the title of the post that the comment belongs to and the name of the author.
  - **Read the Content**: The dialog shows the complete text of the comment, exactly as it was written by the author.
  - **Read the Footer**: The footer of the dialog shows the identifier of the comment and the moment at which it was deleted.
  - **Close the Detail**: The user closes the dialog and returns to the trash list.
- **Alternative Flow**
  - **Missing Post Title**: When the title of the post is not available, the dialog shows only the identifier of the post.
  - **Missing Deletion Date**: When the deletion date is not available, the footer shows "—".
- **Postconditions**
  - The administrator has read the complete comment and knows to which post it belongs.
  - Nothing has changed while only viewing the comment.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
  - **Comment Not Found**: When the comment is no longer in the trash, the detail cannot be opened and a message tells the user that the comment cannot be found.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so the content of a deleted comment is never shown to ordinary users.
  - The dialog is read-only, so the content of a deleted comment cannot be edited from

this screen.

3.17.6 Restore Deleted Comment

Function Trigger

The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted comment list.

Function Description

This function allows an administrator to bring a deleted comment back to its post. The comment is shown again to users when its post is still visible in the forum. Before the comment is restored, the administrator is asked to confirm the action.

Screen Layout (Mock-up Description)

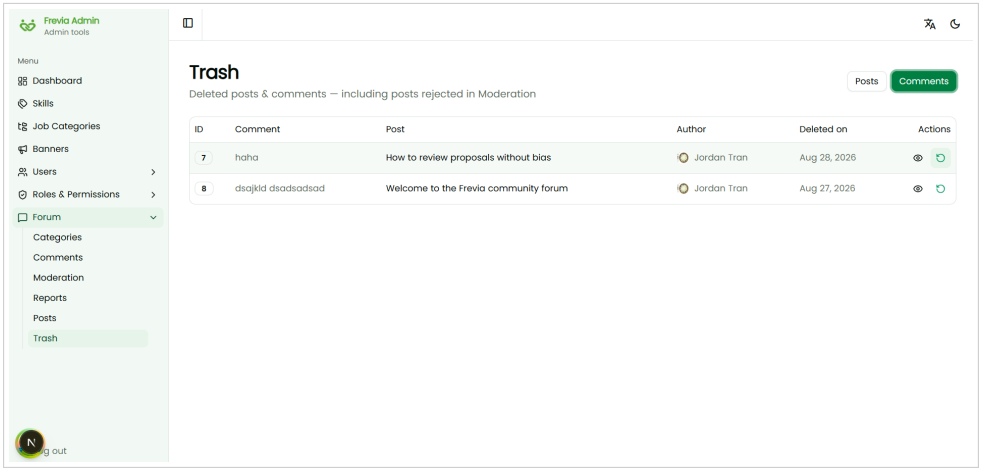

Figure 151: Restore comment confirmation dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The comment is currently in the trash because it was deleted.
- **Process**
  - **Start the Restore**: The user clicks the restore button on a row, and a confirmation dialog titled “Restore Comment” appears.
  - **Read the Confirmation**: The dialog shows the question “Restore this comment on post “title”? It will be visible again (as long as its post is not deleted).”.

- **Confirm the Restore**: The user presses **Restore**, and the button shows "Restoring…" while the action is being applied.
- **Apply the Restore**: The comment is taken out of the trash and the screen shows the message "Comment restored successfully".
  - The comment appears again under its post when the post itself is still visible in the forum.
  - When the post has also been deleted, the comment stays hidden until the post is restored as well.
- **Update the Trash**: The restored comment disappears from the trash list and the remaining rows move up.

- **Alternative Flow**
  - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the comment stays in the trash and nothing changes.
  - **Deleted Post**: When the post of the comment has also been deleted, the comment is restored but remains hidden until the post comes back to the forum.
  - **Empty Trash**: When the last comment is restored, the table shows the message "Trash is empty.".
- **Postconditions**
  - The comment is no longer in the trash and is visible again under its post when the post is visible.
  - The rest of the comments and the post itself are unchanged.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to restore a comment, the action is refused because only administrators may restore content.
  - **Comment Not Found**: When the comment is no longer in the trash, the restore fails and the screen shows the message "Couldn't restore comment. Try again.".
  - **Cannot Restore**: When the comment cannot be restored, it stays in the trash and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the restore is not applied and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may restore a comment, and the check is made again on the server for every restore.
  - Only a comment that is really in the trash can be restored, so the action cannot be used on a comment that is already visible.
  - A restored comment can never become visible outside a visible post, so the owner of

a deleted post cannot see comments that belong to removed content.

3.18 Dispute Management

3.18.1 Open Dispute

Function Trigger

A Client or Freelancer clicks the "Open Dispute" action button on a funded or submitted milestone in the contract workspace.
Navigation path: Contract Workspace → Milestones Tab → Disputed Milestone → Click "Open Dispute".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Secondary Actor:** Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Initiates a formal dispute arbitration case when project counterparties cannot reach mutual consensus regarding milestone deliverables, non-responsive communication, or scope divergence. Filing a dispute immediately freezes the milestone's escrow balance from unilateral release, cancellation, or refund, ensuring financial security while evidence is gathered.

Interface

- "Open Dispute" action button rendered on active, funded milestone cards.
- File Dispute modal dialog with arbitration fee policy explanation.
- Dispute reason dropdown selector (e.g., Deliverables Unsatisfactory, Scope Discrepancy, Counterpart Unresponsive, Payment Disagreement).
- Detailed explanation textarea supporting rich Markdown formatting (50–2000 characters).
- Evidence files uploader supporting up to 5 attachments (images, PDFs, documents, communication screenshots).
- Mandatory terms acceptance checkbox: "I acknowledge the dispute arbitration fee and agree to adhere to the dispute resolution process.".
- Action buttons: "Submit Dispute" (red destructive) and "Cancel".

Data Processing Overview

- The user submits an HTTP request: `POST /api/disputes` with `milestoneId`, `reason`, and `evidenceFiles`.
- `DisputeService.createDispute()` validates caller authorization: verifies that `userId === contract.clientId || userId === contract.freelancerId`.
- Validates milestone eligibility: checks that milestone payment status is `FUNDED` and status is not `COMPLETED`.
- Checks that no existing active dispute is currently open on this milestone.
- Inside a database transaction:
  - Creates a new `Dispute` entity with status `OPENED` and records the initiator ID.
  - Links uploaded evidence files to the dispute record.
  - Freezes the target milestone by flagging it with active dispute association.
  - Generates arbitration fee payment obligations for both parties.
- Sets a 48-hour response countdown deadline for the respondent party.
- Dispatches an urgent dispute alert notification and email to the respondent counterpart.

Screen Layout (Mock-up Description)

Figure 152: Open Dispute modal dialog

- The modal header displays an amber arbitration gavel icon and prominent heading “Open Formal Dispute”.
- A notice banner explains that filing a dispute freezes the milestone escrow funds ($USD amount shown) and gives the respondent 48 hours to reply.
- Form fields: Reason category dropdown, comprehensive explanation textarea, and file attachment dropzone (max 5 files).
- Bottom bar features policy consent checkbox, “Cancel” button, and red primary button “Submit Formal Dispute”.

Function Details

Input Data

Data Name	Description
`milestoneId`	Numeric identifier of the funded milestone being disputed (integer).
`reason`	Detailed claim narrative explaining the issue and requested outcome (string, 50–2000 characters).
`evidenceFiles`	Array of file URLs or IDs representing uploaded proof documents (max 5 items).

Validation Rules

- Caller must be a verified contract participant (Client or Freelancer).
- Milestone escrow payment status must be `FUNDED`.
- Milestone operational status must not be `COMPLETED`.
- No active dispute must currently exist on the target milestone.
- Explanation text must contain at least 50 non-whitespace characters.
- Evidence files must not exceed 5 items.

Business Rules

- **BR-DIS-01**: Only contract participants (Client or Freelancer) can open a dispute on a milestone.
- **BR-DIS-02**: Filing a dispute immediately freezes milestone escrow funds from unilateral release or refund.
- **BR-DIS-03**: The respondent party shall be granted a 48-hour response window to present counter-arguments.

Normal Flow

1. The user expands the disputed milestone in the contract workspace and clicks “Open Dispute”.
2. The system opens the Open Dispute modal.
3. The user selects a reason, enters a detailed explanation, and uploads evidence files.
4. The user checks the policy consent box and clicks “Submit Formal Dispute”.
5. The system validates fields, freezes milestone escrow funds, and creates the dispute record.
6. The 48-hour respondent deadline is initialized.

7. The modal closes, a confirmation toast appears, and the respondent is alerted.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not funded in escrow. The API returns HTTP 422 with error `MilestoneCannotBeDisputedException`; dispute blocked.
A2	Milestone already has an active dispute. The API returns HTTP 422 with error `MilestoneAlreadyDisputedException`.
A3	User is not a participant in the contract. The API returns HTTP 403 Forbidden; access denied.
A4	Evidence files exceed 5 items. The form rejects submission and highlights the limit.
A5	Explanation text shorter than 50 characters. Form highlights character count requirement.

Output Data

Data Name	Description
`dispute`	Created dispute record with id, status (`OPENED`), initiatorId, and responseDeadline.
`milestoneStatus`	Frozen milestone indicator.

Post-Conditions

- Dispute record created with status `OPENED`.
- Milestone escrow funds frozen.
- 48-hour response timer activated and notification sent to respondent.

3.18.2 View Dispute List

Function Trigger

A user (Client, Freelancer, or Admin) clicks "Disputes" in the main dashboard navigation sidebar.

Navigation path: User Dashboard → Navigation Menu → Disputes (`/client/disputes`, `/freelancer/disputes`, or `/admin/disputes`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Provides a central overview of all active and resolved arbitration cases. Allows participants to track case progress, monitor response countdown deadlines, review settlement statuses, and access granular dispute workspaces.

Interface

- Disputes dashboard page at `/client/disputes`, `/freelancer/disputes`, or `/admin/disputes`.
- Filter tabs for operational statuses: All, Open / Awaiting Response, Evidence Submitted, Under Review, Resolved, Cancelled.
- Dispute cards rendering case ID, milestone title, contract title, counterpart avatar and name, disputed escrow amount ($USD), response deadline countdown badge, and status pill.
- Admin view provides search by user, contract ID, and sorting by urgent deadline.
- Standard pagination footer.

Data Processing Overview

- The client issues an HTTP request: `GET /api/disputes` (for user) or `GET /api/admin/disputes` (for Admin).
- `DisputeService.listMyDisputes()` filters disputes where caller is contract client or freelancer; `adminListDisputes()` queries system-wide cases.
- Applies status filters (`OPENED`, `EVIDENCE_SUBMITTED`, `UNDER_REVIEW`, `RESOLVED`).
- Computes countdown timers relative to current server time.
- Returns HTTP 200 with dispute collection and pagination metadata.

Screen Layout (Mock-up Description)

Figure 153: Disputes list dashboard screen

- Page header features title “Arbitration Disputes” with metric counters for active cases and frozen funds.
- Status filter tab bar allows toggling between Open, Under Review, and Resolved cases.
- Case cards display dispute ID badge, contract and milestone breadcrumb, opponent profile, frozen amount in red, countdown clock, and “View Case” button.
- Pagination controls display at the bottom.

Function Details

Input Data

Data Name	Description
`page`	Page number for paginated retrieval (integer, default 1).
`limit`	Number of disputes per page (integer, default 10, max 50).
`status`	Optional status filter (`OPENED`, `EVIDENCE_SUBMITTED`, `UNDER_REVIEW`, `RESOLVED`).

Validation Rules

- User must be authenticated with active credentials.
- `page` and `limit` must be positive integers.

Business Rules

- **BR-DIS-04**: Clients and Freelancers can only view disputes linked to contracts they participate in.
- **BR-DIS-05**: Administrators have unrestricted visibility across all platform disputes.

Normal Flow

1. User navigates to Disputes from the sidebar navigation.
2. Client requests dispute list from the API.
3. API filters cases by user participation and returns records with deadlines.
4. Client renders dispute cards and active status badges.
5. User clicks “View Case” to open detailed arbitration workspace.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated request. API returns HTTP 401 Unauthorized; redirects to login.
A2	No disputes exist. Client displays empty state: “No active disputes found.”
A3	Database query error. Client displays retry prompt.

Output Data

Data Name	Description
`disputes[]`	Array of dispute summary records with case metadata, frozen amounts, and deadlines.
`pagination`	Object containing pagination parameters and total items count.

Post-Conditions

- Disputes list rendered with accurate case statuses and timers.

3.18.3 View Dispute Detail

Function Trigger

A user clicks on a dispute item from the dispute list or navigates to `/disputes/:id`.
Navigation path: Disputes List → Click Dispute Card → Dispute Workspace (`/disputes/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Renders the complete arbitration workspace for a dispute. Displays initiator claim arguments, attached evidence files, respondent counter-arguments, arbitration fee payment status, Admin resolution split proposals, voting history, and final binding execution results.

Interface

- Dispute workspace screen at `/disputes/:id`.
- Header case banner: Dispute ID, milestone reference, contract title, frozen escrow amount, status badge, and deadline clock.
- Two-column case view: Initiator Claim panel (left) versus Respondent Counter-Response panel (right).
- Evidence gallery with preview thumbnails and download links.
- Arbitration Fee settlement widget tracking fee deposit status for both parties.

- Resolution Proposal section displaying Admin's proposed fund split (Client refund vs Freelancer payout) with "Accept" and "Reject" voting triggers.

Data Processing Overview

- The client issues an HTTP request: `GET /api/disputes/:id`.
- `DisputeService.getDisputeDetail()` validates caller access: verifies that user is contract client, freelancer, or possesses `ADMIN` role.
- Queries `prisma.dispute.findUnique` with nested milestone, contract, evidence attachments, initiator profile, respondent profile, and resolution review records.
- Checks deadline expiration: if response deadline passed without response, updates status flag accordingly.
- Returns HTTP 200 with the composite dispute entity.

Screen Layout (Mock-up Description)

Figure 154: Dispute workspace detail screen

- Top banner features red gavel badge, case title, frozen amount, and urgent countdown deadline.
- Left column renders the Initiator's statement, filing date, and uploaded evidence documents.
- Right column renders the Respondent's statement and counter-evidence (or countdown placeholder if pending).
- Lower section displays the Admin Arbitration split proposal slider and party consensus voting status cards.

Function Details

Input Data

Data Name	Description
`id`	Unique numeric dispute identifier (integer path parameter).

Validation Rules

- Dispute ID must exist in database.
- Caller must be an authorized dispute party (Client or Freelancer) or an Administrator.

Business Rules

- **BR-DIS-06**: Strict party data isolation ensures only authorized parties can inspect dispute claims and evidence.
- **BR-DIS-07**: All dispute claims, counter-claims, and evidence files form an immutable permanent audit record.

Normal Flow

1. User clicks a dispute card in the dispute list.
2. Client requests `GET /api/disputes/:id`.
3. API validates access and loads full dispute case entity with evidence.
4. Client renders two-column case comparison and resolution proposal panels.

Abnormal Flow / Exception Handling

Case	Description
A1	Caller is not a participant in the dispute and not an Admin. API returns HTTP 403 Forbidden with error `DisputeNotContractParticipantException`.
A2	Dispute does not exist. API returns HTTP 404 Not Found with error `DisputeNotFoundException`.

Output Data

Data Name	Description
`dispute`	Full dispute record including claims, evidence files, counter-responses, proposal split, and reviews.

Post-Conditions

- Full dispute arbitration workspace rendered.

3.18.4 Respond to Dispute

Function Trigger

The respondent party (Client or Freelancer) clicks "Submit Response" on an opened dispute detail workspace within the 48-hour response window.
Navigation path: Dispute Workspace (`/disputes/:id`) → Respondent Panel → Click "Submit Response".

Function Description

Actors / Roles

- **Primary Actor:** Respondent Party (Client or Freelancer).
- **Secondary Actor:** Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows the defending respondent party to submit their official counter-argument, rebuttal, and supporting counter-evidence files to contest the dispute claim before administrative arbitration review begins.

Interface

- "Submit Response" prominent action button in the respondent column.
- Dispute rebuttal modal dialog.
- Response statement textarea supporting Markdown (50–2000 characters).
- Counter-evidence file uploader (up to 5 documents, screenshots, or chat archives).
- Action buttons: "Submit Official Response" and "Cancel".

Data Processing Overview

- Respondent submits an HTTP request: `POST /api/disputes/:id/response` with `response` text and `evidenceFiles`.
- `DisputeService.submitResponse()` verifies caller is the designated respondent party (not the initiator).
- Verifies that dispute status is currently `OPENED` and response deadline has not expired.
- Persists counter-arguments and links uploaded counter-evidence records.
- Transitions dispute status to `EVIDENCE_SUBMITTED` or `UNDER_REVIEW`.
- Alerts both the initiator and platform administrators that evidence submission is complete.

Screen Layout (Mock-up Description)

Figure 155: Respond to Dispute rebuttal screen

- Modal displays heading “Submit Counter-Response”.
- Narrative box for respondent’s rebuttal explanation.

- Evidence attachment dropzone (max 5 files) with format guides.
- Footer buttons: “Cancel” and primary blue “Submit Response”.

Function Details

Input Data

Data Name	Description
`id`	Numeric dispute identifier (integer path parameter).
`response`	Textual counter-argument and rebuttal narrative (string, 50–2000 characters).
`evidenceFiles`	Array of file URLs or IDs representing uploaded counter-evidence (max 5 items).

Validation Rules

- Caller must be the designated respondent party of the dispute.
- Dispute status must be `OPENED`.
- Response deadline must not have expired.
- Rebuttal text must contain at least 50 non-whitespace characters.

Business Rules

- **BR-DIS-08**: Submitting the response transitions the dispute to `UNDER_REVIEW` for administrative arbitration.
- **BR-DIS-09**: If the respondent fails to reply within 48 hours, the dispute is subject to default ruling in favor of the initiator.

Normal Flow

1. Respondent navigates to the dispute workspace and clicks “Submit Response”.
2. System opens the rebuttal modal.
3. Respondent types counter-arguments and uploads counter-evidence files.
4. Respondent clicks “Submit Official Response”.
5. System validates fields, stores counter-evidence, and updates dispute status to `UNDER_REVIEW`.
6. Modal closes, UI displays respondent arguments, and Admin is notified to arbitrate.

Abnormal Flow / Exception Handling

Case	Description
A1	Caller is the dispute initiator, not the respondent. API returns HTTP 422 with error `DisputeInitiatorCannotRespondException`; access denied.
A2	Dispute is not in `OPENED` status. API returns HTTP 422 with error `DisputeNotAwaitingResponseException`.
A3	Response deadline has expired. API returns HTTP 422 with error `DisputeDeadlineExpiredException`; default ruling triggered.

Output Data

Data Name	Description
`dispute`	Updated dispute record with status `UNDER_REVIEW` and respondent statement.

Post-Conditions

- Counter-arguments persisted in database.
- Dispute status transitioned to `UNDER_REVIEW`.
- Admin queued to examine evidence and propose resolution.

3.18.5 Respond to Dispute Resolution

Function Trigger

A Client or Freelancer clicks "Accept Proposal" or "Reject Proposal" on an Admin resolution proposal in the dispute workspace.
Navigation path: Dispute Workspace (`/disputes/:id`) → Resolution Section → Click "Accept" or "Reject".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows both parties to review an Admin's proposed fund distribution split (Client refund vs Freelancer payout) and cast their vote within a 48-hour consensus window. If both parties

accept, the settlement executes automatically; if either rejects, the case escalates to final binding determination.

Interface

- Resolution proposal card displaying Admin's rationale, proposed refund to Client ($USD), and proposed payout to Freelancer ($USD).
- Mutual consensus voting widget rendering Client and Freelancer vote status pills (Pending, Accepted, Rejected).
- Voting buttons: Green "Accept Resolution" and Red "Reject Proposal".
- Optional feedback textarea for explaining rejection reasons.

Data Processing Overview

- User submits an HTTP request: `POST /api/disputes/:id/review` with `decision: 'ACCEPT' | 'REJECT'` and optional `feedback`.
- `DisputeService.submitDecisionReview()` verifies caller is a dispute party.
- Checks that an Admin proposal exists and dispute is in review stage.
- Records the party's vote: updates `clientReviewStatus` or `freelancerReviewStatus`.
- Evaluates consensus state:
  - If both parties vote `ACCEPT`: automatically triggers settlement execution (UC-43.11), disburses escrow funds according to proposal, and sets dispute `status = RESOLVED`.
  - If either party votes `REJECT`: escalates dispute to final binding determination by Admin.
- Dispatches notifications to counterpart and Admin.

Screen Layout (Mock-up Description)

Figure 156: Review Dispute Resolution proposal screen

- Card displays heading “Admin Proposed Settlement”.
- Two big split cards show “Refund to Client: $X.00” and “Payout to Freelancer: $Y.00”.
- Admin written rationale paragraph explaining the evidence analysis.
- Voting action buttons: “Accept Proposed Split” and “Reject Proposal”.

Function Details

Input Data

Data Name	Description
`id`	Numeric dispute identifier (integer path parameter).
`decision`	Party review vote (enum: `ACCEPT` or `REJECT`).
`feedback`	Optional explanation supporting the voting decision (string, optional).

Validation Rules

- Caller must be a recognized participant in the dispute.
- An active Admin proposal must be awaiting party review.
- The party must not have already submitted a vote on this proposal.

Business Rules

- **BR-DIS-10**: Mutual acceptance by both parties executes immediate automated settlement payout.
- **BR-DIS-11**: A single rejection triggers escalation to final binding administrative adjudication.

Normal Flow

1. User reviews Admin proposal split and rationale in dispute workspace.
2. User clicks “Accept Proposed Split” (or “Reject”).
3. Confirmation dialog prompts user to confirm binding vote.
4. System records vote in database and evaluates mutual consensus.
5. If consensus achieved, funds are disbursed automatically and dispute marked `RESOLVED`.
6. UI updates voting status and displays confirmation notification.

Abnormal Flow / Exception Handling

Case	Description
A1	User has already voted on this proposal. API returns HTTP 422 with error `DisputeAlreadyReviewedException`.
A2	No proposal currently active for review. API returns HTTP 422 with error `DisputeNoProposalAwaitingReviewException`.
A3	Caller is not a participant. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`dispute`	Updated dispute record with party vote flags and resulting status (`RESOLVED` or escalated).

Post-Conditions

- Vote persisted in database.
- If mutually accepted, escrow settlement executed automatically.

3.18.6 Propose Dispute Resolution

Function Trigger

A System Administrator clicks "Propose Resolution" or "Issue Final Decision" in the Admin Dispute Arbitration Console.
Navigation path: Admin Dashboard → Arbitration → Dispute Case (`/admin/disputes/:id`) → Click "Propose Resolution".

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Allows an Administrator to adjudicate evidence, determine a fair fund distribution split (Client refund vs Freelancer transfer) matching the milestone escrow balance, write legal arbitration findings, and present the proposal for party consensus (or issue a final binding ruling).

Interface

- Admin dispute arbitration workspace at `/admin/disputes/:id`.
- Case review dossier: comparative view of claim statements, chat archives, deliverable files, and activity logs.
- Resolution builder form: Client refund amount input ($USD), Freelancer payout amount input ($USD), and automatic escrow total parity validator.
- Arbitration findings and legal rationale rich-text area.
- Action buttons: "Propose Resolution to Parties" and "Issue Final Binding Ruling".

Data Processing Overview

- Administrator submits an HTTP request: `POST /api/admin/disputes/:id/decision` with `clientRefundAmount`, `freelancerTransferAmount`, and `rationale`.
- `DisputeService.adminMakeDecision()` verifies caller possesses `ADMIN` role.
- Validates arithmetic parity: verifies that `clientRefundAmount + freelancerTransferAmount === milestone.amount`.
- Inside a database transaction, persists the proposed split amounts, sets `status = UNDER_REVIEW`, resets party vote flags to `PENDING`, and stores the findings rationale.
- Initializes a 48-hour party review timer.
- Dispatches alert notifications to both Client and Freelancer inviting them to review the proposed resolution.

Screen Layout (Mock-up Description)

Figure 157: Admin propose dispute resolution console

- Header displays dispute case reference, total escrow balance in dispute, and parties' identity cards.
- Fund allocation interactive slider and numeric inputs for Client Refund ($USD) and Freelancer Payout ($USD) with green parity checkmark.
- Rationale editor box for entering administrative findings.
- Action buttons: "Send Proposal for Consensus Review" and "Execute Final Determination".

Function Details

Input Data

Data Name	Description
`id`	Numeric dispute identifier (integer path parameter).
`clientRefundAmount`	Monetary amount in USD to refund to Client (decimal, $\geq$ 0).
`freelancerTransferAmount`	Monetary amount in USD to transfer to Freelancer (decimal, $\geq$ 0).
`rationale`	Administrative arbitration findings explaining the decision (string, 20–3000 characters).

Validation Rules

- Caller must possess verified `ADMIN` role.
- Dispute must be in active arbitration status (`EVIDENCE_SUBMITTED` or `UNDER_REVIEW`).
- The sum of `clientRefundAmount` and `freelancerTransferAmount` must exactly match the total milestone escrow balance.
- Rationale must be at least 20 characters in length.

Business Rules

- **BR-DIS-12**: Only verified Administrators are authorized to propose or execute dispute resolutions.
- **BR-DIS-13**: The sum of proposed resolution splits shall strictly equal 100% of the disputed milestone escrow amount.
- **BR-DIS-14**: Admin resolution proposals initiate a 48-hour consensus window before final binding determination.

Normal Flow

1. Administrator accesses dispute case in Admin Arbitration Console.
2. Admin examines claim statements, evidence files, and deliverable submissions.
3. Admin inputs fund split amounts and types arbitration findings rationale.
4. Admin clicks “Send Proposal for Consensus Review”.
5. System validates amount parity and records proposal split in database.
6. 48-hour consensus timer begins, and notifications are sent to Client and Freelancer.

Abnormal Flow / Exception Handling

Case	Description
A1	Split amounts do not sum to escrow balance. API returns HTTP 400 Bad Request with error `DisputeSettlementInvalidAmountException`; form indicates mismatch.
A2	Caller does not possess Admin role. API returns HTTP 403 Forbidden; access denied.
A3	Dispute is already resolved. API returns HTTP 422; adjudication blocked.

Output Data

Data Name	Description
`dispute`	Updated dispute entity with proposed split amounts, findings rationale, and consensus timers.

Post-Conditions

- Resolution proposal persisted in database.
- Review voting opened for both parties.
- Notifications dispatched to Client and Freelancer.

3.19 Proposal Management

3.19.1 Function: View My Proposal List

Function Trigger

A freelancer clicks "Proposals" in the workspace navigation menu.

Navigation path: Freelancer Dashboard → Proposals (`/freelancer/proposals`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows freelancers to review and monitor all proposals they have submitted across different lifecycle stages (`SUBMITTED`, `INTERVIEWING`, `HIRED`, `REJECTED`, `WITHDRAWN`, `EXPIRED`).

Interface

- Proposals dashboard at `/freelancer/proposals`.
- Tab filters: All, Active, Offers/Hired, Archived.
- Proposal cards displaying target job title, client name, proposed bid amount, delivery timeline, and status badge.

Data Processing Overview

- Client dispatches `GET /api/freelancer/proposals`.
- Backend extracts freelancer user ID from JWT session.
- Queries `Proposal` table joined with `Job` and client profiles.
- Returns sorted results by submission date with pagination.

Screen Layout (Mock-up Description)

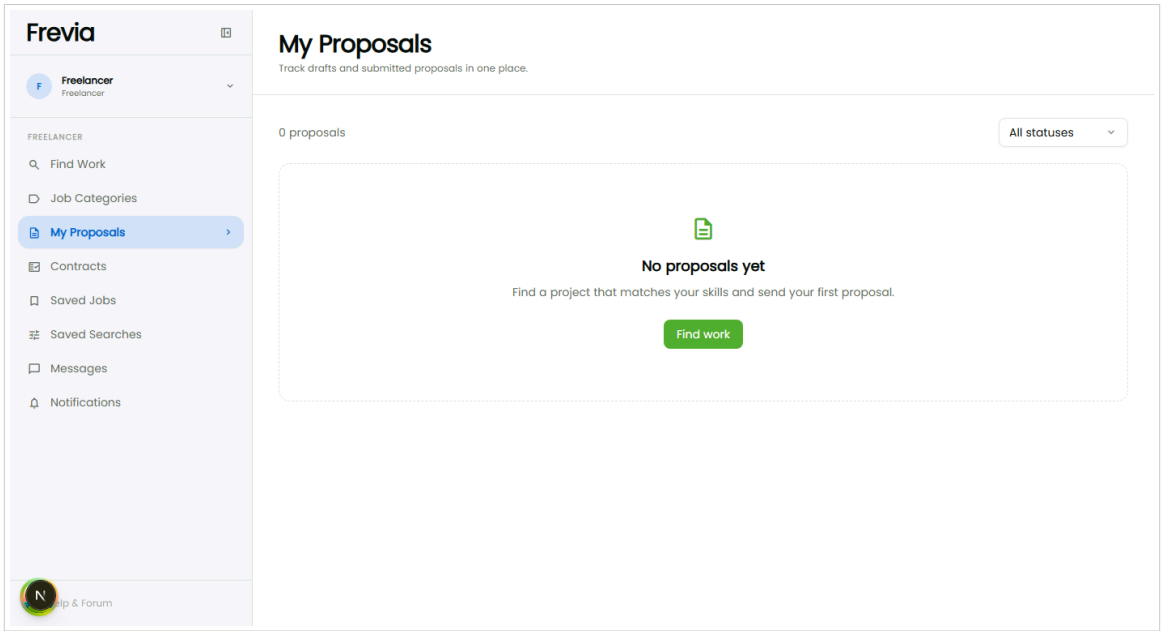

Figure 158: Submitted proposals list interface

- Filter tabs separating active submissions from historical decisions.
- Proposal cards displaying job title, proposed bid, net earnings after fee, and status tag.
- Action menu for inspecting details, editing pending bids, or withdrawing applications.

Function Details

Input Data

Data Name	Description
JWT Token	Authenticated freelancer token.
`status`	Optional proposal status filter.
`page` / `limit`	Pagination integers.

Validation Rules

- Caller must have `FREELANCER` role.

Business Rules

- **BR-PR-01**: Freelancers may only view proposals submitted from their own account.
- **BR-PR-02**: Statuses reflect exact state machine: `SUBMITTED`, `INTERVIEWING`, `HIRED`, `REJECTED`, `WITHDRAWN`, `EXPIRED`.

Normal Flow

1. Freelancer visits `/freelancer/proposals`.
2. Frontend requests proposals from backend.
3. Backend retrieves user's proposals and associated job summaries.
4. UI renders cards with active status tags.

Abnormal Flow / Exception Handling

Case	Description
A1	No proposals submitted yet. UI displays empty state with a "Find Work" shortcut button.

Output Data

Data Name	Description
`proposals`	Array of proposal objects (`id`, `jobTitle`, `bidAmount`, `deliveryDays`, `status`, `submittedAt`).

Post-Conditions

- Freelancer views list and status of submitted bids.

3.19.2 Function: View My Proposal Detail

Function Trigger

A freelancer clicks on a proposal card from their proposals list.

Navigation path: Proposals List → Click Proposal (`/freelancer/proposals/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Displays full details of the freelancer's proposal, including cover letter, bid amount, fee deduction breakdown, milestone plans, and client response status.

Interface

- Proposal detail screen with two-column layout.
- Left panel displaying cover letter, answers to screening questions, and attachments.
- Right sidebar displaying fee breakdown, timeline estimate, and proposal management actions.

Data Processing Overview

- Client dispatches `GET /api/freelancer/proposals/:id`.
- Backend confirms that the proposal was created by the calling user.
- Returns full proposal object with linked job and client information.

Screen Layout (Mock-up Description)

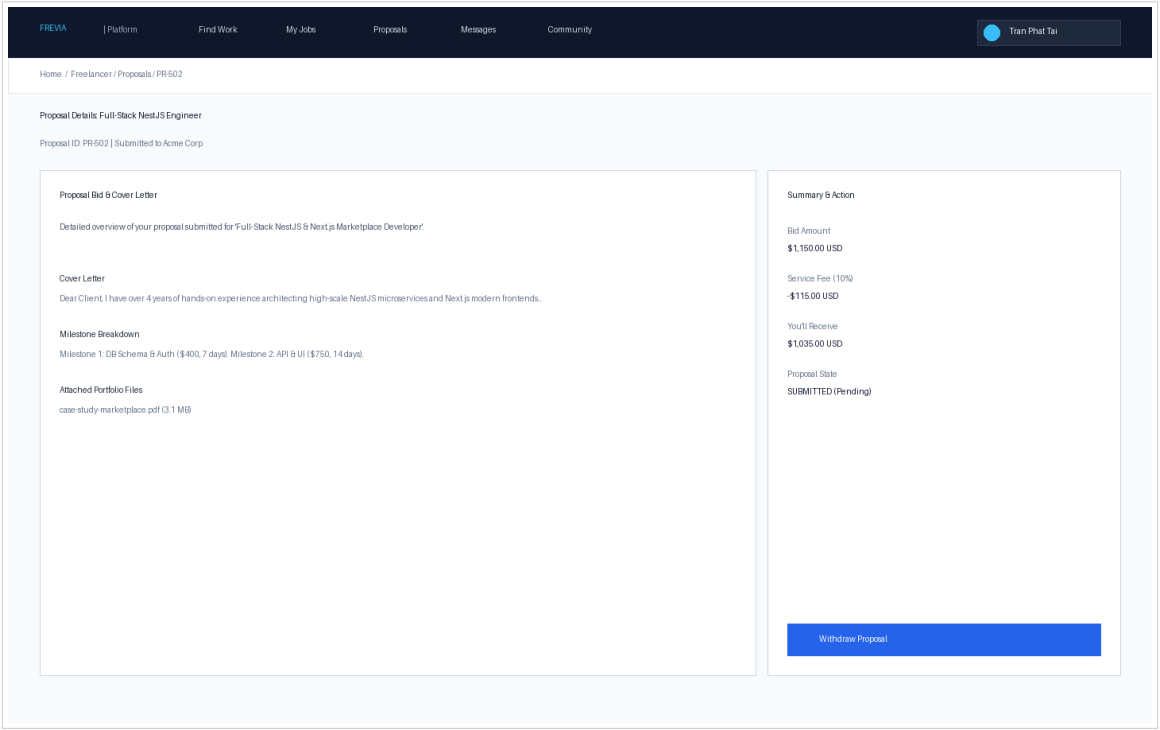

Figure 159: My proposal detail interface

- Detailed view showing cover letter text, milestone breakdown, and uploaded portfolio files.
- Financial calculation sidebar: Gross Bid, Platform Service Fee (10%), and Net Receivable.
- Action buttons: “Edit Proposal” and “Withdraw Proposal” (active when status is SUBMITTED).

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal.

Validation Rules

- Proposal must belong to authenticated freelancer.

Business Rules

- **BR-PR-03**: Non-owners cannot view private proposal details (HTTP 403 / 404).
- **BR-PR-04**: Displays full 10% Frevia service fee deduction calculation.

Normal Flow

1. Freelancer opens proposal detail.
2. Backend validates ownership and returns proposal entity.
3. UI renders proposal text, fee summary, and status timeline.

Abnormal Flow / Exception Handling

Case	Description
A1	Proposal not found or belongs to another user. Returns HTTP 404.

Output Data

Data Name	Description
Proposal Object	Detailed entity (`id`, `coverLetter`, `bidAmount`, `deliveryDays`, `attachments`, `status`, `job`).

Post-Conditions

- Complete proposal terms and current decision state are displayed.

3.19.3 Function: Submit Proposal

Function Trigger

A freelancer clicks "Submit a Proposal" on a job detail page.

Navigation path: Job Detail (`/jobs/:slug`) → Click "Submit a Proposal" (`/jobs/:slug/apply`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a freelancer to apply for an open job by providing a bid price, delivery timeframe, comprehensive cover letter, and optional supporting work samples.

Interface

- Proposal submission page at `/jobs/:slug/apply`.
- Bid terms calculator: inputs for gross bid with automatic calculation of the 10% platform fee and net earnings.
- Rich text cover letter area and file uploader (up to 3 files, 10MB each).

Data Processing Overview

- Client submits payload to `POST /api/freelancer/proposals`.
- Backend verifies that: (1) user has `FREELANCER` role, (2) target job is `OPEN`, (3) user is not the job's client, (4) user has not already submitted an active proposal for this job.
- Creates `Proposal` record in `SUBMITTED` status and increments job's proposal count.

Screen Layout (Mock-up Description)

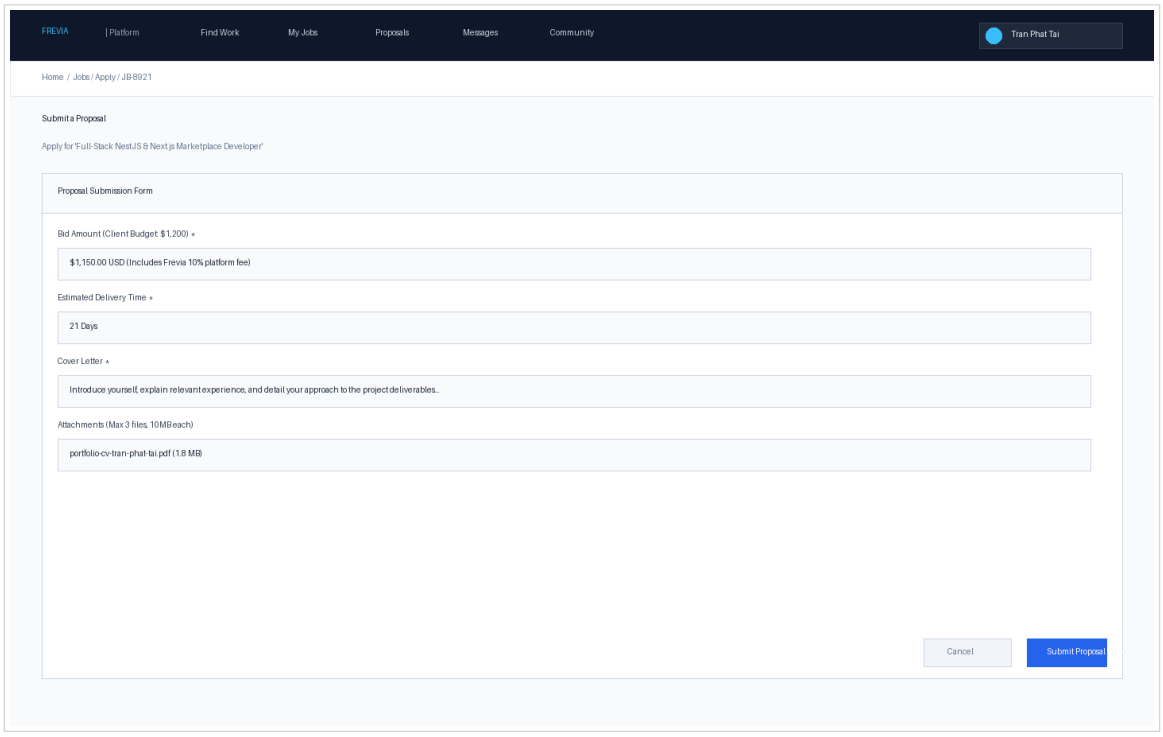

Figure 160: Submit proposal form interface

- Job summary header with client budget reference.
- Interactive bid calculator displaying gross amount, platform service fee, and net earnings.
- Textarea for cover letter and attachment uploader.
- Primary "Submit Proposal Now" button.

Function Details

Input Data

Data Name	Description
`jobId`	UUID of the target job posting.
`bidAmount`	Number, gross proposed compensation in USD.
`deliveryDays`	Integer ≥ 1, estimated completion days.
`coverLetter`	String (50–3000 chars), proposal text.
`attachments`	Optional array of uploaded file URLs.

Validation Rules

- `bidAmount` must be > 0.
- `deliveryDays` must be an integer between 1 and 365.
- `coverLetter` must contain at least 50 characters.

Business Rules

- **BR-PR-05**: Proposals may only be submitted to jobs with status `OPEN`.
- **BR-PR-06**: Clients cannot submit proposals to their own posted jobs.
- **BR-PR-07**: Freelancers may submit only 1 active proposal per job.
- **BR-PR-08**: Automatic fee calculation deducts standard 10% marketplace fee.

Normal Flow

1. Freelancer completes application terms and cover letter.
2. Clicks “Submit Proposal Now”.
3. Backend validates eligibility and persists proposal as `SUBMITTED`.
4. UI redirects to proposal detail view with a success confirmation toast.

Abnormal Flow / Exception Handling

Case	Description
A1	Freelancer already submitted a proposal for this job. Backend returns HTTP 400 Bad Request with message “You have already applied for this job.”

Case	Description
A2	Target job has closed while application was being drafted. Returns HTTP 400 with "Job is no longer open for proposals."

Output Data

Data Name	Description
Created Proposal	New entity with `id`, `status = SUBMITTED`, and submission timestamp.

Post-Conditions

- Proposal is submitted and visible in client's job proposals list.

3.19.4 Function: Update Proposal

Function Trigger

A freelancer clicks "Edit Proposal" on a pending application.

Navigation path: Proposal Detail (`/freelancer/proposals/:id`) → Click "Edit Proposal".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a freelancer to revise their proposed bid, delivery timeframe, or cover letter while the proposal is still in `SUBMITTED` status and has not yet been accepted or rejected.

Interface

- Pre-filled proposal editing form.

Data Processing Overview

- Client submits `PATCH /api/freelancer/proposals/:id`.
- Backend validates ownership and confirms that `status = 'SUBMITTED'`.

- Updates bid values and cover letter.

Screen Layout (Mock-up Description)

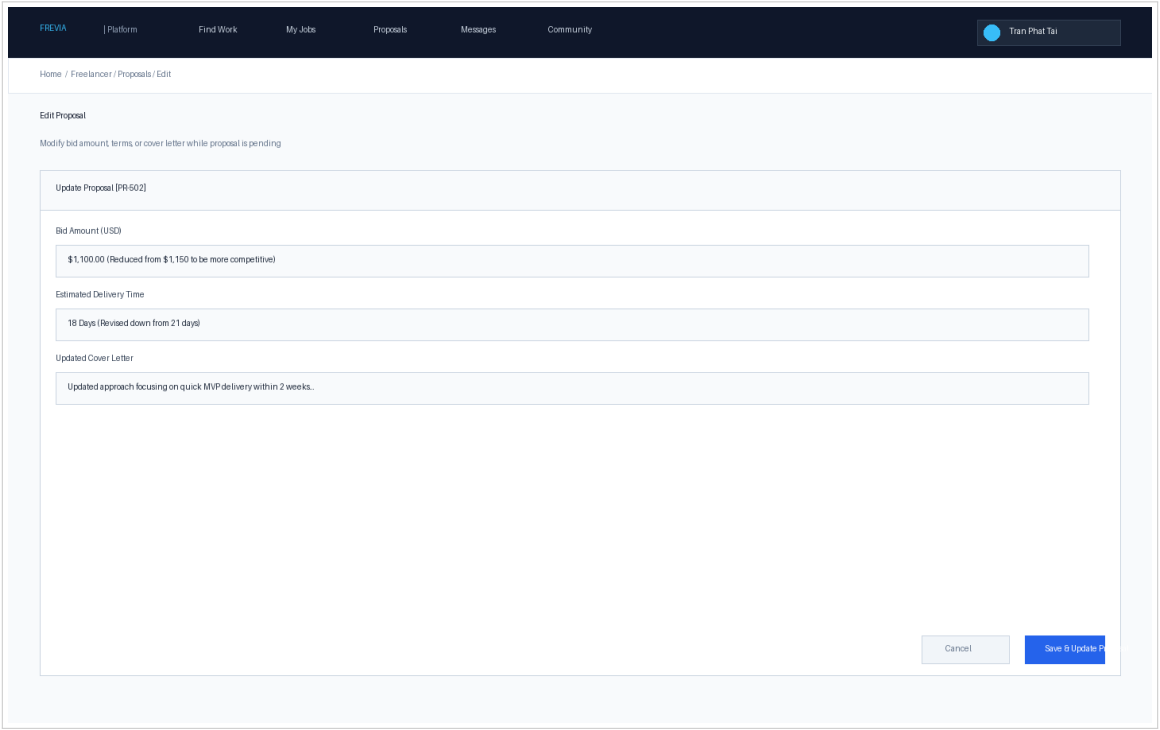

Figure 161: Edit proposal interface

- Form modal with editable bid amount, timeline, and cover letter text.
- Primary “Save & Update Proposal” action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal.
Payload	Modified fields (`bidAmount`, `deliveryDays`, `coverLetter`).

Validation Rules

- Proposal status must be `SUBMITTED`.
- Inputs must satisfy standard proposal validation constraints.

Business Rules

- **BR-PR-09**: Proposals can only be edited while in `SUBMITTED` status.

- **BR-PR-10**: Proposals in `HIRED`, `REJECTED`, or `WITHDRAWN` states cannot be edited.

Normal Flow

1. Freelancer opens edit dialog.
2. Modifies bid parameters or text.
3. Clicks "Save & Update Proposal".
4. Backend updates record and confirms.

Abnormal Flow / Exception Handling

Case	Description
A1	Proposal has already been accepted or rejected by client. Backend returns HTTP 409 Conflict with message "Proposal cannot be edited once reviewed."

Output Data

Data Name	Description
Updated Proposal	Proposal entity reflecting modified values.

Post-Conditions

- Revisions are committed and visible to the hiring client.

3.19.5 Function: Withdraw Proposal

Function Trigger

A freelancer clicks "Withdraw Proposal" on a submitted bid.

Navigation path: Proposal Detail (`/freelancer/proposals/:id`) → Click "Withdraw Proposal".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a freelancer to retract their application from consideration before the client hires or rejects them.

Interface

- Withdraw action button triggering confirmation dialog.
- Optional withdrawal reason selector.

Data Processing Overview

- Client invokes `POST /api/freelancer/proposals/:id/withdraw`.
- Backend verifies that proposal status is `SUBMITTED` or `INTERVIEWING`.
- Sets `status = 'WITHDRAWN'` and notifies client.

Screen Layout (Mock-up Description)

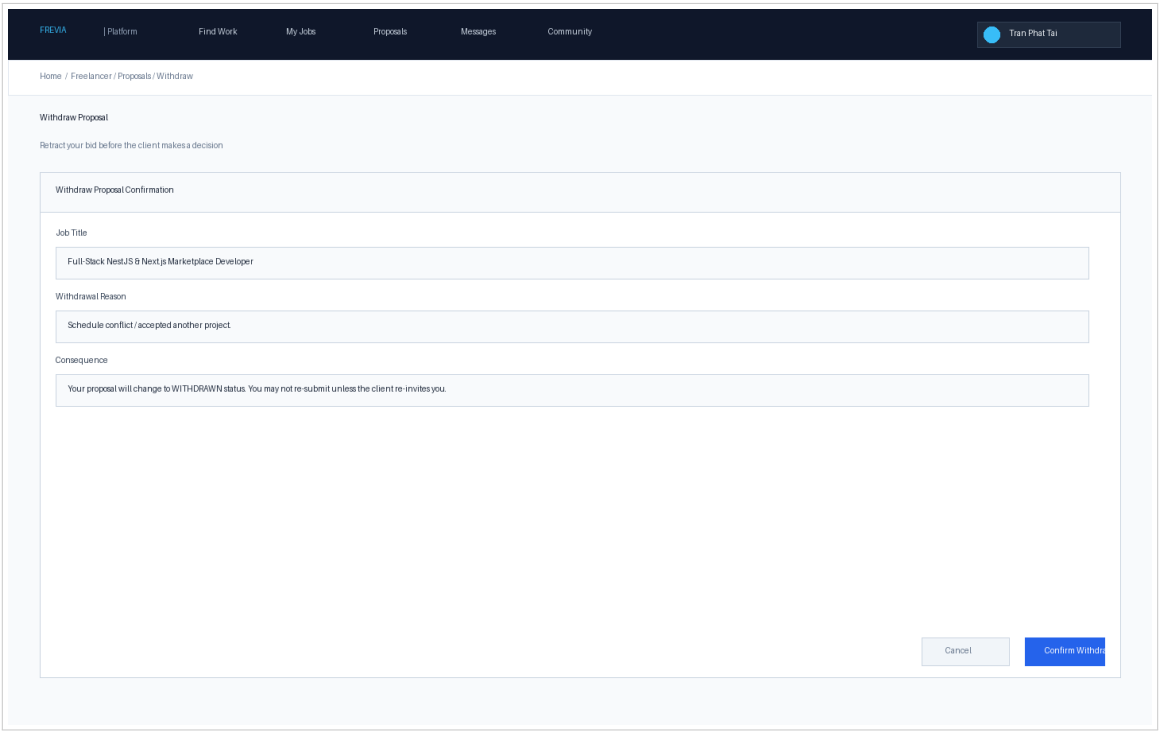

Figure 162: Withdraw proposal dialog interface

- Confirmation modal explaining consequences of withdrawal.
- Reason dropdown and optional remarks field.
- “Confirm Withdrawal” action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal to withdraw.
`reason`	Optional string explaining withdrawal reason.

Validation Rules

- Target proposal must be in `SUBMITTED` or `INTERVIEWING` state.

Business Rules

- **BR-PR-11**: A proposal cannot be withdrawn once accepted (`HIRED`).
- **BR-PR-12**: Withdrawing transitions state to `WITHDRAWN`; freelancer cannot re-submit without invitation.

Normal Flow

1. Freelancer clicks “Withdraw Proposal”.
2. Selects reason and confirms in modal.
3. Backend sets status to `WITHDRAWN`.
4. UI updates status and notifies client.

Abnormal Flow / Exception Handling

Case	Description
A1	Proposal already accepted. Backend returns HTTP 400 Bad Request.

Output Data

Data Name	Description
Result	Status confirmation confirming proposal withdrawal.

Post-Conditions

- Proposal status is `WITHDRAWN` and excluded from active candidate list.

3.19.6 Function: Accept Proposal

Function Trigger

A client clicks "Accept Proposal" or "Hire" on an applicant's bid.

Navigation path: Client Dashboard → Job Proposals → Proposal Detail → Click "Accept Proposal".

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Secondary Actor:** Payment/Escrow Gateway.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a client to accept a freelancer's proposal, automatically initialize and activate a binding contract (`Contract`), increment the filled positions quota, and transition the job status if fully hired.

Interface

- Hiring confirmation and contract initialization screen.
- Terms summary card detailing payment value, escrow deposit requirement, and project milestones.

Data Processing Overview

- Client submits `POST /api/client/proposals/:id/accept`.
- Backend verifies in a database transaction: (1) client owns the job, (2) proposal is `SUBMITTED` or `INTERVIEWING`, (3) job still has open hiring slots (`positionsFilled < positionsRequired`).

- Updates proposal to `status = 'HIRED'`, creates a new `Contract` entity with terms matching the bid, and increments `positionsFilled`.
- If `positionsFilled === positionsRequired`, transitions job from `OPEN` to `IN_PROGRESS`.

Screen Layout (Mock-up Description)

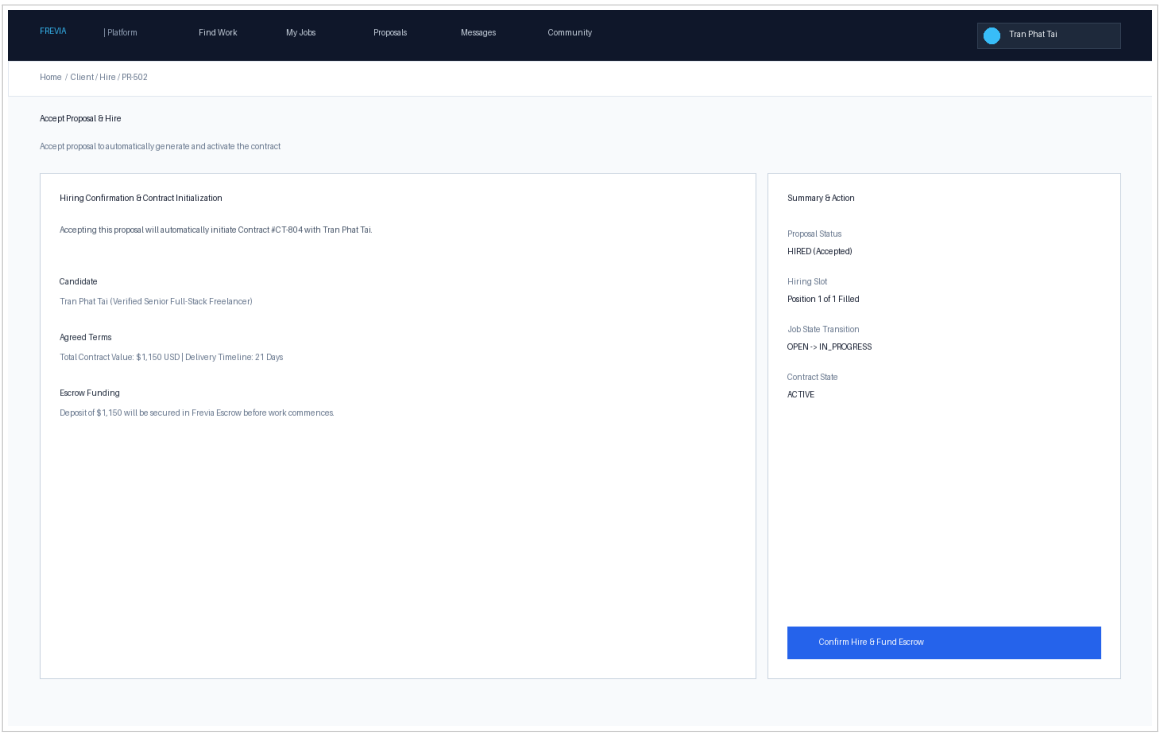

Figure 163: Accept proposal and hire confirmation

- Overview card detailing freelancer credentials, bid price, and delivery duration.
- Contract terms preview and escrow funding notice.
- Primary “Confirm Hire & Fund Escrow” action button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal being accepted.

Validation Rules

- Calling client must be the creator of the job posting.
- Job must have at least one open hiring position available.

Business Rules

- **BR-PR-13**: Accepting a proposal transitions its status to `HIRED` (Accepted).
- **BR-PR-14**: Automatically generates a formal binding contract with terms matching the proposal.
- **BR-PR-15**: Increments `positionsFilled`; when quota is full, transitions job to `IN_PROGRESS`.

Normal Flow

1. Client clicks "Accept Proposal".
2. Reviews terms and escrow details.
3. Confirms hiring decision.
4. Backend executes atomic transaction: updates proposal, initializes contract, and adjusts job status.
5. UI redirects to active contract view.

Abnormal Flow / Exception Handling

Case	Description
A1	Job hiring quota already filled. Backend aborts transaction and returns HTTP 409 Conflict.

Output Data

Data Name	Description
Created Contract	Contract entity linked to client, freelancer, and originating job.

Post-Conditions

- Proposal status is HIRED and active contract is initialized.

3.19.7 Function: Reject Proposal

Function Trigger

A client clicks “Reject” or “Decline” on an applicant’s proposal.

Navigation path: Job Proposals → Proposal Actions → Click “Reject Proposal”.

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Allows a client to decline an unsuitable proposal and respectfully inform the applicant.

Interface

- Reject proposal modal with optional decline reason field.

Data Processing Overview

- Client submits `POST /api/client/proposals/:id/reject`.
- Backend confirms job ownership.
- Updates proposal to `status = 'REJECTED'` and notifies freelancer.

Screen Layout (Mock-up Description)

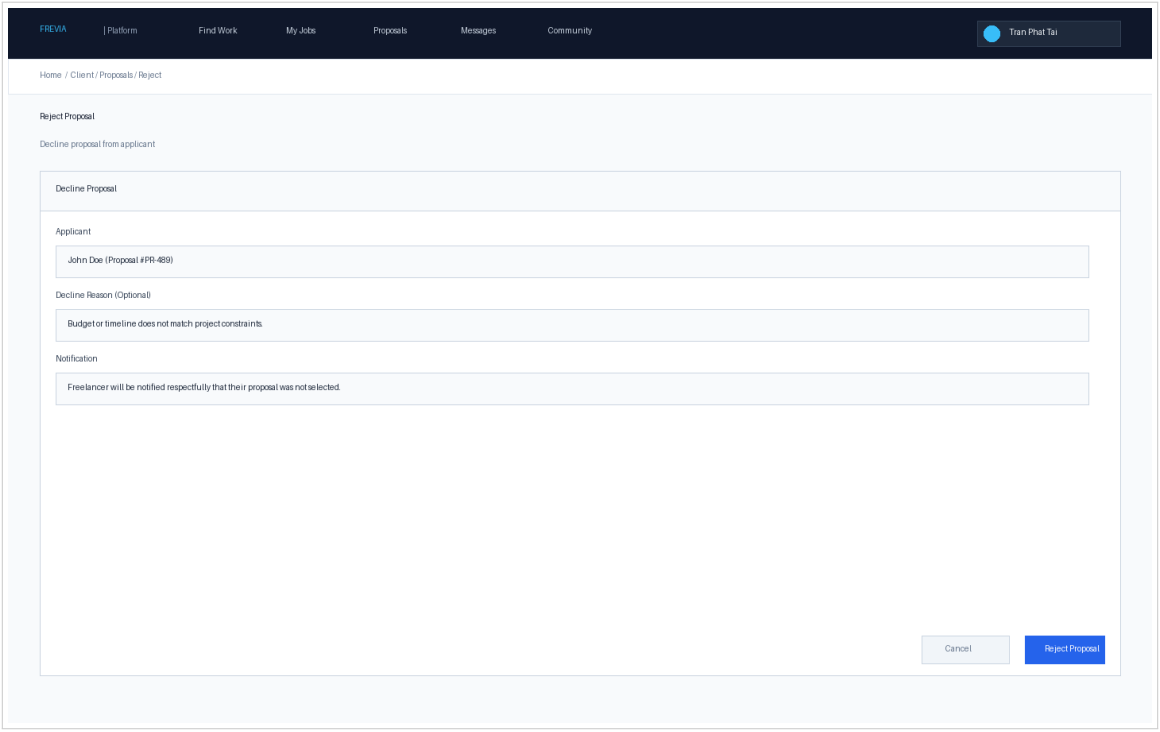

Figure 164: Reject proposal modal interface

- Modal dialog with reason dropdown (e.g. Budget mismatch, Timeline, Skill mismatch).
- Rejection confirmation button.

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal.
`reason`	Optional string explaining rejection.

Validation Rules

- Proposal must not already be in `HIRED` state.

Business Rules

- **BR-PR-16**: Rejection is irreversible; proposal status becomes `REJECTED`.

Normal Flow

1. Client selects “Reject”.

2. Chooses optional reason and confirms.
3. Backend updates status to `REJECTED`.
4. UI updates proposal card to archived state.

Abnormal Flow / Exception Handling

Case	Description
A1	Proposal already accepted. Returns HTTP 400 Bad Request.

Output Data

Data Name	Description
Result	Status confirmation confirming rejection.

Post-Conditions

- Proposal is marked REJECTED and applicant is notified.

3.19.8 Function: View Job Proposal List

Function Trigger

A client clicks "View Proposals" on one of their posted jobs.

Navigation path: Client Dashboard → My Jobs → Job Row → Click "Proposals" (`/client/jobs/:id/proposals`).

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Displays all bids received for a specific job posting, including applicant profiles, ratings, proposed prices, and timeline estimates.

Interface

- Candidate pipeline review table at `/client/jobs/:id/proposals`.

- Sorting options: Bid Amount, Delivery Time, Freelancer Rating.
- Quick action buttons: Chat, Review Proposal, Accept, Reject.

Data Processing Overview

- Client issues `GET /api/client/jobs/:id/proposals`.
- Backend verifies client owns the job.
- Retrieves all proposals submitted for this job with candidate statistics.

Screen Layout (Mock-up Description)

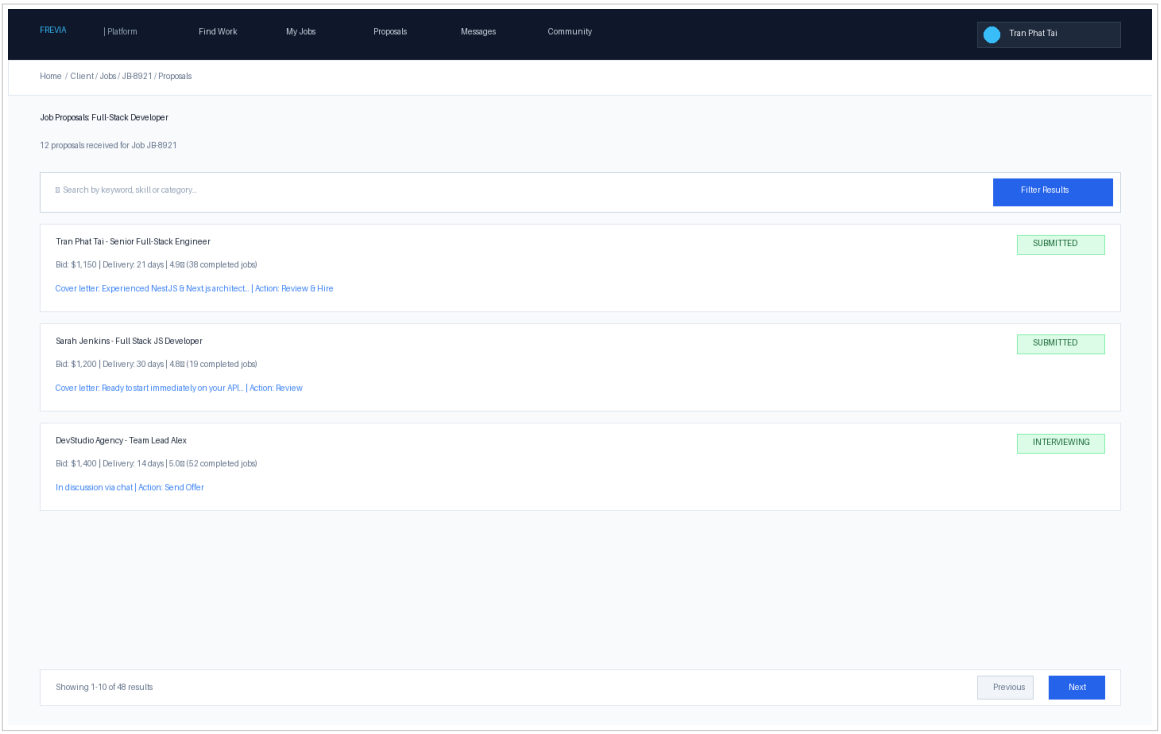

Figure 165: Client job proposals pipeline interface

- Job header showing total proposals count and current hiring status.
- Candidate cards displaying avatar, name, rating, bid value, and cover letter excerpt.
- Action toolbar to start conversation or initiate hiring.

Function Details

Input Data

Data Name	Description
`id`	UUID of the job posting.

Validation Rules

- Calling user must own the job posting.

Business Rules

- **BR-PR-17**: Only the job owner (or Admin) can inspect incoming proposals.
- **BR-PR-18**: Excludes proposals withdrawn by freelancers.

Normal Flow

1. Client opens proposals view for a job.
2. Backend verifies ownership and returns candidate bids.
3. UI renders candidate pipeline with action triggers.

Abnormal Flow / Exception Handling

Case	Description
A1	Non-owner accesses proposals. Backend returns HTTP 403 Forbidden.

Output Data

Data Name	Description
`proposals`	Array of proposal entities with populated freelancer profiles.

Post-Conditions

- Client inspects all candidate bids for the specified job.

3.19.9 Function: View Proposal Detail

Function Trigger

A client clicks on an applicant's proposal card.

Navigation path: Job Proposals → Click Candidate Row (`/client/proposals/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Client.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Gives clients an in-depth view of a specific candidate's application, including full cover letter, qualifications match percentage, portfolio attachments, and direct hiring controls.

Interface

- In-depth application review drawer or screen at `/client/proposals/:id`.
- Action toolbar with "Message Freelancer", "Accept Proposal", and "Reject".

Data Processing Overview

- Client requests `GET /api/client/proposals/:id`.
- Backend confirms that client is the author of the linked job.
- Returns full proposal payload, candidate profile data, and attachment links.

Screen Layout (Mock-up Description)

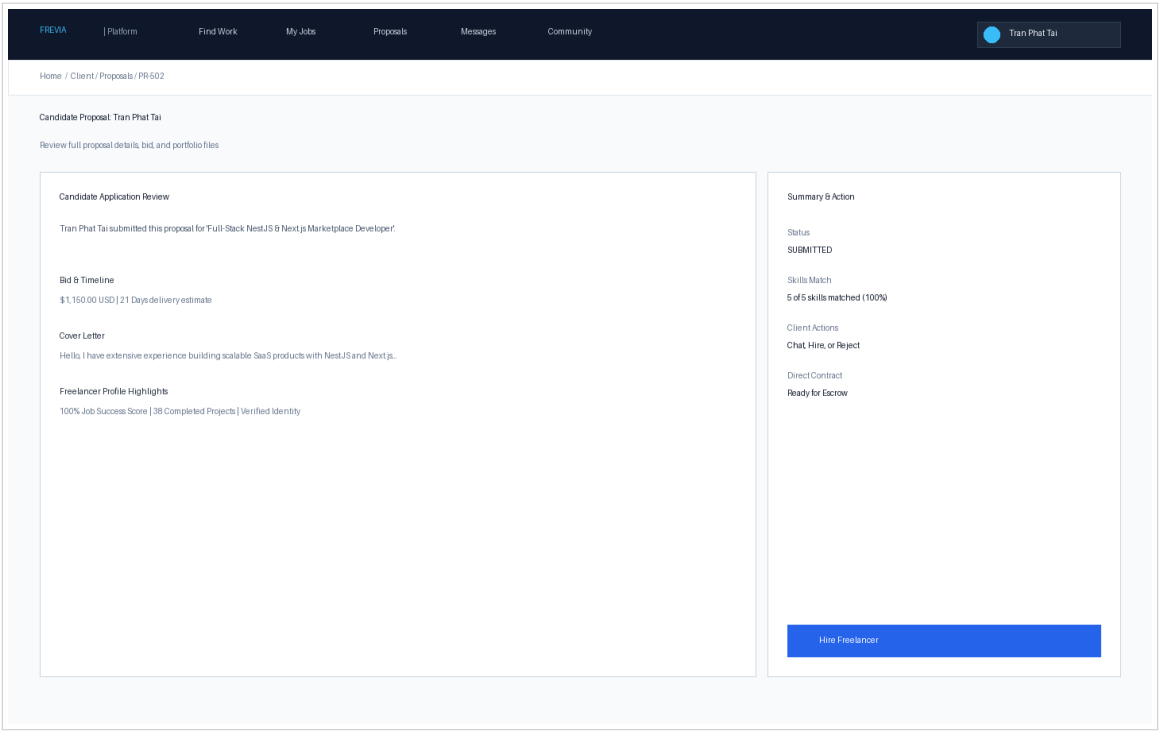

Figure 166: Client candidate proposal review interface

- Candidate overview with rating, earnings, and skills match badge.
- Full text of the applicant's cover letter and downloadable attachments.
- Prominent "Hire Freelancer" and "Decline" decision buttons.

Function Details

Input Data

Data Name	Description
`id`	UUID of the proposal.

Validation Rules

- Client must be the author of the job associated with this proposal.

Business Rules

- **BR-PR-19**: Protects candidate proposal privacy against third-party unauthorized viewers.

Normal Flow

1. Client selects a candidate proposal.

2. Backend returns comprehensive application data.
3. UI renders full application and decision action buttons.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthorized access attempt. Returns HTTP 403 Forbidden.

Output Data

Data Name	Description
Proposal Details	Complete proposal entity with applicant profile and work samples.

Post-Conditions

- Client reviews candidate proposal and chooses to hire, chat, or decline.

3.20 Banner Management

3.20.1 View Banner List

Function Trigger

An administrator navigates to the Banners management page from the admin dashboard sidebar navigation.
Navigation path: Admin Dashboard → Marketing & Banners → Banners (`/admin/banners`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view, search, filter, and sort all promotional and advertisement banners registered in the system. The catalog displays essential banner attributes including ID, title, target position slot, publication schedule, active status, thumbnail preview, and destination URL.

Interface

- Banner management screen at `/admin/banners`.
- Filter bar component (`BannersFilterBar`) containing search input, position slot dropdown, sort column selector, and sort direction toggle.
- Data table component (`BannersTable`) rendering banner rows, thumbnail image previews, position badges, active status toggles, schedule ranges, and contextual action menus.
- Standard pagination footer displaying total banners and page navigation controls.
- "Create Banner" action button in the page header.

Data Processing Overview

- The client initiates an HTTP request: `GET /api/admin/banners` with query parameters (`page`, `limit`, `search`, `position`, `sortBy`, `sortOrder`).
- The backend `AuthGuard` validates the administrator's JWT access token, and `PermissionGuard` verifies administrative route authorization.
- `BannersAdminController.listBanners()` parses query parameters via `BannerAdminQueryDto`.
- `BannersAdminRepository.listBanners()` executes `prisma.advertisementBanner.findMany` with:
  - Substring search on `title` using case-insensitive mode if `search` is provided.
  - Exact filter on `position` if a specific position slot is selected.
  - Sorting on the requested field (`id` or `createdAt`) with specified order (`asc` or `desc`).
- The repository executes `prisma.advertisementBanner.count()` in parallel to obtain total matching records.
- The system returns HTTP 200 with the banners array and pagination metadata.
- The frontend renders the data table, applies position badge styling, and displays schedule ranges.

Screen Layout (Mock-up Description)

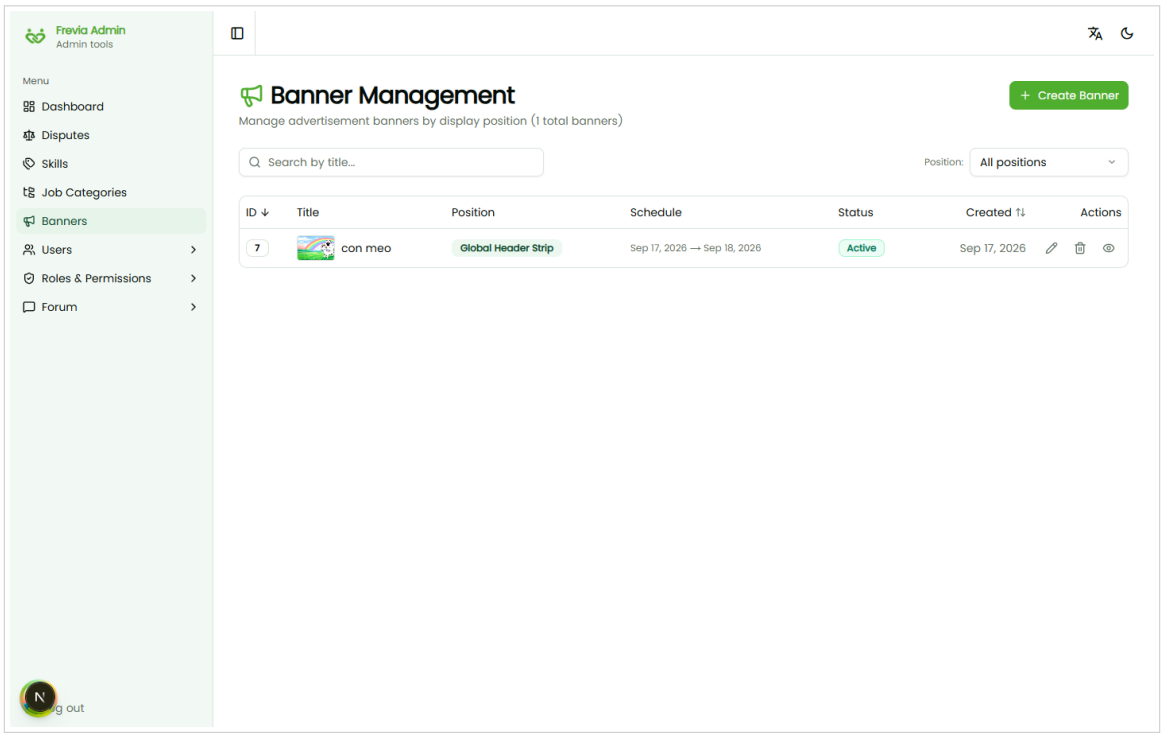

Figure 167: Advertisement banner list interface

- Page header displaying “Banners” title with megaphone icon, summary count, and “Create Banner” button.
- Filter bar including search box, position dropdown selector (All Positions, Global Header, Home Hero, Home Body, Search Results, Footer), and sort order controls.
- Table columns: **Banner** (Thumbnail, Title, and ID), **Position** (Colored badge), **Status** (Green “Active” or Gray “Inactive”), **Schedule** (Start date to End date), **Destination Link**, and **Actions**.
- Row-level action buttons providing quick access to view detail, edit, and delete banner.
- Pagination footer indicating total count and page navigation buttons.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum records returned per page (defaults to 10).
search	Optional text query matching banner title.
position	Optional position filter (`GLOBAL_HEADER`, `HOME_HERO`, `HOME_BODY`, `SEARCH_RESULTS`, `FOOTER`).
sortBy	Sort column field (`id` or `createdAt`; defaults to `id`).
sortOrder	Sort direction sequence (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters; whitespace trimmed.
position	Optional string, must strictly match one of the defined `BannerPositionEnum` values.
sortBy	Must strictly be one of: `id`, `createdAt`.
sortOrder	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-BAN-01**: Admin Permission Verification: Only authenticated users with role `ADMIN` and matching route permission can query the administrative banner catalog.
- **BR-BAN-02**: Search and Position Filter Matching: Text search performs case-insensitive substring matching on `title`, while position filtering performs exact equality matching.
- **BR-BAN-03**: Default Pagination and Ordering: Default page size is 10 records, ordered by `id` in descending order so newly created banners appear first.
- **BR-BAN-04**: Active and Schedule Range Representation: Displays visual badges for active/inactive state along with formatted publication date intervals (`startDate` to `endDate`).
- **BR-BAN-05**: Direct Detail and Modification Actions: Each table row provides direct triggers to view detail, open update form modal, or initiate hard deletion.

Normal Flow

1. The administrator opens the `/admin/banners` page from the admin dashboard.
2. The client issues `GET /api/admin/banners` with query parameters (`page=1&limit=10&sortBy=id&so`
3. The backend validates administrative authentication and route permissions.
4. The repository queries advertisement banners and counts total matching records.
5. The backend returns HTTP 200 with the banners array and pagination metadata.
6. The frontend renders the banner table and pagination controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client clears authentication cookies and redirects to `/login`.
User lacks Admin privileges	The backend throws HTTP 403 Forbidden. The UI displays: "You do not have permission to access this page."
Invalid pagination parameter values	The DTO validator rejects non-integer or negative values with HTTP 400 Bad Request. The client resets to page 1.
Invalid position filter parameter	The validator rejects unknown position strings with HTTP 400 Bad Request. The UI clears invalid position filter.
Unsupported sort column parameter	The validator rejects invalid sort keys with HTTP 400 Bad Request. The client defaults to sorting by `id desc`.

Abnorma Case	Handling & System Response
Database query error or timeout	The service catches the Prisma error and throws `FailedToLoadBannerListException` (HTTP 500). The UI displays: "Error.FailedToLoadBanners".
No banners match search or position filter	The query returns `banners: []` and `total: 0`. The table displays an empty placeholder: "No banners found."
Network connection failure	The client displays a connection failure toast notification and offers a "Try again" button.

Output Data

Output Field	Description
banners	Array of banner objects: `id`, `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`, `createdAt`, `updatedAt`.
pagination.page	Current page number.
pagination.limit	Current limit per page.
pagination.total	Total advertisement banners matching the query.
pagination.totalPages	Total calculated pages available.

Post-Conditions

The administrator views the paginated list of advertisement banners, inspects status and positions, and can initiate creation, inspection, modification, or deletion actions.

3.20.2 View Banner Detail

Function Trigger

An administrator clicks on a banner row or the "View detail" action on the banner list page, navigating to `/admin/banners/[id]`.
Navigation path: Admin Dashboard → Marketing & Banners → Banners → Banner Details (`/admin/banners/[id]`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view comprehensive attributes and publication configuration of a specific advertisement banner, including full-size image preview, destination link, target position slot, scheduling timeline, and creation/update timestamps.

Interface

- Banner detail screen at `/admin/banners/[id]`.
- Back navigation button ("Back to Banners").
- Header summary displaying banner title, position badge, ID, and an "Edit Banner" action trigger.
- Banner media preview card rendering the full banner image or placeholder if no image is uploaded.
- Configuration grid displaying position slot, active status badge, target destination URL with clickable link, start date, end date, and audit timestamps.

Data Processing Overview

- The client sends an HTTP request: `GET /api/admin/banners/:id`.
- The backend validates administrative token and route permissions.
- `BannersAdminController.getBannerDetail()` parses `id` via `ParseIntPipe`.
- `BannersAdminRepository.getBannerDetail(id)` queries `prisma.advertisementBanner.findUni` by primary key `id`.
- If the banner record is not found, the service throws `BannerAdminNotFoundException` (HTTP 404).
- If found, returns the complete banner entity with HTTP 200.

- The frontend renders the banner details layout, full image preview, and action controls.

Screen Layout (Mock-up Description)

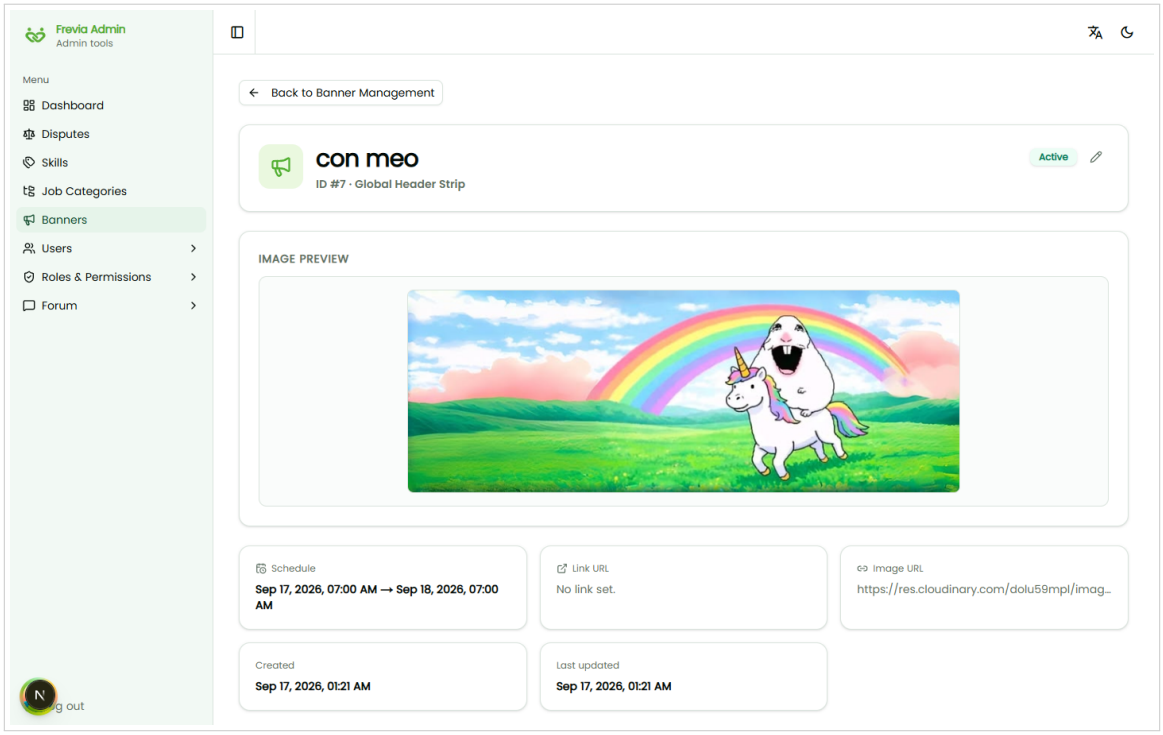

Figure 168: Banner detail interface

- Navigation header with "Back to Banners" button and banner title with megaphone icon.
- Action toolbar containing "Edit Banner" button triggering the edit form modal.
- Banner Image preview panel displaying uploaded advertisement graphic with aspect ratio preservation.
- Information cards: **Display Position** (e.g., Global Header), **Destination Link** (clickable external URL), **Active Status** (Green active badge), **Start Date**, **End Date**, **Created At**, and **Last Updated**.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the advertisement banner (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-BAN-06**: Numeric Identifier Validation: The URL parameter `id` must parse strictly as a valid positive integer.
- **BR-BAN-07**: Advertisement Banner Existence: The requested banner must exist in the database; returns HTTP 404 (`Error.BannerNotFound`) if missing.
- **BR-BAN-08**: Complete Attribute Presentation: Displays all banner configuration data, responsive image asset preview, and external target link.
- **BR-BAN-09**: Direct Modification Access: Provides an "Edit Banner" trigger pre-populating current banner attributes into the form dialog.

Normal Flow

1. The administrator navigates to `/admin/banners/[id]`.
2. The client issues `GET /api/admin/banners/:id`.
3. The backend validates administrative authentication and permissions.
4. The repository queries `prisma.advertisementBanner.findUnique( where: id )`.
5. The server returns HTTP 200 with the banner object.
6. The frontend renders the comprehensive banner detail page.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.
Invalid banner ID format (e.g., non-numeric)	`ParseIntPipe` throws HTTP 400 Bad Request. The UI displays an "Invalid Banner ID" message and back button.
Banner ID does not exist in database	The service throws `BannerAdminNotFoundException` (HTTP 404). The UI renders a "Banner Not Found" screen with navigation back to list.
Database query error or timeout	The service throws `FailedToLoadBannerDetailException` (HTTP 500). The UI displays: "Error.FailedToLoadBannerDetail".
Destination link is empty or null	The UI gracefully displays "No destination link configured" in muted text instead of a broken link.

Abnorma Case	Handling & System Response
Image asset URL broken or unreachable	The UI falls back to rendering a placeholder graphic container with a broken image indicator.

Output Data

Output Field	Description
id	Unique numeric identifier of the banner.
title	Banner title string.
imageUrl	Absolute image URL or relative path on server.
linkUrl	External or internal redirect target URL.
position	Display slot enum value.
startDate	Scheduled activation timestamp or null.
endDate	Scheduled deactivation timestamp or null.
isActive	Boolean indicating whether banner is enabled.
createdAt	Timestamp when banner was created.
updatedAt	Timestamp of last modification.

Post-Conditions

The administrator reviews complete banner configuration, image assets, schedule boundaries, and can initiate editing.

3.20.3 Create Banner

Function Trigger

An administrator clicks the "Create Banner" button on the banner list page (`/admin/banners`), opening the banner creation dialog modal.

Navigation path: Admin Dashboard → Marketing & Banners → Banners → [Create Banner Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Cloudinary media storage service, Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to register a new advertisement banner, specifying title, position slot, optional graphic image upload, target redirect link, publication date range, and initial active status.

Interface

- Banner form dialog modal (`BannerFormDialog`).
- Text input for Title (mandatory).
- Dropdown selector for Position slot (`GLOBAL_HEADER`, `HOME_HERO`, `HOME_BODY`, `SEARCH_RESULTS`, `FOOTER`).
- File upload input with preview for banner image (supports drag-and-drop or file browser, max 10MB).
- Text input for Destination Link URL.
- Date picker inputs for Start Date and End Date.
- Checkbox toggle for "Active" status (defaults to checked).
- "Cancel" and "Create Banner" submission buttons with loading indicator.

Data Processing Overview

- If a local image file is chosen, the client first issues a multipart/form-data request: `POST /api/admin/banners/upload-image`.
  - The service validates file presence, MIME type (`image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/bmp`, `image/tiff`), and file size ($\leq$ 10MB).
  - If Cloudinary is configured, uploads file to Cloudinary under folder `frevia/banners` and obtains `secure_url`; otherwise saves file to local directory `uploads/banners/`.
  - Returns `{ imageUrl, publicId }`.
- The client issues an HTTP request: `POST /api/admin/banners` with JSON payload conforming to `BannerCreateBodySchema`.
- `BannersAdminRepository.createBanner()` checks whether an active banner already occupies the selected position slot (`prisma.advertisementBanner.findFirst(`

`where: position )`).

- If position is already occupied, the service throws `BannerPositionTakenException` (HTTP 409).
- The repository inserts the new banner record via `prisma.advertisementBanner.create()`.
- The backend returns HTTP 201 with the created banner entity.
- The frontend shows a success toast notification, closes the dialog modal, and refreshes the banner list table.

Screen Layout (Mock-up Description)

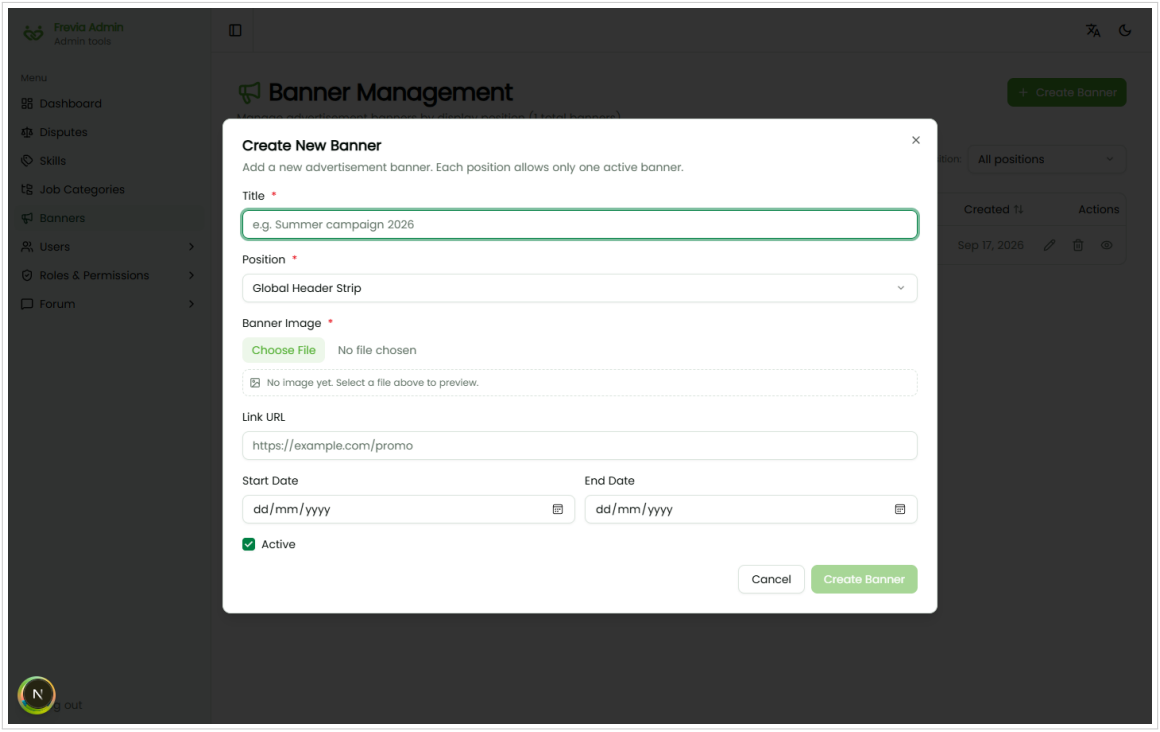

Figure 169: Create banner modal interface

- Dialog header with "Create Banner" title and brief descriptive subtitle.
- Form fields: **Title** (Required text input), **Position** (Select dropdown: Global Header, Home Hero, Home Body, Search Results, Footer), **Banner Image** (Upload zone with image preview and remove button), **Destination Link** (URL input), **Schedule** (Start Date and End Date date pickers), and **Active** (Checkbox toggle).
- Dialog footer with "Cancel" button and "Create Banner" submit button with loading spinner.

Function Details

Input Data

Data Name	Description
title	Banner title string (mandatory).
imageUrl	Direct image URL string or uploaded file URL (optional).
file	Binary image file for upload (optional, multipart/form-data).
linkUrl	Target destination URL string (optional).
position	Target display position slot enum (defaults to `GLOBAL_HEADER`).
startDate	Scheduled start publication date (optional).
endDate	Scheduled end publication date (optional).
isActive	Boolean activation flag (defaults to `true`).

Validation Rules

Data Name	Validation Rule
title	String, trimmed, min 1 char (`Error.BannerTitleRequired`), max 255 chars (`Error.BannerTitleTooLong`).
imageUrl	Optional string, must be valid URL format, max 500 chars (`Error.BannerImageUrlTooLong`).
file	Must be valid image MIME type; size must not exceed 10MB (`Error.BannerImageTooLarge`).
linkUrl	Optional string, must be valid URL format, max 500 chars (`Error.BannerLinkUrlTooLong`).
position	Must match one of `BannerPositionEnum` values (defaults to `GLOBAL_HEADER`).
startDate, endDate	Coerced to valid Date. Refinement: if both present, `startDate <= endDate` (`Error.BannerInvalidDateRange`).
isActive	Boolean value (defaults to `true`).

Business Rules

- **BR-BAN-10**: Single Active Banner Per Position: Each position slot can only be occupied by one banner; if the position is already taken, creation is rejected with HTTP 409 (`Error.BannerPositionTaken`).
- **BR-BAN-11**: Mandatory Title Trimming: Title is mandatory and leading/trailing whitespace is stripped before validation and database persistence.
- **BR-BAN-12**: Target and Image URL Validation: If provided, URLs must be strictly valid URI formats with a maximum length of 500 characters.
- **BR-BAN-13**: Valid Schedule Range: When both dates are specified, `startDate` must be earlier than or equal to `endDate`.
- **BR-BAN-14**: Image Upload Constraints: Image uploads are strictly limited to valid image MIME formats and maximum size of 10MB.
- **BR-BAN-15**: Auto-refresh and Feedback: Shows a success toast notification and refreshes the banner catalog upon successful creation.

Normal Flow

1. The administrator clicks "Create Banner" on the banner management page.
2. The dialog modal opens with an empty form and default position (`GLOBAL_HEADER`).
3. The administrator fills in the title, selects a position slot, optionally attaches an image file, enters a link URL, selects date bounds, and checks "Active".
4. If a file was attached, the client uploads it to `POST /api/admin/banners/upload-image` and receives the image URL.
5. The client submits the form data via `POST /api/admin/banners`.
6. The backend validates the payload and verifies the position slot is available.
7. The repository inserts the new banner record into the database.
8. The backend returns HTTP 201 with the created banner data.
9. The frontend closes the modal, displays a success toast, and refreshes the banner list.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing mandatory title	The form prevents submission; backend returns HTTP 400 Bad Request with `Error.BannerTitleRequired`.

Abnorma Case	Handling & System Response
Selected position already occupied	The service throws `BannerPositionTakenException` (HTTP 409 Conflict). The UI displays: “Error.BannerPositionTaken”.
Uploaded file exceeds 10MB	The upload endpoint throws `BannerImageTooLargeException` (HTTP 400). The UI displays: “Error.BannerImageTooLarge”.
Uploaded file is not a supported image format	The upload endpoint throws `BannerImageInvalidTypeException` (HTTP 400). The UI displays: “Error.BannerImageInvalidType”.
End date is earlier than start date	Zod refinement rejects payload with HTTP 400 Bad Request (`Error.BannerInvalidDateRange`). The UI flags the End Date field.
Invalid URL syntax in image or link URL	Validator returns HTTP 400 Bad Request. The UI highlights the offending URL input field.
Unauthorized or unauthenticated request	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.

Abnorma Case	Handling & System Response
Database insert failure	The service throws `FailedToCreateBannerException` (HTTP 500). The UI displays: “Error.FailedToCreateBanner”.

Output Data

Output Field	Description
id	Auto-generated unique identifier of the created banner.
title	Stored banner title string.
imageUrl	Image asset URL or null.
linkUrl	Destination URL or null.
position	Assigned position slot enum.
startDate	Activation date or null.
endDate	Deactivation date or null.
isActive	Active boolean status.
createdAt	Timestamp of creation.
updatedAt	Timestamp of last update.

Post-Conditions

A new advertisement banner is stored in the database, assigned to the selected position slot, and immediately rendered on the admin table.

3.20.4 Update Banner

Function Trigger

An administrator clicks the Edit icon in the banner table row or the “Edit Banner” button on the banner detail page, opening the edit dialog modal.

Navigation path: Admin Dashboard → Marketing & Banners → Banners → [Edit Action Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Cloudinary media storage service, Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to modify an existing advertisement banner's title, display position, image graphic, destination link, validity period, and active status.

Interface

- Banner form dialog modal (`BannerFormDialog`) pre-populated with existing banner data.
- Text input for Title.
- Select dropdown for Position slot.
- File upload input with current image preview and remove button.
- Text input for Destination Link URL.
- Date pickers for Start Date and End Date.
- Checkbox toggle for "Active" status.
- "Cancel" and "Save Changes" action buttons.

Data Processing Overview

- The client pre-fills the form with existing banner fields.
- If a new replacement image is selected, the client uploads it via `POST /api/admin/banners/upload-ima` before submitting changes.
- The client issues an HTTP request: `PATCH /api/admin/banners/:id` with updated attributes.
- `BannersAdminRepository.updateBanner(id, data)`:
  - Verifies target banner exists; throws `BannerAdminNotFoundException` if not found.
  - If `position` is changed, checks if another banner already occupies the new position slot (`id != currentId`). Throws `BannerPositionTakenException` if taken.
  - Updates modified fields in `prisma.advertisementBanner.update()`.
- The backend returns HTTP 200 with the updated banner entity.
- The frontend displays a success toast, closes the dialog, and refreshes the banner list or detail screen.

Screen Layout (Mock-up Description)

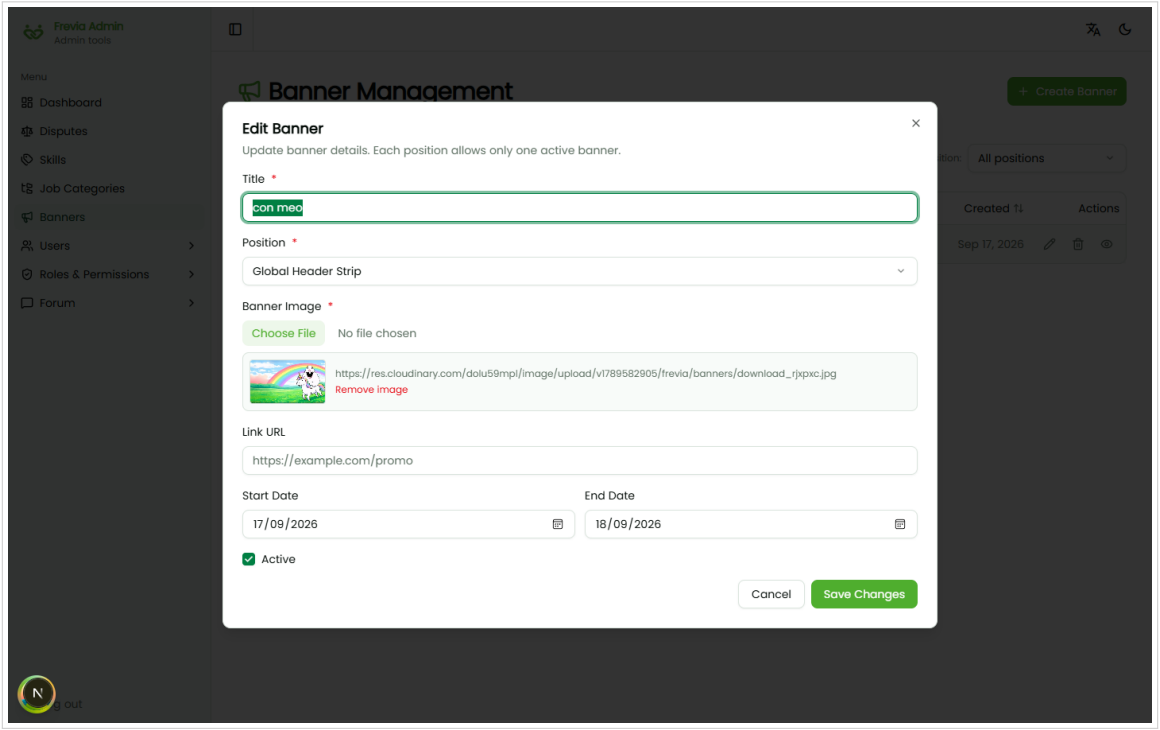

Figure 170: Update banner modal interface

- Modal dialog header with "Edit Banner" title.
- Form pre-populated with current values: **Title**, **Position**, **Banner Image** (with existing preview), **Destination Link**, **Schedule** (Start/End dates), and **Active** toggle.
- Footer buttons: "Cancel" and "Save Changes" button with pending spinner.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the banner (passed in URL path).
title	Updated banner title string (optional).
imageUrl	Updated image URL or null (optional).
file	Binary replacement image file (optional).
linkUrl	Updated destination URL or null (optional).
position	Updated position slot enum (optional).
startDate	Updated start publication date or null (optional).

Data Name	Description
endDate	Updated end publication date or null (optional).
isActive	Updated active boolean flag (optional).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.
title	Optional string, trimmed, min 1 char, max 255 chars (`Error.BannerTitleTooLong`).
imageUrl	Optional string, must be valid URL format, max 500 chars.
file	Must be valid image MIME type; size must not exceed 10MB.
linkUrl	Optional string, must be valid URL format, max 500 chars.
position	Must match one of `BannerPositionEnum` values.
startDate, endDate	Coerced to valid Date. Refinement: `startDate <= endDate` (`Error.BannerInvalidDateRange`).
isActive	Optional boolean value.

Business Rules

- **BR-BAN-16**: Target Banner Existence: The targeted banner ID must exist in the database; returns HTTP 404 (`Error.BannerNotFound`) if not found.
- **BR-BAN-17**: Position Conflict Protection on Update: If updating the `position` slot, the new position must not be occupied by another banner; excludes current banner ID.
- **BR-BAN-18**: Date Range Consistency: If both dates are present after update, `startDate` must be earlier than or equal to `endDate`.
- **BR-BAN-19**: Partial Update Preservation: Unchanged fields retain their current values in the database.
- **BR-BAN-20**: Cache Invalidation and UI Synchronization: Immediately invalidates cached queries and triggers view refresh upon successful update.

Normal Flow

1. The administrator clicks the Edit button on a banner row or detail view.
2. The dialog modal opens pre-filled with the banner's current data.

3. The administrator updates one or more fields (e.g., changes destination link or toggles active status).
4. The administrator clicks “Save Changes”.
5. If a replacement image was uploaded, it is uploaded to Cloudinary/local storage first.
6. The client sends `PATCH /api/admin/banners/:id` with updated attributes.
7. The backend validates data, checks for position conflicts, and updates the record.
8. The server returns HTTP 200 with the updated banner object.
9. The frontend closes the dialog, shows a success toast, and refreshes the view.

Abnormal Flow / Exception Handling

Abnorma Case	**Handling & System Response**
Target banner does not exist	The service throws `BannerAdminNotFoundException` (HTTP 404). The UI displays: “Error.BannerNotFound”.
New position slot already occu-pied	The service throws `BannerPositionTakenException` (HTTP 409). The UI alerts: “Error.BannerPositionTaken”.
End date earlier than start date	Validator rejects payload with HTTP 400 Bad Request (`Error.BannerInvalidDateRange`). The UI highlights the date field.
Replacement image exceeds 10MB	Upload endpoint throws `BannerImageTooLargeException` (HTTP 400). The UI displays: “Error.BannerImageTooLarge”.

Abnorma Case	Handling & System Response
Invalid URL syntax in linkUrl	Validator returns HTTP 400 Bad Request. The UI highlights the link URL field.
Unauthorized or unauthenticated session	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.
Database update failure	The service throws `FailedToUpdateBannerException` (HTTP 500). The UI displays: “Error.FailedToUpdateBanner”.
Network failure during submission	The client displays an error toast notification and leaves the modal open for retry.

Output Data

Output Field	Description
id	Numeric identifier of the updated banner.
title	Updated banner title.
imageUrl	Updated image URL or null.
linkUrl	Updated destination URL or null.
position	Updated position slot enum.
startDate	Updated start publication date or null.
endDate	Updated end publication date or null.
isActive	Updated active status flag.
updatedAt	Updated modification timestamp.

Post-Conditions

The modified banner attributes are committed to the database and immediately reflected across the administrative console.

3.20.5 Delete Banner

Function Trigger

An administrator clicks the Delete action icon (trash can) on a banner table row on the banner list page, triggering the deletion confirmation alert dialog.
Navigation path: Admin Dashboard → Marketing & Banners → Banners → [Delete Action Icon].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to permanently remove an obsolete or unwanted advertisement banner from the system via hard deletion.

Interface

- Delete banner confirmation dialog (`DeleteBannerDialog`) powered by Shadcn `AlertDialog`.
- Modal header with warning title displaying the targeted banner title.
- Descriptive body explaining that this action will permanently remove the banner.
- Modal action buttons: "Cancel" and destructive "Delete" button with pending spinner.

Data Processing Overview

- The administrator confirms the deletion in the modal dialog.
- The client sends an HTTP request: `DELETE /api/admin/banners/:id`.
- `BannersAdminController.deleteBanner(id)` parses `id` via `ParseIntPipe`.
- `BannersAdminRepository.deleteBanner(id)` executes `prisma.advertisementBanner.deleteMan where: id ).`
- If `result.count === 0`, the service throws `BannerAdminNotFoundException` (HTTP

404).

- Returns HTTP 200 with `{ message: "Success.BannerDeleted" }`.
- The frontend displays a success toast, closes the confirmation dialog, and triggers `router.refresh()` to remove the row from the table.

Screen Layout (Mock-up Description)

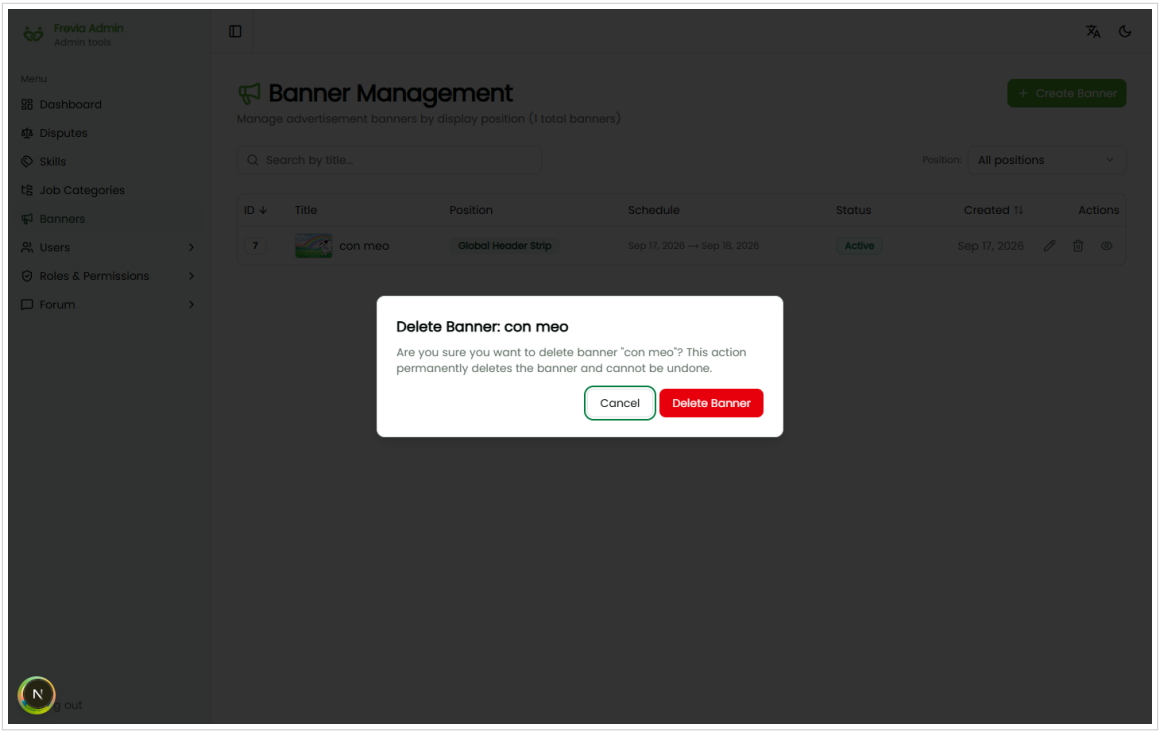

Figure 171: Delete banner confirmation dialog

- Alert dialog displaying “Delete Banner” title including target banner name.
- Warning explanation text: “Are you sure you want to delete this banner? This action cannot be undone.”
- Footer controls: “Cancel” button and red “Delete” button with loading spinner when executing.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the banner to delete (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-BAN-21**: Permanent Hard Deletion: Advertisement banners are permanently removed from the `advertisement_banners` table via hard deletion.
- **BR-BAN-22**: Existence Verification: Throws HTTP 404 (`Error.BannerNotFound`) if the banner record does not exist or was already removed.
- **BR-BAN-23**: Explicit Confirmation Gate: Destruction requires explicit confirmation from the administrator via modal alert to prevent accidental deletion.
- **BR-BAN-24**: Immediate List Synchronization: The deleted banner row is purged from the UI table immediately upon successful response.

Normal Flow

1. The administrator locates the banner in the table and clicks the Delete icon.
2. The deletion confirmation dialog modal appears.
3. The administrator reviews the banner title and confirms by clicking “Delete”.
4. The client sends `DELETE /api/admin/banners/:id`.
5. The backend validates administrative token and route permissions.
6. The repository deletes the record from `advertisementBanner`.
7. The backend returns HTTP 200 with `{ message: "Success.BannerDeleted" }`.
8. The client displays a success toast, closes the modal, and refreshes the banner list table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target banner does not exist	The service throws `BannerAdminNotFoundException` (HTTP 404). The UI displays: “Error.BannerNotFound”.
Missing or invalid authen-tication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privi-leges	The server throws HTTP 403 Forbidden. The UI displays an access denied notification.
Invalid banner ID format	`ParseIntPipe` throws HTTP 400 Bad Request. The UI notifies the user of an invalid parameter.
Administrator cancels dialog	The modal closes immediately; no request is sent, leaving the database unaffected.
Database deletion failure	The service throws `FailedToDeleteBannerException` (HTTP 500). The UI displays: “Error.FailedToDeleteBanner”.
Network failure during request	The client catches the network error, displays an unexpected error toast, and leaves the modal open.

Output Data

Output Field	Description
message	Success confirmation message string (`Success.BannerDeleted`).

Post-Conditions

The banner record is permanently deleted from the database and removed from public and administrative views.

3.21 Skill Management

3.21.1 View Skill List

Function Trigger

An administrator navigates to the Skills management page from the admin dashboard sidebar navigation.
Navigation path: Admin Dashboard → Master Catalog → Skills (`/admin/skills`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view, search, filter, and sort the master catalog of professional skills used by freelancer profiles and job postings. The catalog displays skill name, generated unique slug, description, active job count, soft-delete state, and audit dates.

Interface

- Skill management screen at `/admin/skills`.
- Filter bar component (`SkillsFilterBar`) with search input, status filter dropdown (All, Active Only, Deleted Only), sort column dropdown, and sort direction toggle.
- Data table component (`SkillsTable`) rendering skill name, slug, description, job usage count badge, status badge, creation date, and contextual action menus.
- Standard pagination footer displaying total skills and page selection buttons.
- "Create Skill" action button in the page header.

Data Processing Overview

- The client initiates an HTTP request: `GET /api/admin/skills` with query parameters (`page`, `limit`, `search`, `deleted`, `sortBy`, `sortOrder`).
- The backend `AuthGuard` and `PermissionGuard` verify the administrator's credentials and permissions.
- `SkillsAdminController.listSkills()` parses query parameters via `SkillAdminQueryDto`.
- `SkillsAdminRepository.listSkills()` queries `prisma.skill.findMany`:
  - If `search` is provided, executes case-insensitive `OR` query matching on `name`, `slug`, or `description`.
  - If `deleted === 'true'`, filters records where `deletedAt != null`; if `deleted === 'false'`, filters records where `deletedAt = null`.
  - Includes count of related jobs via `_count: { select: { jobs: true } }`.
  - Sorts records by `id` or `createdAt` in requested direction.
- Executes `prisma.skill.count()` in parallel to determine total matching records.
- Maps database records into response objects including `jobCount`.
- The backend returns HTTP 200 with the skills array and pagination metadata.
- The frontend renders the data table, applies green active badges or red deleted badges, and provides action buttons.

Screen Layout (Mock-up Description)

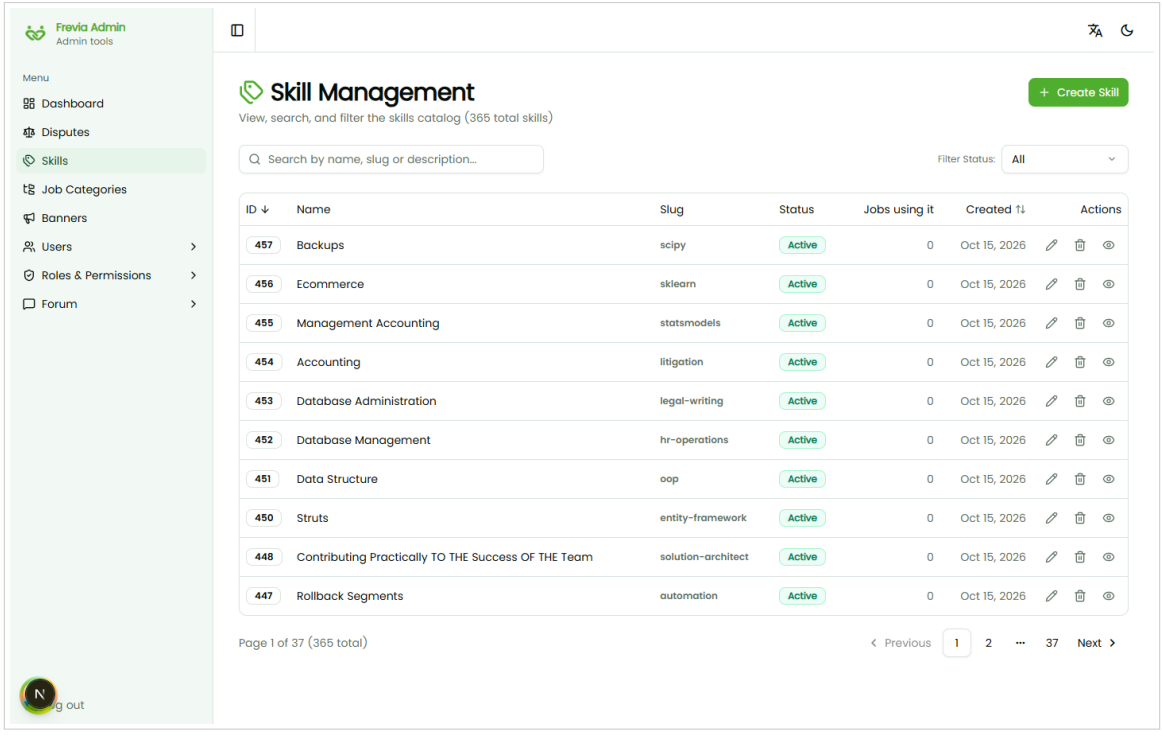

Figure 172: Skill catalog list interface

- Page header displaying “Skills” title with tag icon, summary count of total skills, and “Create Skill” button.
- Filter bar including search box, deleted status filter selector (All, Active, Deleted), and sort direction buttons.
- Table columns: **Skill Name** (with clickable detail link), **Slug**, **Description**, **In Use** (Badge showing number of jobs using this skill), **Status** (Active or Deleted badge), **Created Date**, and **Actions**.
- Contextual action buttons: Update (pencil) and Delete (trash) for active skills; Restore (rotate-ccw) for deleted skills.
- Pagination footer indicating total count and page navigation buttons.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum records returned per page (defaults to 10).
search	Optional text query matching name, slug, or description.
deleted	Optional status filter: `"true"` (deleted only) or `"false"` (active only).
sortBy	Sort column field (`id` or `createdAt`; defaults to `id`).
sortOrder	Sort direction sequence (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters; whitespace trimmed.
deleted	Optional string, must strictly match one of: `"true"`, `"false"`.
sortBy	Must strictly be one of: `id`, `createdAt`.
sortOrder	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-SKL-01**: Admin Authorization: Access to the master skill catalog is strictly restricted to authenticated administrators.
- **BR-SKL-02**: Substring Multi-Field Search: Search performs case-insensitive substring matching across `name`, `slug`, and `description`.
- **BR-SKL-03**: Soft-Deletion State Filtering: Setting `deleted=true` filters for `deletedAt != null`, `deleted=false` filters for `deletedAt = null`, and omitting it lists all skills.
- **BR-SKL-04**: Associated Job Count Aggregation: Automatically aggregates and displays the exact count of job postings utilizing each skill via `_count.jobs`.
- **BR-SKL-05**: Dynamic Status Badge and Action Switching: Active skills display green badges with Edit/Delete buttons; soft-deleted skills display muted badges with a Restore button.

Normal Flow

1. The administrator opens the `/admin/skills` page from the admin dashboard.
2. The client issues `GET /api/admin/skills` with default query parameters.
3. The backend validates administrative token and route permissions.
4. The repository queries master skills with job count aggregation.
5. The server returns HTTP 200 with the skills array and pagination metadata.
6. The frontend renders the skill table and pagination controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authen-tication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.

Abnorma Case	Handling & System Response
User lacks Admin privi-leges	The backend throws HTTP 403 Forbidden. The UI displays: “You do not have permission to access this page.”
Invalid pagina-tion parame-ter values	The DTO validator rejects non-integer values with HTTP 400 Bad Request. The client defaults to page 1.
Invalid deleted filter pa-rameter value	The validator rejects unknown values with HTTP 400 Bad Request. The UI clears invalid filter.
Unsupported sort column parame-ter	The validator rejects invalid sort keys with HTTP 400 Bad Request. The client defaults to sorting by `id desc`.
Database query error or timeout	The service throws `FailedToLoadSkillListException` (HTTP 500). The UI displays: “Error.FailedToLoadSkillList”.
No skills match search or status filter	The query returns `skills: []` and `total: 0`. The table displays an empty placeholder: “No skills found.”

Abnorma Case	Handling & System Response
Network connection timeout	The client displays an error toast notification and offers a “Try again” button.

Output Data

Output Field	Description
skills	Array of skill objects: `id`, `name`, `slug`, `description`, `deletedAt`, `createdAt`, `updatedAt`, `jobCount`.
pagination.page	Current page number.
pagination.limit	Current limit per page.
pagination.total	Total skills matching the query.
pagination.totalPages	Total calculated pages available.

Post-Conditions

The administrator inspects the master skill catalog, views job usage metrics, and can initiate creation, editing, deletion, or restoration.

3.21.2 View Skill Detail

Function Trigger

An administrator clicks on a skill row or name in the skill list, navigating to `/admin/skills/[id]`.

Navigation path: Admin Dashboard → Master Catalog → Skills → Skill Details (`/admin/skills/[id]`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view detailed metadata of a specific skill, including its generated slug, full description, total count of linked jobs, deletion status, and audit timestamps.

Interface

- Skill detail screen at `/admin/skills/[id]`.
- Navigation back button ("Back to Skills").
- Header card displaying skill name, slug, ID, and action buttons (`SkillActions`).
- Metrics card displaying the number of active jobs utilizing this skill.
- Description card displaying the full skill description or a placeholder if empty.
- Lifecycle timestamps card (Created date, Last updated date, and Deletion date if soft-deleted).

Data Processing Overview

- The client sends an HTTP request: `GET /api/admin/skills/:id`.
- The backend validates administrative token and route permissions.
- `SkillsAdminController.getSkillDetail()` parses `id` via `ParseIntPipe`.
- `SkillsAdminRepository.getSkillDetail(id)` queries `prisma.skill.findUnique` by `id`, including `_count: { select: { jobs: true } }`.
- If the skill record is not found, the service throws `SkillAdminNotFoundException` (HTTP 404).
- Returns HTTP 200 with the skill object including `jobCount`.
- The frontend renders the skill details view and contextual action buttons based on whether `deletedAt` is null.

Screen Layout (Mock-up Description)

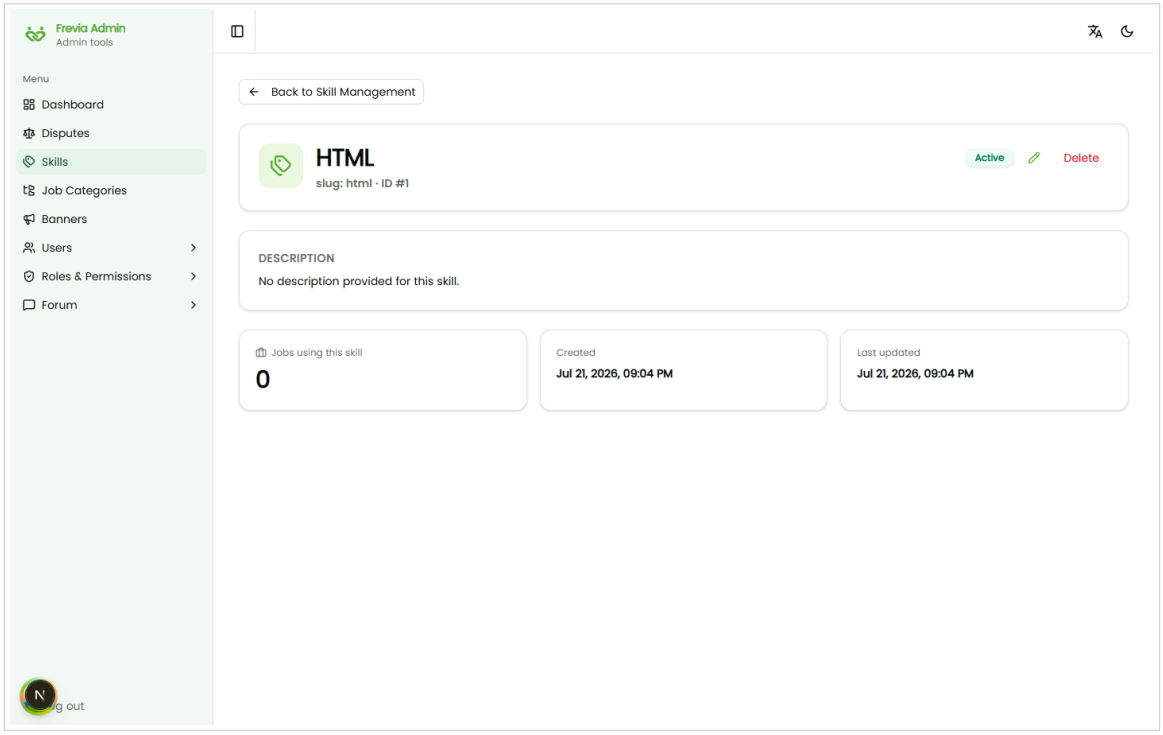

Figure 173: Skill detail interface

- Navigation header with “Back to Skills” button and skill name with tag icon.
- Action toolbar containing Edit (pencil) and Delete (trash) buttons if active, or Restore (rotate-ccw) button if soft-deleted.
- Summary card showing: **Slug** (e.g., `reactjs`), **Status** (Active or Deleted badge), and **ID**.
- Usage metrics card displaying **Active Jobs Count** with a briefcase icon.
- Content card displaying the **Description** of the skill.
- Audit card displaying **Created Date**, **Last Updated Date**, and **Deleted At** (if applicable).

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the skill (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-SKL-06**: Numeric Skill ID Validation: The URL parameter `id` must parse strictly as a valid positive integer.
- **BR-SKL-07**: Skill Record Existence: The requested skill must exist in the database; returns HTTP 404 (`Error.SkillNotFound`) if missing.
- **BR-SKL-08**: Complete Detail and Usage Metrics: Displays name, generated slug, description, creation date, update date, deletion timestamp, and total jobs utilizing this skill.
- **BR-SKL-09**: Contextual Action Toolbar: Shows Edit and Delete actions if skill is active; shows Restore action if skill is soft-deleted.

Normal Flow

1. The administrator navigates to `/admin/skills/[id]`.
2. The client issues `GET /api/admin/skills/:id`.
3. The backend validates administrative authentication and permissions.
4. The repository queries `prisma.skill.findUnique` with job count aggregation.
5. The server returns HTTP 200 with the skill object.
6. The frontend renders the comprehensive skill detail view.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.
Invalid skill ID format (e.g., non-numeric)	`ParseIntPipe` throws HTTP 400 Bad Request. The UI displays an "Invalid Skill ID" message and back button.
Skill ID does not exist in database	The service throws `SkillAdminNotFoundException` (HTTP 404). The UI renders a "Skill Not Found" screen with navigation back to catalog.
Database query error or timeout	The service throws `FailedToLoadSkillDetailException` (HTTP 500). The UI displays: "Error.FailedToLoadSkillDetail".
Skill description is empty or null	The UI gracefully displays "No description provided" in muted text.
Network connection failure	The client displays an error toast notification and offers a reload button.

Output Data

Output Field	Description
id	Unique numeric identifier of the skill.
name	Skill name string.
slug	Generated URL-friendly slug.
description	Detailed description or null.
deletedAt	Soft-deletion timestamp or null.
createdAt	Timestamp of skill creation.
updatedAt	Timestamp of last modification.
jobCount	Total number of jobs referencing this skill.

Post-Conditions

The administrator reviews skill details and usage metrics, and can initiate editing, deletion, or restoration.

3.21.3 Create Skill

Function Trigger

An administrator clicks the "Create Skill" button on the skill catalog page (`/admin/skills`), opening the skill creation dialog modal.
Navigation path: Admin Dashboard → Master Catalog → Skills → [Create Skill Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to register a new professional skill in the master catalog by providing its name and optional description, with automated unique slug generation.

Interface

- Skill creation dialog modal (`CreateSkillDialog`).
- Text input for Skill Name (mandatory).

- Textarea input for Description (optional).
- Action buttons: “Cancel” and “Create Skill” submit button with loading indicator.

Data Processing Overview

- The client submits an HTTP request: `POST /api/admin/skills` with JSON payload conforming to `AdminCreateSkillBodySchema`.
- `SkillsAdminRepository.createSkill()`:
  - Strips leading and trailing whitespace from `name`.
  - Queries `prisma.skill.findFirst` to check if an active skill (`deletedAt = null`) with the same name exists (case-insensitive mode).
  - If an active duplicate exists, throws `SkillNameAlreadyExistsException` (HTTP 409).
  - Generates a unique slug via `generateUniqueSlug()`: normalizes Vietnamese diacritics, converts to lowercase, replaces non-alphanumeric characters with hyphens, and appends sequential numeric suffixes (`-2`, `-3`) if a slug collision occurs.
  - Inserts the skill record via `prisma.skill.create()`.
  - Catches Prisma `P2002` errors (unique constraint violations) and converts to `SkillNameAlreadyExistsException`.
- Returns HTTP 201 with the created skill entity.
- The frontend displays a success toast, closes the modal, and refreshes the skill table.

Screen Layout (Mock-up Description)

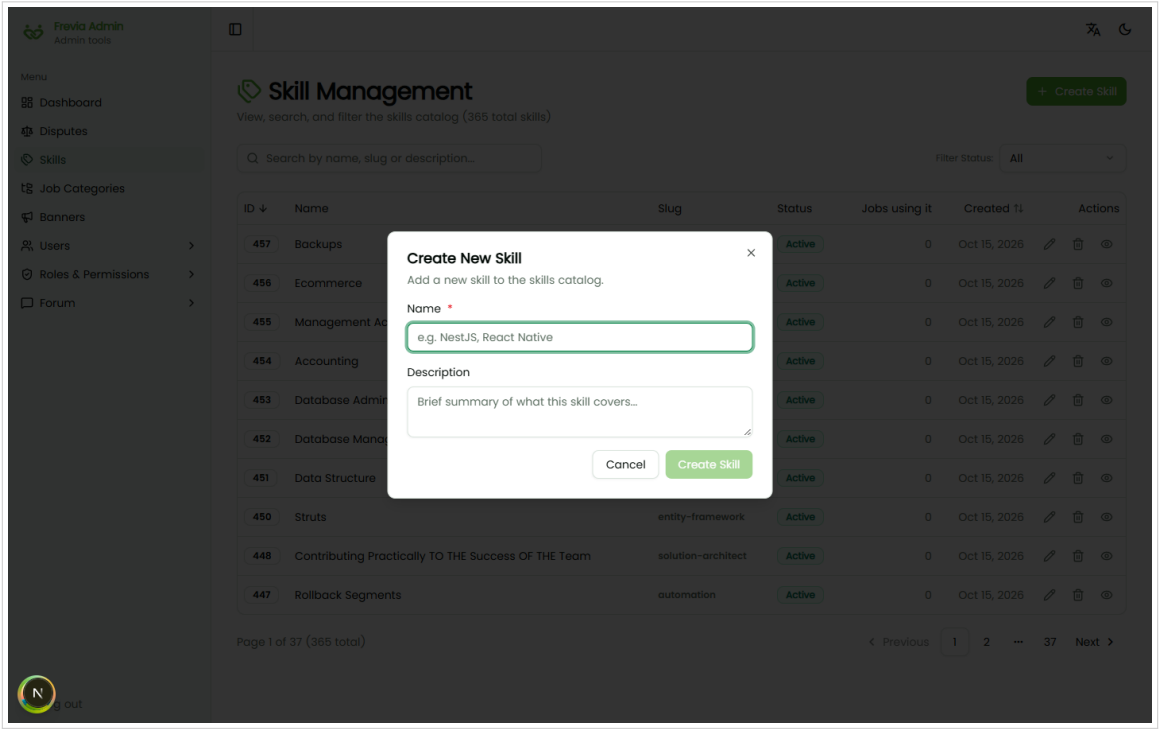

Figure 174: Create skill modal interface

- Dialog header with “Create Skill” title and descriptive prompt.
- Form fields: **Name** (Required input field with asterisk) and **Description** (Multi-line textarea).
- Dialog footer with “Cancel” button and green “Create Skill” button with loading spinner when submitting.

Function Details

Input Data

Data Name	Description
name	Skill name string (mandatory).
description	Optional text description of the skill.

Validation Rules

Data Name	Validation Rule
name	String, trimmed, min 1 char (`Error.SkillNameRequired`), max 100 chars (`Error.SkillNameTooLong`).
description	Optional string, max 500 chars (`Error.SkillDescriptionTooLong`); nullable.

Business Rules

- **BR-SKL-10**: Case-Insensitive Name Uniqueness: Skill name must be unique among active skills; throws HTTP 409 (`Error.SkillNameAlreadyExists`) if conflict occurs.
- **BR-SKL-11**: Automated Unique Slug Generation: Automatically generates a URL-friendly slug by stripping diacritics, replacing special characters with hyphens, and suffixing numbers if collisions occur.
- **BR-SKL-12**: Name and Description Constraints: Name is mandatory trimmed 1 to 100 characters; description is optional up to 500 characters.
- **BR-SKL-13**: Concurrency Race-Condition Guard: Catches database `P2002` unique constraint violations and converts them gracefully to HTTP 409 conflict errors.
- **BR-SKL-14**: Dialog Reset and List Invalidation: Clears input values, closes modal dialog, and triggers Next.js router refresh upon success.

Normal Flow

1. The administrator clicks “Create Skill” on the skills catalog page.
2. The dialog modal opens with empty input fields.
3. The administrator enters the skill name and optional description.
4. The administrator clicks “Create Skill”.
5. The client submits `POST /api/admin/skills`.
6. The backend validates data, checks for duplicate active names, generates a slug, and inserts the record.
7. The server returns HTTP 201 with the created skill entity.
8. The client displays a success toast notification, closes the modal, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing mandatory skill name	Form validation blocks submission; backend throws HTTP 400 with `Error.SkillNameRequired`.
Skill name exceeds 100 characters	Validator rejects input with HTTP 400 Bad Request (`Error.SkillNameTooLong`). The UI highlights the name field.
Description exceeds 500 characters	Validator rejects input with HTTP 400 Bad Request (`Error.SkillDescriptionTooLong`). The UI highlights the description field.
Active skill with identical name already exists	The service throws `SkillNameAlreadyExistsException` (HTTP 409 Conflict). The UI displays: “Error.SkillNameAlreadyExists”.
Concurrent request creates duplicate name	Database catches `P2002` constraint violation; backend converts to HTTP 409 Conflict. The UI alerts the administrator.
Unauthorized or unauthenticated request	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.

Abnorma Case	Handling & System Response
Database insert failure	The service throws `FailedToCreateSkillException` (HTTP 500). The UI displays: "Error.FailedToCreateSkill".
Network timeout	The client catches the network failure, displays an error toast, and leaves the modal open for retry.

Output Data

Output Field	Description
id	Auto-generated unique identifier of the created skill.
name	Stored skill name string.
slug	Auto-generated unique slug.
description	Stored description string or null.
deletedAt	Null for newly created skills.
createdAt	Timestamp of creation.
updatedAt	Timestamp of creation.
jobCount	Zero for newly created skills.

Post-Conditions

A new skill record is added to the master catalog, indexed for search, and immediately selectable by freelancers and clients.

3.21.4 Update Skill

Function Trigger

An administrator clicks the Edit icon (pencil) on a skill table row or detail page, opening the skill edit dialog modal.

Navigation path: Admin Dashboard → Master Catalog → Skills → [Edit Action Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to modify the name and description of an existing master skill, automatically synchronizing its unique slug if the name is altered.

Interface

- Skill update dialog modal (`UpdateSkillDialog`) pre-populated with current skill name and description.
- Text input for Skill Name.
- Textarea input for Description.
- Action buttons: "Cancel" and "Save Changes" submit button with loading indicator.

Data Processing Overview

- The client pre-fills the form with the skill's current name and description.
- The client issues an HTTP request: `PATCH /api/admin/skills/:id` with updated fields.
- `SkillsAdminRepository.updateSkill(id, data)`:
  - Verifies target skill exists; throws `SkillAdminNotFoundException` if missing.
  - If `name` is modified, checks if another active skill (`id != currentId, deletedAt = null`) shares the same case-insensitive name. Throws `SkillNameAlreadyExistsException` if conflict exists.
  - If `name` is modified, regenerates the unique slug via `generateUniqueSlug()`.
  - Updates the record in `prisma.skill.update()`. If description is empty string, converts it to `null`.
- The backend returns HTTP 200 with the updated skill entity.
- The frontend displays a success toast, closes the modal, and refreshes the view.

Screen Layout (Mock-up Description)

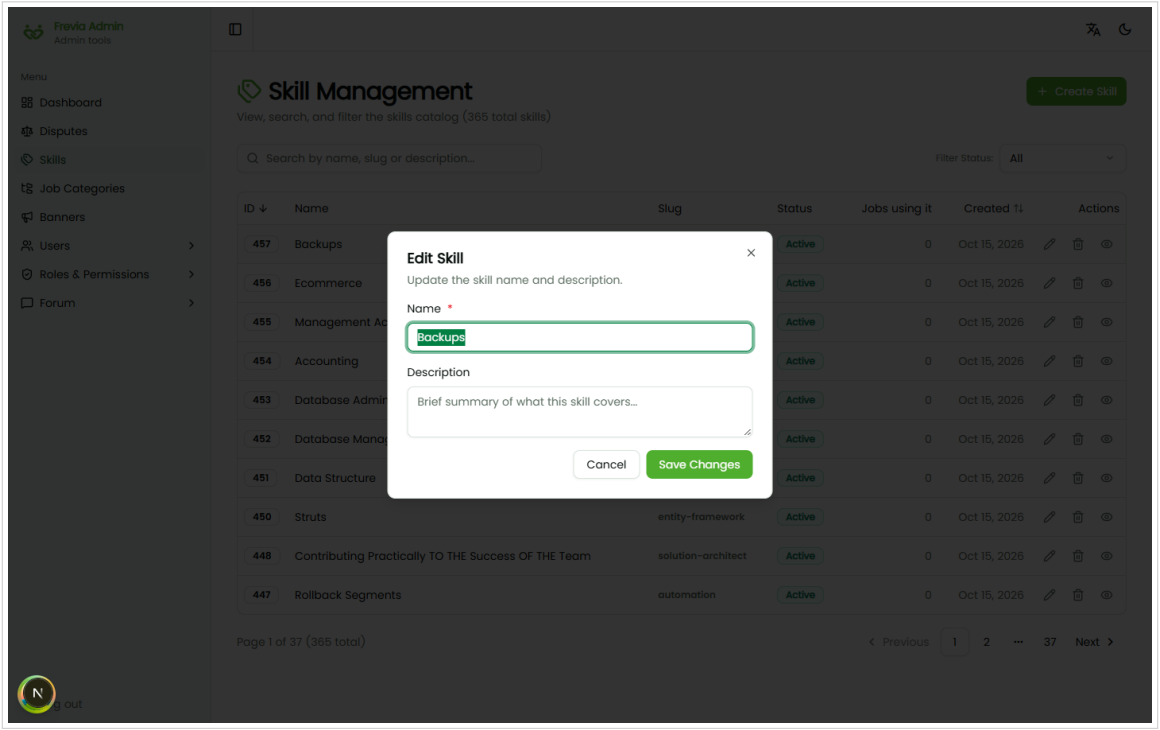

Figure 175: Update skill modal interface

- Modal dialog header with “Edit Skill” title.
- Form pre-populated with current values: **Name** (Text input) and **Description** (Textarea).
- Footer action buttons: “Cancel” and “Save Changes” button with pending spinner.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the skill (passed in URL path).
name	Updated skill name string (optional).
description	Updated description string or null (optional).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.
name	Optional string, trimmed, min 1 char (`Error.SkillNameRequired`), max 100 chars (`Error.SkillNameTooLong`).
description	Optional string, max 500 chars (`Error.SkillDescriptionTooLong`); nullable.

Business Rules

- **BR-SKL-15**: Skill Existence Verification: Target skill must exist in the database; returns HTTP 404 (`Error.SkillNotFound`) if missing.
- **BR-SKL-16**: Name Modification and Conflict Detection: If name is altered, verifies no other active skill shares the same case-insensitive name; throws HTTP 409 if conflict detected.
- **BR-SKL-17**: Automatic Slug Regeneration: If the name is changed, automatically regenerates a new unique URL-friendly slug based on the updated name.
- **BR-SKL-18**: Optional Description Nullability: Submitting an empty description string sets the description attribute to null in the database.
- **BR-SKL-19**: Synchronized UI Updates: Closes modal dialog, triggers success toast, and refreshes table or detail screen upon completion.

Normal Flow

1. The administrator clicks the Edit icon on a skill row or detail page.
2. The dialog modal opens pre-filled with the current skill name and description.
3. The administrator modifies the name or description.
4. The administrator clicks “Save Changes”.
5. The client submits `PATCH /api/admin/skills/:id`.
6. The backend validates fields, checks for duplicate active names, regenerates slug if name changed, and updates the database.
7. The backend returns HTTP 200 with the updated skill entity.
8. The frontend closes the dialog, displays a success toast, and refreshes the view.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target skill does not exist	The service throws `SkillAdminNotFoundException` (HTTP 404). The UI displays: “Error.SkillNotFound”.
Updated name conflicts with active skill	The service throws `SkillNameAlreadyExistsException` (HTTP 409). The UI alerts: “Error.SkillNameAlreadyExists”.
Name is set to empty or whitespace	Form validation blocks submission; backend returns HTTP 400 Bad Request with `Error.SkillNameRequired`.
Name exceeds 100 characters	Validator rejects input with HTTP 400 Bad Request (`Error.SkillNameTooLong`). The UI highlights the name field.
Description exceeds 500 characters	Validator rejects input with HTTP 400 Bad Request (`Error.SkillDescriptionTooLong`). The UI highlights the description field.
Unauthorized or unauthenticated session	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.
Database update failure	The service throws `FailedToUpdateSkillException` (HTTP 500). The UI displays: “Error.FailedToUpdateSkill”.

Abnorma Case	Handling & System Response
Network timeout	The client catches the network error, displays an error toast, and preserves dialog state.

Output Data

Output Field	Description
id	Numeric identifier of the updated skill.
name	Updated skill name.
slug	Updated or preserved unique slug.
description	Updated description or null.
deletedAt	Preserved soft-deletion status.
updatedAt	Updated modification timestamp.
jobCount	Count of active jobs using this skill.

Post-Conditions

The skill record and its generated slug are updated in the database, with immediate propagation across search queries and profile forms.

3.21.5 Delete Skill

Function Trigger

An administrator clicks the Delete action icon (trash can) on a skill table row or detail page, opening the deletion confirmation dialog.
Navigation path: Admin Dashboard → Master Catalog → Skills → [Delete Action Icon].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to soft-delete a skill from the active catalog by setting its `deletedAt`

timestamp, automatically detaching it from freelancer profiles while preserving historical job associations.

Interface

- Delete skill confirmation dialog (`DeleteSkillDialog`) powered by Shadcn `AlertDialog`.
- Header displaying skill name and warning icon if currently utilized by jobs.
- Dependent jobs warning banner if `jobCount > 0`, displaying exact number of linked jobs.
- Explanation note detailing soft deletion behavior.
- Action buttons: "Cancel" and destructive "Delete" button with pending spinner.

Data Processing Overview

- The administrator confirms deletion in the dialog.
- The client sends an HTTP request: `DELETE /api/admin/skills/:id`.
- `SkillsAdminController.deleteSkill(id)` parses `id` via `ParseIntPipe`.
- `SkillsAdminRepository.deleteSkill(id)`:
  - Verifies target skill exists; throws `SkillAdminNotFoundException` if missing.
  - Executes an atomic transaction via `prisma.$transaction`:
    1. Deletes all freelancer associations referencing this skill: `prisma.freelancerSkill.deleteMan where: skillId: id ).`
    2. Sets soft-deletion timestamp on the skill: `prisma.skill.update( where: id , data: deletedAt: new Date() ).`
  - Leaves `job_skills` records intact so existing historical job postings remain accurate.
- The backend returns HTTP 200 with `{ message: "Success.SkillDeleted" }`.
- The frontend shows a success toast, closes the dialog, and refreshes the skill table.

Screen Layout (Mock-up Description)

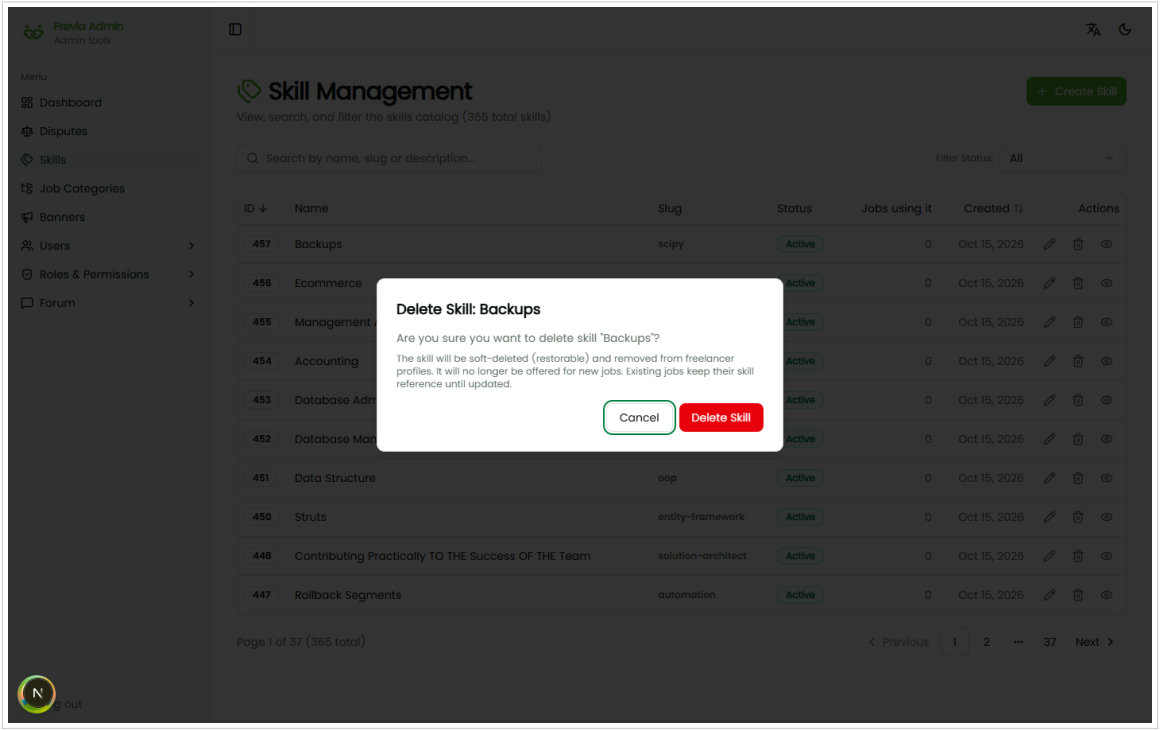

Figure 176: Delete skill confirmation dialog

- Dialog header with “Delete Skill” title and target skill name.
- Amber warning alert banner (when `jobCount > 0`) stating: “Skill in use: This skill is currently used by X jobs. Soft-deleting it will preserve job references.”
- Soft-deletion explanation text stating that the skill can be restored at any time.
- Footer action buttons: “Cancel” button and red “Delete” button with loading spinner.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the skill to delete (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-SKL-20**: Soft-Deletion Principle: Skills are not permanently purged from the database; the system marks them soft-deleted by setting `deletedAt = new Date()`.
- **BR-SKL-21**: Automatic Freelancer Association Detachment: All associations in `freelancer_skills` referencing this skill ID are deleted in the same atomic transaction.
- **BR-SKL-22**: Preservation of Historical Job Associations: Associations in `job_skills` are preserved so existing job postings continue to render their legacy skill requirements accurately.
- **BR-SKL-23**: Dependent Job Warning in UI: The deletion dialog prominently displays an alert warning with the exact number of jobs currently linked to the skill.
- **BR-SKL-24**: Confirmation Modal Guard: Requires explicit confirmation from the administrator before executing soft deletion.

Normal Flow

1. The administrator clicks the Delete icon on a skill table row or detail screen.
2. The deletion confirmation dialog appears, showing job usage warnings if applicable.
3. The administrator reviews the warning and confirms by clicking “Delete”.
4. The client issues `DELETE /api/admin/skills/:id`.
5. The backend validates administrative authentication and permissions.
6. The repository executes an atomic transaction detaching freelancer skills and setting `deletedAt`.
7. The backend returns HTTP 200 with `{ message: "Success.SkillDeleted" }`.
8. The frontend displays a success toast, closes the dialog, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target skill does not exist	The service throws `SkillAdminNotFoundException` (HTTP 404). The UI displays: “Error.SkillNotFound”.

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.
Invalid skill ID format	`ParseIntPipe` throws HTTP 400 Bad Request. The UI notifies the user of an invalid parameter.
Administrator cancels dialog	The modal closes immediately; no request is sent, leaving the database unaffected.
Database transaction failure	The service throws `FailedToDeleteSkillException` (HTTP 500). The UI displays: "Error.FailedToDeleteSkill".
Network failure during request	The client catches the network error, displays an unexpected error toast, and leaves the modal open.

Output Data

Output Field	Description
message	Success confirmation message string (`Success.SkillDeleted`).

Post-Conditions

The skill is marked as soft-deleted (`deletedAt != null`), removed from freelancer profiles, hidden from default active listings, and preserved for historical jobs.

3.21.6 Restore Skill

Function Trigger

An administrator viewing soft-deleted skills (via the "Deleted" status filter or detail page) clicks the Restore icon (rotate-ccw), opening the restore confirmation dialog.
Navigation path: Admin Dashboard → Master Catalog → Skills → [Restore Action Icon].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to restore a previously soft-deleted skill by resetting its `deletedAt` timestamp to null, making it active and selectable again across the platform.

Interface

- Restore skill confirmation dialog (`RestoreSkillDialog`) powered by Shadcn `AlertDialog`.
- Header displaying "Restore Skill" title and target skill name.
- Descriptive body explaining that restoring will make the skill available again for freelancers and clients.
- Action buttons: "Cancel" and green "Restore" button with pending spinner.

Data Processing Overview

- The administrator confirms restoration in the dialog.
- The client sends an HTTP request: `PATCH /api/admin/skills/:id/restore`.

- `SkillsAdminController.restoreSkill(id)` parses `id` via `ParseIntPipe`.
- `SkillsAdminRepository.restoreSkill(id)`:
  - Finds the skill verifying that `deletedAt != null`; throws `SkillAdminNotFoundException` if missing or already active.
  - Checks if another active skill has been created with the same name during the deletion period (`deletedAt = null, id != currentId, name equals skill.name`).
  - If a name collision occurs, throws `SkillNameAlreadyExistsException` (HTTP 409).
  - Sets `deletedAt = null` via `prisma.skill.updateMany( where: id, deletedAt: not: null , data: deletedAt: null )`.
  - Queries the updated skill entity with job count aggregation.
- The backend returns HTTP 200 with the restored skill entity.
- The frontend shows a success toast notification, closes the dialog, and refreshes the view.

Screen Layout (Mock-up Description)

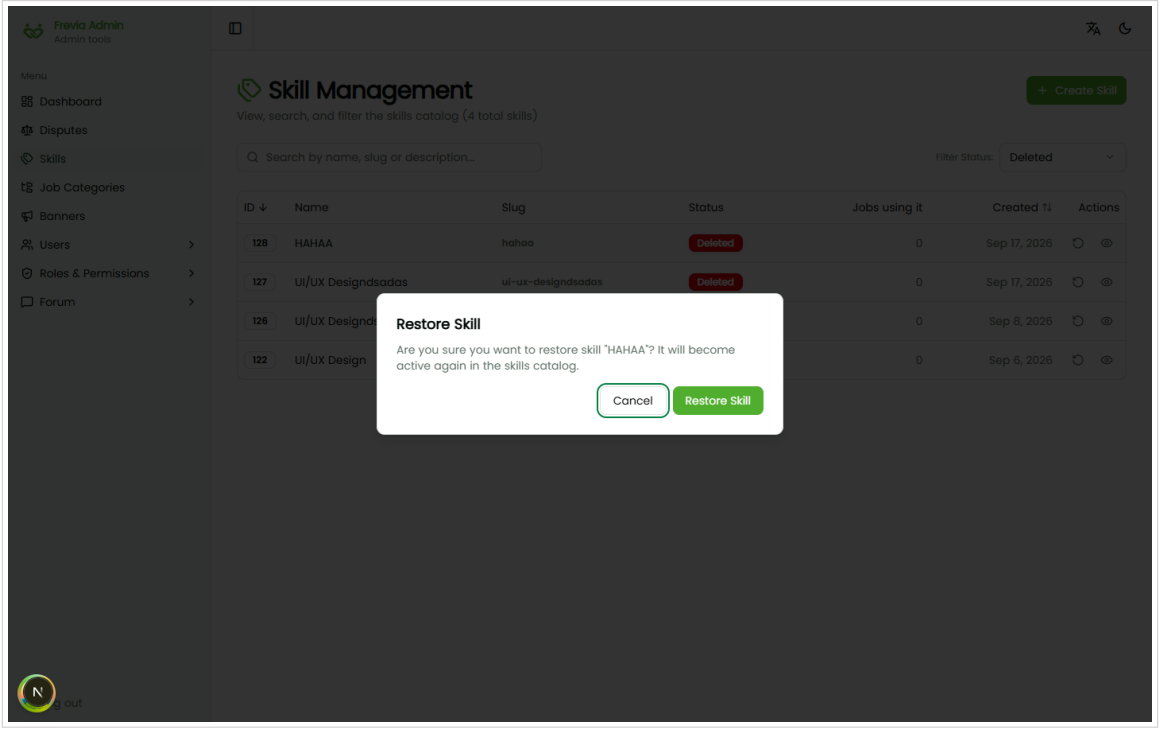

Figure 177: Restore skill confirmation dialog

- Alert dialog header displaying “Restore Skill” title with rotate-counterclockwise icon.
- Confirmation description stating: “Are you sure you want to restore skill [Skill Name]? It will become active and available for users again.”
- Footer controls: “Cancel” button and green “Restore” button with loading spinner when executing.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the soft-deleted skill (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-SKL-25**: Restoration Eligibility: Target skill must currently be soft-deleted with `deletedAt != null`; returns HTTP 404 if not found or already active.
- **BR-SKL-26**: Conflict Prevention Against Interim Duplicates: If another active skill with the same name was created while this skill was deleted, restoration is blocked with HTTP 409 (`Error.SkillNameAlreadyExists`).
- **BR-SKL-27**: Soft-Deletion Flag Clearing: Sets `deletedAt = null`, fully restoring the skill to active catalog availability.
- **BR-SKL-28**: Immediate Status Synchronization: Returns restored skill data, displays success toast, and updates table view immediately.

Normal Flow

1. The administrator filters the catalog by "Deleted" skills.
2. The administrator locates the soft-deleted skill and clicks the Restore icon.
3. The restore confirmation dialog appears.
4. The administrator confirms restoration by clicking "Restore".
5. The client sends `PATCH /api/admin/skills/:id/restore`.
6. The backend verifies the skill is soft-deleted, checks for name conflicts with active skills, and clears `deletedAt`.
7. The backend returns HTTP 200 with the restored skill data.
8. The client displays a success toast, closes the dialog, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target skill does not exist or is already active	The service throws `SkillAdminNotFoundException` (HTTP 404). The UI displays: “Error.SkillNotFound”.
Another active skill now shares the same name	The service throws `SkillNameAlreadyExistsException` (HTTP 409). The UI alerts: “Error.SkillNameAlreadyExists”.
Missing or invalid authen-tication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privi-leges	The server throws HTTP 403 Forbidden. The client renders an access de-nied alert.
Invalid skill ID format	`ParseIntPipe` throws HTTP 400 Bad Request. The UI notifies the user of an invalid parameter.
Administrator cancels dialog	The modal closes immediately; no request is sent, leaving the database unaffected.

Abnorma Case	Handling & System Response
Database update failure	The service throws `FailedToRestoreSkillException` (HTTP 500). The UI displays: "Error.FailedToRestoreSkill".
Network failure during request	The client catches the network error, displays an unexpected error toast, and preserves dialog state.

Output Data

Output Field	Description
id	Numeric identifier of the restored skill.
name	Skill name string.
slug	Preserved or updated unique slug.
description	Description text or null.
deletedAt	Null indicating active status.
createdAt	Timestamp of original creation.
updatedAt	Timestamp of restoration.
jobCount	Count of jobs referencing this skill.

Post-Conditions

The skill is restored to active status (`deletedAt = null`), displayed in the active skill catalog, and available for selection in freelancer profiles and job postings.

3.22 Payment Management

3.22.1 Connect Stripe Account

Function Trigger

A Freelancer clicks "Set up Payouts" or "Connect Stripe" in their Billing and Payout Settings.

Navigation path: Freelancer Dashboard → Settings → Payout Settings (`/freelancer/settings/payouts`) → Click "Connect with Stripe".

Function Description

Actors / Roles

- **Primary Actor:** Freelancer (`FREELANCER` role).
- **Secondary Actor:** Stripe Connect Service.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Enables a Freelancer to link their bank account or debit card via Stripe Connect Express onboarding. Establishing this connection is required to receive automated escrow disbursements, milestone earnings payouts, and dispute resolution settlement funds.

Interface

- Payout Settings screen at `/freelancer/settings/payouts`.
- Status banner indicating connection state: "Unconnected", "Action Required", or "Connected & Verified".
- Primary action button: "Connect with Stripe" or "Update Payout Details".
- Stripe-hosted Express onboarding flow for identity verification, tax details, and bank account entry.
- Return confirmation banner upon redirect back to Frevia.

Data Processing Overview

- Freelancer submits an HTTP request: `POST /api/payments/connect/onboarding-link` with optional `returnUrl`.
- `PaymentService.getOnboardingLink()` validates caller identity and role.
- If no Stripe Account ID exists for the user, creates a new Stripe Express account via `stripe.accounts.create()` and stores `stripeAccountId` in the database.
- Generates a single-use onboarding URL via `stripe.accountLinks.create()` with refresh and return routes.
- Freelancer completes onboarding on Stripe-hosted UI and is redirected back to Frevia.
- Client queries `GET /api/payments/connect/status` which synchronizes `chargesEnabled`, `payoutsEnabled`, and `detailsSubmitted`.
- Returns HTTP 200 with connection status flags.

Screen Layout (Mock-up Description)

Figure 178: Stripe Connect payout settings screen

- Header renders title “Payout Settings” and a description of secure bank transfers powered by Stripe.
- A prominent card displays the Stripe logo, current status badge (e.g., green “Verified”), and bank account snippet.
- If unlinked, a prominent purple “Set Up Stripe Express Payouts” button is centered with security badges.
- A secondary link allows managing tax documents or changing payout schedules directly in Stripe.

Function Details

Input Data

Data Name	Description
`returnUrl`	Optional callback URL to return after onboarding (string, URL format).

Validation Rules

- Caller must be an authenticated Freelancer.
- User account must have status `ACTIVE`.

Business Rules

- **BR-PAY-01**: Freelancers must have a verified Stripe account with payouts enabled to receive contract payments.
- **BR-PAY-02**: Banking credentials and tax identifiers are stored exclusively by Stripe; Frevia stores only the account reference ID.

Normal Flow

1. Freelancer navigates to Payout Settings and clicks "Connect with Stripe".
2. API creates or retrieves Stripe Express account and generates onboarding link.
3. Browser redirects to Stripe-hosted verification flow.
4. Freelancer completes banking and identity verification.
5. Stripe redirects back to Frevia return URL.
6. System synchronizes account status, updates `stripeOnboardingCompleted = true`, and displays verified badge.

Abnormal Flow / Exception Handling

Case	Description
A1	Stripe API connection failure. System displays: "Unable to reach payment gateway. Please try again later."
A2	Verification incomplete or exited prematurely. Stripe flags account with pending requirements; system displays "Action Required" banner.
A3	User is not a Freelancer. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`onboardingUrl`	Temporary Stripe-hosted onboarding URL.
`isConnected`	Boolean status indicating whether payouts are active (`true` / `false`).

Post-Conditions

- Stripe Account ID linked to user profile.
- Payout capabilities verified and enabled.

3.22.2 View Payout History

Function Trigger

A Freelancer clicks the "Payouts" tab in their financial dashboard.

Navigation path: Freelancer Dashboard → Payments → Click "Payouts Tab" (`/freelancer/payments?typ`

Function Description

Actors / Roles

- **Primary Actor:** Freelancer (`FREELANCER` role), Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Displays a paginated ledger of all automated bank transfer payouts sent to the Freelancer's Stripe connected account, showing transfer amounts, net earnings, platform commission deductions, and processing statuses.

Interface

- Payouts tab on the financial transactions dashboard.
- Metric cards summarizing Total Withdrawn, Pending Transfers, and Available Balance.
- Payouts data table with columns: Date, Transfer Reference, Related Contract/Milestone, Gross Amount, Platform Fee (10%), Net Payout, Status, and Actions.
- Filter pills by status (`COMPLETED`, `PENDING`, `FAILED`).
- Pagination controls footer.

Data Processing Overview

- Client requests `GET /api/payments/transactions?type=PAYOUT&page=1&limit=10`.
- `PaymentService.getTransactions()` filters transactions where `userId === callerId` and `type === 'PAYOUT'` (or unrestricted for Admin).
- Orders transactions chronologically by `createdAt desc`.
- Computes pagination count in parallel via Prisma count.
- Returns HTTP 200 with the payout records array.

Screen Layout (Mock-up Description)

Figure 179: Freelancer payout history screen

- Top row renders cards for Total Earnings Disbursed and Last Payout Date.
- Table lists transfer rows with date stamp, milestone title, green net payout amount, and status pill.
- Action icon opens granular transfer detail modal.

Function Details

Input Data

Data Name	Description
`page`	Page number (integer, default 1).
`limit`	Items per page (integer, default 10, max 50).
`status`	Optional filter status (`COMPLETED`, `PENDING`, `FAILED`).

Validation Rules

- User must be an authenticated Freelancer or Administrator.

Business Rules

- **BR-PAY-03**: Freelancers can only view payout transfers associated with their own user account.

Normal Flow

1. Freelancer opens the Payouts tab.
2. Client requests payout transactions from API.
3. API queries database and returns paginated payout records.
4. Client renders payout history table.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated request. API returns HTTP 401; redirects to login.
A2	No payouts exist. Client renders empty state: “No payouts recorded yet.”

Output Data

Data Name	Description
`payouts[]`	Array of payout records with id, amount, netAmount, feeAmount, status, and createdAt.
`pagination`	Object containing page, limit, totalItems, and totalPages.

Post-Conditions

- Payout transfer history displayed.

3.22.3 View Payout Detail

Function Trigger

A user clicks on a payout transfer row in the payout history table.

Navigation path: Payout History → Click Payout Row (`.../transactions/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM, and PostgreSQL database.

Purpose

Displays granular transaction data for a specific payout transfer, including Stripe Transfer ID, destination connected account reference, milestone reference, fee breakdown, and exact settlement timestamps.

Interface

- Payout detail modal dialog or drawer view.
- Summary section: Gross Milestone Amount, Platform Fee (10%), Net Transfer Amount ($USD).
- Metadata breakdown: Stripe Transfer ID, destination account snippet, and completion timestamp.
- Action buttons: “Download Receipt (PDF)” and “Close”.

Data Processing Overview

- Client requests `GET /api/payments/transactions/:id`.

- `PaymentService` validates authorization: verifies that transaction belongs to caller or caller is Admin.
- Queries `prisma.transaction.findUnique` with contract and milestone relations.
- Returns HTTP 200 with the transaction entity.

Screen Layout (Mock-up Description)

Figure 180: Payout detail breakdown screen

- Modal displays receipt icon, transaction ID, and green “Completed” badge.
- Itemized receipt box shows Gross Escrow Release, Platform Service Fee (-10%), and Net Transferred.
- Technical metadata section renders Stripe reference ID and execution timestamp.

Function Details

Input Data

Data Name	Description
`id`	Numeric transaction identifier (integer path parameter).

Validation Rules

- Transaction ID must exist.
- Caller must be the owner of the transaction or an Administrator.

Business Rules

- **BR-PAY-04**: Strict anti-IDOR isolation blocks users from viewing another user's payout receipts.

Normal Flow

1. User clicks a payout row.
2. Client requests transaction details.
3. API validates ownership and returns transaction data.
4. Client renders payout detail modal.

Abnormal Flow / Exception Handling

Case	Description
A1	Caller does not own transaction. API returns HTTP 403 Forbidden with error `PaymentForbiddenException`.
A2	Transaction ID not found. API returns HTTP 404 Not Found with error `TransactionNotFoundException`.

Output Data

Data Name	Description
`transaction`	Full transaction record with gross amount, fee amount, net amount, and Stripe reference.

Post-Conditions

- Payout receipt displayed.

3.22.4 Fund Milestone

Function Trigger

A Client clicks the "Fund Escrow" / "Deposit Funds" button on an unfunded milestone card.
Navigation path: Contract Detail → Milestones Tab → Milestone Card → Click "Fund Milestone".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Stripe Payments Service.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe Elements SDK, Prisma ORM, and PostgreSQL database.

Purpose

Deposits milestone funds into secure platform escrow via Stripe credit card or saved payment method. Securing funds in escrow gives the Freelancer confidence to proceed with deliverable work, knowing payment is guaranteed upon completion approval.

Interface

- "Fund Milestone" action button on unfunded milestone cards.
- Fund Escrow modal dialog displaying milestone amount, escrow terms, and payment method selector.
- Stripe Elements embedded card input or saved card selector (e.g. "Visa ending in 4242").
- Action buttons: "Confirm & Pay $USD" and "Cancel".

Data Processing Overview

- Client initiates funding: requests `POST /api/payments/contracts/:contractId/milestones/:mile` or `/saved-card`.
- `PaymentService` validates caller is contract's Client and contract platform fee is paid.
- Verifies milestone is in `UNPAID` payment status.
- Generates a Stripe PaymentIntent configured with milestone escrow amount and metadata.
- Upon card confirmation, Stripe processes payment and triggers escrow deposit.
- Database updates milestone `paymentStatus = FUNDED` and transitions status to `IN_PROGRESS`.
- Creates ledger transaction record (`ESCROW_DEPOSIT`) and notifies Freelancer to begin work.

Screen Layout (Mock-up Description)

Figure 181: Fund Milestone escrow payment modal

- Modal displays title "Fund Milestone Escrow" and milestone name.

- Deposit amount card shows total amount to be held in secure escrow.
- Payment option: choose saved card or enter new credit card via Stripe Elements.
- Primary blue button "Deposit $X.00 into Escrow" with security lock badge.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).
`paymentMethodId`	Optional Stripe payment method ID for saved card charge (string).

Validation Rules

- Caller must be the contract's Client.
- Contract platform fee must be settled (`platformFeeStatus = PAID`).
- Milestone payment status must be `UNPAID`.

Business Rules

- **BR-PAY-05**: Milestone funds deposited into escrow are held securely until approved, refunded, or resolved via arbitration.
- **BR-PAY-06**: Escrow deposit automatically activates the milestone for work progress.

Normal Flow

1. Client clicks "Fund Milestone" on an unfunded milestone card.
2. System opens Fund Escrow modal with payment method options.
3. Client selects saved card or enters card details.
4. Client clicks "Confirm & Pay".
5. Stripe charges card and confirms PaymentIntent.
6. System marks milestone `paymentStatus = FUNDED` and status `IN_PROGRESS`.
7. Modal closes, card turns blue, and Freelancer is notified to start work.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is already funded. API returns HTTP 422 with error `MilestoneNotEligibleForFundingException`.
A2	Contract platform fee has not been paid. API returns HTTP 422; prompts to pay platform fee first.
A3	Card payment declined by issuer. Stripe returns card error; form prompts for alternative card.
A4	Caller is not the Client. API returns HTTP 403 Forbidden; access denied.

Output Data

Data Name	Description
`paymentIntent`	Stripe PaymentIntent confirmation object with status `succeeded`.
`milestoneStatus`	Updated milestone payment status (`FUNDED`).

Post-Conditions

- Milestone escrow funded and transaction logged.
- Milestone status transitioned to `IN_PROGRESS`.
- Freelancer notified.

3.22.5 Refund Payment

Function Trigger

A Client requests a refund for a cancelled milestone, or an Admin executes an agreed escrow refund.
Navigation path: Contract Detail → Milestones Tab → Funded Milestone → Click "Request Refund".

Function Description

Actors / Roles

- **Primary Actor:** Client, Administrator.
- **Secondary Actor:** Stripe Refund Gateway.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Refunds funded milestone escrow deposits back to the Client's original payment method when a milestone is mutually cancelled, terminated, or granted a refund under platform arbitration terms.

Interface

- Refund action button or modal in the contract management workspace.
- Refund confirmation dialog detailing refund amount ($USD), processing timeline (5–10 business days), and original payment method.
- Action buttons: "Confirm Refund" and "Cancel".

Data Processing Overview

- Client or Admin submits an HTTP request: `POST /api/payments/contracts/:contractId/mileston`
- `PaymentService.refundMilestone()` verifies authorization and milestone eligibility.
- Checks that milestone payment status is `FUNDED`; un-funded or already released milestones are rejected.
- Finds original `ESCROW_DEPOSIT` transaction and extracts Stripe PaymentIntent ID.
- Calls `stripe.refunds.create()` to initiate refund on original charge.
- Inside a database transaction, updates milestone `paymentStatus = REFUNDED` and records `REFUND` transaction.
- Sends refund receipt notification to the Client.

Screen Layout (Mock-up Description)

Figure 182: Refund Payment confirmation screen

- Modal displays refund icon and heading “Refund Milestone Escrow”.
- Summary details refund amount, original deposit date, and destination card.
- Warning notes that refund processing typically takes 5–10 days depending on the card issuer.
- Buttons: “Cancel” and primary orange “Confirm Refund”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Milestone payment status must be `FUNDED`.
- Milestone funds must not have already been released to the freelancer.
- Milestone must not have an active unresolved dispute.

Business Rules

- **BR-PAY-07**: Released milestone funds cannot be refunded via escrow refund; dispute arbitration is required.
- **BR-PAY-08**: Refunds are credited strictly back to the original funding payment method.

Normal Flow

1. User initiates refund on eligible funded milestone.
2. System displays refund confirmation dialog.
3. User clicks “Confirm Refund”.
4. Stripe initiates gateway refund on original charge.
5. System marks milestone `paymentStatus = REFUNDED` and logs refund transaction.
6. Success confirmation appears and Client receives email receipt.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is already released. API returns HTTP 422 with error `MilestoneAlreadyReleasedException`; refund blocked.
A2	Milestone is not funded. API returns HTTP 422 with error `MilestoneNotFundedException`.
A3	Stripe refund API fails. System displays gateway error and prompts retry.

Output Data

Data Name	Description
`refund`	Stripe refund entity with status `succeeded` and refund ID.
`milestoneStatus`	Updated payment status (`REFUNDED`).

Post-Conditions

- Milestone payment status updated to `REFUNDED`.
- Gateway refund initiated and ledger record logged.

3.22.6 Pay Platform Fee

Function Trigger

A Client clicks "Pay Platform Fee" or is redirected to checkout following mutual contract signing.
Navigation path: Contract Detail (`/client/contracts/:slug`) → Payment Banner → Click "Pay Platform Fee".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Stripe Checkout Service.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Collects the mandatory contract activation platform fee from the Client after both parties have signed the contract agreement. Payment of the platform fee activates the contract into operational status, unlocking milestone escrow funding.

Interface

- "Pay Platform Fee" banner in the contract workspace.
- Stripe Checkout redirect or embedded payment modal displaying platform fee breakdown ($10.00 standard or percentage-based).
- Saved card quick-charge option.
- Action button: "Pay Fee & Activate Contract".

Data Processing Overview

- Client requests `POST /api/payments/contracts/:contractId/platform-fee/checkout-session` or `/saved-card`.
- `PaymentService` validates caller is Client and contract is signed by both parties.
- Verifies `platformFeeStatus === 'UNPAID'`.
- Creates Stripe Checkout Session or charges saved card for platform fee.
- Upon successful payment:
  - Updates contract `platformFeeStatus = PAID` and sets `platformFeePaidAt = now()`.
  - Transitions contract `status = ACTIVE`.
  - Creates transaction record (`PLATFORM_FEE`).
- Alerts both parties that the contract is officially activated.

Screen Layout (Mock-up Description)

Figure 183: Pay Platform Fee checkout screen

- Workspace alert banner: “Contract signed! Pay platform fee to activate contract and begin milestones.”.
- Checkout modal itemizes Contract Activation Platform Fee ($10.00).
- Credit card inputs or saved card selector with “Pay $10.00” button.

Function Details

Input Data

Data Name	Description
contractId	Numeric contract identifier (integer path parameter).
paymentMethodId	Optional saved payment method identifier (string).

Validation Rules

- Contract must be signed by both Client and Freelancer.
- Platform fee must currently be in UNPAID status.
- Caller must be the contract's Client.

Business Rules

- **BR-PAY-09**: Contracts cannot transition to ACTIVE until the platform fee is settled.
- **BR-PAY-10**: Platform fees are non-refundable once the contract becomes active.

Normal Flow

1. Both parties sign contract; status transitions to AWAITING_PAYMENT.
2. Client clicks "Pay Platform Fee".
3. Client completes card checkout.
4. System marks platform fee as PAID and transitions contract to ACTIVE.
5. Both parties receive contract activation notification.

Abnormal Flow / Exception Handling

Case	Description
A1	Platform fee is already paid. API returns HTTP 422 with error PlatformFeeAlreadyPaidException.
A2	Contract not signed by both parties. API returns HTTP 422 with error ContractNotSignedException.
A3	Card payment declined. System prompts for alternative payment card.

Output Data

Data Name	Description
`contractStatus`	Updated contract status (`ACTIVE`).
`platformFeeStatus`	Updated fee status (`PAID`).

Post-Conditions

- Contract activated into `ACTIVE` status.
- Platform fee transaction recorded.

3.22.7 Release Payment

Function Trigger

A Client confirms milestone approval or explicitly clicks "Release Escrow Payment" on an approved milestone.
Navigation path: Contract Detail → Milestones Tab → Approved Milestone → Click "Release Payment".

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Secondary Actor:** Freelancer, Stripe Connect Transfer API.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Executes the financial disbursement of funded milestone escrow funds to the Freelancer's connected Stripe account. Deducts the platform service commission (10%) and transfers the net earnings directly to the Freelancer.

Interface

- "Release Payment" action button in milestone review toolbar.
- Payment release summary modal itemizing gross escrow amount, platform service fee, and net freelancer transfer.
- Action buttons: "Confirm & Transfer Funds" and "Cancel".

Data Processing Overview

- Client submits an HTTP request: `POST /api/payments/contracts/:contractId/milestones/:mile`
- `PaymentService.releaseMilestone()` verifies caller is Client and milestone escrow is `FUNDED`.
- Verifies Freelancer has completed Stripe Connect onboarding.
- Calls `stripe.transfers.create()` transferring net amount to `freelancer.stripeAccountId`.
- Inside database transaction:
  - Updates milestone `paymentStatus = RELEASED` and status `COMPLETED`.
  - Creates transaction records (`ESCROW_RELEASE` and `PAYOUT`).
- Dispatches instant payment receipt alert to the Freelancer.

Screen Layout (Mock-up Description)

Figure 184: Release Payment escrow disbursement modal

- Modal displays bank transfer graphic and heading “Release Escrow Funds”.
- Financial box renders Gross Amount, 10% Platform Commission, and Net Freelancer Payout.
- Confirmation button: “Confirm & Release Funds Now”.

Function Details

Input Data

Data Name	Description
`contractId`	Numeric contract identifier (integer path parameter).
`milestoneId`	Numeric milestone identifier (integer path parameter).

Validation Rules

- Milestone payment status must be `FUNDED`.
- Milestone funds must not have been previously released.
- Freelancer must have verified Stripe Connect payout capability.

Business Rules

- **BR-PAY-11**: Releasing milestone payment transfers funds irrevocably to the freelancer’s account.
- **BR-PAY-12**: The platform automatically retains the agreed service fee commission upon release.

Normal Flow

1. Client clicks “Release Payment” on approved milestone.
2. System opens release confirmation dialog.
3. Client clicks “Confirm & Transfer Funds”.
4. Stripe executes transfer to freelancer’s connected account.
5. System marks milestone `paymentStatus = RELEASED` and logs payout ledger record.
6. Freelancer receives real-time payment notification.

Abnormal Flow / Exception Handling

Case	Description
A1	Milestone is not funded. API returns HTTP 422 with error `MilestoneNotFundedException`.
A2	Freelancer Stripe account is incomplete. API returns HTTP 422 with error `StripeOnboardingIncompleteException`; release blocked.
A3	Duplicate release attempt. API returns HTTP 422 with error `MilestoneAlreadyReleasedException`.

Output Data

Data Name	Description
`transfer`	Stripe transfer confirmation object with transfer ID and net amount.
`milestoneStatus`	Updated payment status (`RELEASED`).

Post-Conditions

- Net earnings transferred to freelancer.
- Milestone payment status marked `RELEASED`.

3.22.8 Pay Dispute Fee

Function Trigger

A dispute participant (Client or Freelancer) clicks "Pay Arbitration Fee" in the dispute arbitration workspace.
Navigation path: Dispute Workspace (`/disputes/:id`) → Arbitration Fee Widget → Click "Pay Fee".

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer.
- **Secondary Actor:** Stripe Checkout Service.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Collects the formal arbitration deposit fee from dispute participants. Requiring an arbitration deposit ensures serious, legitimate dispute filings and covers administrative adjudication costs; the fee is refunded to the prevailing party upon settlement.

Interface

- Arbitration Fee notice box in the dispute workspace displaying fee amount ($25.00) and 24-hour payment deadline.
- Payment modal with Stripe Elements card input.
- Action buttons: “Pay Arbitration Fee ($25.00)” and “Cancel”.

Data Processing Overview

- Participant requests `POST /api/payments/disputes/:disputeId/fee/intent` or `/fee/checkout-session`.
- `PaymentService` validates caller is an involved dispute party.
- Checks that party’s dispute fee is currently `UNPAID`.
- Creates Stripe PaymentIntent or Checkout session for the arbitration fee.
- Upon card confirmation, updates `initiatorFeeStatus = PAID` or `respondentFeeStatus = PAID`.
- Records `DISPUTE_FEE` transaction in the ledger.

Screen Layout (Mock-up Description)

Figure 185: Pay Dispute Fee modal screen

- Modal displays legal scale graphic and heading “Pay Arbitration Fee”.
- Itemizes arbitration filing deposit ($25.00) and notes refund conditions.
- Card inputs and “Confirm Payment ($25.00)” primary action trigger.

Function Details

Input Data

Data Name	Description
disputeId	Numeric dispute identifier (integer path parameter).

Validation Rules

- Caller must be a recognized participant in the dispute.
- Party's dispute fee must be currently UNPAID.

Business Rules

- **BR-PAY-13**: Dispute participants must settle arbitration fees within the specified deadline to validate their claim.
- **BR-PAY-14**: Arbitration fees are refunded to the winning party upon formal dispute settlement.

Normal Flow

1. User clicks "Pay Arbitration Fee" in dispute workspace.
2. System opens payment modal with Stripe card fields.
3. User completes payment.
4. System records fee payment as PAID and logs transaction.
5. UI updates fee badge to green "Paid".

Abnormal Flow / Exception Handling

Case	Description
A1	Fee is already paid. API returns HTTP 422 with error DisputeFeeAlreadyPaidException.
A2	Caller is not a dispute party. API returns HTTP 403 Forbidden; access denied.
A3	Card payment declined. User prompted to enter alternative payment method.

Output Data

Data Name	Description
`feeStatus`	Updated fee status (`PAID`).
`transaction`	Ledger transaction record for dispute fee.

Post-Conditions

- Dispute arbitration fee marked `PAID`.
- Case advances in arbitration review queue.

3.22.9 View Payment History

Function Trigger

A user navigates to the "Billing & Transactions" section from the main dashboard navigation.
Navigation path: User Dashboard → Payments → Transactions (`/client/payments` or `/freelancer/payments`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Renders a comprehensive paginated ledger of all financial events associated with the user account, including milestone escrow deposits, platform fees, escrow payouts, refunds, and arbitration fees.

Interface

- Transactions dashboard view with filter bar.
- Transaction type filter tabs: All, Escrow Deposits, Platform Fees, Payout Releases, Refunds, Dispute Fees.
- Ledger table with columns: Date, Transaction ID, Type, Description, Amount ($USD), Status, and Action.
- Date range picker and keyword search input.
- Pagination controls footer.

Data Processing Overview

- Client requests `GET /api/payments/transactions` with query filters (`page`, `limit`, `type`, `status`).
- `PaymentService.getTransactions()` scopes query by caller ID (or all for Admin).
- Queries `prisma.transaction.findMany` with contract and milestone joins.
- Returns HTTP 200 with the transactions collection and count metadata.

Screen Layout (Mock-up Description)

Figure 186: Financial transactions ledger screen

- Header displays summary statistics for Total Spent, In Escrow, and Total Paid Out.
- Filter bar features type dropdown, date range picker, and export CSV button.
- Table renders financial rows with positive/negative colored amounts and status badges.

Function Details

Input Data

Data Name	Description
`page`	Page number (integer, default 1).
`limit`	Number of records per page (integer, default 10, max 50).
`type`	Optional transaction type filter (`ESCROW_DEPOSIT`, `ESCROW_RELEASE`, `PLATFORM_FEE`, `REFUND`, `DISPUTE_FEE`).

Validation Rules

- Caller must be authenticated with active session credentials.

Business Rules

- **BR-PAY-15**: Financial transactions form an append-only immutable ledger; entries are never hard-deleted.

Normal Flow

1. User navigates to Transactions dashboard.
2. Client requests transaction ledger from API.
3. API queries database and returns filtered records.
4. Client renders transaction table and summary metrics.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthenticated access. API returns HTTP 401; redirects to login.
A2	No records found. Client displays empty ledger placeholder.

Output Data

Data Name	Description
`transactions[]`	Array of transaction records with amount, type, status, and related contract/milestone info.
`pagination`	Object containing pagination metadata.

Post-Conditions

- Transaction history displayed.

3.22.10 View Payment Detail

Function Trigger

A user clicks on a transaction row in the payment history table.

Navigation path: Payment History → Click Transaction Row (`.../transactions/:id`).

Function Description

Actors / Roles

- **Primary Actor:** Client, Freelancer, Administrator.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Prisma ORM layer, and PostgreSQL database.

Purpose

Displays full itemized invoice and receipt data for a specific financial transaction, including payment intent ID, fee deductions, payment method details, and contract references.

Interface

- Transaction receipt modal dialog.
- Detailed receipt breakdown: Gross Amount, Fee Breakdown, Net Impact, and Status.
- Technical metadata: Stripe PaymentIntent/Charge ID, timestamp, and card brand/last4.
- Download receipt PDF button.

Data Processing Overview

- Client requests `GET /api/payments/transactions/:id`.
- `PaymentService` validates user ownership or Admin role.
- Queries `prisma.transaction.findUnique` with relations.
- Returns HTTP 200 with the transaction entity.

Screen Layout (Mock-up Description)

Figure 187: Transaction receipt detail modal

- Modal displays receipt layout with platform header and transaction reference.
- Line items show item description, milestone reference, and paid amount.
- Payment method snippet displays card brand icon and last 4 digits.

Function Details

Input Data

Data Name	Description
`id`	Numeric transaction identifier (integer path parameter).

Validation Rules

- Transaction ID must exist.
- Caller must be authorized.

Business Rules

- **BR-PAY-16**: Transaction details must accurately reflect gateway audit IDs.

Normal Flow

1. User clicks a transaction row.
2. System loads transaction details.
3. System opens receipt modal.

Abnormal Flow / Exception Handling

Case	Description
A1	Unauthorized access. API returns HTTP 403 Forbidden with error `PaymentForbiddenException`.
A2	Transaction not found. API returns HTTP 404 Not Found.

Output Data

Data Name	Description
`transaction`	Full transaction entity with itemized fees and gateway metadata.

Post-Conditions

- Transaction receipt displayed.

3.22.11 Execute Dispute Resolution Payment

Function Trigger

A System Administrator executes binding settlement disbursement in the dispute console, or both parties mutually accept an Admin proposal.
Navigation path: Admin Dashboard → Arbitration → Dispute Case (`/admin/disputes/:id`) → Click "Execute Settlement Disbursement".

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Secondary Actor:** Stripe Payment Gateway (Refund & Transfer APIs).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Executes the legally binding financial distribution of frozen milestone escrow funds determined by dispute resolution, splitting funds between a partial refund to the Client and a partial payout to the Freelancer.

Interface

- Dispute settlement execution panel in Admin arbitration view.
- Settlement disbursement confirmation modal itemizing Client Refund amount ($USD) and Freelancer Payout amount ($USD).
- Action buttons: "Execute Binding Disbursement" and "Cancel".

Data Processing Overview

- Administrator submits an HTTP request: `POST /api/payments/disputes/:disputeId/settle` with `clientRefundAmount` and `freelancerTransferAmount`.
- `PaymentService.settleDispute()` verifies caller possesses `ADMIN` role.
- Validates that total split amounts strictly equal the milestone escrow balance.
- If `clientRefundAmount > 0`, calls `stripe.refunds.create()` to refund client's card.
- If `freelancerTransferAmount > 0`, calls `stripe.transfers.create()` to transfer net payout to freelancer's connected account.
- Inside database transaction, sets dispute status to `RESOLVED`, unfreezes milestone, and records settlement ledger entries.
- Dispatches formal dispute resolution decrees to both parties.

Screen Layout (Mock-up Description)

Figure 188: Dispute resolution payment execution screen

- Modal displays red arbitration seal and heading “Execute Binding Settlement Payment”.
- Split summary: Client Refund ($Amount) and Freelancer Transfer ($Amount).
- Destructive button “Execute Irreversible Disbursement”.

Function Details

Input Data

Data Name	Description
`disputeId`	Numeric dispute identifier (integer path parameter).
`clientRefundAmount`	Amount to refund to client (decimal, ≥ 0).
`freelancerTransferAmount`	Amount to transfer to freelancer (decimal, ≥ 0).

Validation Rules

- Caller must possess verified `ADMIN` role.
- The sum of refund and transfer amounts must exactly match total milestone escrow.
- Dispute must be in resolved or ready-for-settlement status.

Business Rules

- **BR-PAY-17**: Executing a dispute settlement disbursement is permanent and unappealable.

Normal Flow

1. Admin opens resolved dispute case and clicks “Execute Settlement Disbursement”.
2. System opens confirmation modal showing calculated splits.
3. Admin clicks “Execute Binding Disbursement”.
4. Stripe executes refund and transfer calls.
5. System updates dispute status to `RESOLVED` and records settlement transactions.
6. Both parties receive resolution decrees and payment receipts.

Abnormal Flow / Exception Handling

Case	Description
A1	Split amounts do not match escrow balance. API returns HTTP 400 Bad Request with error `DisputeSettlementInvalidAmountException`.
A2	Caller is not an Administrator. API returns HTTP 403 Forbidden; access denied.
A3	Gateway transfer failure. System rolls back and reports gateway error.

Output Data

Data Name	Description
settlement	Settlement confirmation object with refund ID, transfer ID, and status.
disputeStatus	Updated dispute status (RESOLVED).

Post-Conditions

- Escrow funds split and disbursed.
- Dispute status updated to RESOLVED.

3.22.12 Handle Stripe Webhook Event

Function Trigger

Stripe asynchronously dispatches an HTTP POST request to the platform webhook endpoint upon payment lifecycle events.
Navigation path: External Payment Gateway → System Webhook Endpoint (POST /api/payments/webhook).

Function Description

Actors / Roles

- **Primary Actor:** Stripe Payment Gateway (Automated System).
- **Supporting Systems:** Frevia REST API (NestJS), Stripe Webhook Verifier, Prisma ORM, and PostgreSQL database.

Purpose

Securely receives and processes asynchronous payment events dispatched by Stripe (e.g. payment_intent.succeeded, payment_intent.payment_failed, checkout.session.completed, account.updated), guaranteeing eventual consistency between gateway state and the platform database.

Interface

- Automated API endpoint at POST /api/payments/webhook protected by HMAC SHA-256 signature verification.

Data Processing Overview

- Server receives raw body payload and stripe-signature header.

- `PaymentService.handleWebhook()` constructs and validates the event using `stripe.webhooks.constructEvent(payload, signature, secret)`.
- Dispatches event handling based on event type:
  - `payment_intent.succeeded`: Marks pending transaction `COMPLETED` and updates milestone/contract status.
  - `checkout.session.completed`: Completes platform fee or milestone funding session.
  - `account.updated`: Updates freelancer's `chargesEnabled` and `payoutsEnabled` status.
- Returns HTTP 200 with acknowledgment `{ received: true }`.

Screen Layout (Mock-up Description)

Figure 189: Stripe webhook events monitoring log

- Admin developer portal view displays the Stripe webhook events log.
- Table shows Event Type, Event ID, Processing Timestamp, and Status (200 OK).

Function Details

Input Data

Data Name	Description
`stripe-signature`	HMAC SHA-256 signature header generated by Stripe.
`payload`	Raw JSON event payload dispatched by Stripe.

Validation Rules

- Webhook signature header must match the configured webhook signing secret.
- Event timestamp must be fresh to prevent replay attacks.

Business Rules

- **BR-PAY-18**: Webhook handlers must be idempotent, safely processing duplicate events without re-applying state changes.

Normal Flow

1. Stripe dispatches webhook event to `/api/payments/webhook`.
2. Server verifies HMAC signature.
3. Server processes event payload and updates database records.
4. Server returns HTTP 200 acknowledgment.

Abnormal Flow / Exception Handling

Case	Description
A1	Signature invalid or missing. Server rejects with HTTP 400 Bad Request and error `WebhookSignatureInvalidException`.
A2	Unhandled event type. Server returns HTTP 200 acknowledgment without modifying state.

Output Data

Data Name	Description
`received`	Boolean acknowledgment confirmation (`true`).

Post-Conditions

- Database records synchronized with gateway event.

3.22.13 Sync Payment Intent Status

Function Trigger

A Client returns from a Stripe Checkout redirect, or manually triggers payment reconciliation.
Navigation path: Contract Detail → Return from Checkout → Automatic Sync Trigger (`POST /api/payments/checkout-sessions/:sessionId/sync`).

Function Description

Actors / Roles

- **Primary Actor:** Client (`CLIENT` role).
- **Supporting Systems:** Frevia Web application (Next.js), Frevia REST API (NestJS), Stripe API SDK, Prisma ORM, and PostgreSQL database.

Purpose

Immediately synchronizes local transaction and milestone states directly with Stripe API following checkout redirect, eliminating dependency on asynchronous webhook latency so the user immediately sees updated contract/milestone statuses.

Interface

- Checkout return page rendering loading spinner: "Verifying payment confirmation…".
- Success redirect to contract workspace with green confirmed banner.

Data Processing Overview

- Client requests `POST /api/payments/checkout-sessions/:sessionId/sync` or `/intents/:id/sync`.
- `PaymentService.syncCheckoutSession()` queries Stripe API directly via `stripe.checkout.sessio`
- Checks `session.payment_status === 'paid'`.
- If local transaction is pending, transitions transaction to `COMPLETED` and marks milestone `FUNDED` or platform fee `PAID`.

- Returns HTTP 200 with synced transaction entity.

Screen Layout (Mock-up Description)

Figure 190: Payment reconciliation confirmation screen

- Screen displays animated checkmark and heading “Payment Verified!”.
- Milestone escrow deposit status confirms funds secured.
- “Return to Contract Workspace” button.

Function Details

Input Data

Data Name	Description
`sessionId`	Stripe Checkout session identifier (string path parameter).

Validation Rules

- Caller must be the authenticated Client.
- Session ID must be valid in Stripe.

Business Rules

- **BR-PAY-19**: Synchronizing reconciles state immediately without waiting for webhook delivery.

Normal Flow

1. Client finishes checkout and returns to Frevia.
2. Client triggers session sync.
3. Server queries Stripe API directly and confirms payment.
4. Server updates local records and returns confirmation.
5. Client navigates to contract workspace with active milestone state.

Abnormal Flow / Exception Handling

Case	Description
A1	Payment session unpaid or expired. Server reports unpaid status; prompts client to complete payment.
A2	Invalid session ID. Server returns HTTP 404 Not Found.

Output Data

Data Name	Description
`status`	Synchronized payment status (`paid` / `succeeded`).
`transaction`	Updated transaction record.

Post-Conditions

- Local database synchronized with Stripe state immediately.

3.23 Job and Talent Recommendation

3.23.1 Function: Recommend Jobs and Freelancers

Function Trigger

Triggered automatically when a freelancer visits their dashboard, uploads/updates their CV, or when a client creates a new job posting.
Navigation path: Freelancer Dashboard → Recommended For You (`/freelancer/recommendations`) OR Client Job Overview → Recommended Candidates (`/client/jobs/:id/recommendations`).

Function Description

Actors / Roles

- **Primary Actor:** AI Recommendation Engine / System.
- **Secondary Actor:** Freelancer (receiving job matches) and Client (receiving candidate matches).
- **Supporting Systems:** Frevia Web application, Frevia REST API, SensitiveAI Vector Proxy, PostgreSQL with pgvector, and OpenAI text-embedding-3-small service.

Purpose

Automatically delivers highly relevant, semantic job recommendations to freelancers based on their profile and uploaded CV embeddings, and simultaneously recommends qualified freelancers to clients for their posted job requirements.

Interface

- "Recommended For You" widget on the Freelancer home workspace.
- "Recommended Candidates" tab on the Client Job Details management dashboard.
- Match percentage indicator badge (e.g., "98% Match") on each recommended card.

Data Processing Overview

- When a freelancer uploads a CV or updates their profile skills, the backend generates a dense vector embedding via the embedding model and stores it in the database.
- When generating recommendations for a freelancer, the system computes cosine similarity between the freelancer's embedding and active `OPEN` job embeddings.
- When generating candidates for a client's job, the system computes cosine similarity between the job description embedding and active freelancer profiles.
- Matches with a similarity score above threshold (≥ 0.75) are ranked in descending order and cached with a 6-hour TTL.

Screen Layout (Mock-up Description)

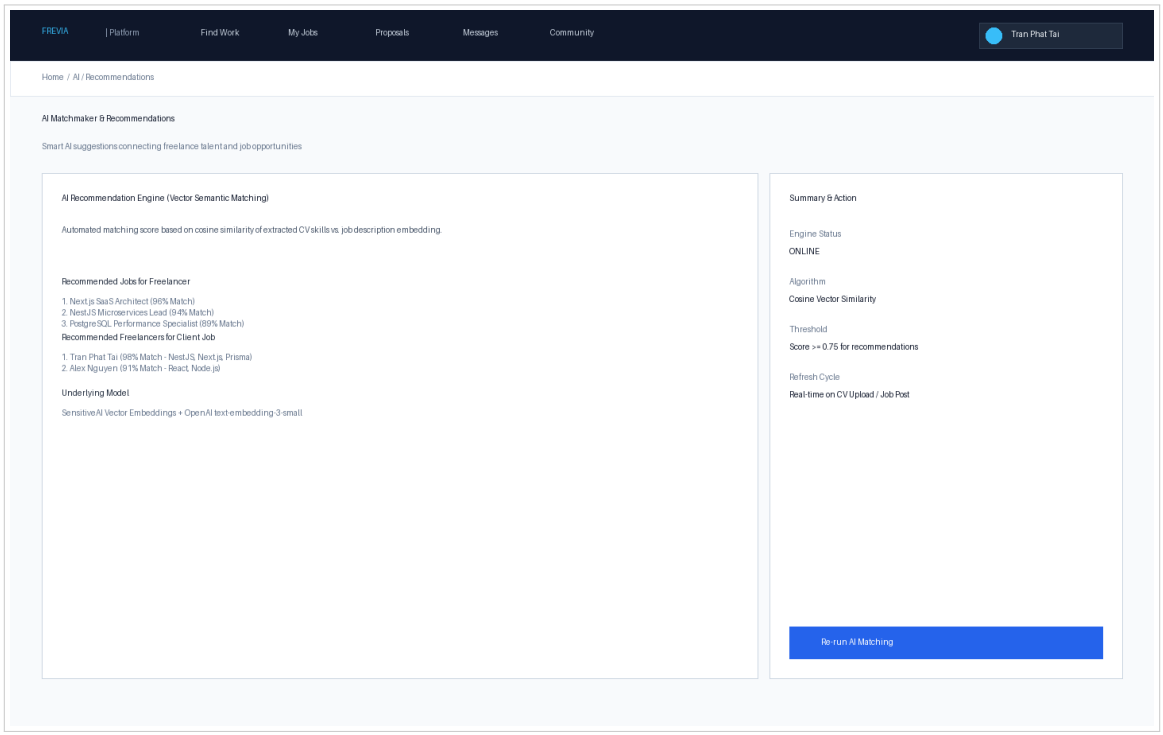

Figure 191: AI matching and recommendation engine view

- Top section summarizing AI vector matching status and model specifications.
- Two-column display: left column shows top matched jobs for freelancers, right column shows top candidate matches for clients.
- Match percentage badges (e.g. 96%, 94%, 89%) and one-click apply or invite actions.

Function Details

Input Data

Data Name	Description
User ID / Job ID	Target freelancer or job posting identifier.
`limit`	Number of recommendations to retrieve, default 10.

Validation Rules

- Only active users and non-deleted jobs are included in vector indexing.

Business Rules

- **BR-REC-01**: Vector embedding matching requires minimum cosine similarity threshold of 0.75.
- **BR-REC-02**: Only jobs with status `OPEN` are recommended to freelancers.
- **BR-REC-03**: Banned or suspended users are excluded from all recommendation pipelines.

Normal Flow

1. User opens dashboard or recommendation tab.
2. Frontend requests recommendations via `GET /api/recommendations`.
3. Backend executes vector similarity query against indexed embeddings.
4. Returns top ranked matches with match scores.
5. UI renders recommendations with match percentage badges.

Abnormal Flow / Exception Handling

Case	Description
A1	Vector service unreachable or user profile has no skills/CV yet. System falls back to recent popularity-based category recommendations.

Output Data

Data Name	Description
recommendations	Array of items containing entity data, match score (0.00–1.00), and top matched skill tags.

Post-Conditions

- Personalized recommendations are presented with match score transparency.

3.24 Job Category Management

3.24.1 View Job Category List

Function Trigger

An administrator navigates to the Job Categories management page from the admin dashboard sidebar navigation.
Navigation path: Admin Dashboard → Job Taxonomy → Job Categories (/admin/job-categories).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (ADMIN role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to retrieve and manage the list of job categories in the system, supporting paginated navigation, multi-field search, operational status filtering (ACTIVE or INACTIVE), soft-deletion status filtering, and sorting.

Interface

- Job category management screen at /admin/job-categories.
- Filter bar component (JobCategoriesFilterBar) with search box, status dropdown (All, Active, Inactive), soft-delete filter dropdown (All, Active Only, Deleted Only), sort column dropdown (id, createdAt, name), and sort direction toggle.
- Data table component (JobCategoriesTable) rendering category rows, slugs, descriptions, operational status badges, active job count badges, created dates, and action controls.
- Standard pagination footer displaying total category count and page selection buttons.
- "Create Job Category" action button in the page header.

Data Processing Overview

- The client initiates an HTTP request: `GET /api/admin/job-categories` with query parameters (`page`, `limit`, `search`, `status`, `deleted`, `sortBy`, `sortOrder`).
- The backend `AuthGuard` validates the administrator's JWT token, and `PermissionGuard` verifies administrative route authorization.
- `JobCategoriesAdminController.listJobCategories()` parses query parameters via `JobCategoryAdminQueryDto`.
- `JobCategoriesAdminRepository.listJobCategories()` queries `prisma.jobCategory.findMany` with:
  - Multi-field case-insensitive search matching on `name`, `slug`, or `description` if `search` is provided.
  - Exact filter on `status` (`ACTIVE` or `INACTIVE`) if selected.
  - Soft-deletion filter on `deletedAt`: if `deleted === 'true'`, filters `deletedAt != null`; if `deleted === 'false'`, filters `deletedAt = null`.
  - Aggregated job count via `_count: { select: { jobs: { where: { job: { deletedAt: null } } } } }`, counting only non-deleted active jobs linked via `JobJobCategory`.
  - Sorting on the requested field (`id`, `createdAt`, or `name`) in specified direction (`asc` or `desc`).
- Executes `prisma.jobCategory.count()` in parallel to determine total matching records.
- Returns HTTP 200 with the array of job categories and pagination metadata.
- The frontend renders the category table, badges, and contextual action menus.

Screen Layout (Mock-up Description)

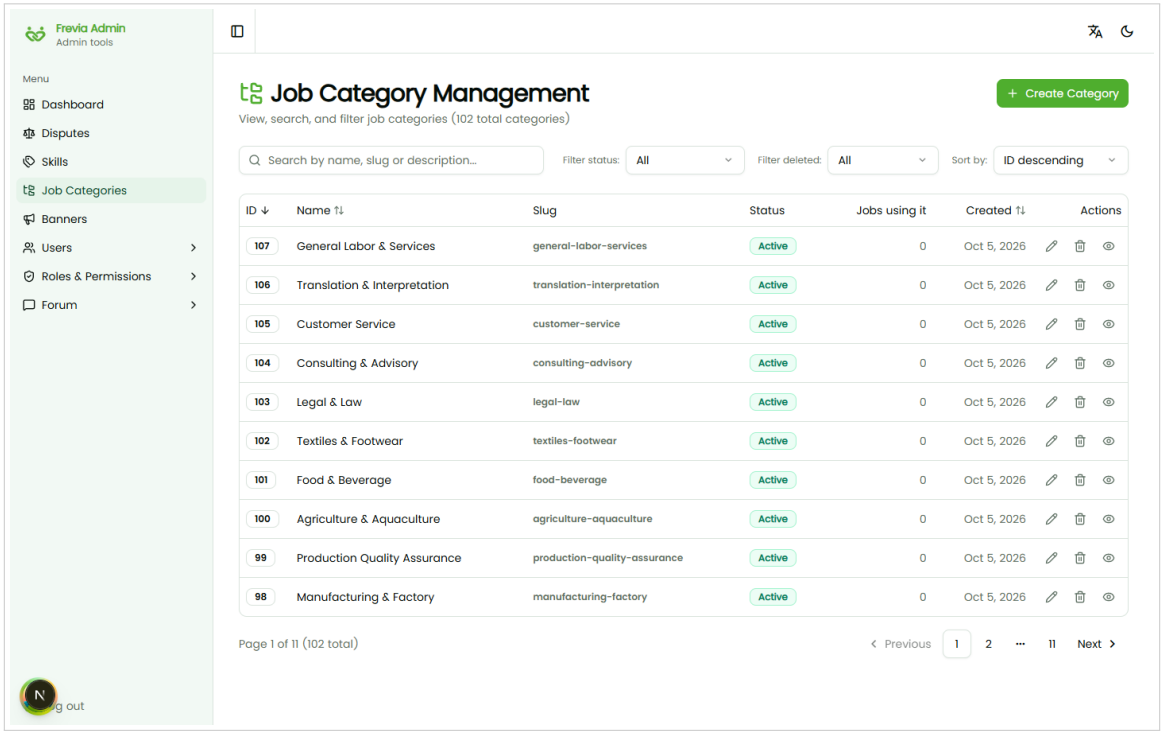

Figure 192: Job category list interface

- Page header displaying “Job Categories” title with folder tree icon, total count summary, and “Create Job Category” button.
- Filter bar including search box, status dropdown (All, Active, Inactive), deleted status dropdown (All, Active, Deleted), and sort direction buttons.
- Table columns: **Name** (with clickable detail link), **Slug**, **Description**, **Status** (Green “Active” or Gray “Inactive”), **In Use** (Badge showing count of active jobs using this category), **Created Date**, and **Actions**.
- Contextual action buttons: Update (pencil) and Delete (trash) for active categories; Restore (rotate-ccw) for deleted categories.
- Pagination footer indicating total count and page navigation controls.

Function Details

Input Data

Data Name	Description
page	Target page number (defaults to 1).
limit	Maximum records returned per page (defaults to 10).
search	Optional text query matching name, slug, or description.
status	Optional operational status filter (`ACTIVE` or `INACTIVE`).
deleted	Optional soft-deletion filter: `"true"` (deleted only) or `"false"` (active only).
sortBy	Sort column field (`id`, `createdAt`, or `name`; defaults to `id`).
sortOrder	Sort direction sequence (`asc` or `desc`; defaults to `desc`).

Validation Rules

Data Name	Validation Rule
page	Must be an integer greater than or equal to 1.
limit	Must be an integer between 1 and 100.
search	String, optional, max length 100 characters; whitespace trimmed.
status	Optional string, must strictly match one of: `ACTIVE`, `INACTIVE`.
deleted	Optional string, must strictly match one of: `"true"`, `"false"`.
sortBy	Must strictly be one of: `id`, `createdAt`, `name`.
sortOrder	Must strictly be one of: `asc`, `desc`.

Business Rules

- **BR-JCAT-01**: Admin Authorization: Access to the administrative job category catalog is strictly restricted to authenticated administrators.
- **BR-JCAT-02**: Multi-Field Case-Insensitive Search: Searches across `name`, `slug`, and `description` case-insensitively using partial substring matching.
- **BR-JCAT-03**: Active Job Count Link Filtering: Counts only jobs that are not soft-deleted (`job.deletedAt = null`) linked via the `JobJobCategory` relation table.
- **BR-JCAT-04**: Operational Status and Soft-Delete Filtering: Supports orthogonal filtering by operational state (`ACTIVE` or `INACTIVE`) and lifecycle deletion state (`deleted=true` or

`deleted=false`).

- **BR-JCAT-05**: Contextual Action Buttons: Active categories display green badges with Edit and Delete action buttons; soft-deleted categories display red badges with a Restore button.

Normal Flow

1. The administrator opens the `/admin/job-categories` page from the admin dashboard.
2. The client issues `GET /api/admin/job-categories` with default query parameters.
3. The backend validates administrative token and route permissions.
4. The repository queries job categories, counting active linked jobs and total matching records.
5. The server returns HTTP 200 with the categories array and pagination metadata.
6. The frontend renders the job category table and pagination controls.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The backend throws HTTP 403 Forbidden. The UI displays: "You do not have permission to access this page."
Invalid pagination parameter values	The DTO validator rejects non-integer values with HTTP 400 Bad Request. The client defaults to page 1.

Abnorma Case	Handling & System Response
Invalid operational status parameter	The validator rejects unknown status values with HTTP 400 Bad Request. The UI clears invalid status filter.
Invalid deleted filter parameter value	The validator rejects unknown values with HTTP 400 Bad Request. The UI clears invalid filter.
Database query error or timeout	The service throws `FailedToLoadJobCategoryListException` (HTTP 500). The UI displays: "Error.FailedToLoadJobCategoryList".
No categories match search or status	The query returns `jobCategories: []` and `total: 0`. The table displays an empty placeholder: "No job categories found."
Network connection timeout	The client displays an error toast notification and offers a reload button.

Output Data

Output Field	Description
jobCategories	Array of category objects: `id`, `name`, `slug`, `description`, `status`, `deletedAt`, `createdAt`, `updatedAt`, `jobCount`.
pagination.page	Current page number.
pagination.limit	Current limit per page.
pagination.total	Total job categories matching the query.
pagination.totalPages	Total calculated pages available.

Post-Conditions

The administrator inspects the job category catalog, views job usage metrics, and can initiate creation, inspection, editing, deletion, or restoration.

3.24.2 View Job Category Detail

Function Trigger

An administrator clicks on a job category row or name in the category list, navigating to `/admin/job-categories/[id]`.
Navigation path: Admin Dashboard → Job Taxonomy → Job Categories → Category Details (`/admin/job-categories/[id]`).

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to view comprehensive metadata of a specific job category, including its generated slug, full description, operational status, total count of active linked jobs, and a list of recently associated active jobs.

Interface

- Job category detail screen at `/admin/job-categories/[id]`.
- Navigation back button (“Back to Job Categories”).

- Header card displaying category name, slug, ID, status badge (`ACTIVE` or `INACTIVE`), deleted badge (if applicable), and action buttons (`JobCategoryActions`).
- Description card displaying the full category description or a placeholder if empty.
- Metrics cards displaying total jobs using this category, creation timestamp, and last updated timestamp.
- Related jobs data table displaying up to 10 latest active jobs linked to this category (columns: ID, Title, Slug, Job Status, and Created Date).

Data Processing Overview

- The client sends an HTTP request: `GET /api/admin/job-categories/:id`.
- The backend validates administrative token and route permissions.
- `JobCategoriesAdminController.getJobCategoryDetail()` parses `id` via `ParseIntPipe`.
- `JobCategoriesAdminRepository.getJobCategoryDetail(id)` queries `prisma.jobCategory.find` by `id`.
- If the category record is not found, the service throws `JobCategoryAdminNotFoundException` (HTTP 404).
- The repository queries `prisma.jobJobCategory.findMany` where `categoryId = id` and `job.deletedAt = null`, ordered by `job.createdAt desc`, taking up to 10 records.
- Returns HTTP 200 with the category detail object including `jobCount` and `jobs` array.
- The frontend renders the comprehensive detail layout, metric cards, and associated jobs table.

Screen Layout (Mock-up Description)

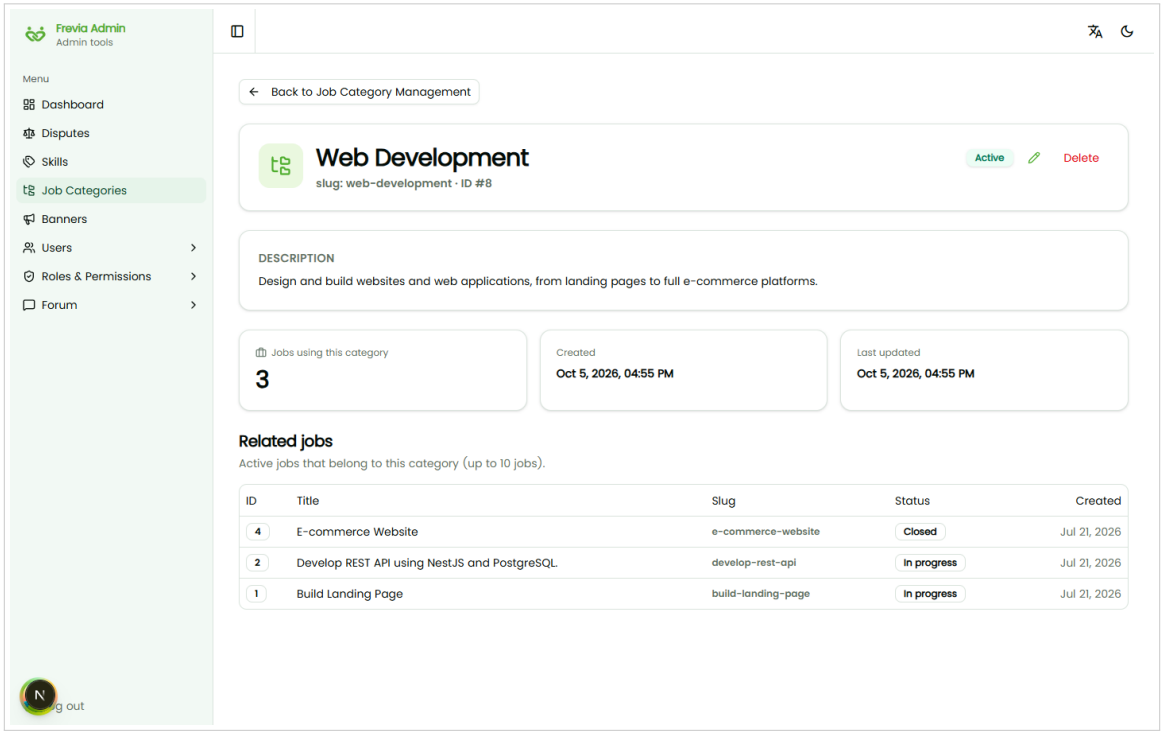

Figure 193: Job category detail interface

- Navigation header with "Back to Job Categories" button, category name with folder tree icon, and status badge.
- Action toolbar containing Edit (pencil) and Delete (trash) buttons if active, or Restore (rotate-ccw) button if soft-deleted.
- Summary card showing: **Slug** (e.g., `software-development`), **Status** (Active or Inactive), and **ID**.
- Description card displaying the category overview text.
- Metric cards: **Jobs Using This Category** (large number with briefcase icon), **Created Date**, and **Last Updated Date**.
- Associated Jobs table: Lists up to 10 latest active jobs with ID badge, Title, Slug, Status badge, and Created Date.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the job category (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-JCAT-06**: Identifier Format Validation: The URL path parameter `id` must parse strictly as a valid positive integer.
- **BR-JCAT-07**: Job Category Existence Check: The targeted job category must exist in the database; returns HTTP 404 (`Error.JobCategoryNotFound`) if missing.
- **BR-JCAT-08**: Associated Job Retrieval Limit: Returns up to 10 latest non-deleted active jobs linked to this category through `JobJobCategory`.
- **BR-JCAT-09**: Contextual Action Toolbar: Shows Edit and Delete actions if category is active; shows Restore action if category is soft-deleted.

Normal Flow

1. The administrator navigates to `/admin/job-categories/[id]`.
2. The client issues `GET /api/admin/job-categories/:id`.
3. The backend validates administrative authentication and permissions.
4. The repository queries the category entity and fetches related active jobs.
5. The server returns HTTP 200 with the category detail object and related jobs.
6. The frontend renders the comprehensive job category detail page and associated jobs table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.
Invalid category ID format (e.g., non-numeric)	`ParseIntPipe` throws HTTP 400 Bad Request. The UI displays an "Invalid Category ID" message and back button.
Category ID does not exist in database	The service throws `JobCategoryAdminNotFoundException` (HTTP 404). The UI renders a "Category Not Found" screen with navigation back to catalog.
Database query error or timeout	The service throws `FailedToLoadJobCategoryDetailException` (HTTP 500). The UI displays: "Error.FailedToLoadJobCategoryDetail".
Category description is empty or null	The UI gracefully displays "No description provided" in muted text.

Abnorma Case	Handling & System Response
No jobs associated with this category	The associated jobs table displays an empty row stating: “No jobs found in this category.”

Output Data

Output Field	Description
id	Unique numeric identifier of the job category.
name	Category name string.
slug	Generated URL-friendly slug.
description	Detailed description text or null.
status	Operational status enum (`ACTIVE` or `INACTIVE`).
deletedAt	Soft-deletion timestamp or null.
createdAt	Timestamp of category creation.
updatedAt	Timestamp of last modification.
jobCount	Total count of active jobs associated with this category.
jobs	Array of up to 10 latest active jobs: `id`, `title`, `slug`, `status`, `deletedAt`, `createdAt`.

Post-Conditions

The administrator reviews category details, operational status, and associated jobs, and can initiate editing, deletion, or restoration.

3.24.3 Create Job Category

Function Trigger

An administrator clicks the “Create Job Category” button on the category catalog page (`/admin/job-categories`), opening the creation dialog modal.

Navigation path: Admin Dashboard → Job Taxonomy → Job Categories → [Create Job Category Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to register a new job category by providing its name, optional description, and operational status, with automated unique slug generation.

Interface

- Job category creation dialog modal (`CreateJobCategoryDialog`).
- Text input for Category Name (mandatory).
- Textarea input for Description (optional).
- Select dropdown for Status (`ACTIVE` or `INACTIVE`, defaults to `ACTIVE`).
- Action buttons: "Cancel" and "Create Job Category" submit button with loading indicator.

Data Processing Overview

- The client submits an HTTP request: `POST /api/admin/job-categories` with JSON payload conforming to `AdminCreateJobCategoryBodySchema`.
- `JobCategoriesAdminRepository.createJobCategory()`:
  - Strips leading and trailing whitespace from `name`.
  - Queries `prisma.jobCategory.findFirst` to check if an active category (`deletedAt = null`) with the same name exists (case-insensitive mode).
  - If an active duplicate exists, throws `JobCategoryNameAlreadyExistsException` (HTTP 409).
  - Generates a unique slug via `generateUniqueSlug()`: strips Vietnamese diacritics, converts to lowercase, replaces non-alphanumeric characters with hyphens, and appends sequential numeric suffixes (`-2`, `-3`) if a slug collision occurs.
  - Inserts the category record via `prisma.jobCategory.create()`.
  - Catches Prisma `P2002` errors (unique constraint violations) and converts to `JobCategoryNameAlreadyExistsException`.
- Returns HTTP 201 with the created category entity.
- The frontend displays a success toast, closes the modal, and refreshes the category table.

Screen Layout (Mock-up Description)

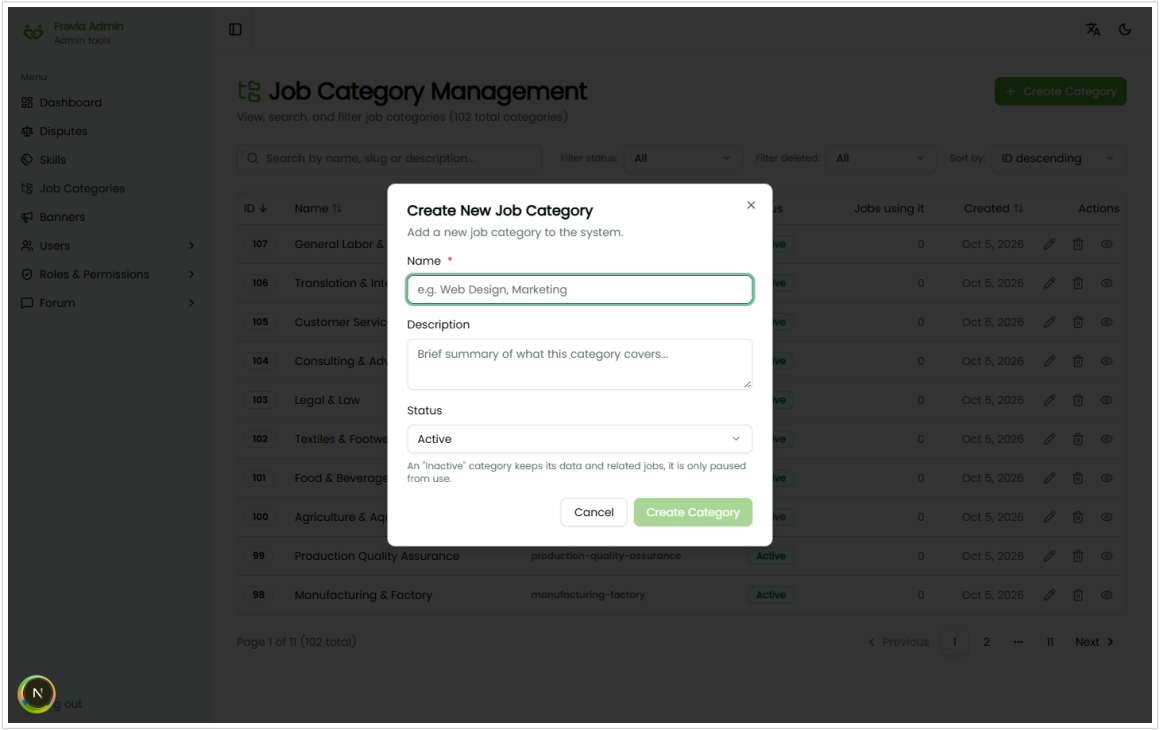

Figure 194: Create job category modal interface

- Dialog header with “Create Job Category” title and descriptive prompt.
- Form fields: **Name** (Required input field with asterisk), **Description** (Textarea), and **Status** (Select dropdown: Active / Inactive).
- Dialog footer with “Cancel” button and green “Create Job Category” button with loading spinner when submitting.

Function Details

Input Data

Data Name	Description
name	Job category name string (mandatory).
description	Optional text description of the category.
status	Operational status enum (`ACTIVE` or `INACTIVE`, defaults to `ACTIVE`).

Validation Rules

Data Name	Validation Rule
name	String, trimmed, min 1 char (`Error.JobCategoryNameRequired`), max 100 chars (`Error.JobCategoryNameTooLong`).
description	Optional string, max 500 chars (`Error.JobCategoryDescriptionTooLong`); nullable.
status	Optional enum, must strictly match one of: `ACTIVE`, `INACTIVE` (defaults to `ACTIVE`).

Business Rules

- **BR-JCAT-10**: Name Uniqueness Among Active Categories: Job category name must be unique among active categories (`deletedAt = null`), evaluated case-insensitively; throws HTTP 409 (`Error.JobCategoryNameAlreadyExists`) if conflict occurs.
- **BR-JCAT-11**: Automated Unique Slug Generation: Automatically generates a URL-friendly slug with diacritics stripping, hyphen replacement, and automatic numeric collision resolution (`-2`, `-3`).
- **BR-JCAT-12**: Default Operational Status: Newly created job categories default to `ACTIVE` status unless explicitly set to `INACTIVE`.
- **BR-JCAT-13**: Concurrency Race-Condition Guard: Catches database `P2002` unique constraint violations and converts them gracefully to HTTP 409 conflict errors.
- **BR-JCAT-14**: Modal Reset and Table Invalidation: Clears input values, closes modal dialog, and triggers Next.js router refresh upon success.

Normal Flow

1. The administrator clicks “Create Job Category” on the categories catalog page.
2. The dialog modal opens with empty input fields and status set to Active.
3. The administrator enters the category name, optional description, and chooses operational status.
4. The administrator clicks “Create Job Category”.
5. The client submits `POST /api/admin/job-categories`.
6. The backend validates data, checks for duplicate active names, generates a slug, and inserts the record.
7. The server returns HTTP 201 with the created category entity.
8. The client displays a success toast notification, closes the modal, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Missing mandatory category name	Form validation blocks submission; backend throws HTTP 400 with `Error.JobCategoryNameRequired`.
Category name exceeds 100 characters	Validator rejects input with HTTP 400 Bad Request (`Error.JobCategoryNameTooLong`). The UI highlights the name field.
Description exceeds 500 characters	Validator rejects input with HTTP 400 Bad Request (`Error.JobCategoryDescriptionTooLong`). The UI highlights the description field.
Active category with identical name already exists	The service throws `JobCategoryNameAlreadyExistsException` (HTTP 409 Conflict). The UI displays: "Error.JobCategoryNameAlreadyExists".
Concurrent request creates duplicate name	Database catches `P2002` constraint violation; backend converts to HTTP 409 Conflict. The UI alerts the administrator.

Abnorma Case	Handling & System Response
Invalid status enum parame-ter	Validator rejects invalid status strings with HTTP 400 Bad Request. The UI flags the status field.
Unauthorized or unau-thenti-cated request	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.
Database insert failure	The service throws `FailedToCreateJobCategoryException` (HTTP 500). The UI displays: “Error.FailedToCreateJobCategory”.

Output Data

Output Field	Description
id	Auto-generated unique identifier of the created category.
name	Stored category name string.
slug	Auto-generated unique slug.
description	Stored description string or null.
status	Operational status (`ACTIVE` or `INACTIVE`).
deletedAt	Null for newly created categories.
createdAt	Timestamp of creation.
updatedAt	Timestamp of creation.
jobCount	Zero for newly created categories.
jobs	Empty array for newly created categories.

Post-Conditions

A new job category record is stored in the database, indexed for search, and immediately available for classification of new job postings.

3.24.4 Update Job Category

Function Trigger

An administrator clicks the Edit icon (pencil) on a category table row or detail page, opening the category edit dialog modal.
Navigation path: Admin Dashboard → Job Taxonomy → Job Categories → [Edit Action Button].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to modify the name, description, and operational status (`ACTIVE` or `INACTIVE`) of an existing job category, automatically synchronizing its unique slug if the name is altered.

Interface

- Job category update dialog modal (`UpdateJobCategoryDialog`) pre-populated with current category name, description, and status.
- Text input for Category Name.
- Textarea input for Description.
- Select dropdown for Status (`ACTIVE` or `INACTIVE`).
- Action buttons: "Cancel" and "Save Changes" submit button with loading indicator.

Data Processing Overview

- The client pre-fills the form with the category's current values.
- The client issues an HTTP request: `PATCH /api/admin/job-categories/:id` with updated fields.
- `JobCategoriesAdminRepository.updateJobCategory(id, data)`:
  - Verifies target category exists and is active (`deletedAt = null`); throws `JobCategoryAdminNotFoundException` if missing or soft-deleted (soft-deleted categories must be restored before editing).
  - If `name` is modified, checks if another active category (`id != currentId`, `deletedAt = null`) shares the same case-insensitive name. Throws `JobCategoryNameAlreadyExis` if conflict exists.

- If `name` is modified, regenerates the unique slug via `generateUniqueSlug()`.
- Updates the record in `prisma.jobCategory.update()`. If description is empty string, converts it to `null`.

- The backend returns HTTP 200 with the updated category entity.
- The frontend displays a success toast, closes the modal, and refreshes the view.

Screen Layout (Mock-up Description)

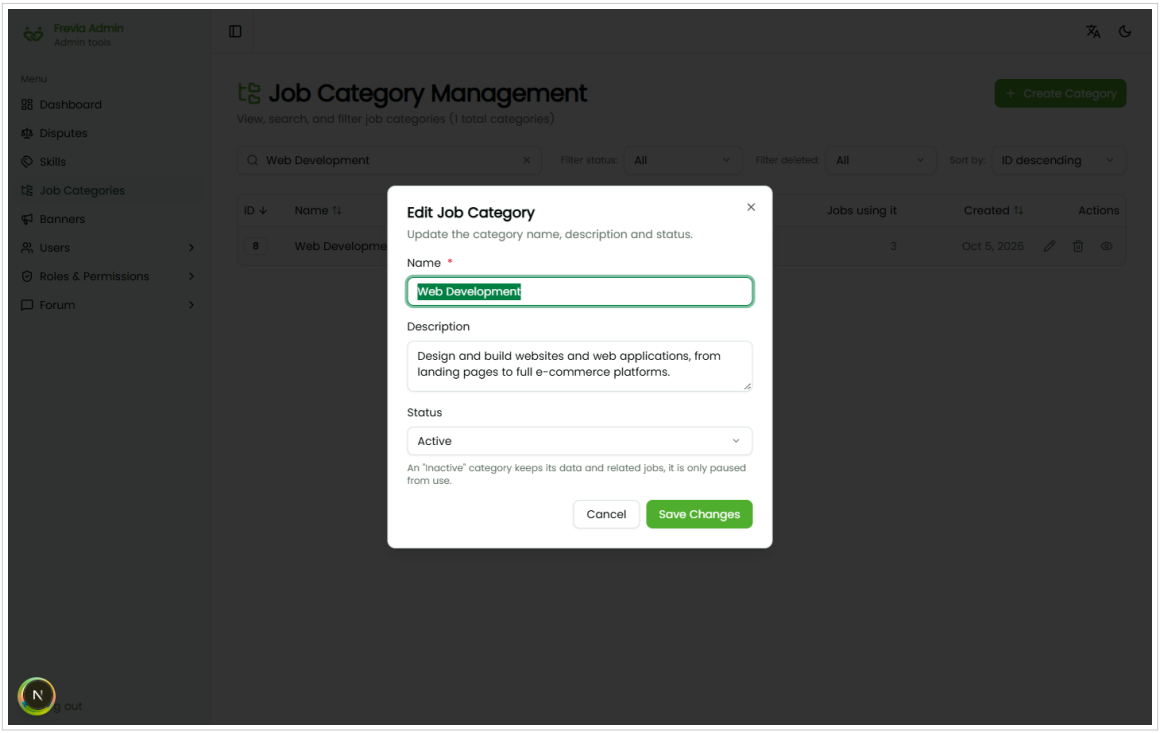

Figure 195: Update job category modal interface

- Modal dialog header with "Edit Job Category" title.
- Form pre-populated with current values: **Name** (Text input), **Description** (Textarea), and **Status** (Select dropdown: Active / Inactive).
- Footer action buttons: "Cancel" and "Save Changes" button with pending spinner.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the category (passed in URL path).
name	Updated category name string (optional).
description	Updated description string or null (optional).
status	Updated operational status enum (`ACTIVE` or `INACTIVE`, optional).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.
name	Optional string, trimmed, min 1 char (`Error.JobCategoryNameRequired`), max 100 chars (`Error.JobCategoryNameTooLong`).
description	Optional string, max 500 chars (`Error.JobCategoryDescriptionTooLong`); nullable.
status	Optional enum, must strictly match one of: `ACTIVE`, `INACTIVE`.

Business Rules

- **BR-JCAT-15**: Target Category Active State Requirement: Only active categories (`deletedAt = null`) can be updated; soft-deleted categories must be restored first.
- **BR-JCAT-16**: Name Conflict Detection on Modification: If name is altered, verifies no other active category shares the same case-insensitive name; throws HTTP 409 if conflict detected.
- **BR-JCAT-17**: Automatic Slug Regeneration: If the name is changed, automatically regenerates a new unique URL-friendly slug based on the updated name.
- **BR-JCAT-18**: Status Transition Freedom: Administrators can freely switch status between `ACTIVE` and `INACTIVE` at any time to temporarily suspend new job associations without deleting data.
- **BR-JCAT-19**: Synchronized UI Updates: Closes modal dialog, triggers success toast, and refreshes category table or detail screen upon completion.

Normal Flow

1. The administrator clicks the Edit icon on a category row or detail page.
2. The dialog modal opens pre-filled with the current category name, description, and status.
3. The administrator modifies the name, description, or status.
4. The administrator clicks “Save Changes”.
5. The client submits `PATCH /api/admin/job-categories/:id`.
6. The backend validates fields, checks for duplicate active names, regenerates slug if name changed, and updates the database.
7. The backend returns HTTP 200 with the updated category entity.
8. The frontend closes the dialog, displays a success toast, and refreshes the view.

Abnormal Flow / Exception Handling

Abnorma Case	**Handling & System Response**
Target category does not exist or is soft-deleted	The service throws `JobCategoryAdminNotFoundException` (HTTP 404). The UI displays: “Error.JobCategoryNotFound”.
Updated name conflicts with active category	The service throws `JobCategoryNameAlreadyExistsException` (HTTP 409). The UI alerts: “Error.JobCategoryNameAlreadyExists”.
Name is set to empty or whitespace	Form validation blocks submission; backend returns HTTP 400 Bad Request with `Error.JobCategoryNameRequired`.

Abnorma Case	Handling & System Response
Name exceeds 100 charac-ters	Validator rejects input with HTTP 400 Bad Request (`Error.JobCategoryNameTooLong`). The UI highlights the name field.
Description exceeds 500 charac-ters	Validator rejects input with HTTP 400 Bad Request (`Error.JobCategoryDescriptionTooLong`). The UI highlights the description field.
Unauthorized or unau-thenti-cated session	The server returns HTTP 401/403. The UI redirects to login or displays an access denied message.
Database update failure	The service throws `FailedToUpdateJobCategoryException` (HTTP 500). The UI displays: "Error.FailedToUpdateJobCategory".
Network timeout	The client catches the network error, displays an error toast, and preserves dialog state.

Output Data

Output Field	Description
id	Numeric identifier of the updated category.
name	Updated category name.
slug	Updated or preserved unique slug.
description	Updated description or null.
status	Updated operational status (`ACTIVE` or `INACTIVE`).
deletedAt	Preserved soft-deletion status.
updatedAt	Updated modification timestamp.
jobCount	Count of active jobs using this category.

Output Field	Description
jobs	Array of related jobs.

Post-Conditions

The category record and its generated slug are updated in the database, with immediate propagation across search queries and job submission forms.

3.24.5 Delete Job Category

Function Trigger

An administrator clicks the Delete action icon (trash can) on a category table row or detail page, opening the deletion confirmation dialog.
Navigation path: Admin Dashboard → Job Taxonomy → Job Categories → [Delete Action Icon].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to soft-delete a job category after verifying that no active jobs are currently associated with it, preventing accidental breaking of active job listings.

Interface

- Delete job category confirmation dialog (`DeleteJobCategoryDialog`) powered by Shadcn `AlertDialog`.
- Header displaying category name and warning icon if currently utilized by jobs.
- Dependent jobs warning banner if `jobCount > 0`, displaying exact number of linked active jobs.
- In-dialog conflict notification banner when HTTP 409 occurs, suggesting the administrator change the category status to `INACTIVE` instead.
- Action buttons: “Cancel” and destructive “Delete” button with pending spinner.

Data Processing Overview

- The administrator confirms deletion in the dialog.
- The client sends an HTTP request: `DELETE /api/admin/job-categories/:id`.
- `JobCategoriesAdminController.deleteJobCategory(id)` parses `id` via `ParseIntPipe`.
- `JobCategoriesAdminRepository.deleteJobCategory(id)`:
  - Verifies target category exists and is active (`deletedAt = null`); throws `JobCategoryAdminNotFoundException` if missing.
  - Checks count of active associated jobs: `_count: { select: { jobs: { where: { job: { deletedAt: null } } } } }`.
  - If `_count.jobs > 0`, throws `JobCategoryHasJobsException` (HTTP 409 Conflict with message `Error.JobCategoryHasJobs`). Deletion is strictly blocked.
  - If `_count.jobs === 0`, executes soft delete: `prisma.jobCategory.update({ where: { id }, data: { deletedAt: new Date() } })`.
- The backend returns HTTP 200 with `{ message: "Success.JobCategoryDeleted" }`.
- The frontend shows a success toast, closes the dialog, and refreshes the category table.

Screen Layout (Mock-up Description)

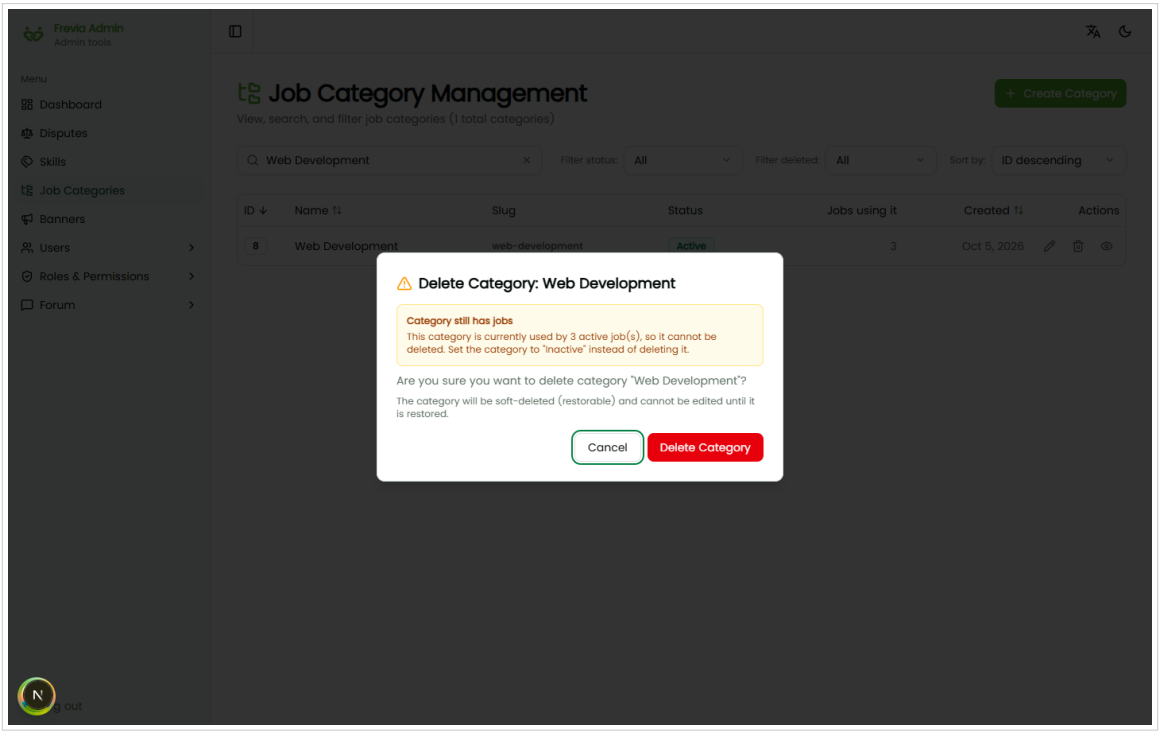

Figure 196: Delete job category confirmation dialog

- Dialog header with "Delete Job Category" title and target category name.
- Warning alert banner (when `jobCount > 0`) stating: "Category in use: This category is currently used by X jobs."

- Inline conflict callout (when deletion blocked) explaining that active jobs prevent deletion and advising switching status to Inactive instead.
- Soft-deletion explanation text stating that the category can be restored at any time.
- Footer action buttons: “Cancel” button and red “Delete” button with loading spinner.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the category to delete (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-JCAT-20**: Active Job Dependency Deletion Guard: Deleting a job category is strictly blocked if one or more active jobs are linked to it (`_count.jobs > 0`); the system throws HTTP 409 (`Error.JobCategoryHasJobs`).
- **BR-JCAT-21**: Inactive Status Recommendation: If active jobs are attached, the UI advises administrators to set category status to `INACTIVE` instead of deleting it.
- **BR-JCAT-22**: Soft-Deletion Lifecycle: Categories without active jobs are soft-deleted by setting `deletedAt = new Date()` without purging relational records.
- **BR-JCAT-23**: Explicit Confirmation Modal: Deletion requires explicit confirmation via modal alert dialog to prevent accidental catalog damage.
- **BR-JCAT-24**: Immediate List Synchronization: Purges the category from default active views immediately upon successful soft deletion.

Normal Flow

1. The administrator clicks the Delete icon on a category table row or detail screen.
2. The deletion confirmation dialog appears, displaying job usage warning if applicable.
3. The administrator reviews the warning and confirms by clicking “Delete”.
4. The client issues `DELETE /api/admin/job-categories/:id`.
5. The backend validates administrative authentication and checks for active linked jobs.

6. Since active job count is 0, the repository updates `deletedAt = new Date()`.
7. The backend returns HTTP 200 with `{ message: "Success.JobCategoryDeleted" }`.
8. The frontend displays a success toast, closes the dialog, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Category has active linked jobs	The service throws `JobCategoryHasJobsException` (HTTP 409 Conflict). The dialog renders an inline callout advising switching to Inactive.
Target category does not exist	The service throws `JobCategoryAdminNotFoundException` (HTTP 404). The UI displays: "Error.JobCategoryNotFound".
Category is already soft-deleted	The service throws HTTP 404. The UI displays that the category is no longer active.
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.

Abnorma Case	Handling & System Response
Invalid category ID format	`ParseIntPipe` throws HTTP 400 Bad Request. The UI notifies the user of an invalid parameter.
Administrator cancels dialog	The modal closes immediately; no request is sent, leaving the database unaffected.
Database update failure	The service throws `FailedToDeleteJobCategoryException` (HTTP 500). The UI displays: "Error.FailedToDeleteJobCategory".

Output Data

Output Field	Description
message	Success confirmation message string (`Success.JobCategoryDeleted`).

Post-Conditions

The category is marked as soft-deleted (`deletedAt != null`), hidden from default active listings, and can be restored if needed.

3.24.6 Restore Job Category

Function Trigger

An administrator viewing soft-deleted job categories (via the "Deleted" status filter or detail page) clicks the Restore icon (rotate-ccw), opening the restore confirmation dialog.
Navigation path: Admin Dashboard → Job Taxonomy → Job Categories → [Restore Action Icon].

Function Description

Actors / Roles

- **Primary Actor:** System Administrator (`ADMIN` role).
- **Supporting Systems:** Frevia Web management console (Next.js), Frevia REST API service (NestJS), Prisma ORM layer, and PostgreSQL relational database.

Purpose

Allows administrators to restore a previously soft-deleted job category by resetting its `deletedAt` timestamp to null, making it active and selectable again across the platform.

Interface

- Restore job category confirmation dialog (`RestoreJobCategoryDialog`) powered by Shadcn `AlertDialog`.
- Header displaying "Restore Job Category" title and target category name.
- Descriptive body explaining that restoring will make the category available again for job classification.
- Action buttons: "Cancel" and green "Restore" button with pending spinner.

Data Processing Overview

- The administrator confirms restoration in the dialog.
- The client sends an HTTP request: `PATCH /api/admin/job-categories/:id/restore`.
- `JobCategoriesAdminController.restoreJobCategory(id)` parses `id` via `ParseIntPipe`.
- `JobCategoriesAdminRepository.restoreJobCategory(id)`:
  - Finds the category verifying that `deletedAt != null`; throws `JobCategoryAdminNotFoundExcept` if missing or already active.
  - Checks if another active category has been created with the same name during the deletion period (`deletedAt = null, id != currentId, name equals category.name`).
  - If a name collision occurs, throws `JobCategoryNameAlreadyExistsException` (HTTP 409).
  - Sets `deletedAt = null` via `prisma.jobCategory.updateMany({ where: { id, deletedAt: { not: null } }, data: { deletedAt: null } })`.
  - Queries the updated category entity with job count aggregation.
- The backend returns HTTP 200 with the restored category entity.
- The frontend shows a success toast notification, closes the dialog, and refreshes the view.

Screen Layout (Mock-up Description)

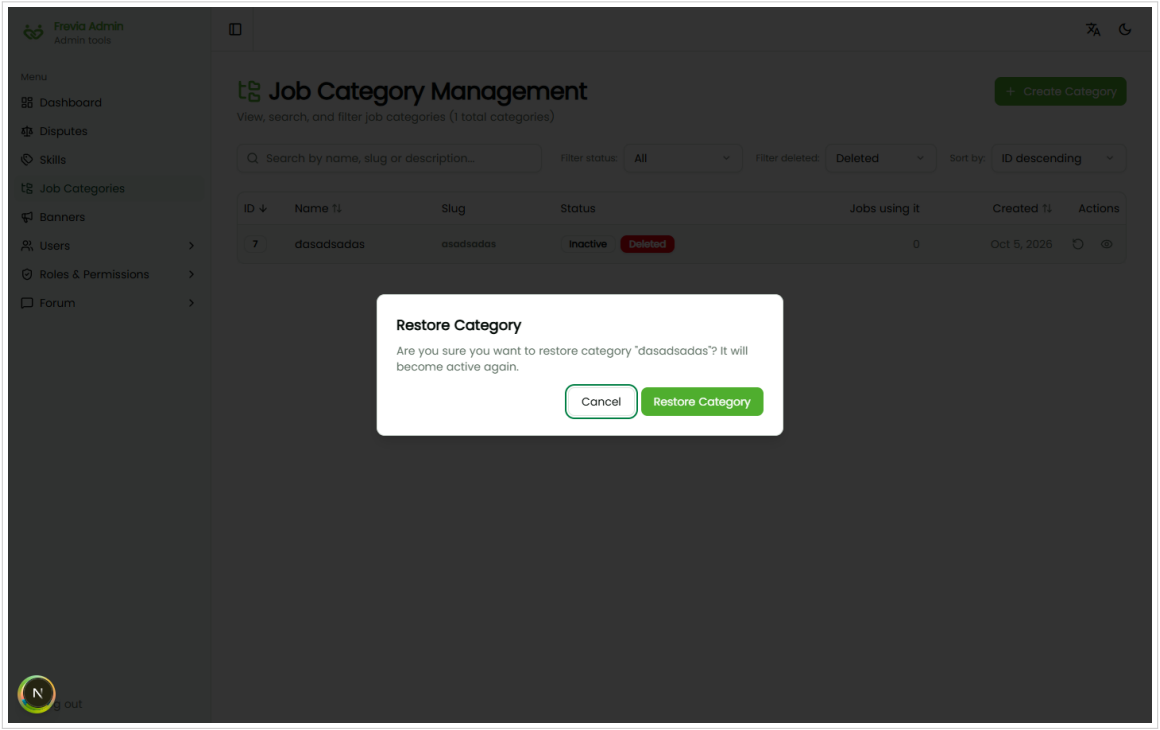

Figure 197: Restore job category confirmation dialog

- Alert dialog header displaying “Restore Job Category” title with rotate-counterclockwise icon.
- Confirmation description stating: “Are you sure you want to restore job category [Category Name]? It will become active and available for users again.”
- Footer controls: “Cancel” button and green “Restore” button with loading spinner when executing.

Function Details

Input Data

Data Name	Description
id	Unique numeric identifier of the soft-deleted category (passed in URL path).

Validation Rules

Data Name	Validation Rule
id	Must be a positive integer greater than or equal to 1.

Business Rules

- **BR-JCAT-25**: Restoration Eligibility Requirement: Target category must currently be soft-deleted with `deletedAt != null`; returns HTTP 404 if not found or already active.
- **BR-JCAT-26**: Conflict Prevention Against Interim Duplicates: If another active category with the same name was created while this category was deleted, restoration is blocked with HTTP 409 (`Error.JobCategoryNameAlreadyExists`).
- **BR-JCAT-27**: Soft-Deletion Timestamp Reset: Clears `deletedAt = null`, fully restoring the category to active catalog availability.
- **BR-JCAT-28**: Real-Time UI Re-indexing: Returns restored category data, displays success toast, and updates table view immediately.

Normal Flow

1. The administrator filters the catalog by "Deleted" job categories.
2. The administrator locates the soft-deleted category and clicks the Restore icon.
3. The restore confirmation dialog appears.
4. The administrator confirms restoration by clicking "Restore".
5. The client sends `PATCH /api/admin/job-categories/:id/restore`.
6. The backend verifies the category is soft-deleted, checks for name conflicts with active categories, and clears `deletedAt`.
7. The backend returns HTTP 200 with the restored category data.
8. The client displays a success toast, closes the dialog, and refreshes the table.

Abnormal Flow / Exception Handling

Abnorma Case	Handling & System Response
Target category does not exist or is already active	The service throws `JobCategoryAdminNotFoundException` (HTTP 404). The UI displays: “Error.JobCategoryNotFound”.
Another active category now shares the same name	The service throws `JobCategoryNameAlreadyExistsException` (HTTP 409). The UI alerts: “Error.JobCategoryNameAlreadyExists”.
Missing or invalid authentication token	The server returns HTTP 401 Unauthorized. The client redirects to `/login`.
User lacks Admin privileges	The server throws HTTP 403 Forbidden. The client renders an access denied alert.
Invalid category ID format	`ParseIntPipe` throws HTTP 400 Bad Request. The UI notifies the user of an invalid parameter.
Administrator cancels dialog	The modal closes immediately; no request is sent, leaving the database unaffected.

Abnorma Case	Handling & System Response
Database update failure	The service throws `FailedToRestoreJobCategoryException` (HTTP 500). The UI displays: “Error.FailedToRestoreJobCategory”.
Network failure during request	The client catches the network error, displays an unexpected error toast, and preserves dialog state.

Output Data

Output Field	Description
id	Numeric identifier of the restored category.
name	Category name string.
slug	Preserved or updated unique slug.
description	Description text or null.
status	Operational status (`ACTIVE` or `INACTIVE`).
deletedAt	Null indicating active status.
createdAt	Timestamp of original creation.
updatedAt	Timestamp of restoration.
jobCount	Count of active jobs referencing this category.
jobs	Array of associated jobs.

Post-Conditions

The job category is restored to active status (`deletedAt = null`), displayed in the active category catalog, and available for selection when posting and filtering jobs.

3.25 Job Category Browsing

3.25.1 Function: View Job Category List

Function Trigger

A freelancer or guest clicks on “Categories” or “Explore Disciplines” from the main navigation.
Navigation path: Home → Browse Categories (`/categories`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Guest.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Presents the public directory of all active job categories and technical disciplines, displaying the count of open jobs and trending skills within each domain.

Interface

- Category directory landing screen at `/categories`.
- Grid of category cards with icon, description, active job count badge, and top skill tags.

Data Processing Overview

- Frontend requests `GET /api/job-categories`.
- Backend queries active categories where `deletedAt = null` and `status = 'ACTIVE'`.
- Aggregates the count of published jobs currently in `OPEN` status for each category.
- Returns sorted category array.

Screen Layout (Mock-up Description)

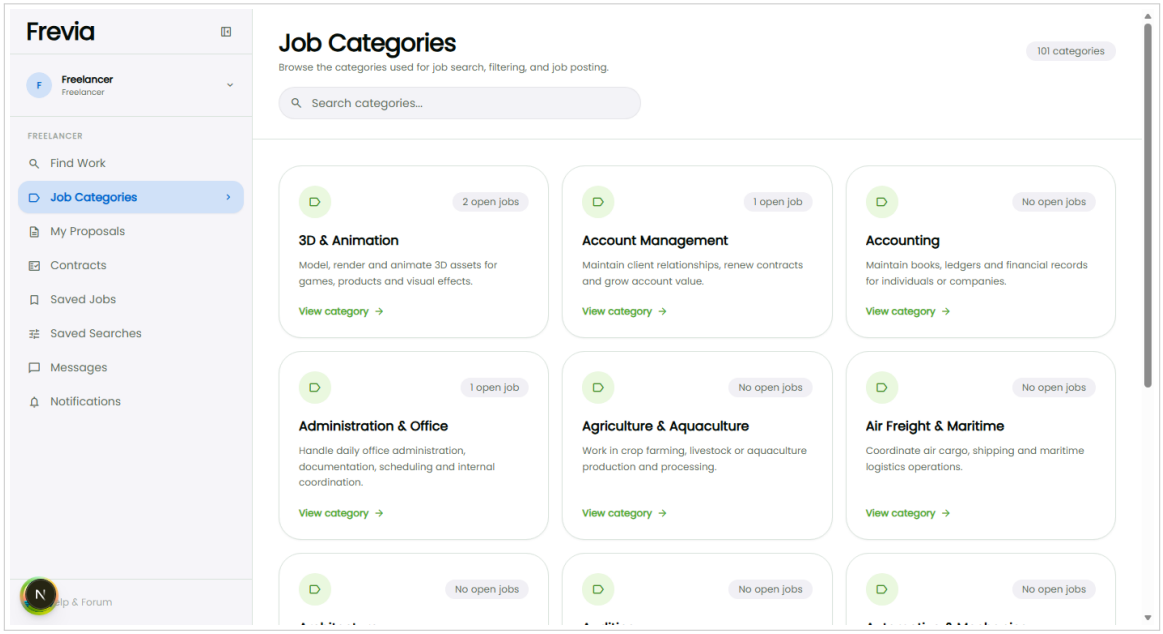

Figure 198: Public job categories directory interface

- Directory header explaining marketplace disciplines.
- Category cards showing discipline name, description, active jobs badge, and popular skills.
- Direct link to view all jobs within each discipline.

Function Details

Input Data

Data Name	Description
Endpoint	`GET /api/job-categories`. No query parameters required.

Validation Rules

- Publicly accessible to both guests and authenticated users.

Business Rules

- **BR-CAT-01**: Only categories with `status = 'ACTIVE'` and `deletedAt = null` are returned.
- **BR-CAT-02**: `jobCount` exclusively tallies jobs in `OPEN` status.

Normal Flow

1. User opens Categories directory.
2. Frontend issues `GET /api/job-categories`.
3. Backend fetches active categories with job aggregates.
4. UI renders interactive category cards.

Abnormal Flow / Exception Handling

Case	Description
A1	Database connection failure. Backend returns HTTP 500; frontend displays retry message.

Output Data

Data Name	Description
`categories`	Array of objects (`id`, `name`, `slug`, `description`, `jobCount`).

Post-Conditions

- Directory of active job categories is displayed.

3.25.2 Function: View Job Category Detail

Function Trigger

A user clicks on a category card in the categories directory.

Navigation path: Browse Categories (`/categories`) → Click Category (`/categories/:slug`).

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Guest.
- **Supporting Systems:** Frevia Web application, Frevia REST API, and PostgreSQL database.

Purpose

Displays information about a specific job category and provides a filtered catalog of all active jobs listed under that discipline.

Interface

- Category detail and filtered jobs view at `/categories/:slug`.
- Category banner with description and total available jobs.
- Job listings feed filtered strictly by this category.

Data Processing Overview

- Frontend requests `GET /api/job-categories/:slug`.
- Backend validates that category exists, is active, and not deleted.
- Queries associated active jobs in `OPEN` status.
- Returns category metadata along with paginated jobs.

Screen Layout (Mock-up Description)

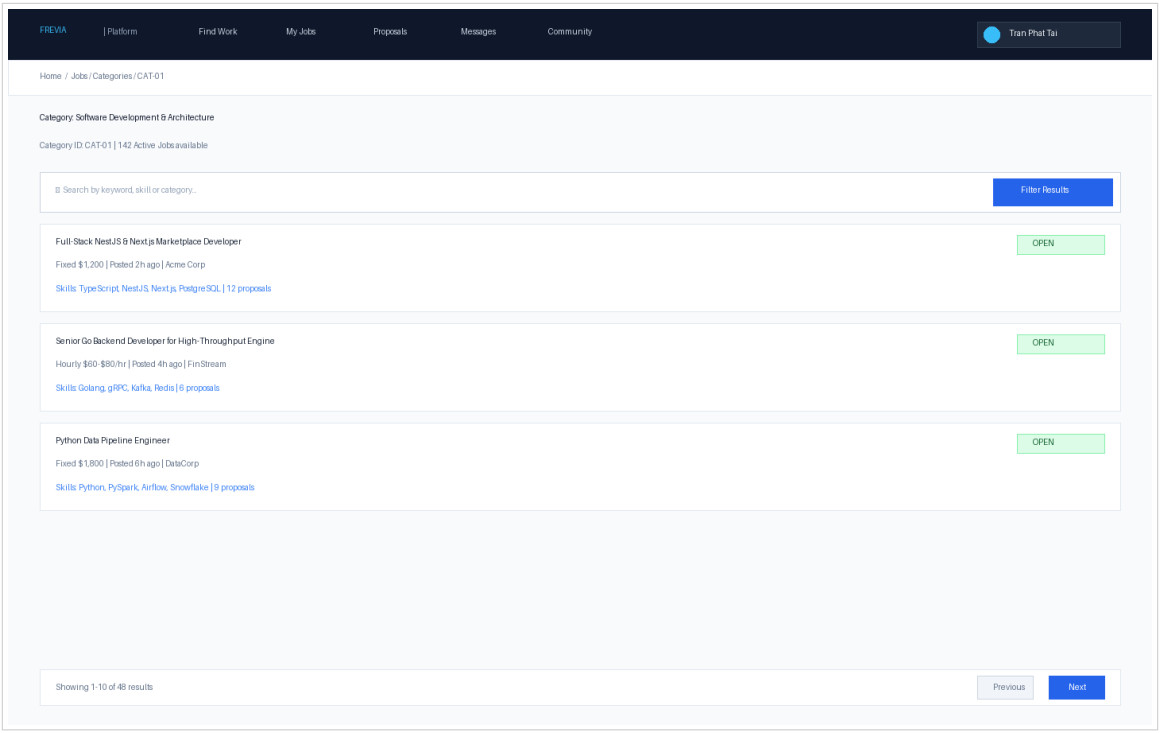

Figure 199: Category detail and jobs feed interface

- Header showing category title, description, and job count badge.
- Filtered job cards matching this technical discipline.
- Standard pagination bar.

Function Details

Input Data

Data Name	Description
`slug`	String identifier for the category in URL.
`page` / `limit`	Standard pagination numbers.

Validation Rules

- Category must be active and valid.

Business Rules

- **BR-CAT-03**: Accessing an inactive or soft-deleted category returns HTTP 404.
- **BR-CAT-04**: Only jobs with status `OPEN` are displayed in the category feed.

Normal Flow

1. User clicks on a category card.
2. Backend returns category details and matching active jobs.
3. UI renders category header and paginated job list.

Abnormal Flow / Exception Handling

Case	Description
A1	Category slug does not exist or category was deleted. Backend returns HTTP 404 Not Found.

Output Data

Data Name	Description
Category Detail	Object with `id`, `name`, `description`, and array of matching `jobs`.

Post-Conditions

- Category details and filtered job opportunities are presented.

4. Non-Functional Requirements

This section describes the non-functional requirements of the Frevia platform: the external interfaces through which the system communicates, and the quality attributes that the delivered system must satisfy.

4.1 External Interfaces

The system communicates with four kinds of external element. Each interface is listed below with the constraint that the implementation must respect.

- **User interfaces.** The platform is delivered as a responsive web client that runs in a current version of Chrome, Edge, Firefox, or Safari. The layout must remain usable from a viewport width of 320 px upwards, and every screen must be available in English and Vietnamese.
- **Software interfaces.** Payments are processed through Stripe (Connect Express, PaymentIntent, Checkout Session, and signature-verified webhooks); profile, portfolio, and portfolio media are stored on Cloudinary; one-time verification and password reset codes are delivered over SMTP through Nodemailer; curriculum vitae analysis and identity document verification are performed by external AI services called over HTTPS; and all persistent data is stored in PostgreSQL and accessed through Prisma.
- **Communication interfaces.** Browser-to-server traffic uses HTTPS with TLS 1.3. Calls from the API to Stripe, Cloudinary, the mail server, and the external AI services are made server-side over HTTPS with an explicit timeout, and a failure of any of them must be reported to the caller as a handled error rather than as an unhandled exception. Real-time messaging between the web client and the API uses a Socket.IO connection authenticated with the same JWT as the REST endpoints.
- **Hardware interfaces.** No dedicated hardware is required beyond the device that runs the browser and the camera or file system used to supply identity documents, avatars, and portfolio media.

4.2 Quality Attributes

4.2.1 Usability

- **Responsive Design:** The user interface must be fully responsive, supporting mobile devices, tablets, and desktop viewports (minimum width 320px).
- **Toast Notification System:** Interactive toast notifications (success, warning, error) must have the highest z-index to overlay all content, auto-dismiss after 3000ms, and must not freeze or block other user interactions.
- **Client-side Image Processing:** The avatar cropping tool must handle images locally before upload. The system must restrict upload sizes to under 5MB and validate formats (.jpg, .jpeg, .png, .webp) before initiating API requests.

4.2.2 Reliability

- **System Uptime:** The system core services must maintain a 99.9% monthly uptime, excluding scheduled maintenance.
- **Blockchain Fault Tolerance:** In case of RPC node connection failures, the platform must gracefully fall back to backup RPC endpoints within 3 seconds to avoid breaking the wallet balance display or transaction signing flows.
- **Escrow Reliability:** Funds locked in the Escrow Smart Contracts must be secure. Contracts must be verified on Etherscan/BscScan and use standard libraries (e.g., OpenZeppelin) to prevent vulnerabilities.

4.2.3 Performance

- **Response Time:** The system's API read operations must respond within 200ms under normal load (up to 500 concurrent requests). Write/update operations, excluding blockchain transactions, must complete within 500ms.
- **Page Load Time:** The web application must achieve a Largest Contentful Paint (LCP) of less than 2.5 seconds, adhering to Google's Core Web Vitals guidelines.
- **Concurrency:** The system backend must support at least 1,000 concurrent active users and handle a database connection pool efficiently.
- **Search and Filtering Efficiency:** The job search page must employ debouncing (300ms delay) and pagination (default 10 items per page) to optimize query performance and reduce server load.

4.2.4 Security & Privacy

- **Authentication and Authorization:** All network communications must be encrypted using HTTPS/TLS 1.3. User passwords must be hashed using bcrypt with a work factor of 12.
- **Session Management:** JWT Access Tokens must have a short lifespan (e.g., 15 minutes) and be stored in memory. Refresh Tokens must have a longer lifespan (e.g., 7 days), be stored in HTTP-only, Secure, and SameSite=Strict cookies, and be invalidated in the database upon user logout.
- **Role-Based Access Control (RBAC):** The system must strictly enforce role boundaries (Guest, Freelancer, Client, Admin). Administrative features, such as changing smart contract addresses or viewing audit logs, must be blocked by a server-side gateway for non-admin accounts.
- **Web3 Wallet and Transaction Safety:** Input wallet addresses must be verified on the client and server side to conform to the standard EVM format (starts with `0x` followed by 40 hexadecimal characters).

5. Requirement Appendix

This appendix collects the requirements that apply across features rather than to a single screen: the business rules that every use case must respect, the common requirements that hold for the whole system, and the standard application messages that the interface reuses.

5.1 Business Rules

The rules below are grouped by the feature they constrain. A use case that violates a rule is rejected, and the identifier of the rule is quoted in the corresponding alternative flow.

Table 430: Business Rules

ID	Rule Definition
BR-PROF-01	Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together.
BR-PROF-02	Hidden or private fields should not be displayed to unauthorized users (e.g., clients vs. the profile owner).
BR-PROF-03	Freelancers can only update their own profile.
BR-PROF-04	Mandatory fields (e.g., display_name, title) cannot be left empty.

ID	Rule Definition
BR-SKILL-01	All skills belonging to the freelancer must be displayed along with their proficiency level.
BR-SKILL-02	A freelancer cannot add duplicate skills.
BR-SKILL-03	Both skill name and proficiency level are required.
BR-SKILL-04	Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table.
BR-SKILL-05	Only the owning freelancer or an administrator can delete a skill from the profile.
BR-PORT-01	Only portfolios with deleted_at = NULL shall be displayed.
BR-PORT-02	Portfolio title is a required field.
BR-PORT-03	Media uploads must conform to system-defined size and type restrictions.
BR-PORT-04	Only portfolios with deleted_at = NULL can be accessed.
BR-PORT-05	Only the portfolio owner can edit it.
BR-PORT-06	Portfolio title remains a required field.
BR-PORT-07	Portfolio deletion must be a soft delete (updating the deleted_at column).
BR-PORT-08	Soft-deleted portfolios must not be visible to clients or guests.
BR-FORUM-01	Only categories with is_active = TRUE shall be displayed.
BR-FORUM-02	Category information displayed includes name and description.
BR-FORUM-03	Only categories with is_active = TRUE shall be accessible.
BR-FORUM-04	Category information displayed includes name and description.
BR-FORUM-05	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-06	Posts with is_pinned = TRUE shall appear before regular posts.
BR-FORUM-07	Posts shall be sorted by created_at in descending order within each group.

ID	Rule Definition
BR-FORUM-08	The system shall support pagination for post listings.
BR-FORUM-09	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-10	Post details shall include title, content, author, category, and created_at information.
BR-FORUM-11	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-12	Comment information shall include content, author, and created_at.
BR-FORUM-13	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-14	Comment details shall include content, author, and created_at information.
BR-FORUM-15	Only authenticated users can create posts.
BR-FORUM-16	Every post must belong to exactly one category.
BR-FORUM-17	Title and content are required fields.
BR-FORUM-18	Only the author of a post can edit that post.
BR-FORUM-19	Every post must belong to a valid category.
BR-FORUM-20	Title and content are required fields.
BR-FORUM-21	Only the author of a post can delete that post.
BR-FORUM-22	Post deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-23	Soft-deleted posts shall not appear in public forum listings.
BR-FORUM-24	Only authenticated users can like posts.
BR-FORUM-25	A user can have only one active like reaction per post.
BR-FORUM-26	The reaction shall be stored in the forum_likes table.
BR-FORUM-27	Only authenticated users can remove post reactions.
BR-FORUM-28	A like reaction can only be removed if a corresponding like record exists.

ID	Rule Definition
BR-FORUM-29	Removing a reaction shall delete the corresponding record from the forum_likes table.
BR-FORUM-30	Only authenticated users can create comments.
BR-FORUM-31	Comment content cannot be empty.
BR-FORUM-32	Comment content must contain at least 2 characters.
BR-FORUM-33	Each comment shall be associated with a valid post_id and user_id.
BR-FORUM-34	Only the owner of a comment can edit it.
BR-FORUM-35	Comment content cannot be empty.
BR-FORUM-36	Comment content must contain at least 2 characters.
BR-FORUM-37	The system shall record the updated_at timestamp when a comment is modified.
BR-FORUM-38	Only the owner of a comment can delete it.
BR-FORUM-39	Comment deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-40	Soft-deleted comments shall not be displayed in normal comment listings.
BR-FORUM-41	Only authenticated users can like comments.
BR-FORUM-42	A user can have only one active like reaction per comment.
BR-FORUM-43	The reaction shall be stored in the forum_likes table using comment_id.
BR-FORUM-44	Only authenticated users can remove comment reactions.
BR-FORUM-45	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-46	Removing a reaction shall delete the corresponding record from the forum_likes table using comment_id.
BR-FORUM-47	Only authenticated users can submit reports.
BR-FORUM-48	Reports must be associated with either a post_id or comment_id.

ID	Rule Definition
BR-FORUM-49	New reports shall be created with status = PENDING.
BR-FORUM-50	A user may submit only one active report for the same content item.
BR-JOB-01	Only jobs with status = 'open' or 'in_progress' shall be displayed to freelancers and guests.
BR-JOB-02	Clients shall see all their jobs regardless of status.
BR-JOB-03	Default sorting shall be by created_at in descending order.
BR-JOB-04	The system shall support pagination with default page size of 10.
BR-JOB-05	Search shall match keywords in title and description fields (case-insensitive).
BR-JOB-06	When multiple filters are applied, all conditions shall be combined using AND logic.
BR-JOB-07	Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status.
BR-JOB-08	If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
BR-JOB-09	Job title is required and must not exceed 200 characters.
BR-JOB-10	Job description is required and must not exceed 5000 characters.
BR-JOB-11	Budget must be a positive number (greater than 0).
BR-JOB-12	Category must be selected from predefined categories.
BR-JOB-13	New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
BR-JOB-14	Only the job owner can edit the job.
BR-JOB-15	Editing is restricted if the job has an active contract.
BR-JOB-16	All validation rules from UC-06.2 apply to edits.
BR-JOB-17	Edit actions shall be logged for auditing purposes.
BR-JOB-18	Only the job owner can delete the job.

ID	Rule Definition
BR-JOB-19	Deletion should be soft delete (set deleted_at timestamp, status = 'deleted').
BR-JOB-20	Jobs with active proposals or contracts cannot be deleted.
BR-JOB-21	Delete actions shall be logged for auditing purposes.
BR-JOB-22	Job status workflow: draft → open → in_progress → completed → closed.
BR-JOB-23	Allowed transitions: draft → open (client); open → in_progress (client, when contract created); open → closed (client, if cancelled); in_progress → completed (client, when work done); in_progress → closed (client, if cancelled); completed → closed (system/auto); Any status → deleted (soft delete).
BR-JOB-24	Only the job owner can manually change status.
BR-JOB-25	Status changes shall be logged for auditing purposes.
BR-JOB-26	Notifications shall be sent to freelancers when status changes from draft to open.
BR-JOB-27	Only the freelancer can view their own bookmarks.
BR-JOB-28	Bookmarks shall be sorted by created_at in descending order by default.
BR-JOB-29	Displayed information includes: job title, budget, client name, bookmark date.
BR-JOB-30	The system shall support pagination with default page size of 10.
BR-JOB-31	Only freelancers can bookmark jobs.
BR-JOB-32	Bookmark is a toggle action. Clicking the button toggles between adding and removing.
BR-JOB-33	If the bookmark does not exist, the system creates it and changes the button to "Bookmarked".
BR-JOB-34	If the bookmark exists, the system deletes it and changes the button to "Bookmark".

ID	Rule Definition
BR-JOB-35	Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
BR-JOB-36	Only the freelancer can remove their own bookmarks.
BR-JOB-37	The system must check if the bookmark exists before deletion.
BR-JOB-38	If the bookmark does not exist, the system shall return a 404 Not Found error.
BR-JOB-39	Removing a bookmark is a hard delete from the database.

5.2 Common Requirements

The following requirements apply to every feature of the platform and are therefore not repeated in each use case.

- **Authorisation.** Every endpoint is protected by the server-side permission guard, which checks the method, the path, and the module against the permissions of the caller's role. A hidden button or a disabled menu item is never treated as an access control.
- **Pagination.** Every list endpoint is paginated, with a default page size of 10 items and an explicit page parameter, so that no screen loads an unbounded result set.
- **Deletion.** Records that other records refer to are soft-deleted by setting `deleted_at`; a soft-deleted record is excluded from every public listing and from every lookup used by a normal user. Hard deletion is limited to join records such as skills, bookmarks, and reactions.
- **Timestamps.** Creation and modification times are stored in UTC and displayed in the time zone of the browser.
- **Money.** All amounts are stored in the smallest currency unit and displayed with two decimals and an explicit currency; a financial record is never updated in place but is corrected by a new ledger entry.
- **File uploads.** Every uploaded file is validated for type and size on the client before the request is sent and on the server before the file is stored.
- **Language.** Every label, message, and validation error is read from the message catalogue, so that the same screen can be rendered in English and in Vietnamese without a code change.

5.3 Application Messages List

The messages below are reused across features. The context column states when the message is raised, and the content column gives the English source text that the message catalogue translates.

#	Message code	Message Type	Context	Content
1	MSG01	Toast, red	The API cannot be reached or the network is down	Cannot connect to the server. Please try again later.
2	MSG02	Toast, red	The server returns an unexpected error	Unable to process your request. Please try again later.
3	MSG03	Toast, red	The caller requests a route outside the permissions of its role	You do not have permission to access this page.
4	MSG04	Inline, under the field	A required field is left empty	All fields are required.
5	MSG05	Inline, under the field	The email does not match a valid format	Please enter a valid email address.
6	MSG06	Inline, under the field	The password does not meet the password policy	Password must be at least 8 characters and include at least one uppercase letter, one number, and one special character.
7	MSG07	Inline, under the field	The password confirmation differs from the password	Passwords do not match.
8	MSG08	Toast, red	A requested record does not exist or has been soft-deleted	Post not found. / Comment not found. / Proposal not found.
9	MSG09	Toast, green	A record has been saved	Profile updated successfully. / Skill added successfully. / Portfolio created successfully.

Table 431: Application Messages List